

RTI Connext DDS Micro C++ API

2.4.14.2

Contents

1	Main Page	1
2	Module Index	1
2.1	Reference Manual	1
3	Class Index	4
3.1	Class Hierarchy	4
4	Class Index	8
4.1	Class List	8
5	File Index	14
5.1	File List	15
6	Module Documentation	15
6.1	Domain Module	16
6.1.1	Detailed Description	16
6.1.2	Macro Definition Documentation	16
6.1.3	Typedef Documentation	16
6.2	DomainParticipantFactory	18
6.2.1	Detailed Description	18
6.2.2	Variable Documentation	18
6.3	DomainParticipant	19
6.3.1	Detailed Description	19
6.3.2	Variable Documentation	19
6.4	Discovery	21
6.4.1	Detailed Description	22
6.4.2	Macro Definition Documentation	22
6.4.3	Typedef Documentation	23
6.5	Topic	25
6.5.1	Detailed Description	25
6.6	Topic	26
6.6.1	Detailed Description	26
6.7	User Data Type Support	27
6.7.1	Detailed Description	27
6.7.2	Macro Definition Documentation	28
6.7.3	Typedef Documentation	28
6.7.4	Function Documentation	28
6.7.5	Variable Documentation	29
6.8	Publication Module	30
6.8.1	Detailed Description	30
6.9	Publisher	31
6.9.1	Detailed Description	31
6.9.2	Variable Documentation	31
6.10	DataWriter	32
6.10.1	Detailed Description	32
6.10.2	Macro Definition Documentation	33
6.10.3	Typedef Documentation	33
6.11	Subscription Module	34
6.11.1	Detailed Description	34
6.11.2	Access to data samples	34
6.12	Subscriber	36
6.12.1	Detailed Description	36
6.12.2	Variable Documentation	36
6.13	DataReader	37
6.13.1	Detailed Description	37
6.13.2	Enumeration Type Documentation	38

6.14	Data Sample	40
6.14.1	Detailed Description	40
6.15	Sample States	41
6.15.1	Detailed Description	41
6.15.2	Typedef Documentation	41
6.15.3	Enumeration Type Documentation	41
6.15.4	Variable Documentation	42
6.16	View States	43
6.16.1	Detailed Description	43
6.16.2	Typedef Documentation	43
6.16.3	Enumeration Type Documentation	43
6.16.4	Variable Documentation	44
6.17	Instance States	45
6.17.1	Detailed Description	45
6.17.2	Typedef Documentation	45
6.17.3	Enumeration Type Documentation	45
6.17.4	Variable Documentation	46
6.18	Infrastructure Module	47
6.18.1	Detailed Description	47
6.19	Documentation Guide	48
6.19.1	Unsupported Features	48
6.19.2	API Naming Conventions	48
6.19.3	API Documentation Terms	48
6.19.4	Stereotypes	48
6.20	DDS C++ API	50
6.20.1	Detailed Description	50
6.20.2	Overview	50
6.20.3	Conceptual Model	51
6.20.4	Modules	52
6.21	DPSE Static Discovery API	53
6.21.1	Detailed Description	53
6.21.2	Macro Definition Documentation	53
6.22	DPDE Dynamic Discovery API	54
6.22.1	Detailed Description	54
6.22.2	Macro Definition Documentation	54
6.23	RT Run-Time API	55
6.23.1	Detailed Description	55
6.24	RHSM Reader History API	56
6.24.1	Detailed Description	56
6.25	WHSM Writer History API	57
6.25.1	Detailed Description	57
6.26	UDP Transport API	58
6.26.1	Detailed Description	58
6.26.2	Macro Definition Documentation	58
6.26.3	Function Documentation	59
6.27	UDP Transform API	60
6.28	RTPS Transport API	61
6.28.1	Detailed Description	62
6.28.2	Typedef Documentation	62
6.28.3	Enumeration Type Documentation	62
6.28.4	Variable Documentation	63
6.29	OS API	64
6.29.1	Detailed Description	64
6.30	Log API	65
6.30.1	Detailed Description	66
6.30.2	Typedef Documentation	66
6.30.3	Enumeration Type Documentation	66
6.30.4	Function Documentation	66

6.31	DDS-Specific Primitive Types	70
6.31.1	Detailed Description	70
6.31.2	Macro Definition Documentation	71
6.31.3	Typedef Documentation	71
6.32	Participant Built-in Topic	73
6.32.1	Detailed Description	73
6.32.2	Macro Definition Documentation	73
6.33	Publication Built-in Topic	74
6.33.1	Detailed Description	74
6.33.2	Macro Definition Documentation	74
6.34	Subscription Built-in Topic	75
6.34.1	Detailed Description	75
6.34.2	Macro Definition Documentation	75
6.35	DISCOVERY	76
6.35.1	Detailed Description	76
6.35.2	Variable Documentation	76
6.36	Time Support	77
6.36.1	Detailed Description	77
6.36.2	Macro Definition Documentation	77
6.36.3	Variable Documentation	77
6.37	GUID Support	79
6.37.1	Detailed Description	79
6.37.2	Variable Documentation	79
6.38	Sequence Number Support	80
6.38.1	Detailed Description	80
6.38.2	Function Documentation	80
6.39	Return Codes	81
6.39.1	Detailed Description	81
6.39.2	Standard Return Codes	81
6.39.3	Enumeration Type Documentation	81
6.40	Status Kinds	83
6.40.1	Detailed Description	83
6.40.2	Changes in Status	84
6.40.3	Macro Definition Documentation	85
6.40.4	Typedef Documentation	85
6.40.5	Enumeration Type Documentation	85
6.41	QoS Policies	89
6.41.1	Detailed Description	90
6.41.2	Specifying QoS on entities	90
6.41.3	QoS compatibility	91
6.41.4	Enumeration Type Documentation	91
6.42	DEADLINE	92
6.42.1	Detailed Description	92
6.42.2	Variable Documentation	92
6.43	OWNERSHIP	93
6.43.1	Detailed Description	93
6.43.2	Enumeration Type Documentation	93
6.43.3	Variable Documentation	93
6.44	OWNERSHIP_STRENGTH	94
6.44.1	Detailed Description	94
6.44.2	Variable Documentation	94
6.45	LIVELINESS	95
6.45.1	Detailed Description	95
6.45.2	Enumeration Type Documentation	95
6.45.3	Variable Documentation	96
6.46	RELIABILITY	97
6.46.1	Detailed Description	97
6.46.2	Enumeration Type Documentation	97

6.46.3	Variable Documentation	97
6.47	HISTORY	98
6.47.1	Detailed Description	98
6.47.2	Enumeration Type Documentation	98
6.47.3	Variable Documentation	98
6.48	DURABILITY	99
6.48.1	Detailed Description	99
6.48.2	Enumeration Type Documentation	99
6.48.3	Variable Documentation	99
6.49	RESOURCE_LIMITS	100
6.49.1	Detailed Description	100
6.49.2	Variable Documentation	100
6.50	PRESENTATION	101
6.50.1	Detailed Description	101
6.50.2	Enumeration Type Documentation	101
6.51	DESTINATION_ORDER	102
6.51.1	Detailed Description	102
6.51.2	Enumeration Type Documentation	102
6.52	DataReader Resource Limits	103
6.52.1	Detailed Description	103
6.52.2	Enumeration Type Documentation	103
6.52.3	Variable Documentation	103
6.53	DataWriter Resource Limits	104
6.53.1	Detailed Description	104
6.53.2	Variable Documentation	104
6.54	ENTITY_FACTORY	105
6.54.1	Detailed Description	105
6.54.2	Variable Documentation	105
6.55	Extended Qos Support	106
6.55.1	Detailed Description	106
6.56	SYSTEM_RESOURCE_LIMITS	107
6.56.1	Detailed Description	107
6.56.2	Variable Documentation	107
6.57	WIRE_PROTOCOL	108
6.57.1	Detailed Description	108
6.57.2	Macro Definition Documentation	109
6.57.3	Typedef Documentation	109
6.57.4	Variable Documentation	110
6.58	DATA_READER_PROTOCOL	112
6.58.1	Detailed Description	112
6.58.2	Variable Documentation	112
6.59	DATA_WRITER_PROTOCOL	113
6.59.1	Detailed Description	113
6.59.2	Variable Documentation	113
6.60	TRANSPORT	114
6.60.1	Detailed Description	114
6.60.2	Variable Documentation	114
6.61	DOMAIN_PARTICIPANT_RESOURCE_LIMITS	115
6.61.1	Detailed Description	115
6.61.2	Variable Documentation	115
6.62	Conditions and WaitSets	116
6.62.1	Detailed Description	116
6.63	Entity Support	117
6.63.1	Detailed Description	117
6.64	ENTITY_NAME	118
6.64.1	Detailed Description	118
6.64.2	Macro Definition Documentation	118
6.64.3	Variable Documentation	118

6.65	DDS_C	119
6.65.1	Detailed Description	131
6.65.2	Macro Definition Documentation	131
6.66	Sequence Support	153
6.66.1	Detailed Description	153
6.67	String Support	154
6.67.1	Detailed Description	154
6.67.2	Function Documentation	155
6.68	REDA	158
6.68.1	Detailed Description	158
6.68.2	Macro Definition Documentation	158
6.69	OSAPI Heap	160
6.69.1	Detailed Description	160
6.69.2	Function Documentation	160
6.69.3	Variable Documentation	163
6.70	OSAPI Mutex	164
6.70.1	Detailed Description	164
6.70.2	Typedef Documentation	164
6.70.3	Function Documentation	164
6.71	OSAPI AutoSAR	166
6.71.1	Detailed Description	166
6.71.2	Typedef Documentation	166
6.71.3	Enumeration Type Documentation	167
6.71.4	Function Documentation	167
6.71.5	Variable Documentation	168
6.72	OSAPI Semaphore	169
6.72.1	Detailed Description	169
6.72.2	Typedef Documentation	169
6.72.3	Function Documentation	169
6.73	OSAPI System API	171
6.73.1	Detailed Description	171
6.73.2	Macro Definition Documentation	171
6.74	OSAPI Thread API	172
6.74.1	Detailed Description	172
6.74.2	Macro Definition Documentation	172
6.74.3	Typedef Documentation	173
6.75	OSAPI Types	174
6.75.1	Detailed Description	174
6.75.2	Macro Definition Documentation	174
6.75.3	Typedef Documentation	175
6.76	DB	176
6.76.1	Detailed Description	177
6.76.2	Macro Definition Documentation	177
6.77	CDR	180
6.77.1	Detailed Description	180
6.77.2	Macro Definition Documentation	180
6.78	RT	181
6.78.1	Detailed Description	181
6.78.2	Macro Definition Documentation	181
6.79	NETIO	183
6.79.1	Detailed Description	187
6.79.2	Macro Definition Documentation	187
6.80	RTPS	195
6.80.1	Detailed Description	200
6.80.2	Macro Definition Documentation	200
6.81	RH	211
6.81.1	Detailed Description	212
6.81.2	Macro Definition Documentation	212

6.82	WH	214
6.82.1	Detailed Description	214
6.82.2	Macro Definition Documentation	214
6.83	DPSE	216
6.83.1	Detailed Description	218
6.83.2	Macro Definition Documentation	218
6.84	DPDE	224
6.84.1	Detailed Description	226
6.84.2	Macro Definition Documentation	226
6.85	Configuring Resource Limits	231
6.85.1	Detailed Description	231
6.86	Dynamic Participant Dynamic Endpoint (DPDE)	232
6.86.1	Detailed Description	232
6.87	max_participant_locators	233
6.88	max_locators_per_discovered_participant	234
6.89	max_samples_per_builtin_endpoint_reader	235
6.90	Log Codes	236
6.90.1	Detailed Description	236
7	Class Documentation	237
7.1	DDS_BuiltinTopicKey_t Struct Reference	237
7.1.1	Detailed Description	237
7.1.2	Member Data Documentation	237
7.2	DDS_ChecksumProperty Struct Reference	237
7.2.1	Detailed Description	238
7.2.2	Member Data Documentation	238
7.3	DDS_ChecksumProperty_t Struct Reference	238
7.3.1	Detailed Description	238
7.3.2	Member Data Documentation	238
7.4	DDS_DataReaderInstanceReplacedStatus Struct Reference	238
7.4.1	Detailed Description	239
7.4.2	Member Data Documentation	239
7.5	DDS_DataReaderProtocolQosPolicy Struct Reference	239
7.5.1	Detailed Description	240
7.5.2	Member Data Documentation	240
7.6	DDS_DataReaderQos Struct Reference	240
7.6.1	Detailed Description	241
7.6.2	Member Data Documentation	241
7.7	DDS_DataReaderResourceLimitsQosPolicy Struct Reference	242
7.7.1	Detailed Description	242
7.7.2	Member Data Documentation	242
7.8	DDS_DataWriterProtocolQosPolicy Struct Reference	244
7.8.1	Detailed Description	244
7.8.2	Member Data Documentation	244
7.9	DDS_DataWriterQos Struct Reference	245
7.9.1	Detailed Description	246
7.9.2	Member Data Documentation	246
7.10	DDS_DataWriterResourceLimitsQosPolicy Struct Reference	247
7.10.1	Detailed Description	247
7.10.2	Member Data Documentation	247
7.11	DDS_DeadlineQosPolicy Struct Reference	248
7.11.1	Detailed Description	248
7.11.2	Usage	248
7.11.3	Compatibility	248
7.11.4	Member Data Documentation	249
7.12	DDS_DestinationOrderQosPolicy Struct Reference	249
7.12.1	Detailed Description	249
7.12.2	Member Data Documentation	249

7.13	DDS_DiscoveryComponent Struct Reference	249
7.13.1	Detailed Description	250
7.13.2	Member Data Documentation	250
7.14	DDS_DiscoveryQosPolicy Struct Reference	250
7.14.1	Detailed Description	250
7.14.2	Usage	250
7.14.3	Member Data Documentation	250
7.15	DDS_DomainParticipantFactoryQos Struct Reference	251
7.15.1	Detailed Description	252
7.15.2	Member Data Documentation	252
7.16	DDS_DomainParticipantQos Struct Reference	252
7.16.1	Detailed Description	253
7.16.2	Member Data Documentation	253
7.17	DDS_DomainParticipantResourceLimitsQosPolicy Struct Reference	253
7.17.1	Detailed Description	254
7.17.2	Member Data Documentation	254
7.18	DDS_DurabilityQosPolicy Struct Reference	257
7.18.1	Detailed Description	257
7.18.2	Member Data Documentation	257
7.19	DDS_Duration_t Struct Reference	257
7.19.1	Detailed Description	258
7.19.2	Member Data Documentation	258
7.20	DDS_EntityFactoryQosPolicy Struct Reference	258
7.20.1	Detailed Description	258
7.20.2	Usage	258
7.20.3	Member Data Documentation	259
7.21	DDS_EntityNameQosPolicy Struct Reference	259
7.21.1	Detailed Description	259
7.21.2	Usage	259
7.21.3	Member Data Documentation	259
7.22	DDS_GUID_t Struct Reference	260
7.22.1	Detailed Description	260
7.22.2	Member Data Documentation	260
7.23	DDS_HistoryQosPolicy Struct Reference	260
7.23.1	Detailed Description	260
7.23.2	Usage	261
7.23.3	Consistency	261
7.23.4	Member Data Documentation	261
7.24	DDS_InconsistentTopicStatus Struct Reference	261
7.24.1	Detailed Description	261
7.25	DDS_LivelinessChangedStatus Struct Reference	262
7.25.1	Detailed Description	262
7.25.2	Member Data Documentation	262
7.26	DDS_LivelinessLostStatus Struct Reference	262
7.26.1	Detailed Description	263
7.26.2	Member Data Documentation	263
7.27	DDS_LivelinessQosPolicy Struct Reference	263
7.27.1	Detailed Description	263
7.27.2	Usage	264
7.27.3	Compatibility	265
7.27.4	Member Data Documentation	265
7.28	DDS_Locator_t Struct Reference	265
7.28.1	Detailed Description	266
7.28.2	Member Data Documentation	266
7.29	DDS_LocatorSeq Struct Reference	266
7.29.1	Detailed Description	266
7.30	DDS_OfferedDeadlineMissedStatus Struct Reference	266
7.30.1	Detailed Description	266

7.30.2	Member Data Documentation	267
7.31	DDS_OfferedIncompatibleQosStatus Struct Reference	267
7.31.1	Detailed Description	267
7.31.2	Member Data Documentation	268
7.32	DDS_OwnershipQosPolicy Struct Reference	268
7.32.1	Detailed Description	268
7.32.2	Usage	269
7.32.3	Compatibility	269
7.32.4	Member Data Documentation	271
7.33	DDS_OwnershipStrengthQosPolicy Struct Reference	271
7.33.1	Detailed Description	271
7.33.2	Member Data Documentation	272
7.34	DDS_ParticipantBuiltinTopicData Struct Reference	272
7.34.1	Detailed Description	272
7.34.2	Member Data Documentation	273
7.35	DDS_PresentationQosPolicy Struct Reference	273
7.35.1	Detailed Description	274
7.35.2	Compatibility	275
7.35.3	Member Data Documentation	276
7.36	DDS_ProductVersion Struct Reference	276
7.36.1	Detailed Description	276
7.36.2	Member Data Documentation	276
7.37	DDS_ProductVersion_t Struct Reference	277
7.37.1	Detailed Description	277
7.37.2	Member Data Documentation	277
7.38	DDS_ProtocolVersion Struct Reference	277
7.38.1	Detailed Description	278
7.38.2	Member Data Documentation	278
7.39	DDS_PublicationBuiltinTopicData Struct Reference	278
7.39.1	Detailed Description	279
7.39.2	Member Data Documentation	279
7.40	DDS_PublicationMatchedStatus Struct Reference	280
7.40.1	Detailed Description	280
7.40.2	Member Data Documentation	280
7.41	DDS_PublisherQos Struct Reference	281
7.41.1	Detailed Description	281
7.41.2	Member Data Documentation	281
7.42	DDS_ReliabilityQosPolicy Struct Reference	281
7.42.1	Detailed Description	281
7.42.2	Usage	282
7.42.3	Compatibility	282
7.42.4	Member Data Documentation	282
7.43	DDS_ReliableReaderActivityChangedStatus Struct Reference	283
7.43.1	Detailed Description	283
7.43.2	Member Data Documentation	283
7.44	DDS_ReliableSampleUnacknowledgedStatus Struct Reference	284
7.44.1	Detailed Description	284
7.44.2	Member Data Documentation	284
7.45	DDS_RequestedDeadlineMissedStatus Struct Reference	284
7.45.1	Detailed Description	285
7.45.2	Member Data Documentation	285
7.46	DDS_RequestedIncompatibleQosStatus Struct Reference	285
7.46.1	Detailed Description	285
7.46.2	Member Data Documentation	286
7.47	DDS_ResourceLimitsQosPolicy Struct Reference	286
7.47.1	Detailed Description	286
7.47.2	Usage	286
7.47.3	Consistency	287

7.47.4	Member Data Documentation	287
7.48	DDS_RtpsReliableReaderProtocol_t Struct Reference	289
7.48.1	Detailed Description	289
7.48.2	Member Data Documentation	289
7.49	DDS_RtpsReliableWriterProtocol_t Struct Reference	290
7.49.1	Detailed Description	290
7.49.2	Member Data Documentation	290
7.50	DDS_RtpsWellKnownPorts_t Struct Reference	291
7.50.1	Detailed Description	291
7.50.2	Member Data Documentation	292
7.51	DDS_SampleInfo Struct Reference	294
7.51.1	Detailed Description	295
7.51.2	Interpretation of the SampleInfo	295
7.51.3	Member Data Documentation	295
7.52	DDS_SampleInfoSeq Struct Reference	296
7.52.1	Detailed Description	296
7.53	DDS_SampleLostStatus Struct Reference	297
7.53.1	Detailed Description	297
7.53.2	Member Data Documentation	297
7.54	DDS_SampleRejectedStatus Struct Reference	297
7.54.1	Detailed Description	298
7.54.2	Member Data Documentation	298
7.55	DDS_SequenceNumber_t Struct Reference	298
7.55.1	Detailed Description	298
7.55.2	Member Data Documentation	299
7.56	DDS_SubscriberQos Struct Reference	299
7.56.1	Detailed Description	299
7.56.2	Member Data Documentation	299
7.57	DDS_SubscriptionBuiltinTopicData Struct Reference	299
7.57.1	Detailed Description	300
7.57.2	Member Data Documentation	300
7.58	DDS_SubscriptionMatchedStatus Struct Reference	301
7.58.1	Detailed Description	301
7.58.2	Member Data Documentation	302
7.59	DDS_SystemResourceLimitsQosPolicy Struct Reference	302
7.59.1	Detailed Description	302
7.59.2	Member Data Documentation	303
7.60	DDS_Time_t Struct Reference	303
7.60.1	Detailed Description	303
7.60.2	Member Data Documentation	303
7.61	DDS_TopicQos Struct Reference	303
7.61.1	Detailed Description	304
7.62	DDS_TransportQosPolicy Struct Reference	304
7.62.1	Detailed Description	304
7.62.2	Member Data Documentation	304
7.63	DDS_UserTrafficQosPolicy Struct Reference	304
7.63.1	Detailed Description	305
7.63.2	Member Data Documentation	305
7.64	DDS_VendorId Struct Reference	305
7.64.1	Detailed Description	305
7.64.2	Member Data Documentation	305
7.65	DDS_VendorId_t Struct Reference	305
7.65.1	Detailed Description	305
7.65.2	Member Data Documentation	306
7.66	DDS_WireProtocolQosPolicy Struct Reference	306
7.66.1	Detailed Description	306
7.66.2	Usage	307
7.66.3	Member Data Documentation	307

7.67	DDSTimeCondition Class Reference	309
7.67.1	Detailed Description	309
7.68	DDSTimeConditionSeq Class Reference	309
7.68.1	Detailed Description	310
7.69	DDSTimeDataReader Class Reference	310
7.69.1	Detailed Description	311
7.69.2	Member Function Documentation	312
7.69.3	SEQUENCES USAGE IN TAKE AND READ	315
7.70	DDSTimeDataReaderListener Class Reference	322
7.70.1	Detailed Description	323
7.70.2	Member Function Documentation	323
7.71	DDSTimeDataWriter Class Reference	325
7.71.1	Detailed Description	326
7.71.2	Member Function Documentation	327
7.72	DDSTimeDataWriterListener Class Reference	337
7.72.1	Detailed Description	338
7.72.2	Member Function Documentation	338
7.73	DDSTimeDomainEntity Class Reference	340
7.73.1	Detailed Description	340
7.74	DDSTimeDomainParticipant Class Reference	340
7.74.1	Detailed Description	341
7.74.2	Member Function Documentation	342
7.75	DDSTimeDomainParticipantFactory Class Reference	354
7.75.1	Detailed Description	355
7.75.2	Member Function Documentation	355
7.76	DDSTimeDomainParticipantListener Class Reference	360
7.76.1	Detailed Description	361
7.76.2	Member Function Documentation	361
7.77	DDSTimeEntity Class Reference	364
7.77.1	Detailed Description	365
7.77.2	Abstract operations	365
7.77.3	Member Function Documentation	366
7.78	DDSTimeGuardCondition Class Reference	368
7.78.1	Detailed Description	368
7.78.2	Member Function Documentation	368
7.79	DDSTimeListener Class Reference	369
7.79.1	Detailed Description	369
7.79.2	Access to Plain Communication Status	369
7.79.3	Access to Read Communication Status	370
7.80	DDSTimePublisher Class Reference	371
7.80.1	Detailed Description	372
7.80.2	Member Function Documentation	372
7.81	DDSTimePublisherListener Class Reference	376
7.81.1	Detailed Description	376
7.82	DDSTimeStatusCondition Class Reference	377
7.82.1	Detailed Description	377
7.82.2	Member Function Documentation	378
7.83	DDSTimeSubscriber Class Reference	378
7.83.1	Detailed Description	379
7.83.2	Member Function Documentation	380
7.84	DDSTimeSubscriberListener Class Reference	384
7.84.1	Detailed Description	385
7.84.2	Member Function Documentation	385
7.85	DDSTimeTopic Class Reference	385
7.85.1	Detailed Description	386
7.85.2	Member Function Documentation	386
7.86	DDSTimeTopicDescription Class Reference	389
7.86.1	Detailed Description	390

7.86.2	Member Function Documentation	390
7.87	DDSTopicListener Class Reference	391
7.87.1	Detailed Description	391
7.87.2	Member Function Documentation	391
7.88	DDSWaitSet Class Reference	392
7.88.1	Detailed Description	392
7.88.2	Usage	392
7.88.3	Trigger State of a ::DDSStatusCondition	393
7.88.4	Trigger State of a ::DDSGuardCondition	393
7.88.5	Member Function Documentation	393
7.89	DPDE_DiscoveryPluginProperty Struct Reference	395
7.89.1	Detailed Description	396
7.89.2	Member Data Documentation	396
7.90	DPDEDiscoveryFactory Class Reference	398
7.90.1	Detailed Description	398
7.90.2	Member Function Documentation	398
7.91	DPSE_DiscoveryPluginProperty Struct Reference	399
7.91.1	Detailed Description	399
7.91.2	Member Data Documentation	399
7.92	DPSEDiscoveryFactory Class Reference	400
7.92.1	Detailed Description	401
7.92.2	Member Function Documentation	401
7.93	DPSEDiscoveryPlugin Class Reference	401
7.93.1	Detailed Description	401
7.93.2	Member Function Documentation	402
7.94	FooDataReader Struct Reference	403
7.94.1	Detailed Description	404
7.94.2	Member Function Documentation	404
7.94.3	SEQUENCES USAGE IN TAKE AND READ	408
7.95	FooDataWriter Struct Reference	412
7.95.1	Detailed Description	412
7.95.2	Member Function Documentation	412
7.96	FooSeq Struct Reference	420
7.96.1	Detailed Description	421
7.96.2	Constructor & Destructor Documentation	421
7.96.3	Member Function Documentation	422
7.96.4	Member Data Documentation	429
7.97	FooTypeSupport Struct Reference	430
7.97.1	Detailed Description	430
7.97.2	Member Function Documentation	430
7.98	NDDS_Discovery_Property Struct Reference	433
7.98.1	Detailed Description	433
7.99	NDDS_Type_Plugin Struct Reference	433
7.99.1	Detailed Description	433
7.100	NDDS_Type_PluginVersion Struct Reference	434
7.100.1	Detailed Description	434
7.100.2	Member Data Documentation	434
7.101	OSAPI_LogProperty Struct Reference	434
7.101.1	Detailed Description	434
7.101.2	Member Data Documentation	434
7.102	OSAPI_MutexBase Struct Reference	435
7.102.1	Detailed Description	435
7.103	OSAPI_PortProperty Struct Reference	435
7.103.1	Detailed Description	436
7.103.2	Member Data Documentation	436
7.104	OSAPI_SystemI Struct Reference	439
7.104.1	Detailed Description	439
7.104.2	Member Data Documentation	439

7.105OSAPI_SystemProperty Struct Reference	440
7.105.1 Detailed Description	440
7.105.2 Member Data Documentation	440
7.106OSAPI_ThreadProperty Struct Reference	441
7.106.1 Detailed Description	441
7.106.2 Member Data Documentation	441
7.107OSAPI_TimerProperty Struct Reference	441
7.107.1 Detailed Description	441
7.107.2 Member Data Documentation	441
7.108RHSMHistoryFactory Class Reference	442
7.108.1 Detailed Description	442
7.108.2 Member Function Documentation	442
7.109RTI_ManagementQosPolicy Struct Reference	442
7.109.1 Detailed Description	442
7.109.2 Member Data Documentation	442
7.110RTPS_Checksum Union Reference	443
7.110.1 Detailed Description	443
7.111RTPS_ChecksumClass Struct Reference	443
7.111.1 Detailed Description	443
7.112RTPS_ChecksumProperty Struct Reference	443
7.112.1 Detailed Description	444
7.113RTPS_InterfaceFactoryProperty Struct Reference	444
7.113.1 Detailed Description	444
7.114RTRegistry Class Reference	444
7.114.1 Detailed Description	444
7.114.2 Member Function Documentation	445
7.115UDP_InterfaceFactoryProperty Struct Reference	446
7.115.1 Detailed Description	446
7.115.2 Member Data Documentation	446
7.116UDP_InterfaceTableEntry Struct Reference	448
7.116.1 Detailed Description	448
7.116.2 Member Data Documentation	448
7.117UDP_InterfaceTableEntrySeq Struct Reference	448
7.117.1 Detailed Description	449
7.118UDP_NatEntry Struct Reference	449
7.118.1 Detailed Description	449
7.119UDP_NatEntrySeq Struct Reference	449
7.119.1 Detailed Description	449
7.120UDPInterfaceFactory Class Reference	449
7.120.1 Detailed Description	449
7.120.2 Member Function Documentation	449
7.121UDPInterfaceTable Class Reference	450
7.121.1 Detailed Description	450
7.122WHSMHistoryFactory Class Reference	450
7.122.1 Detailed Description	450
7.122.2 Member Function Documentation	450
8 File Documentation	450
8.1 cdr_log.h File Reference	450
8.1.1 Detailed Description	451
8.2 db_log.h File Reference	451
8.2.1 Detailed Description	452
8.3 dds_c_config.h File Reference	452
8.3.1 Detailed Description	452
8.4 dds_c_log.h File Reference	452
8.4.1 Detailed Description	465
8.5 disc_dpde_log.h File Reference	465
8.5.1 Detailed Description	467

8.6	disc_dpse_log.h File Reference	467
8.6.1	Detailed Description	471
8.6.2	Macro Definition Documentation	471
8.7	netio_log.h File Reference	472
8.7.1	Detailed Description	476
8.7.2	Macro Definition Documentation	476
8.8	osapi_semaphore.h File Reference	477
8.8.1	Detailed Description	478
8.8.2	Macro Definition Documentation	478
8.9	osapi_stdio.h File Reference	478
8.9.1	Detailed Description	478
8.10	osapi_time.h File Reference	478
8.10.1	Detailed Description	478
8.11	reda_log.h File Reference	478
8.11.1	Detailed Description	479
8.12	rh_sm_log.h File Reference	479
8.12.1	Detailed Description	481
8.12.2	Macro Definition Documentation	481
8.13	rt_log.h File Reference	482
8.13.1	Detailed Description	482
8.14	rtps_config.h File Reference	482
8.14.1	Detailed Description	482
8.15	rtps_dll.h File Reference	482
8.15.1	Detailed Description	482
8.16	rtps_log.h File Reference	483
8.16.1	Detailed Description	488
8.16.2	Macro Definition Documentation	488
8.17	rtps_rtps_impl.h File Reference	489
8.17.1	Detailed Description	489
8.18	wh_sm_log.h File Reference	489
8.18.1	Detailed Description	489
8.18.2	Macro Definition Documentation	490

1 Main Page

Real-Time Innovations, Inc.

RTI Connex Micro provides a small-footprint, modular messaging solution for resource-limited devices that have limited memory and CPU power, and may not even be running an operating system. It provides the communications services that developers need to distribute time-critical data. Additionally, RTI Connex Micro is designed as a certifiable component in high-assurance systems.

RTI Connex Micro benefits include:

- Accommodations for resource-constrained environments.
- Modular and user extensible architecture.
- Designed to be a certifiable component for safety-critical systems.
- Seamless interoperability with Connex DDS and Connex Messaging.

2 Module Index

2.1 Reference Manual

Here is a list of all modules:

Documentation Guide	48
DDS C++ API	50
Domain Module	16
DomainParticipantFactory	18
DomainParticipant	19
Discovery	21
Participant Built-in Topic	73
Publication Built-in Topic	74
Subscription Built-in Topic	75
Topic	25
Topic	26
User Data Type Support	27
DDS-Specific Primitive Types	70
Publication Module	30
Publisher	31
DataWriter	32
Subscription Module	34
Subscriber	36
DataReader	37
Data Sample	40
Sample States	41
View States	43
Instance States	45
Infrastructure Module	47
Time Support	77
GUID Support	79
Sequence Number Support	80
Return Codes	81
Status Kinds	83
QoS Policies	89
DISCOVERY	76
DEADLINE	92

OWNERSHIP	93
OWNERSHIP_STRENGTH	94
LIVELINESS	95
RELIABILITY	97
HISTORY	98
DURABILITY	99
RESOURCE_LIMITS	100
PRESENTATION	101
DESTINATION_ORDER	102
ENTITY_FACTORY	105
Extended Qos Support	106
DataReader Resource Limits	103
DataWriter Resource Limits	104
SYSTEM_RESOURCE_LIMITS	107
WIRE_PROTOCOL	108
DATA_READER_PROTOCOL	112
DATA_WRITER_PROTOCOL	113
TRANSPORT	114
DOMAIN_PARTICIPANT_RESOURCE_LIMITS	115
ENTITY_NAME	118
Conditions and WaitSets	116
Entity Support	117
Sequence Support	153
String Support	154
DPSE Static Discovery API	53
DPDE Dynamic Discovery API	54
RT Run-Time API	55
RHSM Reader History API	56
WHSM Writer History API	57
UDP Transport API	58
UDP Transform API	60
RTPS Transport API	61

OS API	64
OSAPI Heap	160
OSAPI Mutex	164
OSAPI AutoSAR	166
OSAPI Semaphore	169
OSAPI System API	171
OSAPI Thread API	172
OSAPI Types	174
Log API	65
Configuring Resource Limits	231
Dynamic Participant Dynamic Endpoint (DPDE)	232
max_participant_locators	233
max_locators_per_discovered_participant	234
max_samples_per_builtin_endpoint_reader	235
Log Codes	236
DDS_C	119
REDA	158
DB	176
CDR	180
RT	181
NETIO	183
RTPS	195
RH	211
WH	214
DPSE	216
DPDE	224

3 Class Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

DDS_BuiltinTopicKey_t	237
DDS_ChecksumProperty	237

DDS_ChecksumProperty_t	238
DDS_DataReaderInstanceReplacedStatus	238
DDS_DataReaderProtocolQosPolicy	239
DDS_DataReaderQos	240
DDS_DataReaderResourceLimitsQosPolicy	242
DDS_DataWriterProtocolQosPolicy	244
DDS_DataWriterQos	245
DDS_DataWriterResourceLimitsQosPolicy	247
DDS_DeadlineQosPolicy	248
DDS_DestinationOrderQosPolicy	249
DDS_DiscoveryComponent	249
DDS_DiscoveryQosPolicy	250
DDS_DomainParticipantFactoryQos	251
DDS_DomainParticipantQos	252
DDS_DomainParticipantResourceLimitsQosPolicy	253
DDS_DurabilityQosPolicy	257
DDS_Duration_t	257
DDS_EntityFactoryQosPolicy	258
DDS_EntityNameQosPolicy	259
DDS_GUID_t	260
DDS_HistoryQosPolicy	260
DDS_InconsistentTopicStatus	261
DDS_LivelinessChangedStatus	262
DDS_LivelinessLostStatus	262
DDS_LivelinessQosPolicy	263
DDS_Locator_t	265
DDS_LocatorSeq	266
DDS_OfferedDeadlineMissedStatus	266
DDS_OfferedIncompatibleQosStatus	267
DDS_OwnershipQosPolicy	268
DDS_OwnershipStrengthQosPolicy	271
DDS_ParticipantBuiltinTopicData	272

DDS_PresentationQosPolicy	273
DDS_ProductVersion	276
DDS_ProductVersion_t	277
DDS_ProtocolVersion	277
DDS_PublicationBuiltinTopicData	278
DDS_PublicationMatchedStatus	280
DDS_PublisherQos	281
DDS_ReliabilityQosPolicy	281
DDS_ReliableReaderActivityChangedStatus	283
DDS_ReliableSampleUnacknowledgedStatus	284
DDS_RequestedDeadlineMissedStatus	284
DDS_RequestedIncompatibleQosStatus	285
DDS_ResourceLimitsQosPolicy	286
DDS_RtpsReliableReaderProtocol_t	289
DDS_RtpsReliableWriterProtocol_t	290
DDS_RtpsWellKnownPorts_t	291
DDS_SampleInfo	294
DDS_SampleInfoSeq	296
DDS_SampleLostStatus	297
DDS_SampleRejectedStatus	297
DDS_SequenceNumber_t	298
DDS_SubscriberQos	299
DDS_SubscriptionBuiltinTopicData	299
DDS_SubscriptionMatchedStatus	301
DDS_SystemResourceLimitsQosPolicy	302
DDS_Time_t	303
DDS_TopicQos	303
DDS_TransportQosPolicy	304
DDS_UserTrafficQosPolicy	304
DDS_VendorId	305
DDS_VendorId_t	305
DDS_WireProtocolQosPolicy	306

DDSCondition	309
DDSGuardCondition	368
DDSStatusCondition	377
DDSConditionSeq	309
DDSDomainParticipantFactory	354
DDSDomainParticipantListener	360
DDSEntity	364
DDSDataReader	310
DDSDataWriter	325
DDSDomainEntity	340
DDSDomainParticipant	340
DDSPublisher	371
DDSSubscriber	378
DDSTopic	385
DDSListener	369
DDSDataReaderListener	322
DDSSubscriberListener	384
DDSDataWriterListener	337
DDSPublisherListener	376
DDSTopicListener	391
DDSTopicDescription	389
DDSTopic	385
DDSWaitSet	392
DPDE_DiscoveryPluginProperty	395
DPDEDiscoveryFactory	398
DPSE_DiscoveryPluginProperty	399
DPSEDiscoveryFactory	400
DPSEDiscoveryPlugin	401
FooDataReader	403
FooDataWriter	412
FooSeq	420
FooTypeSupport	430

NDDS_Discovery_Property	433
NDDS_Type_Plugin	433
NDDS_Type_PluginVersion	434
OSAPI_LogProperty	434
OSAPI_MutexBase	435
OSAPI_PortProperty	435
OSAPI_SystemI	439
OSAPI_SystemProperty	440
OSAPI_ThreadProperty	441
OSAPI_TimerProperty	441
RHSMHistoryFactory	442
RTI_ManagementQosPolicy	442
RTPS_Checksum	443
RTPS_ChecksumClass	443
RTPS_ChecksumProperty	443
RTPS_InterfaceFactoryProperty	444
RTRegistry	444
UDP_InterfaceFactoryProperty	446
UDP_InterfaceTableEntry	448
UDP_InterfaceTableEntrySeq	448
UDP_NatEntry	449
UDP_NatEntrySeq	449
UDPInterfaceFactory	449
UDPInterfaceTable	450
WHSMHistoryFactory	450

4 Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

DDS_BuiltinTopicKey_t	
The key type of the built-in topic types	237
DDS_ChecksumProperty	
<<eXtension>> Type to define the checksum properties of the participant RTI Connex Micro	237

DDS_ChecksumProperty_t	
<<eXtension>> Type to define the checksum properties of the participant RTI Connex Micro	238
DDS_DataReaderInstanceReplacedStatus	
<<eXtension>> <<cert>> DDS_INSTANCE_REPLACED_STATUS	238
DDS_DataReaderProtocolQosPolicy	
<<eXtension>> Protocol that applies only to DDSDataReader instances	239
DDS_DataReaderQos	
<<cert>> QoS policies supported by a DDSDataReader entity	240
DDS_DataReaderResourceLimitsQosPolicy	
<<cert>> Resource limits that apply only to DDSDataReader instances	242
DDS_DataWriterProtocolQosPolicy	
<<eXtension>> <<cert>> Protocol that applies only to DDSDataWriter instances	244
DDS_DataWriterQos	
<<cert>> QoS policies supported by a DDSDataWriter entity	245
DDS_DataWriterResourceLimitsQosPolicy	
<<cert>> Resource limits that apply only to DDSDataWriter instances	247
DDS_DeadlineQosPolicy	
<<cert>> Expresses the maximum duration (deadline) within which an instance is expected to be updated	248
DDS_DestinationOrderQosPolicy	
Destination order policies that affect the ordering of subscribed data	249
DDS_DiscoveryComponent	
Specifies the discovery plugin the DDSDomainParticipant should use	249
DDS_DiscoveryQosPolicy	
<<cert>> Specifies the attributes required to discover participants in the domain	250
DDS_DomainParticipantFactoryQos	
<<cert>> QoS policies supported by a DDSDomainParticipantFactory	251
DDS_DomainParticipantQos	
<<cert>> QoS policies supported by a DDSDomainParticipant entity	252
DDS_DomainParticipantResourceLimitsQosPolicy	
<<cert>> Resource limits that apply only to DDSDomainParticipant instances	253
DDS_DurabilityQosPolicy	
Durability properties that affect the life-cycle of published data	257
DDS_Duration_t	
Type for <i>duration</i> representation	257
DDS_EntityFactoryQosPolicy	
<<cert>> A QoS policy for all DDSEntity types that can act as factories for one or more other DDSEntity types	258
DDS_EntityNameQosPolicy	
<<cert>> Policy used to name an entity	259
DDS_GUID_t	
Type for <i>GUID</i> (Global Unique Identifier) representation	260

DDS_HistoryQosPolicy	
<<cert>> Specifies the behavior of RTI Connex Micro in the case where the value of a sample changes (one or more times) before it can be successfully communicated to one or more existing subscribers	260
DDS_InconsistentTopicStatus	
DDS_INCONSISTENT_TOPIC_STATUS	261
DDS_LivelinessChangedStatus	
<<cert>> DDS_LIVELINESS_CHANGED_STATUS	262
DDS_LivelinessLostStatus	
<<cert>> DDS_LIVELINESS_LOST_STATUS	262
DDS_LivelinessQosPolicy	
<<cert>> Determines the mechanism and parameters used by the application to determine whether a DDSEntity is alive	263
DDS_Locator_t	
<<eXtension>> <<cert>> Type used to represent the addressing information needed to send a message to an RTPS Endpoint using one of the supported transports	265
DDS_LocatorSeq	
Declares IDL sequence < DDS_Locator_t >	266
DDS_OfferedDeadlineMissedStatus	
<<cert>> DDS_OFFERED_DEADLINE_MISSED_STATUS	266
DDS_OfferedIncompatibleQosStatus	
<<cert>> DDS_OFFERED_INCOMPATIBLE_QOS_STATUS	267
DDS_OwnershipQosPolicy	
<<cert>> Specifies whether it is allowed for multiple DDSDataWriter (s) to write the same instance of the data and if so, how these modifications should be arbitrated	268
DDS_OwnershipStrengthQosPolicy	
<<cert>> Specifies the value of the strength used to arbitrate among multiple DDSDataWriter objects that attempt to modify the same instance of a data type (identified by DDSTopic + key)	271
DDS_ParticipantBuiltinTopicData	
<<cert>> Object representing a remote DomainParticipant	272
DDS_PresentationQosPolicy	
Specifies how the samples representing changes to data instances are presented to a subscribing application	273
DDS_ProductVersion	
<<eXtension>> <<cert>> Type used to represent the current version of RTI Connex Micro	276
DDS_ProductVersion_t	
<<eXtension>> <<cert>> Type used to represent the current version of RTI Connex Micro	277
DDS_ProtocolVersion	
<<eXtension>> <<cert>> Type used to represent the version of the RTPS protocol	277
DDS_PublicationBuiltinTopicData	
Object describing a remote Publication	278
DDS_PublicationMatchedStatus	
<<cert>> DDS_PUBLICATION_MATCHED_STATUS	280

DDS_PublisherQos	
<<cert>> QoS policies supported by a DDSPublisher entity	281
DDS_ReliabilityQosPolicy	
<<cert>> Indicates the level of reliability offered/requested by RTI Connex Micro	281
DDS_ReliableReaderActivityChangedStatus	
<<eXtension>> <<cert>> Describes the activity (i.e. are acknowledgements forthcoming) of reliable readers matched to a reliable writer	283
DDS_ReliableSampleUnacknowledgedStatus	
<<cert>> Status about unacknowledged samples	284
DDS_RequestedDeadlineMissedStatus	
<<cert>> DDS_REQUESTED_DEADLINE_MISSED_STATUS	284
DDS_RequestedIncompatibleQosStatus	
<<cert>> DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS	285
DDS_ResourceLimitsQosPolicy	
<<cert>> Specifies the resources that RTI Connex Micro can consume in order to meet the requested QoS	286
DDS_RtpsReliableReaderProtocol_t	
Qos related to reliable reader protocol defined in RTPS	289
DDS_RtpsReliableWriterProtocol_t	
<<cert>> QoS related to the reliable writer protocol defined in RTPS	290
DDS_RtpsWellKnownPorts_t	
RTPS well-known port mapping configuration	291
DDS_SampleInfo	
Information that accompanies each sample that is read or taken	294
DDS_SampleInfoSeq	
Declares IDL sequence < DDS_SampleInfo >	296
DDS_SampleLostStatus	
<<cert>> DDS_SAMPLE_LOST_STATUS	297
DDS_SampleRejectedStatus	
DDS_SAMPLE_REJECTED_STATUS	297
DDS_SequenceNumber_t	
Type for <i>sequence</i> number representation	298
DDS_SubscriberQos	
<<cert>> QoS policies supported by a DDSSubscriber entity	299
DDS_SubscriptionBuiltinTopicData	
Entry created when a DDSDataReader is discovered in association with its Subscriber	299
DDS_SubscriptionMatchedStatus	
<<cert>> DDS_SUBSCRIPTION_MATCHED_STATUS	301
DDS_SystemResourceLimitsQosPolicy	
<<cert>> Resource limits that apply only to DDSDomainParticipantFactory	302
DDS_Time_t	
Type for <i>time</i> representation	303

DDS_TopicQos		
QoS policies supported by a DDSTopic entity		303
DDS_TransportQosPolicy		
<<eXtension>> <<cert>> Settings for available transports		304
DDS_UserTrafficQosPolicy		
A QoS policy to configure properties of network traffic caused by the exchange of user data		304
DDS_VendorId		
<<eXtension>> <<cert>> Type used to represent the vendor of the service implementing the RTPS protocol		305
DDS_VendorId_t		
<<eXtension>> <<cert>> Type used to represent the vendor of the service implementing the RTPS protocol		305
DDS_WireProtocolQosPolicy		
<<cert>> Specifies the wire-protocol-related attributes for the DDSDomainParticipant		306
DDSDCondition		
<<interface>> <<cert>> Root class for all the conditions that may be attached to a DDS-WaitSet		309
DDSDConditionSeq		
A sequence object containing references to one or more DDSDCondition objects		309
DDSDDataReader		
<<interface>> <<cert>> Allows the application to: (1) declare the data it wishes to receive (i.e. make a subscription) and (2) access the data received by the attached DDSSubscriber		310
DDSDDataReaderListener		
<<interface>> <<cert>> DDSLListener for reader status		322
DDSDDataWriter		
<<interface>> <<cert>> Allows an application to set the value of the data to be published under a given DDSTopic		325
DDSDDataWriterListener		
<<interface>> <<cert>> DDSLListener for writer status		337
DDSDDomainEntity		
<<interface>> Abstract base class for all DDS entities except for the DDSDomainParticipant		340
DDSDDomainParticipant		
<<interface>> <<cert>> Container for all DDSDomainEntity objects		340
DDSDDomainParticipantFactory		
<<singleton>> <<interface>> <<cert>> Allows creation and destruction of DDSDomainParticipant objects		354
DDSDDomainParticipantListener		
<<interface>> <<cert>> Listener for participant status		360
DDSEntity		
<<interface>> Abstract base class for all the DDS objects that support QoS policies, and a listener		364
DDSGuardCondition		
<<interface>> <<cert>> A specific DDSDCondition whose <code>trigger_value</code> is completely under the control of the application		368

DDSListener		
<<interface>>	Abstract base class for all Listener interfaces	369
DDSPublisher		
<<interface>>	<<cert>> A publisher is the object responsible for the actual dissemination of publications	371
DDSPublisherListener		
<<interface>>	<<cert>> DDSListener for DDSPublisher status	376
DDSStatusCondition		
<<interface>>	<<cert>> A specific DDSCondition that is associated with each DDSEntity	377
DDSSubscriber		
<<interface>>	A subscriber is the object responsible for actually receiving data from a subscription	378
DDSSubscriberListener		
<<interface>>	<<cert>> DDSListener for status about a subscriber	384
DDSTopic		
<<interface>>	The most basic description of the data to be published and subscribed	385
DDSTopicDescription		
<<interface>>	Base class for DDSTopic	389
DDSTopicListener		
<<interface>>	DDSListener for DDSTopic entities	391
DDSWaitSet		
<<interface>>	<<cert>> Allows an application to wait until one or more of the attached DDSCondition objects has a trigger_value of DDS_BOOLEAN_TRUE or else until the timeout expires	392
DPDE_DiscoveryPluginProperty		
<<extension>>	Properties for the Dynamic Participant/Dynamic Endpoint (DPDE) discovery plugin. This includes all discovery timing properties for participant discovery	395
DPDEDiscoveryFactory		
<<extension>>	Discovery plug-in used for asserting remote entities	398
DPSE_DiscoveryPluginProperty		
<<extension>>	<<cert>> Properties for the Dynamic Participant/Static Endpoint (DPSE) discovery plugin. This includes all discovery timing properties for participant discovery	399
DPSEDiscoveryFactory		
<<extension>>	<<cert>> A discovery factory that will be used by the DDSDomainParticipantFactory to create a discovery plugin	400
DPSEDiscoveryPlugin		
<<extension>>	<<cert>> Discovery plug-in used for asserting remote entities	401
FooDataReader		
	Declares the interface required to support a user data type-specific data reader	403
FooDataWriter		
	Declares the interface required to support a user data type-specific data writer	412
FooSeq		
<<interface>>	<<generic>> <<cert>> A type-safe, ordered collection of elements. The type of these elements is referred to in this documentation as "Foo"	420

FooTypeSupport	
<<interface>> <<generic>> User data type specific interface	430
NDDS_Discovery_Property	
Specifies internal information used to store discovery information about a DDSDomain-Participant in the registry	433
NDDS_Type_Plugin	
<<interface>> Type plugin interface	433
NDDS_Type_PluginVersion	
Data type used to keep track of versioning information about a NDDS_Type_Plugin	434
OSAPI_LogProperty	
Configuration of logging functionality	434
OSAPI_MutexBase	435
OSAPI_PortProperty	
AutoSAR System properties	435
OSAPI_SystemI	
<<cert>> System API	439
OSAPI_SystemProperty	
<<cert>> The System Properties	440
OSAPI_ThreadProperty	
Properties for thread creation	441
OSAPI_TimerProperty	
Timer properies	441
RHSMHistoryFactory	
<<eXtension>> <<cert>> A reader history factory that will be used by the DataReader to create a history cache	442
RTI_ManagementQosPolicy	
<<cert>> Policy used determine how DDSDataReader , DDSDataWriter , DDSPublisher , and DDSSubscriber is managed	442
RTPS_Checksum	
<<eXtension>> <<cert>> RTPS checksum type	443
RTPS_ChecksumClass	
<<eXtension>> <<cert>> Definition of a checksum function	443
RTPS_ChecksumProperty	
<<eXtension>> <<cert>> Checksum properties for the RTPS plugin	443
RTPS_InterfaceFactoryProperty	
<<eXtension>> <<cert>> Properties for the RTPS Transport	444
RTRegistry	
<<eXtension>> A run-time component factory	444
UDP_InterfaceFactoryProperty	
<<eXtension>> <<cert>> Properties for the UDP Transport	446
UDP_InterfaceTableEntry	
Generic structure to describe a network interface	448

UDP_InterfaceTableEntrySeq	
<<eXtension>> Declares IDL sequence< UDP_InterfaceTableEntry >	448
UDP_NatEntry	
<<eXtension>> Properties for a Network Address Translation entry	449
UDP_NatEntrySeq	
<<eXtension>> Declares IDL sequence< UDP_NatEntry >	449
UDPInterfaceFactory	
<<eXtension>> <<cert>> A UDP transport factory that will be used by a DomainParticipant to create a UDP interface. Only one UDP interface can be registered with the DomainParticipant-Factory	449
UDPInterfaceTable	450
WHSMHistoryFactory	
<<eXtension>> <<cert>> A writer history factory that will be used by the DataWriter to create a history cache	450

5 File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

cdr_log.h	450
db_log.h	
DB module log codes	451
dds_c_config.h	
DDS C configuration file	452
dds_c_log.h	
DDS_C module log codes	452
disc_dpde_log.h	465
disc_dpse_log.h	467
netio_log.h	
NETIO module log codes	472
osapi_semaphore.h	
Semaphore interface definition	477
osapi_stdio.h	
Standard I/O interface definition	478
osapi_time.h	
Time API definition	478
reda_log.h	
REDA module log codes	478
rh_sm_log.h	
RH module log codes	479

rt_log.h	482
rtps_config.h RTPS Configuration	482
rtps_dll.h RTPS Configuration	482
rtps_log.h RTPS module log codes	483
rtps_rtps_impl.h Implementation of RTPS interface functions and types	489
wh_sm_log.h WH module log codes	489

6 Module Documentation

6.1 Domain Module

Contains the [DDSDomainParticipant](#) class that acts as an entrypoint of RTI Connex Micro and acts as a factory for many of the classes. The [DDSDomainParticipant](#) also acts as a container for the other objects that make up RTI Connex Micro. Testing RTI Connex Micro command.

Modules

- [DomainParticipantFactory](#)
DDSDomainParticipantFactory entity and associated elements
- [DomainParticipant](#)
DDSDomainParticipant entity and associated elements
- [Discovery](#)
Descriptions of remote entities used for discovery.

Macros

- `#define DDSTheParticipantFactory DDSDomainParticipantFactory::get_instance()`
Can be used as an alias for the singleton factory returned by the operation [DDSDomainParticipantFactory::get_instance\(\)](#).

Typedefs

- `typedef DDS_DOMAINID_TYPE_NATIVE DDS_DomainId_t`
<<cert>> An integer that indicates in which domain a [DDSDomainParticipant](#) communicates.

6.1.1 Detailed Description

Contains the [DDSDomainParticipant](#) class that acts as an entrypoint of RTI Connex Micro and acts as a factory for many of the classes. The [DDSDomainParticipant](#) also acts as a container for the other objects that make up RTI Connex Micro. Testing RTI Connex Micro command. Defines the DDS domain package.

This image describes the DDS Domain Package, as described by the DDS standard. Note that not all functions and objects shown in this diagram are supported by RTI Connex Micro. Please read the information on the individual types to determine whether a function is supported.

6.1.2 Macro Definition Documentation

6.1.2.1 `#define DDSTheParticipantFactory DDSDomainParticipantFactory::get_instance()`

Can be used as an alias for the singleton factory returned by the operation [DDSDomainParticipantFactory::get_instance\(\)](#).

See also

[DDSDomainParticipantFactory::get_instance](#)

6.1.3 Typedef Documentation

6.1.3.1 `typedef DDS_DOMAINID_TYPE_NATIVE DDS_DomainId_t`

`<<cert>>` An integer that indicates in which domain a [DDSDomainParticipant](#) communicates.

Participants with the same [DDS_DomainId_t](#) are said to be in the same domain, and can thus communicate with one another.

The lower limit for a domain ID is 0. The upper limit for a domain ID is determined by the guidelines of the RTPS specification of the well-known ports on which a Participant will listen for discovery metatraffic from other Participants.

6.2 DomainParticipantFactory

[DDSDomainParticipantFactory](#) entity and associated elements

Classes

- class [DDSDomainParticipantFactory](#)
<<singleton>> <<interface>> <<cert>> Allows creation and destruction of [DDSDomainParticipant](#) objects.
- struct [DDS_DomainParticipantFactoryQos](#)
<<cert>> QoS policies supported by a [DDSDomainParticipantFactory](#).

Variables

- struct
[DDS_DomainParticipantFactoryQos DDS_PARTICIPANT_FACTORY_QOS_DEFAULT](#)
<<cert>> Special value for creating domain participant with default QoS.
- struct [DDS_DomainParticipantQos DDS_PARTICIPANT_QOS_DEFAULT](#)
<<cert>> Special value for creating domain participant with default QoS.

6.2.1 Detailed Description

[DDSDomainParticipantFactory](#) entity and associated elements

6.2.2 Variable Documentation

6.2.2.1 struct [DDS_DomainParticipantFactoryQos DDS_PARTICIPANT_FACTORY_QOS_DEFAULT](#)

<<cert>> Special value for creating domain participant with default QoS.

When used in [DDSDomainParticipantFactory::create_participant](#), this special value is used to indicate that the [DDSDomainParticipant](#) should be created with the default [DDSDomainParticipant](#) QoS.

See also

[DDSDomainParticipantFactory::create_participant\(\)](#)

6.2.2.2 struct [DDS_DomainParticipantQos DDS_PARTICIPANT_QOS_DEFAULT](#)

<<cert>> Special value for creating domain participant with default QoS.

When used in [DDSDomainParticipantFactory::create_participant](#), this special value is used to indicate that the [DDSDomainParticipant](#) should be created with the default [DDSDomainParticipant](#) QoS.

See also

[DDSDomainParticipantFactory::create_participant\(\)](#)

6.3 DomainParticipant

[DDSDomainParticipant](#) entity and associated elements

Classes

- class [DDSDomainParticipantListener](#)
<<interface>> <<cert>> Listener for participant status.
- class [DDSDomainParticipant](#)
<<interface>> <<cert>> Container for all [DDSDomainEntity](#) objects.
- struct [DDS_DomainParticipantQos](#)
<<cert>> QoS policies supported by a [DDSDomainParticipant](#) entity.

Variables

- struct [DDS_TopicQos DDS_TOPIC_QOS_DEFAULT](#)
<<cert>> Special value for creating a [DDSTopic](#) with default QoS. Note that this is the only supported parameter in RTI Connex Micro when creating a topic.
- struct [DDS_PublisherQos DDS_PUBLISHER_QOS_DEFAULT](#)
<<cert>> Special value for creating a [DDSPublisher](#) with default QoS.
- struct [DDS_SubscriberQos DDS_SUBSCRIBER_QOS_DEFAULT](#)
<<cert>> Special value for creating a [DDSSubscriber](#) with default QoS.

6.3.1 Detailed Description

[DDSDomainParticipant](#) entity and associated elements

6.3.2 Variable Documentation

6.3.2.1 struct [DDS_TopicQos DDS_TOPIC_QOS_DEFAULT](#)

<<cert>> Special value for creating a [DDSTopic](#) with default QoS. Note that this is the only supported parameter in RTI Connex Micro when creating a topic.

RTI Connex Micro does not support the Topic QoS. This represents the a placeholder for Topic QoS values. This is the only valid paramater that can be passed into the [DDSDomainParticipant::create_topic](#) function (no other values for QoS are supported).

See also

[DDSDomainParticipant::create_topic](#)

6.3.2.2 struct [DDS_PublisherQos DDS_PUBLISHER_QOS_DEFAULT](#)

<<cert>> Special value for creating a [DDSPublisher](#) with default QoS.

When used in [DDSDomainParticipant::create_publisher](#), this special value is used to indicate that the [DDSPublisher](#) should be created with the default [DDSPublisher](#) QoS.

See also

[DDSDomainParticipant::create_publisher](#)

6.3.2.3 struct `DDS_SubscriberQos` `DDS.SUBSCRIBER_QOS.DEFAULT`

`<<cert>>` Special value for creating a [DDSSubscriber](#) with default QoS.

When used in [DDSDomainParticipant::create_subscriber](#), this special value is used to indicate that the [DDS-Subscriber](#) should be created with the default [DDSSubscriber](#) QoS.

See also

[DDSDomainParticipant::create_subscriber](#)

6.4 Discovery

Descriptions of remote entities used for discovery.

Modules

- [Participant Built-in Topic](#)
Builtin topic for accessing information about the DomainParticipants discovered by RTI Connex Micro.
- [Publication Built-in Topic](#)
Builtin topic for configuring information about the Publications to be discovered by RTI Connex Micro.
- [Subscription Built-in Topic](#)
Builtin topic for configuring information about the Subscriptions to be discovered by RTI Connex Micro.

Classes

- struct [DDS_Locator_t](#)
<<eXtension>> <<cert>> Type used to represent the addressing information needed to send a message to an RTPS Endpoint using one of the supported transports.
- struct [DDS_LocatorSeq](#)
Declares IDL sequence < DDS_Locator_t >
- struct [DDS_ProtocolVersion](#)
<<eXtension>> <<cert>> Type used to represent the version of the RTPS protocol.
- struct [DDS_VendorId](#)
<<eXtension>> <<cert>> Type used to represent the vendor of the service implementing the RTPS protocol.
- struct [DDS_VendorId_t](#)
<<eXtension>> <<cert>> Type used to represent the vendor of the service implementing the RTPS protocol.
- struct [DDS_BuiltinTopicKey_t](#)
The key type of the built-in topic types.

Macros

- [#define DDS_LOCATOR_INVALID](#) RTPS_LOCATOR_INVALID
An invalid locator.
- [#define DDS_LOCATOR_KIND_INVALID](#)
Locator of this kind is invalid.
- [#define DDS_LOCATOR_PORT_INVALID](#)
An invalid port.
- [#define DDS_LOCATOR_ADDRESS_INVALID](#) RTPS_LOCATOR_ADDRESS_INVALID
An invalid address.
- [#define DDS_LOCATOR_KIND_UDPv4](#)
A locator for a UDPv4 address.
- [#define DDS_LOCATOR_KIND_UDPv6](#)
A locator for a UDPv6 address.
- [#define DDS_LOCATOR_KIND_RESERVED](#)
Locator of this kind is reserved.
- [#define DDS_LOCATOR_KIND_SHMEM](#)
A locator for an address accessed via shared memory.
- [#define DDS_PROTOCOLVERSION_1_0](#) { 1, 0 }
The protocol version 1.0.
- [#define DDS_PROTOCOLVERSION_1_1](#) { 1, 1 }
The protocol version 1.1.

- `#define DDS_PROTOCOLVERSION_1_2 { 1, 2 }`
The protocol version 1.2.
- `#define DDS_PROTOCOLVERSION_2_0 { 2, 0 }`
The most recent protocol version. Currently 1.2.
- `#define DDS_PROTOCOLVERSION_2_1 { 2, 1 }`
The protocol version 2.1.
- `#define DDS_PROTOCOLVERSION { 2, 1 }`
The most recent protocol version. Currently 1.2.
- `#define DDS_VENDOR_ID_LENGTH_MAX 2`
Length of vendor id.

Typedefs

- typedef struct `DDS_ProtocolVersion DDS_ProtocolVersion_t`
`<<eXtension>>` `<<cert>>` Type used to represent the version of the RTPS protocol.
- typedef struct `DDS_BuiltinTopicKey_t DDS_BuiltinTopicKey_t`
The key type of the built-in topic types.

6.4.1 Detailed Description

Descriptions of remote entities used for discovery. Given a DDS entity, a remote entity is any other entity that can be discovered and matched. A built-in object is a description of a remote entity, including all the QoS which are important for discovery and matching of remote readers and writers with local readers and writers.

RTI Connex Micro must discover and keep track of the remote entities, such as new participants in the domain. Depending on the type of discovery used, this information about remote entities is gathered in three ways:

1. No data is sent over the network (static discovery)
2. A subset is sent over the network (dynamic participant/static endpoint discovery), while the rest is statically asserted.
3. All data about DomainParticipants DataReaders and DataWriters is over the network (simple discovery)

In the first case, all information about remote DomainParticipants DataReaders and DataWriters is statically configured by the user before starting the middleware. To do this static configuration, the user must fill in the built-in structures for all remote entities it wishes to discover, and must assert them to the middleware. This type of discovery is not supported in this version of RTI Connex Micro.

In the second case, the participant data is sent over the network, but the endpoint data is statically configured. So, the local application fills in builtin structures representing the QoS and object IDs all the remote readers and writers it wishes to discover. For this type of discovery the user does not configure builtin data for remote participants. This type of discovery is faster than Simple Discovery, and does not require reliable communication. This is supported by RTI Connex Micro and is known as Dynamic Participant, Static Endpoint (DPSE) discovery.

In the last case, the user does not need to fill in any data about remote entities, because it is automatically sent over the network by the middleware. The user can access the data through special listeners for discovery traffic.

This is supported by RTI Connex Micro and is known as Dynamic Participant, Dynamic Endpoint (DPDE) discovery. Refer to `DDS_ParticipantBuiltinTopicData`, `DDS_SubscriptionBuiltinTopicData` and `DDS_PublicationBuiltinTopicData` for a description of all the built-in topics and their contents.

`<<eXtension>>` `<<cert>>` Dynamic Participant, Static Endpoint Discovery plugin mechanism for RTI Connex Micro.

6.4.2 Macro Definition Documentation

6.4.2.1 #define DDS_LOCATOR_INVALID RTPS_LOCATOR_INVALID

An invalid locator.

6.4.2.2 #define DDS_LOCATOR_KIND_INVALID

Locator of this kind is invalid.

6.4.2.3 #define DDS_LOCATOR_PORT_INVALID

An invalid port.

6.4.2.4 #define DDS_LOCATOR_ADDRESS_INVALID RTPS_LOCATOR_ADDRESS_INVALID

An invalid address.

6.4.2.5 #define DDS_LOCATOR_KIND_UDPv4

A locator for a UDPv4 address.

6.4.2.6 #define DDS_LOCATOR_KIND_UDPv6

A locator for a UDPv6 address.

6.4.2.7 #define DDS_LOCATOR_KIND_RESERVED

Locator of this kind is reserved.

6.4.2.8 #define DDS_LOCATOR_KIND_SHMEM

A locator for an address accessed via shared memory.

6.4.2.9 #define DDS_PROTOCOLVERSION_1_0 { 1, 0 }

The protocol version 1.0.

6.4.2.10 #define DDS_PROTOCOLVERSION_1_1 { 1, 1 }

The protocol version 1.1.

6.4.2.11 #define DDS_PROTOCOLVERSION_1_2 { 1, 2 }

The protocol version 1.2.

6.4.2.12 #define DDS_PROTOCOLVERSION_2_0 { 2, 0 }

The most recent protocol version. Currently 1.2.

6.4.2.13 #define DDS_PROTOCOLVERSION_2_1 { 2, 1 }

The protocol version 2.1.

6.4.2.14 #define DDS_PROTOCOLVERSION { 2, 1 }

The most recent protocol version. Currently 1.2.

6.4.2.15 #define DDS_VENDOR_ID_LENGTH_MAX 2

Length of vendor id.

6.4.3 Typedef Documentation

6.4.3.1 typedef struct DDS_ProtocolVersion DDS_ProtocolVersion_t

<<*eXtension*>> <<*cert*>> Type used to represent the version of the RTPS protocol.

6.4.3.2 typedef struct DDS_BuiltinTopicKey_t DDS_BuiltinTopicKey_t

The key type of the built-in topic types.

Each remote [DDSEntity](#) to be discovered can be uniquely identified by this key. This is the key of all the built-in topic data types.

The value of element DDS_BUILTIN_TOPIC_KEY_OBJECT_ID is the object ID of the remote entity. In Dynamic Participant/Static Endpoint discovery, this must be set correctly for each remote [DDSDataReader](#) and [DDSDataWriter](#) that an application intends to discover.

For example, to configure a remote [DDSDataWriter](#) with object ID 100 by setting the key in the [DDS_PublicationBuiltinTopicData](#):

```
remote_pub_data.key.value[DDS_BUILTIN_TOPIC_KEY_OBJECT_ID] = 100;
```

See also

[DDS_ParticipantBuiltinTopicData](#)
[DDS_PublicationBuiltinTopicData](#)
[DDS_SubscriptionBuiltinTopicData](#)

6.5 Topic

Contains the [DDSTopic](#), the [DDSTopicListener](#) interface, and more generally, all that is needed by an application to define [DDSTopic](#) objects and the data types that are associated with [DDSTopic](#) objects.

Modules

- [Topic](#)
[DDSTopic](#) entity and associated elements
- [User Data Type Support](#)
Defines generic classes and macros to support user data types.
- [DDS-Specific Primitive Types](#)
Basic DDS value types for use in user data types.

6.5.1 Detailed Description

Contains the [DDSTopic](#), the [DDSTopicListener](#) interface, and more generally, all that is needed by an application to define [DDSTopic](#) objects and the data types that are associated with [DDSTopic](#) objects.

6.6 Topic

[DDSTopic](#) entity and associated elements

Classes

- class [DDSTopicDescription](#)
<<interface>> Base class for [DDSTopic](#).
- class [DDSTopic](#)
<<interface>> The most basic description of the data to be published and subscribed.
- class [DDSTopicListener](#)
<<interface>> [DDSTopicListener](#) for [DDSTopic](#) entities.
- struct [DDS_InconsistentTopicStatus](#)
[DDS_INCONSISTENT_TOPIC_STATUS](#)
- struct [DDS_TopicQos](#)
QoS policies supported by a [DDSTopic](#) entity.

6.6.1 Detailed Description

[DDSTopic](#) entity and associated elements

6.7 User Data Type Support

Defines generic classes and macros to support user data types.

Classes

- struct `FooTypeSupport`
<<interface>> <<generic>> User data type specific interface.

Macros

- #define `DDS_DATAWRITER_CPP`(TDataWriter, TData)
Declares the interface required to support a user data type specific data writer.
- #define `DDS_DATAREADER_CPP`(TDataReader, TData)
Declares the interface required to support a user data type-specific data reader.
- #define `DDS_TYPESUPPORT_CPP`(TTypeSupport, TData)
Declares the interface required to support a user data type.

Typedefs

- typedef `DDS_HANDLE_TYPE_NATIVE DDS_InstanceHandle_t`
Type definition for an instance handle.

Functions

- `DDS_Boolean DDS_InstanceHandle_equals` (const `DDS_InstanceHandle_t` *self, const `DDS_InstanceHandle_t` *other)
<<cert>> Compares this instance handle with another handle for equality.
- `DDS_Boolean DDS_InstanceHandle_is_nil` (const `DDS_InstanceHandle_t` *self)
<<cert>> Compare this handle to `DDS_HANDLE_NIL`

Variables

- const `DDS_InstanceHandle_t DDS_HANDLE_NIL`
<<cert>> The NIL instance handle.

6.7.1 Detailed Description

Defines generic classes and macros to support user data types. Defines the DDS user data type support.

DDS specifies strongly typed interfaces to read and write user data. For each data class defined by the application, there is a number of specialised classes that are required to facilitate the type-safe interaction of the application with RTI Connex Micro.

RTI Connex Micro provides an automatic means to generate all these type-specific classes with the RTI IDL Compiler User Manual utility. The complete set of automatic classes created for a hypothetical user data type named `Foo` are shown below.

The macros defined here declare the strongly typed APIs needed to support an arbitrary user defined data of type `Foo`.

6.7.2 Macro Definition Documentation

6.7.2.1 `#define DDS_DATAWRITER_CPP( TDataWriter, TData )`

Declares the interface required to support a user data type specific data writer.

Uses:

[FooTypeSupport](#) user data type, `Foo`

Defines:

[FooDataWriter](#) `DDSDDataWriter` of type `Foo`, i.e. `FooDataWriter`

6.7.2.2 `#define DDS_DATAREADER_CPP( TDataReader, TData )`

Declares the interface required to support a user data type-specific data reader.

Uses:

[FooTypeSupport](#) user data type, `Foo` [FooSeq](#) sequence of user data type, `sequence<Foo>`

Defines:

[FooDataReader](#) `DDSDDataReader` of type `Foo`, i.e. `FooDataReader`

See also

[FooSeq](#)

6.7.2.3 `#define DDS_TYPESUPPORT_CPP( TTypeSupport, TData )`

Declares the interface required to support a user data type.

Defines:

[FooTypeSupport](#) `TypeSupport` of type `Foo`, i.e. `FooTypeSupport`

6.7.3 Typedef Documentation

6.7.3.1 `typedef DDS_HANDLE_TYPE_NATIVE DDS_InstanceHandle_t`

Type definition for an instance handle.

Handle to identify different instances of the same [DDSTopic](#) of a certain type.

See also

[FooDataWriter::register_instance](#)
[DDS_SampleInfo::instance_handle](#)

6.7.4 Function Documentation

6.7.4.1 `DDS_Boolean DDS_InstanceHandle_equals ( const DDS_InstanceHandle_t * self, const DDS_InstanceHandle_t * other )`

`<<cert>>` Compares this instance handle with another handle for equality.

Parameters

<i>self</i>	<< <i>in</i> >> This handle. Cannot be NULL.
<i>other</i>	<< <i>in</i> >> The other handle to be compared with this handle. Cannot be NULL.

Returns

[DDS_BOOLEAN_TRUE](#) if the two handles have equal values, or [DDS_BOOLEAN_FALSE](#) otherwise.

MT Safety:

UNSAFE. This operation does not protect against the left or right side from being modified by another thread while the comparison is made.

See also

[DDS_InstanceHandle_is_nil](#)

6.7.4.2 DDS_Boolean DDS_InstanceHandle_is_nil (const DDS_InstanceHandle_t * *self*)

<<*cert*>> Compare this handle to [DDS_HANDLE_NIL](#)

Returns

[DDS_BOOLEAN_TRUE](#) if the given instance handle is equal to [DDS_HANDLE_NIL](#) or [DDS_BOOLEAN_FALSE](#) otherwise.

See also

[DDS_InstanceHandle_equals](#)

6.7.5 Variable Documentation

6.7.5.1 const DDS_InstanceHandle_t DDS_HANDLE_NIL

<<*cert*>> The NIL instance handle.

Special [DDS_InstanceHandle_t](#) value

See also

[DDS_InstanceHandle_is_nil](#)

6.8 Publication Module

Contains the [DDSPublisher](#), and [DDSDataWriter](#) classes, and more generally, all that is needed on the publication side.

Modules

- [Publisher](#)
[DDSPublisher](#) entity and associated elements
- [DataWriter](#)
[DDSDataWriter](#) entity and associated elements

6.8.1 Detailed Description

Contains the [DDSPublisher](#), and [DDSDataWriter](#) classes, and more generally, all that is needed on the publication side. Defines the DDS publication package.

This diagram defines the elements that are defined in the DDS specification, and their relationships to each other. Not all entities and functions shown in the diagram are supported by RTI Connex Micro. Please read the information on the individual types to determine whether a function is supported.

6.9 Publisher

[DDSPublisher](#) entity and associated elements

Classes

- class [DDSPublisherListener](#)
<<interface>> <<cert>> [DDSListener](#) for [DDSPublisher](#) status.
- class [DDSPublisher](#)
<<interface>> <<cert>> A publisher is the object responsible for the actual dissemination of publications.
- struct [DDS_PublisherQos](#)
<<cert>> QoS policies supported by a [DDSPublisher](#) entity.

Variables

- struct [DDS_DataWriterQos](#) [DDS_DATAWRITER_QOS_DEFAULT](#)
<<cert>> Special value for creating [DDSDataWriter](#) with default QoS

6.9.1 Detailed Description

[DDSPublisher](#) entity and associated elements

6.9.2 Variable Documentation

6.9.2.1 struct [DDS_DataWriterQos](#) [DDS_DATAWRITER_QOS_DEFAULT](#)

<<cert>> Special value for creating [DDSDataWriter](#) with default QoS

When used in [DDSPublisher::create_datawriter](#), this special value is used to indicate that the [DDSDataWriter](#) should be created with the default [DDSDataWriter](#) QoS.

See also

[DDSPublisher::create_datawriter](#)

6.10 DataWriter

[DDSDataWriter](#) entity and associated elements

Classes

- struct [FooDataWriter](#)
Declares the interface required to support a user data type-specific data writer.
- class [DDSDataWriterListener](#)
<<interface>> <<cert>> [DDSLISTENER](#) for writer status.
- class [DDSDataWriter](#)
<<interface>> <<cert>> Allows an application to set the value of the data to be published under a given [DDS-Topic](#).
- struct [DDS_OfferedDeadlineMissedStatus](#)
<<cert>> [DDS_OFFERED_DEADLINE_MISSED_STATUS](#)
- struct [DDS_LivelinessLostStatus](#)
<<cert>> [DDS_LIVELINESS_LOST_STATUS](#)
- struct [DDS_OfferedIncompatibleQosStatus](#)
<<cert>> [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#)
- struct [DDS_PublicationMatchedStatus](#)
<<cert>> [DDS_PUBLICATION_MATCHED_STATUS](#)
- struct [DDS_ReliableReaderActivityChangedStatus](#)
<<extension>> <<cert>> Describes the activity (i.e. are acknowledgements forthcoming) of reliable readers matched to a reliable writer.
- struct [DDS_ReliableSampleUnacknowledgedStatus](#)
<<cert>> Status about unacknowledged samples.
- struct [DDS_DataWriterQos](#)
<<cert>> QoS policies supported by a [DDSDataWriter](#) entity.

Macros

- #define [DDS_ReliableReaderActivityChangedStatus_INITIALIZER](#)
<<cert>> Initializer for new status instances.

Typedefs

- typedef void(* [DDS_DataWriterListener_ReliableReaderActivityChangedCallback](#))(void *listener_data, DDS_S_DataWriter *writer, const struct [DDS_ReliableReaderActivityChangedStatus](#) *status)
<<extension>> <<cert>> A matched reliable reader has become active or become inactive.
- typedef void(* [DDS_DataWriterListener_ReliableSampleUnacknowledgedCallback](#))(void *listener_data, DDS_S_DataWriter *writer, const struct [DDS_ReliableSampleUnacknowledgedStatus](#) *status)
<<extension>> <<cert>> Unacknowledged samples are removed from the writer cache.

6.10.1 Detailed Description

[DDSDataWriter](#) entity and associated elements

6.10.2 Macro Definition Documentation

6.10.2.1 #define DDS_ReliableReaderActivityChangedStatus_INITIALIZER

<<cert>> Initializer for new status instances.

New [DDS_ReliableReaderActivityChangedStatus](#) instances stored on the stack should be initialized with this value before they are passed to any method. This step ensures that those fields that use dynamic memory are properly initialized. This does not allocate memory.

The simplest way to create a status structure is to initialize it on the stack at the time of its creation.

6.10.3 Typedef Documentation

6.10.3.1 typedef void(* DDS_DataWriterListener_ReliableReaderActivityChangedCallback)(void *listener_data, DDS_DataWriter *writer, const struct DDS_ReliableReaderActivityChangedStatus *status)

<<eXtension>> <<cert>> A matched reliable reader has become active or become inactive.

Parameters

<i>listener_data</i>	<<out>> Data associated with the listener when the listener is set
<i>writer</i>	<<out>> Locally created DDSDDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current reliable reader activity changed status of the locally created DDSDDataWriter

6.10.3.2 typedef void(* DDS_DataWriterListener_ReliableSampleUnacknowledgedCallback)(void *listener_data, DDS_DataWriter *writer, const struct DDS_ReliableSampleUnacknowledgedStatus *status)

<<eXtension>> <<cert>> Unacknowledged samples are removed from the writer cache.

Parameters

<i>listener_data</i>	<<out>> Data associated with the listener when the listener is set
<i>writer</i>	<<out>> Locally created DDSDDataWriter that triggers the listener callback
<i>status</i>	<<out>> Unacknowledged sample status for DDSDDataWriter

6.11 Subscription Module

Contains the [DDSSubscriber](#), and [DDSDataReader](#) classes, and more generally, all that is needed on the subscription side.

Modules

- [Subscriber](#)
[DDSSubscriber](#) entity and associated elements
- [DataReader](#)
[DDSDataReader](#) entity and associated elements
- [Data Sample](#)
[DDS_SampleInfo](#), [DDS_SampleStateKind](#), [DDS_ViewStateKind](#), [DDS_InstanceStateKind](#) and associated elements

6.11.1 Detailed Description

Contains the [DDSSubscriber](#), and [DDSDataReader](#) classes, and more generally, all that is needed on the subscription side. Defines the DDS subscription package.

This diagram defines the elements that are defined in the DDS specification, and their relationships to each other. Not all entities and functions shown in the diagram are supported by RTI Connex Micro. Please read the information on the individual types to determine whether a function is supported.

6.11.2 Access to data samples

Data is made available to the application by the following operations on [DDSDataReader](#) objects: [FooDataReader::read_next_sample](#), [FooDataReader::take_next_sample](#)

The general semantics of the `read()` operation is that the application only gets access to the corresponding data (i.e. a precise instance value); the data remains the responsibility of RTI Connex Micro and can be read again.

The semantics of the `take()` operations is that the application takes full responsibility for the data; that data will no longer be available locally to RTI Connex Micro. Consequently, it is possible to access the same information multiple times only if all previous accesses were `read()` operations, not `take()`.

Each of these operations returns a collection of `Data` values and associated [DDS_SampleInfo](#) objects. Each data value represents an atom of data information (i.e., a value for one instance). This collection may contain samples related to the same or different instances (identified by the key). Multiple samples can refer to the same instance if the settings of the [HISTORY](#) QoS allow for it.

To return the memory back to the middleware, every `read()` or `take()` that retrieves a sequence of samples must be followed with a call to [FooDataReader::return_loan](#).

See also

[Interpretation of the SampleInfo](#)

6.11.2.1 Data access patterns

The application accesses data by means of the operations `read` or `take` on the [DDSDataReader](#). These operations return an ordered collection of `DataSamples` consisting of a [DDS_SampleInfo](#) part and a `Data` part.

The way RTI Connex Micro builds the collection depends on QoS policies set on the [DDSDataReader](#) and [DDSSubscriber](#), as well as the `source_timestamp` of the samples, and the parameters passed to the `read()` / `take()` operations, namely:

- the desired sample states (any combination of [DDS_SampleStateKind](#))
- the desired view states (any combination of [DDS_ViewStateKind](#))

- the desired instance states (any combination of [DDS_InstanceStateKind](#))

The `read()` and `take()` operations are non-blocking and just deliver what is currently available that matches the specified states.

Once the data samples are available to the data readers, they can be read or taken by the application. The basic rule is that the application may do this in any order it wishes. This approach is very flexible and allows the application ultimate control.

6.12 Subscriber

[DDSSubscriber](#) entity and associated elements

Classes

- class [DDSSubscriberListener](#)
<<interface>> <<cert>> [DDSListener](#) for status about a subscriber.
- class [DDSSubscriber](#)
<<interface>> A subscriber is the object responsible for actually receiving data from a subscription.
- struct [DDS_SubscriberQos](#)
<<cert>> QoS policies supported by a [DDSSubscriber](#) entity.

Variables

- struct [DDS_DataReaderQos](#) [DDS_DATAREADER_QOS_DEFAULT](#)
<<cert>> Special value for creating data reader with default QoS.

6.12.1 Detailed Description

[DDSSubscriber](#) entity and associated elements

6.12.2 Variable Documentation

6.12.2.1 struct [DDS_DataReaderQos](#) [DDS_DATAREADER_QOS_DEFAULT](#)

<<cert>> Special value for creating data reader with default QoS.

When used in [DDSSubscriber::create_datareader](#), this special value is used to indicate that the [DDSDataReader](#) should be created with the default [DDSDataReader](#) QoS.

See also

[DDSSubscriber::create_datareader](#)

6.13 DataReader

[DDSDataReader](#) entity and associated elements

Classes

- struct [FooDataReader](#)
Declares the interface required to support a user data type-specific data reader.
- class [DDSDataReaderListener](#)
<<interface>> <<cert>> [DDSListener](#) for reader status.
- class [DDSDataReader](#)
<<interface>> <<cert>> Allows the application to: (1) declare the data it wishes to receive (i.e. make a subscription) and (2) access the data received by the attached [DDSSubscriber](#).
- struct [DDS_RequestedDeadlineMissedStatus](#)
<<cert>> [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#)
- struct [DDS_LivelinessChangedStatus](#)
<<cert>> [DDS_LIVELINESS_CHANGED_STATUS](#)
- struct [DDS_RequestedIncompatibleQosStatus](#)
<<cert>> [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#)
- struct [DDS_SampleRejectedStatus](#)
[DDS_SAMPLE_REJECTED_STATUS](#)
- struct [DDS_DataReaderInstanceReplacedStatus](#)
<<eXtension>> <<cert>> [DDS_INSTANCE_REPLACED_STATUS](#).
- struct [DDS_SubscriptionMatchedStatus](#)
<<cert>> [DDS_SUBSCRIPTION_MATCHED_STATUS](#)
- struct [DDS_SampleLostStatus](#)
<<cert>> [DDS_SAMPLE_LOST_STATUS](#)
- struct [DDS_DataReaderQos](#)
<<cert>> QoS policies supported by a [DDSDataReader](#) entity.

Enumerations

- enum [DDS_SampleRejectedStatusKind](#) {
[DDS_NOT_REJECTED](#), [DDS_REJECTED_BY_INSTANCES_LIMIT](#), [DDS_REJECTED_BY_SAMPLES_LIMIT](#), [DDS_REJECTED_BY_SAMPLES_PER_INSTANCE_LIMIT](#), [DDS_REJECTED_BY_REMOTE_WRITERS_LIMIT](#), [DDS_REJECTED_BY_SAMPLES_PER_REMOTE_WRITER_LIMIT](#) }
Kinds of reasons for rejecting a sample.
- enum [DDS_SampleLostStatusKind](#) {
[DDS_SAMPLE_LOST_NOT_LOST](#), [DDS_SAMPLE_LOST_BY_DATAWRITER](#), [DDS_SAMPLE_LOST_BY_MAX_SAMPLES_LIMIT](#), [DDS_SAMPLE_LOST_BY_HISTORY_DEPTH_LIMIT](#), [DDS_SAMPLE_LOST_BY_META_SAMPLE_LIMIT](#), [DDS_SAMPLE_LOST_BY_NOT_READ_ON_CACHE_DELETION](#) }
Kinds of reasons a sample was lost.

6.13.1 Detailed Description

[DDSDataReader](#) entity and associated elements

6.13.2 Enumeration Type Documentation

6.13.2.1 enum DDS_SampleRejectedStatusKind

Kinds of reasons for rejecting a sample.

Enumerator:

DDS_NOT_REJECTED Samples are never rejected.

See also

[DDS_ResourceLimitsQosPolicy](#)

DDS_REJECTED_BY_INSTANCES_LIMIT A resource limit on the number of instances was reached.

See also

[DDS_ResourceLimitsQosPolicy](#)

DDS_REJECTED_BY_SAMPLES_LIMIT A resource limit on the number of samples was reached.

See also

[DDS_ResourceLimitsQosPolicy](#)

DDS_REJECTED_BY_SAMPLES_PER_INSTANCE_LIMIT A resource limit on the number of samples per instance was reached.

See also

[DDS_ResourceLimitsQosPolicy](#)

DDS_REJECTED_BY_REMOTE_WRITERS_LIMIT A resource limit on the number of remote writers from which a [DDSDataReader](#) may read was reached. This constant is an extension to the DDS standard.

See also

[DDS_DataReaderResourceLimitsQosPolicy](#)

DDS_REJECTED_BY_SAMPLES_PER_REMOTE_WRITER_LIMIT A resource limit on the number of samples from a given remote writer that a [DDSDataReader](#) may store was reached. This constant is an extension to the DDS standard.

See also

[DDS_DataReaderResourceLimitsQosPolicy](#)

6.13.2.2 enum DDS_SampleLostStatusKind

Kinds of reasons a sample was lost.

Enumerator:

DDS_SAMPLE_LOST_NOT_LOST Samples was not lost, normal removal.

See also

[DDS_ResourceLimitsQosPolicy](#)

DDS_SAMPLE_LOST_BY_DATAWRITER Samples was lost was by the DataWriter, it was never received by the DataReader.

See also

[DDS_ResourceLimitsQosPolicy](#)

DDS_SAMPLE_LOST_BY_MAX_SAMPLES_LIMIT A sample was received by the DataReader but lost because the max_samples resource was exceeded and not samples could be freed to receive it.

See also

[DDS_ResourceLimitsQosPolicy](#)

DDS_SAMPLE_LOST_BY_HISTORY_DEPTH_LIMIT A sample was received by the DataReader but lost because the max depth was reached for the instance and no samples could be removed from the history. For example if all the samples in the DataReader cache is loaned by the application they cannot be removed.

See also

[DDS_ResourceLimitsQosPolicy](#)

DDS_SAMPLE_LOST_BY_META_SAMPLE_LIMIT A meta sample was received by the DataReader but lost because the meta sample for the instance was loaned by the application and could not be reused.

See also

[DDS_ResourceLimitsQosPolicy](#)

DDS_SAMPLE_LOST_BY_NOT_READ_ON_CACHE_DELETION The DataReader has samples in the cache that had not been seen by the application and was lost when the DataReader was deleted.

See also

[DDS_ResourceLimitsQosPolicy](#)

6.14 Data Sample

[DDS_SampleInfo](#), [DDS_SampleStateKind](#), [DDS_ViewStateKind](#), [DDS_InstanceStateKind](#) and associated elements

Modules

- [Sample States](#)
[DDS_SampleStateKind](#) and associated elements
- [View States](#)
[DDS_ViewStateKind](#) and associated elements
- [Instance States](#)
[DDS_InstanceStateKind](#) and associated elements

Classes

- struct [DDS_SampleInfo](#)
Information that accompanies each sample that is read or taken.
- struct [DDS_SampleInfoSeq](#)
Declares IDL sequence `< DDS\_SampleInfo >` .

6.14.1 Detailed Description

[DDS_SampleInfo](#), [DDS_SampleStateKind](#), [DDS_ViewStateKind](#), [DDS_InstanceStateKind](#) and associated elements

6.15 Sample States

[DDS_SampleStateKind](#) and associated elements

Typedefs

- typedef [DDS_UnsignedLong](#) [DDS_SampleStateMask](#)
A bit-mask (list) of sample states, i.e. [DDS_SampleStateKind](#).

Enumerations

- enum [DDS_SampleStateKind](#) { [DDS_READ_SAMPLE_STATE](#) = 0x0001 << 0, [DDS_NOT_READ_SAMPLE_STATE](#) = 0x0001 << 1 }
- Indicates whether or not a sample has ever been read.

Variables

- const [DDS_SampleStateMask](#) [DDS_ANY_SAMPLE_STATE](#)
Any sample state [DDS_READ_SAMPLE_STATE](#) | [DDS_NOT_READ_SAMPLE_STATE](#).

6.15.1 Detailed Description

[DDS_SampleStateKind](#) and associated elements

6.15.2 Typedef Documentation

6.15.2.1 typedef [DDS_UnsignedLong](#) [DDS_SampleStateMask](#)

A bit-mask (list) of sample states, i.e. [DDS_SampleStateKind](#).

6.15.3 Enumeration Type Documentation

6.15.3.1 enum [DDS_SampleStateKind](#)

Indicates whether or not a sample has ever been read.

For each sample received, the middleware internally maintains a sample_state relative to each [DDSDataReader](#). The sample state can be either:

- [DDS_READ_SAMPLE_STATE](#) indicates that the [DDSDataReader](#) has already accessed that sample by means of a read or take operation.
- [DDS_NOT_READ_SAMPLE_STATE](#) indicates that the [DDSDataReader](#) has not accessed that sample before.

The sample state will, in general, be different for each sample in the collection returned by read or take.

Enumerator:

- [DDS_READ_SAMPLE_STATE](#)** Sample has been read.
- [DDS_NOT_READ_SAMPLE_STATE](#)** Sample has not been read.

6.15.4 Variable Documentation

6.15.4.1 `const DDS_SampleStateMask DDS_ANY_SAMPLE_STATE`

Any sample state [DDS_READ_SAMPLE_STATE](#) | [DDS_NOT_READ_SAMPLE_STATE](#).

6.16 View States

[DDS_ViewStateKind](#) and associated elements

Typedefs

- typedef [DDS_UnsignedLong](#) [DDS_ViewStateMask](#)
A bit-mask (list) of view states, i.e. [DDS_ViewStateKind](#).

Enumerations

- enum [DDS_ViewStateKind](#) { [DDS_NEW_VIEW_STATE](#) = 0x0001 << 0, [DDS_NOT_NEW_VIEW_STATE](#) = 0x0001 << 1 }
- Indicates whether or not an instance is new.

Variables

- const [DDS_ViewStateMask](#) [DDS_ANY_VIEW_STATE](#)
Any view state [DDS_NEW_VIEW_STATE](#) | [DDS_NOT_NEW_VIEW_STATE](#).

6.16.1 Detailed Description

[DDS_ViewStateKind](#) and associated elements

6.16.2 Typedef Documentation

6.16.2.1 typedef [DDS_UnsignedLong](#) [DDS_ViewStateMask](#)

A bit-mask (list) of view states, i.e. [DDS_ViewStateKind](#).

6.16.3 Enumeration Type Documentation

6.16.3.1 enum [DDS_ViewStateKind](#)

Indicates whether or not an instance is new.

For each instance (identified by the key), the middleware internally maintains a view state relative to each [DDSDataReader](#). The view state can be either:

- [DDS_NEW_VIEW_STATE](#) indicates that either this is the first time that the [DDSDataReader](#) has ever accessed samples of that instance, or else that the [DDSDataReader](#) has accessed previous samples of the instance, but the instance has since been reborn (i.e. become not-alive and then alive again).
- [DDS_NOT_NEW_VIEW_STATE](#) indicates that the [DDSDataReader](#) has already accessed samples of the same instance and that the instance has not been reborn since.

The view_state available in the [DDS_SampleInfo](#) is a snapshot of the view state of the instance relative to the [DDSDataReader](#) used to access the samples at the time the collection was obtained (i.e. at the time read or take was called). The view_state is therefore the same for all samples in the returned collection that refer to the same instance.

Once an instance has been detected as not having any "live" writers and all the samples associated with the instance are "taken" from the [DDSDataReader](#), the middleware can reclaim all local resources regarding the instance. Future samples will be treated as "never seen."

Enumerator:

DDS_NEW_VIEW_STATE New instance. This latest generation of the instance has not previously been accessed.

DDS_NOT_NEW_VIEW_STATE Not a new instance. This latest generation of the instance has previously been accessed.

6.16.4 Variable Documentation

6.16.4.1 `const DDS_ViewStateMask DDS_ANY_VIEW_STATE`

Any view state [DDS_NEW_VIEW_STATE](#) | [DDS_NOT_NEW_VIEW_STATE](#).

6.17 Instance States

[DDS_InstanceStateKind](#) and associated elements

Typedefs

- typedef [DDS_UnsignedLong](#) [DDS_InstanceStateMask](#)
A bit-mask (list) of instance states, i.e. [DDS_InstanceStateKind](#).

Enumerations

- enum [DDS_InstanceStateKind](#) { [DDS_ALIVE_INSTANCE_STATE](#) = 0x0001 << 0, [DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE](#) = 0x0001 << 1, [DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#) = 0x0001 << 2 }
- Indicates if the samples are from a live [DDSDDataWriter](#) or not.

Variables

- const [DDS_InstanceStateMask](#) [DDS_ANY_INSTANCE_STATE](#)
Any instance state [ALIVE_INSTANCE_STATE](#) | [NOT_ALIVE_DISPOSED_INSTANCE_STATE](#) | [NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#).
- const [DDS_InstanceStateMask](#) [DDS_NOT_ALIVE_INSTANCE_STATE](#)
Not alive instance state [NOT_ALIVE_DISPOSED_INSTANCE_STATE](#) | [NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#).

6.17.1 Detailed Description

[DDS_InstanceStateKind](#) and associated elements

6.17.2 Typedef Documentation

6.17.2.1 typedef [DDS_UnsignedLong](#) [DDS_InstanceStateMask](#)

A bit-mask (list) of instance states, i.e. [DDS_InstanceStateKind](#).

6.17.3 Enumeration Type Documentation

6.17.3.1 enum [DDS_InstanceStateKind](#)

Indicates if the samples are from a live [DDSDDataWriter](#) or not.

For each instance, the middleware internally maintains an instance state. The instance state can be:

- [DDS_ALIVE_INSTANCE_STATE](#) indicates that (a) samples have been received for the instance, (b) there are live [DDSDDataWriter](#) entities writing the instance, and (c) the instance has not been explicitly disposed (or else more samples have been received after it was disposed).
- [DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE](#) indicates the instance was explicitly disposed by a [DDSDDataWriter](#) by means of the dispose operation.
- [DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#) indicates the instance has been declared as not-alive by the [DDSDDataReader](#) because it detected that there are no live [DDSDDataWriter](#) entities writing that instance.

The precise behavior events that cause the instance state to change depends on the setting of the OWNERSHIP QoS:

- If **OWNERSHIP** is set to **DDS_EXCLUSIVE_OWNERSHIP_QOS**, then the instance state becomes **DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE** only if the **DDSDDataWriter** that "owns" the instance explicitly disposes it. The instance state becomes **DDS_ALIVE_INSTANCE_STATE** again only if the **DDSDDataWriter** that owns the instance writes it.
- If **OWNERSHIP** is set to **DDS_SHARED_OWNERSHIP_QOS**, then the instance state becomes **DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE** if any **DDSDDataWriter** explicitly disposes the instance. The instance state becomes **DDS_ALIVE_INSTANCE_STATE** as soon as any **DDSDDataWriter** writes the instance again.

The instance state available in the **DDS_SampleInfo** is a snapshot of the instance state of the instance at the time the collection was obtained (i.e. at the time read or take was called). The instance state is therefore the same for all samples in the returned collection that refer to the same instance.

Enumerator:

DDS_ALIVE_INSTANCE_STATE Instance is currently in existence.

DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE Not alive disposed instance. The instance has been disposed by a DataWriter.

DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE Not alive no writers for instance. None of the **DDS-DataWriter** objects are currently alive (according to the **LIVELINESS**) are writing the instance.

6.17.4 Variable Documentation

6.17.4.1 `const DDS_InstanceStateMask DDS_ANY_INSTANCE_STATE`

Any instance state **ALIVE_INSTANCE_STATE** | **NOT_ALIVE_DISPOSED_INSTANCE_STATE** | **NOT_ALIVE_NO_WRITERS_INSTANCE_STATE**.

6.17.4.2 `const DDS_InstanceStateMask DDS_NOT_ALIVE_INSTANCE_STATE`

Not alive instance state **NOT_ALIVE_DISPOSED_INSTANCE_STATE** | **NOT_ALIVE_NO_WRITERS_INSTANCE_STATE**.

6.18 Infrastructure Module

Defines the DDS infrastructure package.

Modules

- [Time Support](#)
Time and duration types and defines.
- [GUID Support](#)
GUID type and defines.
- [Sequence Number Support](#)
Sequence number type and defines.
- [Return Codes](#)
Types of return codes.
- [Status Kinds](#)
Kinds of communication status.
- [QoS Policies](#)
Quality of Service (QoS) policies.
- [Conditions and WaitSets](#)
[DDSCondition](#) and [DDSWaitSet](#) and related items.
- [Entity Support](#)
[DDSEntity](#), [DDSListener](#) and related items.
- [Sequence Support](#)
Defines sequence interface and primitive data types sequences.
- [String Support](#)
String creation, cloning, assignment, and deletion.

6.18.1 Detailed Description

Defines the DDS infrastructure package. Defines the abstract classes and the interfaces that are refined by the other modules. Contains common definitions such as return codes, status values, and QoS policies.

This diagram defines the elements that are defined in the DDS specification, and their relationships to each other. Not all entities and functions shown in the diagram are supported by RTI Connex Micro. Please read the information on the individual types to determine whether a function is supported.

6.19 Documentation Guide

This section describes the conventions used in the API documentation.

6.19.1 Unsupported Features

[Not supported.] This note means that the feature from the DDS specification is not supported in the current release.

6.19.2 API Naming Conventions

6.19.2.1 Structure & Class Names

RTI Connex Micro makes a distinction between *value types* and *interface types*. Value types are types such as primitives, enumerations, strings, and structures whose identity and equality are determined solely by explicit state. Interface types are those abstract opaque data types that conceptually have an identity apart from their explicit state. Examples include all of the [DDSEntity](#) subtypes. Instances of value types are frequently transitory and are declared on the stack. Instances of interface types typically have longer lifecycles, are accessible by pointer only, and may be managed by a factory object.

Value type structures are made more explicit through the use of C's structure tag syntax. For example, a [DDS_Duration_t](#) object must be declared as being of type "struct DDS_Duration_t," not simply "DDS_Duration_t." Interface types, by contrast, are always of typedef'ed types; their underlying representations are opaque.

6.19.3 API Documentation Terms

In the API documentation, the term module refers to a logical grouping of documentation and elements in the API.

At this time, typedefs that occur in the API, such as [DDS_ReturnCode_t](#) do not show up in the compound list or indices. This is a known limitation in the generated HTML.

6.19.4 Stereotypes

Commonly used stereotypes in the API documentation include the following.

6.19.4.1 Extensions

- [<<eXtension>>](#)
 - An RTI Connex Micro product extension to the DDS standard specification.
 - The extension APIs complement the standard APIs specified by the OMG DDS specification. They are provided to improve product usability and enable access to product-specific features.

6.19.4.2 Types

- [<<interface>>](#)
 - Pure interface type with *no state*.
- [<<generic>>](#)
 - A *generic* type is a *skeleton* class written in terms of generic parameters. Type-specific instantiations of such types are conventionally referred to in this documentation in terms of the hypothetical type "Foo"; for example: [FooSeq](#), [FooDataType](#), [FooDataWriter](#), and [FooDataReader](#).
- [<<singleton>>](#)

- Singleton class. There is a single instance of the class.
- Generally accessed via a `get_instance()` static method.

6.19.4.3 Method Parameters

- `<<in>>`
 - An *input* parameter. The input parameter is a value parameter and is expected to be a valid value. If the parameter is a pointer, e.g. a pointer to an *interface types*, it is also an `<<in>>`, even if the content being pointed to has changed on return. `<<in>>` means that the pointer argument is not modified, not that the object pointed to is not modified.
- `<<out>>`
 - An *output* parameter. Only possible with references. The input value is not used, but the value may be changed on return, e.g. a field in a structure may have been updated or a pointer may point to a new object. In general, a parameter is designated with `<<out>>` only if the caller is expected to access the output directly.
- `<<inout>>`
 - An *input* and *output* parameter. Only possible with references. The input value must be valid, including the value of a pointer, and the parameter may have a new value on output; for example, the address of a pointer to a buffer is updated on return. In general, a parameter is designated with `<<inout>>` only if the caller is expected to access the output directly.

6.19.4.4 Cert API

- `<<cert>>`
 - Supported in safety certified version of RTI Connex Micro. Note, specific fields of `<<cert>>` data structures may not be supported and will be described in documentation as unsupported.

6.20 DDS C++ API

RTI Connex Micro modules following the DDS module definitions.

Modules

- [Domain Module](#)

Contains the [DDSDomainParticipant](#) class that acts as an entrypoint of RTI Connex Micro and acts as a factory for many of the classes. The [DDSDomainParticipant](#) also acts as a container for the other objects that make up RTI Connex Micro. Testing RTI Connex Micro command.

- [Topic](#)

Contains the [DDSTopic](#), the [DDSTopicListener](#) interface, and more generally, all that is needed by an application to define [DDSTopic](#) objects and the data types that are associated with [DDSTopic](#) objects.

- [Publication Module](#)

Contains the [DDSPublisher](#), and [DDSDataWriter](#) classes, and more generally, all that is needed on the publication side.

- [Subscription Module](#)

Contains the [DDSSubscriber](#), and [DDSDataReader](#) classes, and more generally, all that is needed on the subscription side.

- [Infrastructure Module](#)

Defines the DDS infrastructure package.

6.20.1 Detailed Description

RTI Connex Micro modules following the DDS module definitions.

6.20.2 Overview

Information flows with the aid of the following constructs: [DDSPublisher](#) and [DDSDataWriter](#) on the sending side, [DDSSubscriber](#) and [DDSDataReader](#) on the receiving side.

- A [DDSPublisher](#) is an object responsible for data distribution. It may publish data of different data types. A [DDSDataWriter](#) acts as an accessor to a publisher - each [DDSDataWriter](#) object is dedicated to one application data type. A [DDSDataWriter](#) is the object the application must use to communicate to a publisher the existence and value of data-objects of a given type. When data-object values have been communicated to the publisher through the appropriate data-writer, it is the publisher's responsibility to perform the distribution (the publisher will do this according to its own QoS, or the QoS attached to the corresponding data-writer). A *publication* is defined by the association of a data-writer to a publisher. This association expresses the intent of the application to publish the data described by the data-writer in the context provided by the publisher.
- A [DDSSubscriber](#) is an object responsible for receiving published data and making it available (according to the Subscriber's QoS) to the receiving application. It may receive and dispatch data of different specified types. To access the received data, the application must use a [DDSDataReader](#) attached to the subscriber. Thus, a *subscription* is defined by the association of a data-reader with a subscriber. This association expresses the intent of the application to subscribe to the data described by the data-reader in the context provided by the subscriber.

[DDSTopic](#) objects conceptually fit between publications and subscriptions. Publications must be known in such a way that subscriptions can refer to them unambiguously. A [DDSTopic](#) is meant to fulfill that purpose: it associates a name (unique in the domain i.e. the set of applications that are communicating with each other), a data type, and QoS related to the data itself. In addition to the topic QoS, the QoS of the [DDSDataWriter](#) associated with that Topic and the QoS of the [DDSPublisher](#) associated to the [DDSDataWriter](#) control the behavior on the publisher's side, while the corresponding [DDSTopic](#), [DDSDataReader](#) and [DDSSubscriber](#) QoS control the behavior on the subscriber's side.

When an application wishes to publish data of a given type, it must create a [DDSPublisher](#) (or reuse an already created one) and a [DDSDataWriter](#) with all the characteristics of the desired publication. Similarly, when an application wishes to receive data, it must create a [DDSSubscriber](#) (or reuse an already created one) and a [DDSDataReader](#) to define the subscription.

6.20.3 Conceptual Model

The overall conceptual model is shown below.

Notice that all the main communication objects (the specializations of Entity) follow unified patterns of:

- Supporting QoS (made up of several QoSPolicy); QoS provides a generic mechanism for the application to control the behavior of the Service and tailor it to its needs. Each [DDSEntity](#) supports its own specialized kind of QoS policies (see [QoS Policies](#)).
- Accepting a [DDSListener](#); listeners provide a generic mechanism for the middleware to notify the application of relevant asynchronous events, such as arrival of data corresponding to a subscription, violation of a QoS setting, etc. Each [DDSEntity](#) supports its own specialized kind of listener. Listeners are related to changes in status conditions (see [Status Kinds](#)).

Note that only one Listener per entity is allowed (instead of a list of them). The reason for that choice is that this allows a much simpler (and, thus, more efficient) implementation as far as the middleware is concerned. Moreover, if it were required, the application could easily implement a listener that, when triggered, triggers in return attached 'sub-listeners'.

All DCPS entities are attached to a [DDSDomainParticipant](#). A domain participant represents the local membership of the application in a domain. A *domain* is a distributed concept that links all the applications able to communicate with each other. It represents a communication plane: only the publishers and the subscribers attached to the same domain may interact.

[DDSDomainEntity](#) is an intermediate object whose only purpose is to state that a DomainParticipant cannot contain other domain participants.

At the DCPS level, data types represent information that is sent atomically. For performance reasons, only plain data structures are handled by this level.

By definition, a [DDSTopic](#) corresponds to a single data type. However, several topics may refer to the same data type. Therefore, a [DDSTopic](#) identifies data of a single type, ranging from one single instance to a whole collection of instances of that given type. This is shown below for the hypothetical data type `F00`.

In case a set of instances is gathered under the same topic, different instances must be distinguishable. This is achieved by means of the values of some data fields that form the **key** to that data set. The *key description* (i.e., the list of data fields whose value forms the key) has to be indicated to the middleware. The rule is simple: *different data samples with the same key value represent successive values for the same instance, while different data samples with different key values represent different instances*. If no key is provided, the data set associated with the [DDSTopic](#) is restricted to a *single instance*.

Topics need to be known by the middleware and potentially propagated. Topic objects are created using the create operations provided by [DDSDomainParticipant](#).

The interaction style is straightforward on the publisher's side: when the application decides that it wants to make data available for publication, it calls the appropriate operation on the related [DDSDataWriter](#) (this, in turn, will trigger its [DDSPublisher](#)).

On the subscriber's side however, there are more choices: relevant information may arrive when the application is busy doing something else or when the application is just waiting for that information. Therefore, depending on the way the application is designed, asynchronous notifications or synchronous access may be more appropriate. Both interaction modes are allowed, a [DDSListener](#) is used to provide a callback for asynchronous access, and polling allows for synchronous access.

6.20.4 Modules

DCPS consists of five modules:

- [Infrastructure module](#) defines the abstract classes and the interfaces that are refined by the other modules. It also provides support for the notification-based interaction style with the middleware.
- [Domain module](#) contains the [DDSDomainParticipant](#) class that acts as an entrypoint of the Service and acts as a factory for many of the classes. The [DDSDomainParticipant](#) also acts as a container for the other objects that make up the Service.
- [Topic module](#) contains the [DDSTopic](#) class, and more generally, all that is needed by the application to define [DDSTopic](#) objects and attach QoS policies to them.
- [Publication module](#) contains the [DDSPublisher](#) and [DDSDataWriter](#) classes, and more generally, all that is needed on the publication side.
- [Subscription module](#) contains the [DDSSubscriber](#), and [DDSDataReader](#), and more generally, all that is needed on the subscription side.

6.21 DPSE Static Discovery API

Classes

- class [DPSEDiscoveryFactory](#)
<<eXtension>> <<cert>> A discovery factory that will be used by the [DDSDomainParticipantFactory](#) to create a discovery plugin.
- class [DPSEDiscoveryPlugin](#)
<<eXtension>> <<cert>> Discovery plug-in used for asserting remote entities.
- struct [DPSE_DiscoveryPluginProperty](#)
<<eXtension>> <<cert>> Properties for the Dynamic Participant/Static Endpoint (DPSE) discovery plugin. This includes all discovery timing properties for participant discovery.

Macros

- `#define DPSE_DiscoveryPluginProperty_INITIALIZER`
Initializer for the [DPSE_DiscoveryPluginProperty](#).

6.21.1 Detailed Description

<<eXtension>> <<cert>> The RTI Connex Micro support for DPSE discovery of remote entities. Please refer to the Discovery chapter in the User's Manual, and the Getting Started Guide, for an example of how to register and use the DPSE plugin.

6.21.2 Macro Definition Documentation

6.21.2.1 `#define DPSE_DiscoveryPluginProperty_INITIALIZER`

Value:

```
{ \
    RT_ComponentFactoryProperty_INITIALIZER, \
    {30, 0}, \
    {100, 0}, \
    5, \
    {1, 0}, \
    DDS_LENGTH_AUTO, \
    4 \
}
```

Initializer for the [DPSE_DiscoveryPluginProperty](#).

6.22 DPDE Dynamic Discovery API

Classes

- class [DPDEDiscoveryFactory](#)
<<eXtension>> Discovery plug-in used for asserting remote entities.
- struct [DPDE_DiscoveryPluginProperty](#)
<<eXtension>> Properties for the Dynamic Participant/Dynamic Endpoint (DPDE) discovery plugin. This includes all discovery timing properties for participant discovery.

Macros

- `#define DPDE_DiscoveryPluginProperty_INITIALIZER`
Initializer for the [DPDE_DiscoveryPluginProperty](#).

6.22.1 Detailed Description

<<eXtension>> The RTI Connex Micro support for DPDE discovery of remote entities. Please refer to Discovery chapter in the User's Manual, and the Getting Started Guide, for an example of how to register and use the DPDE plugin.

6.22.2 Macro Definition Documentation

6.22.2.1 `#define DPDE_DiscoveryPluginProperty_INITIALIZER`

Value:

```
{\
  RT_ComponentFactoryProperty_INITIALIZER,\
  {30,0}, \
  {100,0}, \
  5, \
  {1,0}, \
  DDS_BOOLEAN_FALSE, \
  DDS_LENGTH_AUTO,\
  4,\
  8,\
  DDS_MAX_UNLIMITED,\
  DDS_LENGTH_UNLIMITED,\
  {0,100000000},\
  DDS_LENGTH_UNLIMITED,\
  DDS RTPS_NACK_PERIOD_DEFAULT \
}
```

Initializer for the [DPDE_DiscoveryPluginProperty](#).

6.23 RT Run-Time API

Classes

- class [RTRegistry](#)
<<eXtension>> A run-time component factory.

6.23.1 Detailed Description

The run-time module (RT) manages RTI Connex Micro modules. It provides a common interface to add, remove, and lookup components.

6.24 RHSM Reader History API

Classes

- class [RHSMHistoryFactory](#)
<<eXtension>> <<cert>> A reader history factory that will be used by the DataReader to create a history cache.

6.24.1 Detailed Description

The RHSM plugin is an implementation of the RTI Connext Micro ReaderHistory plugin interface and implements the DataReader reader cache as supported by RTI Connext Micro. RTI Connext Micro requires that a ReaderHistory plugin is registered with the name "rh" in the DomainParticipantFactory before a DataReader can be created. Each instance of a DataReader creates its own instance of the ReaderHistory plugin. Please refer to User Manual for an example on how to register the writer history factory.

6.25 WHSM Writer History API

Classes

- class [WHSMHistoryFactory](#)
<<eXtension>> <<cert>> A writer history factory that will be used by the DataWriter to create a history cache.

6.25.1 Detailed Description

The WHSM plugin is an implementation of the RTI Connex Micro WriterHistory plugin interface and implements the DataWriter writer cache as supported by RTI Connex Micro. RTI Connex Micro requires that a WriterHistory plugin is registered with the name "wh" in the DomainParticipantFactory before a DataWriter can be created. Each instance of a DataWriter creates its own instance of the WriterHistory plugin. Please refer to User Manual for an example on how to register the writer history factory.

6.26 UDP Transport API

Classes

- class `UDPInterfaceFactory`
<<eXtension>> <<cert>> A UDP transport factory that will be used by a DomainParticipant to create a UDP interface. Only one UDP interface can be registered with the DomainParticipantFactory.
- class `UDPInterfaceTable`
- struct `UDP_NatEntry`
<<eXtension>> Properties for a Network Address Translation entry
- struct `UDP_NatEntrySeq`
<<eXtension>> Declares IDL sequence<UDP_NatEntry>
- struct `UDP_InterfaceTableEntry`
Generic structure to describe a network interface.
- struct `UDP_InterfaceTableEntrySeq`
<<eXtension>> Declares IDL sequence<UDP_InterfaceTableEntry>
- struct `UDP_InterfaceFactoryProperty`
<<eXtension>> <<cert>> Properties for the UDP Transport.

Macros

- `#define UDP_INTERFACE_MAX_IFNAME 64`
Maximum length of a UDP network interface.
- `#define UDP_INTERFACE_INTERFACE_UP_FLAG 0x1U`
Generic flag to indicate if a network stack is up.
- `#define UDP_INTERFACE_INTERFACE_MULTICAST_FLAG 0x2U`
Generic flag to indicate if a network stack supports multicast.
- `#define UDP_INTERFACE_MAX_NETMASK_BITS (32U)`
The maximum number of bits in a UDP netmask.

Functions

- static bool `UDPInterfaceTable::add_entry` (struct `UDP_InterfaceTableEntrySeq` *seq, `RTI_UINT32` address, `RTI_UINT32` netmask, const char *ifname, `RTI_UINT32` flags)
Add an entry to the list of available interfaces.

6.26.1 Detailed Description

<<eXtension>> <<cert>> The UDP transport implements the RTI Connex Micro NETIO interface and supports UDPv4. Please refer to the User Manual for information about how to use the UDP transport.

6.26.2 Macro Definition Documentation

6.26.2.1 `#define UDP_INTERFACE_MAX_IFNAME 64`

Maximum length of a UDP network interface.

6.26.2.2 `#define UDP_INTERFACE_INTERFACE_UP_FLAG 0x1U`

Generic flag to indicate if a network stack is up.

6.26.2.3 `#define UDP_INTERFACE_INTERFACE_MULTICAST_FLAG 0x2U`

Generic flag to indicate if a network stack supports multicast.

6.26.2.4 `#define UDP_INTERFACE_MAX_NETMASK_BITS (32U)`

The maximum number of bits in a UDP netmask.

6.26.3 Function Documentation

6.26.3.1 `static bool UDPInterfaceTable::add_entry ( struct UDP_InterfaceTableEntrySeq * seq, RTI_UINT32 address, RTI_UINT32 netmask, const char * ifname, RTI_UINT32 flags ) [static]`

Add an entry to the list of available interfaces.

This function can be used to add an interface to the `if_table` in the [UDP_InterfaceFactoryProperty](#).

Parameters

in	<i>seq</i>	The sequence to add the entry to
in	<i>address</i>	The UDPv4 interface address to add in host order
in	<i>netmask</i>	The interface netmask, maximum of UDP_INTERFACE_MAX_NETMASK_BITS bits.
in	<i>ifname</i>	The interface name. Cannot exceed UDP_INTERFACE_MAX_IFNAME including NUL.
in	<i>flags</i>	Bitmap of UDP_INTERFACE_INTERFACE_UP_FLAG and/or UDP_INTERFACE_INTERFACE_MULTICAST_FLAG

Returns

RTI_TRUE on success, RTI_FALSE on failure

MT Safety:

This operation is not thread safe.

API Restriction:

This function *must* only be called after [DDSDomainParticipantFactory::get_instance](#).

6.27 UDP Transform API

<<*eXtension*>> <<*cert*>> The UDP transform API enables custom transformation of UDP incoming and outgoing UDP datagrams. Please refer to User Manual for the User Manual and how to use the UDP transforms.

6.28 RTPS Transport API

Classes

- union [RTPS_Checksum](#)
<<eXtension>> <<cert>> RTPS checksum type
- struct [RTPS_ChecksumClass](#)
<<eXtension>> <<cert>> Definition of a checksum function
- struct [RTPS_ChecksumProperty](#)
<<eXtension>> <<cert>> Checksum properties for the RTPS plugin
- struct [RTPS_InterfaceFactoryProperty](#)
<<eXtension>> <<cert>> Properties for the RTPS Transport.

Typedefs

- typedef union [RTPS_Checksum](#) [RTPS_Checksum_T](#)
<<eXtension>> <<cert>> RTPS checksum type
- typedef [RTI_BOOL](#)(* [RTPS_ChecksumCalculate_T](#))(void *context, const struct REDA_Buffer *buf, [RTI_UINT32](#) buf_length, [RTPS_Checksum_T](#) *checksum)
<<eXtension>> <<cert>> The checksum calculation function
- typedef struct [RTPS_ChecksumClass](#) [RTPS_ChecksumClass_T](#)
<<eXtension>> <<cert>> Definition of a checksum function
- typedef struct
[RTPS_InterfaceFactoryProperty](#) [RTPS_InterfaceFactoryProperty_T](#)
<<eXtension>> <<cert>> Properties for the RTPS Transport.

Enumerations

- enum [RTPS_ChecksumTxMode_T](#) { [RTPS_CHECKSUM_TXMODE_RTICRC32](#), [RTPS_CHECKSUM_TXMODE_OMG](#) }
<<eXtension>> <<cert>> Transmit mode for checksums

Variables

- [RTI_UINT32](#) [RTPS_Checksum::checksum32](#)
A 32-bit unsigned checksum in host endianness order.
- [RTI_UINT64](#) [RTPS_Checksum::checksum64](#)
A 64-bit unsigned integer checksum in host endianness order.
- [RTI_UINT8](#) [RTPS_Checksum::checksum128](#) [16]
A 128-bit checksum as a 16-byte array.
- [RTPS_ChecksumClassId_T](#) [RTPS_ChecksumClass::class_id](#)
The checksum class ID.
- void * [RTPS_ChecksumClass::context](#)
User defined context.
- [RTPS_ChecksumCalculate_T](#) [RTPS_ChecksumClass::checksum_calculate](#)
User defined function to calculate a checksum.
- [RTPS_ChecksumClass_T](#) [RTPS_ChecksumProperty::builtin_checksum32_class](#)
The builtin 32-bit checksum class.
- [RTPS_ChecksumClass_T](#) [RTPS_ChecksumProperty::builtin_checksum64_class](#)
The builtin 64-bit checksum class.
- [RTPS_ChecksumClass_T](#) [RTPS_ChecksumProperty::builtin_checksum128_class](#)

The builtin 128-bit checksum class.

- `RTPS_ChecksumTxMode_T RTPS_ChecksumProperty::checksum_tx_mode`

The transmit mode for checksums.

- `RTI_BOOL RTPS_ChecksumProperty::allow_builtin_override`

If TRUE, allow overriding the builtin checksum functions.

- `struct RTPS_ChecksumProperty RTPS_InterfaceFactoryProperty::checksum`

The checksum properties for RTPS.

- `struct RTPS_InterfaceFactoryProperty RTPS_INTERFACE_FACTORY_DEFAULT`

`<<eXtension>> <<cert>>` Default properties for the RTPS Transport.

6.28.1 Detailed Description

`<<eXtension>> <<cert>>` The RTPS Plugin implements the RTI Connex Micro RTPS protocol as an RTI Connex Micro NETIO interface.

6.28.2 Typedef Documentation

6.28.2.1 typedef union RTPS_Checksum RTPS_Checksum_T

`<<eXtension>> <<cert>>` RTPS checksum type

6.28.2.2 typedef RTI_BOOL(* RTPS_ChecksumCalculate_T)(void *context, const struct REDA_Buffer *buf, RTI_UINT32 buf_length, RTPS_Checksum_T *checksum)

`<<eXtension>> <<cert>>` The checksum calculation function

Parameters

in	<i>context</i>	Context passed as property
in	<i>buf</i>	Vector of buffers to calculate the checksum over
in	<i>buf_length</i>	Number of elements in buf
out	<i>checksum</i>	Calculated checksum

Returns

RTI_TRUE on success, RTI_FALSE on failure

6.28.2.3 typedef struct RTPS_ChecksumClass RTPS_ChecksumClass_T

`<<eXtension>> <<cert>>` Definition of a checksum function

6.28.2.4 typedef struct RTPS_InterfaceFactoryProperty RTPS_InterfaceFactoryProperty_T

`<<eXtension>> <<cert>>` Properties for the RTPS Transport.

6.28.3 Enumeration Type Documentation

6.28.3.1 enum RTPS_ChecksumTxMode_T

`<<eXtension>> <<cert>>` Transmit mode for checksums

Enumerator:

RTPS_CHECKSUM_TXMODE_RTICRC32 Always use CRC32, fail if not possible.

RTPS_CHECKSUM_TXMODE_OMG Always use OMG's header extension [Default].

6.28.4 Variable Documentation

6.28.4.1 `RTI_UINT32 RTPS_Checksum::checksum32`

A 32-bit unsigned checksum in host endianness order.

6.28.4.2 `RTI_UINT64 RTPS_Checksum::checksum64`

A 64-bit unsigned integer checksum in host endianness order.

6.28.4.3 `RTI_UINT8 RTPS_Checksum::checksum128[16]`

A 128-bit checksum as a 16-byte array.

6.28.4.4 `RTPS_ChecksumClassId_T RTPS_ChecksumClass::class_id`

The checksum class ID.

6.28.4.5 `void* RTPS_ChecksumClass::context`

User defined context.

6.28.4.6 `RTPS_ChecksumCalculate_T RTPS_ChecksumClass::checksum_calculate`

User defined function to calculate a checksum.

6.28.4.7 `RTPS_ChecksumClass_T RTPS_ChecksumProperty::builtin_checksum32_class`

The builtin 32-bit checksum class.

6.28.4.8 `RTPS_ChecksumClass_T RTPS_ChecksumProperty::builtin_checksum64_class`

The builtin 64-bit checksum class.

6.28.4.9 `RTPS_ChecksumClass_T RTPS_ChecksumProperty::builtin_checksum128_class`

The builtin 128-bit checksum class.

6.28.4.10 `RTPS_ChecksumTxMode_T RTPS_ChecksumProperty::checksum_tx_mode`

The transmit mode for checksums.

6.28.4.11 `RTI_BOOL RTPS_ChecksumProperty::allow_builtin_override`

If TRUE, allow overriding the builtin checksum functions.

6.28.4.12 `struct RTPS_ChecksumProperty RTPS_InterfaceFactoryProperty::checksum`

The checksum properties for RTPS.

6.28.4.13 `struct RTPS_InterfaceFactoryProperty RTPS_INTERFACE_FACTORY_DEFAULT`

<<eXtension>> <<cert>> Default properties for the RTPS Transport.

6.29 OS API

Modules

- [OSAPI Heap](#)
- [OSAPI Mutex](#)
Mutex API.
- [OSAPI AutoSAR](#)
AutoSAR System API.
- [OSAPI Semaphore](#)
Abstract Semaphore API.
- [OSAPI System API](#)
Abstract System API.
- [OSAPI Thread API](#)
OSAPI Thread API.
- [OSAPI Types](#)
OSAPI Types.

6.29.1 Detailed Description

6.30 Log API

Classes

- struct [OSAPI_LogProperty](#)
Configuration of logging functionality.

Typedefs

- typedef void(* [OSAPI_Log_write_buffer_T](#))(const char *buffer, [RTI_SIZE_T](#) length)
Optional user-defined function to output a log/trace buffer.
- typedef void(* [OSAPI_LogHandler_T](#))(void *param, [OSAPI_LogEntry_T](#) *entry)
Optional user-defined function for processing new log messages.

Enumerations

- enum [OSAPI_LogVerbosity_T](#) { [OSAPI_LOG_VERBOSITY_SILENT](#) = 0, [OSAPI_LOG_VERBOSITY_ERROR](#) = 1, [OSAPI_LOG_VERBOSITY_WARNING](#) = 2, [OSAPI_LOG_VERBOSITY_DEBUG](#) = 3 }
Logging verbosity.

Functions

- [RTI_BOOL OSAPI_Log_set_log_handler](#) ([OSAPI_LogHandler_T](#) handler, void *param)
Install a log handler.
- [RTI_BOOL OSAPI_Log_get_log_handler](#) ([OSAPI_LogHandler_T](#) *handler, void **param)
Return the current log handler.
- [RTI_BOOL OSAPI_Log_set_trace_handler](#) ([OSAPI_TraceHandler_T](#) handler, void *param)
Install a trace handler.
- [RTI_BOOL OSAPI_Log_get_trace_handler](#) ([OSAPI_TraceHandler_T](#) *handler, void **param)
Return the trace handler.
- [RTI_BOOL OSAPI_Log_set_display_handler](#) ([OSAPI_LogDisplay_T](#) handler, void *param)
Install a display handler.
- [RTI_BOOL OSAPI_Log_get_display_handler](#) ([OSAPI_LogDisplay_T](#) *handler, void **param)
Return the current display function.
- void [OSAPI_Log_set_verbosity](#) ([OSAPI_LogVerbosity_T](#) verbosity)
Set the log verbosity.
- void [OSAPI_Trace_set_trace_mask](#) ([RTI_UINT32](#) mask)
Set the trace mask.
- [RTI_BOOL OSAPI_Log_clear](#) (void)
Clear the log buffer.
- [RTI_INT32 OSAPI_Log_get_last_error_code](#) (void)
Returns the error code for a function that failed.
- void [OSAPI_Log_get_property](#) (struct [OSAPI_LogProperty](#) *property)
Get the current log properties.
- [RTI_BOOL OSAPI_Log_set_property](#) (struct [OSAPI_LogProperty](#) *property)
Set the log properties.

6.30.1 Detailed Description

6.30.2 Typedef Documentation

6.30.2.1 `typedef void(* OSAPI_Log_write_buffer_T)(const char *buffer, RTI_SIZE_T length)`

Optional user-defined function to output a log/trace buffer.

The handler is set by [OSAPI_Log_set_property\(\)](#).

Parameters

in	<i>buffer</i>	Pointer to buffer with log to write
in	<i>length</i>	Length of the buffer to write

6.30.2.2 `typedef void(* OSAPI_LogHandler_T)(void *param, OSAPI_LogEntry_T *entry)`

Optional user-defined function for processing new log messages.

Definition of a function that can be installed with the logger and that will be called for each new log event.

The handler is set by [OSAPI_Log_set_log_handler\(\)](#).

When called, provides parameters containing the raw log entry and additional optional information.

Parameters

in	<i>param</i>	User-defined payload
in	<i>msg</i>	Log entry
in	<i>kind</i>	Kind of log message
in	<i>ec</i>	Unique error code of log message
in	<i>module</i>	Module of log message
in	<i>file</i>	Name of source file. Optional.
in	<i>func</i>	Name of function. Optional.
in	<i>line</i>	Line number. Optional.
in	<i>argc</i>	Variable length argument count
in	<i>ap</i>	Variable length argument list

6.30.3 Enumeration Type Documentation

6.30.3.1 `enum OSAPI_LogVerbosity_T`

Logging verbosity.

Enumerator:

`OSAPI_LOG_VERBOSITY_SILENT` Logs are not written.

`OSAPI_LOG_VERBOSITY_ERROR` Only error logs are written.

`OSAPI_LOG_VERBOSITY_WARNING` Error and warning logs are written.

`OSAPI_LOG_VERBOSITY_DEBUG` All logs are written.

6.30.4 Function Documentation

6.30.4.1 `RTI_BOOL OSAPI_Log_set_log_handler ( OSAPI_LogHandler_T handler, void * param )`

Install a log handler.

The log functionality allows the user to specify a log handler. The log handler is a function which is called for every logged event. It is up to the user to decide what to do with the log message. The handler is a global function pointer.

Parameters

in	<i>handler</i>	Pointer to log handler function
in	<i>param</i>	Parameter passed to the log handler function. This parameter is transparent to the log functionality.

Returns

RTI_TRUE on success, RTI_FALSE on failure

6.30.4.2 RTI_BOOL OSAPI_Log_get_log_handler (OSAPI_LogHandler_T * *handler*, void ** *param*)

Return the current log handler.

Return the current log handler.

Parameters

in	<i>handler</i>	Pointer to store log handler function
in	<i>param</i>	Pointer to store the current log handler parameter.

Returns

RTI_TRUE on success, RTI_FALSE on failure

6.30.4.3 RTI_BOOL OSAPI_Log_set_trace_handler (OSAPI_TraceHandler_T *handler*, void * *param*)

Install a trace handler.

Install a custom trace handler. Traces are not stored in the log-buffer and is generally used to analyze behavior interactively. the default trace handler outputs the trace data to a console.

Parameters

in	<i>handler</i>	Pointer to trace handler function
in	<i>param</i>	Parameter passed to the trace handler function. This parameter is transparent to the trace functionality.

Returns

RTI_TRUE on success, RTI_FALSE on failure

6.30.4.4 RTI_BOOL OSAPI_Log_get_trace_handler (OSAPI_TraceHandler_T * *handler*, void ** *param*)

Return the trace handler.

Return the current trace handler and trace parameter. This information can be used to daisy-chain calls to multiple trace-handlers.

Parameters

in	<i>handler</i>	Pointer to store trace handler function
in	<i>param</i>	Pointer to store the trace handler parameter.

Returns

RTI_TRUE on success, RTI_FALSE on failure

6.30.4.5 RTI_BOOL OSAPI_Log_set_display_handler (OSAPI_LogDisplay_T handler, void * param)

Install a display handler.

The display handler is responsible for outputting log messages to a console.

Parameters

in	handler	Pointer to display function
in	param	Parameter passed to the display handler function. This parameter is transparent to the log functionality.

Returns

RTI_TRUE on success, RTI_FALSE on failure

6.30.4.6 RTI_BOOL OSAPI_Log_get_display_handler (OSAPI_LogDisplay_T * handler, void ** param)

Return the current display function.

Return the current log handler.

Parameters

in	handler	Pointer to store display handler function
in	param	Pointer to store the current display handler parameters.

Returns

RTI_TRUE on success, RTI_FALSE on failure

6.30.4.7 void OSAPI_Log_set_verbosity (OSAPI_LogVerbosity_T verbosity)

Set the log verbosity.

Change the log verbosity. The new setting takes immediate effect.

Parameters

in	verbosity	New log verbosity
----	-----------	-------------------

6.30.4.8 void OSAPI_Trace_set_trace_mask (RTI_UINT32 mask)

Set the trace mask.

Parameters

in	mask	New trace mask
----	------	----------------

6.30.4.9 RTI_BOOL OSAPI_Log_clear (void)

Clear the log buffer.

Clear the log buffer, all the current entries are lost

NOTE: This function must only be called inside a log handler as it is not thread-safe.

Returns

RTI_TRUE on success, RTI_FALSE on failure

6.30.4.10 RTI_INT32 OSAPI_Log_get_last_error_code (void)

Returns the error code for a function that failed.

Many functions returns RTI_FALSE or NULL on failure. In order to provide additional information about reason for the failure fucntions may set an additional error code. This function returns the last error-code recorded for the calling thread.

Returns

Last error-code recorded for this thread

6.30.4.11 void OSAPI_Log_get_property (struct OSAPI_LogProperty * *property*)

Get the current log properties.

Return the current log properties

Parameters

out	<i>property</i>	- Current log properties
-----	-----------------	--------------------------

See Also OSAPI_Log_set_property

6.30.4.12 RTI_BOOL OSAPI_Log_set_property (struct OSAPI_LogProperty * *property*)

Set the log properties.

Set the current log properties. It is not possible to set new properties after OSAPI_Log_initialize() has been called/

Parameters

in	<i>property</i>	New log properties
----	-----------------	--------------------

Returns

RTI_TRUE if new properties can be set, RTI_FALSE otherwise See Also OSAPI_Log_get_property

6.31 DDS-Specific Primitive Types

Basic DDS value types for use in user data types.

Macros

- `#define DDS_BOOLEAN_TRUE`
Defines TRUE value of IDL boolean data type.
- `#define DDS_BOOLEAN_FALSE`
Defines FALSE value of IDL boolean data type.

Typedefs

- `typedef CDR_Char DDS_Char`
Defines IDL char data type.
- `typedef CDR_Wchar DDS_Wchar`
Defines IDL wchar data type.
- `typedef CDR_Octet DDS_Octet`
Defines IDL octet data type.
- `typedef CDR_Short DDS_Short`
Defines IDL short data type.
- `typedef CDR_UnsignedShort DDS_UnsignedShort`
Defines IDL unsigned short data type.
- `typedef CDR_Long DDS_Long`
Defines IDL long data type.
- `typedef CDR_UnsignedLong DDS_UnsignedLong`
Defines IDL unsigned long data type.
- `typedef CDR_LongLong DDS_LongLong`
Defines IDL long long data type.
- `typedef CDR_UnsignedLongLong DDS_UnsignedLongLong`
Defines IDL unsigned long long data type.
- `typedef CDR_Float DDS_Float`
Defines IDL float data type.
- `typedef CDR_Double DDS_Double`
Defines IDL double data type.
- `typedef CDR_LongDouble DDS_LongDouble`
Defines IDL long double data type.
- `typedef CDR_Boolean DDS_Boolean`
Defines IDL boolean data type.
- `typedef CDR_Enum DDS_Enum`
Defines IDL enum data type.
- `typedef CDR_String DDS_String`
IDL string type representation.
- `typedef CDR_Wstring DDS_Wstring`
Defines IDL wstring data type.

6.31.1 Detailed Description

Basic DDS value types for use in user data types. As part of the finalization of the DDS standard, a number of DDS-specific primitive types will be introduced. By using these types, you will ensure that your data is serialized consistently across platforms even if the built-in types have different sizes on those platforms.

In this version of RTI Connex Micro, the DDS primitive types are defined using the OMG's Common Data Representation (CDR) standard.

6.31.2 Macro Definition Documentation

6.31.2.1 #define DDS_BOOLEAN_TRUE

Defines TRUE value of IDL boolean data type.

6.31.2.2 #define DDS_BOOLEAN_FALSE

Defines FALSE value of IDL boolean data type.

6.31.3 Typedef Documentation

6.31.3.1 typedef CDR_Char DDS_Char

Defines IDL char data type.

An 8-bit quantity that encodes a single byte character from any byte-oriented code set.

6.31.3.2 typedef CDR_Wchar DDS_Wchar

Defines IDL wchar data type.

An 16-bit quantity that encodes a wide character from any character set.

6.31.3.3 typedef CDR_Octet DDS_Octet

Defines IDL octet data type.

An 8-bit quantity that is guaranteed not to undergo any conversion when transmitted by the middleware.

6.31.3.4 typedef CDR_Short DDS_Short

Defines IDL short data type.

A 16-bit signed short integer value.

6.31.3.5 typedef CDR_UnsignedShort DDS_UnsignedShort

Defines IDL unsigned short data type.

A 16-bit unsigned short integer value.

6.31.3.6 typedef CDR_Long DDS_Long

Defines IDL long data type.

A 32-bit signed long integer value.

6.31.3.7 typedef CDR_UnsignedLong DDS_UnsignedLong

Defines IDL unsigned long data type.

A 32-bit unsigned long integer value.

6.31.3.8 typedef CDR_LongLong DDS_LongLong

Defines IDL long long data type.

A 64-bit signed long long integer value.

Since some architectures do not support long long (8 bytes long), RTI has defined character arrays that match the expected size of these types.

6.31.3.9 typedef CDR.UnsignedLongLong DDS_UnsignedLongLong

Defines IDL unsigned long long data type.

An 64-bit unsigned long long integer value.

Since some architectures do not support (unsigned) long long (8 byte long), RTI has defined character arrays that match the expected size of these types.

6.31.3.10 typedef CDR.Float DDS_Float

Defines IDL float data type.

A 32-bit floating point value.

6.31.3.11 typedef CDR.Double DDS_Double

Defines IDL double data type.

A 64-bit floating point value.

6.31.3.12 typedef CDR.LongDouble DDS_LongDouble

Defines IDL long double data type.

A 16-byte floating point value.

Since some architectures do not support long double, RTI has defined character arrays that match the expected size of this type. On systems that do have native long double, you have to define `RTI_CDR_SIZEOF_LONG_DOUBLE` as 16 to map them to native types.

6.31.3.13 typedef CDR.Boolean DDS_Boolean

Defines IDL boolean data type.

An 8-bit boolean value that is used to denote a data item that can only take one of the values [DDS_BOOLEAN_TRUE](#) (1) or [DDS_BOOLEAN_FALSE](#) (0).

6.31.3.14 typedef CDR.Enum DDS_Enum

Defines IDL enum data type.

Encoded as unsigned long value. The first enum identifier has the numeric value zero (0). Successive enum identifiers take ascending numeric values, in order of declaration from left to right.

6.31.3.15 typedef CDR.String DDS_String

IDL `string` type representation.

Has a range of ISO Latin-1 (except ASCII NULL) and a variable length.

See also

[String Support](#)

6.31.3.16 typedef CDR.Wstring DDS_Wstring

Defines IDL wstring data type.

Each character composing the string is a 32-bit quantity that encodes a wide character from any character set.

6.32 Participant Built-in Topic

Builtin topic for accessing information about the DomainParticipants discovered by RTI Connex Micro.

Classes

- struct [DDS_ParticipantBuiltinTopicData](#)
<<cert>> Object representing a remote DomainParticipant.

Macros

- #define [DDS_PARTICIPANT_TOPIC_NAME](#) DDS_PARTICIPANT_BUILTIN_TOPIC_NAME
<<cert>> Participant topic name

6.32.1 Detailed Description

Builtin topic for accessing information about the DomainParticipants discovered by RTI Connex Micro.

6.32.2 Macro Definition Documentation

6.32.2.1 #define DDS_PARTICIPANT_TOPIC_NAME DDS_PARTICIPANT_BUILTIN_TOPIC_NAME

<<cert>> Participant topic name

Topic name of participant builtin topic data DataReader.

See also

[DDS_ParticipantBuiltinTopicData](#)

6.33 Publication Built-in Topic

Builtin topic for configuring information about the Publications to be discovered by RTI Connext Micro.

Classes

- struct [DDS_PublicationBuiltinTopicData](#)
Object describing a remote Publication.

Macros

- `#define DDS_PUBLICATION_TOPIC_NAME DDS_PUBLICATION_BUILTIN_TOPIC_NAME`
Publication topic name.

6.33.1 Detailed Description

Builtin topic for configuring information about the Publications to be discovered by RTI Connext Micro.

6.33.2 Macro Definition Documentation

6.33.2.1 `#define DDS_PUBLICATION_TOPIC_NAME DDS_PUBLICATION_BUILTIN_TOPIC_NAME`

Publication topic name.

Topic name of publication builtin topic data DataReader.

See also

[DDS_PublicationBuiltinTopicData](#)

6.34 Subscription Built-in Topic

Builtin topic for configuring information about the Subscriptions to be discovered by RTI Connex Micro.

Classes

- struct [DDS_SubscriptionBuiltinTopicData](#)

Entry created when a [DDSDataReader](#) is discovered in association with its Subscriber.

Macros

- `#define DDS_SUBSCRIPTION_TOPIC_NAME DDS_SUBSCRIPTION_BUILTIN_TOPIC_NAME`

Subscription topic name.

6.34.1 Detailed Description

Builtin topic for configuring information about the Subscriptions to be discovered by RTI Connex Micro.

6.34.2 Macro Definition Documentation

6.34.2.1 `#define DDS_SUBSCRIPTION_TOPIC_NAME DDS_SUBSCRIPTION_BUILTIN_TOPIC_NAME`

Subscription topic name.

Topic name of subscription builtin topic data DataReader.

See also

[DDS_SubscriptionBuiltinTopicData](#)

6.35 DISCOVERY

<<*eXtension*>> Specifies the attributes required to discover participants in the domain.

Classes

- struct [DDS_DiscoveryQosPolicy](#)
 <<*cert*>> *Specifies the attributes required to discover participants in the domain.*

Variables

- const char *const [DDS_DISCOVERY_QOS_POLICY_NAME](#)
 Stringified human-readable name for [DDS_DiscoveryQosPolicy](#).

6.35.1 Detailed Description

<<*eXtension*>> Specifies the attributes required to discover participants in the domain.

6.35.2 Variable Documentation

6.35.2.1 const char* const DDS_DISCOVERY_QOS_POLICY_NAME

Stringified human-readable name for [DDS_DiscoveryQosPolicy](#).

6.36 Time Support

Time and duration types and defines.

Classes

- struct [DDS_Time_t](#)
Type for time representation.
- struct [DDS_Duration_t](#)
Type for duration representation.

Macros

- `#define DDS_TIME_ZERO`
The default instant in time: zero seconds and zero nanoseconds.

Variables

- const [DDS_Long](#) [DDS_TIME_INVALID_SEC](#)
A sentinel indicating an invalid second of time.
- const [DDS_UnsignedLong](#) [DDS_TIME_INVALID_NSEC](#)
A sentinel indicating an invalid nano-second of time.
- struct [DDS_Time_t](#) [DDS_TIME_INVALID](#)
A sentinel indicating an invalid time.
- const [DDS_Long](#) [DDS_DURATION_INFINITE_SEC](#)
An infinite second period of time.
- const [DDS_UnsignedLong](#) [DDS_DURATION_INFINITE_NSEC](#)
An infinite nano-second period of time.
- struct [DDS_Duration_t](#) [DDS_DURATION_INFINITE](#)
An infinite period of time.
- const [DDS_Long](#) [DDS_DURATION_ZERO_SEC](#)
A zero-length second period of time.
- const [DDS_UnsignedLong](#) [DDS_DURATION_ZERO_NSEC](#)
A zero-length nano-second period of time.
- struct [DDS_Duration_t](#) [DDS_DURATION_ZERO](#)
A zero-length period of time.

6.36.1 Detailed Description

Time and duration types and defines.

6.36.2 Macro Definition Documentation

6.36.2.1 `#define DDS_TIME_ZERO`

The default instant in time: zero seconds and zero nanoseconds.

6.36.3 Variable Documentation

6.36.3.1 const [DDS_Long](#) [DDS_TIME_INVALID_SEC](#)

A sentinel indicating an invalid second of time.

6.36.3.2 const DDS_UnsignedLong DDS_TIME_INVALID_NSEC

A sentinel indicating an invalid nano-second of time.

6.36.3.3 struct DDS_Time_t DDS_TIME_INVALID

A sentinel indicating an invalid time.

6.36.3.4 const DDS_Long DDS_DURATION_INFINITE_SEC

An infinite second period of time.

6.36.3.5 const DDS_UnsignedLong DDS_DURATION_INFINITE_NSEC

An infinite nano-second period of time.

6.36.3.6 struct DDS_Duration_t DDS_DURATION_INFINITE

An infinite period of time.

6.36.3.7 const DDS_Long DDS_DURATION_ZERO_SEC

A zero-length second period of time.

6.36.3.8 const DDS_UnsignedLong DDS_DURATION_ZERO_NSEC

A zero-length nano-second period of time.

6.36.3.9 struct DDS_Duration_t DDS_DURATION_ZERO

A zero-length period of time.

6.37 GUID Support

GUID type and defines.

Classes

- struct [DDS_GUID_t](#)
Type for GUID (Global Unique Identifier) representation.

Variables

- struct [DDS_GUID_t](#) [DDS_GUID_UNKNOWN](#)
Unknown GUID.

6.37.1 Detailed Description

GUID type and defines.

6.37.2 Variable Documentation

6.37.2.1 struct [DDS_GUID_t](#) [DDS_GUID_UNKNOWN](#)

Unknown GUID.

6.38 Sequence Number Support

Sequence number type and defines.

Classes

- struct [DDS_SequenceNumber_t](#)
Type for sequence number representation.

Functions

- int [DDS_SequenceNumber_compare](#) (const struct [DDS_SequenceNumber_t](#) *sn1, const struct [DDS_SequenceNumber_t](#) *sn2)
Compares two sequence numbers.

6.38.1 Detailed Description

Sequence number type and defines.

6.38.2 Function Documentation

6.38.2.1 int [DDS_SequenceNumber_compare](#) (const struct [DDS_SequenceNumber_t](#) * *sn1*, const struct [DDS_SequenceNumber_t](#) * *sn2*)

Compares two sequence numbers.

Parameters

<i>sn1</i>	<< <i>in</i> >> Sequence number to compare. Cannot be NULL.
<i>sn2</i>	<< <i>in</i> >> Sequence number to compare. Cannot be NULL.

Returns

If the two sequence numbers are equal, the function returns 0. If sn1 is greater than sn2 the function returns a positive number; otherwise, it returns a negative number.

6.39 Return Codes

Types of return codes.

Enumerations

- enum `DDS_ReturnCode_t` {
`DDS_RETCODE_OK`, `DDS_RETCODE_ERROR`, `DDS_RETCODE_UNSUPPORTED`, `DDS_RETCODE_BAD_PARAMETER`,
`DDS_RETCODE_PRECONDITION_NOT_MET`, `DDS_RETCODE_OUT_OF_RESOURCES`, `DDS_RETCODE_NOT_ENABLED`, `DDS_RETCODE_IMMUTABLE_POLICY`,
`DDS_RETCODE_INCONSISTENT_POLICY`, `DDS_RETCODE_ALREADY_DELETED`, `DDS_RETCODE_TIMEOUT`, `DDS_RETCODE_NO_DATA`,
`DDS_RETCODE_ILLEGAL_OPERATION` }

Type for return codes.

6.39.1 Detailed Description

Types of return codes.

6.39.2 Standard Return Codes

Any operation with return type `DDS_ReturnCode_t` may return `DDS_RETCODE_OK`, `DDS_RETCODE_ERROR` or `DDS_RETCODE_ILLEGAL_OPERATION`. Any operation that takes one or more input parameters may additionally return `DDS_RETCODE_BAD_PARAMETER`. Any operation on an object created from any of the factories may additionally return `DDS_RETCODE_ALREADY_DELETED`. Any operation that is stated as optional may additionally return `DDS_RETCODE_UNSUPPORTED`.

Thus, the standard return codes are:

- `DDS_RETCODE_ERROR`
- `DDS_RETCODE_ILLEGAL_OPERATION`
- `DDS_RETCODE_ALREADY_DELETED`
- `DDS_RETCODE_BAD_PARAMETER`
- `DDS_RETCODE_UNSUPPORTED`

Operations that may return any of the additional return codes will state so explicitly.

6.39.3 Enumeration Type Documentation

6.39.3.1 enum `DDS_ReturnCode_t`

Type for return codes.

Errors are modeled as operation return codes of this type.

Enumerator:

`DDS_RETCODE_OK` Successful return.

`DDS_RETCODE_ERROR` Generic, unspecified error.

`DDS_RETCODE_UNSUPPORTED` Unsupported operation. Can only returned by operations that are unsupported.

DDS_RETCODE_BAD_PARAMETER Illegal parameter value. The value of the parameter that is passed in has illegal value. Things that falls into this category includes NULL parameters and parameter values that are out of range.

DDS_RETCODE_PRECONDITION_NOT_MET A pre-condition for the operation was not met. The system is not in the expected state when the function is called, or the parameter itself is not in the expected state when the function is called.

DDS_RETCODE_OUT_OF_RESOURCES RTI Connex Micro ran out of the resources needed to complete the operation.

DDS_RETCODE_NOT_ENABLED Operation invoked on a [DDSEntity](#) that is not yet enabled.

DDS_RETCODE_IMMUTABLE_POLICY Application attempted to modify an immutable QoS policy.

DDS_RETCODE_INCONSISTENT_POLICY Application specified a set of QoS policies that are not consistent with each other.

DDS_RETCODE_ALREADY_DELETED The object target of this operation has already been deleted.

DDS_RETCODE_TIMEOUT The operation timed out.

DDS_RETCODE_NO_DATA Indicates a transient situation where the operation did not return any data but there is no inherent error.

DDS_RETCODE_ILLEGAL_OPERATION The operation was called under improper circumstances. An operation was invoked on an inappropriate object or at an inappropriate time. This return code is similar to [DDS_RETCODE_PRECONDITION_NOT_MET](#), except that there is no precondition that could be changed to make the operation succeed.

6.40 Status Kinds

Kinds of communication status.

Macros

- `#define DDS_STATUS_MASK_NONE`
No bits are set.
- `#define DDS_STATUS_MASK_ALL`
All bits are set.

Typedefs

- `typedef DDS_UnsignedLong DDS_StatusMask`
A bit-mask (list) of concrete status types, i.e. `DDS_StatusKind[]`.

Enumerations

- `enum DDS_StatusKind {`
`DDS_INCONSISTENT_TOPIC_STATUS, DDS_OFFERED_DEADLINE_MISSED_STATUS, DDS_REQUESTED_DEADLINE_MISSED_STATUS, DDS_OFFERED_INCOMPATIBLE_QOS_STATUS,`
`DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS, DDS_SAMPLE_LOST_STATUS, DDS_SAMPLE_REJECTED_STATUS, DDS_DATA_ON_READERS_STATUS,`
`DDS_DATA_AVAILABLE_STATUS, DDS_LIVELINESS_LOST_STATUS, DDS_LIVELINESS_CHANGED_STATUS, DDS_PUBLICATION_MATCHED_STATUS,`
`DDS_SUBSCRIPTION_MATCHED_STATUS, DDS_INSTANCE_REPLACED_STATUS, DDS_RELIABLE_READER_ACTIVITY_CHANGED_STATUS }`
Type for status kinds.

6.40.1 Detailed Description

Kinds of communication status.

Entity:

[DDSEntity](#)

QoS:

[QoS Policies](#)

Listener:

[DDSListener](#)

Each concrete [DDSEntity](#) is associated with a set of Status objects whose value represents the communication status of that entity. Each status value can be accessed with a corresponding method on the [DDSEntity](#).

RTI Connex Micro supports a subset of these statuses.

When these status values change, the proper [DDSListener](#) objects are invoked to asynchronously inform the application.

An application is notified of communication status by means of the [DDSListener](#) mechanism.

The statuses may be classified into:

- *read communication statuses:* i.e., those that are related to arrival of data, namely [DDS_DATA_ON_READERS_STATUS](#) and [DDS_DATA_AVAILABLE_STATUS](#).

- *plain communication statuses*: i.e., all the others.

Read communication statuses are treated slightly differently than the others because they don't change independently. In other words, at least two changes will appear at the same time ([DDS_DATA_ON_READERS_STATUS](#) and [DDS_DATA_AVAILABLE_STATUS](#)) and even several of the last kind may be part of the set. This 'grouping' has to be communicated to the application.

For each plain communication status, there is a corresponding structure to hold the status value. These values contain the information related to the change of status, as well as information related to the statuses themselves (e.g., contains cumulative counts).

6.40.2 Changes in Status

Associated with each one of an [DDSEntity](#)'s communication status is a logical `StatusChangedFlag`. This flag indicates whether that particular communication status has changed since the last time the status was read by the application. The way the status changes is slightly different for the Plain Communication Status and the Read Communication status.

6.40.2.1 Changes in plain communication status

For the plain communication status, the `StatusChangedFlag` flag is initially set to `FALSE`. It becomes `TRUE` whenever the plain communication status changes and it is reset to [DDS_BOOLEAN_FALSE](#) each time the application accesses the plain communication status via the proper `get_<plain communication status>()` operation on the [DDSEntity](#).

The communication status is also reset to `FALSE` whenever the associated listener operation is called as the listener implicitly accesses the status which is passed as a parameter to the operation. The fact that the status is reset prior to calling the listener means that if the application calls the `get_<plain communication status>` from inside the listener it will see the status already reset.

An exception to this rule is when the associated listener is the 'nil' listener. The 'nil' listener is treated as a NOOP and the act of calling the 'nil' listener does not reset the communication status.

6.40.2.2 Changes in read communication status

For the read communication status, the `StatusChangedFlag` flag is initially set to `FALSE`. The `StatusChangedFlag` becomes `TRUE` when either a data-sample arrives or else the [DDS_ViewStateKind](#), [DDS_SampleStateKind](#), or [DDS_InstanceStateKind](#) of any existing sample changes for any reason other than a call to [FooDataReader::read_next_sample](#), or [FooDataReader::take_next_sample](#). Specifically any of the following events will cause the `StatusChangedFlag` to become `TRUE`:

- The arrival of new data.
- A change in the [DDS_InstanceStateKind](#) of a contained instance. This can be caused by either:
 - The arrival of the notification that an instance has been disposed by:
 - * the [DDSDDataWriter](#) that owns it if `OWNERSHIP` QoS kind= [DDS_EXCLUSIVE_OWNERSHIP_QOS](#)
 - * or by any [DDSDDataWriter](#) if `OWNERSHIP` QoS kind= [DDS_SHARED_OWNERSHIP_QOS](#)
 - The loss of liveliness of the [DDSDDataWriter](#) of an instance for which there is no other [DDSDDataWriter](#).
 - The arrival of the notification that an instance has been unregistered by the only [DDSDDataWriter](#) that is known to be writing the instance.

Depending on the kind of `StatusChangedFlag`, the flag transitions to `FALSE` again as follows:

- The [DDS_DATA_AVAILABLE_STATUS](#) `StatusChangedFlag` becomes `FALSE` when either the corresponding listener operation (`on_data_available`) is called or the `read_next_sample` or `take_next_sample` operation is called on the associated [DDSDDataReader](#).

The communication statuses whose changes can be communicated to the application depend on the [DDSEntity](#).

Each status value can be accessed with a corresponding method on the [DDSEntity](#). The changes on these status values cause activation of the corresponding trigger invocation of the proper [DDSListener](#) objects to asynchronously inform the application.

See also

[DDSEntity](#), [DDSListener](#)

Enumerator:

DDS_INCONSISTENT_TOPIC_STATUS Another topic exists with the same name but different characteristics.

Entity:

[DDSTopic](#)

Status:

[DDS_InconsistentTopicStatus](#)

Listener:

[DDSTopicListener](#)

DDS_OFFERED_DEADLINE_MISSED_STATUS The deadline that the [DDSDataWriter](#) has committed through its [DDS_DeadlineQosPolicy](#) was not respected for a specific instance.

Entity:

[DDSDataWriter](#)

QoS:

[DEADLINE](#)

Status:

[DDS_OfferedDeadlineMissedStatus](#)

Listener:

[DDSDataWriterListener](#)

DDS_REQUESTED_DEADLINE_MISSED_STATUS The deadline that the [DDSDataReader](#) was expecting through its [DDS_DeadlineQosPolicy](#) was not respected for a specific instance.

Entity:

[DDSDataReader](#)

QoS:

[DEADLINE](#)

Status:

[DDS_RequestedDeadlineMissedStatus](#)

Listener:

[DDSDataReaderListener](#)

DDS_OFFERED_INCOMPATIBLE_QOS_STATUS A QosPolicy value was incompatible with what was requested.

Entity:

[DDSDataWriter](#)

Status:

[DDS_OfferedIncompatibleQosStatus](#)

Listener:

[DDSDataWriterListener](#)

DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS A QosPolicy value was incompatible with what is offered.

Entity:

[DDSDataReader](#)

Status:

[DDS_RequestedIncompatibleQosStatus](#)

Listener:

[DDSDataReaderListener](#)

DDS_SAMPLE_LOST_STATUS A sample has been lost (i.e. was never received).

Entity:

[DDSSubscriber](#)

Status:

[DDS_SampleLostStatus](#)

Listener:

[DDSSubscriberListener](#)

DDS_SAMPLE_REJECTED_STATUS A (received) sample has been rejected.

Entity:

[DDSDataReader](#)

QoS:

[RESOURCE_LIMITS](#)

Status:

[DDS_SampleRejectedStatus](#)

Listener:

[DDSDataReaderListener](#)

DDS_DATA_ON_READERS_STATUS New data is available.

Entity:

[DDSSubscriber](#)

Listener:

[DDSSubscriberListener](#)

DDS_DATA_AVAILABLE_STATUS One or more new data samples have been received.

Entity:

[DDSDataReader](#)

Listener:

[DDSDataReaderListener](#)

DDS_LIVELINESS_LOST_STATUS The liveliness that the [DDSDataWriter](#) has committed to through its [DDS_LivelinessQosPolicy](#) was not respected, thus [DDSDataReader](#) entities will consider the [DDSDataWriter](#) as no longer alive.

Entity:

[DDSDataWriter](#)

QoS:

[LIVELINESS](#)

Status:

[DDS_LivelinessLostStatus](#)

Listener:

[DDSDataWriterListener](#)

DDS_LIVELINESS_CHANGED_STATUS The liveliness of one or more [DDSDataWriter](#) that were writing instances read through the [DDSDataReader](#) has changed. Some [DDSDataWriter](#) have become alive or not_alive.

Entity:

[DDSDataReader](#)

QoS:

[LIVELINESS](#)

Status:

[DDS_LivelinessChangedStatus](#)

Listener:

[DDSDataReaderListener](#)

DDS_PUBLICATION_MATCHED_STATUS The [DDSDataWriter](#) has found [DDSDataReader](#) that matches the [DDSTopic](#) and has compatible QoS.

Entity:

[DDSDataWriter](#)

Status:

[DDS_PublicationMatchedStatus](#)

Listener:

[DDSDataWriterListener](#)

DDS_SUBSCRIPTION_MATCHED_STATUS The [DDSDataReader](#) has found [DDSDataWriter](#) that matches the [DDSTopic](#) and has compatible QoS.

Entity:

[DDSDataReader](#)

Status:

[DDS_SubscriptionMatchedStatus](#)

Listener:

[DDSDataReaderListener](#)

DDS_INSTANCE_REPLACED_STATUS The [DDSDataReader](#) has replaced an instance.

Entity:

[DDSDataReader](#)

Status:

[DDS_DataReaderInstanceReplacedStatus](#)

Listener:

[DDSDataReaderListener](#)

DDS_RELIABLE_READER_ACTIVITY_CHANGED_STATUS The [DDSDataWriter](#) has detected a change of activity in a reliable [DDSDataReader](#).

Entity:

[DDSDataWriter](#)

Status:

[DDS_ReliableReaderActivityChangedStatus](#)

Listener:

[DDSDataWriterListener](#)

6.41 QoS Policies

Quality of Service (QoS) policies.

Modules

- **DISCOVERY**
<<eXtension>> Specifies the attributes required to discover participants in the domain.
- **DEADLINE**
Expresses the maximum duration (deadline) within which an instance is expected to be updated.
- **OWNERSHIP**
Specifies whether it is allowed for multiple [DDSDataWriter](#) (s) to write the same instance of the data and if so, how these modifications should be arbitrated.
- **OWNERSHIP_STRENGTH**
Specifies the value of the strength used to arbitrate among multiple [DDSDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).
- **LIVELINESS**
Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.
- **RELIABILITY**
Indicates the level of reliability offered/requested by RTI Connext Micro.
- **HISTORY**
Specifies the behavior of RTI Connext Micro in the case where the value of an instance changes (one or more times) before it can be successfully communicated to one or more existing subscribers.
- **DURABILITY**
<<eXtension>> Specifies the Durability properties used by a [DDSDataWriter](#) and/or [DDSDataReader](#).
- **RESOURCE_LIMITS**
Specifies the resources that RTI Connext Micro can consume in order to meet the requested QoS.
- **PRESENTATION**
Specifies how the samples representing changes to data instances are presented to a subscribing application.
- **DESTINATION_ORDER**
<<eXtension>> Specifies the DestinationOrder policy.
- **ENTITY_FACTORY**
A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.
- **Extended Qos Support**
<<eXtension>> Types and defines used in extended QoS policies.
- **SYSTEM_RESOURCE_LIMITS**
<<eXtension>> Specifies the system-specific resources that RTI Connext Micro can use.
- **WIRE_PROTOCOL**
<<eXtension>> Specifies the wire protocol related attributes for the [DDSDomainParticipant](#).
- **DATA_READER_PROTOCOL**
<<eXtension>> Specifies the DataReader-specific protocol QoS.
- **DATA_WRITER_PROTOCOL**
<<eXtension>> Specifies the DataWriter-specific protocol QoS.
- **TRANSPORT**
<<eXtension>> <<cert>> Specify transport settings
- **DOMAIN_PARTICIPANT_RESOURCE_LIMITS**
<<eXtension>> Specifies the DomainParticipant-specific resources that RTI Connext Micro can use.
- **ENTITY_NAME**
<<eXtension>> Name of the entity.

Classes

- struct [RTI_ManagementQosPolicy](#)
<<cert>> Policy used determine how [DDSDataReader](#), [DDSDataWriter](#), [DDSPublisher](#), and [DDSSubscriber](#) is managed.

Enumerations

- enum [DDS_QosPolicyId_t](#) {
[DDS_INVALID_QOS_POLICY_ID](#) , [DDS_PRESENTATION_QOS_POLICY_ID](#), [DDS_DEADLINE_QOS_POLICY_ID](#) , [DDS_OWNERSHIP_QOS_POLICY_ID](#),
[DDS_OWNERSHIPSTRENGTH_QOS_POLICY_ID](#), [DDS_LIVELINESS_QOS_POLICY_ID](#) , [DDS_RELIABILITY_QOS_POLICY_ID](#) , [DDS_HISTORY_QOS_POLICY_ID](#) ,
[DDS_ENTITYFACTORY_QOS_POLICY_ID](#) }
Type to identify QosPolicies.

6.41.1 Detailed Description

Quality of Service (QoS) policies. Data Distribution Service (DDS) relies on the use of QoS. A QoS is a set of characteristics that controls some aspect of the behavior of DDS. A QoS is comprised of individual QoS policies (objects conceptually deriving from an *abstract QosPolicy* class).

RTI Connex Micro supports a subset of these QoS.

The QosPolicy provides the basic mechanism for an application to specify quality of service parameters. It has an attribute name that is used to uniquely identify each QosPolicy.

QosPolicy implementation is comprised of a name, an ID, and a type. The type of a QosPolicy value may be atomic, such as an integer or float, or compound (a structure). Compound types are used whenever multiple parameters must be set coherently to define a consistent value for a QosPolicy.

QoS (i.e., a list of QosPolicy objects) may be associated with all [DDSEntity](#) objects in the system such as [DDSTopic](#), [DDSDataWriter](#), [DDSDataReader](#), [DDSPublisher](#), [DDSSubscriber](#), and [DDSDomainParticipant](#).

6.41.2 Specifying QoS on entities

QosPolicies can be set programmatically when an [DDSEntity](#) is created, or modified with the [DDSEntity's set_qos \(abstract\)](#) method.

To customize a [DDSEntity's](#) QoS before creating the entity, the correct pattern is:

- First, initialize a QoS object with the appropriate INITIALIZER constructor.
- Call the relevant `get_<entity>_default_qos()` method.
- Modify the QoS values as desired.
- Finally, create the entity.

Each QosPolicy is treated independently from the others. This approach has the advantage of being very extensible. However, there may be cases where several policies are in conflict. Consistency checking is performed each time the policies are modified via the [set_qos \(abstract\)](#) operation, or when the [DDSEntity](#) is created.

When a policy is changed after being set to a given value, it is not required that the new value be applied instantaneously; RTI Connex Micro is allowed to apply it after a transition phase. In addition, some QosPolicy have immutable semantics, meaning that they can only be specified either at [DDSEntity](#) creation time or else prior to calling the [DDSEntity::enable](#) operation on the entity.

Each [DDSEntity](#) can be configured with a list of QosPolicy objects. However, not all QosPolicies are supported by each [DDSEntity](#). For instance, a [DDSDomainParticipant](#) supports a different set of QosPolicies than a [DDSTopic](#) or a [DDSPublisher](#).

6.41.3 QoS compatibility

In several cases, for communications to occur properly (or efficiently), a `QoSPolicy` on the publisher side must be compatible with a corresponding policy on the subscriber side. For example, if a `DDSSubscriber` requests to receive data reliably while the corresponding `DDSPublisher` defines a best-effort policy, communication will not happen as requested.

To address this issue and maintain the desirable decoupling of publication and subscription as much as possible, the `QoSPolicy` specification follows the **subscriber-requested, publisher-offered pattern**.

In this pattern, the subscriber side can specify a "requested" value for a particular `QoSPolicy`. The publisher side specifies an "offered" value for that `QoSPolicy`. RTI Connex Micro will then determine whether the value requested by the subscriber side is compatible with what is offered by the publisher side. If the two policies are compatible, then communication will be established. If the two policies are not compatible, RTI Connex Micro will not establish communications between the two `DDSEntity` objects and will record this fact by means of the `DDS_OFFERED_INCOMPATIBLE_QOS_STATUS` on the publisher end and `DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS` on the subscriber end. The application can detect this fact by means of a `DDSListener`.

The following **properties** are defined on a `QoSPolicy`.

- **RxO property**

The `QoSPolicy` objects that need to be set in a compatible manner between the **publisher** and **subscriber** end are indicated by the setting of the `RxO` property:

- `RxO = YES` indicates that the policy can be set both at the publishing and subscribing ends and the values must be set in a compatible manner. In this case the compatible values are explicitly defined.
- `RxO = NO` indicates that the policy can be set both at the publishing and subscribing ends but the two settings are independent. That is, all combinations of values are compatible.
- `RxO = N/A` indicates that the policy can only be specified at either the publishing or the subscribing end, but not at both ends. So compatibility does not apply.

- **Changeable property**

Determines whether a `QoSPolicy` can be changed.

NO – policy can only be specified at `DDSEntity` creation time.

UNTIL ENABLE – policy can only be changed before the `DDSEntity` is enabled.

YES – policy can be changed at any time.

6.41.4 Enumeration Type Documentation

6.41.4.1 enum `DDS_QoSPolicyId_t`

Type to identify `QoSPolicies`.

Note that the value of these constants disagree with the values of the corresponding parameter IDs in the RTPS protocol. This conflict is unavoidable since these values are given in the DDS specification, which is not tied to RTPS.

Enumerator:

`DDS_INVALID_QOS_POLICY_ID` Identifier for an invalid QoS policy.

`DDS_PRESENTATION_QOS_POLICY_ID` Identifier for `DDS_PresentationQoSPolicy`.

`DDS_DEADLINE_QOS_POLICY_ID` Identifier for `DDS_DeadlineQoSPolicy`.

`DDS_OWNERSHIP_QOS_POLICY_ID` Identifier for `DDS_OwnershipQoSPolicy`.

`DDS_OWNERSHIPSTRENGTH_QOS_POLICY_ID` Identifier for `DDS_OwnershipStrengthQoSPolicy`.

`DDS_LIVELINESS_QOS_POLICY_ID` Identifier for `DDS_LivelinessQoSPolicy`.

`DDS_RELIABILITY_QOS_POLICY_ID` Identifier for `DDS_ReliabilityQoSPolicy`.

`DDS_HISTORY_QOS_POLICY_ID` Identifier for `DDS_HistoryQoSPolicy`.

`DDS_ENTITYFACTORY_QOS_POLICY_ID` Identifier for `DDS_EntityFactoryQoSPolicy`.

6.42 DEADLINE

Expresses the maximum duration (deadline) within which an instance is expected to be updated.

Classes

- struct [DDS_DeadlineQosPolicy](#)
<<cert>> Expresses the maximum duration (deadline) within which an instance is expected to be updated.

Variables

- const char *const [DDS_DEADLINE_QOS_POLICY_NAME](#)
Stringified human-readable name for [DDS_DeadlineQosPolicy](#).

6.42.1 Detailed Description

Expresses the maximum duration (deadline) within which an instance is expected to be updated.

6.42.2 Variable Documentation

6.42.2.1 const char* const DDS_DEADLINE_QOS_POLICY_NAME

Stringified human-readable name for [DDS_DeadlineQosPolicy](#).

6.43 OWNERSHIP

Specifies whether it is allowed for multiple [DDSDataWriter](#) (s) to write the same instance of the data and if so, how these modifications should be arbitrated.

Classes

- struct [DDS_OwnershipQosPolicy](#)
<<cert>> Specifies whether it is allowed for multiple [DDSDataWriter](#) (s) to write the same instance of the data and if so, how these modifications should be arbitrated.

Enumerations

- enum [DDS_OwnershipQosPolicyKind](#) { [DDS_SHARED_OWNERSHIP_QOS](#), [DDS_EXCLUSIVE_OWNERSHIP_QOS](#) }
Kinds of ownership.

Variables

- const char *const [DDS_OWNERSHIP_QOS_POLICY_NAME](#)
Stringified human-readable name for [DDS_OwnershipQosPolicy](#).

6.43.1 Detailed Description

Specifies whether it is allowed for multiple [DDSDataWriter](#) (s) to write the same instance of the data and if so, how these modifications should be arbitrated.

6.43.2 Enumeration Type Documentation

6.43.2.1 enum [DDS_OwnershipQosPolicyKind](#)

Kinds of ownership.

QoS:

[DDS_OwnershipQosPolicy](#)

Enumerator:

[DDS_SHARED_OWNERSHIP_QOS](#) [default] Indicates shared ownership for each instance. Multiple writers are allowed to update the same instance and all the updates are made available to the readers. In other words there is no concept of an owner for the instances.

This is the **default** behavior if the [OWNERSHIP](#) policy is not specified or supported.

[DDS_EXCLUSIVE_OWNERSHIP_QOS](#) Indicates each instance can only be owned by one [DDSDataWriter](#), but the owner of an instance can change dynamically. The selection of the owner is controlled by the setting of the [OWNERSHIP_STRENGTH](#) policy. The owner is always set to be the highest-strength [DDSDataWriter](#) object among the ones currently active (as determined by the [LIVELINESS](#)).

6.43.3 Variable Documentation

6.43.3.1 const char* const [DDS_OWNERSHIP_QOS_POLICY_NAME](#)

Stringified human-readable name for [DDS_OwnershipQosPolicy](#).

6.44 OWNERSHIP_STRENGTH

Specifies the value of the strength used to arbitrate among multiple [DDSDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).

Classes

- struct [DDS_OwnershipStrengthQosPolicy](#)
<<cert>> Specifies the value of the strength used to arbitrate among multiple [DDSDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).

Variables

- const char *const [DDS_OWNERSHIPSTRENGTH_QOS_POLICY_NAME](#)
Stringified human-readable name for [DDS_OwnershipStrengthQosPolicy](#).

6.44.1 Detailed Description

Specifies the value of the strength used to arbitrate among multiple [DDSDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).

6.44.2 Variable Documentation

6.44.2.1 const char* const DDS_OWNERSHIPSTRENGTH_QOS_POLICY_NAME

Stringified human-readable name for [DDS_OwnershipStrengthQosPolicy](#).

6.45 LIVELINESS

Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.

Classes

- struct [DDS_LivelinessQosPolicy](#)
<<cert>> Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.

Enumerations

- enum [DDS_LivelinessQosPolicyKind](#) { [DDS_AUTOMATIC_LIVELINESS_QOS](#), [DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS](#), [DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS](#) }
Kinds of liveliness.

Variables

- const char *const [DDS_LIVELINESS_QOS_POLICY_NAME](#)
Stringified human-readable name for [DDS_LivelinessQosPolicy](#).

6.45.1 Detailed Description

Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.

6.45.2 Enumeration Type Documentation

6.45.2.1 enum [DDS_LivelinessQosPolicyKind](#)

Kinds of liveliness.

QoS:

[DDS_LivelinessQosPolicy](#)

Enumerator:

[DDS_AUTOMATIC_LIVELINESS_QOS](#) [default] The infrastructure will automatically signal liveliness for the [DDSDDataWriter](#) (s) at least as often as required by the [DDSDDataWriter](#) (S) `lease_duration`. A [DDSDDataWriter](#) with this setting does not need to take any specific action in order to be considered 'alive.' The [DDSDDataWriter](#) is only 'not alive' when the participant to which it belongs terminates (gracefully or not), or when there is a network problem that prevents the current participant from contacting that remote participant.

Only [DDS_DURATION_INFINITE](#) is supported for `lease_duration`. Finite `lease_duration` is not supported with this liveliness setting.

[DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS](#) RTI Connex Micro will assume that as long as at least one [DDSDDataWriter](#) belonging to the [DDSDomainParticipant](#) (or the [DDSDomainParticipant](#) itself) has asserted its liveliness, then the other DataWriters belonging to that same [DDSDomainParticipant](#) are also alive. Only [DDS_DURATION_INFINITE](#) is supported for `lease_duration`. Finite `lease_duration` is not supported with this liveliness setting.

The user application takes responsibility to signal liveliness to RTI Connex Micro by calling [DDSDDataWriter::assert_liveliness](#), or [FooDataWriter::write](#) on any [DDSDDataWriter](#) belonging to the [DDSDomainParticipant](#).

DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS RTI Connex Micro will only assume liveliness of the [DDS-DataWriter](#) if the application has asserted liveliness of that [DDSDataWriter](#) itself. The user application takes responsibility to signal liveliness to RTI Connex Micro using the [DDSDataWriter::assert_liveliness](#) method, or by writing some data.

6.45.3 Variable Documentation

6.45.3.1 `const char* const DDS_LIVELINESS_QOS_POLICY_NAME`

Stringified human-readable name for [DDS_LivelinessQosPolicy](#).

6.46 RELIABILITY

Indicates the level of reliability offered/requested by RTI Connext Micro.

Classes

- struct [DDS_ReliabilityQosPolicy](#)
<<cert>> Indicates the level of reliability offered/requested by RTI Connext Micro.

Enumerations

- enum [DDS_ReliabilityQosPolicyKind](#) { [DDS_BEST_EFFORT_RELIABILITY_QOS](#), [DDS_RELIABLE_RELIABILITY_QOS](#) }
Kinds of reliability.

Variables

- const char *const [DDS_RELIABILITY_QOS_POLICY_NAME](#)
Stringified human-readable name for [DDS_ReliabilityQosPolicy](#).

6.46.1 Detailed Description

Indicates the level of reliability offered/requested by RTI Connext Micro.

6.46.2 Enumeration Type Documentation

6.46.2.1 enum [DDS_ReliabilityQosPolicyKind](#)

Kinds of reliability.

QoS:

[DDS_ReliabilityQosPolicy](#)

Enumerator:

[DDS_BEST_EFFORT_RELIABILITY_QOS](#) Indicates that it is acceptable to not retry propagation of any samples. Presumably new values for the samples are generated often enough that it is not necessary to re-send or acknowledge any samples.

[default]

[DDS_RELIABLE_RELIABILITY_QOS](#) Specifies RTI Connext Micro will attempt to deliver all samples in its history. Missed samples may be retried. In steady-state (no modifications communicated via the [DDS-DataWriter](#)) the middleware guarantees that all samples in the [DDSDDataWriter](#) history will eventually be delivered to all the [DDSDDataReader](#) objects (subject to timeouts that indicate loss of communication with a particular [DDSSubscriber](#)).

Outside steady state the [HISTORY](#) and [RESOURCE_LIMITS](#) policies will determine how samples become part of the history and whether samples can be discarded from it.

NOTE: The RTPS spec defines reliability as 0x03 to comply with RTPS spec. However, RTI Connext Core uses 0x3, as well as others.

6.46.3 Variable Documentation

6.46.3.1 const char* const [DDS_RELIABILITY_QOS_POLICY_NAME](#)

Stringified human-readable name for [DDS_ReliabilityQosPolicy](#).

6.47 HISTORY

Specifies the behavior of RTI Connex Micro in the case where the value of an instance changes (one or more times) before it can be successfully communicated to one or more existing subscribers.

Classes

- struct [DDS_HistoryQosPolicy](#)
<<cert>> Specifies the behavior of RTI Connex Micro in the case where the value of a sample changes (one or more times) before it can be successfully communicated to one or more existing subscribers.

Enumerations

- enum [DDS_HistoryQosPolicyKind](#) { [DDS_KEEP_LAST_HISTORY_QOS](#) }
Kinds of history.

Variables

- const char *const [DDS_HISTORY_QOS_POLICY_NAME](#)
Stringified human-readable name for [DDS_HistoryQosPolicy](#).

6.47.1 Detailed Description

Specifies the behavior of RTI Connex Micro in the case where the value of an instance changes (one or more times) before it can be successfully communicated to one or more existing subscribers.

6.47.2 Enumeration Type Documentation

6.47.2.1 enum [DDS_HistoryQosPolicyKind](#)

Kinds of history.

QoS:

[DDS_HistoryQosPolicy](#)

Enumerator:

[DDS_KEEP_LAST_HISTORY_QOS](#) [default] Keep the last `depth` samples. On the publishing side, RTI Connex Micro will only attempt to keep the most recent `depth` samples of each instance of data (identified by its key) managed by the [DDSDDataWriter](#).

On the subscribing side, the [DDSDDataReader](#) will only attempt to keep the most recent `depth` samples received for each instance (identified by its key) until the application takes them via the [DDSDDataReader](#)'s **take()** operation.

6.47.3 Variable Documentation

6.47.3.1 const char* const [DDS_HISTORY_QOS_POLICY_NAME](#)

Stringified human-readable name for [DDS_HistoryQosPolicy](#).

6.48 DURABILITY

<<*eXtension*>> Specifies the Durability properties used by a [DDSDDataWriter](#) and/or [DDSDDataReader](#).

Classes

- struct [DDS_DurabilityQosPolicy](#)
Durability properties that affect the life-cycle of published data.

Enumerations

- enum [DDS_DurabilityQosPolicyKind](#) { [DDS_VOLATILE_DURABILITY_QOS](#), [DDS_TRANSIENT_LOCAL_DURABILITY_QOS](#) }
Enumerated values used to describe the durability of published data.

Variables

- const char *const [DDS_DURABILITY_QOS_POLICY_NAME](#)
Stringified human-readable name for [DDS_DurabilityQosPolicy](#).

6.48.1 Detailed Description

<<*eXtension*>> Specifies the Durability properties used by a [DDSDDataWriter](#) and/or [DDSDDataReader](#).

6.48.2 Enumeration Type Documentation

6.48.2.1 enum [DDS_DurabilityQosPolicyKind](#)

Enumerated values used to describe the durability of published data.

Enumerator:

[DDS_VOLATILE_DURABILITY_QOS](#) Volatile durability.
[DDS_TRANSIENT_LOCAL_DURABILITY_QOS](#) Transient local durability.

6.48.3 Variable Documentation

6.48.3.1 const char* const [DDS_DURABILITY_QOS_POLICY_NAME](#)

Stringified human-readable name for [DDS_DurabilityQosPolicy](#).

6.49 RESOURCE_LIMITS

Specifies the resources that RTI Connex Micro can consume in order to meet the requested QoS.

Classes

- struct [DDS_ResourceLimitsQosPolicy](#)
<<cert>> Specifies the resources that RTI Connex Micro can consume in order to meet the requested QoS.

Variables

- const char *const [DDS_RESOURCELIMITS_QOS_POLICY_NAME](#)
Stringified human-readable name for [DDS_ResourceLimitsQosPolicy](#).
- const [DDS_Long](#) [DDS_LENGTH_UNLIMITED](#)
A special value indicating an unlimited quantity.
- const [DDS_Long](#) [DDS_LENGTH_AUTO](#)
A special value indicating the resource limit is determined based on other settings.

6.49.1 Detailed Description

Specifies the resources that RTI Connex Micro can consume in order to meet the requested QoS.

6.49.2 Variable Documentation

6.49.2.1 const char* const DDS_RESOURCELIMITS_QOS_POLICY_NAME

Stringified human-readable name for [DDS_ResourceLimitsQosPolicy](#).

6.49.2.2 const DDS_Long DDS_LENGTH_UNLIMITED

A special value indicating an unlimited quantity.

6.49.2.3 const DDS_Long DDS_LENGTH_AUTO

A special value indicating the resource limit is determined based on other settings.

Some resource limits can be derived from other limits using rules. However, the rules are usually conservative and typically the resulting limit is larger than needed. Thus, while this is a useful value when starting development, it should be adjusted properly when the exact limits are known.

6.50 PRESENTATION

Specifies how the samples representing changes to data instances are presented to a subscribing application.

Classes

- struct [DDS_PresentationQosPolicy](#)

Specifies how the samples representing changes to data instances are presented to a subscribing application.

Enumerations

- enum [DDS_PresentationQosPolicyAccessScopeKind](#) { [DDS_INSTANCE_PRESENTATION_QOS](#), [DDS_TOPIC_PRESENTATION_QOS](#), [DDS_GROUP_PRESENTATION_QOS](#), [DDS_HIGHEST_OFFERED_PRESENTATION_QOS](#) }

Kinds of presentation "access scope".

6.50.1 Detailed Description

Specifies how the samples representing changes to data instances are presented to a subscribing application.

6.50.2 Enumeration Type Documentation

6.50.2.1 enum [DDS_PresentationQosPolicyAccessScopeKind](#)

Kinds of presentation "access scope".

Access scope determines the largest scope spanning the entities for which the order and coherency of changes can be preserved.

QoS:

[DDS_PresentationQosPolicy](#)

Enumerator:

[DDS_INSTANCE_PRESENTATION_QOS](#) [default] for [DDSDataReader](#). Scope spans only a single instance. Indicates that changes to one instance need not be coherent nor ordered with respect to changes to any other instance. In other words, order and coherent changes apply to each instance separately.

[DDS_TOPIC_PRESENTATION_QOS](#) [default] for [DDSDataWriter](#). Scope spans all instances within the same [DDSDataWriter](#) (or [DDSDataReader](#)), but not across instances in different [DDSDataWriter](#) (or [DDSDataReader](#)) entities.

[DDS_GROUP_PRESENTATION_QOS](#) Scope spans all instances belonging to [DDSDataWriter](#) (or [DDSDataReader](#)) entities within the same [DDSPublisher](#) (or [DDSSubscriber](#)).

[DDS_HIGHEST_OFFERED_PRESENTATION_QOS](#) This value only applies to a [DDSSubscriber](#). The [DDSSubscriber](#) will use the access scope specified by each remote [DDSPublisher](#).

6.51 DESTINATION_ORDER

<<*eXtension*>> Specifies the DestinationOrder policy.

Classes

- struct [DDS_DestinationOrderQosPolicy](#)
Destination order policies that affect the ordering of subscribed data.

Enumerations

- enum [DDS_DestinationOrderQosPolicyKind](#) { [DDS_BY_RECEPTION_TIMESTAMP_DESTINATIONORDER_QOS](#), [DDS_BY_SOURCE_TIMESTAMP_DESTINATIONORDER_QOS](#) }
Enumerated values used to describe the destination order of published data.

6.51.1 Detailed Description

<<*eXtension*>> Specifies the DestinationOrder policy.

6.51.2 Enumeration Type Documentation

6.51.2.1 enum [DDS_DestinationOrderQosPolicyKind](#)

Enumerated values used to describe the destination order of published data.

Enumerator:

[DDS_BY_RECEPTION_TIMESTAMP_DESTINATIONORDER_QOS](#) destination ordering by reception timestamp.

Entity:

[DDSDataReader](#), [DDSDataWriter](#)

Properties:

[Rxo](#) = YES;
[Changeable](#) = NO

[default] Indicates that data is ordered based on the reception time at each [DDSDataReader](#).

Since each subscriber may receive the data at different times there is no guarantee that the changes will be seen in the same order. Consequently, it is possible for each subscriber to end up with a different final value for the data.

[DDS_BY_SOURCE_TIMESTAMP_DESTINATIONORDER_QOS](#) destination ordering by source timestamp.

Entity:

[DDSDataWriter](#)

Properties:

[Rxo](#) = YES;
[Changeable](#) = NO

Indicates that data is ordered based on the source time provided by the source application.

A consistent final value is guaranteed for the data in all subscribers.

6.52 DataReader Resource Limits

<<eXtension>> Specifies the DataReader-specific resources that RTI Connex Micro can use.

Classes

- struct [DDS_DataReaderResourceLimitsQosPolicy](#)
 <<cert>> Resource limits that apply only to [DDSDataReader](#) instances.

Enumerations

- enum [DDS_DataReaderResourceLimitsInstanceReplacementKind](#) { [DDS_NO_INSTANCE_REPLACEMENT_QOS](#), [DDS_REPLACE_OLDEST_INSTANCE_REPLACEMENT_QOS](#) }
 Kinds of instance replacement.

Variables

- const char *const [DDS_DATAREADERRESOURCELIMITS_QOS_POLICY_NAME](#)
 Stringified human-readable name for [DDS_DataReaderResourceLimitsQosPolicy](#).

6.52.1 Detailed Description

<<eXtension>> Specifies the DataReader-specific resources that RTI Connex Micro can use.

6.52.2 Enumeration Type Documentation

6.52.2.1 enum [DDS_DataReaderResourceLimitsInstanceReplacementKind](#)

Kinds of instance replacement.

QoS:

[DDS_DataReaderResourceLimitsQosPolicy](#)

Enumerator:

[DDS_NO_INSTANCE_REPLACEMENT_QOS](#) No instances will be replaced when a new instance is detected and resources are exhausted.

[DDS_REPLACE_OLDEST_INSTANCE_REPLACEMENT_QOS](#) Replace the oldest instance when out of instance resources. When a new instance is detected and resources are exhausted the oldest instance will be removed even if samples with a not read state exists for the instance. Thus, this is a forceful removal of samples and the instance. The freed instance resources will be re-used for the newly detected instance. If there are outstanding loans for the replaced instance the samples can not be freed until returned. However, when the loan is returned the samples are returned back to the free sample pool. Note that the instance will not be rejected.

6.52.3 Variable Documentation

6.52.3.1 const char* const [DDS_DATAREADERRESOURCELIMITS_QOS_POLICY_NAME](#)

Stringified human-readable name for [DDS_DataReaderResourceLimitsQosPolicy](#).

6.53 DataWriter Resource Limits

<<*eXtension*>> Specifies the DataWriter-specific resources that RTI Connex Micro can use.

Classes

- struct [DDS_DataWriterResourceLimitsQosPolicy](#)
<<*cert*>> Resource limits that apply only to [DDSDataWriter](#) instances.

Variables

- const char *const [DDS_DATAWRITERRESOURCELIMITS_QOS_POLICY_NAME](#)
Stringified human-readable name for [DDS_DataWriterResourceLimitsQosPolicy](#).

6.53.1 Detailed Description

<<*eXtension*>> Specifies the DataWriter-specific resources that RTI Connex Micro can use.

6.53.2 Variable Documentation

6.53.2.1 const char* const DDS_DATAWRITERRESOURCELIMITS_QOS_POLICY_NAME

Stringified human-readable name for [DDS_DataWriterResourceLimitsQosPolicy](#).

6.54 ENTITY_FACTORY

A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.

Classes

- struct [DDS_EntityFactoryQosPolicy](#)
<<cert>> A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.

Variables

- const char *const [DDS_ENTITYFACTORY_QOS_POLICY_NAME](#)
Stringified human-readable name for [DDS_EntityFactoryQosPolicy](#).

6.54.1 Detailed Description

A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.

6.54.2 Variable Documentation

6.54.2.1 const char* const DDS_ENTITYFACTORY_QOS_POLICY_NAME

Stringified human-readable name for [DDS_EntityFactoryQosPolicy](#).

6.55 Extended Qos Support

`<<eXtension>>` Types and defines used in extended QoS policies.

Modules

- [DataReader Resource Limits](#)
`<<eXtension>>` Specifies the DataReader-specific resources that RTI Connex Micro can use.
- [DataWriter Resource Limits](#)
`<<eXtension>>` Specifies the DataWriter-specific resources that RTI Connex Micro can use.

Classes

- struct [DDS_RtpsReliableReaderProtocol_t](#)
Qos related to reliable reader protocol defined in RTPS.
- struct [DDS_RtpsReliableWriterProtocol_t](#)
`<<cert>>` QoS related to the reliable writer protocol defined in RTPS.

6.55.1 Detailed Description

`<<eXtension>>` Types and defines used in extended QoS policies.

6.56 SYSTEM_RESOURCE_LIMITS

<<*eXtension*>> Specifies the system-specific resources that RTI Connex Micro can use.

Classes

- struct [DDS_SystemResourceLimitsQosPolicy](#)
 <<*cert*>> Resource limits that apply only to [DDSDomainParticipantFactory](#)

Variables

- const char *const [DDS_SYSTEMRESOURCELIMITS_QOS_POLICY_NAME](#)
 Stringified human-readable name for [DDS_SystemResourceLimitsQosPolicy](#).

6.56.1 Detailed Description

<<*eXtension*>> Specifies the system-specific resources that RTI Connex Micro can use.

6.56.2 Variable Documentation

6.56.2.1 const char* const DDS_SYSTEMRESOURCELIMITS_QOS_POLICY_NAME

Stringified human-readable name for [DDS_SystemResourceLimitsQosPolicy](#).

6.57 WIRE_PROTOCOL

<<eXtension>> Specifies the wire protocol related attributes for the [DDSDomainParticipant](#).

Classes

- struct [DDS_RtpsWellKnownPorts_t](#)
RTPS well-known port mapping configuration.
- struct [DDS_WireProtocolQosPolicy](#)
<<cert>> Specifies the wire-protocol-related attributes for the [DDSDomainParticipant](#).

Macros

- #define [DDS_CHECKSUM_NONE](#) (0)
Bitmap indicating no checksum selection.
- #define [DDS_CHECKSUM_AUTO](#) (0xffffU)
Bitmap indicating automatic checksum selection.
- #define [DDS_CHECKSUM_BUILTIN32](#) (0x1U)
Bitmap for selecting the builtin 32bit checksum.
- #define [DDS_CHECKSUM_BUILTIN64](#) (0x2U)
Bitmap for selecting the builtin 64bit checksum.
- #define [DDS_CHECKSUM_BUILTIN128](#) (0x4U)
Bitmap for selecting the builtin 128bit checksum.

Typedefs

- typedef [DDS_UnsignedShort DDS_ChecksumKind_t](#)
Datatype for checksum kind.
- typedef [DDS_UnsignedShort DDS_ChecksumKindMask_t](#)
Datatype for an OR'ing for [DDS_ChecksumKind_t](#).

Variables

- struct [DDS_RtpsWellKnownPorts_t DDS_RTI_BACKWARDS_COMPATIBLE_RTPS_WELL_KNOWN_PORTS](#)
Assign to use well-known port mappings which are compatible with previous versions of the RTI Connex Micro middleware.
- struct [DDS_RtpsWellKnownPorts_t DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS](#)
Assign to use well-known port mappings which are compliant with OMG's DDS Interoperability Wire Protocol.
- const char *const [DDS_WIREPROTOCOL_QOS_POLICY_NAME](#)
Stringified human-readable name for [DDS_WireProtocolQosPolicy](#).

6.57.1 Detailed Description

<<eXtension>> Specifies the wire protocol related attributes for the [DDSDomainParticipant](#).

6.57.2 Macro Definition Documentation

6.57.2.1 #define DDS_CHECKSUM_NONE (0)

Bitmap indicating no checksum selection.

QoS:

[DDS_WireProtocolQosPolicy](#)

6.57.2.2 #define DDS_CHECKSUM_AUTO (0xffffU)

Bitmap indicating automatic checksum selection.

QoS:

[DDS_WireProtocolQosPolicy](#)

6.57.2.3 #define DDS_CHECKSUM_BUILTIN32 (0x1U)

Bitmap for selecting the builtin 32bit checksum.

QoS:

[DDS_WireProtocolQosPolicy](#)

6.57.2.4 #define DDS_CHECKSUM_BUILTIN64 (0x2U)

Bitmap for selecting the builtin 64bit checksum.

QoS:

[DDS_WireProtocolQosPolicy](#)

6.57.2.5 #define DDS_CHECKSUM_BUILTIN128 (0x4U)

Bitmap for selecting the builtin 128bit checksum.

QoS:

[DDS_WireProtocolQosPolicy](#)

6.57.3 Typedef Documentation

6.57.3.1 typedef DDS_UnsignedShort DDS_ChecksumKind_t

Datatype for checksum kind.

QoS:

[DDS_WireProtocolQosPolicy](#)

6.57.3.2 typedef DDS_UnsignedShort DDS_ChecksumKindMask_t

Datatype for an OR'ing for [DDS_ChecksumKind_t](#).

QoS:

[DDS_WireProtocolQosPolicy](#)

6.57.4 Variable Documentation

6.57.4.1 struct DDS_RtpsWellKnownPorts_t DDS_RTI_BACKWARDS_COMPATIBLE_RTPS_WELL_KNOWN_PORTS

Assign to use well-known port mappings which are compatible with previous versions of the RTI Connex Micro middleware.

Assign [DDS_WireProtocolQosPolicy::rtps_well_known_ports](#) to this value to remain compatible with previous versions of the RTI Connex Micro middleware that used fixed port mappings.

The following are the `rtps_well_known_ports` values for [DDS_RTI_BACKWARDS_COMPATIBLE_RTPS_WELL_KNOWN_PORTS](#):

```
port_base = 7400
domain_id_gain = 10
participant_id_gain = 1000
builtin_multicast_port_offset = 2
builtin_unicast_port_offset = 0
user_multicast_port_offset = 1
user_unicast_port_offset = 3
```

These settings are *not* compliant with OMG's DDS Interoperability Wire Protocol. To comply with the specification, please use [DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS](#).

See also

[DDS_WireProtocolQosPolicy::rtps_well_known_ports](#)
[DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS](#)

6.57.4.2 struct DDS_RtpsWellKnownPorts_t DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS

Assign to use well-known port mappings which are compliant with OMG's DDS Interoperability Wire Protocol.

Assign [DDS_WireProtocolQosPolicy::rtps_well_known_ports](#) to this value to use well-known port mappings which are compliant with OMG's DDS Interoperability Wire Protocol.

The following are the `rtps_well_known_ports` values for [DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS](#):

```
port_base = 7400
domain_id_gain = 250
participant_id_gain = 2
builtin_multicast_port_offset = 0
builtin_unicast_port_offset = 10
user_multicast_port_offset = 1
user_unicast_port_offset = 11
```

Assuming a maximum port number of 65535 (UDPv4), the above settings enable the use of about 230 domains with up to 120 Participants per node per domain.

These settings are *not* backwards compatible with previous versions of the RTI Connex Micro middleware that used fixed port mappings. For backwards compability, please use [DDS_RTI_BACKWARDS_COMPATIBLE_RTPS_WELL_KNOWN_PORTS](#).

See also

[DDS_WireProtocolQosPolicy::rtps_well_known_ports](#)
[DDS_RTI_BACKWARDS_COMPATIBLE_RTPS_WELL_KNOWN_PORTS](#)

6.57.4.3 `const char* const DDS_WIREPROTOCOL_QOS_POLICY_NAME`

Stringified human-readable name for [DDS_WireProtocolQosPolicy](#).

6.58 DATA_READER_PROTOCOL

<<*eXtension*>> Specifies the DataReader-specific protocol QoS.

Classes

- struct [DDS_DataReaderProtocolQosPolicy](#)
<<eXtension>> Protocol that applies only to [DDSDataReader](#) instances.

Variables

- const char *const [DDS_DATAREADERPROTOCOL_QOS_POLICY_NAME](#)
Stringified human-readable name for [DDS_DataReaderProtocolQosPolicy](#).

6.58.1 Detailed Description

<<*eXtension*>> Specifies the DataReader-specific protocol QoS.

6.58.2 Variable Documentation

6.58.2.1 const char* const DDS_DATAREADERPROTOCOL_QOS_POLICY_NAME

Stringified human-readable name for [DDS_DataReaderProtocolQosPolicy](#).

6.59 DATA_WRITER_PROTOCOL

<<*eXtension*>> Specifies the DataWriter-specific protocol QoS.

Classes

- struct [DDS_DataWriterProtocolQosPolicy](#)
 <<*eXtension*>> <<*cert*>> Protocol that applies only to [DDSDataWriter](#) instances.

Variables

- const char *const [DDS_DATAWRITERPROTOCOL_QOS_POLICY_NAME](#)
 Stringified human-readable name for [DDS_DataWriterProtocolQosPolicy](#).

6.59.1 Detailed Description

<<*eXtension*>> Specifies the DataWriter-specific protocol QoS.

6.59.2 Variable Documentation

6.59.2.1 const char* const DDS_DATAWRITERPROTOCOL_QOS_POLICY_NAME

Stringified human-readable name for [DDS_DataWriterProtocolQosPolicy](#).

6.60 TRANSPORT

<<*eXtension*>> <<*cert*>> Specify transport settings

Classes

- struct [DDS_TransportQosPolicy](#)
<<*eXtension*>> <<*cert*>> *Settings for available transports*

Variables

- const char *const [DDS_TRANSPORT_QOS_POLICY_NAME](#)
Stringified human-readable name for [DDS_TransportQosPolicy](#).

6.60.1 Detailed Description

<<*eXtension*>> <<*cert*>> Specify transport settings

6.60.2 Variable Documentation

6.60.2.1 const char* const DDS_TRANSPORT_QOS_POLICY_NAME

Stringified human-readable name for [DDS_TransportQosPolicy](#).

6.61 DOMAIN_PARTICIPANT_RESOURCE_LIMITS

<<*eXtension*>> Specifies the DomainParticipant-specific resources that RTI Connext Micro can use.

Classes

- struct [DDS_DomainParticipantResourceLimitsQosPolicy](#)
 <<*cert*>> Resource limits that apply only to [DDSDomainParticipant](#) instances.

Variables

- const char *const [DDS_DOMAINPARTICIPANTRESOURCELIMITS_QOS_POLICY_NAME](#)
 Stringified human-readable name for [DDS_DomainParticipantResourceLimitsQosPolicy](#).

6.61.1 Detailed Description

<<*eXtension*>> Specifies the DomainParticipant-specific resources that RTI Connext Micro can use.

6.61.2 Variable Documentation

6.61.2.1 const char* const DDS_DOMAINPARTICIPANTRESOURCELIMITS_QOS_POLICY_NAME

Stringified human-readable name for [DDS_DomainParticipantResourceLimitsQosPolicy](#).

6.62 Conditions and WaitSets

[DDSCondition](#) and [DDSWaitSet](#) and related items.

Classes

- class [DDSCondition](#)
<<interface>> <<cert>> Root class for all the conditions that may be attached to a [DDSWaitSet](#).
- class [DDSGuardCondition](#)
<<interface>> <<cert>> A specific [DDSCondition](#) whose `trigger_value` is completely under the control of the application.
- class [DDSStatusCondition](#)
<<interface>> <<cert>> A specific [DDSCondition](#) that is associated with each [DDSEntity](#).
- class [DDSWaitSet](#)
<<interface>> <<cert>> Allows an application to wait until one or more of the attached [DDSCondition](#) objects has a `trigger_value` of [DDS_BOOLEAN_TRUE](#) or else until the timeout expires.

6.62.1 Detailed Description

[DDSCondition](#) and [DDSWaitSet](#) and related items.

6.63 Entity Support

[DDSEntity](#), [DDSListener](#) and related items.

Classes

- class [DDSListener](#)
<<interface>> Abstract base class for all Listener interfaces.
- class [DDSDomainEntity](#)
<<interface>> Abstract base class for all DDS entities except for the [DDSDomainParticipant](#).
- class [DDSEntity](#)
<<interface>> Abstract base class for all the DDS objects that support QoS policies, and a listener.

6.63.1 Detailed Description

[DDSEntity](#), [DDSListener](#) and related items. [DDSEntity](#) subtypes are created and destroyed by factory objects. With the exception of [DDSDomainParticipant](#), whose factory is [DDSDomainParticipantFactory](#), all [DDSEntity](#) factory objects are themselves [DDSEntity](#) subtypes as well.

Important: all [DDSEntity](#) delete operations are inherently thread-unsafe. The user must take extreme care that a given [DDSEntity](#) is not destroyed in one thread while being used concurrently (including being deleted concurrently) in another thread. An operation's effect in the presence of the concurrent deletion of the operation's target [DDSEntity](#) is undefined.

6.64 ENTITY_NAME

<<*eXtension*>> Name of the entity.

Classes

- struct [DDS_EntityNameQosPolicy](#)
 <<*cert*>> Policy used to name an entity.

Macros

- #define [DDS_ENTITYNAME_QOS_NAME_MAX](#) 255
 Constant determining the maximum length of the name string contained in a [DDS_EntityNameQosPolicy](#) instance.

Variables

- const char *const [DDS_ENTITYNAME_QOS_POLICY_NAME](#)
 Stringified human-readable name for [DDS_EntityNameQosPolicy](#).
- const char *const [DDS_AUTO_NAME_ENTITY](#)
 Constant used to request that an entity be assigned an automatically generated name.

6.64.1 Detailed Description

<<*eXtension*>> Name of the entity.

6.64.2 Macro Definition Documentation

6.64.2.1 #define DDS_ENTITYNAME_QOS_NAME_MAX 255

Constant determining the maximum length of the name string contained in a [DDS_EntityNameQosPolicy](#) instance.

6.64.3 Variable Documentation

6.64.3.1 const char* const DDS_ENTITYNAME_QOS_POLICY_NAME

Stringified human-readable name for [DDS_EntityNameQosPolicy](#).

6.64.3.2 const char* const DDS_AUTO_NAME_ENTITY

Constant used to request that an entity be assigned an automatically generated name.

6.65 DDS_C

DDS C. ModuleID = 7.

Macros

- #define `DDSC_LOG_INVALID_DURATION_EC` (DDSC_LOG_BASE + 1)
The specified duration is not valid.
- #define `DDSC_LOG_INVALID_PARTICIPANT_NAME_EC` (DDSC_LOG_BASE + 2)
An invalid participant name was specified with an unknown GUID.
- #define `DDSC_LOG_INVALID_PARTICIPANT_GUID_PREFIX_EC` (DDSC_LOG_BASE + 3)
An invalid participant GUID prefix was specified, typically the GUID prefix does not match an already detected participant.
- #define `DDSC_LOG_SYS_GETTIME_EC` (DDSC_LOG_BASE + 4)
A call to OSAPI_System_get_time failed.
- #define `DDSC_LOG_GET_NEXT_OBJECT_ID_EC` (DDSC_LOG_BASE + 5)
Failed to get the next automatically generated object ID for an entity's GUID. This typically means the object id pool has been exhausted.
- #define `DDSC_LOG_SET_ENTITY_NAME_EC` (DDSC_LOG_BASE + 6)
Failed to set name string for a DDS entity in the DomainParticipantQos entity_name policy.
- #define `DDSC_LOG_SYS_GET_HOSTNAME_EC` (DDSC_LOG_BASE + 7)
A call to OSAPI_System_get_hostname failed.
- #define `DDSC_LOG_IO_SNPRINTF_FAILED_EC` (DDSC_LOG_BASE + 8)
A call to OSAPI_Stdio_snprintf failed. Typically this means the destination buffer was too small.
- #define `DDSC_LOG_TOPIC_FIND_EC` (DDSC_LOG_BASE + 9)
Failed to find a topic created by a DomainParticipant.
- #define `DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_NAME_EC` (DDSC_LOG_BASE + 10)
Endpoint discovery failed because the name of the remote participant parent for an endpoint was not found.
- #define `DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_KEY_EC` (DDSC_LOG_BASE + 11)
Endpoint discovery failed because the key of the remote participant parent for an endpoint was not found. Note that the key is logged as 4 integers in host endianness format.
- #define `DDSC_LOG_REMOTE_PARTICIPANT_KEY_NOT_EQUAL_EC` (DDSC_LOG_BASE + 12)
Failed endpoint discovery when key does not match the remote participant.
- #define `DDSC_LOG_INVALID_ENDPOINT_GUID_EC` (DDSC_LOG_BASE + 13)
Failed endpoint discovery due to an invalid or unknown endpoint GUID.
- #define `DDSC_LOG_ENDPOINT_NOT_CHILD_OF_PARTICIPANT_EC` (DDSC_LOG_BASE + 14)
Failed endpoint discovery when an endpoint is determined to belong to a different remote participant.
- #define `DDSC_LOG_PARTICIPANT_DOES_NOT_EXIST_EC` (DDSC_LOG_BASE + 15)
Failed participant discovery because a remote participant that should have been already asserted locally was not found.
- #define `DDSC_LOG_REFRESH_REM_PARTICIPANT_EC` (DDSC_LOG_BASE + 16)
Did not find a remote participant when asserting participant liveliness for it. Note that the key is logged as 4 integers in host endianness format.
- #define `DDSC_LOG_REFRESH_REM_PARTICIPANT_TIMEOUT_EC` (DDSC_LOG_BASE + 17)
Failed to assert participant liveliness to a remote participant.
- #define `DDSC_LOG_PARTICIPANT_LOOKUP_EC` (DDSC_LOG_BASE + 18)
Failed to find a remote participant as previously discovered.
- #define `DDSC_LOG_REMOVE_PUBLICATION_EC` (DDSC_LOG_BASE + 19)
Failed to remove resources for a remote publication.
- #define `DDSC_LOG_REMOVE_SUBSCRIPTION_EC` (DDSC_LOG_BASE + 20)
Failed to remove resources for a remote subscription.
- #define `DDSC_LOG_FIND_PUBLICATION_PARENT_EC` (DDSC_LOG_BASE + 21)

- Cannot determine the participant of a remote publication.*

 - #define [DDSC_LOG_FIND_SUBSCRIPTION_PARENT_EC](#) (DDSC_LOG_BASE + 22)
- Cannot determine the participant of a remote subscription.*

 - #define [DDSC_LOG_MAX_PARTICIPANT_ID_REACHED_EC](#) (DDSC_LOG_BASE + 23)

Failed to create DomainParticipant due to running out of participant IDs. This is typically caused by all UDP ports being used.
- #define [DDSC_LOG_RESERVE_LOCATORS_EC](#) (DDSC_LOG_BASE + 24)

Failed to reserve endpoint locators.
- #define [DDSC_LOG_TIMER_CREATE_TIMEOUT_EC](#) (DDSC_LOG_BASE + 25)

Failed to create a timeout.
- #define [DDSC_LOG_ILLEGAL_OBJECTID_EC](#) (DDSC_LOG_BASE + 26)

Illegal object id specified.
- #define [DDSC_LOG_TOPIC_NARROW_EC](#) (DDSC_LOG_BASE + 27)

Failed to narrow a TopicDescription to the named Topic.
- #define [DDSC_LOG_FAILED_UPDATE_STATUS_CONDITION_EC](#) (DDSC_LOG_BASE + 28)

Failed to update state of StatusCondition.
- #define [DDSC_LOG_WS_REMOVE_COND_REFERENCE_EC](#) (DDSC_LOG_BASE + 29)

Failed to remove a condition reference from a waitset.
- #define [DDSC_LOG_WS_ADD_COND_REFERENCE_EC](#) (DDSC_LOG_BASE + 30)

Failed to add a condition reference to a waitset.
- #define [DDSC_LOG_RELEASE_META_MC_EC](#) (DDSC_LOG_BASE + 31)

Failed to release resources for multicast discovery locators.
- #define [DDSC_LOG_RELEASE_META_UC_EC](#) (DDSC_LOG_BASE + 32)

Failed to release resources for unicast discovery locators.
- #define [DDSC_LOG_RELEASE_USER_MC_EC](#) (DDSC_LOG_BASE + 33)

Failed to release resources for multicast user locators.
- #define [DDSC_LOG_RELEASE_USER_UC_EC](#) (DDSC_LOG_BASE + 34)

Failed to release resources for unicast user locators.
- #define [DDSC_LOG_INVALID_DOMAINID_EC](#) (DDSC_LOG_BASE + 35)

The domain ID specified exceeds what is allowed based on the parameters specified in dp_qos.protocol.rtps_well_known_ports.
- #define [DDSC_LOG_DATABASE_CREATE_EC](#) (DDSC_LOG_BASE + 100)

Failed to create database.
- #define [DDSC_LOG_DATABASE_DELETE_EC](#) (DDSC_LOG_BASE + 101)

Failed to delete database.
- #define [DDSC_LOG_TABLE_CREATE_EC](#) (DDSC_LOG_BASE + 102)

Failed to create database table of the specified name.
- #define [DDSC_LOG_TABLE_INUSE_EC](#) (DDSC_LOG_BASE + 103)

Failed to delete a database table because it is not empty.
- #define [DDSC_LOG_TABLE_DELETE_EC](#) (DDSC_LOG_BASE + 104)

Failed to delete a database table.
- #define [DDSC_LOG_TABLE_SELECT_EC](#) (DDSC_LOG_BASE + 105)

A selection operation failed on the specified database table.
- #define [DDSC_LOG_CREATE_INDEX_EC](#) (DDSC_LOG_BASE + 106)

Failed to create an index on a database table.
- #define [DDSC_LOG_DELETE_INDEX_EC](#) (DDSC_LOG_BASE + 107)

Failed to delete an index on a database table.
- #define [DDSC_LOG_DB_CURSOR_INVALIDATED_EC](#) (DDSC_LOG_BASE + 108)

A database table cursor was invalidated while in use.
- #define [DDSC_LOG_SUBSCRIPTION_RECORD](#) 1

Database record for a Subscription.

- `#define DDSC_LOG_PUBLICATION_RECORD 2`
Database record for a Publication.
- `#define DDSC_LOG_PARTICIPANT_RECORD 3`
Database record for a Remote Participant.
- `#define DDSC_LOG_TOPIC_RECORD 4`
Database record for a Topic.
- `#define DDSC_LOG_DATAWRITER_RECORD 5`
Database record for a DataWriter.
- `#define DDSC_LOG_DATAREADER_RECORD 6`
Database record for a DataReader.
- `#define DDSC_LOG_PUBLISHER_RECORD 7`
Database record for a Publisher.
- `#define DDSC_LOG_SUBSCRIBER_RECORD 8`
Database record for a Subscriber.
- `#define DDSC_LOG_ROUTE_RECORD 9`
Database record for a route record.
- `#define DDSC_LOG_BIND_RECORD 10`
Database record for a bind record.
- `#define DDSC_LOG_TYPE_RECORD 11`
Database record for a type plugin.
- `#define DDSC_LOG_STRING_RECORD 14`
String record.
- `#define DDSC_LOG_RECORD_CREATE_EC (DDSC_LOG_BASE + 109)`
Failed to create a database record of the specified kind.
- `#define DDSC_LOG_RECORD_DELETE_EC (DDSC_LOG_BASE + 110)`
Failed to delete a database record of the specified kind.
- `#define DDSC_LOG_RECORD_INSERT_EC (DDSC_LOG_BASE + 111)`
Failed to insert a database record of the specified kind.
- `#define DDSC_LOG_RECORD_ERROR_EC (DDSC_LOG_BASE + 112)`
Unknown error for database record of the specified kind.
- `#define DDSC_LOG_RECORD_EXISTS_EC (DDSC_LOG_BASE + 113)`
A database record of the specified kind already exists.
- `#define DDSC_LOG_RECORD_LOOKUP_EC (DDSC_LOG_BASE + 114)`
A lookup of a database record of the specified kind failed.
- `#define DDSC_LOG_RECORD_NOT_EXISTS_EC (DDSC_LOG_BASE + 115)`
A database record of the specified kind does not exist.
- `#define DDSC_LOG_RECORD_SELECT_EC (DDSC_LOG_BASE + 116)`
A database select on the specified record kind failed.
- `#define DDSC_LOG_RECORD_REMOVE_EC (DDSC_LOG_BASE + 117)`
Removal a database record of the specified kind failed.
- `#define DDSC_LOG_RECORD_INITIALIZE_EC (DDSC_LOG_BASE + 118)`
A database record of the specified kind could not be initialized.
- `#define DDSC_LOG_RECORD_FINALIZE_EC (DDSC_LOG_BASE + 119)`
A database record of the specified kind could not be finalized.
- `#define DDSC_LOG_PUBLICATIONDATA_OBJECT 1`
DDS PublicationBuiltinTopicData object.
- `#define DDSC_LOG_SUBSCRIPTIONDATA_OBJECT 2`
DDS SubscriptionBuiltinTopicData object.
- `#define DDSC_LOG_PARTICIPANTDATA_OBJECT 3`
DDS ParticipantBuiltinTopicData object.
- `#define DDSC_LOG_DATAREADERQOS_OBJECT 4`

- *DDS DataReaderQos object.*
- #define [DDSC_LOG_DATAWRITERQOS_OBJECT](#) 5
- *DDS DataWriterQos object.*
- #define [DDSC_LOG_TOPICQOS_OBJECT](#) 6
- *DDS TopicQos object.*
- #define [DDSC_LOG_PUBLISHERQOS_OBJECT](#) 7
- *DDS PublisherQos object.*
- #define [DDSC_LOG_SUBSCRIBERQOS_OBJECT](#) 8
- *DDS SubscriberQos object.*
- #define [DDSC_LOG_PARTICIPANTQOS_OBJECT](#) 9
- *DDS DomainParticipantQos object.*
- #define [DDSC_LOG_ENTITY_OBJECT](#) 10
- *DDS Entity object.*
- #define [DDSC_LOG_PARTICIPANTFACTORYQOS_OBJECT](#) 11
- *DDS DomainParticipantFactoryQos object.*
- #define [DDSC_LOG_TYPE_OBJECT](#) 12
- *A DDS Type object.*
- #define [DDSC_LOG_BINDRESOLVER_OBJECT](#) 13
- *A NETIO BindResolver object.*
- #define [DDSC_LOG_ROUTERESOLVER_OBJECT](#) 14
- *A NETIO RouteResolver object.*
- #define [DDSC_LOG_ADDRESSRESOLVER_OBJECT](#) 15
- *A NETIO AddressResolver object.*
- #define [DDSC_LOG_DATAWRITERIO_OBJECT](#) 16
- *A NETIO DataWriterInterface object.*
- #define [DDSC_LOG_DATAREADERIO_OBJECT](#) 17
- *A NETIO DataReaderInterface object.*
- #define [DDSC_LOG_LOG_OBJECT](#) 18
- *A OSAPI Log object.*
- #define [DDSC_LOG_SYSTEM_OBJECT](#) 19
- *A OSAPI system object.*
- #define [DDSC_LOG_MUTEX_OBJECT](#) 20
- *A OSAPI mutex object.*
- #define [DDSC_LOG_CONDREF_OBJECT](#) 21
- *A DDS Waitset condition reference object.*
- #define [DDSC_LOG_WSREF_OBJECT](#) 22
- *A DDS Condition waitset reference object.*
- #define [DDSC_LOG_MD5STREAM_OBJECT](#) 23
- *A MD5 stream object.*
- #define [DDSC_LOG_CONDITION_OBJECT](#) 24
- *A DDS Condition object.*
- #define [DDSC_LOG_RT_OBJECT](#) 25
- *A RT Condition object.*
- #define [DDSC_LOG_PARTICIPANT_POOL_OBJECT](#) 26
- *A Participant pool object.*
- #define [DDSC_LOG_TIMER_OBJECT](#) 27
- *A OSAPI_Timer object.*
- #define [DDSC_LOG_TIMEROUT_OBJECT](#) 28
- *A OSAPI_Timer timeout object.*
- #define [DDSC_LOG_TOPIC_OBJECT](#) 29
- *A DDS Topic object.*

- #define `DDSC_LOG_WAITSET_OBJECT` 30
A DDS WaitSet object.
- #define `DDSC_LOG_UUID_OBJECT` 31
A UUID object.
- #define `DDSC_LOG_STRING_OBJECT` 40
A String object.
- #define `DDSC_LOG_STRINGMANAGER_OBJECT` 43
A String Manager object.
- #define `DDSC_LOG_OBJECT_INITIALIZE_EC` (DDSC_LOG_BASE + 200)
Out of resources to initialize object of the specified kind.
- #define `DDSC_LOG_OBJECT_ALLOCATE_EC` (DDSC_LOG_BASE + 201)
Out of resources to allocate an object of the specified kind.
- #define `DDSC_LOG_OBJECT_FINALIZE_EC` (DDSC_LOG_BASE + 202)
Failed to finalize object of specified kind.
- #define `DDSC_LOG_OBJECT_DELETE_EC` (DDSC_LOG_BASE + 203)
Failed to delete object of specified kind.
- #define `DDSC_LOG_OBJECT_COPY_EC` (DDSC_LOG_BASE + 204)
Failed to copy object of specified kind.
- #define `DDSC_LOG_OBJECT_REFCOUNT_EC` (DDSC_LOG_BASE + 205)
Failed to delete/finalize an object because other objects are referencing it.
- #define `DDSC_LOG_OBJECT_GET_PROPERTY_EC` (DDSC_LOG_BASE + 206)
Failed to get the object properties.
- #define `DDSC_LOG_OBJECT_SET_PROPERTY_EC` (DDSC_LOG_BASE + 207)
Failed to set the object properties.
- #define `DDSC_LOG_OBJECT_EMPTY_EC` (DDSC_LOG_BASE + 208)
An object is empty, typically applies only to buffer-pool objects.
- #define `DDSC_LOG_OBJECT_INUSECOUNT_EC` (DDSC_LOG_BASE + 209)
Failed to delete/finalize an object because other objects are using it.
- #define `DDSC_LOG_NETMASK_SEQUENCE` 1
A NETIO Netmask sequence.
- #define `DDSC_LOG_ROUTE_SEQUENCE` 2
A NETIO Route sequence.
- #define `DDSC_LOG_RESERVED_SEQUENCE` 3
A NETIO Reserved address sequence.
- #define `DDSC_LOG_ENABLED_USER_TRANSPORT_SEQUENCE` 4
A DDS sequence of enabled user NETIO transports.
- #define `DDSC_LOG_ENABLED_TRANSPORT_SEQUENCE` 5
A DDS sequence of enabled transports.
- #define `DDSC_LOG_ENABLED_DISCOVERY_TRANSPORT_SEQUENCE` 6
A DDS sequence of enabled discovery transports.
- #define `DDSC_LOG_INITIAL_PEER_SEQUENCE` 7
A DDS sequence of initial peers.
- #define `DDSC_LOG_DESTINATION_SEQUENCE` 8
A NETIO sequence of destinations to send to.
- #define `DDSC_LOG_METAUNICAST_SEQUENCE` 9
A DDS sequence of meta-traffic unicast locators.
- #define `DDSC_LOG_METAMULTICAST_SEQUENCE` 10
A DDS sequence of meta-traffic multicast locators.
- #define `DDSC_LOG_USERUNICAST_SEQUENCE` 11
A DDS sequence of user-traffic unicast locators.
- #define `DDSC_LOG_USERMULTICAST_SEQUENCE` 12

- A DDS sequence of user-traffic multicast locators.*
- #define `DDSC_LOG_READTAKE_SEQUENCE` 13
 - A DDS sequence used in a read/take call.*
- #define `DDSC_LOG_SEQUENCE_SETMAX_EC` (DDSC_LOG_BASE + 300)
 - Failed to set the maximum length of a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_SETLENGTH_EC` (DDSC_LOG_BASE + 301)
 - Failed to set the length of a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_GETREF_EC` (DDSC_LOG_BASE + 302)
 - Failed to get a reference at the specified index for a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_INITIALIZE_EC` (DDSC_LOG_BASE + 303)
 - Failed to initialize a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_FINALIZE_EC` (DDSC_LOG_BASE + 304)
 - Failed to finalize a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_COPY_EC` (DDSC_LOG_BASE + 305)
 - Failed to copy a sequence of the specified kind.*
- #define `DDSC_LOG_SEQUENCE_INVALID_EC` (DDSC_LOG_BASE + 306)
 - The sequence of the specified kind was invalid in the context it is used.*
- #define `DDSC_LOG_DISCOVERY_COMPONENT` 1
 - A component of discovery kind.*
- #define `DDSC_LOG_RTPS_COMPONENT` 2
 - A component of RTPS kind.*
- #define `DDSC_LOG_READERHISTORY_COMPONENT` 3
 - A component of reader-history kind.*
- #define `DDSC_LOG_WRITERHISTORY_COMPONENT` 4
 - A component of writer-history kind.*
- #define `DDSC_LOG_DATAREADERIO_COMPONENT` 5
 - A component of DataReaderInterface kind.*
- #define `DDSC_LOG_DATAWRITERIO_COMPONENT` 6
 - A component of DataWriterInterface kind.*
- #define `DDSC_LOG_NETIO_COMPONENT` 7
 - A component of NETIO kind.*
- #define `DDSC_LOG_TRANSPORT_COMPONENT` 8
 - A component of transport kind.*
- #define `DDSC_LOG_COMPONENT_LOOKUP_EC` (DDSC_LOG_BASE + 400)
 - Did not find a component factory with the given name in the registry.*
- #define `DDSC_LOG_COMPONENT_CREATE_EC` (DDSC_LOG_BASE + 401)
 - Could not create a component of the specified kind using the specified factory.*
- #define `DDSC_LOG_COMPONENT_DELETE_EC` (DDSC_LOG_BASE + 402)
 - Could not delete a component of the specified kind using the specified factory.*
- #define `DDSC_LOG_HISTORY_RESOURCE` 1
 - Exceeded resource limits for writer history queue.*
- #define `DDSC_LOG_SAMPLE_RESOURCES` 2
 - Failed to get resource for new sample due to resource limit.*
- #define `DDSC_LOG_PARTICIPANT_RESOURCES` 3
 - Failed to allocate a new participant.*
- #define `DDSC_LOG_RESOURCE_EXCEEDED_EC` (DDSC_LOG_BASE + 500)
 - Could not allocate a resource of the specified kind.*
- #define `DDSC_LOG_TOPIC_QOS` 1
 - The TopicQos kind.*
- #define `DDSC_LOG_DATAREADER_QOS` 2
 - The DataReaderQos kind.*

- #define [DDSC_LOG_DATAWRITER_QOS](#) 3
The DataWriterQos kind.
- #define [DDSC_LOG_SUBSCRIBER_QOS](#) 4
The SubscriberQos kind.
- #define [DDSC_LOG_PUBLISHER_QOS](#) 5
The PublisherQos kind.
- #define [DDSC_LOG_PARTICIPANT_QOS](#) 6
The DomainParticipantQos kind.
- #define [DDSC_LOG_PARTICIPANTFACTORY_QOS](#) 7
The DomainParticipantFactoryQos kind.
- #define [DDSC_LOG_DEFAULTTOPIC_QOS](#) 8
The default TopicQos kind.
- #define [DDSC_LOG_DEFAULTDATAREADER_QOS](#) 9
The default DataReaderQos kind.
- #define [DDSC_LOG_DEFAULTDATAWRITER_QOS](#) 10
The default DataWriterQos kind.
- #define [DDSC_LOG_DEFAULTSUBSCRIBER_QOS](#) 11
The default SubscriberQos kind.
- #define [DDSC_LOG_DEFAULTPUBLISHER_QOS](#) 12
The default PublisherQos kind.
- #define [DDSC_LOG_DEFAULTPARTICIPANT_QOS](#) 13
The default PublisherQos kind.
- #define [DDSC_LOG_DEADLINE_QOS_POLICY](#) 14
The deadline qos policy kind.
- #define [DDSC_LOG_LIVELINESS_QOS_POLICY](#) 15
The liveliness qos policy kind.
- #define [DDSC_LOG_HISTORY_QOS_POLICY](#) 16
The history qos policy kind.
- #define [DDSC_LOG_RESOURCE_LIMIT_QOS_POLICY](#) 17
The resource limits qos policy kind.
- #define [DDSC_LOG_PROTOCOL_QOS_POLICY](#) 18
The protocol qos policy kind.
- #define [DDSC_LOG_TYPE_SUPPORT_QOS_POLICY](#) 19
The type-support qos policy kind.
- #define [DDSC_LOG_RELIABILITY_QOS_POLICY](#) 20
The reliability qos policy kind.
- #define [DDSC_LOG_DURABILITY_QOS_POLICY](#) 21
The durability qos policy kind.
- #define [DDSC_LOG_OWNERSHIP_QOS_POLICY](#) 22
The ownership qos policy kind.
- #define [DDSC_LOG_OWNERSHIP_STRENGTH_QOS_POLICY](#) 23
The ownership-strength qos policy kind.
- #define [DDSC_LOG_TRANSPORT_QOS_POLICY](#) 24
The transport qos policy kind.
- #define [DDSC_LOG_PARTICIPANT_ID_QOS_POLICY](#) 25
The participant id qos policy kind.
- #define [DDSC_LOG_HEARTBEATS_QOS_POLICY](#) 26
The heartbeat qos policy kind.
- #define [DDSC_LOG_DATAWRITER_RESOURCE_QOS_POLICY](#) 27
The DataWriterQos writer_resource_limits policy.
- #define [DDSC_LOG_DATAREADER_RESOURCE_QOS_POLICY](#) 28

- The DataReaderQos reader_resource_limits policy.*
- #define [DDSC_LOG_DESTINATION_ORDER_POLICY](#) 29
 - The DestinationOrderQos destination_order policy.*
- #define [DDSC_LOG_USER_TRAFFIC_QOS_POLICY](#) 30
 - The UserTraffic policy.*
- #define [DDSC_LOG_DISCOVERY_TRAFFIC_QOS_POLICY](#) 31
 - The Discovery.enabled_transport policy.*
- #define [DDSC_LOG_QOS_INCONSISTENT_POLICY_EC](#) (DDSC_LOG_BASE + 501)
 - An inconsistent Qos policy for the specified Qos kind was found.*
- #define [DDSC_LOG_QOS_INCONSISTENT_POLICIES_EC](#) (DDSC_LOG_BASE + 502)
 - Inconsistency between two Qos policies for the specified Qos kind was found.*
- #define [DDSC_LOG_QOS_INCONSISTENT_EC](#) (DDSC_LOG_BASE + 503)
 - Failed to create an entity or set a qos due to inconsistent policy.*
- #define [DDSC_LOG_QOS_COPY_EC](#) (DDSC_LOG_BASE + 504)
 - Failed to copy a Qos of the specified kind.*
- #define [DDSC_LOG_QOS_INITIALIZE_EC](#) (DDSC_LOG_BASE + 505)
 - Failed to initialize a Qos of the specified kind.*
- #define [DDSC_LOG_QOS_FINALIZE_EC](#) (DDSC_LOG_BASE + 506)
 - Failed to finalize a Qos of the specified kind.*
- #define [DDSC_LOG_QOS_SET_EC](#) (DDSC_LOG_BASE + 507)
 - Failed to set a Qos of the specified kind.*
- #define [DDSC_LOG_QOS_SET_ON_ENABLED_EC](#) (DDSC_LOG_BASE + 508)
 - Failed to set a Qos of the specified kind because the entity is already enabled.*
- #define [DDSC_LOG_QOS_IMMUTABLE_EC](#) (DDSC_LOG_BASE + 509)
 - Failed to set a Qos of the specified kind the immutable Qos policies have been changed.*
- #define [DDSC_LOG_QOS_CHANGED_EC](#) (DDSC_LOG_BASE + 510)
 - A discovered Qos changed (the entity already existed)*
- #define [DDSC_LOG_QOS_GET_EC](#) (DDSC_LOG_BASE + 511)
 - Failed to get a Qos of the specified kind.*
- #define [DDSC_LOG_TOPIC_LISTENER](#) 1
 - The topic listener kind.*
- #define [DDSC_LOG_DATAREADER_LISTENER](#) 2
 - The datareader listener kind.*
- #define [DDSC_LOG_DATAWRITER_LISTENER](#) 3
 - The datawriter listener kind.*
- #define [DDSC_LOG_SUBSCRIBER_LISTENER](#) 4
 - The subscriber listener kind.*
- #define [DDSC_LOG_PUBLISHER_LISTENER](#) 5
 - The publisher listener kind.*
- #define [DDSC_LOG_PARTICIPANT_LISTENER](#) 6
 - The participant listener kind.*
- #define [DDSC_LOG_PARTICIPANTFACTORY_LISTENER](#) 7
 - The participant-factory listener kind.*
- #define [DDSC_LOG_LISTENER_INCONSISTENT_EC](#) (DDSC_LOG_BASE + 512)
 - Failed to create an entity due to inconsistent listener and status mask.*
- #define [DDSC_LOG_LISTENER_SET_EC](#) (DDSC_LOG_BASE + 513)
 - Failed to set the listener of the specified kind.*
- #define [DDSC_LOG_LISTENER_GET_EC](#) (DDSC_LOG_BASE + 514)
 - Failed to get the listener of the specified kind.*
- #define [DDSC_LOG_LISTENER_SET_ILLEGAL_NULL_EC](#) (DDSC_LOG_BASE + 515)
 - Illegal combination of NULL listener and non-NONE status mask when setting a listener for an Entity.*

- #define `DDSC_LOG_TOPIC_ENTITY` 1
The topic entity kind.
- #define `DDSC_LOG_DATAWRITER_ENTITY` 2
The datawriter entity kind.
- #define `DDSC_LOG_DATAREADER_ENTITY` 3
The datareader entity kind.
- #define `DDSC_LOG_PUBLISHER_ENTITY` 4
The publisher entity kind.
- #define `DDSC_LOG_SUBSCRIBER_ENTITY` 5
The subscriber entity kind.
- #define `DDSC_LOG_PARTICIPANT_ENTITY` 6
The participant entity kind.
- #define `DDSC_LOG_PUBLICATION_ENTITY` 7
The publication entity kind.
- #define `DDSC_LOG_SUBSCRIPTION_ENTITY` 8
The subscription entity kind.
- #define `DDSC_LOG_ENTITY_ENABLE_EC` (DDSC_LOG_BASE + 600)
Failed to enable an entity of the specified kind.
- #define `DDSC_LOG_ENTITY_NOT_EMPTY_EC` (DDSC_LOG_BASE + 601)
Failed to an delete/finalize an entity of the specified kind because it is not empty.
- #define `DDSC_LOG_ENTITY_FINALIZE_EC` (DDSC_LOG_BASE + 602)
Failed to finalize an entity of the specified kind.
- #define `DDSC_LOG_ENTITY_INITIALIZE_EC` (DDSC_LOG_BASE + 603)
Failed to initialize an entity of the specified kind.
- #define `DDSC_LOG_ENTITY_NOT_ENABLED_EC` (DDSC_LOG_BASE + 604)
An operation was attempted on an entity that is not enabled.
- #define `DDSC_LOG_ENTITY_DIFFERENT_FACTORY_EC` (DDSC_LOG_BASE + 605)
Entities are in different factories.
- #define `DDSC_LOG_DATAWRITER_CDR` 1
The datawriter CDR kind.
- #define `DDSC_LOG_DATAREADER_CDR` 2
The datareader CDR kind.
- #define `DDSC_LOG_CDR_POOL_ALLOC_EC` (DDSC_LOG_BASE + 700)
Failed to allocate a pool of the specified kind.
- #define `DDSC_LOG_CDR_BUFFER_SET_EC` (DDSC_LOG_BASE + 701)
Failed to set the CDR buffer for a packet.
- #define `DDSC_LOG_CDR_POOL_DELETE_EC` (DDSC_LOG_BASE + 702)
Failed to delete the CDR pool.
- #define `DDSC_LOG_CDR_SERIALIZE_PID_EC` (DDSC_LOG_BASE + 703)
Failed to serialize a parameter ID.
- #define `DDSC_LOG_CDR_SERIALIZE_PID_LENGTH_EC` (DDSC_LOG_BASE + 704)
Failed to serialize a parameter length.
- #define `DDSC_LOG_CDR_SERIALIZE_KEYHASH_EC` (DDSC_LOG_BASE + 705)
Failed to serialize a key-hash.
- #define `DDSC_LOG_CDR_SERIALIZE_DATA_EC` (DDSC_LOG_BASE + 706)
Failed to serialize payload data.
- #define `DDSC_LOG_DESERIALIZE_BAD_PID_LENGTH_EC` (DDSC_LOG_BASE + 707)
Deserialized an invalid parameter length for a specific parameter ID.
- #define `DDSC_LOG_CDR_DESERIALIZE_PID_EC` (DDSC_LOG_BASE + 708)
Failed to deserialize the ID of an inline parameter.
- #define `DDSC_LOG_CDR_DESERIALIZE_PID_LENGTH_EC` (DDSC_LOG_BASE + 709)

- Failed to deserialize the length of an inline parameter.*

 - #define [DDSC_LOG_CDR_INCREMENT_POS_EC](#) (DDSC_LOG_BASE + 710)

Failed to increment to the position of the next inline parameter.
- #define [DDSC_LOG_CDR_SET_POS_EC](#) (DDSC_LOG_BASE + 711)

Failed to set the reception stream position.
- #define [DDSC_LOG_CDR_DESERIALIZE_HEADER_EC](#) (DDSC_LOG_BASE + 712)

Failed to deserialize the encapsulation header.
- #define [DDSC_LOG_CDR_DESERIALIZE_DATA_EC](#) (DDSC_LOG_BASE + 713)

Failed to deserialize CDR payload data.
- #define [DDSC_LOG_CDR_INITIALIZE_SAMPLE_EC](#) (DDSC_LOG_BASE + 714)

Failed to initialize CDR sample.
- #define [DDSC_LOG_CDR_FINALIZE_SAMPLE_EC](#) (DDSC_LOG_BASE + 715)

Failed to finalize sample.
- #define [DDSC_LOG_CDR_SERIALIZE_STATUS_INFO_EC](#) (DDSC_LOG_BASE + 716)

Failed to serialize the status info parameter.
- #define [DDSC_LOG_CDR_DESERIALIZE_KEYHASH_EC](#) (DDSC_LOG_BASE + 717)

Failed to deserialize a key-hash.
- #define [DDSC_LOG_CDR_DESERIALIZE_KEY_EC](#) (DDSC_LOG_BASE + 718)

Failed to deserialize CDR payload key.
- #define [DDSC_LOG_NETIO_ADD_ANON_TOPIC_ROUTE_EC](#) (DDSC_LOG_BASE + 800)

Failed to add a route to an anonymous participant discovery datawriter.
- #define [DDSC_LOG_NETIO_ADD_TOPIC_ROUTE_EC](#) (DDSC_LOG_BASE + 801)

Failed to add a route to a topic from a datawriter.
- #define [DDSC_LOG_NETIO_DELETE_TOPIC_ROUTE_EC](#) (DDSC_LOG_BASE + 802)

Failed to delete a route to a topic.
- #define [DDSC_LOG_NETIO_FORWARD_TOPIC_EC](#) (DDSC_LOG_BASE + 803)

Failed to forward a topic.
- #define [DDSC_LOG_NETIO_NETIO_KIND](#) 1

Any NETIO interface kind, typically UDP.
- #define [DDSC_LOG_INTRA_NETIO_KIND](#) 2

The intra interface kind.
- #define [DDSC_LOG_RTPS_NETIO_KIND](#) 3

The RTPS interface kind.
- #define [DDSC_LOG_DATAREADER_NETIO_KIND](#) 4

The DataReader interface kind.
- #define [DDSC_LOG_DATAWRITER_NETIO_KIND](#) 5

The DataWriter interface kind.
- #define [DDSC_LOG_NETIO_BIND_EXTERNAL_EC](#) (DDSC_LOG_BASE + 804)

Failed to bind two external interface of the specified kind.
- #define [DDSC_LOG_NETIO_UNBIND_EXTERNAL_EC](#) (DDSC_LOG_BASE + 805)

Failed to unbind two external interfaces of the specified kind.
- #define [DDSC_LOG_NETIO_BIND_EC](#) (DDSC_LOG_BASE + 806)

Failed to bind an interface to a peer interface.
- #define [DDSC_LOG_NETIO_UNBIND_EC](#) (DDSC_LOG_BASE + 807)

Failed to unbind an interface from a peer interface.
- #define [DDSC_LOG_NETIO_ADD_ROUTE_EC](#) (DDSC_LOG_BASE + 808)

Failed to add a route from an interface to a peer interface.
- #define [DDSC_LOG_NETIO_DELETE_ROUTE_EC](#) (DDSC_LOG_BASE + 809)

Failed to delete a route from an interface to a peer interface.
- #define [DDSC_LOG_NETIO_GET_EXTERNAL_INTF_EC](#) (DDSC_LOG_BASE + 810)

Failed to get an external interface for the specified interface kind.

- #define `DDSC_LOG_NETIO_NO_ROUTE_EC` (DDSC_LOG_BASE + 811)
A DataReader failed a bind due to no existing route.
- #define `DDSC_LOG_NETIO_ROUTE_LOOKUP_FAILED_EC` (DDSC_LOG_BASE + 812)
Lookup a route to a destination failed.
- #define `DDSC_LOG_NETIO_GET_ROUTE_TABLE_FAILED_EC` (DDSC_LOG_BASE + 813)
Failed to get the route table for an interface.
- #define `DDSC_LOG_NETIO_SEND_FAILED_EC` (DDSC_LOG_BASE + 814)
Failure when sending on an interface.
- #define `DDSC_LOG_NETIO_SET_STATE_EC` (DDSC_LOG_BASE + 815)
Failed to set an interface state.
- #define `DDSC_LOG_NETIO_PEER_LOOKUP_EC` (DDSC_LOG_BASE + 816)
Datawriter did not find a peer.
- #define `DDSC_LOG_NETIO_FORCED_REMOVE_EC` (DDSC_LOG_BASE + 817)
Forced removal of sample downstream failed.
- #define `DDSC_LOG_PACKET_INIT_EC` (DDSC_LOG_BASE + 818)
Failed to initialize a packet.
- #define `DDSC_LOG_PACKET_SET_HEAD_EC` (DDSC_LOG_BASE + 819)
Failed to set the head of a packet.
- #define `DDSC_LOG_PACKET_SET_TAIL_EC` (DDSC_LOG_BASE + 820)
Failed to set the tail of a packet.
- #define `DDSC_LOG_DW_ACKNACK_FAILED_EC` (DDSC_LOG_BASE + 900)
Failed to ACKNACK sample in the writer history.
- #define `DDSC_LOG_DW_COMMIT_EC` (DDSC_LOG_BASE + 901)
Failed to commit a sample to the writer queue.
- #define `DDSC_LOG_DW_KEYHASH_CREATE_EC` (DDSC_LOG_BASE + 902)
Failed to create keyhash of instance handle.
- #define `DDSC_LOG_DW_ILLEGAL_KEY_KIND_EC` (DDSC_LOG_BASE + 903)
Failed a write due to an invalid key kind for the type being written.
- #define `DDSC_LOG_DW_CREATE_TYPED_WRITER_EC` (DDSC_LOG_BASE + 904)
Failed to create a typed writer.
- #define `DDSC_LOG_DW_HISTORY_REGISTER_KEY_EC` (DDSC_LOG_BASE + 905)
Failed to register the key of an instance.
- #define `DDSC_LOG_DR_CREATE_TYPED_READER_EC` (DDSC_LOG_BASE + 1000)
Failed to create a typed datareader.
- #define `DDSC_LOG_DR_COPY_DATA_SAMPLE_EC` (DDSC_LOG_BASE + 1001)
Failed to copy a sample upon reception, read, or take.
- #define `DDSC_LOG_DR_COMMIT_SAMPLE_EC` (DDSC_LOG_BASE + 1002)
Failed to commit a sample to be made available to be read or taken.
- #define `DDSC_LOG_DR_FILTER_ERROR_EC` (DDSC_LOG_BASE + 1003)
A datareader filter function failed.
- #define `DDSC_LOG_DR_DESERIALIZE_KEYHASH_EC` (DDSC_LOG_BASE + 1004)
Failed to deserialize a key-hash parameter.
- #define `DDSC_LOG_DR_GET_ENTRY_FAILED_EC` (DDSC_LOG_BASE + 1005)
Failed to get a Reader History entry for a received sample.
- #define `DDSC_LOG_DR_COMMIT_ENTRY_EC` (DDSC_LOG_BASE + 1006)
Failed to commit a receive sample to Reader History to be read or taken.
- #define `DDSC_LOG_DR_UNREGISTER_KEY_EC` (DDSC_LOG_BASE + 1007)
A DataReader failed to unregister an instance.
- #define `DDSC_LOG_DR_DISPOSE_KEY_EC` (DDSC_LOG_BASE + 1008)
A DataReader failed to dispose an instance.
- #define `DDSC_LOG_DR_READ_TAKE_FAILURE_EC` (DDSC_LOG_BASE + 1009)

- A call to a reader/take function failed.*

 - #define `DDSC_LOG_DR_INSTANCE_TO_KEYHASH_EC` (DDSC_LOG_BASE + 1010)

Failed to create keyhash of instance handle.
- #define `DDSC_LOG_TYPE_NAME_CMP_EC` (DDSC_LOG_BASE + 1100)

Two type names are incompatible.
- #define `DDSC_LOG_TOPIC_NAME_CMP_EC` (DDSC_LOG_BASE + 1101)

Two topic names are incompatible.
- #define `DDSC_LOG_TYPE_FUNCTION_GET_SERIALIZED_SAMPLE_MAX_SIZE` 1

The get_serialized_sample_max_size function kind.
- #define `DDSC_LOG_TYPE_FUNCTION_SERIALIZE_DATA` 2

The serialize_data function kind.
- #define `DDSC_LOG_TYPE_FUNCTION_DESERIALIZE_DATA` 3

The deserialize_data function kind.
- #define `DDSC_LOG_TYPE_FUNCTION_CREATE_SAMPLE` 4

The create_sample function kind.
- #define `DDSC_LOG_TYPE_FUNCTION_COPY_SAMPLE` 5

The copy_sample function kind.
- #define `DDSC_LOG_TYPE_FUNCTION_DELETE_SAMPLE` 6

The delete_sample function kind.
- #define `DDSC_LOG_TYPE_FUNCTION_GET_KEY_KIND` 7

The get_key_kind function kind.
- #define `DDSC_LOG_TYPE_FUNCTION_INSTANCE_TO_KEYHASH` 8

The instance_to_keyhash function kind.
- #define `DDSC_LOG_TYPE_FUNCTION_NULL_EC` (DDSC_LOG_BASE + 1102)

Invalid type plugin, The specified function pointer is NULL.
- #define `DDSC_LOG_TOPIC_TOO_LONG_EC` (DDSC_LOG_BASE + 1103)

Failed to create a topic because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)
- #define `DDSC_LOG_TYPE_TOO_LONG_EC` (DDSC_LOG_BASE + 1104)

Failed to create a type because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)
- #define `DDSC_LOG_LOOKUP_TYPE_PLUGIN_EC` (DDSC_LOG_BASE + 1105)

A type-plugin for the given type could not be found.
- #define `DDSC_LOG_TYPE_KEY_TYPE_EC` (DDSC_LOG_BASE + 1106)

Two types have incompatible keys.
- #define `DDSC_LOG_DUPLICATE_GUID_PREFIX_EC` (DDSC_LOG_BASE + 1107)

A DomainParticipant with the specified GUID already exists in the DomainParticipant factory.
- #define `DDSC_LOG_DISC_LOCAL_PARTICIPANT_ENABLED_EC` (DDSC_LOG_BASE + 1200)

Discovery plugin failed its update after a local DomainParticipant was enabled.
- #define `DDSC_LOG_DISC_BEFORE_LOCAL_PARTICIPANT_CREATED_EC` (DDSC_LOG_BASE + 1201)

Discovery plugin failed its update before a local DomainParticipant was created.
- #define `DDSC_LOG_DISC_AFTER_LOCAL_PARTICIPANT_CREATED_EC` (DDSC_LOG_BASE + 1202)

Discovery plugin failed its update after a local DomainParticipant was created.
- #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_ENABLED_EC` (DDSC_LOG_BASE + 1203)

Discovery plugin failed its update after a local DataReader was enabled.
- #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_DELETED_EC` (DDSC_LOG_BASE + 1204)

Discovery plugin failed its update after a local DataReader was deleted.
- #define `DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_ENABLED_EC` (DDSC_LOG_BASE + 1205)

Discovery plugin failed its update after a local DataWriter was enabled.

- `#define DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_DELETED_EC` (DDSC_LOG_BASE + 1206)
Discovery plugin failed its update after a local DataWriter was deleted.
- `#define DDSC_LOG_DISC_REMOTE_PARTICIPANT_EXPIRED_EC` (DDSC_LOG_BASE + 1207)
Discovery plugin failed its update after a remote DomainParticipant's liveliness expired.
- `#define DDSC_LOG_DISC_ADD_PEER_EC` (DDSC_LOG_BASE + 1208)
Failed to add a peer with discovery plugin.
- `#define DDSC_LOG_CHECKSUM_REQUIRED_EC` (DDSC_LOG_BASE + 1301)
A participant requires checksum, but the discovered participant does not send a checksum.
- `#define DDSC_LOG_CHECKSUM_INCOMPATIBLE_EC` (DDSC_LOG_BASE + 1302)
A participant does not support checksum's sent by a discovered participant.
- `#define DDSC_LOG_CHECKSUM_INCONSISTENT_CLASS_EC` (DDSC_LOG_BASE + 1303)
The checksum class id for a custom checksum does match.
- `#define DDSC_LOG_CHECKSUM_INCONSISTENT_COMPUTE_EC` (DDSC_LOG_BASE + 1304)
The computed_crc_kind is inconsistent with what is allowed or supported.
- `#define DDSC_LOG_CHECKSUM_INCONSISTENT_ALLOW_EC` (DDSC_LOG_BASE + 1305)
The allowed_crc_mask is inconsistent with what is supported.

6.65.1 Detailed Description

DDS C. ModuleID = 7.

6.65.2 Macro Definition Documentation

6.65.2.1 `#define DDSC_LOG_INVALID_DURATION_EC` (DDSC_LOG_BASE + 1)

The specified duration is not valid.

6.65.2.2 `#define DDSC_LOG_INVALID_PARTICIPANT_NAME_EC` (DDSC_LOG_BASE + 2)

An invalid participant name was specified with an unknown GUID.

6.65.2.3 `#define DDSC_LOG_INVALID_PARTICIPANT_GUID_PREFIX_EC` (DDSC_LOG_BASE + 3)

An invalid participant GUID prefix was specified, typically the GUID prefix does not match an already detected participant.

6.65.2.4 `#define DDSC_LOG_SYS_GETTIME_EC` (DDSC_LOG_BASE + 4)

A call to `OSAPI_System_get_time` failed.

6.65.2.5 `#define DDSC_LOG_GET_NEXT_OBJECT_ID_EC` (DDSC_LOG_BASE + 5)

Failed to get the next automatically generated object ID for an entity's GUID. This typically means the object id pool has been exhausted.

6.65.2.6 `#define DDSC_LOG_SET_ENTITY_NAME_EC` (DDSC_LOG_BASE + 6)

Failed to set name string for a DDS entity in the `DomainParticipantQos` `entity_name` policy.

6.65.2.7 `#define DDSC_LOG_SYS_GET_HOSTNAME_EC` (DDSC_LOG_BASE + 7)

A call to `OSAPI_System_get_hostname` failed.

6.65.2.8 `#define DDSC_LOG_IO_SNPRINTF_FAILED_EC` (DDSC_LOG_BASE + 8)

A call to `OSAPI_Stdio_snprintf` failed. Typically this means the destination buffer was too small.

6.65.2.9 #define DDSC_LOG_TOPIC_FIND_EC (DDSC_LOG_BASE + 9)

Failed to find a topic created by a DomainParticipant.

6.65.2.10 #define DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_NAME_EC (DDSC_LOG_BASE + 10)

Endpoint discovery failed because the name of the remote participant parent for an endpoint was not found.

6.65.2.11 #define DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_KEY_EC (DDSC_LOG_BASE + 11)

Endpoint discovery failed because the key of the remote participant parent for an endpoint was not found. Note that the key is logged as 4 integers in host endianness format.

6.65.2.12 #define DDSC_LOG_REMOTE_PARTICIPANT_KEY_NOT_EQUAL_EC (DDSC_LOG_BASE + 12)

Failed endpoint discovery when key does not match the remote participant.

6.65.2.13 #define DDSC_LOG_INVALID_ENDPOINT_GUID_EC (DDSC_LOG_BASE + 13)

Failed endpoint discovery due to an invalid or unknown endpoint GUID.

6.65.2.14 #define DDSC_LOG_ENDPOINT_NOT_CHILD_OF_PARTICIPANT_EC (DDSC_LOG_BASE + 14)

Failed endpoint discovery when an endpoint is determined to belong to a different remote participant.

6.65.2.15 #define DDSC_LOG_PARTICIPANT_DOES_NOT_EXIST_EC (DDSC_LOG_BASE + 15)

Failed participant discovery because a remote participant that should have been already asserted locally was not found.

6.65.2.16 #define DDSC_LOG_REFRESH_REM_PARTICIPANT_EC (DDSC_LOG_BASE + 16)

Did not find a remote participant when asserting participant liveness for it. Note that the key is logged as 4 integers in host endianness format.

6.65.2.17 #define DDSC_LOG_REFRESH_REM_PARTICIPANT_TIMEOUT_EC (DDSC_LOG_BASE + 17)

Failed to assert participant liveness to a remote participant.

6.65.2.18 #define DDSC_LOG_PARTICIPANT_LOOKUP_EC (DDSC_LOG_BASE + 18)

Failed to find a remote participant as previously discovered.

6.65.2.19 #define DDSC_LOG_REMOVE_PUBLICATION_EC (DDSC_LOG_BASE + 19)

Failed to remove resources for a remote publication.

6.65.2.20 #define DDSC_LOG_REMOVE_SUBSCRIPTION_EC (DDSC_LOG_BASE + 20)

Failed to remove resources for a remote subscription.

6.65.2.21 #define DDSC_LOG_FIND_PUBLICATION_PARENT_EC (DDSC_LOG_BASE + 21)

Cannot determine the participant of a remote publication.

6.65.2.22 #define DDSC_LOG_FIND_SUBSCRIPTION_PARENT_EC (DDSC_LOG_BASE + 22)

Cannot determine the participant of a remote subscription.

6.65.2.23 #define DDSC_LOG_MAX_PARTICIPANT_ID_REACHED_EC (DDSC_LOG_BASE + 23)

Failed to create DomainParticipant due to running out of participant IDs. This is typically caused by all UDP ports being used.

6.65.2.24 #define DDSC_LOG_RESERVE_LOCATORS_EC (DDSC_LOG_BASE + 24)

Failed to reserve endpoint locators.

6.65.2.25 #define DDSC_LOG_TIMER_CREATE_TIMEOUT_EC (DDSC_LOG_BASE + 25)

Failed to create a timeout.

6.65.2.26 #define DDSC_LOG_ILLEGAL_OBJECTID_EC (DDSC_LOG_BASE + 26)

Illegal object id specified.

Manually specified object id must be in the range [0,0xffffffff]

6.65.2.27 #define DDSC_LOG_TOPIC_NARROW_EC (DDSC_LOG_BASE + 27)

Failed to narrow a TopicDescription to the named Topic.

6.65.2.28 #define DDSC_LOG_FAILED_UPDATE_STATUS_CONDITION_EC (DDSC_LOG_BASE + 28)

Failed to update state of StatusCondition.

An error occurred when trying to update the state (i.e. the trigger value) of a StatusCondition.

6.65.2.29 #define DDSC_LOG_WS_REMOVE_COND_REFERENCE_EC (DDSC_LOG_BASE + 29)

Failed to remove a condition reference from a waitset.

6.65.2.30 #define DDSC_LOG_WS_ADD_COND_REFERENCE_EC (DDSC_LOG_BASE + 30)

Failed to add a condition reference to a waitset.

6.65.2.31 #define DDSC_LOG_RELEASE_META_MC_EC (DDSC_LOG_BASE + 31)

Failed to release resources for multicast discovery locators.

6.65.2.32 #define DDSC_LOG_RELEASE_META_UC_EC (DDSC_LOG_BASE + 32)

Failed to release resources for unicast discovery locators.

6.65.2.33 #define DDSC_LOG_RELEASE_USER_MC_EC (DDSC_LOG_BASE + 33)

Failed to release resources for multicast user locators.

6.65.2.34 #define DDSC_LOG_RELEASE_USER_UC_EC (DDSC_LOG_BASE + 34)

Failed to release resources for unicast user locators.

6.65.2.35 #define DDSC_LOG_INVALID_DOMAINID_EC (DDSC_LOG_BASE + 35)

The domain ID specified exceeds what is allowed based on the parameters specified in `dp_qos.protocol.rtps_well-known_ports`.

6.65.2.36 #define DDSC_LOG_DATABASE_CREATE_EC (DDSC_LOG_BASE + 100)

Failed to create database.

6.65.2.37 **#define DDSC_LOG_DATABASE_DELETE_EC (DDSC_LOG_BASE + 101)**

Failed to delete database.

6.65.2.38 **#define DDSC_LOG_TABLE_CREATE_EC (DDSC_LOG_BASE + 102)**

Failed to create database table of the specified name.

6.65.2.39 **#define DDSC_LOG_TABLE_INUSE_EC (DDSC_LOG_BASE + 103)**

Failed to delete a database table because it is not empty.

6.65.2.40 **#define DDSC_LOG_TABLE_DELETE_EC (DDSC_LOG_BASE + 104)**

Failed to delete a database table.

6.65.2.41 **#define DDSC_LOG_TABLE_SELECT_EC (DDSC_LOG_BASE + 105)**

A selection operation failed on the specified database table.

6.65.2.42 **#define DDSC_LOG_CREATE_INDEX_EC (DDSC_LOG_BASE + 106)**

Failed to create an index on a database table.

6.65.2.43 **#define DDSC_LOG_DELETE_INDEX_EC (DDSC_LOG_BASE + 107)**

Failed to delete an index on a database table.

6.65.2.44 **#define DDSC_LOG_DB_CURSOR_INVALIDATED_EC (DDSC_LOG_BASE + 108)**

A database table cursor was invalidated while in use.

6.65.2.45 **#define DDSC_LOG_SUBSCRIPTION_RECORD 1**

Database record for a Subscription.

6.65.2.46 **#define DDSC_LOG_PUBLICATION_RECORD 2**

Database record for a Publication.

6.65.2.47 **#define DDSC_LOG_PARTICIPANT_RECORD 3**

Database record for a Remote Participant.

6.65.2.48 **#define DDSC_LOG_TOPIC_RECORD 4**

Database record for a Topic.

6.65.2.49 **#define DDSC_LOG_DATAWRITER_RECORD 5**

Database record for a DataWriter.

6.65.2.50 **#define DDSC_LOG_DATAREADER_RECORD 6**

Database record for a DataReader.

6.65.2.51 **#define DDSC_LOG_PUBLISHER_RECORD 7**

Database record for a Publisher.

6.65.2.52 #define DDSC_LOG_SUBSCRIBER_RECORD 8

Database record for a Subscriber.

6.65.2.53 #define DDSC_LOG_ROUTE_RECORD 9

Database record for a route record.

6.65.2.54 #define DDSC_LOG_BIND_RECORD 10

Database record for a bind record.

6.65.2.55 #define DDSC_LOG_TYPE_RECORD 11

Database record for a type plugin.

6.65.2.56 #define DDSC_LOG_STRING_RECORD 14

String record.

6.65.2.57 #define DDSC_LOG_RECORD_CREATE_EC (DDSC_LOG_BASE + 109)

Failed to create a database record of the specified kind.

The creation of an internal database record failed. The failure may have been caused by insufficient resources based on the record kind. The following resource-limits may apply:

DomainParticipantQos.resource_limit.local_publisher_allocation limits the number of DDS Publishers.

DomainParticipantQos.resource_limit.local_subscriber_allocation limits the number of DDS Subscribers.

DomainParticipantQos.resource_limit.local_topic_allocation limits the number of DDS Topics.

DomainParticipantQos.resource_limits.local_reader_allocation limits the number of DDS DataReader records.

DomainParticipantQos.resource_limits.local_writer_allocation limits the number of DDS DataWriter records.

DomainParticipantQos.resource_limits.remote_writer_allocation limits the number of remote publication records.

DomainParticipantQos.resource_limits.remote_reader_allocation limits the number of remote subscription records.

DomainParticipantQos.resource_limits.remote_participant_allocation limits the number of remote participant records.

DataWriterQos.writer_resource_limits.max_remote_readers limits the number of DDS DataReaders a DDS DataWriter can communicate with.

DataReaderQos.reader_resource_limits.max_remote_writers limits the number of DDS DataWriter a DDS DataReader can communicate with.

6.65.2.58 #define DDSC_LOG_RECORD_DELETE_EC (DDSC_LOG_BASE + 110)

Failed to delete a database record of the specified kind.

6.65.2.59 #define DDSC_LOG_RECORD_INSERT_EC (DDSC_LOG_BASE + 111)

Failed to insert a database record of the specified kind.

6.65.2.60 #define DDSC_LOG_RECORD_ERROR_EC (DDSC_LOG_BASE + 112)

Unknown error for database record of the specified kind.

6.65.2.61 #define DDSC_LOG_RECORD_EXISTS_EC (DDSC_LOG_BASE + 113)

A database record of the specified kind already exists.

6.65.2.62 `#define DDSC_LOG_RECORD_LOOKUP_EC (DDSC_LOG_BASE + 114)`

A lookup of a database record of the specified kind failed.

6.65.2.63 `#define DDSC_LOG_RECORD_NOT_EXISTS_EC (DDSC_LOG_BASE + 115)`

A database record of the specified kind does not exist.

6.65.2.64 `#define DDSC_LOG_RECORD_SELECT_EC (DDSC_LOG_BASE + 116)`

A database select on the specified record kind failed.

6.65.2.65 `#define DDSC_LOG_RECORD_REMOVE_EC (DDSC_LOG_BASE + 117)`

Removal a database record of the specified kind failed.

6.65.2.66 `#define DDSC_LOG_RECORD_INITIALIZE_EC (DDSC_LOG_BASE + 118)`

A database record of the specified kind could not be initialized.

6.65.2.67 `#define DDSC_LOG_RECORD_FINALIZE_EC (DDSC_LOG_BASE + 119)`

A database record of the specified kind could not be finalized.

6.65.2.68 `#define DDSC_LOG_PUBLICATIONDATA_OBJECT 1`

DDS PublicationBuiltinTopicData object.

6.65.2.69 `#define DDSC_LOG_SUBSCRIPTIONDATA_OBJECT 2`

DDS SubscriptionBuiltinTopicData object.

6.65.2.70 `#define DDSC_LOG_PARTICIPANTDATA_OBJECT 3`

DDS ParticipantBuiltinTopicData object.

6.65.2.71 `#define DDSC_LOG_DATAREADERQOS_OBJECT 4`

DDS DataReaderQos object.

6.65.2.72 `#define DDSC_LOG_DATAWRITERQOS_OBJECT 5`

DDS DataWriterQos object.

6.65.2.73 `#define DDSC_LOG_TOPICQOS_OBJECT 6`

DDS TopicQos object.

6.65.2.74 `#define DDSC_LOG_PUBLISHERQOS_OBJECT 7`

DDS PublisherQos object.

6.65.2.75 `#define DDSC_LOG_SUBSCRIBERQOS_OBJECT 8`

DDS SubscriberQos object.

6.65.2.76 `#define DDSC_LOG_PARTICIPANTQOS_OBJECT 9`

DDS DomainParticipantQos object.

6.65.2.77 **#define DDSC_LOG_ENTITY_OBJECT 10**

DDS Entity object.

6.65.2.78 **#define DDSC_LOG_PARTICIPANTFACTORYQOS_OBJECT 11**

DDS DomainParticipantFactoryQos object.

6.65.2.79 **#define DDSC_LOG_TYPE_OBJECT 12**

A DDS Type object.

6.65.2.80 **#define DDSC_LOG_BINDRESOLVER_OBJECT 13**

A NETIO BindResolver object.

6.65.2.81 **#define DDSC_LOG_ROUTERESOLVER_OBJECT 14**

A NETIO RouteResolver object.

6.65.2.82 **#define DDSC_LOG_ADDRESSRESOLVER_OBJECT 15**

A NETIO AddressResolver object.

6.65.2.83 **#define DDSC_LOG_DATAWRITERIO_OBJECT 16**

A NETIO DataWriterInterface object.

6.65.2.84 **#define DDSC_LOG_DATAREADERIO_OBJECT 17**

A NETIO DataReaderInterface object.

6.65.2.85 **#define DDSC_LOG_LOG_OBJECT 18**

A OSAPI Log object.

6.65.2.86 **#define DDSC_LOG_SYSTEM_OBJECT 19**

A OSAPI system object.

6.65.2.87 **#define DDSC_LOG_MUTEX_OBJECT 20**

A OSAPI mutex object.

6.65.2.88 **#define DDSC_LOG_CONDREF_OBJECT 21**

A DDS Waitset condition reference object.

6.65.2.89 **#define DDSC_LOG_WSREF_OBJECT 22**

A DDS Condition waitset reference object.

6.65.2.90 **#define DDSC_LOG_MD5STREAM_OBJECT 23**

A MD5 stream object.

6.65.2.91 **#define DDSC_LOG_CONDITION_OBJECT 24**

A DDS Condition object.

6.65.2.92 #define DDSC_LOG_RT_OBJECT 25

A RT Condition object.

6.65.2.93 #define DDSC_LOG_PARTICIPANT_POOL_OBJECT 26

A Participant pool object.

6.65.2.94 #define DDSC_LOG_TIMER_OBJECT 27

A OSAPI_Timer object.

6.65.2.95 #define DDSC_LOG_TIMEROUT_OBJECT 28

A OSAPI_Timer timeout object.

6.65.2.96 #define DDSC_LOG_TOPIC_OBJECT 29

A DDS Topic object.

6.65.2.97 #define DDSC_LOG_WAITSET_OBJECT 30

A DDS WaitSet object.

6.65.2.98 #define DDSC_LOG_UUID_OBJECT 31

A UUID object.

6.65.2.99 #define DDSC_LOG_STRING_OBJECT 40

A String object.

6.65.2.100 #define DDSC_LOG_STRINGMANAGER_OBJECT 43

A String Manager object.

6.65.2.101 #define DDSC_LOG_OBJECT_INITIALIZE_EC (DDSC_LOG_BASE + 200)

Out of resources to initialize object of the specified kind.

6.65.2.102 #define DDSC_LOG_OBJECT_ALLOCATE_EC (DDSC_LOG_BASE + 201)

Out of resources to allocate an object of the specified kind.

6.65.2.103 #define DDSC_LOG_OBJECT_FINALIZE_EC (DDSC_LOG_BASE + 202)

Failed to finalize object of specified kind.

6.65.2.104 #define DDSC_LOG_OBJECT_DELETE_EC (DDSC_LOG_BASE + 203)

Failed to delete object of specified kind.

6.65.2.105 #define DDSC_LOG_OBJECT_COPY_EC (DDSC_LOG_BASE + 204)

Failed to copy object of specified kind.

6.65.2.106 #define DDSC_LOG_OBJECT_REFCOUNT_EC (DDSC_LOG_BASE + 205)

Failed to delete/finalize an object because other objects are referencing it.

6.65.2.107 `#define DDSC_LOG_OBJECT_GET_PROPERTY_EC (DDSC_LOG_BASE + 206)`

Failed to get the object properties.

6.65.2.108 `#define DDSC_LOG_OBJECT_SET_PROPERTY_EC (DDSC_LOG_BASE + 207)`

Failed to set the object properties.

6.65.2.109 `#define DDSC_LOG_OBJECT_EMPTY_EC (DDSC_LOG_BASE + 208)`

An object is empty, typically applies only to buffer-pool objects.

6.65.2.110 `#define DDSC_LOG_OBJECT_INUSECOUNT_EC (DDSC_LOG_BASE + 209)`

Failed to delete/finalize an object because other objects are using it.

6.65.2.111 `#define DDSC_LOG_NETMASK_SEQUENCE 1`

A NETIO Netmask sequence.

6.65.2.112 `#define DDSC_LOG_ROUTE_SEQUENCE 2`

A NETIO Route sequence.

6.65.2.113 `#define DDSC_LOG_RESERVED_SEQUENCE 3`

A NETIO Reserved address sequence.

6.65.2.114 `#define DDSC_LOG_ENABLED_USER_TRANSPORT_SEQUENCE 4`

A DDS sequence of enabled user NETIO transports.

6.65.2.115 `#define DDSC_LOG_ENABLED_TRANSPORT_SEQUENCE 5`

A DDS sequence of enabled transports.

6.65.2.116 `#define DDSC_LOG_ENABLED_DISCOVERY_TRANSPORT_SEQUENCE 6`

A DDS sequence of enabled discovery transports.

6.65.2.117 `#define DDSC_LOG_INITIAL_PEER_SEQUENCE 7`

A DDS sequence of initial peers.

6.65.2.118 `#define DDSC_LOG_DESTINATION_SEQUENCE 8`

A NETIO sequence of destinations to send to.

6.65.2.119 `#define DDSC_LOG_METAUNICAST_SEQUENCE 9`

A DDS sequence of meta-traffic unicast locators.

6.65.2.120 `#define DDSC_LOG_METAMULTICAST_SEQUENCE 10`

A DDS sequence of meta-traffic multicast locators.

6.65.2.121 `#define DDSC_LOG_USERUNICAST_SEQUENCE 11`

A DDS sequence of user-traffic unicast locators.

6.65.2.122 #define DDSC_LOG_USERMULTICAST_SEQUENCE 12

A DDS sequence of user-traffic multicast locators.

6.65.2.123 #define DDSC_LOG_READTAKE_SEQUENCE 13

A DDS sequence used in a read/take call.

6.65.2.124 #define DDSC_LOG_SEQUENCE_SETMAX_EC (DDSC_LOG_BASE + 300)

Failed to set the maximum length of a sequence of the specified kind.

6.65.2.125 #define DDSC_LOG_SEQUENCE_SETLENGTH_EC (DDSC_LOG_BASE + 301)

Failed to set the length of a sequence of the specified kind.

6.65.2.126 #define DDSC_LOG_SEQUENCE_GETREF_EC (DDSC_LOG_BASE + 302)

Failed to get a reference at the specified index for a sequence of the specified kind.

6.65.2.127 #define DDSC_LOG_SEQUENCE_INITIALIZE_EC (DDSC_LOG_BASE + 303)

Failed to initialize a sequence of the specified kind.

6.65.2.128 #define DDSC_LOG_SEQUENCE_FINALIZE_EC (DDSC_LOG_BASE + 304)

Failed to finalize a sequence of the specified kind.

6.65.2.129 #define DDSC_LOG_SEQUENCE_COPY_EC (DDSC_LOG_BASE + 305)

Failed to copy a sequence of the specified kind.

6.65.2.130 #define DDSC_LOG_SEQUENCE_INVALID_EC (DDSC_LOG_BASE + 306)

The sequence of the specified kind was invalid in the context it is used.

6.65.2.131 #define DDSC_LOG_DISCOVERY_COMPONENT 1

A component of discovery kind.

6.65.2.132 #define DDSC_LOG_RTPS_COMPONENT 2

A component of RTPS kind.

6.65.2.133 #define DDSC_LOG_READERHISTORY_COMPONENT 3

A component of reader-history kind.

6.65.2.134 #define DDSC_LOG_WRITERHISTORY_COMPONENT 4

A component of writer-history kind.

6.65.2.135 #define DDSC_LOG_DATAREADERIO_COMPONENT 5

A component of DataReaderInterface kind.

6.65.2.136 #define DDSC_LOG_DATAWRITERIO_COMPONENT 6

A component of DataWriterInterface kind.

6.65.2.137 #define DDSC_LOG_NETIO_COMPONENT 7

A component of NETIO kind.

6.65.2.138 #define DDSC_LOG_TRANSPORT_COMPONENT 8

A component of transport kind.

6.65.2.139 #define DDSC_LOG_COMPONENT_LOOKUP_EC (DDSC_LOG_BASE + 400)

Did not find a component factory with the given name in the registry.

6.65.2.140 #define DDSC_LOG_COMPONENT_CREATE_EC (DDSC_LOG_BASE + 401)

Could not create a component of the specified kind using the specified factory.

6.65.2.141 #define DDSC_LOG_COMPONENT_DELETE_EC (DDSC_LOG_BASE + 402)

Could not delete a component of the specified kind using the specified factory.

6.65.2.142 #define DDSC_LOG_HISTORY_RESOURCE 1

Exceeded resource limits for writer history queue.

A DataWriter failed to get a queue entry for a new sample it is attempting to send, due to exceeding a limit:

- When the sample is for new instance: DataWriterQos.resource_limits.max_instances may have been exceeded
- When the sample is for an existing instance, and History kind is KEEP_ALL: DataWriterQos.resource_limits.max_samples_per_instance may have been exceeded.
- The limit on the total number of samples in the queue: DataWriterQos.resource_limits.max_samples, may have been exceeded.

6.65.2.143 #define DDSC_LOG_SAMPLE_RESOURCES 2

Failed to get resource for new sample due to resource limit.

A DataWriter failed to get a buffer for a new sample being written because the limit DataWriterQos.resource_limits.max_samples was exceeded.

A DataReader failed in getting a buffer for a newly received sample because DataReaderQos.resource_limits.max_samples was exceeded.

6.65.2.144 #define DDSC_LOG_PARTICIPANT_RESOURCES 3

Failed to allocate a new participant.

The DomainParticipantFactory failed to get a buffer for a new DomainParticipant because DomainParticipantFactoryQos.system_resource.max_participants because was exceeded.

6.65.2.145 #define DDSC_LOG_RESOURCE_EXCEEDED_EC (DDSC_LOG_BASE + 500)

Could not allocate a resource of the specified kind.

6.65.2.146 #define DDSC_LOG_TOPIC_QOS 1

The TopicQos kind.

6.65.2.147 #define DDSC_LOG_DATAREADER_QOS 2

The DataReaderQos kind.

6.65.2.148 #define DDSC_LOG_DATAWRITER_QOS 3

The DataWriterQos kind.

6.65.2.149 #define DDSC_LOG_SUBSCRIBER_QOS 4

The SubscriberQos kind.

6.65.2.150 #define DDSC_LOG_PUBLISHER_QOS 5

The PublisherQos kind.

6.65.2.151 #define DDSC_LOG_PARTICIPANT_QOS 6

The DomainParticipantQos kind.

6.65.2.152 #define DDSC_LOG_PARTICIPANTFACTORY_QOS 7

The DomainParticipantFactoryQos kind.

6.65.2.153 #define DDSC_LOG_DEFAULTTOPIC_QOS 8

The default TopicQos kind.

6.65.2.154 #define DDSC_LOG_DEFAULTDATAREADER_QOS 9

The default DataReaderQos kind.

6.65.2.155 #define DDSC_LOG_DEFAULTDATAWRITER_QOS 10

The default DataWriterQos kind.

6.65.2.156 #define DDSC_LOG_DEFAULTSUBSCRIBER_QOS 11

The default SubscriberQos kind.

6.65.2.157 #define DDSC_LOG_DEFAULTPUBLISHER_QOS 12

The default PublisherQos kind.

6.65.2.158 #define DDSC_LOG_DEFAULTPARTICIPANT_QOS 13

The default PublisherQos kind.

6.65.2.159 #define DDSC_LOG_DEADLINE_QOS_POLICY 14

The deadline qos policy kind.

6.65.2.160 #define DDSC_LOG_LIVELINESS_QOS_POLICY 15

The liveliness qos policy kind.

6.65.2.161 #define DDSC_LOG_HISTORY_QOS_POLICY 16

The history qos policy kind.

6.65.2.162 #define DDSC_LOG_RESOURCE_LIMIT_QOS_POLICY 17

The resource limits qos policy kind.

6.65.2.163 `#define DDSC_LOG_PROTOCOL_QOS_POLICY 18`

The protocol qos policy kind.

6.65.2.164 `#define DDSC_LOG_TYPE_SUPPORT_QOS_POLICY 19`

The type-support qos policy kind.

6.65.2.165 `#define DDSC_LOG_RELIABILITY_QOS_POLICY 20`

The reliability qos policy kind.

6.65.2.166 `#define DDSC_LOG_DURABILITY_QOS_POLICY 21`

The durability qos policy kind.

6.65.2.167 `#define DDSC_LOG_OWNERSHIP_QOS_POLICY 22`

The ownership qos policy kind.

6.65.2.168 `#define DDSC_LOG_OWNERSHIP_STRENGTH_QOS_POLICY 23`

The ownership-strength qos policy kind.

6.65.2.169 `#define DDSC_LOG_TRANSPORT_QOS_POLICY 24`

The transport qos policy kind.

6.65.2.170 `#define DDSC_LOG_PARTICIPANT_ID_QOS_POLICY 25`

The participant id qos policy kind.

6.65.2.171 `#define DDSC_LOG_HEARTBEATS_QOS_POLICY 26`

The heartbeat qos policy kind.

When configured for reliable communication, `heartbeats_per_max_samples` must be fit within `max_samples`

6.65.2.172 `#define DDSC_LOG_DATAWRITER_RESOURCE_QOS_POLICY 27`

The DataWriterQos `writer_resource_limits` policy.

An invalid value has been set for a limit of DataWriterQos.`writer_resource_limits`. Each value must be positive and finite. May be logged by Discovery writers with no `initial_peers` set.

6.65.2.173 `#define DDSC_LOG_DATAREADER_RESOURCE_QOS_POLICY 28`

The DataReaderQos `reader_resource_limits` policy.

6.65.2.174 `#define DDSC_LOG_DESTINATION_ORDER_POLICY 29`

The DestinationOrderQos `destination_order` policy.

6.65.2.175 `#define DDSC_LOG_USER_TRAFFIC_QOS_POLICY 30`

The UserTraffic policy.

6.65.2.176 `#define DDSC_LOG_DISCOVERY_TRAFFIC_QOS_POLICY 31`

The Discovery.`enabled_transport` policy.

6.65.2.177 `#define DDSC_LOG_QOS_INCONSISTENT_POLICY_EC (DDSC_LOG_BASE + 501)`

An inconsistent Qos policy for the specified Qos kind was found.

6.65.2.178 `#define DDSC_LOG_QOS_INCONSISTENT_POLICIES_EC (DDSC_LOG_BASE + 502)`

Inconsistency between two Qos policies for the specified Qos kind was found.

6.65.2.179 `#define DDSC_LOG_QOS_INCONSISTENT_EC (DDSC_LOG_BASE + 503)`

Failed to create an entity or set a qos due to inconsistent policy.

6.65.2.180 `#define DDSC_LOG_QOS_COPY_EC (DDSC_LOG_BASE + 504)`

Failed to copy a Qos of the specified kind.

6.65.2.181 `#define DDSC_LOG_QOS_INITIALIZE_EC (DDSC_LOG_BASE + 505)`

Failed to initialize a Qos of the specified kind.

6.65.2.182 `#define DDSC_LOG_QOS_FINALIZE_EC (DDSC_LOG_BASE + 506)`

Failed to finalize a Qos of the specified kind.

6.65.2.183 `#define DDSC_LOG_QOS_SET_EC (DDSC_LOG_BASE + 507)`

Failed to set a Qos of the specified kind.

6.65.2.184 `#define DDSC_LOG_QOS_SET_ON_ENABLED_EC (DDSC_LOG_BASE + 508)`

Failed to set a Qos of the specified kind because the entity is already enabled.

6.65.2.185 `#define DDSC_LOG_QOS_IMMUTABLE_EC (DDSC_LOG_BASE + 509)`

Failed to set a Qos of the specified kind the immutable Qos policies have been changed.

6.65.2.186 `#define DDSC_LOG_QOS_CHANGED_EC (DDSC_LOG_BASE + 510)`

A discovered Qos changed (the entity already existed)

6.65.2.187 `#define DDSC_LOG_QOS_GET_EC (DDSC_LOG_BASE + 511)`

Failed to get a Qos of the specified kind.

6.65.2.188 `#define DDSC_LOG_TOPIC_LISTENER 1`

The topic listener kind.

6.65.2.189 `#define DDSC_LOG_DATAREADER_LISTENER 2`

The datareader listener kind.

6.65.2.190 `#define DDSC_LOG_DATAWRITER_LISTENER 3`

The datawriter listener kind.

6.65.2.191 `#define DDSC_LOG_SUBSCRIBER_LISTENER 4`

The subscriber listener kind.

6.65.2.192 `#define DDSC_LOG_PUBLISHER_LISTENER 5`

The publisher listener kind.

6.65.2.193 `#define DDSC_LOG_PARTICIPANT_LISTENER 6`

The participant listener kind.

6.65.2.194 `#define DDSC_LOG_PARTICIPANTFACTORY_LISTENER 7`

The participant-factory listener kind.

6.65.2.195 `#define DDSC_LOG_LISTENER_INCONSISTENT_EC (DDSC_LOG_BASE + 512)`

Failed to create an entity due to inconsistent listener and status mask.

6.65.2.196 `#define DDSC_LOG_LISTENER_SET_EC (DDSC_LOG_BASE + 513)`

Failed to set the listener of the specified kind.

6.65.2.197 `#define DDSC_LOG_LISTENER_GET_EC (DDSC_LOG_BASE + 514)`

Failed to get the listener of the specified kind.

6.65.2.198 `#define DDSC_LOG_LISTENER_SET_ILLEGAL_NULL_EC (DDSC_LOG_BASE + 515)`

Illegal combination of NULL listener and non-NONE status mask when setting a listener for an Entity.

6.65.2.199 `#define DDSC_LOG_TOPIC_ENTITY 1`

The topic entity kind.

6.65.2.200 `#define DDSC_LOG_DATAWRITER_ENTITY 2`

The datawriter entity kind.

6.65.2.201 `#define DDSC_LOG_DATAREADER_ENTITY 3`

The datareader entity kind.

6.65.2.202 `#define DDSC_LOG_PUBLISHER_ENTITY 4`

The publisher entity kind.

6.65.2.203 `#define DDSC_LOG_SUBSCRIBER_ENTITY 5`

The subscriber entity kind.

6.65.2.204 `#define DDSC_LOG_PARTICIPANT_ENTITY 6`

The participant entity kind.

6.65.2.205 `#define DDSC_LOG_PUBLICATION_ENTITY 7`

The publication entity kind.

6.65.2.206 `#define DDSC_LOG_SUBSCRIPTION_ENTITY 8`

The subscription entity kind.

6.65.2.207 `#define DDSC_LOG_ENTITY_ENABLE_EC (DDSC_LOG_BASE + 600)`

Failed to enable an entity of the specified kind.

6.65.2.208 `#define DDSC_LOG_ENTITY_NOT_EMPTY_EC (DDSC_LOG_BASE + 601)`

Failed to an delete/finalize an entity of the specified kind because it is not empty.

6.65.2.209 `#define DDSC_LOG_ENTITY_FINALIZE_EC (DDSC_LOG_BASE + 602)`

Failed to finalize an entity of the specified kind.

6.65.2.210 `#define DDSC_LOG_ENTITY_INITIALIZE_EC (DDSC_LOG_BASE + 603)`

Failed to initialize an entity of the specified kind.

6.65.2.211 `#define DDSC_LOG_ENTITY_NOT_ENABLED_EC (DDSC_LOG_BASE + 604)`

An operation was attempted on an entity that is not enabled.

6.65.2.212 `#define DDSC_LOG_ENTITY_DIFFERENT_FACTORY_EC (DDSC_LOG_BASE + 605)`

Entities are in different factories.

A entity tried to use an entity created by a different factory. For example, it is illegal to create a datawriter using a topic from a different participant.

6.65.2.213 `#define DDSC_LOG_DATAWRITER_CDR 1`

The datawriter CDR kind.

6.65.2.214 `#define DDSC_LOG_DATAREADER_CDR 2`

The datareader CDR kind.

6.65.2.215 `#define DDSC_LOG_CDR_POOL_ALLOC_EC (DDSC_LOG_BASE + 700)`

Failed to allocate a pool of the specified kind.

6.65.2.216 `#define DDSC_LOG_CDR_BUFFER_SET_EC (DDSC_LOG_BASE + 701)`

Failed to set the CDR buffer for a packet.

6.65.2.217 `#define DDSC_LOG_CDR_POOL_DELETE_EC (DDSC_LOG_BASE + 702)`

Failed to delete the CDR pool.

6.65.2.218 `#define DDSC_LOG_CDR_SERIALIZE_PID_EC (DDSC_LOG_BASE + 703)`

Failed to serialize a parameter ID.

6.65.2.219 `#define DDSC_LOG_CDR_SERIALIZE_PID_LENGTH_EC (DDSC_LOG_BASE + 704)`

Failed to serialize a parameter length.

6.65.2.220 `#define DDSC_LOG_CDR_SERIALIZE_KEYHASH_EC (DDSC_LOG_BASE + 705)`

Failed to serialize a key-hash.

6.65.2.221 `#define DDSC_LOG_CDR_SERIALIZE_DATA_EC (DDSC_LOG_BASE + 706)`

Failed to serialize payload data.

6.65.2.222 `#define DDSC_LOG_DESERIALIZE_BAD_PID_LENGTH_EC (DDSC_LOG_BASE + 707)`

Deserialized an invalid parameter length for a specific parameter ID.

6.65.2.223 `#define DDSC_LOG_CDR_DESERIALIZE_PID_EC (DDSC_LOG_BASE + 708)`

Failed to deserialize the ID of an inline parameter.

6.65.2.224 `#define DDSC_LOG_CDR_DESERIALIZE_PID_LENGTH_EC (DDSC_LOG_BASE + 709)`

Failed to deserialize the length of an inline parameter.

6.65.2.225 `#define DDSC_LOG_CDR_INCREMENT_POS_EC (DDSC_LOG_BASE + 710)`

Failed to increment to the position of the next inline parameter.

6.65.2.226 `#define DDSC_LOG_CDR_SET_POS_EC (DDSC_LOG_BASE + 711)`

Failed to set the reception stream position.

6.65.2.227 `#define DDSC_LOG_CDR_DESERIALIZE_HEADER_EC (DDSC_LOG_BASE + 712)`

Failed to deserialize the encapsulation header.

6.65.2.228 `#define DDSC_LOG_CDR_DESERIALIZE_DATA_EC (DDSC_LOG_BASE + 713)`

Failed to deserialize CDR payload data.

6.65.2.229 `#define DDSC_LOG_CDR_INITIALIZE_SAMPLE_EC (DDSC_LOG_BASE + 714)`

Failed to initialize CDR sample.

6.65.2.230 `#define DDSC_LOG_CDR_FINALIZE_SAMPLE_EC (DDSC_LOG_BASE + 715)`

Failed to finalize sample.

6.65.2.231 `#define DDSC_LOG_CDR_SERIALIZE_STATUS_INFO_EC (DDSC_LOG_BASE + 716)`

Failed to serialize the status info parameter.

6.65.2.232 `#define DDSC_LOG_CDR_DESERIALIZE_KEYHASH_EC (DDSC_LOG_BASE + 717)`

Failed to deserialize a key-hash.

6.65.2.233 `#define DDSC_LOG_CDR_DESERIALIZE_KEY_EC (DDSC_LOG_BASE + 718)`

Failed to deserialize CDR payload key.

6.65.2.234 `#define DDSC_LOG_NETIO_ADD_ANON_TOPIC_ROUTE_EC (DDSC_LOG_BASE + 800)`

Failed to add a route to an anonymous participant discovery datawriter.

6.65.2.235 `#define DDSC_LOG_NETIO_ADD_TOPIC_ROUTE_EC (DDSC_LOG_BASE + 801)`

Failed to add a route to a topic from a datawriter.

6.65.2.236 `#define DDSC_LOG_NETIO_DELETE_TOPIC_ROUTE_EC (DDSC_LOG_BASE + 802)`

Failed to delete a route to a topic.

6.65.2.237 `#define DDSC_LOG_NETIO_FORWARD_TOPIC_EC (DDSC_LOG_BASE + 803)`

Failed to forward a topic.

6.65.2.238 `#define DDSC_LOG_NETIO_NETIO_KIND 1`

Any NETIO interface kind, typically UDP.

6.65.2.239 `#define DDSC_LOG_INTRA_NETIO_KIND 2`

The intra interface kind.

6.65.2.240 `#define DDSC_LOG_RTPS_NETIO_KIND 3`

The RTPS interface kind.

6.65.2.241 `#define DDSC_LOG_DATAREADER_NETIO_KIND 4`

The DataReader interface kind.

6.65.2.242 `#define DDSC_LOG_DATAWRITER_NETIO_KIND 5`

The DataWriter interface kind.

6.65.2.243 `#define DDSC_LOG_NETIO_BIND_EXTERNAL_EC (DDSC_LOG_BASE + 804)`

Failed to bind two external interface of the specified kind.

6.65.2.244 `#define DDSC_LOG_NETIO_UNBIND_EXTERNAL_EC (DDSC_LOG_BASE + 805)`

Failed to unbind two external interfaces of the specified kind.

6.65.2.245 `#define DDSC_LOG_NETIO_BIND_EC (DDSC_LOG_BASE + 806)`

Failed to bind an interface to a peer interface.

6.65.2.246 `#define DDSC_LOG_NETIO_UNBIND_EC (DDSC_LOG_BASE + 807)`

Failed to unbind an interface from a peer interface.

6.65.2.247 `#define DDSC_LOG_NETIO_ADD_ROUTE_EC (DDSC_LOG_BASE + 808)`

Failed to add a route from an interface to a peer interface.

6.65.2.248 `#define DDSC_LOG_NETIO_DELETE_ROUTE_EC (DDSC_LOG_BASE + 809)`

Failed to delete a route from an interface to a peer interface.

6.65.2.249 `#define DDSC_LOG_NETIO_GET_EXTERNAL_INTF_EC (DDSC_LOG_BASE + 810)`

Failed to get an external interface for the specified interface kind.

6.65.2.250 `#define DDSC_LOG_NETIO_NO_ROUTE_EC (DDSC_LOG_BASE + 811)`

A DataReader failed a bind due to no existing route.

6.65.2.251 `#define DDSC_LOG_NETIO_ROUTE_LOOKUP_FAILED_EC (DDSC_LOG_BASE + 812)`

Lookup a route to a destination failed.

A failure was encountered when trying to lookup a route to a destination, this log message is preceded by a more specific message. Failure to lookup a route does not mean the route does not exist, it means it failed to determine if a route did exist.

6.65.2.252 `#define DDSC_LOG_NETIO_GET_ROUTE_TABLE_FAILED_EC (DDSC_LOG_BASE + 813)`

Failed to get the route table for an interface.

6.65.2.253 `#define DDSC_LOG_NETIO_SEND_FAILED_EC (DDSC_LOG_BASE + 814)`

Failure when sending on an interface.

6.65.2.254 `#define DDSC_LOG_NETIO_SET_STATE_EC (DDSC_LOG_BASE + 815)`

Failed to set an interface state.

6.65.2.255 `#define DDSC_LOG_NETIO_PEER_LOOKUP_EC (DDSC_LOG_BASE + 816)`

Datawriter did not find a peer.

6.65.2.256 `#define DDSC_LOG_NETIO_FORCED_REMOVE_EC (DDSC_LOG_BASE + 817)`

Forced removal of sample downstream failed.

The datawriter failed to force a sample removal downstream

6.65.2.257 `#define DDSC_LOG_PACKET_INIT_EC (DDSC_LOG_BASE + 818)`

Failed to initialize a packet.

6.65.2.258 `#define DDSC_LOG_PACKET_SET_HEAD_EC (DDSC_LOG_BASE + 819)`

Failed to set the head of a packet.

6.65.2.259 `#define DDSC_LOG_PACKET_SET_TAIL_EC (DDSC_LOG_BASE + 820)`

Failed to set the tail of a packet.

6.65.2.260 `#define DDSC_LOG_DW_ACKNACK_FAILED_EC (DDSC_LOG_BASE + 900)`

Failed to ACKNACK sample in the writer history.

6.65.2.261 `#define DDSC_LOG_DW_COMMIT_EC (DDSC_LOG_BASE + 901)`

Failed to commit a sample to the writer queue.

6.65.2.262 `#define DDSC_LOG_DW_KEYHASH_CREATE_EC (DDSC_LOG_BASE + 902)`

Failed to create keyhash of instance handle.

6.65.2.263 `#define DDSC_LOG_DW_ILLEGAL_KEY_KIND_EC (DDSC_LOG_BASE + 903)`

Failed a write due to an invalid key kind for the type being written.

6.65.2.264 `#define DDSC_LOG_DW_CREATE_TYPED_WRITER_EC (DDSC_LOG_BASE + 904)`

Failed to create a typed writer.

6.65.2.265 `#define DDSC_LOG_DW_HISTORY_REGISTER_KEY_EC (DDSC_LOG_BASE + 905)`

Failed to register the key of an instance.

A DataWriter failed to register the key of a new instance because `DataWriterQos.resource_limits.max_instances` was exceeded.

6.65.2.266 `#define DDSC_LOG_DR_CREATE_TYPED_READER_EC (DDSC_LOG_BASE + 1000)`

Failed to create a typed datareader.

6.65.2.267 `#define DDSC_LOG_DR_COPY_DATA_SAMPLE_EC (DDSC_LOG_BASE + 1001)`

Failed to copy a sample upon reception, read, or take.

6.65.2.268 `#define DDSC_LOG_DR_COMMIT_SAMPLE_EC (DDSC_LOG_BASE + 1002)`

Failed to commit a sample to be made available to be read or taken.

6.65.2.269 `#define DDSC_LOG_DR_FILTER_ERROR_EC (DDSC_LOG_BASE + 1003)`

A datareader filter function failed.

6.65.2.270 `#define DDSC_LOG_DR_DESERIALIZE_KEYHASH_EC (DDSC_LOG_BASE + 1004)`

Failed to deserialize a key-hash parameter.

6.65.2.271 `#define DDSC_LOG_DR_GET_ENTRY_FAILED_EC (DDSC_LOG_BASE + 1005)`

Failed to get a Reader History entry for a received sample.

6.65.2.272 `#define DDSC_LOG_DR_COMMIT_ENTRY_EC (DDSC_LOG_BASE + 1006)`

Failed to commit a receive sample to Reader History to be read or taken.

6.65.2.273 `#define DDSC_LOG_DR_UNREGISTER_KEY_EC (DDSC_LOG_BASE + 1007)`

A DataReader failed to unregister an instance.

6.65.2.274 `#define DDSC_LOG_DR_DISPOSE_KEY_EC (DDSC_LOG_BASE + 1008)`

A DataReader failed to dispose an instance.

6.65.2.275 `#define DDSC_LOG_DR_READ_TAKE_FAILURE_EC (DDSC_LOG_BASE + 1009)`

A call to a reader/take function failed.

6.65.2.276 `#define DDSC_LOG_DR_INSTANCE_TO_KEYHASH_EC (DDSC_LOG_BASE + 1010)`

Failed to create keyhash of instance handle.

6.65.2.277 `#define DDSC_LOG_TYPE_NAME_CMP_EC (DDSC_LOG_BASE + 1100)`

Two type names are incompatible.

6.65.2.278 `#define DDSC_LOG_TOPIC_NAME_CMP_EC (DDSC_LOG_BASE + 1101)`

Two topic names are incompatible.

6.65.2.279 `#define DDSC_LOG_TYPE_FUNCTION_GET_SERIALIZED_SAMPLE_MAX_SIZE 1`

The `get_serialized_sample_max_size` function kind.

6.65.2.280 `#define DDSC_LOG_TYPE_FUNCTION_SERIALIZE_DATA 2`

The `serialize_data` function kind.

6.65.2.281 `#define DDSC_LOG_TYPE_FUNCTION_DESERIALIZE_DATA 3`

The `deserialize_data` function kind.

6.65.2.282 `#define DDSC_LOG_TYPE_FUNCTION_CREATE_SAMPLE 4`

The `create_sample` function kind.

6.65.2.283 `#define DDSC_LOG_TYPE_FUNCTION_COPY_SAMPLE 5`

The `copy_sample` function kind.

6.65.2.284 `#define DDSC_LOG_TYPE_FUNCTION_DELETE_SAMPLE 6`

The `delete_sample` function kind.

6.65.2.285 `#define DDSC_LOG_TYPE_FUNCTION_GET_KEY_KIND 7`

The `get_key_kind` function kind.

6.65.2.286 `#define DDSC_LOG_TYPE_FUNCTION_INSTANCE_TO_KEYHASH 8`

The `instance_to_keyhash` function kind.

6.65.2.287 `#define DDSC_LOG_TYPE_FUNCTION_NULL_EC (DDSC_LOG_BASE + 1102)`

Invalid type plugin, The specified function pointer is NULL.

6.65.2.288 `#define DDSC_LOG_TOPIC_TOO_LONG_EC (DDSC_LOG_BASE + 1103)`

Failed to create a topic because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)

6.65.2.289 `#define DDSC_LOG_TYPE_TOO_LONG_EC (DDSC_LOG_BASE + 1104)`

Failed to create a type because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)

6.65.2.290 `#define DDSC_LOG_LOOKUP_TYPE_PLUGIN_EC (DDSC_LOG_BASE + 1105)`

A type-plugin for the given type could not be found.

6.65.2.291 `#define DDSC_LOG_TYPE_KEY_TYPE_EC (DDSC_LOG_BASE + 1106)`

Two types have incompatible keys.

6.65.2.292 `#define DDSC_LOG_DUPLICATE_GUID_PREFIX_EC (DDSC_LOG_BASE + 1107)`

A DomainParticipant with the specified GUID already exists in the DomainParticipant factory.

A DomainParticipant with the specified GUID already exists in the domain in the DomainParticipant factory. The GUID is either generated (default) or specified manually in the DomainParticipantQos.protocol policy. This error typically indicates that the DomainParticipantQos.protocol is manually specified and is a duplicate.

6.65.2.293 `#define DDSC_LOG_DISC_LOCAL_PARTICIPANT_ENABLED_EC (DDSC_LOG_BASE + 1200)`

Discovery plugin failed its update after a local DomainParticipant was enabled.

6.65.2.294 `#define DDSC_LOG_DISC_BEFORE_LOCAL_PARTICIPANT_CREATED_EC (DDSC_LOG_BASE + 1201)`

Discovery plugin failed its update before a local DomainParticipant was created.

6.65.2.295 `#define DDSC_LOG_DISC_AFTER_LOCAL_PARTICIPANT_CREATED_EC (DDSC_LOG_BASE + 1202)`

Discovery plugin failed its update after a local DomainParticipant was created.

6.65.2.296 `#define DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_ENABLED_EC (DDSC_LOG_BASE + 1203)`

Discovery plugin failed its update after a local DataReader was enabled.

6.65.2.297 `#define DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_DELETED_EC (DDSC_LOG_BASE + 1204)`

Discovery plugin failed its update after a local DataReader was deleted.

6.65.2.298 `#define DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_ENABLED_EC (DDSC_LOG_BASE + 1205)`

Discovery plugin failed its update after a local DataWriter was enabled.

6.65.2.299 `#define DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_DELETED_EC (DDSC_LOG_BASE + 1206)`

Discovery plugin failed its update after a local DataWriter was deleted.

6.65.2.300 `#define DDSC_LOG_DISC_REMOTE_PARTICIPANT_EXPIRED_EC (DDSC_LOG_BASE + 1207)`

Discovery plugin failed its update after a remote DomainParticipant's liveliness expired.

6.65.2.301 `#define DDSC_LOG_DISC_ADD_PEER_EC (DDSC_LOG_BASE + 1208)`

Failed to add a peer with discovery plugin.

6.65.2.302 `#define DDSC_LOG_CHECKSUM_REQUIRED_EC (DDSC_LOG_BASE + 1301)`

A participant requires checksum, but the discovered participant does not send a checksum.

6.65.2.303 `#define DDSC_LOG_CHECKSUM_INCOMPATIBLE_EC (DDSC_LOG_BASE + 1302)`

A participant does not support checksum's sent by a discovered participant.

6.65.2.304 `#define DDSC_LOG_CHECKSUM_INCONSISTENT_CLASS_EC (DDSC_LOG_BASE + 1303)`

The checksum class id for a custom checksum does match.

6.65.2.305 `#define DDSC_LOG_CHECKSUM_INCONSISTENT_COMPUTE_EC (DDSC_LOG_BASE + 1304)`

The computed_crc_kind is inconsistent with what is allowed or supported.

6.65.2.306 `#define DDSC_LOG_CHECKSUM_INCONSISTENT_ALLOW_EC (DDSC_LOG_BASE + 1305)`

The allowed_crc_mask is inconsistent with what is supported.

6.66 Sequence Support

Defines sequence interface and primitive data types sequences.

Classes

- struct [FooSeq](#)

<<interface>> <<generic>> <<cert>> A type-safe, ordered collection of elements. The type of these elements is referred to in this documentation as "Foo".

6.66.1 Detailed Description

Defines sequence interface and primitive data types sequences.

6.67 String Support

String creation, cloning, assignment, and deletion.

Functions

- `char * DDS_String_alloc (DDS_UnsignedLong length)`
Create a new empty string that can hold up to `length` characters.
- `char * DDS_String_dup (const char *str)`
Clone a string. Creates a new string that duplicates the value of `string`.
- `void DDS_String_free (char *str)`
Delete a string.
- `int DDS_String_cmp (const char *s1, const char *s2)`
Compare two strings.
- `int DDS_String_ncmp (const char *s1, const char *s2, DDS_UnsignedLong num)`
Compare two strings.
- `int DDS_String_length (const char *string)`
Return the length of a string.
- `int DDS_Wstring_compare (const DDS_Wstring *s1, const DDS_Wstring *s2)`
Return the length of a string.

6.67.1 Detailed Description

String creation, cloning, assignment, and deletion. The methods in this class ensure consistent cross-platform implementations for string creation (`DDS_String_alloc()`), deletion (`DDS_String_free()`), and cloning (`DDS_String_dup()`) that preserve the mutable value type semantics. These are to be viewed as methods that define a `string` class whose data is represented by a `'char*'`.

The following conventions govern the memory management of strings in RTI Connex Micro:

- The DDS implementation ensures that when value types containing strings are passed back and forth to the DDS APIs, the strings are created/deleted/assigned/cloned using the `string` class methods.
- Value types containing strings have ownership of the contained string. Thus, when a value type is deleted, the contained string field is also deleted.
- The user must ensure that when value types containing strings are passed back and forth to the DDS APIs, the strings are created/deleted/assigned/cloned using the String class methods.

The representation of a string in C/C++ unfortunately does not allow programs to detect how much memory has been allocated for a string. RTI Connex Micro must therefore make some assumptions when a user requests that a string be copied into. The following rules apply when RTI Connex Micro is copying into a string or string sequence:

- If the `'char*'` is NULL, RTI Connex Micro will log a warning and allocate a new string on behalf of the user. To avoid leaking memory, you must ensure that the string will be freed (see below).
- If the `'char*'` is not NULL, RTI Connex Micro will assume that you are managing the string's memory yourself and have allocated enough memory to store the string to be copied. RTI Connex Micro will copy into your memory; to avoid memory corruption, be sure to allocate enough of it. Also, do not pass structures containing junk pointers into RTI Connex Micro; you are likely to crash.

This requirement can generally be assured by adhering to the following *idiom* for manipulating strings:

- Always use `DDS_String_alloc()` to create a string

- Always use [DDS_String_dup\(\)](#) to clone a string
- Always use [DDS_String_free\(\)](#) to delete a string 'char*' that is passed back and forth between user code and the DDS C/C++ APIs

Not adhering to this idiom can result in bad pointers, and incorrect memory being freed.

In addition, the user code should be vigilant to avoid memory leaks. It is good practice to:

- Balance occurrences of [DDS_String_alloc\(\)](#), [DDS_String_dup\(\)](#), with matching occurrences of [DDS_String_free\(\)](#) in the code.
- Finalize value types containing strings. In C++ the destructor accomplishes this automatically. in C, explicit "destructor" functions are provided; these functions are typically called "finalize."

6.67.2 Function Documentation

6.67.2.1 char* DDS_String_alloc (DDS_UnsignedLong length)

Create a new empty string that can hold up to `length` characters.

A string created by this method must be deleted using [DDS_String_free\(\)](#).

This function will allocate enough memory to hold a string of `length` characters, **plus** one additional byte to hold the NULL terminating character.

Parameters

<i>length</i>	<<in>> Capacity of the string.
---------------	--------------------------------

Returns

A newly created non-NULL string upon success or NULL upon failure.

MT Safety:

This operation is thread safe.

API Restriction:

This function *must* only be called after [DDSDomainParticipantFactory::get_instance](#).

6.67.2.2 char* DDS_String_dup (const char * str)

Clone a string. Creates a new string that duplicates the value of `string`.

A string created by this method must be deleted using [DDS_String_free\(\)](#)

Parameters

<i>str</i>	<<in>> The string to duplicate.
------------	---------------------------------

Returns

If `string == NULL`, this method always returns NULL. Otherwise, upon success it returns a newly created string whose value is `string`; upon failure it returns NULL.

MT Safety:

UNSAFE. This operation is not thread safe. This operation does not protect the source or destination from being modified by another thread while the source is being copied to the destination.

API Restriction:

This function *must* only be called after [DDSDomainParticipantFactory::get_instance](#).

6.67.2.3 void DDS_String_free (char * *str*)

Delete a string.

Precondition

string must be either NULL, or must have been created using [DDS_String_alloc\(\)](#), [DDS_String_dup\(\)](#)

Parameters

<i>str</i>	<< <i>in</i> >> The string to delete.
------------	---------------------------------------

6.67.2.4 int DDS_String_cmp (const char * *s1*, const char * *s2*)

Compare two strings.

Precondition

s1 and *s2* can be NULL or non-NULL.

Parameters

<i>s1</i>	<< <i>in</i> >> String 1 to compare.
<i>s2</i>	<< <i>in</i> >> String 2 to compare.

Returns

0 if *s1* equals *s2*, positive integer if *s1* is greater than *s2*, negative integer if *s1* is less than *s2*

MT Safety:

UNSAFE. This operation does not protect against the left or right side from being modified by another thread while the comparison is made.

6.67.2.5 int DDS_String_ncmp (const char * *s1*, const char * *s2*, DDS_UnsignedLong *num*)

Compare two strings.

This function compares two strings for lexicographic equivalence, but only up to *num* bytes.

Parameters

<i>in</i>	<i>s1</i>	<< <i>in</i> >> Left side of comparison
<i>in</i>	<i>s2</i>	<< <i>in</i> >> Right side of comparison
<i>in</i>	<i>num</i>	Maximum number of bytes to compare

Returns

0 if *s1* equals *s2*, positive integer if *s1* is greater than *s2*, negative integer if *s1* is less than *s2*

MT Safety:

UNSAFE. This operation does not protect against the left or right side from being modified by another thread while the comparison is made.

6.67.2.6 `int DDS_String_length ( const char * string )`

Return the length of a string.

Precondition

`string` must be non-NULL

Parameters

<i>string</i>	<< <i>in</i> >> String to return length of
---------------	--

Returns

length of string. Does not include the NUL-termination character. If a NULL value is passed as argument, RTI_SIZE_INVALID will be returned.

MT Safety:

This operation is not thread safe.

6.67.2.7 `int DDS_Wstring_compare ( const DDS_Wstring * s1, const DDS_Wstring * s2 )`

Return the length of a string.

Precondition

`s1` and `s2` can be NULL or non-NULL.

Parameters

<i>s1</i>	<< <i>in</i> >> String 1 to compare.
<i>s2</i>	<< <i>in</i> >> String 2 to compare.

Returns

- 0 if `*s1` is lexicographically equal to `*s2`
- a value larger than 0 if `*s1` is lexicographically larger than to `*s2`
- a value less than 0 if `*s1` is lexicographically less than to `*s2`
- 0 if `s1` or `s2` is equal to 'nil'
- 0 if `*s1` and `*s2` is equal to 'nil'
- a value less than 0 if `*s1` is 'nil'
- a value larger than 0 if `*s2` is 'nil'

MT Safety:

UNSAFE. This operation does not protect against the left or right side from being modified by another thread while the comparison is made.

6.68 REDA

Real-time Efficient Data Structures and Algorithms. ModuleID = 1.

Macros

- #define REDA_LOG_BUFFERPOOL_OUT_OF_RESOURCES_EC (REDA_LOG_BASE + 1)
Not sufficient memory to allocate buffer-pool.
- #define REDA_LOG_BUFFERPOOL_BUFFER_INITIALIZATION_FAILED_EC (REDA_LOG_BASE + 2)
The buffer initialization method failed.
- #define REDA_LOG_BUFFERPOOL_NOT_EMPTY_EC (REDA_LOG_BASE + 3)
Cannot delete buffer-pool due to buffer(s) not returned to pool.
- #define REDA_LOG_BUFFERPOOL_DOUBLE_FREE_EC (REDA_LOG_BASE + 4)
A buffer was freed more than once.
- #define REDA_LOG_BUFFERPOOL_NULL_POINTER_EC (REDA_LOG_BASE + 5)
A pointer in a pool's linked list was NULL.
- #define REDA_LOG_SEQUENCE_COPY_FAILED_EC (REDA_LOG_BASE + 100)
An error occurred while copying a sequence.
- #define REDA_LOG_SEQUENCE_ALLOC_FAILED_EC (REDA_LOG_BASE + 101)
An error occurred while allocating space for a sequence.
- #define REDA_LOG_SEQUENCE_SET_MAX_FAILED_EC (REDA_LOG_BASE + 102)
An error occurred while setting the maximum sequence size.
- #define REDA_LOG_SEQUENCE_REALLOCATION_EC (REDA_LOG_BASE + 103)
An attempt was made to resize an already allocated sequence.
- #define REDA_LOG_SEQUENCE_INVALID_OPERATION_EC (REDA_LOG_BASE + 104)
An error occurred while operating on two sequences (copy/compare)
- #define REDA_LOG_SEQUENCE_INVALID_LENGTH_EC (REDA_LOG_BASE + 105)
An attempt was made to set an illegal length (length < 0 or length > max_length)
- #define REDA_LOG_SEQUENCE_INDEX_OUT_OF_BOUNDS_EC (REDA_LOG_BASE + 106)
An illegal sequence index was specified (index < 0 or index > length)
- #define REDA_LOG_STRING_ALLOC_FAILED_EC (REDA_LOG_BASE + 200)
An error occurred while allocating memory for a string.
- #define REDA_LOG_INDEX_FULL_EC (REDA_LOG_BASE + 300)
An attempt was made to add an element to a full index.
- #define REDA_LOG_INDEX_ENTRY_EXISTS_EC (REDA_LOG_BASE + 301)
An attempt was made to add an existing element to an index.
- #define REDA_LOG_INVALID_LIST_NODE_EC (REDA_LOG_BASE + 302)
An attempt was made to add an existing element to an index.

6.68.1 Detailed Description

Real-time Efficient Data Structures and Algorithms. ModuleID = 1.

6.68.2 Macro Definition Documentation

6.68.2.1 #define REDA_LOG_BUFFERPOOL_OUT_OF_RESOURCES_EC (REDA_LOG_BASE + 1)

Not sufficient memory to allocate buffer-pool.

6.68.2.2 #define REDA_LOG_BUFFERPOOL_BUFFER_INITIALIZATION_FAILED_EC (REDA_LOG_BASE + 2)

The buffer initialization method failed.

6.68.2.3 `#define REDA_LOG_BUFFERPOOL_NOT_EMPTY_EC (REDA_LOG_BASE + 3)`

Cannot delete buffer-pool due to buffer(s) not returned to pool.

6.68.2.4 `#define REDA_LOG_BUFFERPOOL_DOUBLE_FREE_EC (REDA_LOG_BASE + 4)`

A buffer was freed more than once.

6.68.2.5 `#define REDA_LOG_BUFFERPOOL_NULL_POINTER_EC (REDA_LOG_BASE + 5)`

A pointer in a pool's linked list was NULL.

6.68.2.6 `#define REDA_LOG_SEQUENCE_COPY_FAILED_EC (REDA_LOG_BASE + 100)`

An error occurred while copying a sequence.

6.68.2.7 `#define REDA_LOG_SEQUENCE_ALLOC_FAILED_EC (REDA_LOG_BASE + 101)`

An error occurred while allocating space for a sequence.

6.68.2.8 `#define REDA_LOG_SEQUENCE_SET_MAX_FAILED_EC (REDA_LOG_BASE + 102)`

An error occurred while setting the maximum sequence size.

6.68.2.9 `#define REDA_LOG_SEQUENCE_REALLOCATION_EC (REDA_LOG_BASE + 103)`

An attempt was made to resize an already allocated sequence.

6.68.2.10 `#define REDA_LOG_SEQUENCE_INVALID_OPERATION_EC (REDA_LOG_BASE + 104)`

An error occurred while operating on two sequences (copy/compare)

6.68.2.11 `#define REDA_LOG_SEQUENCE_INVALID_LENGTH_EC (REDA_LOG_BASE + 105)`

An attempt was made to set an illegal length (length < 0 or length > max_length)

6.68.2.12 `#define REDA_LOG_SEQUENCE_INDEX_OUT_OF_BOUNDS_EC (REDA_LOG_BASE + 106)`

An illegal sequence index was specified (index < 0 or index > length)

6.68.2.13 `#define REDA_LOG_STRING_ALLOC_FAILED_EC (REDA_LOG_BASE + 200)`

An error occurred while allocating memory for a string.

6.68.2.14 `#define REDA_LOG_INDEX_FULL_EC (REDA_LOG_BASE + 300)`

An attempt was made to add an element to a full index.

6.68.2.15 `#define REDA_LOG_INDEX_ENTRY_EXISTS_EC (REDA_LOG_BASE + 301)`

An attempt was made to add an existing element to an index.

6.68.2.16 `#define REDA_LOG_INVALID_LIST_NODE_EC (REDA_LOG_BASE + 302)`

An attempt was made to add an existing element to an index.

6.69 OSAPI Heap

Functions

- void * [OSAPI_Heap_allocate](#) (RTI_SIZE_T count, RTI_SIZE_T size)
Allocates zero-initialized memory from the heap.
- void * [OSAPI_Heap_realloc](#) (void *ptr, RTI_SIZE_T size)
Reallocate memory from the heap.
- void [OSAPI_Heap_free](#) (void *ptr)
Frees memory allocated from the heap.
- void [OSAPI_Heap_allocate_struct](#) (TYPE **pointer, TYPE)
Allocates space on the heap for a C structure and initializes the structure with 0's.
- void [OSAPI_Heap_free_struct](#) (TYPE *pointer)
Returns previously allocated (with [OSAPI_Heap_allocate_struct](#)) space to the heap.
- void [OSAPI_Heap_allocate_array](#) (TYPE **pointer, int count, TYPE)
A static method to allocate space on the heap for an array of C structures; the array is initialized with 0's.
- void [OSAPI_Heap_free_array](#) (TYPE *storage)
A static method to return space (that was previously allocated with [OSAPI_Heap_allocate_array](#)) to the heap.
- void [OSAPI_Heap_allocate_string](#) (char **pointer, RTI_UINT32 size)
A static method to allocate space on the heap for a string of up to a given size. The string is initialized with 0's.
- void [OSAPI_Heap_free_string](#) (char *pointer)
A static method to return space string space to the heap.
- void [OSAPI_Heap_allocate_buffer](#) (char **buffer, RTI_SIZE_T size, OSAPI_Alignment_T alignment)
A static method to allocate a block of memory of the provided size from the heap.
- void [OSAPI_Heap_free_buffer](#) (void *buffer)
A static method to return a block (that was previously allocated with [OSAPI_Heap_allocate_buffer](#)) to the heap.
- RTI_SIZE_T [OSAPI_Heap_get_allocated_byte_count](#) (void)
Return current allocated memory.
- void [OSAPI_Heap_disable_alloc](#) (void)
Disable memory allocation.

Variables

- RTI_SIZE_T [OSAPI_gv_AllocatedByteCount](#)
Current allocated bytes.

6.69.1 Detailed Description

6.69.2 Function Documentation

6.69.2.1 void* OSAPI_Heap_allocate (RTI_SIZE_T count, RTI_SIZE_T size)

Allocates zero-initialized memory from the heap.

The function allocates `count` elements of size `size` and sets the memory region to 0.

NOTE: This operation assumes the specified size to be > 0; this operation is never used directly but it is only accessed by using one of the other allocate operations of OSAPI_Heap (e.g. `allocate_struct`, `allocate_array`, `allocate_string...`); these operations are implemented as macros and accept a "type" argument which is always converted to a size > 0 using the `sizeof` operator.

Parameters

in	<i>count</i>	The number of elements to allocate.
in	<i>size</i>	The size of the element; size is assumed to be always greater than 0.

Returns

. A pointer to a contiguous memory area guaranteed to be large enough store `count` elements of size `size`.

See Also [OSAPI_Heap_free](#)

6.69.2.2 void* OSAPI_Heap_realloc (void * *ptr*, RTI_SIZE_T *size*)

Reallocate memory from the heap.

Parameters

in	<i>ptr</i>	Currently allocated buffer
in	<i>size</i>	The new desired size

Returns

. A pointer to a contiguous memory area guaranteed to be large enough store `size` bytes, NULL if the reallocation failed. See Also [OSAPI_Heap_free](#)

6.69.2.3 void OSAPI_Heap_free (void * *ptr*)

Frees memory allocated from the heap.

The function frees memory previously allocated with [OSAPI_Heap_allocate](#) back to the heap.

Parameters

in	<i>ptr</i>	Pointer to region previous allocated with OSAPI_Heap_allocate
----	------------	---

See Also [OSAPI_Heap_allocate](#)

6.69.2.4 void OSAPI_Heap_allocate_struct (TYPE ** *pointer*, TYPE)

Allocates space on the heap for a C structure and initializes the structure with 0's.

Example:

```
struct MyStruct *myStruct;
OSAPI_Heap_allocate_struct(&myStruct, struct
    MyStruct);
if (myStruct==NULL) {
    return;
}
```

The returned space must be freed with [OSAPI_Heap_free_struct](#). NOTE: This operation is implemented as a macro in order to support the specification of a "type" argument (converted to a size value using the sizeof operator).

6.69.2.5 void OSAPI_Heap_free_struct (TYPE * *pointer*)

Returns previously allocated (with [OSAPI_Heap_allocate_struct](#)) space to the heap.

Parameters

in	<i>pointer</i>	If NULL, no op.
----	----------------	-----------------

Example:

```
char *c;
OSAPI_Heap_allocate_struct(&c, char);
OSAPI_Heap_free_struct(c);
```

6.69.2.6 void OSAPI_Heap_allocate_array (TYPE ** *pointer*, int *count*, TYPE)

A static method to allocate space on the heap for an array of C structures; the array is initialized with 0's.

Example:

```
struct MyElement *array;
OSAPI_Heap_allocate_struct(&array, 7, struct
    MyElement);
if (array==NULL) {
    return;
}
// elements of the array can be accessed as array[i]
```

The returned space must be freed with [OSAPI_Heap_free_array](#). NOTE: This operation is implemented as a macro in order to support the specification of a "type" argument (converted to a size value using the sizeof operator).

6.69.2.7 void OSAPI_Heap_free_array (TYPE * *storage*)

A static method to return space (that was previously allocated with [OSAPI_Heap_allocate_array](#)) to the heap.

Example:

```
char *c;
OSAPI_Heap_allocate_array(&c, 1, char);
OSAPI_Heap_free_array(c);
```

Parameters

in	<i>storage</i>	If NULL, no op.
----	----------------	-----------------

6.69.2.8 void OSAPI_Heap_allocate_string (char ** *pointer*, RTI_UINT32 *size*)

A static method to allocate space on the heap for a string of up to a given size. The string is initialized with 0's.

Example:

```
char* myString;
OSAPI_Heap_allocate_string(&myString, 128);
```

Parameters

out	<i>pointer</i>	String buffer.
in	<i>size</i>	Size in bytes to allocate; this length must not include the string terminator, which is automatically added to the specified value. The returned space must be freed with OSAPI_Heap_free_string .

6.69.2.9 void OSAPI_Heap_free_string (char * *pointer*)

A static method to return space string space to the heap.

Parameters

in	<i>pointer</i>	If NULL, no op.
----	----------------	-----------------

6.69.2.10 void OSAPI_Heap_allocate_buffer (char ** *buffer*, RTI_SIZE_T *size*, OSAPI.Alignment_T *alignment*)

A static method to allocate a block of memory of the provided size from the heap.

Postcondition

The returned address will be a multiple of $2^{\wedge}$ alignment; its memory will be initialized with 0's.

Parameters

out	<i>buffer</i>	
in	<i>size</i>	> 0
in	<i>alignment</i>	Power of 2 and >0 or OSAPI_ALIGNMENT_DEFAULT. If the alignment is OSAPI_ALIGNMENT_DEFAULT, the returned pointer will have "default alignment" (meaning that any C-structure can start at the beginning of the buffer).

The block must be returned to the heap with [OSAPI_Heap_free_buffer](#).

6.69.2.11 void OSAPI_Heap_free_buffer (void * *buffer*)

A static method to return a block (that was previously allocated with [OSAPI_Heap_allocate_buffer](#)) to the heap.

Parameters

in	<i>buffer</i>	If NULL, no op.
----	---------------	-----------------

6.69.2.12 RTI_SIZE_T OSAPI_Heap_get_allocated_byte_count (void)

Return current allocated memory.

MT Safety:

UNSAFE

6.69.2.13 void OSAPI_Heap_disable_alloc (void)

Disable memory allocation.

6.69.3 Variable Documentation

6.69.3.1 RTI_SIZE_T OSAPI_gv_AllocatedByteCount

Current allocated bytes.

6.70 OSAPI Mutex

Mutex API.

Classes

- struct [OSAPI_MutexBase](#)

Typedefs

- typedef struct OSAPI_Mutex [OSAPI_Mutex_T](#)

Functions

- [RTI_BOOL OSAPI_Mutex_delete](#) ([OSAPI_Mutex_T](#) *mutex)
Delete a mutex.
- [OSAPI_Mutex_T * OSAPI_Mutex_new](#) (void)
Create a mutex.
- [RTI_BOOL OSAPI_Mutex_take](#) ([OSAPI_Mutex_T](#) *self)
Take a mutex.
- [RTI_BOOL OSAPI_Mutex_give](#) ([OSAPI_Mutex_T](#) *self)
Give a mutex.

6.70.1 Detailed Description

Mutex API.

6.70.2 Typedef Documentation

6.70.2.1 typedef struct OSAPI_Mutex OSAPI_Mutex_T

Abstract Mutex type

6.70.3 Function Documentation

6.70.3.1 [RTI_BOOL OSAPI_Mutex_delete](#) ([OSAPI_Mutex_T](#) * *mutex*)

Delete a mutex.

Parameters

<i>in</i>	<i>mutex</i>	Delete a mutex created with OSAPI_Mutex_new .
-----------	--------------	---

Returns

RTI_TRUE on success, RTI_FALSE on failure

6.70.3.2 [OSAPI_Mutex_T*](#) [OSAPI_Mutex_new](#) (void)

Create a mutex.

Returns

Pointer to a mutex in a not taken condition. NULL on failure.

See Also [OSAPI_Mutex_delete](#)

6.70.3.3 RTI_BOOL OSAPI_Mutex_take (OSAPI_Mutex_T * *self*)

Take a mutex.

A mutex can only be taken if it is not currently already taken by another thread; it CAN be taken if is it already taken by the same thread. In order to release a mutex, it must be given as many times a it has been taken. Note that `take` will block indefinitely.

Note: the mutex is not required to support priority inversion; there is no protection against deadlocks or starvation.

Parameters

<i>in</i>	<i>self</i>	Take a mutex previously created with OSAPI_Mutex_new .
-----------	-------------	--

Returns

RTI_TRUE on success, RTI_FALSE on failure

See Also [OSAPI_Mutex_give](#), [OSAPI_Mutex_new](#)

6.70.3.4 RTI_BOOL OSAPI_Mutex_give (OSAPI_Mutex_T * *self*)

Give a mutex.

A mutex can only be given if it is owned by the calling thread. Furthermore, it must be given as many times as it has been taken.

Parameters

<i>in</i>	<i>self</i>	Take a mutex previously created with OSAPI_Mutex_new .
-----------	-------------	--

Returns

RTI_TRUE on success, RTI_FALSE on failure

See Also [OSAPI_Mutex_take](#), [OSAPI_Mutex_new](#)

6.71 OSAPI AutoSAR

AutoSAR System API.

Classes

- struct [OSAPI_PortProperty](#)
AutoSAR System properties.

Typedefs

- typedef Std_ReturnType(* [NETIO_Autosar_Tcplp_get_socket](#))(Tcplp_DomainType domain, Tcplp_ProtocolType protocol, Tcplp_SocketIdType *socket)
Function used to create a socket. Users must provide this function in case SocketOwner is used. The implementation of this function normally calls Tcplp_<Up>GetSocket().
- typedef Std_ReturnType(* [NETIO_Autosar_Tcplp_send_dyn_data](#))(void const *data_ptr, uint16 length, Tcplp_SockAddrType const *remote_addr_ptr)
Function used to send data. Users must provide this function only in case SocketOwner is not used. If SocketOwner is used RTI Connex Micro will use a socket to send UDP packets.

Enumerations

- enum [OSAPI_Autosar_SyncKind_T](#) { [OSAPI_AUTOSAR_SYNCKIND_RESOURCES](#) }
Synchronization method enum.

Functions

- boolean [NETIO_Autosar_Tcplp_pdu_callout](#) (PduldType rx_pdu_id, PduInfoType const *pdu_info_ptr)
This callback can be used only when SocketOwner is NOT configured. Callback function to be called on UDP packet reception.
- void [NETIO_Autosar_Tcplp_udp_rx_indication](#) (Tcplp_SocketIdType socket, Tcplp_SockAddrType const *remote_addr_ptr, uint8 *buf_ptr, uint16 length)
Callback function to be called on UDP packet reception.

Variables

- uint32 [OSAPI_PortProperty::log_dst_address](#)
- uint16 [OSAPI_PortProperty::log_dst_port](#)

6.71.1 Detailed Description

AutoSAR System API. The AutoSAR System is defined as specific functionality needed to run RTI Connex Micro on an AutoSAR based OS.

6.71.2 Typedef Documentation

6.71.2.1 typedef Std_ReturnType(* [NETIO_Autosar_Tcplp_get_socket](#))(Tcplp_DomainType domain, Tcplp_ProtocolType protocol, Tcplp_SocketIdType *socket)

Function used to create a socket. Users must provide this function in case SocketOwner is used. The implementation of this function normally calls Tcplp_<Up>GetSocket().

Parameters

in	<i>domain</i>	- IP address family.
in	<i>protocol</i>	- Socket protocol as sub-family of parameter type.
out	<i>socket</i>	- Pointer to socket identifier representing the requested socket. This socket identifier must be provided for all further API calls which requires a SocketId. Note: value of 'socket' parameter is only valid if return value is E_OK

Returns

E_OK on success, error code on failure.

6.71.2.2 `typedef Std_ReturnType(* NETIO_Autosar_Tcplp_send_dyn_data)(void const *data_ptr, uint16 length, Tcplp_SockAddrType const *remote_addr_ptr)`

Function used to send data. Users must provide this function only in case SocketOwner is not used. If SocketOwner is used RTI Connex Micro will use a socket to send UDP packets.

Parameters

in	<i>data_ptr</i>	- Pointer to data to send.
in	<i>length</i>	- Length of data to send.
in	<i>remote_addr_ptr</i>	- Address and port where to send the data.

Returns

E_OK on success, error code on failure.

6.71.3 Enumeration Type Documentation**6.71.3.1 enum OSAPI_Autosar_SyncKind_T**

Synchronization method enum.

Enumerator:

OSAPI_AUTOSAR_SYNCIND_RESOURCES Use Resources as synchronization method. Use spinlock as synchronization method.

6.71.4 Function Documentation**6.71.4.1** `boolean NETIO_Autosar_Tcplp_pdu_callout ( PduldType rx_pdu_id, PdulInfoType const * pdu_info_ptr )`

This callback can be used only when SocketOwner is NOT configured. Callback function to be called on UDP packet reception.

Parameters

in	<i>rx_pdu_id</i>	Rx PDU id.
in	<i>pdu_info_ptr</i>	Pointer to PDU information

Returns

TRUE on success, FALSE on failure.

6.71.4.2 void NETIO_Autosar_Tcplp_udp_rx_indication (Tcplp_SocketIdType *socket*, Tcplp_SockAddrType const * *remote_addr_ptr*, uint8 * *buf_ptr*, uint16 *length*)

Callback function to be called on UDP packet reception.

This callback can be used only when SocketOwner is configured.

Parameters

in	<i>socket</i>	Local socket ID. Use 0 if SocketOwner is not configured.
in	<i>remote_addr_ptr</i>	Pointer to remote address. Can be NULL_PTR if unknown.
in	<i>buf_ptr</i>	Pointer to buffer with incoming data.
in	<i>length</i>	Length of incoming data

6.71.5 Variable Documentation**6.71.5.1 uint32 OSAPI_PortProperty::log_dst_address**

Destination IP address used to send logs to. The format of the address must be compatible with the format needed for function Tcplp_UdpTransmit(), that is in host format instead of network format. I.e. 0xCA00A8C0 would be IP address 192.168.0.202. If 0, logs will not be send to any address. This is the default value.

6.71.5.2 uint16 OSAPI_PortProperty::log_dst_port

Destination IP port used to send logs to. The format of the port must be compatible with the format needed for function Tcplp_UdpTransmit().

6.72 OSAPI Semaphore

Abstract Semaphore API.

Typedefs

- typedef struct OSAPI_Semaphore [OSAPI_Semaphore_T](#)
Abstract semaphore object.

Functions

- [OSAPI_Semaphore_T * OSAPI_Semaphore_new](#) (void)
Create a semaphore.
- [RTI_BOOL OSAPI_Semaphore_delete](#) (OSAPI_Semaphore_T *self)
Delete a semaphore.
- [RTI_BOOL OSAPI_Semaphore_take](#) (OSAPI_Semaphore_T *self, RTI_INT32 timeout_ms, RTI_INT32 *fail_reason)
Take a semaphore.
- [RTI_BOOL OSAPI_Semaphore_give](#) (OSAPI_Semaphore_T *self)
Give a semaphore.

6.72.1 Detailed Description

Abstract Semaphore API.

6.72.2 Typedef Documentation

6.72.2.1 typedef struct OSAPI_Semaphore OSAPI_Semaphore_T

Abstract semaphore object.

6.72.3 Function Documentation

6.72.3.1 OSAPI_Semaphore_T* OSAPI_Semaphore_new (void)

Create a semaphore.

The created semaphore is a binary semaphore with an initial semaphore value set to 0 and a maximum value of 1. Thus, a call to OSAPI_Semaphore_take will block until the semaphore is signaled with a call to OSAPI_Semaphore_give.

Returns

Pointer to a semaphore with a value of 0 on success, NULL on failure.

See Also [OSAPI_Semaphore_delete](#)

6.72.3.2 RTI_BOOL OSAPI_Semaphore_delete (OSAPI_Semaphore_T * self)

Delete a semaphore.

Parameters

in	self	Semaphore created with OSAPI_Semaphore_new .
----	------	--

Returns

RTI_TRUE on success, RTI_FALSE on failure.

See Also [OSAPI_Semaphore_new](#)

6.72.3.3 RTI_BOOL OSAPI_Semaphore.take (OSAPI_Semaphore_T * *self*, RTI_INT32 *timeout_ms*, RTI_INT32 * *fail_reason*)

Take a semaphore.

Parameters

in	<i>self</i>	Semaphore previously created with OSAPI_Semaphore_new .
in	<i>timeout_ms</i>	Timeout in ms. The <code>take</code> call will wait at most <code>timeout_ms</code> before returning. If OSAPI_SEMAPHORE_TIMEOUT_INFINITE is specified, the call will not return until the semaphore has a value of 1 or higher.
out	<i>fail_reason</i>	Additional information when the call returns. If RTI_TRUE is returned, but timed out, <code>fail_reason</code> is set to OSAPI_SEMAPHORE_RESULT_TIMEOUT , otherwise <code>fail_reason</code> is set to OSAPI_SEMAPHORE_RESULT_OK . If RTI_FALSE is returned, <code>fail_reason</code> is set to OSAPI_SEMAPHORE_RESULT_ERROR .

Returns

RTI_TRUE on success, RTI_FALSE on failure.

See Also [OSAPI_Semaphore_give](#)

6.72.3.4 RTI_BOOL OSAPI_Semaphore.give (OSAPI_Semaphore_T * *self*)

Give a semaphore.

Parameters

in	<i>self</i>	Semaphore previously created with OSAPI_Semaphore_new .
----	-------------	---

Returns

RTI_TRUE on success, RTI_FALSE on failure.

See Also [OSAPI_Semaphore_take](#)

6.73 OSAPI System API

Abstract System API.

Classes

- struct `OSAPI_SystemI`
<<cert>> System API.
- struct `OSAPI_SystemProperty`
<<cert>> The System Properties.

Macros

- `#define OSAPI_SYSTEM_MAX_HOSTNAME (64)`
Maximum hostname length.

6.73.1 Detailed Description

Abstract System API. The System is defined as the physical or virtual execution environment and implements functions that are not necessarily provided by the operating system. For example, a custom embedded board may include a special real-time clock which is synchronized via GPS, but is not available via a regular OS system call.

The following functions are not thread-safe:

- `OSAPI_System_set_interface`
- `OSAPI_System_get_property`
- `OSAPI_System_set_property`
- `OSAPI_System_get_native_interface`

6.73.2 Macro Definition Documentation

6.73.2.1 `#define OSAPI_SYSTEM_MAX_HOSTNAME (64)`

Maximum hostname length.

6.74 OSAPI Thread API

OSAPI Thread API.

Classes

- struct [OSAPI_ThreadProperty](#)
Properties for thread creation.

Macros

- #define [OSAPI_THREAD_PRIORITY_LOW](#) -1
Low priority.
- #define [OSAPI_THREAD_PRIORITY_BELOW_NORMAL](#) -2
Below normal priority.
- #define [OSAPI_THREAD_PRIORITY_NORMAL](#) -3
Normal priority.
- #define [OSAPI_THREAD_PRIORITY_ABOVE_NORMAL](#) -4
Above normal priority.
- #define [OSAPI_THREAD_PRIORITY_HIGH](#) -5
High priority.
- #define [OSAPI_THREAD_PRIORITY_INHERIT](#) -6
Use the same priority as the spawning thread. Only applies to POSIX.
- #define [OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE](#) 0
Use the default stack size when creating a thread.
- #define [OSAPI_THREAD_DEFAULT_OPTIONS](#) 0x00
Use only the default options the OS gives you.
- #define [OSAPI_THREAD_FLOATING_POINT](#) 0x01
Hint to the OS to create a thread that supports floating point.
- #define [OSAPI_THREAD_STDIO](#) 0x02
Hint to the OS to create a thread that supports standard I/O.
- #define [OSAPI_THREAD_REALTIME_PRIORITY](#) 0x08
Hint to the OS to create a thread that runs in real-time priority mode.
- #define [OSAPI_THREAD_PROPERTY_DEFAULT](#)
Default thread properties.
- #define [OSAPI_ThreadProperty_INITIALIZER](#) [OSAPI_THREAD_PROPERTY_DEFAULT](#)
Initializer for [OSAPI_ThreadProperty](#).

Typedefs

- typedef [RTI_UINT32](#) [OSAPI_ThreadOptions](#)
Thread options.

6.74.1 Detailed Description

OSAPI Thread API.

6.74.2 Macro Definition Documentation

6.74.2.1 #define OSAPI_THREAD_PRIORITY_LOW -1

Low priority.

6.74.2.2 #define OSAPI_THREAD_PRIORITY_BELOW_NORMAL -2

Below normal priority.

6.74.2.3 #define OSAPI_THREAD_PRIORITY_NORMAL -3

Normal priority.

6.74.2.4 #define OSAPI_THREAD_PRIORITY_ABOVE_NORMAL -4

Above normal priority.

6.74.2.5 #define OSAPI_THREAD_PRIORITY_HIGH -5

High priority.

6.74.2.6 #define OSAPI_THREAD_PRIORITY_INHERIT -6

Use the same priority as the spawning thread. Only applies to POSIX.

6.74.2.7 #define OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE 0

Use the default stack size when creating a thread.

6.74.2.8 #define OSAPI_THREAD_DEFAULT_OPTIONS 0x00

Use only the default options the OS gives you.

6.74.2.9 #define OSAPI_THREAD_FLOATING_POINT 0x01

Hint to the OS to create a thread that supports floating point.

6.74.2.10 #define OSAPI_THREAD_STDIO 0x02

Hint to the OS to create a thread that supports standard I/O.

6.74.2.11 #define OSAPI_THREAD_REALTIME_PRIORITY 0x08

Hint to the OS to create a thread that runs in real-time priority mode.

6.74.2.12 #define OSAPI_THREAD_PROPERTY_DEFAULT

Value:

```
{ \
    OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE, \
    OSAPI_THREAD_PRIORITY_DEFAULT, \
    OSAPI_THREAD_DEFAULT_OPTIONS \
    OSAPI_PORT_THREAD_PROPERTY_INITIALIZER \
}
```

Default thread properties.

6.74.2.13 #define OSAPI_ThreadProperty_INITIALIZER OSAPI_THREAD_PROPERTY_DEFAULT

Initializer for [OSAPI_ThreadProperty](#).

6.74.3 Typedef Documentation**6.74.3.1 typedef RTI_UINT32 OSAPI_ThreadOptions**

Thread options.

6.75 OSAPI Types

OSAPI Types.

Macros

- `#define RTI_INT8`
A 8-bit signed integer.
- `#define RTI_UINT8`
A 8-bit unsigned integer.
- `#define RTI_INT16`
A 16-bit signed integer.
- `#define RTI_UINT16`
A 16-bit unsigned integer.
- `#define RTI_INT32`
A 32-bit signed integer.
- `#define RTI_UINT32`
A 32-bit unsigned integer.
- `#define RTI_INT64`
A 64-bit signed integer.
- `#define RTI_UINT64`
A 64-bit unsigned integer.
- `#define RTI_FALSE ((RTI_BOOL) 0)`
- `#define RTI_TRUE ((RTI_BOOL) 1)`

Typedefs

- `typedef RTI_INT32 RTI_BOOL`
- `typedef RTI_UINT32 RTI_SIZE_T`

6.75.1 Detailed Description

OSAPI Types.

6.75.2 Macro Definition Documentation

6.75.2.1 `#define RTI_INT8`

A 8-bit signed integer.

6.75.2.2 `#define RTI_UINT8`

A 8-bit unsigned integer.

6.75.2.3 `#define RTI_INT16`

A 16-bit signed integer.

6.75.2.4 `#define RTI_UINT16`

A 16-bit unsigned integer.

6.75.2.5 `#define RTI_INT32`

A 32-bit signed integer.

6.75.2.6 `#define RTI_UINT32`

A 32-bit unsigned integer.

6.75.2.7 `#define RTI_INT64`

A 64-bit signed integer.

6.75.2.8 `#define RTI_UINT64`

A 64-bit unsigned integer.

6.75.2.9 `#define RTI_FALSE ((RTI_BOOL) 0)`

Functions that return [RTI_BOOL](#) return `RTI_FALSE` as logical false.

6.75.2.10 `#define RTI_TRUE ((RTI_BOOL) 1)`

Functions that return [RTI_BOOL](#) return `RTI_TRUE` as logical true.

Referenced by `DDSDDataReaderListener::on_before_sample_commit()`, `DDSDomainParticipantListener::on_before_sample_commit()`, `DDSDDataReaderListener::on_before_sample_deserialize()`, and `DDSDomainParticipantListener::on_before_sample_deserialize()`.

6.75.3 Typedef Documentation

6.75.3.1 `typedef RTI_INT32 RTI_BOOL`

The type `RTI_BOOL` uses [RTI_TRUE](#) as logical true and [RTI_FALSE](#) as logical false. In general, a zero value is considered false, and a non-zero value is considered true.

6.75.3.2 `typedef RTI_UINT32 RTI_SIZE_T`

6.76 DB

Database. ModuleID = 2.

Macros

- #define [DB_LOG_SORTED_ALLOC_EC](#) (DB_LOG_BASE + 1)
Not sufficient memory to allocate sorted list for an index.
- #define [DB_LOG_NAME_TOO_LONG_EC](#) (DB_LOG_BASE + 2)
The specified database name is too long.
- #define [DB_LOG_ILLEGAL_TABLE_SIZE_EC](#) (DB_LOG_BASE + 3)
Illegal table size specified.
- #define [DB_LOG_ILLEGAL_LOCK_MODE_EC](#) (DB_LOG_BASE + 4)
Illegal combination of lock mode and mutex given.
- #define [DB_LOG_MUTEX_ALLOC_EC](#) (DB_LOG_BASE + 5)
Failed to allocate database mutex.
- #define [DB_LOG_ALLOC_TABLE_POOL_EC](#) (DB_LOG_BASE + 6)
Failed to allocate buffer pool.
- #define [DB_LOG_TABLES_INUSE_EC](#) (DB_LOG_BASE + 7)
Table is in use.
- #define [DB_LOG_TABLE_NAME_TOO_LONG_EC](#) (DB_LOG_BASE + 8)
Specified table name too long.
- #define [DB_LOG_ILLEGAL_RECORD_COUNT_EC](#) (DB_LOG_BASE + 9)
Illegal record count specified.
- #define [DB_LOG_TABLE_EXISTS_EC](#) (DB_LOG_BASE + 10)
Specified table already exists.
- #define [DB_LOG_OUT_OF_TABLES_EC](#) (DB_LOG_BASE + 11)
Table resources exceeded.
- #define [DB_LOG_OUT_OF_RECORDS_EC](#) (DB_LOG_BASE + 12)
Table record resources exceeded.
- #define [DB_LOG_OUT_OF_INDICES_EC](#) (DB_LOG_BASE + 13)
Table index resources exceeded.
- #define [DB_LOG_OUT_OF_CURSORS_EC](#) (DB_LOG_BASE + 14)
Table cursor resources exceeded.
- #define [DB_LOG_RECORDS_INUSE_EC](#) (DB_LOG_BASE + 15)
Cannot delete table, records are still in table.
- #define [DB_LOG_CURSORS_INUSE_EC](#) (DB_LOG_BASE + 16)
Cannot delete table, cursors are still in use.
- #define [DB_LOG_INDEX_INUSE_EC](#) (DB_LOG_BASE + 17)
Cannot delete table, indices are still in use.
- #define [DB_LOG_ALLOC_CURSOR_POOL_EC](#) (DB_LOG_BASE + 18)
Failed to allocate cursor buffer pool.
- #define [DB_LOG_ALLOC_INDEX_POOL_EC](#) (DB_LOG_BASE + 19)
Failed to allocate index buffer pool.
- #define [DB_LOG_ALLOC_RECORD_POOL_EC](#) (DB_LOG_BASE + 20)
Failed to allocate record buffer pool.
- #define [DB_LOG_ALLOC_DATABASE_EC](#) (DB_LOG_BASE + 21)
Failed to allocate database.
- #define [DB_LOG_TABLE_NOT_INUSE_EC](#) (DB_LOG_BASE + 22)
An attempt was made to delete a table not in use.
- #define [DB_LOG_RECORD_ALREADY_EXISTS_EC](#) (DB_LOG_BASE + 23)

- An attempt was made to insert an already existing record.*
- `#define DB_LOG_RECORD_DOES_NOT_EXIST_EC (DB_LOG_BASE + 24)`
- An attempt was made to delete a record that does not exist.*
- `#define DB_LOG_CURSOR_INVALIDATED_EC (DB_LOG_BASE + 25)`
- An attempt was made to use a cursor that has been invalidated.*
- `#define DB_LOG_LOCK_FAILURE_EC (DB_LOG_BASE + 26)`
- Failed to lock the database.*
- `#define DB_LOG_UNLOCK_FAILURE_EC (DB_LOG_BASE + 27)`
- Failed to unlock the database.*

6.76.1 Detailed Description

Database. ModuleID = 2.

6.76.2 Macro Definition Documentation

6.76.2.1 `#define DB_LOG_SORTED_ALLOC_EC (DB_LOG_BASE + 1)`

Not sufficient memory to allocate sorted list for an index.

6.76.2.2 `#define DB_LOG_NAME_TOO_LONG_EC (DB_LOG_BASE + 2)`

The specified database name is too long.

6.76.2.3 `#define DB_LOG_ILLEGAL_TABLE_SIZE_EC (DB_LOG_BASE + 3)`

Illegal table size specified.

6.76.2.4 `#define DB_LOG_ILLEGAL_LOCK_MODE_EC (DB_LOG_BASE + 4)`

Illegal combination of lock mode and mutex given.

6.76.2.5 `#define DB_LOG_MUTEX_ALLOC_EC (DB_LOG_BASE + 5)`

Failed to allocate database mutex.

6.76.2.6 `#define DB_LOG_ALLOC_TABLE_POOL_EC (DB_LOG_BASE + 6)`

Failed to allocate buffer pool.

6.76.2.7 `#define DB_LOG_TABLES_INUSE_EC (DB_LOG_BASE + 7)`

Table is in use.

6.76.2.8 `#define DB_LOG_TABLE_NAME_TOO_LONG_EC (DB_LOG_BASE + 8)`

Specified table name too long.

6.76.2.9 `#define DB_LOG_ILLEGAL_RECORD_COUNT_EC (DB_LOG_BASE + 9)`

Illegal record count specified.

6.76.2.10 `#define DB_LOG_TABLE_EXISTS_EC (DB_LOG_BASE + 10)`

Specified table already exists.

6.76.2.11 `#define DB_LOG_OUT_OF_TABLES_EC (DB_LOG_BASE + 11)`

Table resources exceeded.

6.76.2.12 `#define DB_LOG_OUT_OF_RECORDS_EC (DB_LOG_BASE + 12)`

Table record resources exceeded.

6.76.2.13 `#define DB_LOG_OUT_OF_INDICES_EC (DB_LOG_BASE + 13)`

Table index resources exceeded.

6.76.2.14 `#define DB_LOG_OUT_OF_CURSORS_EC (DB_LOG_BASE + 14)`

Table cursor resources exceeded.

6.76.2.15 `#define DB_LOG_RECORDS_INUSE_EC (DB_LOG_BASE + 15)`

Cannot delete table, records are still in table.

6.76.2.16 `#define DB_LOG_CURSORS_INUSE_EC (DB_LOG_BASE + 16)`

Cannot delete table, cursors are still in use.

6.76.2.17 `#define DB_LOG_INDEX_INUSE_EC (DB_LOG_BASE + 17)`

Cannot delete table, indices are still in use.

6.76.2.18 `#define DB_LOG_ALLOC_CURSOR_POOL_EC (DB_LOG_BASE + 18)`

Failed to allocate cursor buffer pool.

6.76.2.19 `#define DB_LOG_ALLOC_INDEX_POOL_EC (DB_LOG_BASE + 19)`

Failed to allocate index buffer pool.

6.76.2.20 `#define DB_LOG_ALLOC_RECORD_POOL_EC (DB_LOG_BASE + 20)`

Failed to allocate record buffer pool.

6.76.2.21 `#define DB_LOG_ALLOC_DATABASE_EC (DB_LOG_BASE + 21)`

Failed to allocate database.

6.76.2.22 `#define DB_LOG_TABLE_NOT_INUSE_EC (DB_LOG_BASE + 22)`

An attempt was made to delete a table not in use.

6.76.2.23 `#define DB_LOG_RECORD_ALREADY_EXISTS_EC (DB_LOG_BASE + 23)`

An attempt was made to insert an already existing record.

6.76.2.24 `#define DB_LOG_RECORD_DOES_NOT_EXIST_EC (DB_LOG_BASE + 24)`

An attempt was made to delete a record that does not exist.

6.76.2.25 `#define DB_LOG_CURSOR_INVALIDATED_EC (DB_LOG_BASE + 25)`

An attempt was made to use a cursor that has been invalidated.

6.76.2.26 `#define DB_LOG_LOCK_FAILURE_EC (DB_LOG_BASE + 26)`

Failed to lock the database.

6.76.2.27 #define DB_LOG_UNLOCK_FAILURE_EC (DB_LOG_BASE + 27)

Failed to unlock the database.

6.77 CDR

CDR. ModuleID = 5.

Macros

- #define `CDR_LOG_STREAM_ALLOC_EC` (CDR_LOG_BASE + 1)
Failed to allocate stream or stream buffer.
- #define `CDR_LOG_SET_OFFSET_EC` (CDR_LOG_BASE + 2)
Failed to set stream's current offset.
- #define `CDR_LOG_INCR_OFFSET_EC` (CDR_LOG_BASE + 3)
Failed to increment stream's current offset.
- #define `CDR_LOG_INVALID_ENUM_EC` (CDR_LOG_BASE + 4)
Invalid enum size.
- #define `CDR_LOG_UNSUPPORTED_OPTIONS_EC` (CDR_LOG_BASE + 5)
Unsupported CDR encapsulations were received. The sample is dropped.

6.77.1 Detailed Description

CDR. ModuleID = 5.

6.77.2 Macro Definition Documentation

6.77.2.1 #define CDR_LOG_STREAM_ALLOC_EC (CDR_LOG_BASE + 1)

Failed to allocate stream or stream buffer.

6.77.2.2 #define CDR_LOG_SET_OFFSET_EC (CDR_LOG_BASE + 2)

Failed to set stream's current offset.

6.77.2.3 #define CDR_LOG_INCR_OFFSET_EC (CDR_LOG_BASE + 3)

Failed to increment stream's current offset.

6.77.2.4 #define CDR_LOG_INVALID_ENUM_EC (CDR_LOG_BASE + 4)

Invalid enum size.

6.77.2.5 #define CDR_LOG_UNSUPPORTED_OPTIONS_EC (CDR_LOG_BASE + 5)

Unsupported CDR encapsulations were received. The sample is dropped.

6.78 RT

RT. ModuleID = 3.

Macros

- `#define RT_LOG_SET_IMMUTABLE_PROPERTY_EC (RT_LOG_BASE + 1)`
Failed to set registry property due to registry already being enabled.
- `#define RT_LOG_REGISTRY_INIT_FAILURE_EC (RT_LOG_BASE + 2)`
Failed to initialize registry due to failed table creation.
- `#define RT_LOG_REGISTRY_FINALIZE_EC (RT_LOG_BASE + 3)`
Failed to finalize registry due to failed table deletion.
- `#define RT_LOG_REGISTRY_EXISTS_EC (RT_LOG_BASE + 4)`
An attempt was made to register a factory that already existed.
- `#define RT_LOG_REGISTRY_REGISTER_EC (RT_LOG_BASE + 5)`
Error registering a component factory. May have exceeded RT_RegistryProperty.max_factories.
- `#define RT_LOG_REGISTRY_INIT_FACTORY_EC (RT_LOG_BASE + 7)`
A registered factory failed to initialize.
- `#define RT_LOG_REGISTRY_NAME_TOO_LONG_EC (RT_LOG_BASE + 8)`
Factory name longer than maximum length of 8.
- `#define RT_LOG_REGISTRY_INCONSISTENT_CID_EC (RT_LOG_BASE + 9)`
Factory name exists, but the class ID is not of the requested type.
- `#define RT_LOG_REGISTRY_NOT_INITIALIZED_EC (RT_LOG_BASE + 10)`
An attempt was made to operate on an uninitialized registry.

6.78.1 Detailed Description

RT. ModuleID = 3.

6.78.2 Macro Definition Documentation

6.78.2.1 `#define RT_LOG_SET_IMMUTABLE_PROPERTY_EC (RT_LOG_BASE + 1)`

Failed to set registry property due to registry already being enabled.

6.78.2.2 `#define RT_LOG_REGISTRY_INIT_FAILURE_EC (RT_LOG_BASE + 2)`

Failed to initialize registry due to failed table creation.

6.78.2.3 `#define RT_LOG_REGISTRY_FINALIZE_EC (RT_LOG_BASE + 3)`

Failed to finalize registry due to failed table deletion.

6.78.2.4 `#define RT_LOG_REGISTRY_EXISTS_EC (RT_LOG_BASE + 4)`

An attempt was made to register a factory that already existed.

6.78.2.5 `#define RT_LOG_REGISTRY_REGISTER_EC (RT_LOG_BASE + 5)`

Error registering a component factory. May have exceeded RT_RegistryProperty.max_factories.

6.78.2.6 `#define RT_LOG_REGISTRY_INIT_FACTORY_EC (RT_LOG_BASE + 7)`

A registered factory failed to initialize.

6.78.2.7 `#define RT_LOG_REGISTRY_NAME_TOO_LONG_EC (RT_LOG_BASE + 8)`

Factory name longer than maximum length of 8.

6.78.2.8 `#define RT_LOG_REGISTRY_INCONSISTENT_CID_EC (RT_LOG_BASE + 9)`

Factory name exists, but the class ID is not of the requested type.

6.78.2.9 `#define RT_LOG_REGISTRY_NOT_INITIALIZED_EC (RT_LOG_BASE + 10)`

An attempt was made to operate on an uninitialized registry.

6.79 NETIO

Network I/O. ModuleID = 4.

Macros

- #define `NETIO_LOG_GETHOST_BYNAME_EC` (NETIO_LOG_BASE + 1)
Failed to get host address from a string representation.
- #define `NETIO_LOG_BIND_NEW_EC` (NETIO_LOG_BASE + 2)
Failed to allocate memory for a bind-resolver.
- #define `NETIO_LOG_BIND_NEW_RTABLE_EC` (NETIO_LOG_BASE + 3)
Failed to create a database table for routes of a bind-resolver.
- #define `NETIO_LOG_BIND_CREATE_RTABLE_EC` (NETIO_LOG_BASE + 4)
Failed to create route record.
- #define `NETIO_LOG_BIND_DELETE_RTABLE_EC` (NETIO_LOG_BASE + 5)
Failed to delete a database table for routes of a bind_resolver.
- #define `NETIO_LOG_BIND_SET_LENGTH_EC` (NETIO_LOG_BASE + 6)
Failed to set the length for a sequence of resolved addresses.
- #define `NETIO_LOG_BIND_SET_MAX_EC` (NETIO_LOG_BASE + 7)
Failed to set the maximum length of an internal sequence of addresses.
- #define `NETIO_LOG_BIND_EXTERNAL_FAILED_EC` (NETIO_LOG_BASE + 8)
An interface failed on bind_external.
- #define `NETIO_LOG_UNBIND_EXTERNAL_FAILED_EC` (NETIO_LOG_BASE + 10)
An interface failed on unbind_external.
- #define `NETIO_LOG_ROUTE_RTABLE_ALLOC_EC` (NETIO_LOG_BASE + 11)
Failed to allocate memory for a route resolver.
- #define `NETIO_LOG_ROUTE_RTABLE_CREATE_EC` (NETIO_LOG_BASE + 12)
Failed to create a database table for routes of a route-resolver.
- #define `NETIO_LOG_ROUTE_RTABLE_DELETE_EC` (NETIO_LOG_BASE + 13)
Failed to delete a database table for routes of a route-resolver.
- #define `NETIO_LOG_ROUTE_RTABLE_ADD_EC` (NETIO_LOG_BASE + 14)
Failed to create route resolver record.
- #define `NETIO_LOG_ROUTE_RTABLE_UPDATE_EC` (NETIO_LOG_BASE + 15)
Failed to find routes to update for a route-resolver.
- #define `NETIO_LOG_AR_ALLOC_FAILED_EC` (NETIO_LOG_BASE + 16)
Failed to allocate memory for address resolver.
- #define `NETIO_LOG_AR_ADD_INVALID_INTERFACE_EC` (NETIO_LOG_BASE + 17)
An attempt was made to add an existing interface with a different port resolver to the address resolver.
- #define `NETIO_LOG_AR_ADDRESS_RESOLVE_FAILED_EC` (NETIO_LOG_BASE + 18)
An interface failed to resolve an address.
- #define `NETIO_LOG_AR_PORT_RESOLVE_FAILED_EC` (NETIO_LOG_BASE + 19)
An interface failed to resolve a port.
- #define `NETIO_LOG_AR_CONTEXT_UNINITIALIZED_EC` (NETIO_LOG_BASE + 21)
NETIO_AddressResolver_resolve/get_next was called with an uninitialized context.
- #define `NETIO_LOG_AR_INVALID_TOKEN_EC` (NETIO_LOG_BASE + 22)
An invalid token was encountered in the address string.
- #define `NETIO_LOG_AR_ADDRESS_STRING_EXCEEDED_EC` (NETIO_LOG_BASE + 23)
The length of the address exceeded the address buffer passed in.
- #define `NETIO_LOG_INVALID_ADDRESS_INDEX_EC` (NETIO_LOG_BASE + 24)
An address index was not parsed correctly.
- #define `NETIO_LOG_ILLEGAL_ROUTE_OPERATION_EC` (NETIO_LOG_BASE + 25)

- Illegal route operation attempted.*
- #define [NETIO_LOG_SET_NAME_EC](#) (UDP_LOG_BASE + 26)
Failed to set the name of a runtime component interface.
- #define [NETIO_LOG_PACKET_SET_HEAD_EC](#) (UDP_LOG_BASE + 27)
Failed to set packet's head.
- #define [NETIO_LOG_PACKET_SET_TAIL_EC](#) (UDP_LOG_BASE + 28)
Failed to set packet's tail.
- #define [NETIO_LOG_PACKET_INITIALIZE_EC](#) (UDP_LOG_BASE + 29)
Failed to initialize packet.
- #define [NETIO_LOG_LOOP_DELETE_TABLE_EC](#) (NETIO_LOG_BASE + 100)
Failed to delete a database table for a NETIO loopback interface.
- #define [NETIO_LOG_LOOP_CREATE_TABLE_EC](#) (NETIO_LOG_BASE + 101)
Failed to create a database table for a NETIO loopback interface.
- #define [NETIO_LOG_LOOP_SELECT_TABLE_EC](#) (NETIO_LOG_BASE + 102)
Failed to select a valid record when sending with the NETIO loopback interface.
- #define [NETIO_LOG_LOOP_FWD_EC](#) (NETIO_LOG_BASE + 103)
Failed to send forward with the NETIO loopback interface.
- #define [NETIO_LOG_LOOP_BINDX_DB_EC](#) (NETIO_LOG_BASE + 104)
Failed to bind an external interface with the NETIO loopback interface.
- #define [NETIO_LOG_LOOP_UNBINDX_DB_EC](#) (NETIO_LOG_BASE + 105)
Failed to unbind an external interface with the NETIO loopback interface.
- #define [NETIO_LOG_LOOP_SET_LENGTH_EC](#) (NETIO_LOG_BASE + 106)
Failed to set the length of an address sequence.
- #define [NETIO_LOG_LOOP_INVALID_PROPERTY_EC](#) (NETIO_LOG_BASE + 107)
Invalid property passed in when creating a loopback interface.
- #define [NETIO_LOG_LOOP_INITIALIZE_FAILED_EC](#) (NETIO_LOG_BASE + 108)
Failed to initialize the loopback interface.
- #define [NETIO_LOG_LOOP_CURSOR_ERROR_EC](#) (NETIO_LOG_BASE + 109)
An error occurred with a cursor while iterating over a table.
- #define [NETIO_LOG_LOOP_INVALID_FACTORY_EC](#) (NETIO_LOG_BASE + 110)
An invalid factory was passed in to create a loopback interface.
- #define [NETIO_LOG_LOOP_INVALID_COMPONENT_EC](#) (NETIO_LOG_BASE + 111)
An invalid component was passed in to delete a loopback interface.
- #define [UDP_LOG_SOCKET_INIT_EC](#) (UDP_LOG_BASE + 200)
Failed to initialize use of sockets.
- #define [UDP_LOG_GETHOSTNAME_EC](#) (UDP_LOG_BASE + 201)
Failed to get the host address given the host name.
- #define [UDP_LOG_SOCKET_SET_MCAST_EC](#) (UDP_LOG_BASE + 202)
Failed to set a socket for multicast loopback.
- #define [UDP_LOG_SOCKET_SET_MCASTIF_EC](#) (UDP_LOG_BASE + 203)
Failed to set a socket for multicast bound to a specific interface.
- #define [UDP_LOG_SOCKET_CREATE_EC](#) (UDP_LOG_BASE + 204)
Failed to create a new socket, the sysrc code is the OS return code and reason for failure.
- #define [UDP_LOG_SOCKET_SETFD_EC](#) (UDP_LOG_BASE + 205)
Failed to set the close-on-exec flag for a socket.
- #define [UDP_LOG_SOCKET_SNDBUF_EC](#) (UDP_LOG_BASE + 206)
Failed to set the socket send buffer size.
- #define [UDP_LOG_SOCKET_TTL_EC](#) (UDP_LOG_BASE + 207)
Failed to set multicast TTL for a socket.
- #define [UDP_LOG_PACKET_INIT_EC](#) (UDP_LOG_BASE + 208)
Failed to initialize a receive packet.

- `#define UDP_LOG_PACKET_HEAD_EC (UDP_LOG_BASE + 209)`
Failed to set the head of a receive packet.
- `#define UDP_LOG_PACKET_FWD_EC (UDP_LOG_BASE + 210)`
Failed reception upstream for a received packet.
- `#define UDP_LOG_NAT_DELETE_EC (UDP_LOG_BASE + 211)`
Failed to delete a database record or index from an internal NAT.
- `#define UDP_LOG_FINALIZE_EC (UDP_LOG_BASE + 212)`
Failed to finalize the parent NETIO interface.
- `#define UDP_LOG_SOCKET_REUSE_PORT_EC (UDP_LOG_BASE + 213)`
Failed to set socket to reuse a port or address.
- `#define UDP_LOG_SOCKET_BIND_EC (UDP_LOG_BASE + 214)`
Failed a socket bind.
- `#define UDP_LOG_SOCKET_ADDGRP_EC (UDP_LOG_BASE + 215)`
Failed to add membership to a multicast group for a socket.
- `#define UDP_LOG_PORT_ENTRY_EC (UDP_LOG_BASE + 216)`
Failed to create a bind entry for a receive port.
- `#define UDP_LOG_PORT_ALLOC_EC (UDP_LOG_BASE + 217)`
Failed to allocate memory for a receive buffer of a specific port.
- `#define UDP_LOG_SOCKET_RXBUF_EC (UDP_LOG_BASE + 218)`
Failed to set the receive buffer size of a socket.
- `#define UDP_LOG_PORT_ADD_EC (UDP_LOG_BASE + 219)`
Failed to add a port entry record to a database table.
- `#define UDP_LOG_PORT_THREAD_EC (UDP_LOG_BASE + 220)`
Failed to create a receive thread for a receive port.
- `#define UDP_LOG_SOCKET_IFLIST_EC (UDP_LOG_BASE + 221)`
Failed to get a list of interface addresses.
- `#define UDP_LOG_SOCKET_IFFLAGS_EC (UDP_LOG_BASE + 222)`
Failed to get the network interface flags or mask of a socket.
- `#define UDP_LOG_SOCKET_CLOSE_EC (UDP_LOG_BASE + 223)`
Failed to close a socket.
- `#define UDP_LOG_LOADLIB_EC (UDP_LOG_BASE + 224)`
Failed to load a dynamic library.
- `#define UDP_LOG_GET_BUF_SIZE_EC (UDP_LOG_BASE + 225)`
Failed to get interface buffer size.
- `#define UDP_LOG_GET_BUF_ALLOC_EC (UDP_LOG_BASE + 226)`
Out of memory to allocate interface buffer.
- `#define UDP_LOG_GET_IFLIST_EC (UDP_LOG_BASE + 227)`
Failed to get an interface list.
- `#define UDP_LOG_GET_NAMEINFO_EC (UDP_LOG_BASE + 228)`
Failed getnameinfo()
- `#define UDP_LOG_MULTICAST_ENABLE_EC (UDP_LOG_BASE + 229)`
Failed to enable multicast loopback.
- `#define UDP_LOG_UDP_CREATE_TABLE_EC (UDP_LOG_BASE + 230)`
Failed to create a database table for a UDP interface.
- `#define UDP_LOG_UDP_CREATE_INDEX_EC (UDP_LOG_BASE + 231)`
Failed to create an index for a UDP interface.
- `#define UDP_LOG_RECORD_EC (UDP_LOG_BASE + 232)`
Failed to create or insert a database record for a UDP interface.
- `#define UDP_LOG_PROPERTY_FINALIZE_EC (UDP_LOG_BASE + 233)`
Failed to finalize UDP interface factory property.
- `#define UDP_LOG_ALLOC_EC (UDP_LOG_BASE + 234)`

- Failed to allocate memory for a UDP interface.*

 - #define `UDP_LOG_SEND_PING_EC` (UDP_LOG_BASE + 235)

Failed sending a ping message, upon adding a route or waking up a receive thread.

- #define `UDP_LOG_DROPGRP_EC` (UDP_LOG_BASE + 236)

Failed to drop membership from a multicast group.

- #define `UDP_LOG_GET_LENGTH_EC` (UDP_LOG_BASE + 237)

Length of address sequence exceeds its maximum.

- #define `UDP_LOG_SET_LENGTH_EC` (UDP_LOG_BASE + 240)

Failed to set the length of an address sequence.

- #define `UDP_LOG_WSA_STARTUP_EC` (UDP_LOG_BASE + 241)

Failed WSStartup()

- #define `UDP_LOG_WINSOCK_INCOMPATIBLE_EC` (UDP_LOG_BASE + 242)

Incompatible version of Winsock, must not be older than 2.0.

- #define `UDP_LOG_INCONSISTENT_MAX_MESSAGE_SIZE_EC` (UDP_LOG_BASE + 243)

Inconsistent max_message_size.

- #define `UDP_LOG_INITIALIZE_FAILED_EC` (UDP_LOG_BASE + 244)

Failed to initialize the loopback interface.

- #define `UDP_LOG_PORT_NOT_FOUND_EC` (UDP_LOG_BASE + 245)

Did not find the specified thread port.

- #define `UDP_LOG_CURSOR_ERROR_EC` (UDP_LOG_BASE + 246)

An error occurred while iterating over a cursor.

- #define `UDP_LOG_SEND_ERROR_EC` (UDP_LOG_BASE + 247)

Failed to send message.

- #define `UDP_LOG_MULTICAST_NOT_ENABLED_EC` (UDP_LOG_BASE + 248)

Multicast address ignored because multicast supported is not compiled in.

- #define `UDP_LOG_TRANSPORT_NO_INTERFACES_EC` (UDP_LOG_BASE + 249)

Transport does not have any interfaces.

- #define `UDP_LOG_RECV_ERROR_EC` (UDP_LOG_BASE + 250)

Failed to receive message.

- #define `UDP_TRANSFORM_LOG_INVALID_FACTORY_REFERENCE_EC` (UDP_LOG_BASE + 300)

The returned factory reference is NULL.

- #define `UDP_TRANSFORM_LOG_FACTORY_NOT_FOUND_EC` (UDP_LOG_BASE + 301)

Could not find the factory used as part of a transform rule.

- #define `UDP_TRANSFORM_LOG_ALLOC_EC` (UDP_LOG_BASE + 302)

Failed to allocate entry of the specified kind.

- #define `UDP_TRANSFORM_LOG_CREATE_EC` (UDP_LOG_BASE + 303)

Failed to create a transformation of the specified kind.

- #define `UDP_TRANSFORM_LOG_ADD_EC` (UDP_LOG_BASE + 304)

Failed to add object of specified kind.

- #define `UDP_TRANSFORM_LOG_FIND_EC` (UDP_LOG_BASE + 305)

Failed to find a transformation.

- #define `UDP_TRANSFORM_LOG_DUPLICATE_EC` (UDP_LOG_BASE + 306)

Duplicate object.

- #define `UDP_TRANSFORM_LOG_TRANSFORM_EC` (UDP_LOG_BASE + 307)

Source or destination transformation function failed.

- #define `UDP_TRANSFORM_LOG_INVALID_UDP_MODE_EC` (UDP_LOG_BASE + 308)

Invalid UDP mode specified.

- #define `UDP_TRANSFORM_LOG_INVALID_TUDP_LOCATOR_KIND_EC` (UDP_LOG_BASE + 309)

Invalid transformation UDP locator kind specified.

6.79.1 Detailed Description

Network I/O. ModuleID = 4.

6.79.2 Macro Definition Documentation

6.79.2.1 `#define NETIO_LOG_GETHOST_BYNAME_EC (NETIO_LOG_BASE + 1)`

Failed to get host address from a string representation.

6.79.2.2 `#define NETIO_LOG_BIND_NEW_EC (NETIO_LOG_BASE + 2)`

Failed to allocate memory for a bind-resolver.

6.79.2.3 `#define NETIO_LOG_BIND_NEW_RTABLE_EC (NETIO_LOG_BASE + 3)`

Failed to create a database table for routes of a bind-resolver.

6.79.2.4 `#define NETIO_LOG_BIND_CREATE_RTABLE_EC (NETIO_LOG_BASE + 4)`

Failed to create route record.

May have exceeded DomainParticipantQos.resource_limits.max_receive_ports or DomainParticipantQos.resource_limits.local_writer_allocation

6.79.2.5 `#define NETIO_LOG_BIND_DELETE_RTABLE_EC (NETIO_LOG_BASE + 5)`

Failed to delete a database table for routes of a bind_resolver.

6.79.2.6 `#define NETIO_LOG_BIND_SET_LENGTH_EC (NETIO_LOG_BASE + 6)`

Failed to set the length for a sequence of resolved addresses.

6.79.2.7 `#define NETIO_LOG_BIND_SET_MAX_EC (NETIO_LOG_BASE + 7)`

Failed to set the maximum length of an internal sequence of addresses.

6.79.2.8 `#define NETIO_LOG_BIND_EXTERNAL_FAILED_EC (NETIO_LOG_BASE + 8)`

An interface failed on bind_external.

6.79.2.9 `#define NETIO_LOG_UNBIND_EXTERNAL_FAILED_EC (NETIO_LOG_BASE + 10)`

An interface failed on unbind_external.

6.79.2.10 `#define NETIO_LOG_ROUTE_RTABLE_ALLOC_EC (NETIO_LOG_BASE + 11)`

Failed to allocate memory for a route resolver.

6.79.2.11 `#define NETIO_LOG_ROUTE_RTABLE_CREATE_EC (NETIO_LOG_BASE + 12)`

Failed to create a database table for routes of a route-resolver.

6.79.2.12 `#define NETIO_LOG_ROUTE_RTABLE_DELETE_EC (NETIO_LOG_BASE + 13)`

Failed to delete a database table for routes of a route-resolver.

6.79.2.13 `#define NETIO_LOG_ROUTE_RTABLE_ADD_EC (NETIO_LOG_BASE + 14)`

Failed to create route resolver record.

May have exceeded DomainParticipantQos.resource_limits.max_destination_ports

6.79.2.14 `#define NETIO_LOG_ROUTE_RTABLE_UPDATE_EC (NETIO_LOG_BASE + 15)`

Failed to find routes to update for a route-resolver.

6.79.2.15 `#define NETIO_LOG_AR_ALLOC_FAILED_EC (NETIO_LOG_BASE + 16)`

Failed to allocate memory for address resolver.

6.79.2.16 `#define NETIO_LOG_AR_ADD_INVALID_INTERFACE_EC (NETIO_LOG_BASE + 17)`

An attempt was made to add an existing interface with a different port resolver to the address resolver.

6.79.2.17 `#define NETIO_LOG_AR_ADDRESS_RESOLVE_FAILED_EC (NETIO_LOG_BASE + 18)`

An interface failed to resolve an address.

6.79.2.18 `#define NETIO_LOG_AR_PORT_RESOLVE_FAILED_EC (NETIO_LOG_BASE + 19)`

An interface failed to resolve a port.

6.79.2.19 `#define NETIO_LOG_AR_CONTEXT_UNINITIALIZED_EC (NETIO_LOG_BASE + 21)`

NETIO_AddressResolver_resolve/get_next was called with an uninitialized context.

6.79.2.20 `#define NETIO_LOG_AR_INVALID_TOKEN_EC (NETIO_LOG_BASE + 22)`

An invalid token was encountered in the address string.

6.79.2.21 `#define NETIO_LOG_AR_ADDRESS_STRING_EXCEEDED_EC (NETIO_LOG_BASE + 23)`

The length of the address exceeded the address buffer passed in.

6.79.2.22 `#define NETIO_LOG_INVALID_ADDRESS_INDEX_EC (NETIO_LOG_BASE + 24)`

An address index was not parsed correctly.

When parsing an address string the index was not specified correctly. Legal formats are: N@ [,N]@ [N]@ [M,N]@

Where N and M are integers ≥ 0

6.79.2.23 `#define NETIO_LOG_ILLEGAL_ROUTE_OPERATION_EC (NETIO_LOG_BASE + 25)`

Illegal route operation attempted.

6.79.2.24 `#define NETIO_LOG_SET_NAME_EC (UDP_LOG_BASE + 26)`

Failed to set the name of a runtime component interface.

6.79.2.25 `#define NETIO_LOG_PACKET_SET_HEAD_EC (UDP_LOG_BASE + 27)`

Failed to set packet's head.

NETIO_Packet_set_head() failed, due to an invalid delta that would have moved the head position out-of-bounds of the packet.

6.79.2.26 `#define NETIO_LOG_PACKET_SET_TAIL_EC (UDP_LOG_BASE + 28)`

Failed to set packet's tail.

NETIO_Packet_set_tail() failed, due to an invalid delta that would have moved the tail position out-of-bounds of the packet.

6.79.2.27 `#define NETIO_LOG_PACKET_INITIALIZE_EC (UDP_LOG_BASE + 29)`

Failed to initialize packet.

NETIO_Packet_initialize() failed due to invalid arguments

6.79.2.28 `#define NETIO_LOG_LOOP_DELETE_TABLE_EC (NETIO_LOG_BASE + 100)`

Failed to delete a database table for a NETIO loopback interface.

6.79.2.29 `#define NETIO_LOG_LOOP_CREATE_TABLE_EC (NETIO_LOG_BASE + 101)`

Failed to create a database table for a NETIO loopback interface.

6.79.2.30 `#define NETIO_LOG_LOOP_SELECT_TABLE_EC (NETIO_LOG_BASE + 102)`

Failed to select a valid record when sending with the NETIO loopback interface.

6.79.2.31 `#define NETIO_LOG_LOOP_FWD_EC (NETIO_LOG_BASE + 103)`

Failed to send forward with the NETIO loopback interface.

6.79.2.32 `#define NETIO_LOG_LOOP_BINDX_DB_EC (NETIO_LOG_BASE + 104)`

Failed to bind an external interface with the NETIO loopback interface.

6.79.2.33 `#define NETIO_LOG_LOOP_UNBINDX_DB_EC (NETIO_LOG_BASE + 105)`

Failed to unbind an external interface with the NETIO loopback interface.

6.79.2.34 `#define NETIO_LOG_LOOP_SET_LENGTH_EC (NETIO_LOG_BASE + 106)`

Failed to set the length of an address sequence.

6.79.2.35 `#define NETIO_LOG_LOOP_INVALID_PROPERTY_EC (NETIO_LOG_BASE + 107)`

Invalid property passed in when creating a loopback interface.

6.79.2.36 `#define NETIO_LOG_LOOP_INITIALIZE_FAILED_EC (NETIO_LOG_BASE + 108)`

Failed to initialize the loopback interface.

6.79.2.37 `#define NETIO_LOG_LOOP_CURSOR_ERROR_EC (NETIO_LOG_BASE + 109)`

An error occurred with a cursor while iterating over a table.

6.79.2.38 `#define NETIO_LOG_LOOP_INVALID_FACTORY_EC (NETIO_LOG_BASE + 110)`

An invalid factory was passed in to create a loopback interface.

6.79.2.39 `#define NETIO_LOG_LOOP_INVALID_COMPONENT_EC (NETIO_LOG_BASE + 111)`

An invalid component was passed in to delete a loopback interface.

6.79.2.40 `#define UDP_LOG_SOCKET_INIT_EC (UDP_LOG_BASE + 200)`

Failed to initialize use of sockets.

6.79.2.41 `#define UDP_LOG_GETHOSTNAME_EC (UDP_LOG_BASE + 201)`

Failed to get the host address given the host name.

6.79.2.42 `#define UDP_LOG_SOCKET_SET_MCAST_EC (UDP_LOG_BASE + 202)`

Failed to set a socket for multicast loopback.

6.79.2.43 `#define UDP_LOG_SOCKET_SET_MCASTIF_EC (UDP_LOG_BASE + 203)`

Failed to set a socket for multicast bound to a specific interface.

6.79.2.44 `#define UDP_LOG_SOCKET_CREATE_EC (UDP_LOG_BASE + 204)`

Failed to create a new socket, the sysrc code is the OS return code and reason for failure.

6.79.2.45 `#define UDP_LOG_SOCKET_SETFD_EC (UDP_LOG_BASE + 205)`

Failed to set the close-on-exec flag for a socket.

6.79.2.46 `#define UDP_LOG_SOCKET_SNDBUF_EC (UDP_LOG_BASE + 206)`

Failed to set the socket send buffer size.

6.79.2.47 `#define UDP_LOG_SOCKET_TTL_EC (UDP_LOG_BASE + 207)`

Failed to set multicast TTL for a socket.

6.79.2.48 `#define UDP_LOG_PACKET_INIT_EC (UDP_LOG_BASE + 208)`

Failed to initialize a receive packet.

6.79.2.49 `#define UDP_LOG_PACKET_HEAD_EC (UDP_LOG_BASE + 209)`

Failed to set the head of a receive packet.

6.79.2.50 `#define UDP_LOG_PACKET_FWD_EC (UDP_LOG_BASE + 210)`

Failed reception upstream for a received packet.

6.79.2.51 `#define UDP_LOG_NAT_DELETE_EC (UDP_LOG_BASE + 211)`

Failed to delete a database record or index from an internal NAT.

6.79.2.52 `#define UDP_LOG_FINALIZE_EC (UDP_LOG_BASE + 212)`

Failed to finalize the parent NETIO interface.

6.79.2.53 `#define UDP_LOG_SOCKET_REUSE_PORT_EC (UDP_LOG_BASE + 213)`

Failed to set socket to reuse a port or address.

6.79.2.54 `#define UDP_LOG_SOCKET_BIND_EC (UDP_LOG_BASE + 214)`

Failed a socket bind.

6.79.2.55 `#define UDP_LOG_SOCKET_ADDGRP_EC (UDP_LOG_BASE + 215)`

Failed to add membership to a multicast group for a socket.

6.79.2.56 `#define UDP_LOG_PORT_ENTRY_EC (UDP_LOG_BASE + 216)`

Failed to create a bind entry for a receive port.

6.79.2.57 `#define UDP_LOG_PORT_ALLOC_EC (UDP_LOG_BASE + 217)`

Failed to allocate memory for a receive buffer of a specific port.

6.79.2.58 `#define UDP_LOG_SOCKET_RXBUF_EC (UDP_LOG_BASE + 218)`

Failed to set the receive buffer size of a socket.

6.79.2.59 `#define UDP_LOG_PORT_ADD_EC (UDP_LOG_BASE + 219)`

Failed to add a port entry record to a database table.

6.79.2.60 `#define UDP_LOG_PORT_THREAD_EC (UDP_LOG_BASE + 220)`

Failed to create a receive thread for a receive port.

6.79.2.61 `#define UDP_LOG_SOCKET_IFLIST_EC (UDP_LOG_BASE + 221)`

Failed to get a list of interface addresses.

6.79.2.62 `#define UDP_LOG_SOCKET_IFFLAGS_EC (UDP_LOG_BASE + 222)`

Failed to get the network interface flags or mask of a socket.

6.79.2.63 `#define UDP_LOG_SOCKET_CLOSE_EC (UDP_LOG_BASE + 223)`

Failed to close a socket.

6.79.2.64 `#define UDP_LOG_LOADLIB_EC (UDP_LOG_BASE + 224)`

Failed to load a dynamic library.

6.79.2.65 `#define UDP_LOG_GET_BUF_SIZE_EC (UDP_LOG_BASE + 225)`

Failed to get interface buffer size.

6.79.2.66 `#define UDP_LOG_GET_BUF_ALLOC_EC (UDP_LOG_BASE + 226)`

Out of memory to allocate interface buffer.

6.79.2.67 `#define UDP_LOG_GET_IFLIST_EC (UDP_LOG_BASE + 227)`

Failed to get an interface list.

6.79.2.68 `#define UDP_LOG_GET_NAMEINFO_EC (UDP_LOG_BASE + 228)`

Failed getnameinfo()

6.79.2.69 `#define UDP_LOG_MULTICAST_ENABLE_EC (UDP_LOG_BASE + 229)`

Failed to enable multicast loopback.

6.79.2.70 `#define UDP_LOG_UDP_CREATE_TABLE_EC (UDP_LOG_BASE + 230)`

Failed to create a database table for a UDP interface.

6.79.2.71 `#define UDP_LOG_UDP_CREATE_INDEX_EC (UDP_LOG_BASE + 231)`

Failed to create an index for a UDP interface.

6.79.2.72 **#define UDP_LOG_RECORD_EC (UDP_LOG_BASE + 232)**

Failed to create or insert a database record for a UDP interface.

6.79.2.73 **#define UDP_LOG_PROPERTY_FINALIZE_EC (UDP_LOG_BASE + 233)**

Failed to finalize UDP interface factory property.

6.79.2.74 **#define UDP_LOG_ALLOC_EC (UDP_LOG_BASE + 234)**

Failed to allocate memory for a UDP interface.

6.79.2.75 **#define UDP_LOG_SEND_PING_EC (UDP_LOG_BASE + 235)**

Failed sending a ping message, upon adding a route or waking up a receive thread.

6.79.2.76 **#define UDP_LOG_DROPGRP_EC (UDP_LOG_BASE + 236)**

Failed to drop membership from a multicast group.

6.79.2.77 **#define UDP_LOG_GET_LENGTH_EC (UDP_LOG_BASE + 237)**

Length of address sequence exceeds its maximum.

6.79.2.78 **#define UDP_LOG_SET_LENGTH_EC (UDP_LOG_BASE + 240)**

Failed to set the length of an address sequence.

6.79.2.79 **#define UDP_LOG_WSA_STARTUP_EC (UDP_LOG_BASE + 241)**

Failed WSStartup()

6.79.2.80 **#define UDP_LOG_WINSOCK_INCOMPATIBLE_EC (UDP_LOG_BASE + 242)**

Incompatible version of Winsock, must not be older than 2.0.

6.79.2.81 **#define UDP_LOG_INCONSISTENT_MAX_MESSAGE_SIZE_EC (UDP_LOG_BASE + 243)**

Inconsistent max_message_size.

UDP_InterfaceProperty.max_message_size is set to an inconsistent value, larger than UDP_InterfaceProperty.max_send_buffer_size and/or larger than UDP_InterfaceProperty.max_receive_buffer_size .

6.79.2.82 **#define UDP_LOG_INITIALIZE_FAILED_EC (UDP_LOG_BASE + 244)**

Failed to initialize the loopback interface.

6.79.2.83 **#define UDP_LOG_PORT_NOT_FOUND_EC (UDP_LOG_BASE + 245)**

Did not find the specified thread port.

6.79.2.84 **#define UDP_LOG_CURSOR_ERROR_EC (UDP_LOG_BASE + 246)**

An error occurred while iterating over a cursor.

6.79.2.85 **#define UDP_LOG_SEND_ERROR_EC (UDP_LOG_BASE + 247)**

Failed to send message.

A message could not be sent due to system error

6.79.2.86 `#define UDP_LOG_MULTICAST_NOT_ENABLED_EC (UDP_LOG_BASE + 248)`

Multicast address ignored because multicast supported is not compiled in.

A multicast address was specified as an address, but multicast support has not been enabled with `NETIO_CONFIG_ENABLE_MULTICAST` in `netio_config.h`

6.79.2.87 `#define UDP_LOG_TRANSPORT_NO_INTERFACES_EC (UDP_LOG_BASE + 249)`

Transport does not have any interfaces.

UDP configuration might be wrong.

6.79.2.88 `#define UDP_LOG_RECV_ERROR_EC (UDP_LOG_BASE + 250)`

Failed to receive message.

A message could not be received due to system error

6.79.2.89 `#define UDP_TRANSFORM_LOG_INVALID_FACTORY_REFERENCE_EC (UDP_LOG_BASE + 300)`

The returned factory reference is NULL.

This error indicates that the system has run out of memory

6.79.2.90 `#define UDP_TRANSFORM_LOG_FACTORY_NOT_FOUND_EC (UDP_LOG_BASE + 301)`

Could not find the factory used as part of a transform rule.

This error indicates that a factory for the specified rule has not been registered before the UDP transport is registered.

6.79.2.91 `#define UDP_TRANSFORM_LOG_ALLOC_EC (UDP_LOG_BASE + 302)`

Failed to allocate entry of the specified kind.

6.79.2.92 `#define UDP_TRANSFORM_LOG_CREATE_EC (UDP_LOG_BASE + 303)`

Failed to create a transformation of the specified kind.

6.79.2.93 `#define UDP_TRANSFORM_LOG_ADD_EC (UDP_LOG_BASE + 304)`

Failed to add object of specified kind.

6.79.2.94 `#define UDP_TRANSFORM_LOG_FIND_EC (UDP_LOG_BASE + 305)`

Failed to find a transformation.

6.79.2.95 `#define UDP_TRANSFORM_LOG_DUPLICATE_EC (UDP_LOG_BASE + 306)`

Duplicate object.

6.79.2.96 `#define UDP_TRANSFORM_LOG_TRANSFORM_EC (UDP_LOG_BASE + 307)`

Source or destination transformation function failed.

6.79.2.97 `#define UDP_TRANSFORM_LOG_INVALID_UDP_MODE_EC (UDP_LOG_BASE + 308)`

Invalid UDP mode specified.

6.79.2.98 `#define UDP_TRANSFORM_LOG_INVALID_TUDP_LOCATOR_KIND_EC (UDP_LOG_BASE + 309)`

Invalid transformation UDP locator kind specified.

[UDP_InterfaceFactoryProperty::transform_locator_kind](#) must be greater or equal than `NETIO_ADDRESS_KIND_`

USER and not equal to NETIO_ADDRESS_KIND_SHMEM or NETIO_ADDRESS_KIND_INTRA.

6.80 RTPS

Real-Time Publish-Subscribe. ModuleID = 6.

Macros

- #define `RTPS_LOG_INITIALIZE_INTERFACE_EC` (RTPS_LOG_BASE + 1)
Failed to initialize an RTPS interface.
- #define `RTPS_LOG_ALLOCATE_EC` (RTPS_LOG_BASE + 2)
Failed to allocate heap memory for internal resources.
- #define `RTPS_LOG_CREATE_ROUTE_TABLE_EC` (RTPS_LOG_BASE + 3)
Failed to create a database table for storing RTPS route info.
- #define `RTPS_LOG_DB_CREATE_ROUTE_PEER_INDEX_EC` (RTPS_LOG_BASE + 4)
Failed to create database index for route-peer info.
- #define `RTPS_LOG_DB_CREATE_ROUTE_XPORT_INDEX_EC` (RTPS_LOG_BASE + 5)
Failed to create database index for route-transport info.
- #define `RTPS_LOG_DB_CREATE_BIND_PEER_INDEX_EC` (RTPS_LOG_BASE + 6)
Failed to create database index for bind-peer info.
- #define `RTPS_LOG_INDEX_ADD_ENTRY_EC` (RTPS_LOG_BASE + 8)
Failed to add an entry to a database index.
- #define `RTPS_LOG_DB_SELECT_ALL_EC` (RTPS_LOG_BASE + 9)
Failed to select all entries of a database table.
- #define `RTPS_LOG_DB_SELECT_MATCH_EC` (RTPS_LOG_BASE + 10)
Matching entry not found in a database table.
- #define `RTPS_LOG_DB_SELECT_RANGE_EC` (RTPS_LOG_BASE + 11)
No matching entries found in a database table.
- #define `RTPS_LOG_DB_CREATE_ROUTE_ENTRY_EC` (RTPS_LOG_BASE + 12)
Failed to create database entry for route.
- #define `RTPS_LOG_DB_INSERT_ROUTE_ENTRY_EC` (RTPS_LOG_BASE + 13)
Failed to insert entry into route table.
- #define `RTPS_LOG_DB_DELETE_ROUTE_ENTRY_EC` (RTPS_LOG_BASE + 15)
Failed to delete entry from route table.
- #define `RTPS_LOG_DB_CREATE_BIND_ENTRY_EC` (RTPS_LOG_BASE + 16)
Failed to create database entry for bind.
- #define `RTPS_LOG_DB_INSERT_BIND_ENTRY_EC` (RTPS_LOG_BASE + 17)
Failed to insert an entry into the bind table.
- #define `RTPS_LOG_DB_REMOVE_BIND_ENTRY_EC` (RTPS_LOG_BASE + 18)
Failed to remove an entry from the bind table.
- #define `RTPS_LOG_DB_REMOVE_EXTERNAL_BIND_ENTRY_EC` (RTPS_LOG_BASE + 19)
Failed to remove an entry from the external bind table.
- #define `RTPS_LOG_DB_DELETE_EXTERNAL_BIND_ENTRY_EC` (RTPS_LOG_BASE + 20)
Failed to delete entry from external bind table.
- #define `RTPS_LOG_SEND_EC` (RTPS_LOG_BASE + 21)
Failed to send a packet due to a lower module failing to send the packet.
- #define `RTPS_LOG_RECEIVE_EC` (RTPS_LOG_BASE + 22)
Failed to receive a packet due to a higher module failing to receive the packet.
- #define `RTPS_LOG_ROUTE_PACKET_EC` (RTPS_LOG_BASE + 23)
Failed to send a packet, when routing to a lower module or peer.
- #define `RTPS_LOG_FORWARD_UPSTREAM_EC` (RTPS_LOG_BASE + 24)
Failed to receive a packet, when forwarding to a higher upstream module.
- #define `RTPS_LOG_BAD_PARAMETER_EC` (RTPS_LOG_BASE + 25)

- Bad parameter to an RTPS interface function.*

 - #define [RTPS_LOG_NOT_ENABLED_EC](#) (RTPS_LOG_BASE + 26)

Failed because interface is not enabled.
- #define [RTPS_LOG_READER_UNSUPPORTED_EC](#) (RTPS_LOG_BASE + 27)

RTPS reader does not support send.
- #define [RTPS_LOG_INTERFACE_MISMATCH_EC](#) (RTPS_LOG_BASE + 28)

Upstream interface does not match source or destination of packet being sent or received.
- #define [RTPS_LOG_FULL_SEND_WINDOW_EC](#) (RTPS_LOG_BASE + 29)

Failed to send due to reaching maximum send window size.
- #define [RTPS_LOG_NETIO_PACKET_SET_TAIL_EC](#) (RTPS_LOG_BASE + 31)

Failed to set tail of packet to send.
- #define [RTPS_LOG_FUNC_UNSUPPORTED_EC](#) (RTPS_LOG_BASE + 32)

RTPS interface does not support this function.
- #define [RTPS_LOG_EXCEEDED_LIMIT_TRANSPORTS_EC](#) (RTPS_LOG_BASE + 33)

Out of transport entries.
- #define [RTPS_LOG_EXCEEDED_LIMIT_PEERS_EC](#) (RTPS_LOG_BASE + 34)

Out of peer entries.
- #define [RTPS_LOG_ASSERT_PEER_EC](#) (RTPS_LOG_BASE + 35)

Failed to assert a remote writer or reader.
- #define [RTPS_LOG_ASSERT_TRANSPORT_EC](#) (RTPS_LOG_BASE + 36)

Failed to assert a downstream transport.
- #define [RTPS_LOG_FIND_EXTERNAL_INTERFACE_EC](#) (RTPS_LOG_BASE + 37)

Failed to lookup an external interface.
- #define [RTPS_LOG_NONEXISTENT_ROUTE_EC](#) (RTPS_LOG_BASE + 38)

Failed to delete a nonexistent route.
- #define [RTPS_LOG_NONEXISTENT_BIND_EC](#) (RTPS_LOG_BASE + 39)

Failed to remove a nonexistent bind entry.
- #define [RTPS_LOG_NONEXISTENT_EXTERNAL_BIND_EC](#) (RTPS_LOG_BASE + 40)

Failed to remove a nonexistent external bind entry.
- #define [RTPS_LOG_STALE_ACK_EPOCH_EC](#) (RTPS_LOG_BASE + 41)

Dropped an ACKNACK submessage with an old epoch.
- #define [RTPS_LOG_STALE_HB_EPOCH_EC](#) (RTPS_LOG_BASE + 42)

Dropped a HEARTBEAT submessage with an old epoch.
- #define [RTPS_LOG_ACK_EC](#) (RTPS_LOG_BASE + 43)

Failed an acknack() upstream.
- #define [RTPS_LOG_REQUEST_EC](#) (RTPS_LOG_BASE + 45)

Failed a request() upstream.
- #define [RTPS_LOG_RETURN_LOAN_EC](#) (RTPS_LOG_BASE + 46)

Failed a return_loan() upstream.
- #define [RTPS_LOG_NETIO_PACKET_SET_HEAD_EC](#) (RTPS_LOG_BASE + 47)

Failed to set the head of a packet.
- #define [RTPS_LOG_SEND_HEARTBEAT_EC](#) (RTPS_LOG_BASE + 48)

Failed to send a HEARTBEAT submessage.
- #define [RTPS_LOG_SHIFT_BITMAP_EC](#) (RTPS_LOG_BASE + 49)

Failed to shift a bitmap.
- #define [RTPS_LOG_FULLY_ACKED_READER_EC](#) (RTPS_LOG_BASE + 50)

A Reader is fully acknowledged and does not need to respond to a final HEARTBEAT.
- #define [RTPS_LOG_DATA_OUT_OF_RANGE_EC](#) (RTPS_LOG_BASE + 51)

Dropping a DATA submessage whose sequence number is outside of a Reader's receive window.
- #define [RTPS_LOG_DATA_ALREADY_RECEIVED_EC](#) (RTPS_LOG_BASE + 52)

A DATA submessage whose sequence number was previously received was discarded. This is not an error and may happen for example when a message is retransmitted.

- #define `RTPS_LOG_INTERFACE_NOT_ENABLED_EC` (RTPS_LOG_BASE + 53)
Failed to receive a message because interface is not enabled.
- #define `RTPS_LOG_INVALID_PACKET_EC` (RTPS_LOG_BASE + 54)
Dropped a message with an invalid packet header.
- #define `RTPS_LOG_UNSUPPORTED_INTERFACE_EC` (RTPS_LOG_BASE + 55)
Received a message on an unsupported interface.
- #define `RTPS_LOG_UNKNOWN_SUBMESSAGE_EC` (RTPS_LOG_BASE + 56)
Received a submessage with an unknown ID.
- #define `RTPS_LOG_PROCESS_ACKNACK_EC` (RTPS_LOG_BASE + 57)
Failed to process received ACKNACK submessage.
- #define `RTPS_LOG_PROCESS_DATA_EC` (RTPS_LOG_BASE + 58)
Failed to process received DATA submessage.
- #define `RTPS_LOG_PROCESS_GAP_EC` (RTPS_LOG_BASE + 59)
Failed to process received GAP submessage.
- #define `RTPS_LOG_PROCESS_HEARTBEAT_EC` (RTPS_LOG_BASE + 60)
Failed to process received HEARTBEAT submessage.
- #define `RTPS_LOG_CREATE_HB_EVENT_EC` (RTPS_LOG_BASE + 61)
Failed to create periodic HEARTBEAT event.
- #define `RTPS_LOG_CREATE_EVENT_TIMER_EC` (RTPS_LOG_BASE + 62)
Failed to create periodic HEARTBEAT event timer.
- #define `RTPS_LOG_SEQ_NUM_OUT_OF_RANGE_EC` (RTPS_LOG_BASE + 63)
Failed to set bitmap bit due to out-of-range sequence number.
- #define `RTPS_LOG_FIRST_SN_GREATER_LAST_SN_EC` (RTPS_LOG_BASE + 65)
Invalid range of sequence numbers to fill in bitmap.
- #define `RTPS_LOG_BITCOUNT_OUT_OF_BOUNDS_EC` (RTPS_LOG_BASE + 66)
Deserialized an invalid out-of-bounds bitcount for a bitmap.
- #define `RTPS_LOG_SHIFT_SN_OUT_OF_RANGE_EC` (RTPS_LOG_BASE + 67)
Failed to shift bitmap due to invalid starting sequence number.
- #define `RTPS_LOG_SERIALIZE_HOST_ID_EC` (RTPS_LOG_BASE + 68)
Failed to serialize host ID of GUID.
- #define `RTPS_LOG_SERIALIZE_APP_ID_EC` (RTPS_LOG_BASE + 69)
Failed to serialize app ID of GUID.
- #define `RTPS_LOG_SERIALIZE_INSTANCE_ID_EC` (RTPS_LOG_BASE + 70)
Failed to serialize instance ID of GUID.
- #define `RTPS_LOG_SERIALIZE_OBJECT_ID_EC` (RTPS_LOG_BASE + 71)
Failed to serialize object ID of GUID.
- #define `RTPS_LOG_DESERIALIZE_HOST_ID_EC` (RTPS_LOG_BASE + 72)
Failed to deserialize host ID of GUID.
- #define `RTPS_LOG_DESERIALIZE_APP_ID_EC` (RTPS_LOG_BASE + 73)
Failed to deserialize app ID of GUID.
- #define `RTPS_LOG_DESERIALIZE_INSTANCE_ID_EC` (RTPS_LOG_BASE + 74)
Failed to deserialize instance ID of GUID.
- #define `RTPS_LOG_DESERIALIZE_OBJECT_ID_EC` (RTPS_LOG_BASE + 75)
Failed to deserialize object ID of GUID.
- #define `RTPS_LOG_DB_CREATE_EXT_BIND_ENTRY_EC` (RTPS_LOG_BASE + 76)
Failed to create database entry for external bind table.
- #define `RTPS_LOG_BITMAP_SET_BIT_EC` (RTPS_LOG_BASE + 77)
Failed to set bit in bitmap.
- #define `RTPS_LOG_UNSUPPORTED_INFO_REPLY_EC` (RTPS_LOG_BASE + 78)

- Received and dropped currently unsupported INFO_REPLY submessage.*

 - #define [RTPS_LOG_UNSUPPORTED_INFO_REPLY_IP4_EC](#) (RTPS_LOG_BASE + 79)
- Received and dropped currently unsupported INFO_REPLY_IP4 submessage.*

 - #define [RTPS_LOG_UNSUPPORTED_INFO_SRC_EC](#) (RTPS_LOG_BASE + 80)
- Received and dropped currently unsupported INFO_SRC submessage.*

 - #define [RTPS_LOG_DELETE_INDEX_EC](#) (RTPS_LOG_BASE + 81)
- Failed to delete index of a database table.*

 - #define [RTPS_LOG_DELETE_TABLE_EC](#) (RTPS_LOG_BASE + 82)
- Failed to delete a database table.*

 - #define [RTPS_LOG_DB_LOCK_EC](#) (RTPS_LOG_BASE + 83)
- Failed to take database lock.*

 - #define [RTPS_LOG_DB_UNLOCK_EC](#) (RTPS_LOG_BASE + 84)
- Failed to give database lock.*

 - #define [RTPS_LOG_CREATE_BIND_TABLE_EC](#) (RTPS_LOG_BASE + 85)
- Failed to create a database table for storing RTPS bind info.*

 - #define [RTPS_LOG_STATUS_CHANGE_EC](#) (RTPS_LOG_BASE + 86)
- Failed to update status for reliable reader activity changed.*

 - #define [RTPS_LOG_SET_GAP_EC](#) (RTPS_LOG_BASE + 87)
- Failed to update status for reliable reader activity changed.*

 - #define [RTPS_LOG_WINDOW_POOL_ALLOC_EC](#) (RTPS_LOG_BASE + 88)
- Out of memory to allocate send window pool.*

 - #define [RTPS_LOG_GET_WINDOW_BUFFER_EC](#) (RTPS_LOG_BASE + 89)
- Failed to get buffer from send window pool.*

 - #define [RTPS_LOG_INSERT_WINDOW_FULL_EC](#) (RTPS_LOG_BASE + 90)
- Cannot insert sample into full window.*

 - #define [RTPS_LOG_WINDOW_INSERT_EC](#) (RTPS_LOG_BASE + 91)
- Failed to insert sample into send window.*

 - #define [RTPS_LOG_TIMER_DELETE_TIMEOUT_EC](#) (RTPS_LOG_BASE + 92)
- Failed to delete a timeout event.*

 - #define [RTPS_LOG_BUFFERPOOL_DELETE_EC](#) (RTPS_LOG_BASE + 93)
- Failed to delete a buffer pool.*

 - #define [RTPS_LOG_DIRECT_SEND_EC](#) (RTPS_LOG_BASE + 94)
- Failed to send a message to a specific peer.*

 - #define [RTPS_LOG_PACKET_INITIALIZE_EC](#) (RTPS_LOG_BASE + 95)
- RTPS writer failed to initialize a packet.*

 - #define [RTPS_LOG_WRITER_REQUEST_SENT_HISTORY_EC](#) (RTPS_LOG_BASE + 96)
- RTPS reliable writer failed to request and resend history.*

 - #define [RTPS_LOG_BITMAP_SHIFT_EC](#) (RTPS_LOG_BASE + 97)
- Failed to shift bitmap to new lead sequence number.*

 - #define [RTPS_LOG_WINDOW_ADVANCE_EC](#) (RTPS_LOG_BASE + 98)
- Send window of a remote reader failed to advance.*

 - #define [RTPS_LOG_WINDOW_ADVANCE_UNACKED_EC](#) (RTPS_LOG_BASE + 99)
- Failed when advancing window ahead of unacknowledged sequence number.*

 - #define [RTPS_LOG_LOOKUP_PEER_EC](#) (RTPS_LOG_BASE + 100)
- Failed to find peer that should exist.*

 - #define [RTPS_LOG_READER_SEND_ACKNACK_EC](#) (RTPS_LOG_BASE + 101)
- RTPS reader failed to send an ACKNACK message.*

 - #define [RTPS_LOG_FINALIZE_INTERFACE_EC](#) (RTPS_LOG_BASE + 102)
- Failed to finalize an RTPS interface.*

 - #define [RTPS_LOG_INDEXER_REMOVE_ENTRY_EC](#) (RTPS_LOG_BASE + 103)
- Failed to remove index entry for external interface upon deleting RTPS interface.*

- `#define RTPS_LOG_LOOKUP_EXT_BIND_ENTRY_EC (RTPS_LOG_BASE + 104)`
Failed when looking up external bind entry from database.
- `#define RTPS_LOG_SEQ_INIT_EC (RTPS_LOG_BASE + 105)`
Failed to initialize a local sequence.
- `#define RTPS_LOG_SEQ_MAX_EC (RTPS_LOG_BASE + 106)`
Failed to set maximum of a local sequence.
- `#define RTPS_LOG_SEQ_LEN_EC (RTPS_LOG_BASE + 107)`
Failed to set length of a local sequence.
- `#define RTPS_LOG_INTF_MODE_UNDEF_EC (RTPS_LOG_BASE + 108)`
Initializing an RTPS interface with undefined mode.
- `#define RTPS_LOG_DELETE_BUFFER_POOL_EC (RTPS_LOG_BASE + 109)`
Failed to delete buffer pool.
- `#define RTPS_LOG_CREATE_BUFFER_POOL_EC (RTPS_LOG_BASE + 110)`
Failed to create buffer pool.
- `#define RTPS_LOG_CREATE_PEERS_INDEX_EC (RTPS_LOG_BASE + 111)`
Failed to create index of peers.
- `#define RTPS_LOG_DELETE_INDEXER_EC (RTPS_LOG_BASE + 112)`
Failed to delete indexer.
- `#define RTPS_LOG_GET_PEER_BUFFER_EC (RTPS_LOG_BASE + 113)`
Failed to get buffer from peer buffer pool.
- `#define RTPS_LOG_DB_CREATE_DIRECT_INDEX_EC (RTPS_LOG_BASE + 114)`
Failed to create database index for direct sends.
- `#define RTPS_LOG_DB_CREATE_GROUP_INDEX_EC (RTPS_LOG_BASE + 115)`
Failed to create database index for group sends.
- `#define RTPS_LOG_NETIO_PACKET_INSUFFICIENT_BUFFER_EC (RTPS_LOG_BASE + 116)`
Insufficient buffer in packet.
- `#define RTPS_LOG_PROCESS_DATA_BATCH_EC (RTPS_LOG_BASE + 117)`
Failed to process received DATA BATCH submessage.
- `#define RTPS_LOG_PROCESS_HEARTBEAT_BATCH_EC (RTPS_LOG_BASE + 118)`
Failed to process received HEARTBEAT_BATCH submessage.
- `#define RTPS_LOG_NEGATIVE_RESERVATION_COUNT_EC (RTPS_LOG_BASE + 119)`
The reservation count is zero.
- `#define RTPS_LOG_DB_CREATE_SELECTED_GROUP_INDEX_EC (RTPS_LOG_BASE + 148)`
Failed to create database index for the selected route group.
- `#define RTPS_LOG_CHECKSUM_MISSING_CHECKSUM_LENGTH_EC (RTPS_LOG_BASE + 149)`
A header extension was received, but the checksum length is not present.
- `#define RTPS_LOG_CHECKSUM_UNSUPPORTED_EC (RTPS_LOG_BASE + 150)`
Unsupported checksum bits received, message dropped.
- `#define RTPS_LOG_CHECKSUM_LENGTH_DESERIALIZE_ERROR_EC (RTPS_LOG_BASE + 151)`
Checksum length could not be deserialized.
- `#define RTPS_LOG_CHECKSUM_INCONSISTENT_LENGTH_EC (RTPS_LOG_BASE + 152)`
The payload length is less than the deserialized checksum length.
- `#define RTPS_LOG_CHECKSUM_CHECKSUM_DESERIALIZE_ERROR_EC (RTPS_LOG_BASE + 153)`
Checksum length could not be deserialized.
- `#define RTPS_LOG_CHECKSUM_CHECKSUM_ERROR_EC (RTPS_LOG_BASE + 154)`
The checksum is invalid.
- `#define RTPS_LOG_DATA_BATCH_ALREADY_RECEIVED_EC (RTPS_LOG_BASE + 155)`
A DATA_BATCH submessage whose sequence number was previously received was discarded. This is not an error and may happen for example when a message is retransmitted.
- `#define RTPS_LOG_DATA_SN_OUT_OF_ORDER_EC (RTPS_LOG_BASE + 156)`
Received a DATA submessage with an out-of-order sequence number.

- `#define RTPS_LOG_DATA_BATCH_SN_OUT_OF_ORDER_EC` (RTPS_LOG_BASE + 157)
Received a DATA_BATCH submessage with an out-of-order sequence number.
- `#define RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM128_EC` (RTPS_LOG_BASE + 200)
The property tries to override the BUILTIN128 checksum without setting allow_builtin_override to TRUE.
- `#define RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM32_EC` (RTPS_LOG_BASE + 201)
The property tries to override the BUILTIN32 checksum without setting allow_builtin_override to TRUE.
- `#define RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM64_EC` (RTPS_LOG_BASE + 202)
The property tries to override the BUILTIN64 checksum without setting allow_builtin_override to TRUE.
- `#define RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN128_EC` (RTPS_LOG_BASE + 203)
The property override of the BUILTIN128 checksum is invalid. It is not legal to reuse any of the built-in functions.
- `#define RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN32_EC` (RTPS_LOG_BASE + 204)
The property override of the BUILTIN32 checksum is invalid. It is not legal to reuse any of the built-in functions.
- `#define RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN64_EC` (RTPS_LOG_BASE + 205)
The property override of the BUILTIN64 checksum is invalid. It is not legal to reuse any of the built-in functions.
- `#define RTPS_LOG_CHECKSUM_INVALID_CUSTOM256_EC` (RTPS_LOG_BASE + 206)
The property specifies an invalid custom256 checksum function.
- `#define RTPS_LOG_CHECKSUM_INVALID_CUSTOM32_EC` (RTPS_LOG_BASE + 207)
The property specifies an invalid custom32 checksum function.
- `#define RTPS_LOG_CHECKSUM_INVALID_CUSTOM64_EC` (RTPS_LOG_BASE + 208)
The property specifies an invalid custom64 checksum function.
- `#define RTPS_LOG_CHECKSUM_INVALID_CUSTOM128_EC` (RTPS_LOG_BASE + 209)
The property specifies an invalid custom128 checksum function.
- `#define RTPS_LOG_CHECKSUM_CHECKSUM_CALCULATION_FAILED_EC` (RTPS_LOG_BASE + 210)
RTPS_Interface_calculate_checksum() failed to calculate the checksum.
- `#define RTPS_LOG_UNSUPPORTED_MANDATORY_HDR_EXT_PID_EC` (RTPS_LOG_BASE + 211)
The HeaderExtension contained a mandatory PID that was not understood.
- `#define RTPS_LOG_INVALID_HDR_EXT_PID_LIST_EC` (RTPS_LOG_BASE + 212)
An invalid HeaderExtension PID was detected.
- `#define RTPS_LOG_INVALID_HDR_EXT_PID_ERROR_EC` (RTPS_LOG_BASE + 213)
Failed to decode the HeaderExtension PID list.
- `#define RTPS_LOG_INCONSISTENT_HDR_EXT_LENGTH_ERROR_EC` (RTPS_LOG_BASE + 214)
The HeaderExtension length is inconsistent with the decoded encoded HeaderExtension length.
- `#define RTPS_LOG_MESSAGE_EXCEED_LENGTH_ERROR_EC` (RTPS_LOG_BASE + 215)
A RTPS message which was longer than the indicated message length in the RTPS header was received. This causes the rest of the RTPS message to be invalidated and the remaining submessages to be ignored.
- `#define RTPS_LOG_CHECKSUM_INVALID_CRC32_FLAGS_EC` (RTPS_LOG_BASE + 216)
The CRC32 submsg flags are invalid.

6.80.1 Detailed Description

Real-Time Publish-Subscribe. ModuleID = 6.

6.80.2 Macro Definition Documentation

6.80.2.1 `#define RTPS_LOG_INITIALIZE_INTERFACE_EC` (RTPS_LOG_BASE + 1)

Failed to initialize an RTPS interface.

6.80.2.2 #define RTPS_LOG_ALLOCATE_EC (RTPS_LOG_BASE + 2)

Failed to allocate heap memory for internal resources.

6.80.2.3 #define RTPS_LOG_CREATE_ROUTE_TABLE_EC (RTPS_LOG_BASE + 3)

Failed to create a database table for storing RTPS route info.

6.80.2.4 #define RTPS_LOG_DB_CREATE_ROUTE_PEER_INDEX_EC (RTPS_LOG_BASE + 4)

Failed to create database index for route-peer info.

6.80.2.5 #define RTPS_LOG_DB_CREATE_ROUTE_XPORT_INDEX_EC (RTPS_LOG_BASE + 5)

Failed to create database index for route-transport info.

6.80.2.6 #define RTPS_LOG_DB_CREATE_BIND_PEER_INDEX_EC (RTPS_LOG_BASE + 6)

Failed to create database index for bind-peer info.

6.80.2.7 #define RTPS_LOG_INDEX_ADD_ENTRY_EC (RTPS_LOG_BASE + 8)

Failed to add an entry to a database index.

6.80.2.8 #define RTPS_LOG_DB_SELECT_ALL_EC (RTPS_LOG_BASE + 9)

Failed to select all entries of a database table.

Failure reason given by database return code (dbrc)

6.80.2.9 #define RTPS_LOG_DB_SELECT_MATCH_EC (RTPS_LOG_BASE + 10)

Matching entry not found in a database table.

Failure reason given by database return code (dbrc)

6.80.2.10 #define RTPS_LOG_DB_SELECT_RANGE_EC (RTPS_LOG_BASE + 11)

No matching entries found in a database table.

6.80.2.11 #define RTPS_LOG_DB_CREATE_ROUTE_ENTRY_EC (RTPS_LOG_BASE + 12)

Failed to create database entry for route.

May have exceeded DomainParticipantQos.resource_limits.matching_writer_reader_pair_allocation, DataWriterQos.writer_resource_limits.max_remote_readers, or DataReaderQos.reader_resource_limits.max_remote_writers

6.80.2.12 #define RTPS_LOG_DB_INSERT_ROUTE_ENTRY_EC (RTPS_LOG_BASE + 13)

Failed to insert entry into route table.

Failure reason given by database return code (dbrc)

6.80.2.13 #define RTPS_LOG_DB_DELETE_ROUTE_ENTRY_EC (RTPS_LOG_BASE + 15)

Failed to delete entry from route table.

Failure reason given by database return code (dbrc)

6.80.2.14 #define RTPS_LOG_DB_CREATE_BIND_ENTRY_EC (RTPS_LOG_BASE + 16)

Failed to create database entry for bind.

If DomainParticipant, may have exceeded DomainParticipantQos.resource_limits.matching_reader_writer_pair_

allocation. Failure reason given by database return code (dbrc).

6.80.2.15 #define RTPS_LOG_DB_INSERT_BIND_ENTRY_EC (RTPS_LOG_BASE + 17)

Failed to insert an entry into the bind table.

Failure reason given by database return code (dbrc)

6.80.2.16 #define RTPS_LOG_DB_REMOVE_BIND_ENTRY_EC (RTPS_LOG_BASE + 18)

Failed to remove an entry from the bind table.

Failure reason given by database return code (dbrc)

6.80.2.17 #define RTPS_LOG_DB_REMOVE_EXTERNAL_BIND_ENTRY_EC (RTPS_LOG_BASE + 19)

Failed to remove an entry from the external bind table.

Failure reason given by database return code (dbrc)

6.80.2.18 #define RTPS_LOG_DB_DELETE_EXTERNAL_BIND_ENTRY_EC (RTPS_LOG_BASE + 20)

Failed to delete entry from external bind table.

Failure reason given by database return code (dbrc)

6.80.2.19 #define RTPS_LOG_SEND_EC (RTPS_LOG_BASE + 21)

Failed to send a packet due to a lower module failing to send the packet.

6.80.2.20 #define RTPS_LOG_RECEIVE_EC (RTPS_LOG_BASE + 22)

Failed to receive a packet due to a higher module failing to receive the packet.

6.80.2.21 #define RTPS_LOG_ROUTE_PACKET_EC (RTPS_LOG_BASE + 23)

Failed to send a packet, when routing to a lower module or peer.

6.80.2.22 #define RTPS_LOG_FORWARD_UPSTREAM_EC (RTPS_LOG_BASE + 24)

Failed to receive a packet, when forwarding to a higher upstream module.

6.80.2.23 #define RTPS_LOG_BAD_PARAMETER_EC (RTPS_LOG_BASE + 25)

Bad parameter to an RTPS interface function.

6.80.2.24 #define RTPS_LOG_NOT_ENABLED_EC (RTPS_LOG_BASE + 26)

Failed because interface is not enabled.

6.80.2.25 #define RTPS_LOG_READER_UNSUPPORTED_EC (RTPS_LOG_BASE + 27)

RTPS reader does not support send.

6.80.2.26 #define RTPS_LOG_INTERFACE_MISMATCH_EC (RTPS_LOG_BASE + 28)

Upstream interface does not match source or destination of packet being sent or received.

6.80.2.27 #define RTPS_LOG_FULL_SEND_WINDOW_EC (RTPS_LOG_BASE + 29)

Failed to send due to reaching maximum send window size.

6.80.2.28 `#define RTPS_LOG_NETIO_PACKET_SET_TAIL_EC (RTPS_LOG_BASE + 31)`

Failed to set tail of packet to send.

6.80.2.29 `#define RTPS_LOG_FUNC_UNSUPPORTED_EC (RTPS_LOG_BASE + 32)`

RTPS interface does not support this function.

6.80.2.30 `#define RTPS_LOG_EXCEEDED_LIMIT_TRANSPORTS_EC (RTPS_LOG_BASE + 33)`

Out of transport entries.

Exceeded either `DataWriterQos.writer_resource_limits.max_remote_readers` or `DataReaderQos.reader_resource_limits.max_remote_writers`

6.80.2.31 `#define RTPS_LOG_EXCEEDED_LIMIT_PEERS_EC (RTPS_LOG_BASE + 34)`

Out of peer entries.

Exceeded either `DataWriterQos.writer_resource_limits.max_remote_readers` or `DataReaderQos.reader_resource_limits.max_remote_writers`

6.80.2.32 `#define RTPS_LOG_ASSERT_PEER_EC (RTPS_LOG_BASE + 35)`

Failed to assert a remote writer or reader.

6.80.2.33 `#define RTPS_LOG_ASSERT_TRANSPORT_EC (RTPS_LOG_BASE + 36)`

Failed to assert a downstream transport.

6.80.2.34 `#define RTPS_LOG_FIND_EXTERNAL_INTERFACE_EC (RTPS_LOG_BASE + 37)`

Failed to lookup an external interface.

6.80.2.35 `#define RTPS_LOG_NONEXISTENT_ROUTE_EC (RTPS_LOG_BASE + 38)`

Failed to delete a nonexistent route.

6.80.2.36 `#define RTPS_LOG_NONEXISTENT_BIND_EC (RTPS_LOG_BASE + 39)`

Failed to remove a nonexistent bind entry.

6.80.2.37 `#define RTPS_LOG_NONEXISTENT_EXTERNAL_BIND_EC (RTPS_LOG_BASE + 40)`

Failed to remove a nonexistent external bind entry.

6.80.2.38 `#define RTPS_LOG_STALE_ACK_EPOCH_EC (RTPS_LOG_BASE + 41)`

Dropped an ACKNACK submessage with an old epoch.

6.80.2.39 `#define RTPS_LOG_STALE_HB_EPOCH_EC (RTPS_LOG_BASE + 42)`

Dropped a HEARTBEAT submessage with an old epoch.

6.80.2.40 `#define RTPS_LOG_ACK_EC (RTPS_LOG_BASE + 43)`

Failed an `acknack()` upstream.

6.80.2.41 `#define RTPS_LOG_REQUEST_EC (RTPS_LOG_BASE + 45)`

Failed a `request()` upstream.

6.80.2.42 `#define RTPS_LOG_RETURN_LOAN_EC (RTPS_LOG_BASE + 46)`

Failed a `return_loan()` upstream.

6.80.2.43 `#define RTPS_LOG_NETIO_PACKET_SET_HEAD_EC (RTPS_LOG_BASE + 47)`

Failed to set the head of a packet.

6.80.2.44 `#define RTPS_LOG_SEND_HEARTBEAT_EC (RTPS_LOG_BASE + 48)`

Failed to send a HEARTBEAT submessage.

6.80.2.45 `#define RTPS_LOG_SHIFT_BITMAP_EC (RTPS_LOG_BASE + 49)`

Failed to shift a bitmap.

6.80.2.46 `#define RTPS_LOG_FULLY_ACKED_READER_EC (RTPS_LOG_BASE + 50)`

A Reader is fully acknowledged and does not need to respond to a final HEARTBEAT.

6.80.2.47 `#define RTPS_LOG_DATA_OUT_OF_RANGE_EC (RTPS_LOG_BASE + 51)`

Dropping a DATA submessage whose sequence number is outside of a Reader's receive window.

6.80.2.48 `#define RTPS_LOG_DATA_ALREADY_RECEIVED_EC (RTPS_LOG_BASE + 52)`

A DATA submessage whose sequence number was previously received was discarded. This is not an error and may happen for example when a message is retransmitted.

6.80.2.49 `#define RTPS_LOG_INTERFACE_NOT_ENABLED_EC (RTPS_LOG_BASE + 53)`

Failed to receive a message because interface is not enabled.

6.80.2.50 `#define RTPS_LOG_INVALID_PACKET_EC (RTPS_LOG_BASE + 54)`

Dropped a message with an invalid packet header.

6.80.2.51 `#define RTPS_LOG_UNSUPPORTED_INTERFACE_EC (RTPS_LOG_BASE + 55)`

Received a message on an unsupported interface.

6.80.2.52 `#define RTPS_LOG_UNKNOWN_SUBMESSAGE_EC (RTPS_LOG_BASE + 56)`

Received a submessage with an unknown ID.

6.80.2.53 `#define RTPS_LOG_PROCESS_ACKNACK_EC (RTPS_LOG_BASE + 57)`

Failed to process received ACKNACK submessage.

6.80.2.54 `#define RTPS_LOG_PROCESS_DATA_EC (RTPS_LOG_BASE + 58)`

Failed to process received DATA submessage.

6.80.2.55 `#define RTPS_LOG_PROCESS_GAP_EC (RTPS_LOG_BASE + 59)`

Failed to process received GAP submessage.

6.80.2.56 `#define RTPS_LOG_PROCESS_HEARTBEAT_EC (RTPS_LOG_BASE + 60)`

Failed to process received HEARTBEAT submessage.

6.80.2.57 `#define RTPS_LOG_CREATE_HB_EVENT_EC (RTPS_LOG_BASE + 61)`

Failed to create periodic HEARTBEAT event.

6.80.2.58 `#define RTPS_LOG_CREATE_EVENT_TIMER_EC (RTPS_LOG_BASE + 62)`

Failed to create periodic HEARTBEAT event timer.

6.80.2.59 `#define RTPS_LOG_SEQ_NUM_OUT_OF_RANGE_EC (RTPS_LOG_BASE + 63)`

Failed to set bitmap bit due to out-of-range sequence number.

6.80.2.60 `#define RTPS_LOG_FIRST_SN_GREATER_LAST_SN_EC (RTPS_LOG_BASE + 65)`

Invalid range of sequence numbers to fill in bitmap.

6.80.2.61 `#define RTPS_LOG_BITCOUNT_OUT_OF_BOUNDS_EC (RTPS_LOG_BASE + 66)`

Deserialized an invalid out-of-bounds bitcount for a bitmap.

6.80.2.62 `#define RTPS_LOG_SHIFT_SN_OUT_OF_RANGE_EC (RTPS_LOG_BASE + 67)`

Failed to shift bitmap due to invalid starting sequence number.

6.80.2.63 `#define RTPS_LOG_SERIALIZE_HOST_ID_EC (RTPS_LOG_BASE + 68)`

Failed to serialize host ID of GUID.

6.80.2.64 `#define RTPS_LOG_SERIALIZE_APP_ID_EC (RTPS_LOG_BASE + 69)`

Failed to serialize app ID of GUID.

6.80.2.65 `#define RTPS_LOG_SERIALIZE_INSTANCE_ID_EC (RTPS_LOG_BASE + 70)`

Failed to serialized instance ID of GUID.

6.80.2.66 `#define RTPS_LOG_SERIALIZE_OBJECT_ID_EC (RTPS_LOG_BASE + 71)`

Failed to serialize object ID of GUID.

6.80.2.67 `#define RTPS_LOG_DESERIALIZE_HOST_ID_EC (RTPS_LOG_BASE + 72)`

Failed to deserialize host ID of GUID.

6.80.2.68 `#define RTPS_LOG_DESERIALIZE_APP_ID_EC (RTPS_LOG_BASE + 73)`

Failed to deserialize app ID of GUID.

6.80.2.69 `#define RTPS_LOG_DESERIALIZE_INSTANCE_ID_EC (RTPS_LOG_BASE + 74)`

Failed to deserialize instance ID of GUID.

6.80.2.70 `#define RTPS_LOG_DESERIALIZE_OBJECT_ID_EC (RTPS_LOG_BASE + 75)`

Failed to deserialize object ID of GUID.

6.80.2.71 `#define RTPS_LOG_DB_CREATE_EXT_BIND_ENTRY_EC (RTPS_LOG_BASE + 76)`

Failed to create database entry for external bind table.

If DomainParticipant, may have exceeded DomainParticipantQos.resource_limits.matching_reader_writer_pair_allocation.

6.80.2.72 `#define RTPS_LOG_BITMAP_SET_BIT_EC (RTPS_LOG_BASE + 77)`

Failed to set bit in bitmap.

6.80.2.73 `#define RTPS_LOG_UNSUPPORTED_INFO_REPLY_EC (RTPS_LOG_BASE + 78)`

Received and dropped currently unsupported INFO_REPLY submessage.

6.80.2.74 `#define RTPS_LOG_UNSUPPORTED_INFO_REPLY_IP4_EC (RTPS_LOG_BASE + 79)`

Received and dropped currently unsupported INFO_REPLY_IP4 submessage.

6.80.2.75 `#define RTPS_LOG_UNSUPPORTED_INFO_SRC_EC (RTPS_LOG_BASE + 80)`

Received and dropped currently unsupported INFO_SRC submessage.

6.80.2.76 `#define RTPS_LOG_DELETE_INDEX_EC (RTPS_LOG_BASE + 81)`

Failed to delete index of a database table.

6.80.2.77 `#define RTPS_LOG_DELETE_TABLE_EC (RTPS_LOG_BASE + 82)`

Failed to delete a database table.

6.80.2.78 `#define RTPS_LOG_DB_LOCK_EC (RTPS_LOG_BASE + 83)`

Failed to take database lock.

6.80.2.79 `#define RTPS_LOG_DB_UNLOCK_EC (RTPS_LOG_BASE + 84)`

Failed to give database lock.

6.80.2.80 `#define RTPS_LOG_CREATE_BIND_TABLE_EC (RTPS_LOG_BASE + 85)`

Failed to create a database table for storing RTPS bind info.

6.80.2.81 `#define RTPS_LOG_STATUS_CHANGE_EC (RTPS_LOG_BASE + 86)`

Failed to update status for reliable reader activity changed.

6.80.2.82 `#define RTPS_LOG_SET_GAP_EC (RTPS_LOG_BASE + 87)`

Failed to update status for reliable reader activity changed.

6.80.2.83 `#define RTPS_LOG_WINDOW_POOL_ALLOC_EC (RTPS_LOG_BASE + 88)`

Out of memory to allocate send window pool.

6.80.2.84 `#define RTPS_LOG_GET_WINDOW_BUFFER_EC (RTPS_LOG_BASE + 89)`

Failed to get buffer from send window pool.

6.80.2.85 `#define RTPS_LOG_INSERT_WINDOW_FULL_EC (RTPS_LOG_BASE + 90)`

Cannot insert sample into full window.

6.80.2.86 `#define RTPS_LOG_WINDOW_INSERT_EC (RTPS_LOG_BASE + 91)`

Failed to insert sample into send window.

6.80.2.87 `#define RTPS_LOG_TIMER_DELETE_TIMEOUT_EC (RTPS_LOG_BASE + 92)`

Failed to delete a timeout event.

6.80.2.88 `#define RTPS_LOG_BUFFERPOOL_DELETE_EC (RTPS_LOG_BASE + 93)`

Failed to delete a buffer pool.

6.80.2.89 `#define RTPS_LOG_DIRECT_SEND_EC (RTPS_LOG_BASE + 94)`

Failed to send a message to a specific peer.

6.80.2.90 `#define RTPS_LOG_PACKET_INITIALIZE_EC (RTPS_LOG_BASE + 95)`

RTPS writer failed to initialize a packet.

6.80.2.91 `#define RTPS_LOG_WRITER_REQUEST_SENT_HISTORY_EC (RTPS_LOG_BASE + 96)`

RTPS reliable writer failed to request and resend history.

6.80.2.92 `#define RTPS_LOG_BITMAP_SHIFT_EC (RTPS_LOG_BASE + 97)`

Failed to shift bitmap to new lead sequence number.

6.80.2.93 `#define RTPS_LOG_WINDOW_ADVANCE_EC (RTPS_LOG_BASE + 98)`

Send window of a remote reader failed to advance.

6.80.2.94 `#define RTPS_LOG_WINDOW_ADVANCE_UNACKED_EC (RTPS_LOG_BASE + 99)`

Failed when advancing window ahead of unacknowledged sequence number.

6.80.2.95 `#define RTPS_LOG_LOOKUP_PEER_EC (RTPS_LOG_BASE + 100)`

Failed to find peer that should exist.

6.80.2.96 `#define RTPS_LOG_READER_SEND_ACKNACK_EC (RTPS_LOG_BASE + 101)`

RTPS reader failed to send an ACKNACK message.

6.80.2.97 `#define RTPS_LOG_FINALIZE_INTERFACE_EC (RTPS_LOG_BASE + 102)`

Failed to finalize an RTPS interface.

6.80.2.98 `#define RTPS_LOG_INDEXER_REMOVE_ENTRY_EC (RTPS_LOG_BASE + 103)`

Failed to remove index entry for external interface upon deleting RTPS interface.

6.80.2.99 `#define RTPS_LOG_LOOKUP_EXT_BIND_ENTRY_EC (RTPS_LOG_BASE + 104)`

Failed when looking up external bind entry from database.

6.80.2.100 `#define RTPS_LOG_SEQ_INIT_EC (RTPS_LOG_BASE + 105)`

Failed to initialize a local sequence.

6.80.2.101 `#define RTPS_LOG_SEQ_MAX_EC (RTPS_LOG_BASE + 106)`

Failed to set maximum of a local sequence.

6.80.2.102 #define RTPS_LOG_SEQ_LEN_EC (RTPS_LOG_BASE + 107)

Failed to set length of a local sequence.

6.80.2.103 #define RTPS_LOG_INTF_MODE_UNDEF_EC (RTPS_LOG_BASE + 108)

Initializing an RTPS interface with undefined mode.

6.80.2.104 #define RTPS_LOG_DELETE_BUFFER_POOL_EC (RTPS_LOG_BASE + 109)

Failed to delete buffer pool.

May still have buffers in use, thus cannot delete pool.

6.80.2.105 #define RTPS_LOG_CREATE_BUFFER_POOL_EC (RTPS_LOG_BASE + 110)

Failed to create buffer pool.

May have insufficient memory to allocate pool.

6.80.2.106 #define RTPS_LOG_CREATE_PEERS_INDEX_EC (RTPS_LOG_BASE + 111)

Failed to create index of peers.

System may have insufficient memory to allocate new index.

6.80.2.107 #define RTPS_LOG_DELETE_INDEXER_EC (RTPS_LOG_BASE + 112)

Failed to delete indexer.

6.80.2.108 #define RTPS_LOG_GET_PEER_BUFFER_EC (RTPS_LOG_BASE + 113)

Failed to get buffer from peer buffer pool.

6.80.2.109 #define RTPS_LOG_DB_CREATE_DIRECT_INDEX_EC (RTPS_LOG_BASE + 114)

Failed to create database index for direct sends.

6.80.2.110 #define RTPS_LOG_DB_CREATE_GROUP_INDEX_EC (RTPS_LOG_BASE + 115)

Failed to create database index for group sends.

6.80.2.111 #define RTPS_LOG_NETIO_PACKET_INSUFFICIENT_BUFFER_EC (RTPS_LOG_BASE + 116)

Insufficient buffer in packet.

Receive buffer does not have enough space for a valid RTPS message

6.80.2.112 #define RTPS_LOG_PROCESS_DATA_BATCH_EC (RTPS_LOG_BASE + 117)

Failed to process received DATA BATCH submessage.

This indicates an invalid DATA_BATCH submessage

6.80.2.113 #define RTPS_LOG_PROCESS_HEARTBEAT_BATCH_EC (RTPS_LOG_BASE + 118)

Failed to process received HEARTBEAT_BATCH submessage.

This indicates an invalid HEARTBEAT_BATCH submessage

6.80.2.114 #define RTPS_LOG_NEGATIVE_RESERVATION_COUNT_EC (RTPS_LOG_BASE + 119)

The reservation count is zero.

The reservation count should never be less than 0, this is an internal error.

6.80.2.115 `#define RTPS_LOG_DB_CREATE_SELECTED_GROUP_INDEX_EC (RTPS_LOG_BASE + 148)`

Failed to create database index for the selected route group.

6.80.2.116 `#define RTPS_LOG_CHECKSUM_MISSING_CHECKSUM_LENGTH_EC (RTPS_LOG_BASE + 149)`

A header extension was received, but the checksum length is not present.

6.80.2.117 `#define RTPS_LOG_CHECKSUM_UNSUPPORTED_EC (RTPS_LOG_BASE + 150)`

Unsupported checksum bits received, message dropped.

6.80.2.118 `#define RTPS_LOG_CHECKSUM_LENGTH_DESERIALIZE_ERROR_EC (RTPS_LOG_BASE + 151)`

Checksum length could not be deserialized.

6.80.2.119 `#define RTPS_LOG_CHECKSUM_INCONSISTENT_LENGTH_EC (RTPS_LOG_BASE + 152)`

The payload length is less than the deserialized checksum length.

6.80.2.120 `#define RTPS_LOG_CHECKSUM_CHECKSUM_DESERIALIZE_ERROR_EC (RTPS_LOG_BASE + 153)`

Checksum length could not be deserialized.

6.80.2.121 `#define RTPS_LOG_CHECKSUM_CHECKSUM_ERROR_EC (RTPS_LOG_BASE + 154)`

The checksum is invalid.

6.80.2.122 `#define RTPS_LOG_DATA_BATCH_ALREADY_RECEIVED_EC (RTPS_LOG_BASE + 155)`

A DATA_BATCH submessage whose sequence number was previously received was discarded. This is not an error and may happen for example when a message is retransmitted.

6.80.2.123 `#define RTPS_LOG_DATA_SN_OUT_OF_ORDER_EC (RTPS_LOG_BASE + 156)`

Received a DATA submessage with an out-of-order sequence number.

The received sequence number is higher than the lowest, next expected sequence number. This is not an error.

6.80.2.124 `#define RTPS_LOG_DATA_BATCH_SN_OUT_OF_ORDER_EC (RTPS_LOG_BASE + 157)`

Received a DATA_BATCH submessage with an out-of-order sequence number.

The received sequence number is higher than the lowest, next expected sequence number. This is not an error.

6.80.2.125 `#define RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM128_EC (RTPS_LOG_BASE + 200)`

The property tries to override the BUILTIN128 checksum without setting allow_builtin_override to TRUE.

6.80.2.126 `#define RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM32_EC (RTPS_LOG_BASE + 201)`

The property tries to override the BUILTIN32 checksum without setting allow_builtin_override to TRUE.

6.80.2.127 `#define RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM64_EC (RTPS_LOG_BASE + 202)`

The property tries to override the BUILTIN64 checksum without setting allow_builtin_override to TRUE.

6.80.2.128 `#define RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN128_EC (RTPS_LOG_BASE + 203)`

The property override of the BUILTIN128 checksum is invalid. It is not legal to reuse any of the built-in functions.

6.80.2.129 #define RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN32_EC (RTPS_LOG_BASE + 204)

The property override of the BUILTIN32 checksum is invalid. It is not legal to reuse any of the built-in functions.

6.80.2.130 #define RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN64_EC (RTPS_LOG_BASE + 205)

The property override of the BUILTIN64 checksum is invalid. It is not legal to reuse any of the built-in functions.

6.80.2.131 #define RTPS_LOG_CHECKSUM_INVALID_CUSTOM256_EC (RTPS_LOG_BASE + 206)

The property specifies an invalid custom256 checksum function.

6.80.2.132 #define RTPS_LOG_CHECKSUM_INVALID_CUSTOM32_EC (RTPS_LOG_BASE + 207)

The property specifies an invalid custom32 checksum function.

6.80.2.133 #define RTPS_LOG_CHECKSUM_INVALID_CUSTOM64_EC (RTPS_LOG_BASE + 208)

The property specifies an invalid custom64 checksum function.

6.80.2.134 #define RTPS_LOG_CHECKSUM_INVALID_CUSTOM128_EC (RTPS_LOG_BASE + 209)

The property specifies an invalid custom128 checksum function.

6.80.2.135 #define RTPS_LOG_CHECKSUM_CHECKSUM_CALCULATION_FAILED_EC (RTPS_LOG_BASE + 210)

RTPS_Interface_calculate_checksum() failed to calculate the checksum.

6.80.2.136 #define RTPS_LOG_UNSUPPORTED_MANDTORY_HDR_EXT_PID_EC (RTPS_LOG_BASE + 211)

The HeaderExtension contained a mandatory PID that was not understood.

6.80.2.137 #define RTPS_LOG_INVALID_HDR_EXT_PID_LIST_EC (RTPS_LOG_BASE + 212)

An invalid HeaderExtension PID was detected.

6.80.2.138 #define RTPS_LOG_INVALID_HDR_EXT_PID_ERROR_EC (RTPS_LOG_BASE + 213)

Failed to decode the HeaderExtension PID list.

6.80.2.139 #define RTPS_LOG_INCONSISTENT_HDR_EXT_LENGTH_ERROR_EC (RTPS_LOG_BASE + 214)

The HeaderExtension length is inconsistent with the decoded encoded HeaderExtension length.

6.80.2.140 #define RTPS_LOG_MESSAGE_EXCEED_LENGTH_ERROR_EC (RTPS_LOG_BASE + 215)

A RTPS message which was longer than the indicated message length in the RTPS header was received. This causes the rest of the RTPS message to be invalidated and the remaining submessages to be ignored.

6.80.2.141 #define RTPS_LOG_CHECKSUM_INVALID_CRC32_FLAGS_EC (RTPS_LOG_BASE + 216)

The CRC32 submsg flags are invalid.

6.81 RH

Reader History. ModuleID = 8.

Macros

- #define `RHSM_LOG_OUTSTANDING_SAMPLE_EC` (RHSM_LOG_BASE + 1)
Failed to delete Reader History due to outstanding, unremoved sample.
- #define `RHSM_LOG_NO_PROPERTY_QOS_EC` (RHSM_LOG_BASE + 2)
Failed to create Reader History due to unspecified property Qos.
- #define `RHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC` (RHSM_LOG_BASE + 3)
KEEP_ALL History kind is not supported.
- #define `RHSM_LOG_MAX_SAMPLE_TOO_SMALL_EC` (RHSM_LOG_BASE + 4)
DataReaderQos.resource_limits.max_samples set too small.
- #define `RHSM_LOG_TBF_NOT_SUPPORTED_EC` (RHSM_LOG_BASE + 5)
Time-based filtering is not supported.
- #define `RHSM_LOG_SAMPLE_INFO_POOL_EC` (RHSM_LOG_BASE + 6)
Failed to allocate a buffer pool for sample info.
- #define `RHSM_LOG_SAMPLE_POOL_EC` (RHSM_LOG_BASE + 7)
Failed to allocate a buffer pool for sample pointers.
- #define `RHSM_LOG_SAMPLE_PTR_ARRAY_EC` (RHSM_LOG_BASE + 8)
Failed to get sample buffer.
- #define `RHSM_LOG_INFO_ARRAY_EC` (RHSM_LOG_BASE + 9)
Failed to get sample info buffer.
- #define `RHSM_LOG_GET_RECEPTION_TIMESTAMP_EC` (RHSM_LOG_BASE + 10)
Failed to get time for reception timestamp.
- #define `RHSM_LOG_INVALID_INSTANCE_REPLACEMENT_EC` (RHSM_LOG_BASE + 11)
An invalid instance replacement policy was specified.
- #define `RHSM_LOG_FAILED_TO_REMOVE_OLDEST_EC` (RHSM_LOG_BASE + 12)
Failed to remove the oldest sample.
- #define `RHSM_LOG_SAMPLE_POOL_EMPTY_EC` (RHSM_LOG_BASE + 13)
Sample pool empty, may have exceeded.
- #define `RHSM_LOG_RW_PRUNE_FAILED_EC` (RHSM_LOG_BASE + 14)
Failed to prune remote writer entry.
- #define `RHSM_LOG_ENTRY_RESERVATION_FAILED_EC` (RHSM_LOG_BASE + 15)
Failed to reserve an entry.
- #define `RHSM_LOG_RETURN_SAMPLE_EC` (RHSM_LOG_BASE + 16)
Failed to return a sample.
- #define `RHSM_LOG_HISTORY_UPDATE_EC` (RHSM_LOG_BASE + 17)
Failed to update the history queue.
- #define `RHSM_LOG_GET_TIME_EC` (RHSM_LOG_BASE + 18)
Failed to get the system time.
- #define `RHSM_LOG_OBJECT_ALLOCATE_EC` (RHSM_LOG_BASE + 19)
Failed to allocate object of the specified kind.
- #define `RHSM_LOG_OBJECT_DELETE_EC` (RHSM_LOG_BASE + 20)
Failed to delete object of the specified kind.
- #define `RHSM_LOG_OBJECT_INDEX_EC` (RHSM_LOG_BASE + 21)
Failed to index an object of the specified kind.
- #define `RHSM_LOG_OBJECT_INVALID_EC` (RHSM_LOG_BASE + 22)
Invalid object specified.
- #define `RHSM_LOG_NO_PROPERTY_EC` (RHSM_LOG_BASE + 23)

No property when creating a reader history instance.

- `#define RHSM_LOG_DESTINATION_BY_SOURCE_NOT_SUPPORTED_EC` (RHSM_LOG_BASE + 24)

Destination ordering by SOURCE_TIMESTAMP ordering is not supported.

- `#define RHSM_LOG_RECOMMIT_FAILURE_EC` (RHSM_LOG_BASE + 25)

Failed to recommit sample.

6.81.1 Detailed Description

Reader History. ModuleID = 8.

6.81.2 Macro Definition Documentation

6.81.2.1 `#define RHSM_LOG_OUTSTANDING_SAMPLE_EC` (RHSM_LOG_BASE + 1)

Failed to delete Reader History due to outstanding, unremoved sample.

6.81.2.2 `#define RHSM_LOG_NO_PROPERTY_QOS_EC` (RHSM_LOG_BASE + 2)

Failed to create Reader History due to unspecified property Qos.

6.81.2.3 `#define RHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC` (RHSM_LOG_BASE + 3)

KEEP_ALL History kind is not supported.

6.81.2.4 `#define RHSM_LOG_MAX_SAMPLE_TOO_SMALL_EC` (RHSM_LOG_BASE + 4)

DataReaderQos.resource_limits.max_samples set too small.

DataReaderQos.resource_limits.max_samples too small, must be at least DataReaderQos.resource_limits.max_instances * DataReaderQos.resource_limits.max_samples_per_instance

6.81.2.5 `#define RHSM_LOG_TBF_NOT_SUPPORTED_EC` (RHSM_LOG_BASE + 5)

Time-based filtering is not supported.

6.81.2.6 `#define RHSM_LOG_SAMPLE_INFO_POOL_EC` (RHSM_LOG_BASE + 6)

Failed to allocate a buffer pool for sample info.

6.81.2.7 `#define RHSM_LOG_SAMPLE_POOL_EC` (RHSM_LOG_BASE + 7)

Failed to allocate a buffer pool for sample pointers.

6.81.2.8 `#define RHSM_LOG_SAMPLE_PTR_ARRAY_EC` (RHSM_LOG_BASE + 8)

Failed to get sample buffer.

May have exceeded DataReaderQos.reader_resource_limits.max_outstanding_reads

6.81.2.9 `#define RHSM_LOG_INFO_ARRAY_EC` (RHSM_LOG_BASE + 9)

Failed to get sample info buffer.

May have exceeded DataReaderQos.reader_resource_limits.max_outstanding_reads

6.81.2.10 `#define RHSM_LOG_GET_RECEPTION_TIMESTAMP_EC` (RHSM_LOG_BASE + 10)

Failed to get time for reception timestamp.

6.81.2.11 `#define RHSM_LOG_INVALID_INSTANCE_REPLACEMENT_EC (RHSM_LOG_BASE + 11)`

An invalid instance replacement policy was specified.

6.81.2.12 `#define RHSM_LOG_FAILED_TO_REMOVE_OLDEST_EC (RHSM_LOG_BASE + 12)`

Failed to remove the oldest sample.

6.81.2.13 `#define RHSM_LOG_SAMPLE_POOL_EMPTY_EC (RHSM_LOG_BASE + 13)`

Sample pool empty, may have exceeded.

The sample pool is empty, may have exceeded `DataReaderQos.resource_limits.max_samples`

6.81.2.14 `#define RHSM_LOG_RW_PRUNE_FAILED_EC (RHSM_LOG_BASE + 14)`

Failed to prune remote writer entry.

6.81.2.15 `#define RHSM_LOG_ENTRY_RESERVATION_FAILED_EC (RHSM_LOG_BASE + 15)`

Failed to reserve an entry.

An entry reservation failed beyond normal resource settings

6.81.2.16 `#define RHSM_LOG_RETURN_SAMPLE_EC (RHSM_LOG_BASE + 16)`

Failed to return a sample.

6.81.2.17 `#define RHSM_LOG_HISTORY_UPDATE_EC (RHSM_LOG_BASE + 17)`

Failed to update the history queue.

6.81.2.18 `#define RHSM_LOG_GET_TIME_EC (RHSM_LOG_BASE + 18)`

Failed to get the system time.

6.81.2.19 `#define RHSM_LOG_OBJECT_ALLOCATE_EC (RHSM_LOG_BASE + 19)`

Failed to allocate object of the specified kind.

6.81.2.20 `#define RHSM_LOG_OBJECT_DELETE_EC (RHSM_LOG_BASE + 20)`

Failed to delete object of the specified kind.

6.81.2.21 `#define RHSM_LOG_OBJECT_INDEX_EC (RHSM_LOG_BASE + 21)`

Failed to index an object of the specified kind.

6.81.2.22 `#define RHSM_LOG_OBJECT_INVALID_EC (RHSM_LOG_BASE + 22)`

Invalid object specified.

6.81.2.23 `#define RHSM_LOG_NO_PROPERTY_EC (RHSM_LOG_BASE + 23)`

No property when creating a reader history instance.

6.81.2.24 `#define RHSM_LOG_DESTINATION_BY_SOURCE_NOT_SUPPORTED_EC (RHSM_LOG_BASE + 24)`

Destination ordering by `SOURCE_TIMESTAMP` ordering is not supported.

6.81.2.25 `#define RHSM_LOG_RECOMMIT_FAILURE_EC (RHSM_LOG_BASE + 25)`

Failed to recommit sample.

6.82 WH

Writer History. ModuleID = 9.

Macros

- #define `WHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC` (WHSM_LOG_BASE + 1)
KEEP_ALL History kind is not supported.
- #define `WHSM_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC` (WHSM_LOG_BASE + 2)
Unlimited length resource limits unsupported.
- #define `WHSM_LOG_MAX_SAMPLES_TOO_SMALL_EC` (WHSM_LOG_BASE + 3)
DataWriterQos.resource_limits.max_samples set too small.
- #define `WHSM_LOG_OBJECT_ALLOCATE_EC` (WHSM_LOG_BASE + 4)
Failed to allocate object of the specified kind.
- #define `WHSM_LOG_OBJECT_DELETE_EC` (WHSM_LOG_BASE + 5)
Failed to delete object of the specified kind.
- #define `WHSM_LOG_OBJECT_INDEX_EC` (WHSM_LOG_BASE + 6)
Failed to index an object of the specified kind.
- #define `WHSM_LOG_NO_PROPERTY_EC` (WHSM_LOG_BASE + 7)
No property when creating a writer history instance.
- #define `WHSM_LOG_INVALID_INDEX_OBJECT_EC` (WHSM_LOG_BASE + 8)
The sample removed from an index did not match sample's key.

6.82.1 Detailed Description

Writer History. ModuleID = 9.

6.82.2 Macro Definition Documentation

6.82.2.1 #define WHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC (WHSM_LOG_BASE + 1)

`KEEP_ALL` History kind is not supported.

6.82.2.2 #define WHSM_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC (WHSM_LOG_BASE + 2)

Unlimited length resource limits unsupported.

Ensure that `max_samples`, `max_samples_per_instance`, and `max_instances` resource limits are all finite values

6.82.2.3 #define WHSM_LOG_MAX_SAMPLES_TOO_SMALL_EC (WHSM_LOG_BASE + 3)

`DataReaderQos.resource_limits.max_samples` set too small.

`DataReaderQos.resource_limits.max_samples` must be no less than `max_instances * max_samples_per_instance`

6.82.2.4 #define WHSM_LOG_OBJECT_ALLOCATE_EC (WHSM_LOG_BASE + 4)

Failed to allocate object of the specified kind.

6.82.2.5 #define WHSM_LOG_OBJECT_DELETE_EC (WHSM_LOG_BASE + 5)

Failed to delete object of the specified kind.

6.82.2.6 #define WHSM_LOG_OBJECT_INDEX_EC (WHSM_LOG_BASE + 6)

Failed to index an object of the specified kind.

6.82.2.7 `#define WHSM_LOG_NO_PROPERTY_EC (WHSM_LOG_BASE + 7)`

No property when creating a writer history instance.

6.82.2.8 `#define WHSM_LOG_INVALID_INDEX_OBJECT_EC (WHSM_LOG_BASE + 8)`

The sample removed from an index did not match sample's key.

6.83 DPSE

DPSE. ModuleID = 10.

Macros

- #define `DPSE_LOG_INVALID_PEER_ADDRESS_EC` (DPSE_LOG_BASE + 1)
A peer address string specifies an invalid address.
- #define `DPSE_LOG_CREATE_DISCOVERY_PUBLISHER_EC` (DPSE_LOG_BASE + 2)
Failed to create the publisher for a participant discovery writer.
- #define `DPSE_LOG_CREATE_DISCOVERY_SUBSCRIBER_EC` (DPSE_LOG_BASE + 3)
Failed to create the subscriber for a participant discovery datareader.
- #define `DPSE_LOG_REGISTER_TYPE_EC` (DPSE_LOG_BASE + 4)
Failed to register a built-in participant discovery type.
- #define `DPSE_LOG_CREATE_TOPIC_EC` (DPSE_LOG_BASE + 5)
Failed to create a topic for the built-in participant discovery topic.
- #define `DPSE_LOG_CREATE_WRITER_EC` (DPSE_LOG_BASE + 6)
Failed to create a DataWriter for participant discovery.
- #define `DPSE_LOG_CREATE_READER_EC` (DPSE_LOG_BASE + 7)
Failed to create a DataReader for participant discovery.
- #define `DPSE_LOG_DISPOSE_EC` (DPSE_LOG_BASE + 8)
Failed to dispose a participant discovery instance.
- #define `DPSE_LOG_ANNOUNCE_LOCAL_PARTICIPANT_DELETION_EC` (DPSE_LOG_BASE + 9)
Failed to dispose a participant discovery instance upon deletion.
- #define `DPSE_LOG_DELETE_WRITER_EC` (DPSE_LOG_BASE + 10)
Failed to delete a participant discovery DataWriter.
- #define `DPSE_LOG_DELETE_PUBLISHER_EC` (DPSE_LOG_BASE + 11)
Failed to delete a participant discovery Publisher.
- #define `DPSE_LOG_DELETE_READER_EC` (DPSE_LOG_BASE + 12)
Failed to delete a participant discovery DataReader.
- #define `DPSE_LOG_DELETE_TOPIC_EC` (DPSE_LOG_BASE + 13)
Failed to delete a participant discovery Topic.
- #define `DPSE_LOG_DELETE_SUBSCRIBER_EC` (DPSE_LOG_BASE + 14)
Failed to delete a participant discovery Subscriber.
- #define `DPSE_LOG_OBJECT_ALLOCATE_EC` (DPSE_LOG_BASE + 15)
Failed to allocate an object of the specified kind.
- #define `DPSE_LOG_OBJECT_INITIALIZE_EC` (DPSE_LOG_BASE + 16)
Failed to initialize an object of the specified kind.
- #define `DPSE_LOG_OBJECT_FINALIZE_EC` (DPSE_LOG_BASE + 17)
Failed to finalize an object of the specified kind.
- #define `DPSE_LOG_OBJECT_DELETE_EC` (DPSE_LOG_BASE + 18)
Failed to delete an object of the specified kind.
- #define `DPSE_LOG_OBJECT_INVALID_EC` (DPSE_LOG_BASE + 19)
The object was invalid in the context it was used.
- #define `DPSE_LOG_CDR_SET_POSITION_EC` (DPSE_LOG_BASE + 20)
Failed to set the CDR stream position.
- #define `DPSE_LOG_CDR_SERIALIZE_EC` (DPSE_LOG_BASE + 21)
Failed to serialize the specified kind.
- #define `DPSE_LOG_CDR_DESERIALIZE_EC` (DPSE_LOG_BASE + 22)
Failed to deserialize the specified kind.
- #define `DPSE_LOG_SERIALIZE_GUID_EC` (DPSE_LOG_BASE + 23)

- Failed to serialize a GUID key parameter.*

 - #define `DPSE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC` (DPSE_LOG_BASE + 24)
- Failed to serialize a Builtin Endpoint Mask parameter.*

 - #define `DPSE_LOG_SERIALIZE_PROTOCOL_VERSION_EC` (DPSE_LOG_BASE + 25)
- Failed to serialize a Protocol Version parameter.*

 - #define `DPSE_LOG_SERIALIZE_VENDOR_ID_EC` (DPSE_LOG_BASE + 26)
- Failed to serialize a Vendor ID parameter.*

 - #define `DPSE_LOG_SERIALIZE_DEFAULT_UNICAST_EC` (DPSE_LOG_BASE + 27)
- Failed to serialize a Default Unicast Locator parameter.*

 - #define `DPSE_LOG_SERIALIZE_META_UNICAST_EC` (DPSE_LOG_BASE + 28)
- Failed to serialize a Meta Unicast Locator parameter.*

 - #define `DPSE_LOG_SERIALIZE_META_MULTICAST_EC` (DPSE_LOG_BASE + 29)
- Failed to serialize a Meta Multicast Locator parameter.*

 - #define `DPSE_LOG_SERIALIZE_LEASE_DURATION_EC` (DPSE_LOG_BASE + 30)
- Failed to serialize a Lease Duration parameter.*

 - #define `DPSE_LOG_SERIALIZE_PRODUCT_VERSION_EC` (DPSE_LOG_BASE + 31)
- Failed to serialize a Product Version parameter.*

 - #define `DPSE_LOG_DESERIALIZE_PRODUCT_VERSION_EC` (DPSE_LOG_BASE + 32)
- Failed to serialize a Product Version parameter.*

 - #define `DPSE_LOG_SERIALIZE_PARTICIPANT_NAME_EC` (DPSE_LOG_BASE + 33)
- Failed to serialize a Participant Name parameter.*

 - #define `DPSE_LOG_DESERIALIZE_GUID_EC` (DPSE_LOG_BASE + 34)
- Failed to deserialize a GUID key parameter.*

 - #define `DPSE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC` (DPSE_LOG_BASE + 35)
- Failed to deserialize a Builtin Endpoint Mask parameter.*

 - #define `DPSE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC` (DPSE_LOG_BASE + 36)
- Failed to deserialize a Protocol Version parameter.*

 - #define `DPSE_LOG_DESERIALIZE_VENDOR_ID_EC` (DPSE_LOG_BASE + 37)
- Failed to deserialize a Vendor ID parameter.*

 - #define `DPSE_LOG_DESERIALIZE_TOO_MANY_LOCATORS_EC` (DPSE_LOG_BASE + 38)
- Cannot deserialize another locator parameter, having reached maximum of 4 unicast or 4 multicast locators.*

 - #define `DPSE_LOG_DESERIALIZE_PARTICIPANT_NAME_EC` (DPSE_LOG_BASE + 39)
- Failed to deserialize a Participant Name parameter.*

 - #define `DPSE_LOG_DESERIALIZE_UNKNOWN_PID_EC` (DPSE_LOG_BASE + 40)
- Failed to deserialize a parameter with an unknown ID.*

 - #define `DPSE_LOG_ANNOUNCEMENT_EC` (DPSE_LOG_BASE + 41)
- Failed to send a participant discovery announcement.*

 - #define `DPSE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC` (DPSE_LOG_BASE + 42)
- Failed to schedule an event to send the next participant discovery announcement.*

 - #define `DPSE_LOG_ADVANCE_SN_EC` (DPSE_LOG_BASE + 43)
- Failed to advance the sequence number of a participant discovery announcement.*

 - #define `DPSE_LOG_ANNOUNCE_WRITE_EC` (DPSE_LOG_BASE + 44)
- Failed to write a participant discovery announcement message.*

 - #define `DPSE_LOG_SCHEDULE_FAST_ASSERTION_EC` (DPSE_LOG_BASE + 45)
- Failed to schedule an event to send the next participant discovery announcement.*

 - #define `DPSE_LOG_PARTICIPANT_TAKE_EC` (DPSE_LOG_BASE + 46)
- Failed to take a sample from a participant discovery DataReader.*

 - #define `DPSE_LOG_ASSERT_REMOTE_PARTICIPANT_EC` (DPSE_LOG_BASE + 47)
- Failed to assert remote participant.*

 - #define `DPSE_LOG_ON_ASSERT_REMOTE_PARTICIPANT_EC` (DPSE_LOG_BASE + 48)
- Failed to assert and complete discovery of a remote participant.*

- `#define DPSE_LOG_ADD_ANONYMOUS_ROUTE_EC` (DPSE_LOG_BASE + 49)
Failed to add an anonymous route.
- `#define DPSE_LOG_DELETE_ANONYMOUS_ROUTE_EC` (DPSE_LOG_BASE + 50)
Failed to delete an anonymous route.
- `#define DPSE_LOG_MAX_REMOTE_PARTICIPANT_EC` (DPSE_LOG_BASE + 51)
Exceeded resource limit, remote participant allocation.
- `#define DPSE_LOG_REFRESH_REMOTE_PARTICIPANT_EC` (DPSE_LOG_BASE + 52)
Failed to refresh liveliness for a remote participant.
- `#define DPSE_LOG_RESET_REMOTE_PARTICIPANT_EC` (DPSE_LOG_BASE + 53)
Failed to reset liveliness for a remote participant.
- `#define DPSE_LOG_INVALID_DISCOVERY_SAMPLE_EC` (DPSE_LOG_BASE + 54)
Received a participant discovery announcement with invalid state.
- `#define DPSE_LOG_RETURN_DISCOVERY_SAMPLE_EC` (DPSE_LOG_BASE + 55)
Failed to return a loan on a participant discovery announcement.
- `#define DPSE_LOG_SERIALIZE_MULTICAST_EC` (DPSE_LOG_BASE + 56)
Failed to serialize a Multicast Locator parameter.
- `#define DPSE_LOG_GET_DDS_PROPERTIES_EC` (DPSE_LOG_BASE + 57)
Failed to serialize a Multicast Locator parameter.
- `#define DPSE_LOG_ENTITY_ENABLE_EC` (DPSE_LOG_BASE + 58)
Failed to enable the specified entity kind.
- `#define DPSE_LOG_SEQUENCE_SETMAX_EC` (DPSE_LOG_BASE + 59)
Failed to set the maximum length of a sequence of the specified kind.
- `#define DPSE_LOG_SEQUENCE_SETLENGTH_EC` (DPSE_LOG_BASE + 60)
Failed to set the length of a sequence of the specified kind.
- `#define DPSE_LOG_SEQUENCE_GETREF_EC` (DPSE_LOG_BASE + 61)
Failed to get a reference at the specified index for a sequence of the specified kind.
- `#define DPSE_LOG_SEQUENCE_INITIALIZE_EC` (DPSE_LOG_BASE + 62)
Failed to initialize a sequence of the specified kind.
- `#define DPSE_LOG_SEQUENCE_FINALIZE_EC` (DPSE_LOG_BASE + 63)
Failed to finalize a sequence of the specified kind.
- `#define DPSE_LOG_SEQUENCE_COPY_EC` (DPSE_LOG_BASE + 64)
Failed to copy a sequence of the specified kind.
- `#define DPSE_LOG_GET_TIMER_EC` (DPSE_LOG_BASE + 65)
Failed to get timer.
- `#define DPSE_LOG_UNSUPPORTED_LOCATOR_EC` (DPSE_LOG_BASE + 66)
The locator kind is not supported by any transport registered with the domain participant and is dropped.
- `#define DPSE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC` (DPSE_LOG_BASE + 67)
A participant with incompatible checksum was discovered and ignored.

6.83.1 Detailed Description

DPSE. ModuleID = 10.

6.83.2 Macro Definition Documentation

6.83.2.1 `#define DPSE_LOG_INVALID_PEER_ADDRESS_EC` (DPSE_LOG_BASE + 1)

A peer address string specifies an invalid address.

6.83.2.2 `#define DPSE_LOG_CREATE_DISCOVERY_PUBLISHER_EC (DPSE_LOG_BASE + 2)`

Failed to create the publisher for a participant discovery writer.

6.83.2.3 `#define DPSE_LOG_CREATE_DISCOVERY_SUBSCRIBER_EC (DPSE_LOG_BASE + 3)`

Failed to create the subscriber for a participant discovery datareader.

6.83.2.4 `#define DPSE_LOG_REGISTER_TYPE_EC (DPSE_LOG_BASE + 4)`

Failed to register a built-in participant discovery type.

6.83.2.5 `#define DPSE_LOG_CREATE_TOPIC_EC (DPSE_LOG_BASE + 5)`

Failed to create a topic for the built-in participant discovery topic.

6.83.2.6 `#define DPSE_LOG_CREATE_WRITER_EC (DPSE_LOG_BASE + 6)`

Failed to create a DataWriter for participant discovery.

6.83.2.7 `#define DPSE_LOG_CREATE_READER_EC (DPSE_LOG_BASE + 7)`

Failed to create a DataReader for participant discovery.

6.83.2.8 `#define DPSE_LOG_DISPOSE_EC (DPSE_LOG_BASE + 8)`

Failed to dispose a participant discovery instance.

6.83.2.9 `#define DPSE_LOG_ANNOUNCE_LOCAL_PARTICIPANT_DELETION_EC (DPSE_LOG_BASE + 9)`

Failed to dispose a participant discovery instance upon deletion.

6.83.2.10 `#define DPSE_LOG_DELETE_WRITER_EC (DPSE_LOG_BASE + 10)`

Failed to delete a participant discovery DataWriter.

6.83.2.11 `#define DPSE_LOG_DELETE_PUBLISHER_EC (DPSE_LOG_BASE + 11)`

Failed to delete a participant discovery Publisher.

6.83.2.12 `#define DPSE_LOG_DELETE_READER_EC (DPSE_LOG_BASE + 12)`

Failed to delete a participant discovery DataReader.

6.83.2.13 `#define DPSE_LOG_DELETE_TOPIC_EC (DPSE_LOG_BASE + 13)`

Failed to delete a participant discovery Topic.

6.83.2.14 `#define DPSE_LOG_DELETE_SUBSCRIBER_EC (DPSE_LOG_BASE + 14)`

Failed to delete a participant discovery Subscriber.

6.83.2.15 `#define DPSE_LOG_OBJECT_ALLOCATE_EC (DPSE_LOG_BASE + 15)`

Failed to allocate an object of the specified kind.

6.83.2.16 `#define DPSE_LOG_OBJECT_INITIALIZE_EC (DPSE_LOG_BASE + 16)`

Failed to initialize an object of the specified kind.

6.83.2.17 `#define DPSE_LOG_OBJECT_FINALIZE_EC (DPSE_LOG_BASE + 17)`

Failed to finalize an object of the specified kind.

6.83.2.18 `#define DPSE_LOG_OBJECT_DELETE_EC (DPSE_LOG_BASE + 18)`

Failed to delete an object of the specified kind.

6.83.2.19 `#define DPSE_LOG_OBJECT_INVALID_EC (DPSE_LOG_BASE + 19)`

The object was invalid in the context it was used.

6.83.2.20 `#define DPSE_LOG_CDR_SET_POSITION_EC (DPSE_LOG_BASE + 20)`

Failed to set the CDR stream position.

6.83.2.21 `#define DPSE_LOG_CDR_SERIALIZE_EC (DPSE_LOG_BASE + 21)`

Failed to serialize the specified kind.

6.83.2.22 `#define DPSE_LOG_CDR_DESERIALIZE_EC (DPSE_LOG_BASE + 22)`

Failed to deserialize the specified kind.

6.83.2.23 `#define DPSE_LOG_SERIALIZE_GUID_EC (DPSE_LOG_BASE + 23)`

Failed to serialize a GUID key parameter.

6.83.2.24 `#define DPSE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC (DPSE_LOG_BASE + 24)`

Failed to serialize a Builtin Endpoint Mask parameter.

6.83.2.25 `#define DPSE_LOG_SERIALIZE_PROTOCOL_VERSION_EC (DPSE_LOG_BASE + 25)`

Failed to serialize a Protocol Version parameter.

6.83.2.26 `#define DPSE_LOG_SERIALIZE_VENDOR_ID_EC (DPSE_LOG_BASE + 26)`

Failed to serialize a Vendor ID parameter.

6.83.2.27 `#define DPSE_LOG_SERIALIZE_DEFAULT_UNICAST_EC (DPSE_LOG_BASE + 27)`

Failed to serialize a Default Unicast Locator parameter.

6.83.2.28 `#define DPSE_LOG_SERIALIZE_META_UNICAST_EC (DPSE_LOG_BASE + 28)`

Failed to serialize a Meta Unicast Locator parameter.

6.83.2.29 `#define DPSE_LOG_SERIALIZE_META_MULTICAST_EC (DPSE_LOG_BASE + 29)`

Failed to serialize a Meta Multicast Locator parameter.

6.83.2.30 `#define DPSE_LOG_SERIALIZE_LEASE_DURATION_EC (DPSE_LOG_BASE + 30)`

Failed to serialize a Lease Duration parameter.

6.83.2.31 `#define DPSE_LOG_SERIALIZE_PRODUCT_VERSION_EC (DPSE_LOG_BASE + 31)`

Failed to serialize a Product Version parameter.

6.83.2.32 `#define DPSE_LOG_DESERIALIZE_PRODUCT_VERSION_EC (DPSE_LOG_BASE + 32)`

Failed to serialize a Product Version parameter.

6.83.2.33 `#define DPSE_LOG_SERIALIZE_PARTICIPANT_NAME_EC (DPSE_LOG_BASE + 33)`

Failed to serialize a Participant Name parameter.

6.83.2.34 `#define DPSE_LOG_DESERIALIZE_GUID_EC (DPSE_LOG_BASE + 34)`

Failed to deserialize a GUID key parameter.

6.83.2.35 `#define DPSE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC (DPSE_LOG_BASE + 35)`

Failed to deserialize a Builtin Endpoint Mask parameter.

6.83.2.36 `#define DPSE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC (DPSE_LOG_BASE + 36)`

Failed to deserialize a Protocol Version parameter.

6.83.2.37 `#define DPSE_LOG_DESERIALIZE_VENDOR_ID_EC (DPSE_LOG_BASE + 37)`

Failed to deserialize a Vendor ID parameter.

6.83.2.38 `#define DPSE_LOG_DESERIALIZE_TOO_MANY_LOCATORS_EC (DPSE_LOG_BASE + 38)`

Cannot deserialize another locator parameter, having reached maximum of 4 unicast or 4 multicast locators.

6.83.2.39 `#define DPSE_LOG_DESERIALIZE_PARTICIPANT_NAME_EC (DPSE_LOG_BASE + 39)`

Failed to deserialize a Participant Name parameter.

6.83.2.40 `#define DPSE_LOG_DESERIALIZE_UNKNOWN_PID_EC (DPSE_LOG_BASE + 40)`

Failed to deserialize a parameter with an unknown ID.

6.83.2.41 `#define DPSE_LOG_ANNOUNCEMENT_EC (DPSE_LOG_BASE + 41)`

Failed to send a participant discovery announcement.

6.83.2.42 `#define DPSE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC (DPSE_LOG_BASE + 42)`

Failed to schedule an event to send the next participant discovery announcement.

6.83.2.43 `#define DPSE_LOG_ADVANCE_SN_EC (DPSE_LOG_BASE + 43)`

Failed to advance the sequence number of a participant discovery announcement.

6.83.2.44 `#define DPSE_LOG_ANNOUNCE_WRITE_EC (DPSE_LOG_BASE + 44)`

Failed to write a participant discovery announcement message.

6.83.2.45 `#define DPSE_LOG_SCHEDULE_FAST_ASSERTION_EC (DPSE_LOG_BASE + 45)`

Failed to schedule an event to send the next participant discovery announcement.

6.83.2.46 `#define DPSE_LOG_PARTICIPANT_TAKE_EC (DPSE_LOG_BASE + 46)`

Failed to take a sample from a participant discovery DataReader.

6.83.2.47 `#define DPSE_LOG_ASSERT_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 47)`

Failed to assert remote participant.

6.83.2.48 `#define DPSE_LOG_ON_ASSERT_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 48)`

Failed to assert and complete discovery of a remote participant.

6.83.2.49 `#define DPSE_LOG_ADD_ANONYMOUS_ROUTE_EC (DPSE_LOG_BASE + 49)`

Failed to add an anonymous route.

Results in a DomainParticipant not having a destination to which it should have sent participant discovery announcements

6.83.2.50 `#define DPSE_LOG_DELETE_ANONYMOUS_ROUTE_EC (DPSE_LOG_BASE + 50)`

Failed to delete an anonymous route.

6.83.2.51 `#define DPSE_LOG_MAX_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 51)`

Exceeded resource limit, `remote_participant_allocation`.

6.83.2.52 `#define DPSE_LOG_REFRESH_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 52)`

Failed to refresh liveliness for a remote participant.

6.83.2.53 `#define DPSE_LOG_RESET_REMOTE_PARTICIPANT_EC (DPSE_LOG_BASE + 53)`

Failed to reset liveliness for a remote participant.

6.83.2.54 `#define DPSE_LOG_INVALID_DISCOVERY_SAMPLE_EC (DPSE_LOG_BASE + 54)`

Received a participant discovery announcement with invalid state.

6.83.2.55 `#define DPSE_LOG_RETURN_DISCOVERY_SAMPLE_EC (DPSE_LOG_BASE + 55)`

Failed to return a loan on a participant discovery announcement.

6.83.2.56 `#define DPSE_LOG_SERIALIZE_MULTICAST_EC (DPSE_LOG_BASE + 56)`

Failed to serialize a Multicast Locator parameter.

6.83.2.57 `#define DPSE_LOG_GET_DDS_PROPERTIES_EC (DPSE_LOG_BASE + 57)`

Failed to serialize a Multicast Locator parameter.

6.83.2.58 `#define DPSE_LOG_ENTITY_ENABLE_EC (DPSE_LOG_BASE + 58)`

Failed to enable the specified entity kind.

6.83.2.59 `#define DPSE_LOG_SEQUENCE_SETMAX_EC (DPSE_LOG_BASE + 59)`

Failed to set the maximum length of a sequence of the specified kind.

6.83.2.60 `#define DPSE_LOG_SEQUENCE_SETLENGTH_EC (DPSE_LOG_BASE + 60)`

Failed to set the length of a sequence of the specified kind.

6.83.2.61 `#define DPSE_LOG_SEQUENCE_GETREF_EC (DPSE_LOG_BASE + 61)`

Failed to get a reference at the specified index for a sequence of the specified kind.

6.83.2.62 `#define DPSE_LOG_SEQUENCE_INITIALIZE_EC (DPSE_LOG_BASE + 62)`

Failed to initialize a sequence of the specified kind.

6.83.2.63 `#define DPSE_LOG_SEQUENCE_FINALIZE_EC (DPSE_LOG_BASE + 63)`

Failed to finalize a sequence of the specified kind.

6.83.2.64 `#define DPSE_LOG_SEQUENCE_COPY_EC (DPSE_LOG_BASE + 64)`

Failed to copy a sequence of the specified kind.

6.83.2.65 `#define DPSE_LOG_GET_TIMER_EC (DPSE_LOG_BASE + 65)`

Failed to get timer.

6.83.2.66 `#define DPSE_LOG_UNSUPPORTED_LOCATOR_EC (DPSE_LOG_BASE + 66)`

The locator kind is not supported by any transport registered with the domain participant and is dropped.

6.83.2.67 `#define DPSE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC (DPSE_LOG_BASE + 67)`

A participant with incompatible checksum was discovered and ignored.

6.84 DPDE

DPDE. ModuleID = 11.

Macros

- #define `DPDE_LOG_CDR_SET_OFFSET_EC` (DPDE_LOG_BASE + 1)
Failed to set current offset of stream.
- #define `DPDE_LOG_SERIALIZE_ENTITY_NAME_EC` (DPDE_LOG_BASE + 2)
Failed to serialize Entity Name parameter.
- #define `DPDE_LOG_SERIALIZE_TOPIC_NAME_EC` (DPDE_LOG_BASE + 3)
Failed to serialize Topic Name parameter.
- #define `DPDE_LOG_SERIALIZE_TYPE_NAME_EC` (DPDE_LOG_BASE + 4)
Failed to serialize Type Name parameter.
- #define `DPDE_LOG_SERIALIZE_GUID_EC` (DPDE_LOG_BASE + 5)
Failed to serialize GUID key.
- #define `DPDE_LOG_SERIALIZE_DEFAULT_UNICAST_EC` (DPDE_LOG_BASE + 6)
Failed to serialize Default Unicast Locator parameter.
- #define `DPDE_LOG_SERIALIZE_PROTOCOL_VERSION_EC` (DPDE_LOG_BASE + 7)
Failed to serialize Protocol Version parameter.
- #define `DPDE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC` (DPDE_LOG_BASE + 8)
Failed to deserialize Protocol Version parameter.
- #define `DPDE_LOG_DESERIALIZE_VENDOR_ID_EC` (DPDE_LOG_BASE + 9)
Failed to deserialize Vendor ID parameter.
- #define `DPDE_LOG_SERIALIZE_VENDOR_ID_EC` (DPDE_LOG_BASE + 10)
Failed to serialize Vendor ID parameter.
- #define `DPDE_LOG_SERIALIZE_PRODUCT_VERSION_EC` (DPDE_LOG_BASE + 11)
Failed to serialize Product Version parameter.
- #define `DPDE_LOG_SERIALIZE_LEASE_DURATION_EC` (DPDE_LOG_BASE + 12)
Failed to serialize Lease Duration parameter.
- #define `DPDE_LOG_CREATE_PUBLISHER_EC` (DPDE_LOG_BASE + 13)
Failed to create Publisher for built-in endpoint DataWriters.
- #define `DPDE_LOG_CREATE_SUBSCRIBER_EC` (DPDE_LOG_BASE + 14)
Failed to create Subscriber for built-in endpoint DataReaders.
- #define `DPDE_LOG_CREATE_PARTICIPANT_DISCOVERY_EC` (DPDE_LOG_BASE + 15)
Failed to create built-in endpoint for participant discovery after creating local user DomainParticipant.
- #define `DPDE_LOG_CREATE_PUBLICATION_DISCOVERY_EC` (DPDE_LOG_BASE + 16)
Failed to create built-in publication endpoint after creating local user DomainParticipant.
- #define `DPDE_LOG_CREATE_SUBSCRIPTION_DISCOVERY_EC` (DPDE_LOG_BASE + 17)
Failed to create built-in subscription endpoint after creating local DomainParticipant.
- #define `DPDE_LOG_DELETE_SUBSCRIPTION_TOPIC_EC` (DPDE_LOG_BASE + 18)
Failed to delete subscription built-in topic.
- #define `DPDE_LOG_DELETE_PUBLICATION_TOPIC_EC` (DPDE_LOG_BASE + 19)
Failed to delete publication built-in topic.
- #define `DPDE_LOG_DELETE_PARTICIPANT_TOPIC_EC` (DPDE_LOG_BASE + 20)
Failed to delete participant built-in topic.
- #define `DPDE_LOG_DISPOSE_PARTICIPANT_EC` (DPDE_LOG_BASE + 21)
Failed to dispose built-in participant when deleting DomainParticipant.
- #define `DPDE_LOG_ALLOCATE_DPDE_EC` (DPDE_LOG_BASE + 22)
Out of memory to allocate discovery plugin.
- #define `DPDE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC` (DPDE_LOG_BASE + 23)

- Failed to serialize Builtin Endpoint Mask parameter.*
- #define `DPDE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC` (DPDE_LOG_BASE + 24)
Failed to deserialize Builtin Endpoint Mask parameter.
- #define `DPDE_LOG_DESERIALIZE_UNKNOWN_PID_EC` (DPDE_LOG_BASE + 25)
Deserialized a parameter of unknown ID.
- #define `DPDE_LOG_ANNOUNCEMENT_EC` (DPDE_LOG_BASE + 26)
Failed to write a dynamic participant discovery message.
- #define `DPDE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC` (DPDE_LOG_BASE + 27)
Failed to schedule an event to assert the next participant discovery announcement.
- #define `DPDE_LOG_ADVANCE_SN_EC` (DPDE_LOG_BASE + 28)
Failed to advance the sequence number of a participant discovery announcement.
- #define `DPDE_LOG_ANNOUNCE_WRITE_EC` (DPDE_LOG_BASE + 29)
Failed to write initial participant discovery announcement.
- #define `DPDE_LOG_SCHEDULE_FAST_ASSERTION_EC` (DPDE_LOG_BASE + 30)
Failed to schedule event for asserting participant discovery announcements.
- #define `DPDE_LOG_REGISTER_TYPE_EC` (DPDE_LOG_BASE + 32)
Failed to register a built-in type for discovery.
- #define `DPDE_LOG_CREATE_TOPIC_EC` (DPDE_LOG_BASE + 33)
Failed to create a topic for discovery.
- #define `DPDE_LOG_CREATE_WRITER_EC` (DPDE_LOG_BASE + 34)
Failed to create a built-in DataWriter for discovery.
- #define `DPDE_LOG_CREATE_READER_EC` (DPDE_LOG_BASE + 35)
Failed to create a built-in DataReader for discovery.
- #define `DPDE_LOG_ASSERT_REMOTE_PARTICIPANT_EC` (DPDE_LOG_BASE + 36)
Failed to assert and complete discovery of a remote participant.
- #define `DPDE_LOG_ENABLE_REMOTE_PARTICIPANT_EC` (DPDE_LOG_BASE + 37)
Failed to enable and complete discovery of a remote participant.
- #define `DPDE_LOG_REFRESH_REMOTE_PARTICIPANT_EC` (DPDE_LOG_BASE + 38)
Failed to refresh liveliness for a discovered remote participant.
- #define `DPDE_LOG_TAKE_PARTICIPANT_SAMPLE_EC` (DPDE_LOG_BASE + 39)
Failed to take a sample from a participant discovery DataReader.
- #define `DPDE_LOG_INVALID_PARTICIPANT_SAMPLE_EC` (DPDE_LOG_BASE + 40)
Received a participant discovery announcement with invalid state.
- #define `DPDE_LOG_RETURN_PARTICIPANT_SAMPLE_EC` (DPDE_LOG_BASE + 41)
Failed to return a loan on a participant discovery sample.
- #define `DPDE_LOG_DISPOSE_PUBLICATION_EC` (DPDE_LOG_BASE + 42)
Failed to dispose an instance for a publication.
- #define `DPDE_LOG_DISPOSE_SUBSCRIPTION_EC` (DPDE_LOG_BASE + 43)
Failed to dispose an instance of a subscription.
- #define `DPDE_LOG_ASSERT_REMOTE_PUBLICATION_EC` (DPDE_LOG_BASE + 44)
Failed to assert and complete discovery of a remote publication.
- #define `DPDE_LOG_ASSERT_REMOTE_SUBSCRIPTION_EC` (DPDE_LOG_BASE + 45)
Failed to assert and complete discovery of a remote subscription.
- #define `DPDE_LOG_TAKE_PUBLICATION_SAMPLE_EC` (DPDE_LOG_BASE + 46)
Failed to take a sample from a publication discovery DataReader.
- #define `DPDE_LOG_REMOVE_REMOTE_PUBLICATION_EC` (DPDE_LOG_BASE + 47)
Failed to dispose or unregister a remote publication instance.
- #define `DPDE_LOG_INVALID_PUBLICATION_SAMPLE_EC` (DPDE_LOG_BASE + 48)
Received a publication discovery announcement with invalid state.
- #define `DPDE_LOG_RETURN_PUBLICATION_SAMPLE_EC` (DPDE_LOG_BASE + 49)
Failed to return a loan on a publication discovery sample.

- `#define DPDE_LOG_TAKE_SUBSCRIPTION_SAMPLE_EC (DPDE_LOG_BASE + 51)`
Failed to take a sample from a subscription discovery DataReader.
- `#define DPDE_LOG_INVALID_SUBSCRIPTION_SAMPLE_EC (DPDE_LOG_BASE + 52)`
Received a subscription discovery sample with invalid state.
- `#define DPDE_LOG_RETURN_SUBSCRIPTION_SAMPLE_EC (DPDE_LOG_BASE + 53)`
Failed to return a loan on a subscription discovery sample.
- `#define DPDE_LOG_REMOVE_REMOTE_SUBSCRIPTION_EC (DPDE_LOG_BASE + 54)`
Failed to dispose or unregister a remote subscription instance.
- `#define DPDE_LOG_LOCATORS_FULL_EC (DPDE_LOG_BASE + 55)`
A locator sequence is full, the remaining locators of the specified kind are dropped.
- `#define DPDE_LOG_DESERIALIZE_LOCATOR_EC (DPDE_LOG_BASE + 56)`
Failed to deserialize a locator of the specified kind.
- `#define DPDE_LOG_UNSUPPORTED_LOCATOR_EC (DPDE_LOG_BASE + 57)`
The locator kind is not supported by any transport registered with the domain participant and is dropped.
- `#define DPDE_LOG_ADD_PEER_EC (DPDE_LOG_BASE + 58)`
Failed to add the specified entity as a peer.
- `#define DPDE_LOG_REMOVE_PEER_EC (DPDE_LOG_BASE + 59)`
Failed to add the specified entity as a peer.
- `#define DPDE_LOG_SERIALIZE_PRESENTATION_EC (DPDE_LOG_BASE + 60)`
Failed to serialize Presentation parameter.
- `#define DPDE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC (DPDE_LOG_BASE + 61)`
A participant with incompatible checksum was discovered and ignored.

6.84.1 Detailed Description

DPDE. ModuleID = 11.

6.84.2 Macro Definition Documentation

6.84.2.1 `#define DPDE_LOG_CDR_SET_OFFSET_EC (DPDE_LOG_BASE + 1)`

Failed to set current offset of stream.

6.84.2.2 `#define DPDE_LOG_SERIALIZE_ENTITY_NAME_EC (DPDE_LOG_BASE + 2)`

Failed to serialize Entity Name parameter.

6.84.2.3 `#define DPDE_LOG_SERIALIZE_TOPIC_NAME_EC (DPDE_LOG_BASE + 3)`

Failed to serialize Topic Name parameter.

6.84.2.4 `#define DPDE_LOG_SERIALIZE_TYPE_NAME_EC (DPDE_LOG_BASE + 4)`

Failed to serialize Type Name parameter.

6.84.2.5 `#define DPDE_LOG_SERIALIZE_GUID_EC (DPDE_LOG_BASE + 5)`

Failed to serialize GUID key.

6.84.2.6 `#define DPDE_LOG_SERIALIZE_DEFAULT_UNICAST_EC (DPDE_LOG_BASE + 6)`

Failed to serialize Default Unicast Locator parameter.

6.84.2.7 `#define DPDE_LOG_SERIALIZE_PROTOCOL_VERSION_EC (DPDE_LOG_BASE + 7)`

Failed to serialize Protocol Version parameter.

6.84.2.8 `#define DPDE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC (DPDE_LOG_BASE + 8)`

Failed to deserialize Protocol Version parameter.

6.84.2.9 `#define DPDE_LOG_DESERIALIZE_VENDOR_ID_EC (DPDE_LOG_BASE + 9)`

Failed to deserialize Vendor ID parameter.

6.84.2.10 `#define DPDE_LOG_SERIALIZE_VENDOR_ID_EC (DPDE_LOG_BASE + 10)`

Failed to serialize Vendor ID parameter.

6.84.2.11 `#define DPDE_LOG_SERIALIZE_PRODUCT_VERSION_EC (DPDE_LOG_BASE + 11)`

Failed to serialize Product Version parameter.

6.84.2.12 `#define DPDE_LOG_SERIALIZE_LEASE_DURATION_EC (DPDE_LOG_BASE + 12)`

Failed to serialize Lease Duration parameter.

6.84.2.13 `#define DPDE_LOG_CREATE_PUBLISHER_EC (DPDE_LOG_BASE + 13)`

Failed to create Publisher for built-in endpoint DataWriters.

6.84.2.14 `#define DPDE_LOG_CREATE_SUBSCRIBER_EC (DPDE_LOG_BASE + 14)`

Failed to create Subscriber for built-in endpoint DataReaders.

6.84.2.15 `#define DPDE_LOG_CREATE_PARTICIPANT_DISCOVERY_EC (DPDE_LOG_BASE + 15)`

Failed to create built-in endpoint for participant discovery after creating local user DomainParticipant.

6.84.2.16 `#define DPDE_LOG_CREATE_PUBLICATION_DISCOVERY_EC (DPDE_LOG_BASE + 16)`

Failed to create built-in publication endpoint after creating local user DomainParticipant.

6.84.2.17 `#define DPDE_LOG_CREATE_SUBSCRIPTION_DISCOVERY_EC (DPDE_LOG_BASE + 17)`

Failed to create built-in subscription endpoint after creating local DomainParticipant.

6.84.2.18 `#define DPDE_LOG_DELETE_SUBSCRIPTION_TOPIC_EC (DPDE_LOG_BASE + 18)`

Failed to delete subscription built-in topic.

6.84.2.19 `#define DPDE_LOG_DELETE_PUBLICATION_TOPIC_EC (DPDE_LOG_BASE + 19)`

Failed to delete publication built-in topic.

6.84.2.20 `#define DPDE_LOG_DELETE_PARTICIPANT_TOPIC_EC (DPDE_LOG_BASE + 20)`

Failed to delete participant built-in topic.

6.84.2.21 `#define DPDE_LOG_DISPOSE_PARTICIPANT_EC (DPDE_LOG_BASE + 21)`

Failed to dispose built-in participant when deleting DomainParticipant.

6.84.2.22 `#define DPDE_LOG_ALLOCATE_DPDE_EC (DPDE_LOG_BASE + 22)`

Out of memory to allocate discovery plugin.

6.84.2.23 `#define DPDE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC (DPDE_LOG_BASE + 23)`

Failed to serialize Builtin Endpoint Mask parameter.

6.84.2.24 `#define DPDE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC (DPDE_LOG_BASE + 24)`

Failed to deserialize Builtin Endpoint Mask parameter.

6.84.2.25 `#define DPDE_LOG_DESERIALIZE_UNKNOWN_PID_EC (DPDE_LOG_BASE + 25)`

Deserialized a parameter of unknown ID.

6.84.2.26 `#define DPDE_LOG_ANNOUNCEMENT_EC (DPDE_LOG_BASE + 26)`

Failed to write a dynamic participant discovery message.

6.84.2.27 `#define DPDE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC (DPDE_LOG_BASE + 27)`

Failed to schedule an event to assert the next participant discovery announcement.

6.84.2.28 `#define DPDE_LOG_ADVANCE_SN_EC (DPDE_LOG_BASE + 28)`

Failed to advance the sequence number of a participant discovery announcement.

6.84.2.29 `#define DPDE_LOG_ANNOUNCE_WRITE_EC (DPDE_LOG_BASE + 29)`

Failed to write initial participant discovery announcement.

6.84.2.30 `#define DPDE_LOG_SCHEDULE_FAST_ASSERTION_EC (DPDE_LOG_BASE + 30)`

Failed to schedule event for asserting participant discovery announcements.

6.84.2.31 `#define DPDE_LOG_REGISTER_TYPE_EC (DPDE_LOG_BASE + 32)`

Failed to register a built-in type for discovery.

6.84.2.32 `#define DPDE_LOG_CREATE_TOPIC_EC (DPDE_LOG_BASE + 33)`

Failed to create a topic for discovery.

6.84.2.33 `#define DPDE_LOG_CREATE_WRITER_EC (DPDE_LOG_BASE + 34)`

Failed to create a built-in DataWriter for discovery.

6.84.2.34 `#define DPDE_LOG_CREATE_READER_EC (DPDE_LOG_BASE + 35)`

Failed to create a built-in DataReader for discovery.

6.84.2.35 `#define DPDE_LOG_ASSERT_REMOTE_PARTICIPANT_EC (DPDE_LOG_BASE + 36)`

Failed to assert and complete discovery of a remote participant.

6.84.2.36 `#define DPDE_LOG_ENABLE_REMOTE_PARTICIPANT_EC (DPDE_LOG_BASE + 37)`

Failed to enable and complete discovery of a remote participant.

6.84.2.37 `#define DPDE_LOG_REFRESH_REMOTE_PARTICIPANT_EC (DPDE_LOG_BASE + 38)`

Failed to refresh liveliness for a discovered remote participant.

6.84.2.38 `#define DPDE_LOG_TAKE_PARTICIPANT_SAMPLE_EC (DPDE_LOG_BASE + 39)`

Failed to take a sample from a participant discovery DataReader.

6.84.2.39 `#define DPDE_LOG_INVALID_PARTICIPANT_SAMPLE_EC (DPDE_LOG_BASE + 40)`

Received a participant discovery announcement with invalid state.

6.84.2.40 `#define DPDE_LOG_RETURN_PARTICIPANT_SAMPLE_EC (DPDE_LOG_BASE + 41)`

Failed to return a loan on a participant discovery sample.

6.84.2.41 `#define DPDE_LOG_DISPOSE_PUBLICATION_EC (DPDE_LOG_BASE + 42)`

Failed to dispose an instance for a publication.

6.84.2.42 `#define DPDE_LOG_DISPOSE_SUBSCRIPTION_EC (DPDE_LOG_BASE + 43)`

Failed to dispose an instance of a subscription.

6.84.2.43 `#define DPDE_LOG_ASSERT_REMOTE_PUBLICATION_EC (DPDE_LOG_BASE + 44)`

Failed to assert and complete discovery of a remote publication.

6.84.2.44 `#define DPDE_LOG_ASSERT_REMOTE_SUBSCRIPTION_EC (DPDE_LOG_BASE + 45)`

Failed to assert and complete discovery of a remote subscription.

6.84.2.45 `#define DPDE_LOG_TAKE_PUBLICATION_SAMPLE_EC (DPDE_LOG_BASE + 46)`

Failed to take a sample from a publication discovery DataReader.

6.84.2.46 `#define DPDE_LOG_REMOVE_REMOTE_PUBLICATION_EC (DPDE_LOG_BASE + 47)`

Failed to dispose or unregister a remote publication instance.

6.84.2.47 `#define DPDE_LOG_INVALID_PUBLICATION_SAMPLE_EC (DPDE_LOG_BASE + 48)`

Received a publication discovery announcement with invalid state.

6.84.2.48 `#define DPDE_LOG_RETURN_PUBLICATION_SAMPLE_EC (DPDE_LOG_BASE + 49)`

Failed to return a loan on a publication discovery sample.

6.84.2.49 `#define DPDE_LOG_TAKE_SUBSCRIPTION_SAMPLE_EC (DPDE_LOG_BASE + 51)`

Failed to take a sample from a subscription discovery DataReader.

6.84.2.50 `#define DPDE_LOG_INVALID_SUBSCRIPTION_SAMPLE_EC (DPDE_LOG_BASE + 52)`

Received a subscription discovery sample with invalid state.

6.84.2.51 `#define DPDE_LOG_RETURN_SUBSCRIPTION_SAMPLE_EC (DPDE_LOG_BASE + 53)`

Failed to return a loan on a subscription discovery sample.

6.84.2.52 `#define DPDE_LOG_REMOVE_REMOTE_SUBSCRIPTION_EC (DPDE_LOG_BASE + 54)`

Failed to dispose or unregister a remote subscription instance.

6.84.2.53 `#define DPDE_LOG_LOCATORS_FULL_EC (DPDE_LOG_BASE + 55)`

A locator sequence is full, the remaining locators of the specified kind are dropped.

6.84.2.54 `#define DPDE_LOG_DESERIALIZE_LOCATOR_EC (DPDE_LOG_BASE + 56)`

Failed to deserialize a locator of the specified kind.

6.84.2.55 `#define DPDE_LOG_UNSUPPORTED_LOCATOR_EC (DPDE_LOG_BASE + 57)`

The locator kind is not supported by any transport registered with the domain participant and is dropped.

6.84.2.56 `#define DPDE_LOG_ADD_PEER_EC (DPDE_LOG_BASE + 58)`

Failed to add the specified entity as a peer.

6.84.2.57 `#define DPDE_LOG_REMOVE_PEER_EC (DPDE_LOG_BASE + 59)`

Failed to add the specified entity as a peer.

6.84.2.58 `#define DPDE_LOG_SERIALIZE_PRESENTATION_EC (DPDE_LOG_BASE + 60)`

Failed to serialize Presentation parameter.

6.84.2.59 `#define DPDE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC (DPDE_LOG_BASE + 61)`

A participant with incompatible checksum was discovered and ignored.

6.85 Configuring Resource Limits

Modules

- [Dynamic Participant Dynamic Endpoint \(DPDE\)](#)

6.85.1 Detailed Description

6.86 Dynamic Participant Dynamic Endpoint (DPDE)

Modules

- [max_participant_locators](#)
- [max_locators_per_discovered_participant](#)
- [max_samples_per_builtin_endpoint_reader](#)

6.86.1 Detailed Description

The Dynamic Participant Dynamic Endpoint (DPSE) discovery plugin creates one datawriter and one datareader for each of the ParticipantBuiltinTopicData, PublicationBuiltinTopicData, and SubscriptionBuiltinTopicData. The memory for the plugin includes the memory allocated by these datareaders and datawriters. The memory allocated by the properties must be added for total memory usage.

6.87 max_participant_locators

NAME

[DPDE_DiscoveryPluginProperty::max_participant_locators](#)

DESCRIPTION

The DPDE builtin participant topic datawriter allocates max_participant_locators to send participant announcements to. The default value DDS_LENGTH_AUTO calculates max_participant_locators as remote_participant_allocation + (DPDE_MAX_ANON_PARTICIPANT * (peer_length + 1)) where the constant DPDE_MAX_ANON_PARTICIPANT = 6.

A participant locator is defined as a unique index@address to send to where index is an integer ≥ 0 and only applies to unicast addresses. For multicast address the index is 0. This index is the highest participant_id that will receive participant announcements at the address.

The maximum number of addresses to send to is defined by the peer list (either specified in [DDS_Discovery-QosPolicy::initial_peers](#) or by calling [DDSDomainParticipant::add_peer](#)). The maximum number of addresses needed is determined by adding all indices for all unicast addresses and adding one for each multicast address.

If memory is limited it is advised to set this value as low as possible.

6.88 max_locators_per_discovered_participant**NAME**

[DPDE_DiscoveryPluginProperty::max_locators_per_discovered_participant](#)

DESCRIPTION

When a participant discovers another participant it must keep track of which addresses to respond to for each participant. This resource-limit sets the maximum number of addresses which can be saved.

NOTES

If two participants respond with the same addresses, for example multicast, only one address is saved.

6.89 max_samples_per_builtin_endpoint_reader

NAME

[DPDE_DiscoveryPluginProperty::max_samples_per_builtin_endpoint_reader](#)

DESCRIPTION

The builtin endpoint datareaders are reliable and may cache samples received out-of-order to improve discovery performance. This resource-limit determines the maximum number of outstanding samples at any given time.

6.90 Log Codes

Modules

- [DDS_C](#)
DDS C. ModuleID = 7.
- [REDA](#)
Real-time Efficient Data Structures and Algorithms. ModuleID = 1.
- [DB](#)
Database. ModuleID = 2.
- [CDR](#)
CDR. ModuleID = 5.
- [RT](#)
RT. ModuleID = 3.
- [NETIO](#)
Network I/O. ModuleID = 4.
- [RTPS](#)
Real-Time Publish-Subscribe. ModuleID = 6.
- [RH](#)
Reader History. ModuleID = 8.
- [WH](#)
Writer History. ModuleID = 9.
- [DPSE](#)
DPSE. ModuleID = 10.
- [DPDE](#)
DPDE. ModuleID = 11.

6.90.1 Detailed Description

This page contains references to all log codes reported by RTI Connex Micro. Please refer to the [Debugging](#) section in the [User's Manual](#) for how to interpret log messages and [Log API](#) for the Log API. Please note that logging is enabled in the Debug build of RTI Connex Micro Cert.

7 Class Documentation

7.1 DDS_BuiltinTopicKey_t Struct Reference

The key type of the built-in topic types.

Public Attributes

- `DDS_BUILTIN_TOPIC_KEY_TYPE_NATIVE` [value](#) [`DDS_BUILTIN_TOPIC_KEY_TYPE_NATIVE_LENGTH`]

An array of four integers that uniquely represents a remote [DDSEntity](#).

7.1.1 Detailed Description

The key type of the built-in topic types.

Each remote [DDSEntity](#) to be discovered can be uniquely identified by this key. This is the key of all the built-in topic data types.

The value of element `DDS_BUILTIN_TOPIC_KEY_OBJECT_ID` is the object ID of the remote entity. In Dynamic Participant/Static Endpoint discovery, this must be set correctly for each remote [DDSDataReader](#) and [DDSDataWriter](#) that an application intends to discover.

For example, to configure a remote [DDSDataWriter](#) with object ID 100 by setting the key in the [DDS_PublicationBuiltinTopicData](#):

```
remote_pub_data.key.value[DDS_BUILTIN_TOPIC_KEY_OBJECT_ID] = 100;
```

See also

[DDS_ParticipantBuiltinTopicData](#)
[DDS_PublicationBuiltinTopicData](#)
[DDS_SubscriptionBuiltinTopicData](#)

7.1.2 Member Data Documentation

7.1.2.1 `DDS_BUILTIN_TOPIC_KEY_TYPE_NATIVE` `DDS_BuiltinTopicKey_t::value[DDS_BUILTIN_TOPIC_KEY_TYPE_NATIVE_LENGTH]`

An array of four integers that uniquely represents a remote [DDSEntity](#).

7.2 DDS_ChecksumProperty Struct Reference

<<[eXtension](#)>> Type to define the checksum properties of the participant RTI Connex Micro.

Public Attributes

- `DDS_ChecksumKind_t` `computed_crc_kind`
The checksum kind for the computed and sent checksum.
- `DDS_ChecksumKindMask_t` `allowed_crc_mask`
The allowed checksum for checked checksum.
- `DDS_Boolean` `require_crc`
Whether the discovered participant requires checksum or not.

7.2.1 Detailed Description

<<eXtension>> Type to define the checksum properties of the participant RTI Connex Micro.

7.2.2 Member Data Documentation

7.2.2.1 DDS_ChecksumKind_t DDS_ChecksumProperty::computed_crc_kind

The checksum kind for the computed and sent checksum.

7.2.2.2 DDS_ChecksumKindMask_t DDS_ChecksumProperty::allowed_crc_mask

The allowed checksum for checked checksum.

7.2.2.3 DDS_Boolean DDS_ChecksumProperty::require_crc

Whether the discovered participant requires checksum or not.

7.3 DDS_ChecksumProperty_t Struct Reference

<<eXtension>> Type to define the checksum properties of the participant RTI Connex Micro.

Public Attributes

- [DDS_ChecksumKind_t computed_crc_kind](#)
The checksum kind for the computed and sent checksum.
- [DDS_ChecksumKindMask_t allowed_crc_mask](#)
The allowed checksum for checked checksum.
- [DDS_Boolean require_crc](#)
Whether the discovered participant requires checksum or not.

7.3.1 Detailed Description

<<eXtension>> Type to define the checksum properties of the participant RTI Connex Micro.

7.3.2 Member Data Documentation

7.3.2.1 DDS_ChecksumKind_t DDS_ChecksumProperty_t::computed_crc_kind

The checksum kind for the computed and sent checksum.

7.3.2.2 DDS_ChecksumKindMask_t DDS_ChecksumProperty_t::allowed_crc_mask

The allowed checksum for checked checksum.

7.3.2.3 DDS_Boolean DDS_ChecksumProperty_t::require_crc

Whether the discovered participant requires checksum or not.

7.4 DDS_DataReaderInstanceReplacedStatus Struct Reference

<<eXtension>> <<cert>> [DDS_INSTANCE_REPLACED_STATUS](#).

Public Attributes

- [DDS_Long total_count](#)
The total number of instances that have been replaced during the life-span of the [DDSDataReader](#).
- [DDS_Long total_count_change](#)
The incremental number of instances replaced since the last time the listener was called or the status was read.
- [DDS_InstanceHandle_t last_instance_handle](#)
The instance that is being replaced.
- [DDS_InstanceHandle_t last_replacement_instance](#)
Handle to the instance which last replaced an instance.
- [DDS_InstanceHandle_t publication_handle](#)
Handle to the publisher which last replaced an instance.
- [DDS_Long lost_samples](#)
The number of samples which where lost due to the instance being replaced. Note that this number is an approximation because the middle-ware does not.

7.4.1 Detailed Description

[<<eXtension>>](#) [<<cert>>](#) [DDS_INSTANCE_REPLACED_STATUS](#).

A status structure describing the current instance replacement status for the reader.

7.4.2 Member Data Documentation

7.4.2.1 [DDS_Long DDS_DataReaderInstanceReplacedStatus::total_count](#)

The total number of instances that have been replaced during the life-span of the [DDSDataReader](#).

7.4.2.2 [DDS_Long DDS_DataReaderInstanceReplacedStatus::total_count_change](#)

The incremental number of instances replaced since the last time the listener was called or the status was read.

7.4.2.3 [DDS_InstanceHandle_t DDS_DataReaderInstanceReplacedStatus::last_instance_handle](#)

The instance that is being replaced.

7.4.2.4 [DDS_InstanceHandle_t DDS_DataReaderInstanceReplacedStatus::last_replacement_instance](#)

Handle to the instance which last replaced an instance.

7.4.2.5 [DDS_InstanceHandle_t DDS_DataReaderInstanceReplacedStatus::publication_handle](#)

Handle to the publisher which last replaced an instance.

7.4.2.6 [DDS_Long DDS_DataReaderInstanceReplacedStatus::lost_samples](#)

The number of samples which where lost due to the instance being replaced. Note that this number is an approximation because the middle-ware does not.

7.5 DDS_DataReaderProtocolQosPolicy Struct Reference

[<<eXtension>>](#) Protocol that applies only to [DDSDataReader](#) instances.

Public Attributes

- [DDS_UnsignedLong rtps_object_id](#)
The RTPS Object ID.
- struct [DDS_RtpsReliableReaderProtocol_t rtps_reliable_reader](#)
RTPS protocol-related configuration settings for the RTPS reliable reader associated to a [DDSDataReader](#). This parameter only has effect if the reader is configured with [DDS_RELIABLE_RELIABILITY_QOS](#) [DDS_ReliabilityQosPolicyKind](#).

7.5.1 Detailed Description

<<*e*Extension>> Protocol that applies only to [DDSDataReader](#) instances.

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDataReader](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = NO

7.5.2 Member Data Documentation

7.5.2.1 [DDS_UnsignedLong DDS_DataReaderProtocolQosPolicy::rtps_object_id](#)

The RTPS Object ID.

This value is used to determine the RTPS object ID of a data reader according to the DDS-RTPS Interoperability Wire Protocol.

Only the last 3 bytes are used; the most significant byte is ignored.

[default] [DDS_RTPS_AUTO_ID](#)

[range] [0,0x00ffffff]

7.5.2.2 struct [DDS_RtpsReliableReaderProtocol_t DDS_DataReaderProtocolQosPolicy::rtps_reliable_reader](#)

RTPS protocol-related configuration settings for the RTPS reliable reader associated to a [DDSDataReader](#). This parameter only has effect if the reader is configured with [DDS_RELIABLE_RELIABILITY_QOS](#) [DDS_ReliabilityQosPolicyKind](#).

For details, refer to the [DDS_RtpsReliableReaderProtocol_t](#)

[default]

nack_period 50ms seconds;

7.6 [DDS_DataReaderQos](#) Struct Reference

<<*cert*>> QoS policies supported by a [DDSDataReader](#) entity.

Public Attributes

- struct [DDS_DeadlineQosPolicy deadline](#)
Deadline policy, [DEADLINE](#).

- struct [DDS_LivelinessQosPolicy](#) liveliness
Liveliness policy, [LIVELINESS](#).
- struct [DDS_HistoryQosPolicy](#) history
History policy, [HISTORY](#).
- struct [DDS_ResourceLimitsQosPolicy](#) resource_limits
Resource limits policy, [RESOURCE_LIMITS](#).
- struct [DDS_OwnershipQosPolicy](#) ownership
Ownership policy, [OWNERSHIP](#).
- struct [DDS_DurabilityQosPolicy](#) durability
<<eXtension>> Durability policy, [DURABILITY](#).
- struct [DDS_DestinationOrderQosPolicy](#) destination_order
<<eXtension>> Destination Order policy, [DESTINATION_ORDER](#).
- struct [DDS_DataReaderProtocolQosPolicy](#) protocol
<<eXtension>> [DDSDataReader](#) protocol policy, [DATA_READER_PROTOCOL](#).
- struct [DDS_TransportQosPolicy](#) transport
<<eXtension>> Transport policy, [TRANSPORT](#).
- struct [DDS_DataReaderResourceLimitsQosPolicy](#) reader_resource_limits
<<eXtension>> [DDSDataReader](#) resource limits policy, [DataReader Resource Limits](#). This policy is an extension to the DDS standard.

7.6.1 Detailed Description

<<cert>> QoS policies supported by a [DDSDDataReader](#) entity.

The [DDS_DataReaderQos](#) controls the resources used by the [DDSDDataReader](#). Each [DDSDDataReader](#) allocates its own resources, even DataReaders of the same [DDSTopic](#). For this reason is it advised to limit the number of DataReaders per [DDSTopic](#).

7.6.2 Member Data Documentation

7.6.2.1 struct [DDS_DeadlineQosPolicy](#) [DDS_DataReaderQos::deadline](#)

Deadline policy, [DEADLINE](#).

7.6.2.2 struct [DDS_LivelinessQosPolicy](#) [DDS_DataReaderQos::liveliness](#)

Liveliness policy, [LIVELINESS](#).

7.6.2.3 struct [DDS_HistoryQosPolicy](#) [DDS_DataReaderQos::history](#)

History policy, [HISTORY](#).

7.6.2.4 struct [DDS_ResourceLimitsQosPolicy](#) [DDS_DataReaderQos::resource_limits](#)

Resource limits policy, [RESOURCE_LIMITS](#).

7.6.2.5 struct [DDS_OwnershipQosPolicy](#) [DDS_DataReaderQos::ownership](#)

Ownership policy, [OWNERSHIP](#).

7.6.2.6 struct [DDS_DurabilityQosPolicy](#) [DDS_DataReaderQos::durability](#)

<<eXtension>> Durability policy, [DURABILITY](#).

7.6.2.7 struct `DDS_DestinationOrderQosPolicy` `DDS_DataReaderQos::destination_order`

<<*eXtension*>> Destination Order policy, [DESTINATION_ORDER](#).

7.6.2.8 struct `DDS_DataReaderProtocolQosPolicy` `DDS_DataReaderQos::protocol`

<<*eXtension*>> [DDSDataReader](#) protocol policy, [DATA_READER_PROTOCOL](#)

7.6.2.9 struct `DDS_TransportQosPolicy` `DDS_DataReaderQos::transport`

<<*eXtension*>> Transport policy, [TRANSPORT](#).

7.6.2.10 struct `DDS_DataReaderResourceLimitsQosPolicy` `DDS_DataReaderQos::reader_resource_limits`

<<*eXtension*>> [DDSDataReader](#) resource limits policy, [DataReader Resource Limits](#). This policy is an extension to the DDS standard.

7.7 DDS_DataReaderResourceLimitsQosPolicy Struct Reference

<<*cert*>> Resource limits that apply only to [DDSDataReader](#) instances.

Public Attributes

- [DDS_Long max_remote_writers](#)
The maximum number of remote writers for which the reader will maintain state and communication.
- [DDS_Long max_remote_writers_per_instance](#)
The maximum number of remote writers for a unique instance for which the reader will maintain state and communication.
- [DDS_Long max_samples_per_remote_writer](#)
The maximum number of untaken samples from a given remote DataWriter that a DataReader may store.
- [DDS_Long max_outstanding_reads](#)
The maximum number of outstanding read/take calls (or one of the variants) on the same DataReader for which the memory has not been returned by calling [FooDataReader::return_loan](#).
- [DDS_DataReaderResourceLimitsInstanceReplacementKind instance_replacement](#)
- [DDS_Long max_routes_per_writer](#)
The maximum number of routes the reader will send to per matched writer.

7.7.1 Detailed Description

<<*cert*>> Resource limits that apply only to [DDSDataReader](#) instances.

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDataReader](#)

Properties:

[RxO](#) = N/A
[Changeable](#) = NO

7.7.2 Member Data Documentation

7.7.2.1 DDS_Long DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers

The maximum number of remote writers for which the reader will maintain state and communication.

The `DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers` resource-limit limits the maximum number of remote DataWriters the `DDSDataReader` can receive data from regardless of which instance it is.

Resources controlled by this limit can only be reused when a remote `DDSDataWriter` is either deleted or loses liveliness.

[default] 1

[range] [1, 1000000], $\geq$ max_remote_writers_per_instance

7.7.2.2 DDS_Long DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers_per_instance

The maximum number of remote writers for a unique instance for which the reader will maintain state and communication.

The `DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers_per_instance` limits the maximum number of remote `DDSDataWriter` entities a `DDSDataReader` maintains state for per instance. If this resource-limit is lower than the actual number of remote DataWriters, updating an instance only state for up to `DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers_per_instance` is maintained. It is not guaranteed to be the last `DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers_per_instance` though. However, the last remote `DDSDataWriter` to update an instance is *always* saved. When an instance is no longer updated by a known, to the `DDSDataReader` or `DDSDataWriter`, `DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE` state is signaled on the `DDSDataReader`. Thus, if this resource-limit is lower than the actual number of remote DataWriters updating an instance, `DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE` may be signaled even though there may be one or more DataWriters still updating an instance. In this case the instance will go back to the `DDS_ALIVE_INSTANCE_STATE`.

[default] 1

[range] [1, 2147483647], $\leq$ max_remote_writers

7.7.2.3 DDS_Long DDS_DataReaderResourceLimitsQosPolicy::max_samples_per_remote_writer

The maximum number of untaken samples from a given remote DataWriter that a DataReader may store.

This resource-limit limits the maximum number of samples that can be cached per remote `DDSDataWriter` the `DDSDataReader` is receiving data from. This resource-limit is useful in preventing one `DDSDataWriter` from starving another `DDSDataWriter`. NOTE: The samples cached from a `DDSDataWriter` are allocated from the same sample pool controlled by `DDS_ResourceLimitsQosPolicy::max_samples`.

[default] 1

[range] [1, 100000000], $\leq$ `DDS_ResourceLimitsQosPolicy::max_samples`

7.7.2.4 DDS_Long DDS_DataReaderResourceLimitsQosPolicy::max_outstanding_reads

The maximum number of outstanding read/take calls (or one of the variants) on the same DataReader for which the memory has not been returned by calling `FooDataReader::return_loan`.

The `DDS_DataReaderResourceLimitsQosPolicy::max_outstanding_reads` resource-limit limits the maximum number of simultaneous loans an application can make at any given time using the `FooDataReader::read()` and `FooDataReader::take()` APIs.

Typically a value of 1 is sufficient. It is only necessary to increase this value if an application thread will access the same cache before another thread has returned the loan.

[default] 1

[range] [1, 2147483647]

7.7.2.5 DDS_DataReaderResourceLimitsInstanceReplacementKind DDS_DataReaderResourceLimitsQosPolicy::instance_replacement

[default] `DDS_NO_INSTANCE_REPLACEMENT_QOS`

7.7.2.6 DDS_Long DDS_DataReaderResourceLimitsQosPolicy::max_routes_per_writer

The maximum number of routes the reader will send to per matched writer.

Each [DDSDDataReader](#) maintains information about the state of its peer [DDSDDataWriter](#) (those it has matched with). Part of this state is which locators (or destination addresses) it should use to send data to a particular [DDSDDataWriter](#). The [DDS_DataReaderResourceLimitsQosPolicy::max_routes_per_writer](#) resource-limit limits the maximum number of routes that can be saved per [DDSDDataWriter](#).

This resource-limit is shared across all matched [DDSDDataWriter](#) per [DDSDDataReader](#). Thus, if [DDS_DataReaderResourceLimitsQosPolicy::max_remote_writers](#) is 2 and [DDS_DataReaderResourceLimitsQosPolicy::max_routes_per_writer](#) is 4, a total of 8 routes can be saved for both DataWriters. One [DDSDDataWriter](#) may have 6 routes and the other 2.

[default] 4

[range] [1, 2000]

7.8 DDS_DataWriterProtocolQosPolicy Struct Reference

[<<eXtension>>](#) [<<cert>>](#) Protocol that applies only to [DDSDDataWriter](#) instances.

Public Attributes

- [DDS_UnsignedLong rtps_object_id](#)
The RTPS Object ID.
- struct
[DDS_RtpsReliableWriterProtocol_t rtps_reliable_writer](#)
The reliable protocol defined in RTPS.

7.8.1 Detailed Description

[<<eXtension>>](#) [<<cert>>](#) Protocol that applies only to [DDSDDataWriter](#) instances.

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDDataWriter](#)

Properties:

[RxO](#) = N/A
[Changeable](#) = NO

7.8.2 Member Data Documentation

7.8.2.1 DDS_UnsignedLong DDS_DataWriterProtocolQosPolicy::rtps_object_id

The RTPS Object ID.

This value is used to determine the RTPS object ID of a data writer according to the DDS-RTPS Interoperability Wire Protocol.

Only the last 3 bytes are used; the most significant byte is ignored.

[default] [DDS_RTPS_AUTO_ID](#)

[range] [0,0x00ffffff]

7.8.2.2 struct DDS_RtpsReliableWriterProtocol_t DDS_DataWriterProtocolQosPolicy::rtps_reliable_writer

The reliable protocol defined in RTPS.

[default] heartbeat_period 3.0 seconds;

heartbeats_per_max_samples: 1;

max_send_window: minimum(256, ::DDS_HistoryQosPolicy::depth * ::DDS_ResourceLimitsQosPolicy::max_instances);

max_heartbeat_retries: [DDS_LENGTH_UNLIMITED](#);

first_write_sequence_number: {0,1};

7.9 DDS_DataWriterQos Struct Reference

<<[cert](#)>> QoS policies supported by a [DDSDataWriter](#) entity.

Public Attributes

- struct [DDS_DeadlineQosPolicy](#) deadline
Deadline policy, [DEADLINE](#).
- struct [DDS_LivelinessQosPolicy](#) liveliness
Liveliness policy, [LIVELINESS](#).
- struct [DDS_HistoryQosPolicy](#) history
History policy, [HISTORY](#).
- struct [DDS_ResourceLimitsQosPolicy](#) resource_limits
Resource limits policy, [RESOURCE_LIMITS](#).
- struct [DDS_OwnershipQosPolicy](#) ownership
Ownership policy, [OWNERSHIP](#).
- struct [DDS_OwnershipStrengthQosPolicy](#) ownership_strength
Ownership strength policy, [OWNERSHIP_STRENGTH](#).
- struct [DDS_ReliabilityQosPolicy](#) reliability
Reliability policy, [RELIABILITY](#).
- struct [DDS_DurabilityQosPolicy](#) durability
<<[eXtension](#)>> Durability policy, [DURABILITY](#).
- struct [DDS_DestinationOrderQosPolicy](#) destination_order
<<[eXtension](#)>> Destination Order policy, [DESTINATION_ORDER](#).
- struct [DDS_DataWriterProtocolQosPolicy](#) protocol
<<[eXtension](#)>> [DDSDataWriter](#) protocol policy, [DATA_WRITER_PROTOCOL](#).
- struct [DDS_TransportQosPolicy](#) transport
Transport policy, [TRANSPORT](#). Only unicast transports are supported.
- struct [DDS_DataWriterResourceLimitsQosPolicy](#) writer_resource_limits
Writer resource limits policy, [DataWriter Resource Limits](#).

7.9.1 Detailed Description

<<cert>> QoS policies supported by a [DDSDataWriter](#) entity.

The [DDS_DataWriterQos](#) controls the resources allocated by a [DDSDataWriter](#). Each [DDSDataWriter](#) allocates its own resources, even DataWriters of the same [DDSTopic](#). For this reason is it advised to limit the number of DataWriters per topic.

You must set certain members in a consistent manner:

- [DDS_DataWriterQos::DDS_HistoryQosPolicy::depth](#) <= [DDS_DataWriterQos::DDS_ResourceLimitsQosPolicy::max_samples_per_instance](#)
- [DDS_DataWriterQos::DDS_ResourceLimitsQosPolicy::max_samples_per_instance](#) <= [DDS_DataWriterQos::DDS_ResourceLimitsQosPolicy::max_samples](#)

If any of the above are not true, [DDSDataWriter::set_qos](#) and [DDSPublisher::set_default_datawriter_qos](#) will fail with [DDS_RETCODE_INCONSISTENT_POLICY](#) and [DDSPublisher::create_datawriter](#) and will return NULL.

Entity:

[DDSDataWriter](#)

See also

[QoS Policies](#) allowed ranges within each Qos.

7.9.2 Member Data Documentation

7.9.2.1 struct DDS_DeadlineQosPolicy DDS_DataWriterQos::deadline

Deadline policy, [DEADLINE](#).

7.9.2.2 struct DDS_LivelinessQosPolicy DDS_DataWriterQos::liveliness

Liveliness policy, [LIVELINESS](#).

7.9.2.3 struct DDS_HistoryQosPolicy DDS_DataWriterQos::history

History policy, [HISTORY](#).

7.9.2.4 struct DDS_ResourceLimitsQosPolicy DDS_DataWriterQos::resource_limits

Resource limits policy, [RESOURCE_LIMITS](#).

7.9.2.5 struct DDS_OwnershipQosPolicy DDS_DataWriterQos::ownership

Ownership policy, [OWNERSHIP](#).

7.9.2.6 struct DDS_OwnershipStrengthQosPolicy DDS_DataWriterQos::ownership_strength

Ownership strength policy, [OWNERSHIP_STRENGTH](#).

7.9.2.7 struct DDS_ReliabilityQosPolicy DDS_DataWriterQos::reliability

Reliability policy, [RELIABILITY](#).

7.9.2.8 struct DDS_DurabilityQosPolicy DDS_DataWriterQos::durability

<<eXtension>> Durability policy, [DURABILITY](#).

7.9.2.9 struct DDS_DestinationOrderQosPolicy DDS_DataWriterQos::destination_order

<<eXtension>> Destination Order policy, [DESTINATION_ORDER](#).

7.9.2.10 struct DDS_DataWriterProtocolQosPolicy DDS_DataWriterQos::protocol

<<eXtension>> [DDSDataWriter](#) protocol policy, [DATA_WRITER_PROTOCOL](#)

7.9.2.11 struct DDS_TransportQosPolicy DDS_DataWriterQos::transport

Transport policy, [TRANSPORT](#). Only unicast transports are supported.

7.9.2.12 struct DDS_DataWriterResourceLimitsQosPolicy DDS_DataWriterQos::writer_resource_limits

Writer resource limits policy, [DataWriter Resource Limits](#).

7.10 DDS_DataWriterResourceLimitsQosPolicy Struct Reference

<<cert>> Resource limits that apply only to [DDSDataWriter](#) instances.

Public Attributes

- [DDS_Long max_remote_readers](#)
The maximum number of remote readers for which the writer will maintain state and communication.
- [DDS_Long max_routes_per_reader](#)
The maximum number of routes the writer will send to per matched reader.

7.10.1 Detailed Description

<<cert>> Resource limits that apply only to [DDSDataWriter](#) instances.

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDataWriter](#)

Properties:

[RxO](#) = N/A
[Changeable](#) = NO

7.10.2 Member Data Documentation

7.10.2.1 [DDS_Long DDS_DataWriterResourceLimitsQosPolicy::max_remote_readers](#)

The maximum number of remote readers for which the writer will maintain state and communication.

The [DDS_DataWriterResourceLimitsQosPolicy::max_remote_readers](#) resource-limit limits the maximum number of remote DataReaders with which the [DDSDataWriter](#) will maintain state and communication.

[default] 16

[range] [1, 100000000]

7.10.2.2 [DDS_Long DDS_DataWriterResourceLimitsQosPolicy::max_routes_per_reader](#)

The maximum number of routes the writer will send to per matched reader.

Each [DDSDataWriter](#) maintains information about the state of its peer DataReaders (those it has matched with). Part of this state is which locators (or destination addresses) it should use to send data to a particular [DDSDataReader](#). The [DDS_DataWriterResourceLimitsQosPolicy::max_routes_per_reader](#) resource-limit limits the number of routes that can be saved per [DDSDataReader](#).

This resource-limit is shared across all matched DataReaders per [DDSDataWriter](#). Thus, if [DDS_DataWriterResourceLimitsQosPolicy::max_remote_readers](#) is 2 and [DDS_DataWriterResourceLimitsQosPolicy::max_routes_per_reader](#) is 4, a total of 8 routes can be saved for both DataReaders. One [DDSDataReader](#) may have 6 routes and the other 2.

[default] 4

[range] [1, 2000]

7.11 DDS_DeadlineQosPolicy Struct Reference

[`<<cert>>`](#) Expresses the maximum duration (deadline) within which an instance is expected to be updated.

Public Attributes

- struct [DDS_Duration_t](#) `period`
Duration of the deadline period.

7.11.1 Detailed Description

[`<<cert>>`](#) Expresses the maximum duration (deadline) within which an instance is expected to be updated.

[DDSDataReader](#) expects a new sample updating the value of each instance at least once every `period`. [DDS_DataWriter](#) indicates that the application commits to write a new value (using the [DDSDataWriter](#)) for each instance managed by the [DDSDataWriter](#) at least once every `period`.

If a [DDSDataReader](#) stops receiving samples, it will take at most $(\text{period} + (\text{period} / 8))$ to detect that the deadline has been missed. The deadline period starts again when the next sample is received.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS_OFFERED_DEADLINE_MISSED_STATUS](#), [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#), [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#), [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#)

Properties:

[RxO](#) = YES
[Changeable](#) = NO

7.11.2 Usage

This policy is useful for cases where a [DDSTopic](#) is expected to have each instance updated periodically. On the publishing side this setting establishes a contract that the application must meet. On the subscribing side the setting establishes a minimum requirement for the remote publishers that are expected to supply the data values.

When RTI Connext Micro 'matches' a [DDSDataWriter](#) and a [DDSDataReader](#) it checks whether the settings are compatible (i.e., *offered deadline* $\leq$ *requested deadline*); if they are not, the two entities are informed (via the [DDSListener](#) mechanism) of the incompatibility of the QoS settings and communication will not occur.

Assuming that the reader and writer ends have compatible settings, the fulfilment of this contract is monitored by RTI Connext Micro and the application is informed of any violations by means of the proper [DDSListener](#).

7.11.3 Compatibility

The value offered is considered compatible with the value requested if and only if the inequality *offered period* $\leq$ *requested period* evaluates to 'TRUE'.

7.11.4 Member Data Documentation

7.11.4.1 struct DDS_Duration_t DDS_DeadlineQosPolicy::period

Duration of the deadline period.

[default] [DDS_DURATION_INFINITE](#)

[range] [1 nanosec, 1 year] or [DDS_DURATION_INFINITE](#)

7.12 DDS_DestinationOrderQosPolicy Struct Reference

Destination order policies that affect the ordering of subscribed data.

Public Attributes

- [DDS_DestinationOrderQosPolicyKind](#) kind
The kind of destination ordering that applies to subscribed data.
- struct [DDS_Duration_t](#) source_timestamp_tolerance
<<eXtension>> Allowed tolerance between source timestamps of consecutive samples.

7.12.1 Detailed Description

Destination order policies that affect the ordering of subscribed data.

7.12.2 Member Data Documentation

7.12.2.1 DDS_DestinationOrderQosPolicyKind DDS_DestinationOrderQosPolicy::kind

The kind of destination ordering that applies to subscribed data.

[default] [DDS_BY_RECEPTION_TIMESTAMP_DESTINATIONORDER_QOS](#)

7.12.2.2 struct DDS_Duration_t DDS_DestinationOrderQosPolicy::source_timestamp_tolerance

<<eXtension>> Allowed tolerance between source timestamps of consecutive samples.

When a [DDSDataWriter](#) sets [DDS_DestinationOrderQosPolicy](#) to [DDS_BY_SOURCE_TIMESTAMP_DESTINATIONORDER_QOS](#), when writing a sample, its timestamp must not be less than the timestamp of the previously written sample. However, if it is less than the timestamp of the previously written sample but the difference is less than this tolerance, the sample will use the previously written sample's timestamp as its timestamp. Otherwise, if the difference is greater than this tolerance, the write will fail.

[DDS_BY_SOURCE_TIMESTAMP_DESTINATIONORDER_QOS](#) is not support on the [DDSDataReader](#).

[default] 100 milliseconds for [DDSDataWriter](#)

[range] [1 nanosec, 1 year]

7.13 DDS_DiscoveryComponent Struct Reference

Specifies the discovery plugin the [DDSDomainParticipant](#) should use.

Public Attributes

- [RT_ComponentFactoryId_T](#) name
Specifies the name of a registered discovery plugin the [DDSDomainParticipant](#) should use.

7.13.1 Detailed Description

Specifies the discovery plugin the [DDSDomainParticipant](#) should use.

7.13.2 Member Data Documentation

7.13.2.1 RT_ComponentFactoryId_T DDS_DiscoveryComponent::name

Specifies the name of a registered discovery plugin the [DDSDomainParticipant](#) should use.

This attribute specifies the name of the plug-in to be used for DDS Discovery. The maximum length of the name is RT_MAX_FACTORY_NAME excluding the NUL termination.

7.14 DDS_DiscoveryQosPolicy Struct Reference

<<[cert](#)>> Specifies the attributes required to discover participants in the domain.

Public Attributes

- struct DDS_StringSeq [initial_peers](#)
Determines the initial list of peers that will be contacted by the Discovery mechanism to send announcements about the presence of this participant.
- struct DDS_StringSeq [enabled_transports](#)
The transports available for use by the Discovery mechanism.
- struct [DDS_DiscoveryComponent](#) [discovery](#)
The name of the discovery plugin that will be used to create a discovery plugin for this [DDSDomainParticipant](#).
- [DDS_Boolean](#) [accept_unknown_peers](#)
Whether to accept a new participant that is not in the initial peer list.

7.14.1 Detailed Description

<<[cert](#)>> Specifies the attributes required to discover participants in the domain.

Entity:

[DDSDomainParticipant](#)

Properties:

[RxO](#) = N/A
[Changeable](#) = NO

7.14.2 Usage

This extended Qos is used to define how participants find each other on a network.

7.14.3 Member Data Documentation

7.14.3.1 struct DDS_StringSeq DDS_DiscoveryQosPolicy::initial_peers

Determines the initial list of peers that will be contacted by the Discovery mechanism to send announcements about the presence of this participant.

If there is a remote peer [DDSDomainParticipant](#) such as is described this list, it will become aware of this participant and will engage in the Discovery protocol to exchange meta-data with this participant.

Each element of this list must be a `peer` descriptor in the proper format.

[default] Empty sequence

[range] Sequence of arbitrary length.

7.14.3.2 struct DDS_StringSeq DDS_DiscoveryQosPolicy::enabled_transports

The transports available for use by the Discovery mechanism.

Only these transports can be used by the discovery mechanism to send meta-traffic via the builtin endpoints (built-in [DDSDataReader](#) and [DDSDataWriter](#)).

Also determines the IP addresses on which the Discovery mechanism will listen for meta-traffic. These along with the `domain_id` and `participant_id` determine the IP locators on which the Discovery mechanism can receive meta-data.

The memory for the strings in this sequence is managed according to the conventions described in [String Support](#). In particular, be careful to avoid a situation in which RTI Connex Micro allocates a string on your behalf and you then reuse that string in such a way that RTI Connex Micro believes it to have more memory allocated to it than it actually does.

[default] Empty sequence.

When an empty sequence is passed to [DDSDomainParticipantFactory::create_participant](#) in the [DDS_DomainParticipantQos](#), the domain participant will automatically add the following locators to listen for discovery traffic on: `"_udp://"` and `"_udp://239.255.0.1"`

[range] Sequence of non-null, non-empty strings.

7.14.3.3 struct DDS_DiscoveryComponent DDS_DiscoveryQosPolicy::discovery

The name of the discovery plugin that will be used to create a discovery plugin for this [DDSDomainParticipant](#).

The [DDSDomainParticipantFactory](#) will search through a list of registered discovery plugins. If it finds a registered discovery factory with a name matching this factory name, it will use that discovery plugin to create a new instance of the discovery plugin and register the plugin with the [DDSDomainParticipant](#).

If it does not find a registered discovery factory with this name, it will return [DDS_RETCODE_PRECONDITION_NOT_MET](#)

Precondition

One or more discovery plugins must have been previously registered with the [DDSDomainParticipantFactory](#), each with an associated name.

7.14.3.4 DDS_Boolean DDS_DiscoveryQosPolicy::accept_unknown_peers

Whether to accept a new participant that is not in the initial peer list.

If [DDS_BOOLEAN_FALSE](#), the participant will only communicate with those in the initial peer list

If [DDS_BOOLEAN_TRUE](#), the participant will also communicate with all discovered remote participants.

[default] [DDS_BOOLEAN_TRUE](#)

7.15 DDS_DomainParticipantFactoryQos Struct Reference

<<cert>> QoS policies supported by a [DDSDomainParticipantFactory](#).

Public Attributes

- struct [DDS_EntityFactoryQosPolicy](#) `entity_factory`
Entity factory policy, [ENTITY_FACTORY](#).
- struct [DDS_SystemResourceLimitsQosPolicy](#) `resource_limits`
<<eXtension>> System resource limits, [SYSTEM_RESOURCE_LIMITS](#).

7.15.1 Detailed Description

<<cert>> QoS policies supported by a [DDSDomainParticipantFactory](#).

Entity:

[DDSDomainParticipantFactory](#)

See also

[QoS Policies](#) and allowed ranges within each Qos.

7.15.2 Member Data Documentation

7.15.2.1 struct [DDS_EntityFactoryQosPolicy](#) `DDS_DomainParticipantFactoryQos::entity_factory`

Entity factory policy, [ENTITY_FACTORY](#).

7.15.2.2 struct [DDS_SystemResourceLimitsQosPolicy](#) `DDS_DomainParticipantFactoryQos::resource_limits`

<<eXtension>> System resource limits, [SYSTEM_RESOURCE_LIMITS](#).

7.16 DDS_DomainParticipantQos Struct Reference

<<cert>> QoS policies supported by a [DDSDomainParticipant](#) entity.

Public Attributes

- struct [DDS_EntityFactoryQosPolicy](#) `entity_factory`
Entity factory policy, [ENTITY_FACTORY](#).
- struct [DDS_DiscoveryQosPolicy](#) `discovery`
<<eXtension>> Discovery policy, [DISCOVERY](#).
- struct [DDS_DomainParticipantResourceLimitsQosPolicy](#) `resource_limits`
<<eXtension>> Domain participant resource limits policy, [DOMAIN_PARTICIPANT_RESOURCE_LIMITS](#).
- struct [DDS_EntityNameQosPolicy](#) `participant_name`
<<eXtension>> The participant name. [ENTITY_NAME](#)
- struct [DDS_WireProtocolQosPolicy](#) `protocol`
<<eXtension>> Wire Protocol policy, [WIRE_PROTOCOL](#).
- struct [DDS_TransportQosPolicy](#) `transports`
<<eXtension>> Available transports for all communication to and from this DomainParticipant.
- struct [DDS_UserTrafficQosPolicy](#) `user_traffic`
<<eXtension>> Transports enabled for user traffic.

7.16.1 Detailed Description

<<cert>> QoS policies supported by a [DDSDomainParticipant](#) entity.

The QoS policy must be set in a consistent manner, [DDSDomainParticipant::set_qos](#) and [DDSDomainParticipantFactory::set_default_participant_qos](#) will fail with [DDS_RETCODE_INCONSISTENT_POLICY](#), and [DDSDomainParticipantFactory::create_participant](#) will fail.

Entity:

[DDSDomainParticipant](#)

See also

[QoS Policies](#) and allowed ranges within each QoS.

7.16.2 Member Data Documentation

7.16.2.1 struct [DDS_EntityFactoryQosPolicy](#) [DDS_DomainParticipantQos::entity_factory](#)

Entity factory policy, [ENTITY_FACTORY](#).

7.16.2.2 struct [DDS_DiscoveryQosPolicy](#) [DDS_DomainParticipantQos::discovery](#)

<<eXtension>> Discovery policy, [DISCOVERY](#).

7.16.2.3 struct [DDS_DomainParticipantResourceLimitsQosPolicy](#) [DDS_DomainParticipantQos::resource_limits](#)

<<eXtension>> Domain participant resource limits policy, [DOMAIN_PARTICIPANT_RESOURCE_LIMITS](#).

7.16.2.4 struct [DDS_EntityNameQosPolicy](#) [DDS_DomainParticipantQos::participant_name](#)

<<eXtension>> The participant name. [ENTITY_NAME](#)

7.16.2.5 struct [DDS_WireProtocolQosPolicy](#) [DDS_DomainParticipantQos::protocol](#)

<<eXtension>> Wire Protocol policy, [WIRE_PROTOCOL](#).

The wire protocol (RTPS) attributes associated with the participant.

7.16.2.6 struct [DDS_TransportQosPolicy](#) [DDS_DomainParticipantQos::transports](#)

<<eXtension>> Available transports for all communication to and from this DomainParticipant.

7.16.2.7 struct [DDS_UserTrafficQosPolicy](#) [DDS_DomainParticipantQos::user_traffic](#)

<<eXtension>> Transports enabled for user traffic.

7.17 DDS_DomainParticipantResourceLimitsQosPolicy Struct Reference

<<cert>> Resource limits that apply only to [DDSDomainParticipant](#) instances.

Public Attributes

- [DDS_Long local_writer_allocation](#)
The maximum number of local Writers that can be created.
- [DDS_Long local_reader_allocation](#)
The maximum number of local Readers that can be created.

- [DDS_Long local_publisher_allocation](#)
The maximum number of local Publishers that can be created.
- [DDS_Long local_subscriber_allocation](#)
The maximum number of local Subscribers that can be created.
- [DDS_Long local_topic_allocation](#)
The maximum number of topics that can be created.
- [DDS_Long local_type_allocation](#)
The maximum number of local types that can be registered with this DomainParticipant.
- [DDS_Long remote_participant_allocation](#)
The maximum number of remote participants that can be discovered by this DomainParticipant.
- [DDS_Long remote_writer_allocation](#)
The maximum number of remote writers that can be discovered by this DomainParticipant.
- [DDS_Long remote_reader_allocation](#)
The maximum number of remote readers that can be discovered by this DomainParticipant.
- [DDS_Long matching_writer_reader_pair_allocation](#)
The maximum number of matching local writer and remote/local reader pairs.
- [DDS_Long max_receive_ports](#)
Maximum number of addresses that can be listened on.
- [DDS_Long max_destination_ports](#)
Maximum number of unique destination addresses that a participant can reach.

7.17.1 Detailed Description

<<cert>> Resource limits that apply only to [DDSDomainParticipant](#) instances.

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDomainParticipant](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = NO

7.17.2 Member Data Documentation

7.17.2.1 [DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_writer_allocation](#)

The maximum number of local Writers that can be created.

This resource-limit limits the maximum number of [DDSDataWriter](#) entities that can be created with the [DDS-Publisher::create_datawriter\(\)](#) API within a single [DDSDomainParticipant](#) across all [DDSPublisher](#) entities.

[default] 1

7.17.2.2 [DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_reader_allocation](#)

The maximum number of local Readers that can be created.

This resource-limit limits the maximum number of [DDSDataReader](#) entities that can be created with the [DDS-Subscriber::create_datareader\(\)](#) API within a single [DDSDomainParticipant](#) across all [DDSSubscriber](#) entities.

[default] 1

7.17.2.3 DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_publisher_allocation

The maximum number of local Publishers that can be created.

This resource-limit limits the maximum number of [DDSPublisher](#) entities that can be created within a [DDSDomainParticipant](#) with the [DDSDomainParticipant::create_publisher\(\)](#) API.

[default] 1

7.17.2.4 DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_subscriber_allocation

The maximum number of local Subscribers that can be created.

This resource-limit limits the maximum number of [DDSSubscriber](#) entities that can be created within a [DDSDomainParticipant](#) with the [DDSDomainParticipant::create_subscriber\(\)](#) API.

[default] 1

7.17.2.5 DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_topic_allocation

The maximum number of topics that can be created.

This resource-limit limits the maximum number of unique [DDSTopic](#)'s that can be created within a single [DDS-DomainParticipant](#) with the [DDSDomainParticipant::create_topic\(\)](#) API.

[default] 1

7.17.2.6 DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::local_type_allocation

The maximum number of local types that can be registered with this DomainParticipant.

This resource-limit limits the maximum number of unique [NDDS_Type_Plugin](#)'s that can be registered locally within a single [DDSDomainParticipant](#) with the [FooTypeSupport::register_type\(\)](#) API.

[default] 1

7.17.2.7 DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::remote_participant_allocation

The maximum number of remote participants that can be discovered by this DomainParticipant.

This resource-limit limits the maximum number of [DDSDomainParticipant](#)'s that can be discovered by a local [DDS-DomainParticipant](#).

The DPDE discovery plugin is a first discovered first served basis and the only way to limit discovery is careful use of peer addresses. The DPSE discovery plugin limits the discovery to those that have been asserted based on the DomainParticipant name using [DPSEDiscoveryPlugin::RemoteParticipant_assert](#)

[default] 1

7.17.2.8 DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::remote_writer_allocation

The maximum number of remote writers that can be discovered by this DomainParticipant.

This resource-limit limits the maximum number of remote [DDSDataWriter](#) entities that can be discovered by a local [DDSDomainParticipant](#). Note that a remote [DDSDataWriter](#) cannot be discovered until its parent [DDSDomainParticipant](#) has been discovered.

[default] 1

7.17.2.9 DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::remote_reader_allocation

The maximum number of remote readers that can be discovered by this DomainParticipant.

This resource-limit limits the maximum number of remote [DDSDataReader](#) entities that can be discovered by a local DomainParticipant. Note that a remote [DDSDataReader](#) cannot be discovered until its parent [DDSDomainParticipant](#) has been discovered.

[default] 1

7.17.2.10 DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::matching_writer_reader_pair_allocation

The maximum number of matching local writer and remote/local reader pairs.

When RTI Connex Micro discovers DDS entities using a DDS discovery protocol, such as DPSE (Dynamic Participant Static Endpoint) or DPDE (Dynamic Participant Dynamic Endpoint), it matches the discovered entities with the locally created DataReaders and DataWriters. For each match, either a local DataWriter matching a remote DataReader or a local DataReader matching a remote DataWriter, internal state must be managed.

A match, in this context, is defined as being able to receive packets sent from a [DDSDDataReader](#) to a [DDSDDataWriter](#), or from a [DDSDDataWriter](#) to a [DDSDDataReader](#):

- A data-writer needs one resource for each [DDSDDataReader](#) it matches.
- A data-reader needs one resource for each [DDSDDataWriter](#) it matches.

This resource-limit limits the maximum number of matches that can be managed. When this resource limit is reached no further matching can occur, even if new [DDSDDataReader](#)'s and [DDSDDataWriter](#)'s are discovered.

A simple rule of thumb is to set this resource limit to:

- $(\text{local_reader_allocation} + \text{local_writer_allocation}) * \text{max_remote_participants}$.

This assumes that each [DDSDDataReader](#) and [DDSDDataWriter](#) matches one [DDSDDataWriter](#) and [DDSDDataReader](#) respectively for each discovered participant. However, this may over-allocate resources.

[default] 32

7.17.2.11 DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::max_receive_ports

Maximum number of addresses that can be listened on.

RTI Connex Micro listens for discovery data and user data on different transports as specified by the user. This resource-limit limits the maximum number of addresses that can be listened on. As a rule of thumb, this resource limit can be determined as follows: - A user-traffic unicast port counts as 1. - A meta-traffic unicast port counts as 1. - A user-traffic multicast address counts as 1. - A meta-traffic multicast address counts as 1.

Consider the typical case with the UDP transport, using unicast and multicast for discovery data and unicast for user data:

- Discovery data needs two ports.
- User data needs one port.

Thus, this requires 3 resources.

Consider a case with the UDP transport, using unicast and multicast for discovery data and user data: - Discovery data needs two ports. - User data needs two ports.

Thus, this requires 4 resources.

Consider a case with the UDP transport, using unicast and 2 multicast addresses each for discovery data and user data:

- Discovery data needs three ports.
- User data needs three ports.

Thus, this requires 6 resources.

The default value has been set to cover most cases. This resource-limit should only be changed if a lot of interfaces are supported (increase it) or if memory is extremely limited (decrease it).

[default] 8

7.17.2.12 DDS_Long DDS_DomainParticipantResourceLimitsQosPolicy::max_destination_ports

Maximum number of unique destination addresses that a participant can reach.

RTI Connex Micro maintains a table of destination addresses which it can send to based on information queried from the registered transports. This resource limit is difficult to give a precise rule for how to determine since it is transport dependent. However, as a rule of thumb the following is true: - Each network that can be reached directly by the transport requires 1 resource. - If multicast is supported it counts as 1 resource. - If the transport is the default transport (by default UDP is) it counts as one address

Consider a typical Linux computer with 2 network interfaces, eth0 and lo, with support for multicast:

- 1 for the eth0 network.
- 1 for the lo network.
- 1 for multicast.
- 1 for being the default transport.

This requires 4 resources.

Consider a typical embedded computer with 1 network interfaces, eth0, with no support for multicast:

- 1 for the eth0 network.
- 1 for multicast.
- 1 for being the default transport.

This requires 3 resources.

The default value has been set to cover most cases. This resource-limit should only be changed if a lot of interfaces are supported (increase it) or if memory is extremely limited (decrease it).

[default] 8

7.18 DDS_DurabilityQosPolicy Struct Reference

Durability properties that affect the life-cycle of published data.

Public Attributes

- [DDS_DurabilityQosPolicyKind kind](#)
The kind of durability that applies to published data.

7.18.1 Detailed Description

Durability properties that affect the life-cycle of published data.

7.18.2 Member Data Documentation

7.18.2.1 DDS_DurabilityQosPolicyKind DDS_DurabilityQosPolicy::kind

The kind of durability that applies to published data.

7.19 DDS_Duration_t Struct Reference

Type for *duration* representation.

Public Attributes

- [DDS_Long](#) *sec*
seconds
- [DDS_UnsignedLong](#) *nanosec*
nanoseconds

7.19.1 Detailed Description

Type for *duration* representation.

Represents a time interval.

7.19.2 Member Data Documentation

7.19.2.1 [DDS_Long](#) [DDS_Duration_t::sec](#)

seconds

7.19.2.2 [DDS_UnsignedLong](#) [DDS_Duration_t::nanosec](#)

nanoseconds

7.20 [DDS_EntityFactoryQosPolicy](#) Struct Reference

[<<cert>>](#) A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.

Public Attributes

- [DDS_Boolean](#) [autoenable_created_entities](#)
Specifies whether the entity acting as a factory automatically enables the instances it creates.

7.20.1 Detailed Description

[<<cert>>](#) A QoS policy for all [DDSEntity](#) types that can act as factories for one or more other [DDSEntity](#) types.

Entity:

[DDSDomainParticipantFactory](#), [DDSDomainParticipant](#), [DDSPublisher](#), [DDSSubscriber](#)

Properties:

[RxO](#) = NO
[Changeable](#) = YES

7.20.2 Usage

This policy controls the behavior of the [DDSEntity](#) as a factory for other entities.

This policy is mutable. A change in the policy affects only the entities created after the change; not the previously created entities.

The setting of [autoenable_created_entities](#) to [DDS_BOOLEAN_TRUE](#) indicates that the factory [create_<entity>](#) operation will automatically invoke the [DDSEntity::enable](#) operation each time a new [DDSEntity](#) is created. Therefore, the [DDSEntity](#) returned by [create_<entity>](#) will already be enabled. A setting of

`DDS_BOOLEAN_FALSE` indicates that the `DDSEntity` will not be automatically enabled. The application will need to enable it explicitly by means of `DDSEntity::enable` operation.

The default setting of `autoenable_created_entities` = `DDS_BOOLEAN_TRUE` means that, by default, it is not necessary to explicitly call `DDSEntity::enable` on newly created entities.

7.20.3 Member Data Documentation

7.20.3.1 DDS_Boolean DDS_EntityFactoryQosPolicy::autoenable_created_entities

Specifies whether the entity acting as a factory automatically enables the instances it creates.

If `DDS_BOOLEAN_TRUE`, the factory will automatically enable each created entity. Otherwise, it will not.

[default] `DDS_BOOLEAN_TRUE`

7.21 DDS_EntityNameQosPolicy Struct Reference

<<*cert*>> Policy used to name an entity.

Public Attributes

- char `name` [`DDS_ENTITYNAME_QOS_NAME_MAX+1`]
The actual name string.

7.21.1 Detailed Description

<<*cert*>> Policy used to name an entity.

Entity:

`DDSDomainParticipant`

Properties:

`RxO` = NO;
`Changeable` = NO

7.21.2 Usage

The purpose of this QoS is to allow the user to attach naming information to created `DDSEntity` objects.

It can only be 255 characters in length.

It must be a NULL-terminated string allocated with `DDS_String_alloc` or `DDS_String_dup`.

If the user provides a non-NULL pointer when getting the QoS, then it should point to valid memory that can be written to, to avoid ungraceful failures.

7.21.3 Member Data Documentation

7.21.3.1 char DDS_EntityNameQosPolicy::name[DDS_ENTITYNAME_QOS_NAME_MAX+1]

The actual name string.

[default] "[ENTITY]"

[range] Null terminated string with length not exceeding 255. Can be NULL.

7.22 DDS_GUID_t Struct Reference

Type for *GUID* (Global Unique Identifier) representation.

Public Attributes

- [DDS_Octet value](#) [16]
A 16 byte array containing the GUID value.

7.22.1 Detailed Description

Type for *GUID* (Global Unique Identifier) representation.

Represents a 128 bit GUID.

7.22.2 Member Data Documentation

7.22.2.1 DDS_Octet DDS_GUID_t::value[16]

A 16 byte array containing the GUID value.

7.23 DDS_HistoryQosPolicy Struct Reference

<<[cert](#)>> Specifies the behavior of RTI Connex Micro in the case where the value of a sample changes (one or more times) before it can be successfully communicated to one or more existing subscribers.

Public Attributes

- [DDS_HistoryQosPolicyKind kind](#)
Specifies the kind of history to be kept.
- [DDS_Long depth](#)
*Specifies the number of samples to be kept, when the *kind* is [DDS_KEEP_LAST_HISTORY_QOS](#).*

7.23.1 Detailed Description

<<[cert](#)>> Specifies the behavior of RTI Connex Micro in the case where the value of a sample changes (one or more times) before it can be successfully communicated to one or more existing subscribers.

This `QosPolicy` controls whether RTI Connex Micro should deliver only the most recent value, attempt to deliver all intermediate values, or do something in between.

On the publishing side this policy controls the samples that should be maintained by the [DDSDDataWriter](#) on behalf of existing [DDSDDataReader](#) entities.

On the subscribing side it controls the samples that should be maintained until the application "takes" them from RTI Connex Micro.

RTI Connex Micro supports [DDS_KEEP_LAST_HISTORY_QOS](#) kind.

Entity:

[DDSTopic](#), [DDSDDataReader](#), [DDSDDataWriter](#)

Properties:

[RxO](#) = NO
[Changeable](#) = NO

See also

[DDS_ReliabilityQosPolicy](#)
[DDS_HistoryQosPolicy](#)

7.23.2 Usage

This policy controls the behavior of RTI Connex Micro when the value of an instance changes before it is finally communicated to some of its existing [DDSDataReader](#) entities.

For kind set to [DDS_KEEP_LAST_HISTORY_QOS](#), RTI Connex Micro will only attempt to keep the latest values of the instance and discard the older ones. In this case, the value of `depth` regulates the maximum number of values (up to and including the most current one) RTI Connex Micro will maintain and deliver. The default (and most common setting) for `depth` is 1, indicating that only the most recent value should be delivered.

7.23.3 Consistency

The setting of [HISTORY](#) `depth` must be consistent with the [RESOURCE_LIMITS](#) `max_samples_per_instance`. For these two QoS to be consistent, they must verify that $depth \leq max_samples_per_instance$.

See also

[DDS_ResourceLimitsQosPolicy](#)

7.23.4 Member Data Documentation

7.23.4.1 DDS_HistoryQosPolicyKind DDS_HistoryQosPolicy::kind

Specifies the kind of history to be kept.

[default] [DDS_KEEP_LAST_HISTORY_QOS](#)

7.23.4.2 DDS_Long DDS_HistoryQosPolicy::depth

Specifies the number of samples to be kept, when the `kind` is [DDS_KEEP_LAST_HISTORY_QOS](#).

If a value other than 1 (the default) is specified, it should be consistent with the settings of the [RESOURCE_LIMITS](#) policy. That is:

$depth \leq DDS_ResourceLimitsQosPolicy::max_samples_per_instance$

[default] 1

[range] [1,2147483647]

7.24 DDS_InconsistentTopicStatus Struct Reference

[DDS_INCONSISTENT_TOPIC_STATUS](#)

7.24.1 Detailed Description

[DDS_INCONSISTENT_TOPIC_STATUS](#)

Entity:

[DDSTopic](#)

Listener:

[DDSTopicListener](#)

A remote [DDSTopic](#) will be inconsistent with the locally created [DDSTopic](#) if the type name of the two topics are different.

7.25 DDS_LivelinessChangedStatus Struct Reference

<<cert>> [DDS_LIVELINESS_CHANGED_STATUS](#)

Public Attributes

- [DDS_Long alive_count](#)
The total count of currently alive [DDSDDataWriter](#) entities that write the [DDSTopic](#) the [DDSDDataReader](#) reads.
- [DDS_Long not_alive_count](#)
The total count of currently not_alive [DDSDDataWriter](#) entities that write the [DDSTopic](#) the [DDSDDataReader](#) reads.
- [DDS_Long alive_count_change](#)
The change in the alive_count since the last time the listener was called or the status was read.
- [DDS_Long not_alive_count_change](#)
The change in the not_alive_count since the last time the listener was called or the status was read.
- [DDS_InstanceHandle_t last_publication_handle](#)
An instance handle to the last remote writer to change its liveliness.

7.25.1 Detailed Description

<<cert>> [DDS_LIVELINESS_CHANGED_STATUS](#)

7.25.2 Member Data Documentation

7.25.2.1 DDS_Long DDS_LivelinessChangedStatus::alive_count

The total count of currently alive [DDSDDataWriter](#) entities that write the [DDSTopic](#) the [DDSDDataReader](#) reads.

7.25.2.2 DDS_Long DDS_LivelinessChangedStatus::not_alive_count

The total count of currently not_alive [DDSDDataWriter](#) entities that write the [DDSTopic](#) the [DDSDDataReader](#) reads.

7.25.2.3 DDS_Long DDS_LivelinessChangedStatus::alive_count_change

The change in the alive_count since the last time the listener was called or the status was read.

7.25.2.4 DDS_Long DDS_LivelinessChangedStatus::not_alive_count_change

The change in the not_alive_count since the last time the listener was called or the status was read.

7.25.2.5 DDS_InstanceHandle_t DDS_LivelinessChangedStatus::last_publication_handle

An instance handle to the last remote writer to change its liveliness.

7.26 DDS_LivelinessLostStatus Struct Reference

<<cert>> [DDS_LIVELINESS_LOST_STATUS](#)

Public Attributes

- [DDS_Long total_count](#)
Total cumulative number of times that a previously-alive [DDSDDataWriter](#) became not alive due to a failure to to actively signal its liveliness within the offered liveliness period.
- [DDS_Long total_count_change](#)
The incremental changees in total_count since the last time the listener was called or the status was read.

7.26.1 Detailed Description

<<cert>> [DDS_LIVELINESS_LOST_STATUS](#)

Entity:

[DDSDDataWriter](#)

Listener:

[DDSDDataWriterListener](#)

The liveliness that the [DDSDDataWriter](#) has committed through its [DDS_LivelinessQosPolicy](#) was not respected; thus [DDSDDataReader](#) entities will consider the [DDSDDataWriter](#) as no longer "alive/active".

7.26.2 Member Data Documentation

7.26.2.1 DDS_Long DDS_LivelinessLostStatus::total_count

Total cumulative number of times that a previously-alive [DDSDDataWriter](#) became not alive due to a failure to to actively signal its liveliness within the offered liveliness period.

This count does not change when an already not alive [DDSDDataWriter](#) simply remains not alive for another liveliness period.

7.26.2.2 DDS_Long DDS_LivelinessLostStatus::total_count_change

The incremental changees in total_count since the last time the listener was called or the status was read.

7.27 DDS_LivelinessQosPolicy Struct Reference

<<cert>> Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.

Public Attributes

- [DDS_LivelinessQosPolicyKind kind](#)
The kind of liveliness desired.
- struct [DDS_Duration_t lease_duration](#)
The duration within which a [DDSEntity](#) must be asserted, or else it is assumed to be not alive.

7.27.1 Detailed Description

<<cert>> Determines the mechanism and parameters used by the application to determine whether a [DDSEntity](#) is alive.

Liveliness must be asserted at least once every `lease_duration` otherwise RTI Connex Micro will assume the corresponding [DDSEntity](#) or is no longer alive.

The liveliness status of a [DDSEntity](#) is used to maintain instance ownership in combination with the setting of the [OWNERSHIP](#) policy. The application is also informed via [DDSListener](#) when an [DDSEntity](#) is no longer alive.

A [DDSDDataWriter](#) commits to signalling its liveliness using the stated means at intervals not to exceed the `lease_duration`. That is, a failure by the [DDSDDataWriter](#) to assert liveliness within its offered `lease_duration` will result in a [DDS_LIVELINESS_LOST_STATUS](#) event.

A [DDSDDataReader](#) will match a [DDSDDataWriter](#) if the [DDSDDataReader](#)'s requested `lease_duration` is equal to or larger than the `lease_duration` offered by the [DDSDDataWriter](#) and the offered `kind` is equal to or better than the requested `kind`. In addition, a [DDSDDataReader](#) uses its own `lease_duration` as the check period to check if a [DDSDDataWriter](#) was active or not. This means that a [DDSDDataWriter](#) may be inactive for up to, but not including, twice the [DDSDDataReader](#)'s `lease_duration` before being detected as inactive by the [DDSDDataReader](#). It does not matter when the [DDSDDataWriter](#) was active, or how many times the [DDSDDataWriter](#) was active within the last [DDSDDataReader](#)'s `lease_duration` to be considered inactive.

For example: If a [DDSDDataReader](#) must detect that a [DDSDDataWriter](#) was inactive within at most 100ms, both the [DDSDDataWriter](#) and [DDSDDataReader](#) must specify a `lease_duration` of 50ms.

Listeners are used to notify a [DDSDDataReader](#) of loss of liveliness and [DDSDDataWriter](#) of violations to the liveliness contract. The `on_liveliness_lost()` callback is only called *once*, after the first time the `lease_duration` is exceeded (when the [DDSDDataWriter](#) first loses liveliness).

This QoS policy can be used during system integration to ensure that applications have been coded to meet design specifications. It can also be used during run time to detect when systems are performing outside of design specifications. Receiving applications can take appropriate actions in response to disconnected DataWriters.

Entity:

[DDSTopic](#), [DDSDDataReader](#), [DDSDDataWriter](#)

Status:

[DDS_LIVELINESS_LOST_STATUS](#), [DDS_LivelinessLostStatus](#);
[DDS_LIVELINESS_CHANGED_STATUS](#), [DDS_LivelinessChangedStatus](#);
[DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#), [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#)

Properties:

[RxO](#) = YES
[Changeable](#) = NO

7.27.2 Usage

This policy controls the mechanism and parameters used by RTI Connex Micro to ensure that particular entities on the network are still alive. The liveliness can also affect the ownership of a particular instance, as determined by the [OWNERSHIP](#) policy.

This policy has several settings to support both data types that are updated periodically as well as those that are changed sporadically. It also allows customisation for different application requirements in terms of the kinds of failures that will be detected by the liveliness mechanism.

The [DDS_AUTOMATIC_LIVELINESS_QOS](#) liveliness setting is most appropriate for applications that only need to detect failures at the process-level, but not application-logic failures within a process. RTI Connex Micro takes responsibility for renewing the leases at the required rates and thus, as long as the local process where a [DDS-DomainParticipant](#) is running and the link connecting it to remote participants remains connected, the entities within the [DDSDomainParticipant](#) will be considered alive. This requires the lowest overhead.

The manual settings ([DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS](#), [DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS](#)) require the application on the publishing side to periodically assert the liveliness before the lease expires to indicate the corresponding [DDSEntity](#) is still alive. The action can be explicit by calling the [DDSDDataWriter::assert_liveliness](#) operation or implicit by writing some data.

The two possible manual settings control the granularity at which the application must assert liveliness.

- The setting `DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS` requires only that one `DDSEntity` within a participant is asserted to be alive to deduce all other `DDSEntity` objects within the same `DDSDomainParticipant` are also alive.
- The setting `DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS` requires that at least one instance within the `DDSDataWriter` is asserted.

Changes in `LIVELINESS` must be detected by the Service with a time-granularity greater or equal to the `lease_duration`. This ensures that the value of the `DDS_LivelinessChangedStatus` is updated at least once during each `lease_duration` and the related Listeners are notified within a `lease_duration` from the time the `LIVELINESS` changed.

7.27.3 Compatibility

The value offered is considered compatible with the value requested if and only if the following conditions are met:

- the inequality *offered kind* $\geq$ *requested kind* evaluates to 'TRUE'. For the purposes of this inequality, the values of `DDS_LivelinessQosPolicyKind` kind are considered ordered such that: `DDS_AUTOMATIC_LIVELINESS_QOS` $<$ `DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS` $<$ `DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS`.
- the inequality *offered lease_duration* $\leq$ *requested lease_duration* evaluates to `DDS_BOOLEAN_TRUE`.

7.27.4 Member Data Documentation

7.27.4.1 DDS_LivelinessQosPolicyKind DDS_LivelinessQosPolicy::kind

The kind of liveliness desired.

[default] `DDS_AUTOMATIC_LIVELINESS_QOS`

7.27.4.2 struct DDS_Duration_t DDS_LivelinessQosPolicy::lease_duration

The duration within which a `DDSEntity` must be asserted, or else it is assumed to be not alive.

[default] `DDS_DURATION_INFINITE`

[range] [0,1 year] or `DDS_DURATION_INFINITE`

7.28 DDS_Locator_t Struct Reference

`<<eXtension>>` `<<cert>>` Type used to represent the addressing information needed to send a message to an RTPS Endpoint using one of the supported transports.

Public Attributes

- `DDS_Long` kind
The kind of locator.
- `DDS_UnsignedLong` port
the port number
- `DDS_Octet` address [RTPS_LOCATOR_ADDRESS_LENGTH_MAX]
An octet field (with maximum length of RTPS_LOCATOR_ADDRESS_LENGTH_MAX to hold the IP address.

7.28.1 Detailed Description

[`<<eXtension>>`](#) [`<<cert>>`](#) Type used to represent the addressing information needed to send a message to an RTPS Endpoint using one of the supported transports.

7.28.2 Member Data Documentation

7.28.2.1 `DDS_Long DDS_Locator_t::kind`

The kind of locator.

If the `Locator_t` kind is `DDS_LOCATOR_KIND_UDPv4`, the address contains an IPv4 address. In this case, the leading 12 octets of the `DDS_Locator_t::address` must be zero. The last 4 octets of `DDS_Locator_t::address` are used to store the IPv4 address in big-endian order.

If the `Locator_t` kind is not `DDS_LOCATOR_KIND_UDPv4`, the address is not interpreted and is treated as an array of 16 octets.

7.28.2.2 `DDS_UnsignedLong DDS_Locator_t::port`

the port number

7.28.2.3 `DDS_Octet DDS_Locator_t::address[RTPS_LOCATOR_ADDRESS_LENGTH_MAX]`

An octet field (with maximum length of `RTPS_LOCATOR_ADDRESS_LENGTH_MAX` to hold the IP address.

7.29 `DDS_LocatorSeq` Struct Reference

Declares IDL `sequence` [`< DDS\_Locator\_t >`](#)

7.29.1 Detailed Description

Declares IDL `sequence` [`< DDS\_Locator\_t >`](#)

See also

[`DDS\_Locator\_t`](#)

7.30 `DDS_OfferedDeadlineMissedStatus` Struct Reference

[`<<cert>>`](#) `DDS_OFFERED_DEADLINE_MISSED_STATUS`

Public Attributes

- [`DDS\_Long total\_count`](#)
Total cumulative count of the number of times the `DDSDataWriter` failed to write within its offered deadline.
- [`DDS\_Long total\_count\_change`](#)
The incremental changes in `total_count` since the last time the listener was called or the status was read.
- [`DDS\_InstanceHandle\_t last\_instance\_handle`](#)
Handle to the last instance in the `DDSDataWriter` for which an offered deadline was missed.

7.30.1 Detailed Description

[`<<cert>>`](#) `DDS_OFFERED_DEADLINE_MISSED_STATUS`

Entity:

[DDSDataWriter](#)

Listener:

[DDSDataWriterListener](#)

The deadline that the [DDSDataWriter](#) has committed through its [DDS_DeadlineQosPolicy](#) was not respected for a specific instance.

7.30.2 Member Data Documentation

7.30.2.1 DDS_Long DDS_OfferedDeadlineMissedStatus::total_count

Total cumulative count of the number of times the [DDSDataWriter](#) failed to write within its offered deadline. Missed deadlines accumulate; that is, each deadline period the `total_count` will be incremented by one.

```
total_count
```

7.30.2.2 DDS_Long DDS_OfferedDeadlineMissedStatus::total_count_change

The incremental changes in `total_count` since the last time the listener was called or the status was read.

7.30.2.3 DDS_InstanceHandle_t DDS_OfferedDeadlineMissedStatus::last_instance_handle

Handle to the last instance in the [DDSDataWriter](#) for which an offered deadline was missed.

7.31 DDS_OfferedIncompatibleQosStatus Struct Reference

<<cert>> [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#)

Public Attributes

- [DDS_Long total_count](#)
Total cumulative number of times the concerned [DDSDataWriter](#) discovered a [DDSDataReader](#) for the same [DDS-Topic](#), common partition with a requested QoS that is incompatible with that offered by the [DDSDataWriter](#).
- [DDS_Long total_count_change](#)
The incremental changes in `total_count` since the last time the listener was called or the status was read.
- [DDS_QosPolicyId_t last_policy_id](#)
The [DDS_QosPolicyId_t](#) of one of the policies that was found to be incompatible the last time an incompatibility was detected.
- struct [DDS_QosPolicyCountSeq policies](#)
A list containing for each policy the total number of times that the concerned [DDSDataWriter](#) discovered a [DDSDataReader](#) for the same [DDSTopic](#) and common partition with a requested QoS that is incompatible with that offered by the [DDSDataWriter](#).

7.31.1 Detailed Description

<<cert>> [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#)

Entity:

[DDSDataWriter](#)

Listener:

[DDSDataWriterListener](#)

The qos policy value was incompatible with what was requested.

7.31.2 Member Data Documentation

7.31.2.1 [DDS_Long](#) [DDS_OfferedIncompatibleQosStatus::total_count](#)

Total cumulative number of times the concerned [DDSDataWriter](#) discovered a [DDSDataReader](#) for the same [DDS-Topic](#), common partition with a requested QoS that is incompatible with that offered by the [DDSDataWriter](#).

7.31.2.2 [DDS_Long](#) [DDS_OfferedIncompatibleQosStatus::total_count_change](#)

The incremental changes in total_count since the last time the listener was called or the status was read.

7.31.2.3 [DDS_QosPolicyId_t](#) [DDS_OfferedIncompatibleQosStatus::last_policy_id](#)

The [DDS_QosPolicyId_t](#) of one of the policies that was found to be incompatible the last time an incompatibility was detected.

7.31.2.4 [struct](#) [DDS_QosPolicyCountSeq](#) [DDS_OfferedIncompatibleQosStatus::policies](#)

A list containing for each policy the total number of times that the concerned [DDSDataWriter](#) discovered a [DDSDataReader](#) for the same [DDSTopic](#) and common partition with a requested QoS that is incompatible with that offered by the [DDSDataWriter](#).

7.32 [DDS_OwnershipQosPolicy](#) Struct Reference

[<<cert>>](#) Specifies whether it is allowed for multiple [DDSDataWriter](#) (s) to write the same instance of the data and if so, how these modifications should be arbitrated.

Public Attributes

- [DDS_OwnershipQosPolicyKind](#) kind

The kind of ownership.

7.32.1 Detailed Description

[<<cert>>](#) Specifies whether it is allowed for multiple [DDSDataWriter](#) (s) to write the same instance of the data and if so, how these modifications should be arbitrated.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#), [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#)

Properties:

[RxO](#) = YES

[Changeable](#) = NO

See also

[OWNERSHIP_STRENGTH](#)

7.32.2 Usage

This policy controls whether RTI Connext Micro allows multiple [DDSDataWriter](#) objects to update the same instance (identified by [DDSTopic](#) + key) of a data type.

There are two kinds of owner selected by the setting of the `kind`: SHARED and EXCLUSIVE.

7.32.2.1 SHARED ownership

[DDS_SHARED_OWNERSHIP_QOS](#) indicates that RTI Connext Micro does not enforce unique ownership for each instance. In this case, multiple writers can update the same data type instance. The subscriber to the [DDSTopic](#) will be able to access modifications from all [DDSDataWriter](#) objects, subject to the settings of other QoS that may filter particular samples (e.g. the [HISTORY](#) policy). In any case there is no "filtering" of modifications made based on the identity of the [DDSDataWriter](#) that causes the modification.

7.32.2.2 EXCLUSIVE ownership

[DDS_EXCLUSIVE_OWNERSHIP_QOS](#) indicates that each instance of a data type can only be modified by one [DDSDataWriter](#). In other words, at any point in time a single [DDSDataWriter](#) owns each instance and is the only one whose modifications will be visible to the [DDSDataReader](#) objects. The owner is determined by selecting the [DDSDataWriter](#) with the highest value of the [DDS_OwnershipStrengthQosPolicy::value](#) that is currently alive as defined by the [LIVELINESS](#) policy and has not violated its [DEADLINE](#) contract with regards to the data-instance.

Ownership can therefore change as a result of:

- a [DDSDataWriter](#) in the system with a higher value of the strength that modifies the instance,
- a change in the strength value of the [DDSDataWriter](#) that owns the instance, and
- a change in the liveliness of the [DDSDataWriter](#) that owns the instance.
- a deadline with regards to the instance that is missed by the [DDSDataWriter](#) that owns the instance.

The behavior of the system is as if the determination was made independently by each [DDSDataReader](#). Each [DDSDataReader](#) may detect the change of ownership at a different time. It is not a requirement that at a particular point in time all the [DDSDataReader](#) objects for that [DDSTopic](#) have a consistent picture of who owns each instance.

It is also not a requirement that the [DDSDataWriter](#) objects are aware of whether they own a particular instance. There is no error or notification given to a [DDSDataWriter](#) that modifies an instance it does not currently own.

The requirements are chosen to (a) preserve the decoupling of publishers and subscriber, and (b) allow the policy to be implemented efficiently.

It is possible that multiple [DDSDataWriter](#) objects with the same strength modify the same instance. If this occurs RTI Connext Micro will pick one of the [DDSDataWriter](#) objects as the owner. It is not specified how the owner is selected. However, it is required that the policy used to select the owner is such that all [DDSDataReader](#) objects will make the same choice of the particular [DDSDataWriter](#) that is the owner. It is required that the owner remains the same until there is a change in strength, liveliness, the owner misses a deadline on the instance, or a new [DDSDataWriter](#) with higher same strength, or a new [DDSDataWriter](#) with same strength that should be deemed the owner according to the policy of the Service, modifies the instance.

Exclusive ownership is on an instance-by-instance basis. That is, a subscriber can receive values written by a lower strength [DDSDataWriter](#) as long as they affect instances whose values have not been set by the higher-strength [DDSDataWriter](#).

7.32.3 Compatibility

The value of the [DDS_OwnershipQosPolicyKind](#) offered must exactly match the one requested or else they are considered incompatible.

7.32.3.1 Ownership Resolution on Redundant Systems

It is expected that users may use DDS to set up redundant systems where multiple [DDSDataWriter](#) entities are "capable" of writing the same instance. In this situation, the [DDSDataWriter](#) entities are configured such that:

- Either both are writing the instance "constantly"
- Or else they use some mechanism to classify each other as "primary" and "secondary", such that the primary is the only one writing, and the secondary monitors the primary and only writes when it detects that the primary "writer" is no longer writing.

Both cases above use the [DDS_EXCLUSIVE_OWNERSHIP_QOS](#) and arbitrate themselves by means of the [DDS_OwnershipStrengthQosPolicy](#). Regardless of the scheme, the desired behavior from the [DDSDataReader](#) point of view is that [DDSDataReader](#) normally receives data from the primary unless the "primary" writer stops writing, in which case the [DDSDataReader](#) starts to receive data from the secondary [DDSDataWriter](#).

This approach requires some mechanism to detect that a [DDSDataWriter](#) (the primary) is no longer "writing" the data as it should. There are several reasons why this may happen and all must be detected (but not necessarily distinguished):

- [crash] The writing process is no longer running (e.g. the whole application has crashed)
- [connectivity loss] Connectivity to the writing application has been lost (e.g. network disconnection)
- [application fault] The application logic that was writing the data is faulty and has stopped calling [FooDataWriter::write](#).

Arbitrating from a [DDSDataWriter](#) to one of a higher strength is simple and the decision can be taken autonomously by the [DDSDataReader](#). Switching ownership from a higher strength [DDSDataWriter](#) to one of a lower strength [DDSDataWriter](#) requires that the [DDSDataReader](#) can make a determination that the stronger [DDSDataWriter](#) is "no longer writing the instance".

7.32.3.1.1 Case where the data is periodically updated

This determination is reasonably simple when the data is being written periodically at some rate. The [DDSDataWriter](#) simply states its offered [DDS_DeadlineQosPolicy](#) (maximum interval between updates) and the [DDSDataReader](#) automatically monitors that the [DDSDataWriter](#) indeed updates the instance at least once per [DDS_DeadlineQosPolicy::period](#). If the deadline is missed, the [DDSDataReader](#) considers the [DDSDataWriter](#) "not alive" and automatically gives ownership to the next highest-strength [DDSDataWriter](#) that *is* alive.

7.32.3.1.2 Case where data is not periodically updated

The case where the [DDSDataWriter](#) is not writing data periodically is also a very important use-case. Since the instance is not being updated at any fixed period, the "deadline" mechanism cannot be used to determine ownership. The liveliness solves this situation. Ownership is maintained while the [DDSDataWriter](#) is "alive" and for the [DDSDataWriter](#) to be alive it must fulfill its [DDS_LivelinessQosPolicy](#) contract. The different means to renew liveliness (automatic, manual) combined by the implied renewal each time data is written handle the three conditions above [crash], [connectivity loss], and [application fault]. Note that to handle [application fault], LIVELINESS must be [DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS](#). The [DDSDataWriter](#) can retain ownership by periodically writing data or else calling `assert_liveliness` if it has no data to write. Alternatively if only protection against [crash] or [connectivity loss] is desired, it is sufficient that some task on the [DDSDataWriter](#) process periodically writes data. However, this scenario requires that the [DDSDataReader](#) knows what instances are being "written" by the [DDSDataWriter](#). That is the only way that the [DDSDataReader](#) deduces the ownership of specific instances from the fact that the [DDSDataWriter](#) is still "alive".

7.32.3.2 Detection of Loss in Topological Connectivity

There are applications that are designed in such a way that their correct operation requires some minimal topological connectivity, that is, the writer needs to have a minimum number of readers or alternatively the reader must have a minimum number of writers.

A common scenario is that the application does not start doing its logic until it knows that some specific writers have the minimum configured readers (e.g. the alarm monitor is up).

A *more* common scenario is that the application logic will wait until some writers appear that can provide some needed source of information (e.g. the raw sensor data that must be processed).

Furthermore, once the application is running it is a requirement that this minimal connectivity (from the source of the data) is monitored and the application informed if it is ever lost. For the case where data is being written periodically, the [DDS_DeadlineQosPolicy](#) and the `on_deadline_missed` listener provides the notification. The case where data is not periodically updated requires the use of the [DDS_LivelinessQosPolicy](#) to detect whether the "connectivity" has been lost, and the notification is provided by means of [DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#).

In terms of the required mechanisms, the scenario is very similar to the case of maintaining ownership. In both cases, the reader needs to know whether a writer is still "managing the current value of an instance" even though it is not continually writing it and this knowledge requires the writer to keep its liveliness.

For a [DDSTopic](#) with [DDS_EXCLUSIVE_OWNERSHIP_QOS](#), if the current owner of an instance *disposes* it, the readers accessing the instance will see the `instance_state` as being "DISPOSED" and not see the values being written by the weaker writer (even after the stronger one has disposed the instance). This is because the [DDSDDataWriter](#) that owns the instance is saying that the instance no longer exists (e.g. the master of the database is saying that a record has been deleted) and thus the readers should see it as such.

7.32.4 Member Data Documentation

7.32.4.1 DDS_OwnershipQosPolicyKind DDS_OwnershipQosPolicy::kind

The kind of ownership.

[default] [DDS_SHARED_OWNERSHIP_QOS](#)

7.33 DDS_OwnershipStrengthQosPolicy Struct Reference

<<[cert](#)>> Specifies the value of the strength used to arbitrate among multiple [DDSDDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).

Public Attributes

- [DDS_Long](#) value
The strength value used to arbitrate among multiple writers.

7.33.1 Detailed Description

<<[cert](#)>> Specifies the value of the strength used to arbitrate among multiple [DDSDDataWriter](#) objects that attempt to modify the same instance of a data type (identified by [DDSTopic](#) + key).

This policy only applies if the [OWNERSHIP](#) policy is of kind [DDS_EXCLUSIVE_OWNERSHIP_QOS](#).

Entity:

[DDSDDataWriter](#)

Properties:

[RxO](#) = N/A
[Changeable](#) = NO

The value of the [OWNERSHIP_STRENGTH](#) is used to determine the ownership of a data-instance (identified by the key). The arbitration is performed by the [DDSDDataReader](#).

See also

[EXCLUSIVE ownership](#)

7.33.2 Member Data Documentation

7.33.2.1 DDS_Long DDS_OwnershipStrengthQosPolicy::value

The strength value used to arbitrate among multiple writers.

[default] 0

[range] [0, 1 million]

7.34 DDS_ParticipantBuiltinTopicData Struct Reference

[`<<cert>>`](#) Object representing a remote DomainParticipant.

Public Attributes

- struct [DDS_BuiltinTopicKey_t](#) `key`
DCPS key to distinguish entries.
- struct [DDS_EntityNameQosPolicy](#) `participant_name`
The participant name.
- [DDS_UnsignedLong](#) `dds_builtin_endpoints`
Bitmap of builtin endpoints supported by the participant.
- [DDS_ProtocolVersion_t](#) `rtps_protocol_version`
Version number of the RTPS wire protocol used.
- struct [DDS_VendorId_t](#) `rtps_vendor_id`
ID of vendor implementing the RTPS wire protocol.
- struct [DDS_LocatorSeq](#) `default_unicast_locators`
Unicast locators used when individual entities do not specify unicast locators.
- struct [DDS_LocatorSeq](#) `default_multicast_locators`
Multicast locators used when individual entities do not specify multicast locators.
- struct [DDS_LocatorSeq](#) `metatraffic_unicast_locators`
Unicast locators used for discovery metatraffic.
- struct [DDS_LocatorSeq](#) `metatraffic_multicast_locators`
Multicast locators used for discovery metatraffic.
- struct [DDS_Duration_t](#) `liveliness_lease_duration`
Liveliness lease duration of participant.
- struct [DDS_ProductVersion_t](#) `product_version`
This is a vendor specific parameter. It gives the current version for RTI Connex Micro.
- struct [DDS_ChecksumProperty](#) `checksum`
Checksum properties for the participant.

7.34.1 Detailed Description

[`<<cert>>`](#) Object representing a remote DomainParticipant.

Data associated with the built-in topic [DDS_PARTICIPANT_TOPIC_NAME](#). It contains QoS policies and additional information that apply to the remote [DDSDomainParticipant](#).

7.34.2 Member Data Documentation

7.34.2.1 struct DDS_BuiltinTopicKey_t DDS_ParticipantBuiltinTopicData::key

DCPS key to distinguish entries.

7.34.2.2 struct DDS_EntityNameQosPolicy DDS_ParticipantBuiltinTopicData::participant_name

The participant name.

This is the name of the discovered participant.

7.34.2.3 DDS_UnsignedLong DDS_ParticipantBuiltinTopicData::dds_builtin_endpoints

Bitmap of builtin endpoints supported by the participant.

Each bit indicates a builtin endpoint that may be available on the participant for use in discovery.

7.34.2.4 DDS_ProtocolVersion_t DDS_ParticipantBuiltinTopicData::rtps_protocol_version

Version number of the RTPS wire protocol used.

7.34.2.5 struct DDS_VendorId_t DDS_ParticipantBuiltinTopicData::rtps_vendor_id

ID of vendor implementing the RTPS wire protocol.

7.34.2.6 struct DDS_LocatorSeq DDS_ParticipantBuiltinTopicData::default_unicast_locators

Unicast locators used when individual entities do not specify unicast locators.

7.34.2.7 struct DDS_LocatorSeq DDS_ParticipantBuiltinTopicData::default_multicast_locators

Multicast locators used when individual entities do not specify multicast locators.

7.34.2.8 struct DDS_LocatorSeq DDS_ParticipantBuiltinTopicData::metatraffic_unicast_locators

Unicast locators used for discovery metatraffic.

7.34.2.9 struct DDS_LocatorSeq DDS_ParticipantBuiltinTopicData::metatraffic_multicast_locators

Multicast locators used for discovery metatraffic.

7.34.2.10 struct DDS_Duration_t DDS_ParticipantBuiltinTopicData::liveliness_lease_duration

Liveliness lease duration of participant.

7.34.2.11 struct DDS_ProductVersion_t DDS_ParticipantBuiltinTopicData::product_version

This is a vendor specific parameter. It gives the current version for RTI Connext Micro.

7.34.2.12 struct DDS_ChecksumProperty DDS_ParticipantBuiltinTopicData::checksum

Checksum properties for the participant.

7.35 DDS_PresentationQosPolicy Struct Reference

Specifies how the samples representing changes to data instances are presented to a subscribing application.

Public Attributes

- [DDS_PresentationQosPolicyAccessScopeKind access_scope](#)

Determines the largest scope spanning the entities for which the order and coherency of changes can be preserved.

- [DDS_Boolean coherent_access](#)

Specifies support for coherent access. Controls whether coherent access is supported within the scope `access_scope`.

- [DDS_Boolean ordered_access](#)

Specifies support for ordered access to the samples received at the subscription end. Controls whether ordered access is supported within the scope `access_scope`.

7.35.1 Detailed Description

Specifies how the samples representing changes to data instances are presented to a subscribing application.

NOTE: This QoS policy can only be set when using the static discovery API on the [DDS_Subscription-BuiltinTopicData](#).

However, a [DDSDDataWriter](#) always announces [DDS_TOPIC_PRESENTATION_QOS](#) via discovery, enabling a [DDSDDataWriter](#) to match with a [DDSDDataReader](#) that requests either [DDS_INSTANCE_PRESENTATION_QOS](#) or [DDS_TOPIC_PRESENTATION_QOS](#), increasing interoperability.

A RTI Connex Micro [DDSSubscriber](#) always requests:

- `access_scope` = [DDS_INSTANCE_PRESENTATION_QOS](#)
- `ordered_access` = [DDS_BOOLEAN_FALSE](#)
- `coherent_access` = [DDS_BOOLEAN_FALSE](#)

The following is a general description of the [DDS_PresentationQosPolicy](#).

This QoS policy controls the extent to which changes to data instances can be made dependent on each other and also the kind of dependencies that can be propagated and maintained by RTI Connex Micro. Specifically, this policy affects the application's ability to:

- specify and receive coherent changes to instances
- specify the relative order in which changes are presented

Entity:

[DDSPublisher](#), [DDSSubscriber](#)

Status:

[DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#), [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#)

A [DDSDDataReader](#) will usually receive data in the order that it was sent by a [DDSDDataWriter](#), and the data is presented to the [DDSDDataReader](#) as soon as the application receives the next expected value. However, sometimes you may want a set of data for the same [DDSDDataWriter](#) to be presented to the [DDSDDataReader](#) only after *all* of the elements of the set have been received. Or you may want the data to be presented in a different order than that in which it was received. Specifically for keyed data, you may want the middleware to present the data in keyed – or *instance* – order, such that samples pertaining to the same instance are presented together.

The Presentation QoS policy allows you to specify different *scopes* of presentation: within a [DDSDDataWriter](#), across instances of a single [DDSDDataWriter](#), and even across multiple DataWriters used by different writers of a publisher. It also controls whether or not a set of changes within the scope is delivered at the same time or can be delivered as soon as each element is received.

- `coherent_access` controls whether RTI Connex Micro will preserve the groupings of changes made by a publishing application by means of the operations `begin_coherent_changes()` and `end_coherent_changes()`. These operations are not supported.

- `ordered_access` controls whether RTI Connex Micro will preserve the order of changes.
- `access_scope` controls the granularity of the other settings. See below:

If `coherent_access` is set, then the `access_scope` controls the maximum extent of coherent changes. The behavior is as follows:

- If `access_scope` is set to `DDS_INSTANCE_PRESENTATION_QOS` (the default), the use of `begin_coherent_changes()` and `end_coherent_changes()` has no effect on how the subscriber can access the data, because with the scope limited to each instance, changes to separate instances are considered independent and thus cannot be grouped into a coherent set.
- If `access_scope` is set to `DDS_TOPIC_PRESENTATION_QOS`, then coherent changes (indicated by their enclosure within calls to `begin_coherent_changes()` and `end_coherent_changes()`) will be made available as such to each remote `DDSDataReader` independently. That is, changes made to instances within each individual `DDSDataWriter` will be available as coherent with respect to other changes to instances in that same `DDSDataWriter`, but will not be grouped with changes made to instances belonging to a different `DDSDataWriter`.
- If `access_scope` is set to `DDS_GROUP_PRESENTATION_QOS`, then coherent changes made to instances through a `DDSDataWriter` attached to a common `DDSPublisher` are made available as a unit to remote subscribers.

If `ordered_access` is set, then the `access_scope` controls the maximum extent for which order will be preserved by RTI Connex Micro.

- If `access_scope` is set to `DDS_INSTANCE_PRESENTATION_QOS` (the lowest level), then changes to each instance are considered unordered relative to changes to any other instance. That means that changes (creations, deletions, modifications) made to two instances are not necessarily seen in the order they occur. This is the case even if it is the same application thread making the changes using the same `DDSDataWriter`.
- If `access_scope` is set to `DDS_TOPIC_PRESENTATION_QOS`, changes (creations, deletions, modifications) made by a single `DDSDataWriter` are made available to subscribers in the same order they occur. Changes made to instances through different `DDSDataWriter` entities are not necessarily seen in the order they occur. This is the case, even if the changes are made by a single application thread using `DDSDataWriter` objects attached to the same `DDSPublisher`.
- Finally, if `access_scope` is set to `DDS_GROUP_PRESENTATION_QOS`, changes made to instances via `DDSDataWriter` entities attached to the same `DDSPublisher` object are made available to subscribers in the same order they occur.

Note that this QoS policy controls the scope at which related changes are made available to the subscriber. This means the subscriber **can** access the changes in a coherent manner and in the proper order; however, it does not necessarily imply that the `DDSSubscriber` **will** indeed access the changes in the correct order. For that to occur, the application at the subscriber end must use the proper logic in reading the `DDSDataReader` objects.

7.35.2 Compatibility

The value offered is considered compatible with the value requested if and only if the following conditions are met:

- the inequality `offered_access_scope >= requested_access_scope` evaluates to 'TRUE'. For the purposes of this inequality, the values of `access_scope` are considered ordered such that `DDS_INSTANCE_PRESENTATION_QOS < DDS_TOPIC_PRESENTATION_QOS < DDS_GROUP_PRESENTATION_QOS`.
- `requested_coherent_access` is `DDS_BOOLEAN_FALSE`, or else both offered and requested `coherent_access` are `DDS_BOOLEAN_TRUE`.
- `requested_ordered_access` is `DDS_BOOLEAN_FALSE`, or else both offered and requested `ordered_access` are `DDS_BOOLEAN_TRUE`.

7.35.3 Member Data Documentation

7.35.3.1 DDS_PresentationQosPolicyAccessScopeKind DDS_PresentationQosPolicy::access_scope

Determines the largest scope spanning the entities for which the order and coherency of changes can be preserved.

NOTE: This QoS setting can only be set when using the static discovery API on the [DDS_Subscription-BuiltinTopicData](#).

7.35.3.2 DDS_Boolean DDS_PresentationQosPolicy::coherent_access

Specifies support for *coherent* access. Controls whether coherent access is supported within the scope `access_scope`.

That is, the ability to group a set of changes as a unit on the publishing end such that they are received as a unit at the subscribing end.

To use this feature, the DataWriter must be configured for RELIABLE communication (see [DDS_RELIABLE_RELIABILITY_QOS](#)).

NOTE: This QoS setting can only be set when using the static discovery API on the [DDS_Subscription-BuiltinTopicData](#).

[default] [DDS_BOOLEAN_FALSE](#)

7.35.3.3 DDS_Boolean DDS_PresentationQosPolicy::ordered_access

Specifies support for *ordered* access to the samples received at the subscription end. Controls whether ordered access is supported within the scope `access_scope`.

That is, the ability of the subscriber to see changes in the same order as they occurred on the publishing end.

NOTE: This QoS setting can only be set when using the static discovery API on the [DDS_Subscription-BuiltinTopicData](#).

[default] [DDS_BOOLEAN_TRUE](#) for DataWriters, [DDS_BOOLEAN_FALSE](#) for DataReaders

7.36 DDS_ProductVersion Struct Reference

[<<eXtension>>](#) [<<cert>>](#) Type used to represent the current version of RTI Connex Micro.

Public Attributes

- [DDS_Char major](#)
Major product version.
- [DDS_Char minor](#)
Minor product version.
- [DDS_Char release](#)
Release letter for product version.
- [DDS_Char revision](#)
Revision number of product.

7.36.1 Detailed Description

[<<eXtension>>](#) [<<cert>>](#) Type used to represent the current version of RTI Connex Micro.

7.36.2 Member Data Documentation

7.36.2.1 DDS_Char DDS_ProductVersion::major

Major product version.

7.36.2.2 DDS_Char DDS_ProductVersion::minor

Minor product version.

7.36.2.3 DDS_Char DDS_ProductVersion::release

Release letter for product version.

7.36.2.4 DDS_Char DDS_ProductVersion::revision

Revision number of product.

7.37 DDS_ProductVersion_t Struct Reference

<<eXtension>> <<cert>> Type used to represent the current version of RTI Connex Micro.

Public Attributes

- [DDS_Char major](#)
Major product version.
- [DDS_Char minor](#)
Minor product version.
- [DDS_Char release](#)
Release letter for product version.
- [DDS_Char revision](#)
Revision number of product.

7.37.1 Detailed Description

<<eXtension>> <<cert>> Type used to represent the current version of RTI Connex Micro.

7.37.2 Member Data Documentation

7.37.2.1 DDS_Char DDS_ProductVersion_t::major

Major product version.

7.37.2.2 DDS_Char DDS_ProductVersion_t::minor

Minor product version.

7.37.2.3 DDS_Char DDS_ProductVersion_t::release

Release letter for product version.

7.37.2.4 DDS_Char DDS_ProductVersion_t::revision

Revision number of product.

7.38 DDS_ProtocolVersion Struct Reference

<<eXtension>> <<cert>> Type used to represent the version of the RTPS protocol.

Public Attributes

- [DDS_Octet major](#)
Major protocol version number.
- [DDS_Octet minor](#)
Minor protocol version number.

7.38.1 Detailed Description

[<<eXtension>>](#) [<<cert>>](#) Type used to represent the version of the RTPS protocol.

7.38.2 Member Data Documentation

7.38.2.1 [DDS_Octet DDS.ProtocolVersion::major](#)

Major protocol version number.

7.38.2.2 [DDS_Octet DDS.ProtocolVersion::minor](#)

Minor protocol version number.

7.39 [DDS_PublicationBuiltinTopicData](#) Struct Reference

Object describing a remote Publication.

Public Attributes

- struct [DDS_BuiltinTopicKey_t key](#)
DCPS key to distinguish entries. This is used to configure the object ID of the remote [DDSDDataWriter](#).
- struct [DDS_BuiltinTopicKey_t participant_key](#)
DCPS key of the participant to which the DataWriter belongs.
- char * [topic_name](#)
Name of the related [DDSTopic](#).
- char * [type_name](#)
Name of the type attached to the [DDSTopic](#).
- struct [DDS_DeadlineQosPolicy deadline](#)
Policy of the corresponding DataWriter.
- struct [DDS_OwnershipQosPolicy ownership](#)
Policy of the corresponding DataWriter.
- struct [DDS_OwnershipStrengthQosPolicy ownership_strength](#)
Policy of the corresponding DataWriter.
- struct [DDS_ReliabilityQosPolicy reliability](#)
Policy of the corresponding DataWriter.
- struct [DDS_LivelinessQosPolicy liveliness](#)
Policy of the corresponding DataWriter.
- struct [DDS_DurabilityQosPolicy durability](#)
durability policy of the corresponding DataWriter
- struct [DDS_DestinationOrderQosPolicy destination_order](#)
Policy of the corresponding DataWriter.
- struct [DDS_LocatorSeq unicast_locator](#)
Custom unicast locator sequence that the endpoint can specify. The default locator sequence from the [DDSDomainParticipant](#) will be used if this is not specified.

7.39.1 Detailed Description

Object describing a remote Publication.

This object contains all of the information about a remote [DDSDataWriter](#) and its [DDSPublisher](#) that is necessary for matching the remote [DDSDataWriter](#) to a local [DDSDataReader](#) during discovery.

This object is used to pre-configure remote entities for Dynamic Participant/Static Endpoint discovery.

For example: Application A is designed to discover a remote [DDSDataWriter](#) in application B, and that remote [DDSDataWriter](#) has an object ID of 100, and a deadline of three seconds. In Dynamic Participant/Static Endpoint discovery, this discovery data is not sent over the network, so the user must create an instance of a [DDS_PublicationBuiltinTopicData](#) structure in application A. The user then must fill in the QoS describing this remote [DDSDataWriter](#), including the writer's object ID and three-second deadline. Then the user must assert this [DDS_PublicationBuiltinTopicData](#) describing the remote entity with the discovery plugin. The pre-configured QoS must match the actual QoS of the remote entity, or communication will not occur.

7.39.2 Member Data Documentation

7.39.2.1 struct DDS_BuiltinTopicKey_t DDS_PublicationBuiltinTopicData::key

DCPS key to distinguish entries. This is used to configure the object ID of the remote [DDSDataWriter](#).

7.39.2.2 struct DDS_BuiltinTopicKey_t DDS_PublicationBuiltinTopicData::participant_key

DCPS key of the participant to which the DataWriter belongs.

7.39.2.3 char* DDS_PublicationBuiltinTopicData::topic_name

Name of the related [DDSTopic](#).

The length of this string is limited to 255 characters.

The memory for this field is managed as described in [String Support](#).

See also

[String Support](#)

7.39.2.4 char* DDS_PublicationBuiltinTopicData::type_name

Name of the type attached to the [DDSTopic](#).

The length of this string is limited to 255 characters.

The memory for this field is managed as described in [String Support](#).

See also

[String Support](#)

7.39.2.5 struct DDS_DeadlineQosPolicy DDS_PublicationBuiltinTopicData::deadline

Policy of the corresponding DataWriter.

7.39.2.6 struct DDS_OwnershipQosPolicy DDS_PublicationBuiltinTopicData::ownership

Policy of the corresponding DataWriter.

7.39.2.7 struct DDS_OwnershipStrengthQosPolicy DDS_PublicationBuiltinTopicData::ownership_strength

Policy of the corresponding DataWriter.

7.39.2.8 struct DDS_ReliabilityQosPolicy DDS_PublicationBuiltinTopicData::reliability

Policy of the corresponding DataWriter.

7.39.2.9 struct DDS_LivelinessQosPolicy DDS_PublicationBuiltinTopicData::liveliness

Policy of the corresponding DataWriter.

7.39.2.10 struct DDS_DurabilityQosPolicy DDS_PublicationBuiltinTopicData::durability

durability policy of the corresponding DataWriter

7.39.2.11 struct DDS_DestinationOrderQosPolicy DDS_PublicationBuiltinTopicData::destination_order

Policy of the corresponding DataWriter.

7.39.2.12 struct DDS_LocatorSeq DDS_PublicationBuiltinTopicData::unicast_locator

Custom unicast locator sequence that the endpoint can specify. The default locator sequence from the [DDSDomainParticipant](#) will be used if this is not specified.

7.40 DDS_PublicationMatchedStatus Struct Reference

<<cert>> [DDS_PUBLICATION_MATCHED_STATUS](#)

Public Attributes

- [DDS_Long total_count](#)
The total cumulative number of times the concerned [DDSDataWriter](#) discovered a "match" with a [DDSDataReader](#).
- [DDS_Long total_count_change](#)
The incremental changes in total_count since the last time the listener was called or the status was read.
- [DDS_Long current_count](#)
The current number of readers with which the [DDSDataWriter](#) is matched.
- [DDS_Long current_count_change](#)
The change in current_count since the last time the listener was called or the status was read.
- [DDS_InstanceHandle_t last_subscription_handle](#)
A handle to the last [DDSDataReader](#) that caused the the [DDSDataWriter](#)'s status to change.

7.40.1 Detailed Description

<<cert>> [DDS_PUBLICATION_MATCHED_STATUS](#)

A "match" happens when the [DDSDataWriter](#) finds a [DDSDataReader](#) for the same [DDSTopic](#) and common partition with a requested QoS that is compatible with that offered by the [DDSDataWriter](#).

This status is also changed (and the listener, if any, called) when a match is ended. A local [DDSDataWriter](#) will become "unmatched" from a remote [DDSDataReader](#) when that [DDSDataReader](#) goes away for any reason.

7.40.2 Member Data Documentation

7.40.2.1 DDS_Long DDS_PublicationMatchedStatus::total_count

The total cumulative number of times the concerned [DDSDataWriter](#) discovered a "match" with a [DDSDataReader](#).

This number increases whenever a new match is discovered. It does not change when an existing match goes away.

7.40.2.2 DDS_Long DDS_PublicationMatchedStatus::total_count_change

The incremental changes in total_count since the last time the listener was called or the status was read.

7.40.2.3 DDS_Long DDS_PublicationMatchedStatus::current_count

The current number of readers with which the [DDSDataWriter](#) is matched.

This number increases when a new match is discovered and decreases when an existing match goes away.

7.40.2.4 DDS_Long DDS_PublicationMatchedStatus::current_count_change

The change in current_count since the last time the listener was called or the status was read.

7.40.2.5 DDS_InstanceHandle_t DDS_PublicationMatchedStatus::last_subscription_handle

A handle to the last [DDSDataReader](#) that caused the the [DDSDataWriter](#)'s status to change.

7.41 DDS_PublisherQos Struct Reference

<<[cert](#)>> QoS policies supported by a [DDSPublisher](#) entity.

Public Attributes

- struct [DDS_EntityFactoryQosPolicy](#) entity_factory
Entity factory policy, [ENTITY_FACTORY](#).

7.41.1 Detailed Description

<<[cert](#)>> QoS policies supported by a [DDSPublisher](#) entity.

7.41.2 Member Data Documentation

7.41.2.1 struct DDS_EntityFactoryQosPolicy DDS_PublisherQos::entity_factory

Entity factory policy, [ENTITY_FACTORY](#).

7.42 DDS_ReliabilityQosPolicy Struct Reference

<<[cert](#)>> Indicates the level of reliability offered/requested by RTI Connex Micro.

Public Attributes

- [DDS_ReliabilityQosPolicyKind](#) kind
Kind of reliability.
- struct [DDS_Duration_t](#) max_blocking_time
For a [DDSDataReader](#), the maximum time a receive thread may block to wait for resources. Unsupported on a [DDSDataWriter](#).

7.42.1 Detailed Description

<<[cert](#)>> Indicates the level of reliability offered/requested by RTI Connex Micro.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#), [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#)

Properties:

[RxO](#) = YES
[Changeable](#) = NO

7.42.2 Usage

This policy indicates the level of reliability requested by a [DDSDataReader](#) or offered by a [DDSDataWriter](#).

These levels are ordered, [DDS_BEST_EFFORT_RELIABILITY_QOS](#) < [DDS_RELIABLE_RELIABILITY_QOS](#). A [DDSDataWriter](#) offering a level is implicitly offering all levels below.

If the [DDS_ReliabilityQosPolicy::kind](#) is set to [DDS_RELIABLE_RELIABILITY_QOS](#), data-samples originating from a single [DDSDataWriter](#) cannot be made available to the [DDSDataReader](#) if there are previous data samples that have not been received yet due to a communication error. In other words, the service will repair the error and re-transmit data-samples as needed in order to reconstruct a correct snapshot of the [DDSDataWriter](#) history before it is accessible by the [DDSDataReader](#).

If the [DDS_ReliabilityQosPolicy::kind](#) is set to [DDS_BEST_EFFORT_RELIABILITY_QOS](#), the service will not re-transmit missing data-samples. However for data-samples originating from any one DataWriter the service will ensure they are stored in the [DDSDataReader](#) history in the same order they originated in the [DDSDataWriter](#). In other words, the [DDSDataReader](#) may miss some data samples but it will never see the value of a data-object change from a newer value to an older value.

See also

[DDS_HistoryQosPolicy](#)
[DDS_ResourceLimitsQosPolicy](#)

7.42.3 Compatibility

The value offered is considered compatible with the value requested if and only if the inequality *offered kind* >= *requested kind* evaluates to 'TRUE'. For the purposes of this inequality, the values of [DDS_ReliabilityQosPolicy::kind](#) are considered ordered such that [DDS_BEST_EFFORT_RELIABILITY_QOS](#) < [DDS_RELIABLE_RELIABILITY_QOS](#).

7.42.4 Member Data Documentation

7.42.4.1 [DDS_ReliabilityQosPolicyKind](#) [DDS_ReliabilityQosPolicy::kind](#)

Kind of reliability.

[default] [DDS_BEST_EFFORT_RELIABILITY_QOS](#)

7.42.4.2 [struct DDS_Duration_t](#) [DDS_ReliabilityQosPolicy::max_blocking_time](#)

For a [DDSDataReader](#), the maximum time a receive thread may block to wait for resources. Unsupported on a [DDSDataWriter](#).

For a [DDSDataReader](#), a non-zero maximum blocking time enables the receive thread to block up to the maximum blocking time if the [DDSDataReader](#) is out of resources when new data arrives. This is useful when the history depth is equal to `max_samples_per_instance` and the application is using a combination of loan and waitsets or polling. If the maximum blocking value is 0 a receive thread will not block.

For [DDSDataWriter](#), only a maximum blocking time of 0 is supported. That is, a [DDSDataWriter](#) will not block to wait for resources if the [DDSDataWriter](#) is out of resources at the time of write.

[DDSDataReader](#) [range] [0,1 year] [default] 0

[DDSDataWriter](#) [range] [0,0] [default] 0

See also

[DDS_ResourceLimitsQosPolicy](#)

7.43 DDS_ReliableReaderActivityChangedStatus Struct Reference

[<<eXtension>>](#) [<<cert>>](#) Describes the activity (i.e. are acknowledgements forthcoming) of reliable readers matched to a reliable writer.

Public Attributes

- [DDS_Long active_count](#)
The current number of reliable readers currently matched with this reliable writer.
- [DDS_Long inactive_count](#)
The number of reliable readers that have been dropped by this reliable writer because they failed to send acknowledgements in a timely fashion.
- [DDS_Long active_count_change](#)
The most recent change in the number of active remote reliable readers.
- [DDS_Long inactive_count_change](#)
The most recent change in the number of inactive remote reliable readers.
- [DDS_InstanceHandle_t last_instance_handle](#)
The instance handle of the last reliable remote reader to be determined inactive.

7.43.1 Detailed Description

[<<eXtension>>](#) [<<cert>>](#) Describes the activity (i.e. are acknowledgements forthcoming) of reliable readers matched to a reliable writer.

Entity:

[DDSDataWriter](#)

Listener:

[DDSDataWriterListener](#)

This status is the reciprocal status to the [DDS_LivelinessChangedStatus](#) on the reader. It is different than the [DDS_LivelinessLostStatus](#) on the writer in that the latter informs the writer about its own liveliness; this status informs the writer about the "liveliness" (activity) of its matched readers.

All counts in this status will remain at zero for best effort writers.

7.43.2 Member Data Documentation

7.43.2.1 DDS_Long DDS_ReliableReaderActivityChangedStatus::active_count

The current number of reliable readers currently matched with this reliable writer.

7.43.2.2 **DDS_Long DDS_ReliableReaderActivityChangedStatus::inactive_count**

The number of reliable readers that have been dropped by this reliable writer because they failed to send acknowledgements in a timely fashion.

A reader is considered to be inactive after is has been sent heartbeats [DDS_RtpsReliableWriterProtocol_t::max_heartbeat_retries](#) times, each heartbeat having been separated from the previous by the current heartbeat period.

7.43.2.3 **DDS_Long DDS_ReliableReaderActivityChangedStatus::active_count_change**

The most recent change in the number of active remote reliable readers.

7.43.2.4 **DDS_Long DDS_ReliableReaderActivityChangedStatus::inactive_count_change**

The most recent change in the number of inactive remote reliable readers.

7.43.2.5 **DDS_InstanceHandle_t DDS_ReliableReaderActivityChangedStatus::last_instance_handle**

The instance handle of the last reliable remote reader to be determined inactive.

7.44 **DDS_ReliableSampleUnacknowledgedStatus Struct Reference**

<<[cert](#)>> Status about unacknowledged samples.

Public Attributes

- struct [DDS_SequenceNumber_t](#) [sequence_number](#)
- [DDS_Long](#) [unacknowledged_count](#)
- [DDS_InstanceHandle_t](#) [instance_handle](#)

7.44.1 Detailed Description

<<[cert](#)>> Status about unacknowledged samples.

Entity:

[DDSDDataWriter](#)

Listener:

[DDSDDataWriterListener](#)

7.44.2 Member Data Documentation

7.44.2.1 **struct DDS_SequenceNumber_t DDS_ReliableSampleUnacknowledgedStatus::sequence_number**

7.44.2.2 **DDS_Long DDS_ReliableSampleUnacknowledgedStatus::unacknowledged_count**

7.44.2.3 **DDS_InstanceHandle_t DDS_ReliableSampleUnacknowledgedStatus::instance_handle**

7.45 **DDS_RequestedDeadlineMissedStatus Struct Reference**

<<[cert](#)>> [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#)

Public Attributes

- [DDS_Long total_count](#)
Total cumulative count of the deadlines detected for any instance read by the [DDSDDataReader](#).
- [DDS_Long total_count_change](#)
The incremental number of deadlines detected since the last time the listener was called or the status was read.
- [DDS_InstanceHandle_t last_instance_handle](#)
Handle to the last instance in the [DDSDDataReader](#) for which a deadline was detected.

7.45.1 Detailed Description

<<cert>> [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#)

7.45.2 Member Data Documentation

7.45.2.1 DDS_Long DDS_RequestedDeadlineMissedStatus::total_count

Total cumulative count of the deadlines detected for any instance read by the [DDSDDataReader](#).

7.45.2.2 DDS_Long DDS_RequestedDeadlineMissedStatus::total_count_change

The incremental number of deadlines detected since the last time the listener was called or the status was read.

7.45.2.3 DDS_InstanceHandle_t DDS_RequestedDeadlineMissedStatus::last_instance_handle

Handle to the last instance in the [DDSDDataReader](#) for which a deadline was detected.

7.46 DDS_RequestedIncompatibleQosStatus Struct Reference

<<cert>> [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#)

Public Attributes

- [DDS_Long total_count](#)
Total cumulative count of how many times the concerned [DDSDDataReader](#) discovered a [DDSDDataWriter](#) for the same [DDSTopic](#) with an offered QoS that is incompatible with that requested by the [DDSDDataReader](#).
- [DDS_Long total_count_change](#)
The change in `total_count` since the last time the listener was called or the status was read.
- [DDS_QosPolicyId_t last_policy_id](#)
The `PolicyId_t` of one of the policies that was found to be incompatible the last time an incompatibility was detected.
- struct [DDS_QosPolicyCountSeq policies](#)
A list containing, for each policy, the total number of times that the concerned [DDSDDataReader](#) discovered a [DDSDDataWriter](#) for the same [DDSTopic](#) with an offered QoS that is incompatible with that requested by the [DDSDDataReader](#).

7.46.1 Detailed Description

<<cert>> [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#)

See also

[RELIABILITY](#)
[OWNERSHIP](#)
[LIVELINESS](#)
[DEADLINE](#)

7.46.2 Member Data Documentation

7.46.2.1 DDS_Long DDS_RequestedIncompatibleQosStatus::total_count

Total cumulative count of how many times the concerned [DDSDataReader](#) discovered a [DDSDataWriter](#) for the same [DDSTopic](#) with an offered QoS that is incompatible with that requested by the [DDSDataReader](#).

7.46.2.2 DDS_Long DDS_RequestedIncompatibleQosStatus::total_count_change

The change in `total_count` since the last time the listener was called or the status was read.

7.46.2.3 DDS_QosPolicyId_t DDS_RequestedIncompatibleQosStatus::last_policy_id

The PolicyId_t of one of the policies that was found to be incompatible the last time an incompatibility was detected.

7.46.2.4 struct DDS_QosPolicyCountSeq DDS_RequestedIncompatibleQosStatus::policies

A list containing, for each policy, the total number of times that the concerned [DDSDataReader](#) discovered a [DDSDataWriter](#) for the same [DDSTopic](#) with an offered QoS that is incompatible with that requested by the [DDSDataReader](#).

7.47 DDS_ResourceLimitsQosPolicy Struct Reference

<<[cert](#)>> Specifies the resources that RTI Connex Micro can consume in order to meet the requested QoS.

Public Attributes

- [DDS_Long max_samples](#)
Represents the maximum samples the middleware can store for any one [DDSDataWriter](#) or [DDSDataReader](#).
- [DDS_Long max_instances](#)
Represents the maximum number of instances a [DDSDataWriter](#) (or [DDSDataReader](#)) can manage.
- [DDS_Long max_samples_per_instance](#)
Represents the maximum number of samples of any one instance a [DDSDataWriter](#) or [DDSDataReader](#) can manage.

7.47.1 Detailed Description

<<[cert](#)>> Specifies the resources that RTI Connex Micro can consume in order to meet the requested QoS.

Entity:

[DDSTopic](#), [DDSDataReader](#), [DDSDataWriter](#)

Status:

[DDS_SAMPLE_REJECTED_STATUS](#), [DDS_SampleRejectedStatus](#)

Properties:

[RxO](#) = NO
[Changeable](#) = NO

7.47.2 Usage

This policy controls the resources that RTI Connex Micro can use in order to meet the requirements imposed by the application and other QoS settings.

If [DDSDataWriter](#) objects are communicating samples faster than they are ultimately taken by the [DDSDataReader](#) objects, the middleware will eventually hit against some of the QoS-imposed resource limits. Note that this may occur when just a single [DDSDataReader](#) cannot keep up with its corresponding [DDSDataWriter](#). The behavior in this case depends on the setting for the [RELIABILITY](#). If reliability is [DDS_BEST_EFFORT_RELIABILITY_QOS](#) then RTI Connex Micro is allowed to drop samples. If the reliability is [DDS_RELIABLE_RELIABILITY_QOS](#), RTI Connex Micro will block the [DDSDataWriter](#) or discard the sample at the [DDSDataReader](#) in order not to lose existing samples.

See also

[DDS_ReliabilityQosPolicy](#)
[DDS_HistoryQosPolicy](#)

7.47.3 Consistency

All limits must be set to positive finite values. Furthermore, `max_samples` must be large enough to contain all samples for all instances:

```
max_samples >= max_instances *
    max_samples_per_instance
```

The setting of [RESOURCE_LIMITS](#) `max_samples_per_instance` must be consistent with the [HISTORY](#) `depth`. For these two QoS to be consistent, they must verify that *depth* ≤ *max_samples_per_instance*.

See also

[DDS_HistoryQosPolicy](#)

7.47.4 Member Data Documentation

7.47.4.1 DDS_Long DDS_ResourceLimitsQosPolicy::max_samples

Represents the maximum samples the middleware can store for any one [DDSDataWriter](#) or [DDSDataReader](#).

Specifies the maximum number of data samples a [DDSDataWriter](#) or [DDSDataReader](#) can manage across all the instances associated with it.

For unkeyed types this value has to be equal to `max_samples_per_instance`.

[DDSDataReader](#)

The [DDS_ResourceLimitsQosPolicy::max_samples](#) resource-limit limits the maximum number of samples a [DDS-DataReader](#) can allocate and cache. Samples are cached before they are made available to the application; for example, when samples are received out-of-order. Thus, this resource limit may impact throughput and latency.

What this value should be set to is determined by a number of factors:

- For best-effort, it can be set to 1 since samples are not cached. However, if data is polled or samples are loaned longer than the update frequency, this resource-limit should be increased to prevent samples from being reused before the application has a chance to access the data (along with [DDS_HistoryQosPolicy::depth](#)). If For reliable data, the value should take into account the data-rate, the heartbeat rate of the [DDSDataWriter](#), and the number of piggy-backed heartbeats.

[DDSDataWriter](#)

The [DDS_ResourceLimitsQosPolicy::max_samples](#) resource-limit limits the maximum number of samples a [DDS-DataWriter](#) can allocate and maintain state for at any given time. Every time a write operation is called, a sample is allocated from the cache and filled in with user-data.

A sample allocated from the pool controlled by this resource-limit can be reused based on the following rules:

- If the [DDSDataWriter](#) is using [DDS_BEST_EFFORT_RELIABILITY_QOS](#) effort, the sample is returned for reuse after the transport returns from the write call.

- If the `DDSDDataWriter` is using `DDS_RELIABLE_RELIABILITY_QOS` and `DDS_VOLATILE_DURABILITY_QOS`, the sample is returned when all known `DataReaders` have acknowledged the sample or the `DDSDataWriter` has given up on delivering the sample (controlled by `DDS_RtpsReliableWriterProtocol_t::max_heartbeat_retries`), whichever happens first.
- If the `DDSDDataWriter` is using `DDS_RELIABLE_RELIABILITY_QOS` and not using `DDS_VOLATILE_DURABILITY_QOS`, the sample is returned to the pool for reuse only if the history depth is exceeded and the `DDS_HistoryQosPolicy::kind` is set to `DDS_KEEP_LAST_HISTORY_QOS`

For best-effort communication, this resource-limit can be set to 1.

For reliable communication, the value of this resource-limit determines the maximum number of samples a `DDSDataWriter` can keep in unacknowledged state before reusing potentially unacknowledged samples (for `DDS_KEEP_LAST_HISTORY_QOS`). For reliable data, the value should take into account the data-rate, the `DDS_RtpsReliableWriterProtocol_t::heartbeat_period`, and `DDS_RtpsReliableWriterProtocol_t::heartbeats_per_max_samples`.

See also

`DDS_HistoryQosPolicy::depth`, `DDS_DataWriterQos`

[default] 1

[range] [1, 100 million] $\geq (\text{max_samples_per_instance} * \text{max_instances})$,

7.47.4.2 `DDS_Long DDS_ResourceLimitsQosPolicy::max_instances`

Represents the maximum number of instances a `DDSDDataWriter` (or `DDSDDataReader`) can manage.

`DDSDDataReader`

The `max_instance` resource-limit limits the maximum number of instances a `DDSDDataReader` can manage. Instance state is one of the most expensive resources to maintain, as well as often being the highest one.

The resources allocated by an instance are normally only released when an instance is disposed *and* all known owners have unregistered the instance; please refer to `FooDataWriter::unregister_instance` and `FooDataWriter::dispose`.

The `DDS_DataReaderResourceLimitsQosPolicy::instance_replacement` policy is useful to maintain state for only the last N instances, and when it is not necessary to keep samples in the `DDSDDataReader` history cache. When the `DDS_DataReaderResourceLimitsQosPolicy::instance_replacement` policy is set to replace the oldest instance (`DDS_REPLACE_OLDEST_INSTANCE_REPLACEMENT_QOS`), information about the oldest instance is automatically removed and the newly discovered instance is received. The disadvantage is that `DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE` state cannot be maintained. The `DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE` is maintained for an instance that is accepted (either there is space or another instance is replaced), regardless of whether any samples have previously been received.

See also

`DDS_DataReaderResourceLimitsInstanceReplacementKind` `instance_replacement`

`DDSDDataWriter`

The `DDS_ResourceLimitsQosPolicy::max_instances` resource limit limits the maximum number of instances a `DDSDDataWriter` can manage. An instance resource can only be reused when it is unregistered by the `DDSDDataWriter`.

[default] 1

[range] [1, 1 million]

7.47.4.3 `DDS_Long DDS_ResourceLimitsQosPolicy::max_samples_per_instance`

Represents the maximum number of samples of any one instance a `DDSDDataWriter` or `DDSDDataReader` can manage.

`DDSDDataReader`

This resource-limit limits the maximum number of samples that can be cached for a single instance; it must be less than or equal to `max_samples` and greater or equal to the history depth. When `DDS_ResourceLimitsQosPolicy::max_samples_per_instance` is higher than depth, it makes it possible to loan samples and receive up to depth samples before returning loaned samples.

When this resource-limit or `DDS_HistoryQosPolicy::depth` is exceeded and the `DDS_HistoryQosPolicy::kind` is `DDS_KEEP_LAST_HISTORY_QOS`, the oldest sample is removed from the cache.

Note that it is possible to exceed the `max_samples_per_instance` limit while samples are being received and processed, but before they are added to the reader queue. This typically happens when there are holes in the sequence numbers that must be filled before the samples can be committed to the reader queue. If a sample cannot be committed due to the `max_samples_per_instance` limit, it is not dropped. Instead, it is recommitted when space is made available, such as the application taking samples from the reader queue or older samples being removed due to `KEEP_LAST`.

For unkeyed types this value has to be equal to `max_samples`.

DDSDataWriter

This resource-limit limits the maximum number of samples that can be cached for a single instance; it must be less than or equal to `max_samples` and greater or equal to the `DDS_HistoryQosPolicy::depth`.

`DDS_HistoryQosPolicy::depth` determines how many samples are kept in case of `DDS_TRANSIENT_LOCAL_DURABILITY_QOS` durability and made available to late joining `DDSDataReader`, while `DDS_ResourceLimitsQosPolicy::max_samples_per_instance` is the maximum number of *unacknowledged* samples the `DDSDataWriter` can keep per instance. Thus, if `max_samples_per_instance > depth`, it is possible to keep unacknowledged samples while limiting the number of samples made available to late joining DataReaders.

[default] 1

[range] [1, 100 million], $\leq$ `max_samples`, $\geq$ `DDS_HistoryQosPolicy::depth`

7.48 DDS_RtpsReliableReaderProtocol_t Struct Reference

Qos related to reliable reader protocol defined in RTPS.

Public Attributes

- struct `DDS_Duration_t` `nack_period`
The period at which to send NACKs.

7.48.1 Detailed Description

Qos related to reliable reader protocol defined in RTPS.

It is used to configure the reliable reader according to RTPS protocol.

Properties:

`RxO` = N/A
`Changeable` = NO

QoS:

`DDS_DataReaderProtocolQosPolicy`

7.48.2 Member Data Documentation

7.48.2.1 struct `DDS_Duration_t` `DDS_RtpsReliableReaderProtocol_t::nack_period`

The period at which to send NACKs.

A reliable reader will send periodic NACKs at this rate when it first matches with a reliable writer. The reader will stop sending NACKs when it has received all available historical data from the writer. Note that this NACK period only determines the rate at which NACKs are sent until the first HB is received from the writer. When the first HB is received, the writer's HB period determines how fast historical data is sent.

[default] 50 ms

[range] [1 nanosec, 1 year]

7.49 DDS_RtpsReliableWriterProtocol_t Struct Reference

<<cert>> QoS related to the reliable writer protocol defined in RTPS.

Public Attributes

- struct [DDS_Duration_t heartbeat_period](#)
The period at which to send heartbeats.
- [DDS_Long heartbeats_per_max_samples](#)
The number of heartbeats per send queue.
- [DDS_Long max_send_window](#)
The maximum size of a reliable send window.
- [DDS_Long max_heartbeat_retries](#)
The maximum number of heartbeat retries before marking a remote reader as inactive.

7.49.1 Detailed Description

<<cert>> QoS related to the reliable writer protocol defined in RTPS.

It is used to configure a reliable writer according to RTPS protocol.

Properties:

RxO = N/A
Changeable = NO

QoS:

[DDS_DataWriterProtocolQosPolicy](#)

7.49.2 Member Data Documentation

7.49.2.1 struct DDS_Duration_t DDS_RtpsReliableWriterProtocol_t::heartbeat_period

The period at which to send heartbeats.

A reliable writer will send periodic heartbeat at this rate.

[default] 3.0 seconds

[range] [1 nanosec, 1 year]

7.49.2.2 DDS_Long DDS_RtpsReliableWriterProtocol_t::heartbeats_per_max_samples

The number of heartbeats per send queue.

A piggyback heartbeat will be sent every [[DDS_ResourceLimitsQosPolicy::max_samples](#)/heartbeats_per_max_samples] number of samples.

If set to zero, no piggyback heartbeat will be sent. If [DDS_ResourceLimitsQosPolicy::max_samples](#) is [DDS_LENGTH_UNLIMITED](#), 100 million is assumed as the maximum value in the calculation.

[default] 1

[range] [0, 100 million]

- For [DDS_DataWriterProtocolQosPolicy::rtps_reliable_writer](#):
heartbeats_per_max_samples ≤ [DDS_ResourceLimitsQosPolicy::max_samples](#)

7.49.2.3 DDS_Long DDS_RtpsReliableWriterProtocol_t::max_send_window

The maximum size of a reliable send window.

A reliable DataWriter may maintain a send window of samples in-flight that have not been acknowledged by its DataReaders. The max_send_window is the maximum number of unacknowledged samples in-flight. When the window is at max_send_window, subsequent writes will pass but not publish any data over the network.

[default] DDS_LENGTH_UNLIMITED, meaning the window will be as large as resource limits allow

[range] [1, 256] of [DDS_LENGTH_UNLIMITED](#)

7.49.2.4 DDS_Long DDS_RtpsReliableWriterProtocol_t::max_heartbeat_retries

The maximum number of heartbeat retries before marking a remote reader as inactive.

When a remote reader has not acked all the samples the reliable writer has in its queue, and max_heartbeat_retries number of periodic heartbeats has been sent without receiving any ack/nack back, the remote reader will be marked as inactive (not alive) and be ignored until it resumes sending ack/nack.

[default] [DDS_LENGTH_UNLIMITED](#)

[range] [1, 1 million] or [DDS_LENGTH_UNLIMITED](#)

7.50 DDS_RtpsWellKnownPorts_t Struct Reference

RTPS well-known port mapping configuration.

Public Attributes

- [DDS_Long port_base](#)
The base port offset.
- [DDS_Long domain_id_gain](#)
Tunable domain gain parameter.
- [DDS_Long participant_id_gain](#)
Tunable participant gain parameter.
- [DDS_Long builtin_multicast_port_offset](#)
*Additional offset for **metatraffic** multicast port.*
- [DDS_Long builtin_unicast_port_offset](#)
*Additional offset for **metatraffic** unicast port.*
- [DDS_Long user_multicast_port_offset](#)
*Additional offset for **usertraffic** multicast port.*
- [DDS_Long user_unicast_port_offset](#)
*Additional offset for **usertraffic** unicast port.*

7.50.1 Detailed Description

RTPS well-known port mapping configuration.

RTI Connex Micro uses the RTPS wire protocol. The discovery protocols defined by RTPS rely on well-known ports to initiate discovery. These well-known ports define the multicast and unicast ports on which a Participant will listen

for discovery **metatraffic** from other Participants. The discovery metatraffic contains all the information required to establish the presence of remote DDS entities in the network.

The well-known ports are defined by RTPS in terms of port mapping expressions with several tunable parameters, which allow the user to customize what network ports are used by the middleware. These parameters are exposed in [DDS_RtpsWellKnownPorts_t](#). In order for all Participants in a system to correctly discover each other, it is important that they all use the same port mapping expressions.

The actual port mapping expressions, as defined by the RTPS specification, can be found below. In addition to the parameters listed in [DDS_RtpsWellKnownPorts_t](#), the port numbers depend on:

- `domain_id`, as specified in [DDSDomainParticipantFactory::create_participant](#)
- `participant_id`, as specified using [DDS_WireProtocolQosPolicy::participant_id](#)

The `domain_id` parameter ensures no port conflicts exist between Participants belonging to different domains. This also means that discovery metatraffic in one domain is not visible to Participants in a different domain. The `participant_id` parameter ensures that unique unicast port numbers are assigned to Participants belonging to the same domain on a given host.

The `metatraffic_unicast_port` is used to exchange discovery metatraffic using unicast.

```
metatraffic_unicast_port = port_base + (domain_id_gain * domain_id) + (participant_id_gain * participant_id) +
```

The `metatraffic_multicast_port` is used to exchange discovery metatraffic using multicast.

```
metatraffic_multicast_port = port_base + (domain_id_gain * domain_id) + builtin_multicast_port_offset
```

RTPS also defines the *default* multicast and unicast ports on which DataReaders and DataWriters receive **user-traffic**.

The `usertraffic_unicast_port` is used to exchange user data using unicast.

```
usertraffic_unicast_port = port_base + (domain_id_gain * domain_id) + (participant_id_gain * participant_id) +
```

The `usertraffic_multicast_port` is used to exchange user data using multicast. The corresponding multicast group addresses can be configured using

```
usertraffic_multicast_port = port_base + (domain_id_gain * domain_id) + user_multicast_port_offset
```

By default, the port mapping parameters are configured to compliant with OMG's DDS Interoperability Wire Protocol (see also [DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS](#)).

The OMG's DDS Interoperability Wire Protocol compliant port mapping parameters are *not* backwards compatible with previous versions of the RTI Connex Micro middleware.

When modifying the port mapping parameters, care must be taken to avoid port aliasing. This would result in undefined discovery behavior. The chosen parameter values will also determine the maximum possible number of domains in the system and the maximum number of participants per domain. Additionally, any resulting mapped port number must be within the range imposed by the underlying transport. For example, for UDPv4, this range typically equals [1024 - 65535].

QoS:

[DDS_WireProtocolQosPolicy](#)

7.50.2 Member Data Documentation

7.50.2.1 DDS_Long DDS_RtpsWellKnownPorts_t::port_base

The base port offset.

All mapped well-known ports are offset by this value.

[default] 7400

[range] [≥ 1], but resulting ports must be within the range imposed by the underlying transport.

7.50.2.2 DDS_Long DDS_RtpsWellKnownPorts_t::domain_id_gain

Tunable domain gain parameter.

Multiplier of the `domain_id`. Together with `participant_id_gain`, it determines the highest `domain_id` and `participant_id` allowed on this network.

In general, there are two ways to setup `domain_id_gain` and `participant_id_gain` parameters.

If `domain_id_gain > participant_id_gain`, it results in a port mapping layout where all [DDSDomainParticipant](#) instances within a single domain occupy a consecutive range of `domain_id_gain` ports. Precisely, all ports occupied by the domain fall within:

```
(port_base + (domain_id_gain * domain_id))
```

and:

```
(port_base + (domain_id_gain * (domain_id + 1)) - 1)
```

Under such a case, the highest `domain_id` is limited only by the underlying transport's maximum port. The highest `participant_id`, however, must satisfy:

```
max_participant_id < (domain_id_gain / participant_id_gain)
```

On the contrary, if `domain_id_gain <= participant_id_gain`, it results in a port mapping layout where a given domain's [DDSDomainParticipant](#) instances occupy ports spanned across the entire valid port range allowed by the underlying transport. For instance, it results in the following potential mapping:

Mapped Port	Domain Id	Participant ID
higher port number	Domain Id = 1	Participant ID = 2
	Domain Id = 0	Participant ID = 2
	Domain Id = 1	Participant ID = 1
	Domain Id = 0	Participant ID = 1
	Domain Id = 1	Participant ID = 0
lower port number	Domain Id = 0	Participant ID = 0

Under this case, the highest `participant_id` is limited only by the underlying transport's maximum port. The highest `domain_id`, however, must satisfy:

```
max_domain_id < (participant_id_gain / domain_id_gain)
```

Additionally, `domain_id_gain` also determines the range of the port-specific offsets.

```
domain_id_gain > abs(builtin_multicast_port_offset - user_multicast_port_offset)
```

```
domain_id_gain > abs(builtin_unicast_port_offset - user_unicast_port_offset)
```

Violating this may result in port aliasing and undefined discovery behavior.

[default] 250

[range] [> 0], but resulting ports must be within the range imposed by the underlying transport.

7.50.2.3 DDS_Long DDS_RtpsWellKnownPorts_t::participant_id_gain

Tunable participant gain parameter.

Multiplier of the `participant_id`. See [DDS_RtpsWellKnownPorts_t::domain_id_gain](#) for its implications on the highest `domain_id` and `participant_id` allowed on this network.

Additionally, `participant_id_gain` also determines the range of `builtin_unicast_port_offset` and `user_unicast_port_offset`.

```
participant_id_gain > abs(builtin_unicast_port_offset - user_unicast_port_offset)
```

[default] 2

[range] [> 0], but resulting ports must be within the range imposed by the underlying transport.

7.50.2.4 DDS_Long DDS_RtpsWellKnownPorts_t::builtin_multicast_port_offset

Additional offset for **metatraffic** multicast port.

It must be unique from other port-specific offsets.

[default] 0

[range] [≥ 0], but resulting ports must be within the range imposed by the underlying transport.

7.50.2.5 DDS_Long DDS_RtpsWellKnownPorts_t::builtin_unicast_port_offset

Additional offset for **metatraffic** unicast port.

It must be unique from other port-specific offsets.

[default] 10

[range] [≥ 0], but resulting ports must be within the range imposed by the underlying transport.

7.50.2.6 DDS_Long DDS_RtpsWellKnownPorts_t::user_multicast_port_offset

Additional offset for **usertraffic** multicast port.

It must be unique from other port-specific offsets.

[default] 1

[range] [≥ 0], but resulting ports must be within the range imposed by the underlying transport.

7.50.2.7 DDS_Long DDS_RtpsWellKnownPorts_t::user_unicast_port_offset

Additional offset for **usertraffic** unicast port.

It must be unique from other port-specific offsets.

[default] 11

[range] [≥ 0], but resulting ports must be within the range imposed by the underlying transport.

7.51 DDS_SampleInfo Struct Reference

Information that accompanies each sample that is `read` or `taken`.

Public Attributes

- [DDS_SampleStateKind sample_state](#)
The sample state of the sample.
- [DDS_ViewStateKind view_state](#)
The view state of the instance.
- [DDS_InstanceStateKind instance_state](#)
The instance state of the instance.
- struct [DDS_Time_t source_timestamp](#)
The timestamp when the sample was written by a DataWriter.
- [DDS_InstanceHandle_t instance_handle](#)
Identifies locally the corresponding instance.
- [DDS_InstanceHandle_t publication_handle](#)
Identifies locally the DataWriter that modified the instance.
- [DDS_Boolean valid_data](#)
Indicates whether the DataSample contains data or else it is only used to communicate a change in the instance_state of the instance.
- struct [DDS_Time_t reception_timestamp](#)

- <<eXtension>> The timestamp when the sample was committed by a DataReader.*
- struct [DDS_SequenceNumber_t](#) `publication_sequence_number`
<<eXtension>> The publication sequence number.

7.51.1 Detailed Description

Information that accompanies each sample that is `read` or `taken`.

7.51.2 Interpretation of the SampleInfo

The [DDS_SampleInfo](#) contains information pertaining to the associated `Data` instance sample including:

- the `sample_state` of the `Data` value (i.e., if it has already been read or not)
- the `view_state` of the related instance (i.e., if the instance is new or not)
- the `instance_state` of the related instance (i.e., if the instance is alive or not)
- the `valid_data` flag. This flag indicates whether there is data associated with the sample. Some samples do not contain data indicating only a change on the `instance_state` of the corresponding instance.
- The `source_timestamp` of the sample. This is the timestamp provided by the [DDSDataWriter](#) at the time the sample was produced.

See also

[DDS_SampleStateKind](#), [DDS_InstanceStateKind](#), [DDS_ViewStateKind](#), [DDS_SampleInfo::valid_data](#)

7.51.3 Member Data Documentation

7.51.3.1 DDS_SampleStateKind DDS_SampleInfo::sample_state

The sample state of the sample.

Indicates whether or not the corresponding data sample has already been read.

See also

[DDS_SampleStateKind](#)

7.51.3.2 DDS_ViewStateKind DDS_SampleInfo::view_state

The view state of the instance.

Indicates whether the [DDSDataReader](#) has already seen samples for the most-current generation of the related instance.

See also

[DDS_ViewStateKind](#)

7.51.3.3 DDS_InstanceStateKind DDS_SampleInfo::instance_state

The instance state of the instance.

Indicates whether the instance is currently in existence or, if it has been disposed, the reason why it was disposed.

See also

[DDS_InstanceStateKind](#)

7.51.3.4 struct DDS_Time_t DDS_SampleInfo::source_timestamp

The timestamp when the sample was written by a DataWriter.

7.51.3.5 DDS_InstanceHandle_t DDS_SampleInfo::instance_handle

Identifies locally the corresponding instance.

7.51.3.6 DDS_InstanceHandle_t DDS_SampleInfo::publication_handle

Identifies locally the DataWriter that modified the instance.

7.51.3.7 DDS_Boolean DDS_SampleInfo::valid_data

Indicates whether the `DataSample` contains data or else it is only used to communicate a change in the `instance_state` of the instance.

Normally each `DataSample` contains both a `DDS_SampleInfo` and some Data. However there are situations where a `DataSample` contains only the `DDS_SampleInfo` and does not have any associated data. This occurs when the RTI Connex Micro notifies the application of a change of state for an instance that was caused by some internal mechanism (such as a timeout) for which there is no associated data. An example of this situation is when the RTI Connex Micro detects that an instance has no writers and changes the corresponding `instance_state` to `DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE`.

The application can distinguish whether a particular `DataSample` has data by examining the value of the `valid_data` flag. If this flag is set to `DDS_BOOLEAN_TRUE`, then the `DataSample` contains valid Data. If the flag is set to `DDS_BOOLEAN_FALSE`, the `DataSample` contains no Data.

To ensure correctness and portability, the `valid_data` flag must be examined by the application prior to accessing the Data associated with the `DataSample` and if the flag is set to `DDS_BOOLEAN_FALSE`, the application should not access the Data associated with the `DataSample`, that is, the application should access only the `DDS_SampleInfo`.

7.51.3.8 struct DDS_Time_t DDS_SampleInfo::reception_timestamp

<<eXtension>> The timestamp when the sample was committed by a DataReader.

The `reception_timestamp` is platform dependent and depends on the port of RTI Connex Micro to the platform. Typically, it represents the current local time, but that may not always be the case. E.g, some platforms may not have a real-time clock and the system time is the time since the system was started.

A `reception_timestamp` equal to zero may mean any of the following:

- RTI Connex Micro was unable to read the current time
- The system time is not incrementing
- Zero time has elapsed between the system was started and the first sample was committed.

7.51.3.9 struct DDS_SequenceNumber_t DDS_SampleInfo::publication_sequence_number

<<eXtension>> The publication sequence number.

7.52 DDS_SampleInfoSeq Struct Reference

Declares IDL sequence < `DDS_SampleInfo` > .

7.52.1 Detailed Description

Declares IDL sequence < `DDS_SampleInfo` > .

See also

[FooSeq](#)

7.53 DDS_SampleLostStatus Struct Reference

<<*cert*>> [DDS_SAMPLE_LOST_STATUS](#)

Public Attributes

- [DDS_Long total_count](#)
Total cumulative count of all samples lost across all instances of data published under the [DDSTopic](#).
- [DDS_Long total_count_change](#)
The incremental number of samples lost since the last time the listener was called or the status was read.
- [DDS_SampleLostStatusKind reason](#)
The reason a sample was lost.
- struct [DDS_SampleInfo sample_info](#)
Additional information about the sample lost for reasons other than.

7.53.1 Detailed Description

<<*cert*>> [DDS_SAMPLE_LOST_STATUS](#)

7.53.2 Member Data Documentation

7.53.2.1 DDS_Long DDS_SampleLostStatus::total_count

Total cumulative count of all samples lost across all instances of data published under the [DDSTopic](#).

7.53.2.2 DDS_Long DDS_SampleLostStatus::total_count_change

The incremental number of samples lost since the last time the listener was called or the status was read.

7.53.2.3 DDS_SampleLostStatusKind DDS_SampleLostStatus::reason

The reason a sample was lost.

7.53.2.4 struct DDS_SampleInfo DDS_SampleLostStatus::sample_info

Additional information about the sample lost for reasons other than.

Samples was lost was by the DataWriter, it was never received by the DataReader.

See also

[DDS_ResourceLimitsQosPolicy](#) and Samples was not lost, normal removal.

See also

[DDS_ResourceLimitsQosPolicy](#)

7.54 DDS_SampleRejectedStatus Struct Reference

[DDS_SAMPLE_REJECTED_STATUS](#)

Public Attributes

- [DDS_Long total_count](#)
Total cumulative count of samples rejected by the [DDSDataReader](#).
- [DDS_Long total_count_change](#)
The incremental number of samples rejected since the last time the listener was called or the status was read.
- [DDS_SampleRejectedStatusKind last_reason](#)
Reason for rejecting the last sample rejected.
- [DDS_InstanceHandle_t last_instance_handle](#)
Handle to the instance being updated by the last sample that was rejected.

7.54.1 Detailed Description

[DDS_SAMPLE_REJECTED_STATUS](#)

A status structure describing the samples rejected by the reader.

7.54.2 Member Data Documentation

7.54.2.1 [DDS_Long](#) [DDS_SampleRejectedStatus::total_count](#)

Total cumulative count of samples rejected by the [DDSDataReader](#).

7.54.2.2 [DDS_Long](#) [DDS_SampleRejectedStatus::total_count_change](#)

The incremental number of samples rejected since the last time the listener was called or the status was read.

7.54.2.3 [DDS_SampleRejectedStatusKind](#) [DDS_SampleRejectedStatus::last_reason](#)

Reason for rejecting the last sample rejected.

See also

[DDS_SampleRejectedStatusKind](#)

7.54.2.4 [DDS_InstanceHandle_t](#) [DDS_SampleRejectedStatus::last_instance_handle](#)

Handle to the instance being updated by the last sample that was rejected.

7.55 [DDS_SequenceNumber_t](#) Struct Reference

Type for *sequence* number representation.

Public Attributes

- [DDS_Long high](#)
The most significant part of the sequence number.
- [DDS_UnsignedLong low](#)
The least significant part of the sequence number.

7.55.1 Detailed Description

Type for *sequence* number representation.

Represents a 64-bit sequence number.

7.55.2 Member Data Documentation

7.55.2.1 DDS_Long DDS_SequenceNumber_t::high

The most significant part of the sequence number.

7.55.2.2 DDS_UnsignedLong DDS_SequenceNumber_t::low

The least significant part of the sequence number.

7.56 DDS_SubscriberQos Struct Reference

<<cert>> QoS policies supported by a [DDSSubscriber](#) entity.

Public Attributes

- struct [DDS_EntityFactoryQosPolicy](#) entity_factory
Entity factory policy, [ENTITY_FACTORY](#).

7.56.1 Detailed Description

<<cert>> QoS policies supported by a [DDSSubscriber](#) entity.

You must set the members in a consistent manner. If not, [DDSSubscriber::set_qos](#) and [DDSDomainParticipant::set_default_subscriber_qos](#) will fail with [DDS_RETCODE_INCONSISTENT_POLICY](#), and [DDSDomainParticipant::create_subscriber](#) will fail.

7.56.2 Member Data Documentation

7.56.2.1 struct DDS_EntityFactoryQosPolicy DDS_SubscriberQos::entity_factory

Entity factory policy, [ENTITY_FACTORY](#).

7.57 DDS_SubscriptionBuiltinTopicData Struct Reference

Entry created when a [DDSDataReader](#) is discovered in association with its Subscriber.

Public Attributes

- struct [DDS_BuiltinTopicKey_t](#) key
DCPS key to distinguish entries. This is used to configure the object ID of the remote [DDSDataReader](#).
- struct [DDS_BuiltinTopicKey_t](#) participant_key
DCPS key of the participant to which the DataReader belongs.
- char * [topic_name](#)
Name of the related [DDSTopic](#).
- char * [type_name](#)
Name of the type attached to the [DDSTopic](#).
- struct [DDS_DeadlineQosPolicy](#) deadline
Policy of the corresponding DataReader.
- struct [DDS_OwnershipQosPolicy](#) ownership
Policy of the corresponding DataReader.
- struct [DDS_ReliabilityQosPolicy](#) reliability

- Policy of the corresponding DataReader.*
- struct [DDS_LivelinessQosPolicy](#) liveliness
 - Policy of the corresponding DataReader.*
- struct [DDS_DurabilityQosPolicy](#) durability
 - Policy of the corresponding DataReader.*
- struct [DDS_DestinationOrderQosPolicy](#) destination_order
 - Policy of the corresponding DataReader.*
- struct [DDS_LocatorSeq](#) unicast_locator
 - Custom unicast locator sequence that the endpoint can specify. The default locator sequence from the [DDSDomainParticipant](#) will be used if this is not specified.*
- struct [DDS_LocatorSeq](#) multicast_locator
 - Custom multicast locator that the endpoint can specify. The default multicast locator sequence from the [DDSDomainParticipant](#) will be used if this is not specified.*

7.57.1 Detailed Description

Entry created when a [DDSDDataReader](#) is discovered in association with its Subscriber.

Data associated with the built-in topic [DDS_SUBSCRIPTION_TOPIC_NAME](#). It contains QoS policies and additional information that apply to the remote [DDSDDataReader](#) the related [DDSSubscriber](#).

7.57.2 Member Data Documentation

7.57.2.1 struct [DDS_BuiltinTopicKey_t](#) [DDS_SubscriptionBuiltinTopicData::key](#)

DCPS key to distinguish entries. This is used to configure the object ID of the remote [DDSDDataReader](#).

Key GUID must be first

7.57.2.2 struct [DDS_BuiltinTopicKey_t](#) [DDS_SubscriptionBuiltinTopicData::participant_key](#)

DCPS key of the participant to which the DataReader belongs.

7.57.2.3 [char*](#) [DDS_SubscriptionBuiltinTopicData::topic_name](#)

Name of the related [DDSTopic](#).

The length of this string is limited to 255 characters.

The memory for this field is managed as described in [String Support](#).

See also

[String Support](#)

7.57.2.4 [char*](#) [DDS_SubscriptionBuiltinTopicData::type_name](#)

Name of the type attached to the [DDSTopic](#).

The length of this string is limited to 255 characters.

The memory for this field is managed as described in [String Support](#).

See also

[String Support](#)

7.57.2.5 `struct DDS_DeadlineQosPolicy DDS_SubscriptionBuiltinTopicData::deadline`

Policy of the corresponding DataReader.

7.57.2.6 `struct DDS_OwnershipQosPolicy DDS_SubscriptionBuiltinTopicData::ownership`

Policy of the corresponding DataReader.

7.57.2.7 `struct DDS_ReliabilityQosPolicy DDS_SubscriptionBuiltinTopicData::reliability`

Policy of the corresponding DataReader.

7.57.2.8 `struct DDS_LivelinessQosPolicy DDS_SubscriptionBuiltinTopicData::liveliness`

Policy of the corresponding DataReader.

7.57.2.9 `struct DDS_DurabilityQosPolicy DDS_SubscriptionBuiltinTopicData::durability`

Policy of the corresponding DataReader.

7.57.2.10 `struct DDS_DestinationOrderQosPolicy DDS_SubscriptionBuiltinTopicData::destination_order`

Policy of the corresponding DataReader.

7.57.2.11 `struct DDS_LocatorSeq DDS_SubscriptionBuiltinTopicData::unicast_locator`

Custom unicast locator sequence that the endpoint can specify. The default locator sequence from the [DDSDomainParticipant](#) will be used if this is not specified.

7.57.2.12 `struct DDS_LocatorSeq DDS_SubscriptionBuiltinTopicData::multicast_locator`

Custom multicast locator that the endpoint can specify. The default multicast locator sequence from the [DDS-DomainParticipant](#) will be used if this is not specified.

7.58 DDS_SubscriptionMatchedStatus Struct Reference

<<cert>> [DDS_SUBSCRIPTION_MATCHED_STATUS](#)

Public Attributes

- [DDS_Long total_count](#)
The total cumulative number of times the concerned [DDSDataReader](#) discovered a "match" with a [DDSDataWriter](#).
- [DDS_Long total_count_change](#)
The change in total_count since the last time the listener was called or the status was read.
- [DDS_Long current_count](#)
The current number of writers with which the [DDSDataReader](#) is matched.
- [DDS_Long current_count_change](#)
The change in current_count since the last time the listener was called or the status was read.
- [DDS_InstanceHandle_t last_publication_handle](#)
A handle to the last [DDSDataWriter](#) that caused the status to change.

7.58.1 Detailed Description

<<cert>> [DDS_SUBSCRIPTION_MATCHED_STATUS](#)

A "match" happens when the [DDSDataReader](#) finds a [DDSDataWriter](#) for the same [DDSTopic](#) with an offered QoS that is compatible with that requested by the [DDSDataReader](#).

This status is also changed (and the listener, if any, called) when a match is ended. A local [DDSDDataReader](#) will become "unmatched" from a remote [DDSDDataWriter](#) when that [DDSDDataWriter](#) goes away for any reason.

7.58.2 Member Data Documentation

7.58.2.1 DDS_Long DDS_SubscriptionMatchedStatus::total_count

The total cumulative number of times the concerned [DDSDDataReader](#) discovered a "match" with a [DDSDDataWriter](#). This number increases whenever a new match is discovered. It does not change when an existing match goes away.

7.58.2.2 DDS_Long DDS_SubscriptionMatchedStatus::total_count_change

The change in total_count since the last time the listener was called or the status was read.

7.58.2.3 DDS_Long DDS_SubscriptionMatchedStatus::current_count

The current number of writers with which the [DDSDDataReader](#) is matched. This number increases when a new match is discovered and decreases when an existing match goes away.

7.58.2.4 DDS_Long DDS_SubscriptionMatchedStatus::current_count_change

The change in current_count since the last time the listener was called or the status was read.

7.58.2.5 DDS_InstanceHandle_t DDS_SubscriptionMatchedStatus::last_publication_handle

A handle to the last [DDSDDataWriter](#) that caused the status to change.

7.59 DDS_SystemResourceLimitsQosPolicy Struct Reference

<<[cert](#)>> Resource limits that apply only to [DDSDomainParticipantFactory](#)

Public Attributes

- [DDS_Long max_participants](#)
The maximum number of local participants that can be created by a [DDSDomainParticipantFactory](#).
- [DDS_Long max_components](#)
The maximum number of components that may be registered on the [DDSDomainParticipantFactory](#).

7.59.1 Detailed Description

<<[cert](#)>> Resource limits that apply only to [DDSDomainParticipantFactory](#)

This QoS policy is an extension to the DDS standard.

Entity:

[DDSDomainParticipantFactory](#)

Properties:

RxO = N/A
[Changeable](#) = NO

7.59.2 Member Data Documentation

7.59.2.1 DDS_Long DDS_SystemResourceLimitsQosPolicy::max_participants

The maximum number of local participants that can be created by a [DDSDomainParticipantFactory](#).

This resource-limit limits the number of local DomainParticipants that can be created with the [DDSDomainParticipantFactory::create_participant\(\)](#) API. For the maximum number of participants that can be discovered, please refer to [DDS_DomainParticipantResourceLimitsQosPolicy::remote_participant_allocation](#).

[default] 1

[range] [1, 100] with default QoS settings. Additional [DDSDomainParticipant](#) objects can be allocated by changing the [DDS_WireProtocolQosPolicy::rtps_well_known_ports](#) QoS policy on all [DDSDomainParticipant](#)'s on a node.

7.59.2.2 DDS_Long DDS_SystemResourceLimitsQosPolicy::max_components

The maximum number of components that may be registered on the [DDSDomainParticipantFactory](#).

RTI Connex Micro consists of a number of plugins and components that perform various tasks. For example, the UDP transport and the DPDE discovery plugins are RTI Connex Micro components. This resource-limit limits the number of components that can be registered with RTI Connex Micro.

Note that changing this value is not supported.

[default] 64

7.60 DDS_Time_t Struct Reference

Type for *time* representation.

Public Attributes

- [DDS_Long](#) *sec*
seconds
- [DDS_UnsignedLong](#) *nanosec*
nanoseconds

7.60.1 Detailed Description

Type for *time* representation.

A [DDS_Time_t](#) represents a moment in time.

7.60.2 Member Data Documentation

7.60.2.1 DDS_Long DDS_Time_t::sec

seconds

7.60.2.2 DDS_UnsignedLong DDS_Time_t::nanosec

nanoseconds

7.61 DDS_TopicQos Struct Reference

QoS policies supported by a [DDSTopic](#) entity.

7.61.1 Detailed Description

QoS policies supported by a [DDSTopic](#) entity.

Getting and setting the topic QoS is not supported. [DDS_TOPIC_QOS_DEFAULT](#) is the only value supported as an argument to [DDSDomainParticipant::create_topic](#) .

Entity:

[DDSTopic](#)

7.62 DDS_TransportQosPolicy Struct Reference

[<<eXtension>>](#) [<<cert>>](#) Settings for available transports

Public Attributes

- struct DDS_StringSeq [enabled_transports](#)
The transports available for use.

7.62.1 Detailed Description

[<<eXtension>>](#) [<<cert>>](#) Settings for available transports

Entity:

[DDSDomainParticipant](#), [DDSDataWriter](#), [DDSDataReader](#)

Properties:

[RxO](#) = N/A
[Changeable](#) = NO

7.62.2 Member Data Documentation

7.62.2.1 struct DDS_StringSeq DDS_TransportQosPolicy::enabled_transports

The transports available for use.

A sequence of transport identifiers that defines the set of transports available for use by the associated DDS entity.

See also

[DDS_DomainParticipantQos](#), [DDS_DataWriterQos](#), [DDS_DataReaderQos](#)

[default] Empty sequence.

[range] Sequence of non-null,non-empty strings.

7.63 DDS_UserTrafficQosPolicy Struct Reference

A QosPolicy to configure properties of network traffic caused by the exchange of user data.

Public Attributes

- struct DDS_StringSeq [enabled_transports](#)
Configures the default transports enabled to transmit and received user data traffic.

7.63.1 Detailed Description

A QosPolicy to configure properties of network traffic caused by the exchange of user data.

7.63.2 Member Data Documentation

7.63.2.1 struct DDS_StringSeq DDS_UserTrafficQosPolicy::enabled_transports

Configures the default transports enabled to transmit and received user data traffic.

The memory for the strings in this sequence is managed according to the conventions described in [String Support](#). In particular, be careful to avoid a situation in which RTI Connex Micro allocates a string on your behalf and you then reuse that string in such a way that RTI Connex Micro believes it to have more memory allocated to it than it actually does.

[default] Empty sequence.

When an empty sequence is passed to [DDSDomainParticipantFactory::create_participant](#) in the [DDS_DomainParticipantQos](#), the domain participant will automatically add the following locators to listen for user-traffic on: "_-udp://" and "_intra://".

[range] Sequence of non-null, non-empty strings.

7.64 DDS_VendorId Struct Reference

<<[eXtension](#)>> <<[cert](#)>> Type used to represent the vendor of the service implementing the RTPS protocol.

Public Attributes

- [DDS_Octet vendorId](#) [[DDS_VENDOR_ID_LENGTH_MAX](#)]
The vendor Id.

7.64.1 Detailed Description

<<[eXtension](#)>> <<[cert](#)>> Type used to represent the vendor of the service implementing the RTPS protocol.

7.64.2 Member Data Documentation

7.64.2.1 DDS_Octet DDS_VendorId::vendorId[DDS_VENDOR_ID_LENGTH_MAX]

The vendor Id.

7.65 DDS_VendorId_t Struct Reference

<<[eXtension](#)>> <<[cert](#)>> Type used to represent the vendor of the service implementing the RTPS protocol.

Public Attributes

- [DDS_Octet vendorId](#) [[DDS_VENDOR_ID_LENGTH_MAX](#)]
The vendor Id.

7.65.1 Detailed Description

<<[eXtension](#)>> <<[cert](#)>> Type used to represent the vendor of the service implementing the RTPS protocol.

7.65.2 Member Data Documentation

7.65.2.1 DDS_Octet DDS_VendorId_t::vendorId[DDS_VENDOR_ID_LENGTH_MAX]

The vendor Id.

7.66 DDS_WireProtocolQosPolicy Struct Reference

<<cert>> Specifies the wire-protocol-related attributes for the [DDSDomainParticipant](#).

Public Attributes

- [DDS_Long participant_id](#)
A value used to distinguish between different participants belonging to the same domain on the same host.
- [DDS_UnsignedLong rtps_host_id](#)
The RTPS Host ID of the domain participant.
- [DDS_UnsignedLong rtps_app_id](#)
The RTPS App ID of the domain participant.
- [DDS_UnsignedLong rtps_instance_id](#)
The RTPS Instance ID of the domain participant.
- struct [DDS_RtpsWellKnownPorts_t rtps_well_known_ports](#)
Configures the RTPS well-known port mappings.
- [DDS_Boolean compute_crc](#)
Adds RTPS checksum submessage to every message when this field is set to [DDS_BOOLEAN_TRUE](#).
- [DDS_Boolean check_crc](#)
Checks if the received RTPS message is valid by comparing the computed checksum with the received RTPS checksum submessage when this field is set to [DDS_BOOLEAN_TRUE](#).
- [DDS_Boolean require_crc](#)
Require that discovered participants send one of the checksums enabled in [DDS_WireProtocolQosPolicy::allowed_crc_mask](#).
- [DDS_ChecksumKind_t computed_crc_kind](#)
A bitmask indicating which checksum is used for sent messages when [compute_crc](#) is [DDS_BOOLEAN_TRUE](#). Only one mask is supported.
- [DDS_ChecksumKindMask_t allowed_crc_mask](#)
The checksums which are enabled for validating incoming packets.

7.66.1 Detailed Description

<<cert>> Specifies the wire-protocol-related attributes for the [DDSDomainParticipant](#).

Entity:

[DDSDomainParticipant](#)

Properties:

[RxO](#) = N/A

[Changeable](#) = NO

7.66.2 Usage

The default Qos policies returned by RTI Connex Micro contain the correctly initialized wire protocol attributes.

The defaults are not expected to be modified normally, but are available to the advanced user customizing the implementation behavior.

The default values should not be modified without an understanding of the underlying Real-Time Publish Subscribe (RTPS) wire protocol.

7.66.3 Member Data Documentation

7.66.3.1 DDS_Long DDS_WireProtocolQosPolicy::participant_id

A value used to distinguish between different participants belonging to the same domain on the same host.

Determines the unicast port on which meta-traffic is received. Also defines the *default* unicast port for receiving user-traffic for DataReaders and DataWriters .

For more information on port mapping, please refer to [DDS_RtpsWellKnownPorts_t](#).

Each [DDSDomainParticipant](#) in the same domain, running on the same host, must have a unique `participant_id`. The participants may be in the same address space or in distinct address spaces.

A negative number (-1) means that RTI Connex Micro will *automatically* resolve the participant ID as follows.

- RTI Connex Micro will pick the *smallest* participant ID, based on the unicast ports available on the transports enabled for discovery.
- RTI Connex Micro will attempt to resolve an automatic port index either when a DomainParticipant is enabled, or when a DataReader or a DataWriter is created. Therefore, all the transports enabled for discovery must have been registered by this time. Otherwise, the discovery transports registered after resolving the automatic port index may produce port conflicts when the DomainParticipant is enabled.

[default] -1 [automatic], i.e. RTI Connex Micro will automatically pick the `participant_id`, as described above.

[range] [≥ 0], or -1, and does not violate guidelines stated in [DDS_RtpsWellKnownPorts_t](#).

See also

[DDSEntity::enable\(\)](#)

7.66.3.2 DDS_UnsignedLong DDS_WireProtocolQosPolicy::rtps_host_id

The RTPS Host ID of the domain participant.

Either all of [DDS_WireProtocolQosPolicy::rtps_host_id](#), [DDS_WireProtocolQosPolicy::rtps_app_id](#), and [DDS_WireProtocolQosPolicy::rtps_instance_id](#) for a DomainParticipant are set to [DDS_RTPS_AUTO_ID](#), or none of them are set to [DDS_RTPS_AUTO_ID](#).

[default] [DDS_RTPS_AUTO_ID](#).

[range] [0,0xffffffff]

7.66.3.3 DDS_UnsignedLong DDS_WireProtocolQosPolicy::rtps_app_id

The RTPS App ID of the domain participant.

Either all of [DDS_WireProtocolQosPolicy::rtps_host_id](#), [DDS_WireProtocolQosPolicy::rtps_app_id](#), and [DDS_WireProtocolQosPolicy::rtps_instance_id](#) for a DomainParticipant are set to [DDS_RTPS_AUTO_ID](#), or none of them are set to [DDS_RTPS_AUTO_ID](#).

[default] [DDS_RTPS_AUTO_ID](#).

[range] [0,0xffffffff]

7.66.3.4 **DDS_UnsignedLong** DDS_WireProtocolQosPolicy::rtps_instance_id

The RTPS Instance ID of the domain participant.

Either all of [DDS_WireProtocolQosPolicy::rtps_host_id](#), [DDS_WireProtocolQosPolicy::rtps_app_id](#), and [DDS_WireProtocolQosPolicy::rtps_instance_id](#) for a DomainParticipant are set to [DDS_RTPS_AUTO_ID](#), or none of them are set to [DDS_RTPS_AUTO_ID](#).

[default] [DDS_RTPS_AUTO_ID](#).

[range] [0,0xffffffff]

7.66.3.5 **struct DDS_RtpsWellKnownPorts_t** DDS_WireProtocolQosPolicy::rtps_well_known_ports

Configures the RTPS well-known port mappings.

Determines the well-known multicast and unicast port mappings for discovery (meta) traffic and user traffic.

[default] [DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS](#)

7.66.3.6 **DDS_Boolean** DDS_WireProtocolQosPolicy::compute_crc

Adds RTPS checksum submessage to every message when this field is set to [DDS_BOOLEAN_TRUE](#).

The computed checksum covers the entire RTPS message including the RTPS header itself.

[default] [DDS_BOOLEAN_FALSE](#)

7.66.3.7 **DDS_Boolean** DDS_WireProtocolQosPolicy::check_crc

Checks if the received RTPS message is valid by comparing the computed checksum with the received RTPS checksum submessage when this field is set to [DDS_BOOLEAN_TRUE](#).

[DDS_WireProtocolQosPolicy::compute_crc](#) must be enabled at the publishing application for validating the message at the subscribing application.

If a message is received with an invalid checksum, the message is discarded. If no checksum is received or found, the message is accepted but may still be corrupted.

[default] [DDS_BOOLEAN_FALSE](#)

7.66.3.8 **DDS_Boolean** DDS_WireProtocolQosPolicy::require_crc

Require that discovered participants send one of the checksums enabled in [DDS_WireProtocolQosPolicy::allowed_crc_mask](#).

If an invalid checksum is received, the message is dropped.

if the value is [DDS_BOOLEAN_TRUE](#) the participant requires that discovered participants send one of the checksums enabled in [DDS_WireProtocolQosPolicy::allowed_crc_mask](#). If this is not the case the participant is ignored and not matching will take place.

This is a stronger check than [idref_WireProtocolQosPolicy_check_crc](#). If an invalid checksum is received or not found, the message is dropped.

[default] [DDS_BOOLEAN_FALSE](#)

7.66.3.9 **DDS_ChecksumKind_t** DDS_WireProtocolQosPolicy::computed_crc_kind

A bitmask indicating which checksum is used for sent messages when [compute_crc](#) is [DDS_BOOLEAN_TRUE](#). Only one mask is supported.

- [DDS_CHECKSUM_AUTO](#)
- [DDS_CHECKSUM_BUILTIN32](#)
- [DDS_CHECKSUM_BUILTIN64](#)
- [DDS_CHECKSUM_BUILTIN128](#)

[default] [DDS_CHECKSUM_AUTO](#).

7.66.3.10 DDS_ChecksumKindMask_t DDS_WireProtocolQosPolicy::allowed_crc_mask

The checksums which are enabled for validating incoming packets.

The following bitmask can be OR'ed for all enabled checksums:

- [DDS_CHECKSUM_AUTO](#)
- [DDS_CHECKSUM_BUILTIN32](#)
- [DDS_CHECKSUM_BUILTIN64](#)
- [DDS_CHECKSUM_BUILTIN128](#)

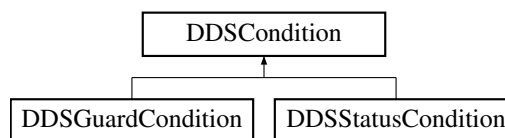
Note that the selected checksums must be a subset of what is supported. If a bitmask that is not supported is selected participant creation will fail.

[default] [DDS_CHECKSUM_AUTO](#).

7.67 DDSCondition Class Reference

<<interface>> <<cert>> Root class for all the conditions that may be attached to a [DDSWaitSet](#).

Inheritance diagram for DDSCondition:



Friends

- class [DDSWaitSet](#)

7.67.1 Detailed Description

<<interface>> <<cert>> Root class for all the conditions that may be attached to a [DDSWaitSet](#).

This basic class is specialised in two classes:

[DDSGuardCondition](#) and [DDSStatusCondition](#).

A [DDSCondition](#) has a `trigger_value` that can be [DDS_BOOLEAN_TRUE](#) or [DDS_BOOLEAN_FALSE](#) and is set automatically by RTI Connex Micro.

See also

[DDSWaitSet](#)

7.68 DDSConditionSeq Class Reference

A sequence object containing references to one or more [DDSCondition](#) objects.

Friends

- class [DDSWaitSet](#)

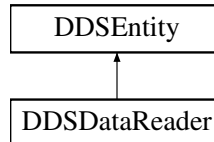
7.68.1 Detailed Description

A sequence object containing references to one or more [DDSSubscriber](#) objects.

7.69 DDSDataReader Class Reference

<<interface>> **<<cert>>** Allows the application to: (1) declare the data it wishes to receive (i.e. make a subscription) and (2) access the data received by the attached [DDSSubscriber](#).

Inheritance diagram for DDSDataReader:



Public Member Functions

- virtual [DDSEntity](#) * [as_entity](#) ()=0
<<cert>> Accesses the [DDSDataReader](#)'s supertype instance.
- virtual [DDSTopicDescription](#) * [get_topicdescription](#) ()=0
<<cert>> Returns the [DDSTopicDescription](#) associated with the [DDSDataReader](#).
- virtual [DDSSubscriber](#) * [get_subscriber](#) ()=0
<<cert>> Returns the [DDSSubscriber](#) to which the [DDSDataReader](#) belongs.
- virtual [DDS_ReturnCode_t](#) [set_qos](#) (const [DDS_DataReaderQos](#) &qos)=0
 Sets the reader QoS.
- virtual [DDS_ReturnCode_t](#) [get_qos](#) ([DDS_DataReaderQos](#) &qos)=0
 Gets the reader QoS.
- virtual [DDS_ReturnCode_t](#) [set_listener](#) (const [DDSDataReaderListener](#) *listener, [DDS_StatusMask](#) mask)=0
 Sets the reader listener.
- virtual [DDSDataReaderListener](#) * [get_listener](#) ()=0
 Get the reader listener.
- virtual [DDS_ReturnCode_t](#) [read_untyped](#) ([DDS_UntypedSampleSeq](#) *received_data, [DDS_SampleInfoSeq](#) &info_seq, [DDS_Long](#) max_samples, [DDS_SampleStateMask](#) sample_states, [DDS_ViewStateMask](#) view_states, [DDS_InstanceStateMask](#) instance_states)=0
<<cert>> Access a collection of data samples from the [DDSDataReader](#).
- virtual [DDS_ReturnCode_t](#) [take_untyped](#) ([DDS_UntypedSampleSeq](#) *received_data, [DDS_SampleInfoSeq](#) &info_seq, [DDS_Long](#) max_samples, [DDS_SampleStateMask](#) sample_states, [DDS_ViewStateMask](#) view_states, [DDS_InstanceStateMask](#) instance_states)=0
<<cert>> Access a collection of data-samples from the [DDSDataReader](#).
- virtual [DDS_ReturnCode_t](#) [read_instance_untyped](#) ([DDS_UntypedSampleSeq](#) *received_data, [DDS_SampleInfoSeq](#) &info_seq, [DDS_Long](#) max_samples, const [DDS_InstanceHandle_t](#) &a_handle, [DDS_SampleStateMask](#) sample_states, [DDS_ViewStateMask](#) view_states, [DDS_InstanceStateMask](#) instance_states)=0
<<cert>> Access a collection of data samples from the [DDSDataReader](#), all of which belong to the same instance.
- virtual [DDS_ReturnCode_t](#) [take_instance_untyped](#) ([DDS_UntypedSampleSeq](#) *received_data, [DDS_SampleInfoSeq](#) &info_seq, [DDS_Long](#) max_samples, const [DDS_InstanceHandle_t](#) &a_handle, [DDS_SampleStateMask](#) sample_states, [DDS_ViewStateMask](#) view_states, [DDS_InstanceStateMask](#) instance_states)=0
<<cert>> Access a collection of data samples from the [DDSDataReader](#), all of which belong to the same instance.

- virtual [DDS_ReturnCode_t read_next_sample_untyped](#) (void *received_data, [DDS_SampleInfo](#) &sample_info)=0
<<cert>> Copies the next not-previously-accessed data value from the [DDSDataReader](#).
- virtual [DDS_ReturnCode_t take_next_sample_untyped](#) (void *received_data, [DDS_SampleInfo](#) &sample_info)=0
<<cert>> Copies the next not-previously-accessed data value from the [DDSDataReader](#).
- virtual [DDS_InstanceHandle_t lookup_instance_untyped](#) (const void *key_holder)=0
<<cert>> Retrieve the [DDS_InstanceHandle_t](#) identifying a specific instance.
- virtual [DDS_ReturnCode_t return_loan_untyped](#) ([DDS_UntypedSampleSeq](#) *received_data, [DDS_SampleInfoSeq](#) &info_seq)=0
*<<cert>> Indicates to the [DDSDataReader](#) that the application is done accessing the collection of *received_data* and *info_seq* obtained by some earlier invocation of *read* or *take* on the [DDSDataReader](#).*
- virtual [DDS_ReturnCode_t get_subscription_matched_status](#) ([DDS_SubscriptionMatchedStatus](#) &status)=0
<<cert>> Accesses the [DDS_SUBSCRIPTION_MATCHED_STATUS](#) communication status.
- virtual [DDS_ReturnCode_t get_liveliness_changed_status](#) ([DDS_LivelinessChangedStatus](#) &status)=0
<<cert>> Accesses the [DDS_LIVELINESS_CHANGED_STATUS](#) communication status.
- virtual [DDS_ReturnCode_t get_sample_rejected_status](#) ([DDS_SampleRejectedStatus](#) &status)=0
<<cert>> Accesses the [DDS_SAMPLE_REJECTED_STATUS](#) communication status.
- virtual [DDS_ReturnCode_t get_sample_lost_status](#) ([DDS_SampleLostStatus](#) &status)=0
<<cert>> Accesses the [DDS_SAMPLE_LOST_STATUS](#) communication status.
- virtual [DDS_ReturnCode_t get_requested_deadline_missed_status](#) ([DDS_RequestedDeadlineMissedStatus](#) &status)=0
<<cert>> Accesses the [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#) communication status.
- virtual [DDS_ReturnCode_t get_requested_incompatible_qos_status](#) ([DDS_RequestedIncompatibleQosStatus](#) &status)=0
<<cert>> Accesses the [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#) communication status.
- virtual [DDS_ReturnCode_t get_instance_replaced_missed_status](#) ([DDS_DataReaderInstanceReplacedStatus](#) &status)=0
<<cert>> Accesses the [DDS_INSTANCE_REPLACED_STATUS](#) communication status.

7.69.1 Detailed Description

<<interface>> <<cert>> Allows the application to: (1) declare the data it wishes to receive (i.e. make a subscription) and (2) access the data received by the attached [DDSSubscriber](#).

QoS:

[DDS_DataReaderQos](#)

Status:

[DDS_DATA_AVAILABLE_STATUS](#);
[DDS_LIVELINESS_CHANGED_STATUS](#), [DDS_LivelinessChangedStatus](#);
[DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#), [DDS_RequestedIncompatibleQosStatus](#);
[DDS_SUBSCRIPTION_MATCHED_STATUS](#), [DDS_SubscriptionMatchedStatus](#);
[DDS_INSTANCE_REPLACED_STATUS](#), [DDS_DataReaderInstanceReplacedStatus](#);

Listener:

[DDSDDataReaderListener](#)

A [DDSDDataReader](#) refers to exactly one [DDSTopicDescription](#) (a [DDSTopic](#)) that identifies the data to be read.

The subscription has a unique resulting type. The data-reader may give access to several instances of the resulting type, which can be distinguished from each other by their *key*.

The following operations may be called even if the [DDSDDataReader](#) is not enabled. Other operations will fail with the value [DDS_RETCODE_NOT_ENABLED](#) if called on a disabled [DDSDDataReader](#):

- The base-class operations [DDSEntity::enable](#) , [DDSDDataReader::get_qos](#) , [DDSDDataReader::get_listener](#) , [DDSDDataReader::set_qos](#) , and [DDSDDataReader::set_listener](#)

7.69.2 Member Function Documentation

7.69.2.1 `virtual DDSEntity* DDSDDataReader::as_entity ( ) [pure virtual]`

`<<cert>>` Accesses the [DDSDDataReader](#)'s supertype instance.

7.69.2.2 `virtual DDSTopicDescription* DDSDDataReader::get_topicdescription ( ) [pure virtual]`

`<<cert>>` Returns the [DDSTopicDescription](#) associated with the [DDSDDataReader](#).

Returns that same [DDSTopicDescription](#) that was used to create the [DDSDDataReader](#).

Returns

[DDSTopicDescription](#) associated with the [DDSDDataReader](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.69.2.3 `virtual DDSSubscriber* DDSDDataReader::get_subscriber ( ) [pure virtual]`

`<<cert>>` Returns the [DDSSubscriber](#) to which the [DDSDDataReader](#) belongs.

Returns

[DDSSubscriber](#) to which the [DDSDDataReader](#) belongs.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.69.2.4 `virtual DDS_ReturnCode_t DDSDDataReader::set_qos ( const DDS_DataReaderQos & qos ) [pure virtual]`

Sets the reader QoS.

This operation modifies the QoS of the [DDSDDataReader](#).

The [DDS_DataReaderQos::deadline](#)

can be changed. The other policies are immutable.

Parameters

<i>qos</i>	<code><<in>></code> The DDS_DataReaderQos to be set to. Policies must be consistent. Policies cannot be changed after DDSDDataReader is enabled. The special value DDS_DATAREADER_QOS_DEFAULT can be used to indicate that the QoS of the DDSDDataReader should be changed to match the current default DDS_DataReaderQos set in the DDSSubscriber .
------------	--

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_IMMUTABLE_POLICY](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#).

See also

[DDS_DataReaderQos](#) for rules on consistency among QoS
[set_qos](#) (abstract)
[DDSDataReader::set_qos](#)

7.69.2.5 `virtual DDS_ReturnCode_t DDSDataReader::get_qos ( DDS_DataReaderQos & qos ) [pure virtual]`

Gets the reader QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<i>qos</i>	<< <i>inout</i> >> The DDS_DataReaderQos to be filled up.
------------	---

Returns

One of the [Standard Return Codes](#)

See also

[get_qos](#) (abstract)

7.69.2.6 `virtual DDS_ReturnCode_t DDSDataReader::set_listener ( const DDSDataReaderListener * listener, DDS_StatusMask mask ) [pure virtual]`

Sets the reader listener.

Parameters

<i>listener</i>	<< <i>in</i> >> DDSDataReaderListener to set to
<i>mask</i>	<< <i>in</i> >> DDS_StatusMask associated with the DDSDataReaderListener .

Returns

One of the [Standard Return Codes](#)

See also

[set_listener](#) (abstract)

7.69.2.7 `virtual DDSDataReaderListener* DDSDataReader::get_listener ( ) [pure virtual]`

Get the reader listener.

Returns

[DDSDataReaderListener](#) of the [DDSDataReader](#).

7.69.2.8 `virtual DDS_ReturnCode_t DDSDataReader::read_untyped ( DDS_UntypedSampleSeq * received_data, DDS_SampleInfoSeq & info_seq, DDS_Long max_samples, DDS_SampleStateMask sample_states, DDS_ViewStateMask view_states, DDS_InstanceStateMask instance_states ) [pure virtual]`

<<*cert*>> Access a collection of data samples from the [DDSDataReader](#).

This operation offers the same functionality and API as [DDSDataReader::take_untyped](#) except that the samples returned remain in the [DDSDataReader](#) such that they can be retrieved again by means of a read or take operation.

Please refer to the documentation of [DDSDataReader::take_untyped\(\)](#) for details on the number of samples returned within the `received_data` and `info_seq` as well as the order in which the samples appear in these sequences.

The act of reading a sample changes its `sample_state` to [DDS_READ_SAMPLE_STATE](#). If the sample belongs to the most recent generation of the instance, it will also set the `view_state` of the instance to be [DDS_NOT_NEW_VIEW_STATE](#). It will not affect the `instance_state` of the instance.

Important: If the samples "returned" by this method are loaned from RTI Connex Micro (see [DDSDataReader::take_untyped](#) for more information on memory loaning), it is important that their contents not be changed. Because the memory in which the data is stored belongs to the middleware, any modifications made to the data will be seen the next time the same samples are read or taken; the samples will no longer reflect the state that was received from the network.

Parameters

<i>received_data</i>	<< <i>inout</i> >> User data untyped pointer where the received data samples will be returned. Must be a valid non-NULL pointer. The method will fail with DDS_RETCODE_BAD_PARAMETER if it is NULL.
<i>info_seq</i>	<< <i>inout</i> >> A DDS_SampleInfoSeq object where the received sample info will be returned.
<i>max_samples</i>	<< <i>in</i> >> The maximum number of samples to be returned. If the special value DDS_LENGTH_UNLIMITED is provided, as many samples will be returned as are available.
<i>sample_states</i>	<< <i>in</i> >> Data samples matching one of these <code>sample_states</code> are returned.
<i>view_states</i>	<< <i>in</i> >> Data samples matching one of these <code>view_state</code> are returned.
<i>instance_states</i>	<< <i>in</i> >> Data samples matching ones of these <code>instance_state</code> are returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

See also

[DDSDataReader::take_untyped](#),
[DDS_LENGTH_UNLIMITED](#)

```
7.69.2.9 virtual DDS_ReturnCode_t DDSDataReader::take_untyped ( DDS_UntypedSampleSeq * received_data,
DDS_SampleInfoSeq & info_seq, DDS_Long max_samples, DDS_SampleStateMask sample_states,
DDS_ViewStateMask view_states, DDS_InstanceStateMask instance_states ) [pure virtual]
```

<<*cert*>> Access a collection of data-samples from the [DDSDataReader](#).

The operation will return the list of samples received by the [DDSDataReader](#) since the last [DDSDataReader::take_untyped](#) operation that match the specified [DDS_SampleStateMask](#), [DDS_ViewStateMask](#) and [DDS_InstanceStateMask](#).

This operation may fail with [DDS_RETCODE_ERROR](#) if [DDS_DataReaderResourceLimitsQosPolicy::max_outstanding_reads](#) limit has been exceeded.

The actual number of samples returned depends on the information that has been received by the middleware as well as the [DDS_HistoryQosPolicy](#), [DDS_ResourceLimitsQosPolicy](#), [DDS_DataReaderResourceLimitsQosPolicy](#) and the characteristics of the data-type that is associated with the [DDSDataReader](#):

- In the case where the [DDS_HistoryQosPolicy::kind](#) is [DDS_KEEP_LAST_HISTORY_QOS](#), the call will return at most [DDS_HistoryQosPolicy::depth](#) samples per instance.
- The maximum number of samples returned is limited by [DDS_ResourceLimitsQosPolicy::max_samples](#).
- For multiple instances, the number of samples returned is additionally limited by the product ([DDS_ResourceLimitsQosPolicy::max_samples_per_instance](#) * [DDS_ResourceLimitsQosPolicy::max_instances](#))

If the read or take succeeds and the number of samples returned has been limited (by means of a maximum limit, as listed above, or insufficient [DDS_SampleInfo](#) resources), the call will complete successfully and provide those samples the reader is able to return. The user may need to make additional calls, or return outstanding loaned buffers in the case of insufficient resources, in order to access remaining samples.

Note that in the case where the [DDSTopic](#) associated with the [DDSDataReader](#) is bound to a data-type that has no key definition, then there will be at most one instance in the [DDSDataReader](#). So the per-sample limits will apply.

The act of *taking* a sample removes it from RTI Connex Micro so it cannot be read or taken again. If the sample belongs to the most recent generation of the instance, it will also set the `view_state` of the sample's instance to [DDS_NOT_NEW_VIEW_STATE](#). It will not affect the `instance_state` of the sample's instance.

After [DDSDataReader::take_untyped](#) completes, `received_data` and `info_seq` will be of the same length and contain the received data.

If the sequences are empty (maximum size equal 0) when the [DDSDataReader::take_untyped](#) is called, the samples returned in the `received_data` and the corresponding `info_seq` are 'loaned' to the application from buffers provided by the [DDSDataReader](#). The application can use them as desired and has guaranteed exclusive access to them.

Once the application completes its use of the samples it must 'return the loan' to the [DDSDataReader](#) by calling the [DDSDataReader::return_loan_untyped](#) operation.

Note: While you must call [DDSDataReader::return_loan_untyped](#) at some point, you do *not* have to do so before the next [DDSDataReader::take_untyped](#) call. However, failure to return the loan will eventually deplete the [DDSDataReader](#) of the buffers it needs to receive new samples and eventually samples will start to be lost. The total number of buffers available to the [DDSDataReader](#) is specified by the [DDS_ResourceLimitsQosPolicy](#) and the [DDS_DataReaderResourceLimitsQosPolicy](#).

If the sequences are not empty (maximum size not equal 0) when the [DDSDataReader::take_untyped](#) is called, samples are copied to `received_data` and `info_seq`. The application will not need to call [DDSDataReader::return_loan](#).

If the [DDSDataReader](#) has no samples that meet the constraints, the method will fail with [DDS_RETCODE_NO_DATA](#).

In addition to the collection of samples, the read and take operations also use a collection of [DDS_SampleInfo](#) structures.

7.69.3 SEQUENCES USAGE IN TAKE AND READ

The initial (input) properties of the `received_data` and `info_seq` collections will determine the precise behavior of the read or take operation. For the purposes of this description, the collections are modeled as having these properties:

- whether the collection container owns the memory of the elements within (`owns`, see [FooSeq::has_ownership](#))
- the current-length (`len`, see [FooSeq::length\(\)](#))
- the maximum length (`max_len`, see [FooSeq::maximum\(\)](#))

The initial values of the `owns`, `len` and `max_len` properties for the `received_data` and `info_seq` collections govern the behavior of the read and take operations as specified by the following rules:

1. The values of `owns`, `len` and `max_len` properties for the two collections must be identical. Otherwise read/take will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).
2. On successful output, the values of `owns`, `len` and `max_len` will be the same for both collections.
3. If the initial `max_len==0`, then the `received_data` and `info_seq` collections will be filled with elements that are loaned by the [DDSDataReader](#). On output, `owns` will be `FALSE`, `len` will be set to the number of values returned, and `max_len` will be set to a value verifying `max_len >= len`. The use of

this variant allows for zero-copy access to the data and the application will need to return the loan to the `DDSDataWriter` using `FooDataReader::return_loan`.

4. If the initial `max_len > 0` and `owns == FALSE`, then the read or take operation will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`. This avoids the potential hard-to-detect memory leaks caused by an application forgetting to return the loan.
5. If initial `max_len > 0` and `owns == TRUE`, then the read or take operation will copy the `received_data` values and `DDS_SampleInfo` values into the elements already inside the collections. On output, `owns` will be `TRUE`, `len` will be set to the number of values copied and `max_len` will remain unchanged. The use of this variant forces a copy but the application can control where the copy is placed and the application will not need to return the loan. The number of samples copied depends on the relative values of `max_len` and `max_samples`:
 - If `max_samples == LENGTH_UNLIMITED`, then at most `max_len` values will be copied. The use of this variant lets the application limit the number of samples returned to what the sequence can accommodate.
 - If `max_samples <= max_len`, then at most `max_samples` values will be copied. The use of this variant lets the application limit the number of samples returned to fewer than what the sequence can accommodate.
 - If `max_samples > max_len`, then the read or take operation will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`. This avoids the potential confusion where the application expects to be able to access up to `max_samples`, but that number can never be returned, even if they are available in the `DDSDataReader`, because the output sequence cannot accommodate them.

As described above, upon completion, the `received_data` and `info_seq` collections may contain elements loaned from the `DDSDataReader`. If this is the case, the application will need to use `DDSDataReader::return_loan_untyped` to return the loan once it is no longer using the `received_data` in the collection. When `DDSDataReader::return_loan_untyped` completes, the collection will have `owns = FALSE` and `max_len = 0`. The application can determine whether it is necessary to return the loan or not based on how the state of the collections when the read/take operation was called or by accessing the `owns` property. However, in many cases it may be simpler to always call `DDSDataReader::return_loan_untyped`, as this operation is harmless (i.e., it leaves all elements unchanged) if the collection does not have a loan.

To avoid potential memory leaks, the implementation of the `Foo` and `DDS_SampleInfo` collections should disallow changing the length of a collection for which `owns == FALSE`. Furthermore, deleting a collection for which `owns == FALSE` should be considered an error.

On output, the collection of `Foo` values and the collection of `DDS_SampleInfo` structures are of the same length and are in a one-to-one correspondence. Each `DDS_SampleInfo` provides information, such as the `source_timestamp`, the `sample_state`, `view_state`, and `instance_state`, etc., about the corresponding sample.

Some elements in the returned collection may not have valid data. If the `instance_state` in the `DDS_SampleInfo` is `DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE` or `DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE`, then the last sample for that instance in the collection (that is, the one whose `DDS_SampleInfo` has `sample_rank == 0`) does not contain valid data.

Samples that contain no data do not count towards the limits imposed by the `DDS_ResourceLimitsQosPolicy`. The act of reading/taking a sample sets its `sample_state` to `DDS_READ_SAMPLE_STATE`.

If the sample belongs to the most recent generation of the instance, it will also set the `view_state` of the instance to `DDS_NOT_NEW_VIEW_STATE`. It will not affect the `instance_state` of the instance.

This operation must be provided on the specialized class that is generated for the particular application data-type that is being read. If the `DDSDataReader` has no samples that meet the constraints, the operation fails with `DDS_RETCODE_NO_DATA`.

Parameters

<code>received_data</code>	<<inout>> User data untyped pointer where the received data samples will be returned.
----------------------------	---

Parameters

<i>info_seq</i>	<< <i>inout</i> >> A DDS_SampleInfoSeq object where the received sample info will be returned.
<i>max_samples</i>	<< <i>in</i> >> The maximum number of samples to be returned. If the special value DDS_LENGTH_UNLIMITED is provided, as many samples will be returned as are available.
<i>sample_states</i>	<< <i>in</i> >> Data samples matching one of these <i>sample_states</i> are returned.
<i>view_states</i>	<< <i>in</i> >> Data samples matching one of these <i>view_state</i> are returned.
<i>instance_states</i>	<< <i>in</i> >> Data samples matching one of these <i>instance_state</i> are returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

See also

[DDSDataReader::read_untyped](#)

7.69.3.1 `virtual DDS_ReturnCode_t DDSDataReader::read_instance_untyped (DDS_UntypedSampleSeq * received_data, DDS_SampleInfoSeq & info_seq, DDS_Long max_samples, const DDS_InstanceHandle_t & a_handle, DDS_SampleStateMask sample_states, DDS_ViewStateMask view_states, DDS_InstanceStateMask instance_states) [pure virtual]`

<<*cert*>> Access a collection of data samples from the [DDSDataReader](#), all of which belong to the same instance .

Parameters

<i>received_data</i>	<< <i>inout</i> >> User data untyped pointer where the received data samples will be returned. Must be a valid non-NULL pointer. The method will fail with DDS_RETCODE_BAD_PARAMETER if it is NULL.
<i>info_seq</i>	<< <i>inout</i> >> A DDS_SampleInfoSeq object where the received sample info will be returned.
<i>max_samples</i>	<< <i>in</i> >> The maximum number of samples to be returned. If the special value DDS_LENGTH_UNLIMITED is provided, as many samples will be returned as are available.
<i>a_handle</i>	<< <i>in</i> >> Instance for which data samples should be read
<i>sample_states</i>	<< <i>in</i> >> Data samples matching one of these <i>sample_states</i> are returned.
<i>view_states</i>	<< <i>in</i> >> Data samples matching one of these <i>view_state</i> are returned.
<i>instance_states</i>	<< <i>in</i> >> Data samples matching ones of these <i>instance_state</i> are returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

See also

[DDSDataReader::read_untyped](#),
[DDS_LENGTH_UNLIMITED](#)

7.69.3.2 `virtual DDS_ReturnCode_t DDSDataReader::take_instance_untyped (DDS_UntypedSampleSeq * received_data, DDS_SampleInfoSeq & info_seq, DDS_Long max_samples, const DDS_InstanceHandle_t & a_handle, DDS_SampleStateMask sample_states, DDS_ViewStateMask view_states, DDS_InstanceStateMask instance_states) [pure virtual]`

<<*cert*>> Access a collection of data samples from the [DDSDataReader](#), all of which belong to the same instance .

Parameters

<i>received_data</i>	<< <i>inout</i> >> User data untyped pointer where the received data samples will be returned. Must be a valid non-NULL pointer. The method will fail with DDS_RETCODE_BAD_PARAMETER if it is NULL.
<i>info_seq</i>	<< <i>inout</i> >> A DDS_SampleInfoSeq object where the received sample info will be returned.
<i>max_samples</i>	<< <i>in</i> >> The maximum number of samples to be returned. If the special value DDS_LENGTH_UNLIMITED is provided, as many samples will be returned as are available.
<i>a_handle</i>	<< <i>in</i> >> Instance for which data samples should be read
<i>sample_states</i>	<< <i>in</i> >> Data samples matching one of these <i>sample_states</i> are returned.
<i>view_states</i>	<< <i>in</i> >> Data samples matching one of these <i>view_state</i> are returned.
<i>instance_states</i>	<< <i>in</i> >> Data samples matching ones of these <i>instance_state</i> are returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

See also

[DDSDataReader::take_untyped](#),
[DDS_LENGTH_UNLIMITED](#)

7.69.3.3 **virtual DDS_ReturnCode_t DDSDataReader::read_next_sample_untyped (void * *received_data*,
 DDS_SampleInfo & *sample_info*)** [pure virtual]

<<*cert*>> Copies the next not-previously-accessed data value from the [DDSDataReader](#).

This operation copies the next not-previously-accessed data value from the [DDSDataReader](#). This operation also copies the corresponding [DDS_SampleInfo](#). The implied order among the samples stored in the [DDSDataReader](#) is the same as for the [DDSDataReader::read_untyped](#) operation.

The [DDSDataReader::read_next_sample_untyped](#) operation is semantically equivalent to the [DDSDataReader::read_untyped](#) operation, where the input data sequences has *max_len*=1, the *sample_states*=NOT_READ, the *view_states*=ANY_VIEW_STATE, and the *instance_states*=ANY_INSTANCE_STATE.

The [DDSDataReader::read_next_sample_untyped](#) operation provides a simplified API to 'read' samples, avoiding the need for the application to manage sequences and specify states.

If there is no unread data in the [DDSDataReader](#), the operation will fail with [DDS_RETCODE_NO_DATA](#) and nothing is copied.

Parameters

<i>received_data</i>	<< <i>inout</i> >> user data type-specific data object where the next received data sample will be returned. The <i>received_data</i> must have been fully allocated. Otherwise, this operation may fail.
<i>sample_info</i>	<< <i>inout</i> >> a DDS_SampleInfo object where the next received sample info will be returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

See also

[DDSDataReader::read_untyped](#)

7.69.3.4 **virtual DDS_ReturnCode_t DDSDataReader::take_next_sample_untyped (void * *received_data*,
 DDS_SampleInfo & *sample_info*)** [pure virtual]

<<*cert*>> Copies the next not-previously-accessed data value from the [DDSDataReader](#).

This operation copies the next not-previously-accessed data value from the [DDSDataReader](#) and 'removes' it from the [DDSDataReader](#) so that it is no longer accessible. This operation also copies the corresponding [DDS_SampleInfo](#). This operation is analogous to the [DDSDataReader::read_next_sample_untyped](#) except for the fact that the sample is removed from the [DDSDataReader](#).

The [DDSDataReader::take_next_sample_untyped](#) operation is semantically equivalent to the [DDSDataReader::take_untyped](#) operation, where the input data sequences has `max_len=1`, the `sample_states=NOT_READ`, the `view_states=ANY_VIEW_STATE`, and the `instance_states=ANY_INSTANCE_STATE`.

The [DDSDataReader::take_next_sample_untyped](#) operation provides a simplified API to 'take' samples, avoiding the need for the application to manage sequences and specify states.

If there is no unread data in the [DDSDataReader](#), the operation will fail with [DDS_RETCODE_NO_DATA](#) and nothing is copied.

Parameters

<i>received_data</i>	<< <i>inout</i> >> user data type-specific Data object where the next received data sample will be returned. The <code>received_data</code> must have been fully allocated. Otherwise, this operation may fail.
<i>sample_info</i>	<< <i>inout</i> >> a DDS_SampleInfo object where the next received sample info will be returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

See also

[DDSDataReader::take_untyped](#)

7.69.3.5 `virtual DDS_InstanceHandle_t DDSDataReader::lookup_instance_untyped ( const void * key_holder )` `[pure virtual]`

<<*cert*>> Retrieve the [DDS_InstanceHandle_t](#) identifying a specific instance.

Parameters

<i>key_holder</i>	<< <i>in</i> >> A data sample containing the desired values for key attributes.
-------------------	---

Returns

The [DDS_InstanceHandle_t](#) for the selected instance or [DDS_HANDLE_NIL](#) if the instance was not found or an error occurred.

7.69.3.6 `virtual DDS_ReturnCode_t DDSDataReader::return_loan_untyped ( DDS_UntypedSampleSeq * received_data, DDS_SampleInfoSeq & info_seq )` `[pure virtual]`

<<*cert*>> Indicates to the [DDSDataReader](#) that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of `read` or `take` on the [DDSDataReader](#).

This operation indicates to the [DDSDataReader](#) that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of `read` or `take` on the [DDSDataReader](#).

The `received_data` and `info_seq` must belong to a single related "pair"; that is, they should correspond to a pair returned from a single call to `read` or `take`. The `received_data` and `info_seq` must also have been obtained from the same [DDSDataReader](#) to which they are returned. If either of these conditions is not met, the operation will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

The operation [DDSDataReader::return_loan_untyped](#) allows implementations of the `read` and `take` operations to "loan" buffers from the [DDSDataReader](#) to the application and in this manner provide "zerocopy" access to the data. During the loan, the [DDSDataReader](#) will guarantee that the data and sample-information are not modified.

It is not necessary for an application to return the loans immediately after the read or take calls. However, as these buffers correspond to internal resources inside the [DDSDDataReader](#), the application should not retain them indefinitely.

The use of [DDSDDataReader::return_loan_untyped](#) is only necessary if the read or take calls "loaned" buffers to the application. This only occurs if the `received_data` and `info_seq` collections had `max_len=0` at the time read or take was called.

The application may also examine the "owns" property of the collection to determine where there is an outstanding loan. However, calling [DDSDDataReader::return_loan_untyped](#) on a collection that does not have a loan is safe and has no side effects.

If the collections had a loan, upon completion of [DDSDDataReader::return_loan_untyped](#), the collections will have `max_len=0`.

Parameters

<i>received_data</i>	<<in>> user data type-specific FooSeq object where the received data samples was obtained from earlier invocation of read or take on the DDSDDataReader .
----------------------	---

Parameters

<i>info_seq</i>	<<in>> a DDS_SampleInfoSeq object where the received sample info was obtained from earlier invocation of read or take on the DDSDDataReader .
-----------------	---

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#) or [DDS_RETCODE_NOT_ENABLED](#).

7.69.3.7 `virtual DDS_ReturnCode_t DDSDataReader::get_subscription_matched_status ( DDS_SubscriptionMatchedStatus & status ) [pure virtual]`

<<cert>> Accesses the [DDS_SUBSCRIPTION_MATCHED_STATUS](#) communication status.

Parameters

<i>status</i>	<<inout>> DDS_SubscriptionMatchedStatus to be filled in.
---------------	--

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.69.3.8 `virtual DDS_ReturnCode_t DDSDataReader::get_liveliness_changed_status ( DDS_LivelinessChangedStatus & status ) [pure virtual]`

<<cert>> Accesses the [DDS_LIVELINESS_CHANGED_STATUS](#) communication status.

Parameters

<i>status</i>	<<inout>> DDS_LivelinessChangedStatus to be filled in.
---------------	--

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.69.3.9 `virtual DDS_ReturnCode_t DDSDataReader::get_sample_rejected_status ( DDS_SampleRejectedStatus & status ) [pure virtual]`

<<cert>> Accesses the [DDS_SAMPLE_REJECTED_STATUS](#) communication status.

Parameters

<i>status</i>	<<inout>> DDS_SampleRejectedStatus to be filled in.
---------------	---

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.69.3.10 `virtual DDS_ReturnCode_t DDSDataReader::get_sample_lost_status ( DDS_SampleLostStatus & status ) [pure virtual]`

<<cert>> Accesses the [DDS_SAMPLE_LOST_STATUS](#) communication status.

Parameters

<i>status</i>	<<inout>> DDS_SampleLostStatus to be filled in.
---------------	---

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.69.3.11 `virtual DDS_ReturnCode_t DDSDataReader::get_requested_deadline_missed_status ( DDS_RequestedDeadlineMissedStatus & status ) [pure virtual]`

<<cert>> Accesses the [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#) communication status.

Parameters

<i>status</i>	<<inout>> DDS_RequestedDeadlineMissedStatus to be filled in.
---------------	--

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.69.3.12 `virtual DDS_ReturnCode_t DDSDataReader::get_requested_incompatible_qos_status ( DDS_RequestedIncompatibleQosStatus & status ) [pure virtual]`

`<<cert>>` Accesses the [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#) communication status.

Parameters

<i>status</i>	<code><<inout>></code> DDS_RequestedIncompatibleQosStatus to be filled in.
---------------	--

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.69.3.13 `virtual DDS_ReturnCode_t DDSDataReader::get_instance_replaced_missed_status ( DDS_DataReaderInstanceReplacedStatus & status ) [pure virtual]`

`<<cert>>` Accesses the [DDS_INSTANCE_REPLACED_STATUS](#) communication status.

Parameters

<i>status</i>	<code><<inout>></code> DDS_DataReaderInstanceReplacedStatus to be filled in.
---------------	--

Returns

One of the [Standard Return Codes](#)

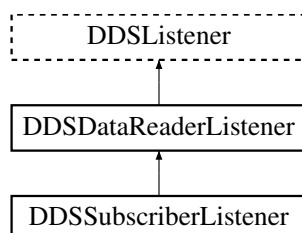
MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.70 DDSDataReaderListener Class Reference

`<<interface>>` `<<cert>>` [DDSListener](#) for reader status.

Inheritance diagram for DDSDataReaderListener:



Public Member Functions

- virtual void `on_data_available` (DDSDataReader *)
<<cert>> Handle the `DDS_DATA_AVAILABLE_STATUS` communication status.
- virtual void `on_requested_deadline_missed` (DDSDataReader *, const DDS_RequestedDeadlineMissedStatus &)
<<cert>> Handles the `DDS_REQUESTED_DEADLINE_MISSED_STATUS` communication status.
- virtual void `on_liveliness_changed` (DDSDataReader *, const DDS_LivelinessChangedStatus &)
<<cert>> Handles the `DDS_LIVELINESS_CHANGED_STATUS` communication status.
- virtual void `on_requested_incompatible_qos` (DDSDataReader *, const DDS_RequestedIncompatibleQosStatus &)
<<cert>> Handles the `DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS` communication status.
- virtual void `on_sample_rejected` (DDSDataReader *, const DDS_SampleRejectedStatus &)
<<cert>> Handles the `DDS_SAMPLE_REJECTED_STATUS` communication status.
- virtual void `on_subscription_matched` (DDSDataReader *, const DDS_SubscriptionMatchedException &)
<<cert>> Handles the `DDS_SUBSCRIPTION_MATCHED_STATUS` communication status.
- virtual void `on_sample_lost` (DDSDataReader *, const DDS_SampleLostStatus &)
<<cert>> Handles the `DDS_SAMPLE_LOST_STATUS` communication status.
- virtual void `on_instance_replaced` (DDSDataReader *, const DDS_DataReaderInstanceReplacedStatus &)
<<cert>> Handles the `DDS_INSTANCE_REPLACED_STATUS` communication status.
- virtual DDS_Boolean `on_before_sample_deserialize` (DDSDataReader *, NDDS_Type_Plugin *, CDR_Stream_t *, DDS_Boolean *)
<<cert>> Callback to filter a received sample based on serialized data
- virtual DDS_Boolean `on_before_sample_commit` (DDSDataReader *, const void *const, const struct DDS_SampleInfo *const, DDS_Boolean *)
<<cert>> Callback to filter a received sample based on deserialized data

7.70.1 Detailed Description

<<interface>> <<cert>> DDSListener for reader status.

Entity:

DDSDataReader

Status:

DDS_DATA_AVAILABLE_STATUS;
 DDS_LIVELINESS_CHANGED_STATUS, DDS_LivelinessChangedStatus;
 DDS_REQUESTED_DEADLINE_MISSED_STATUS, DDS_RequestedDeadlineMissedStatus;
 DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS, DDS_RequestedIncompatibleQosStatus;
 DDS_SAMPLE_LOST_STATUS, DDS_SampleLostStatus;
 DDS_SAMPLE_REJECTED_STATUS, DDS_SampleRejectedStatus;
 DDS_SUBSCRIPTION_MATCHED_STATUS, DDS_SubscriptionMatchedException;
 DDS_INSTANCE_REPLACED_STATUS, DDS_DataReaderInstanceReplacedStatus;

See also

Status Kinds

7.70.2 Member Function Documentation

7.70.2.1 virtual void DDSDataReaderListener::on_data_available (DDSDataReader *) [inline], [virtual]

<<cert>> Handle the `DDS_DATA_AVAILABLE_STATUS` communication status.

7.70.2.2 `virtual void DDSDataReaderListener::on_requested_deadline_missed ( DDSDataReader *, const DDS_RequestedDeadlineMissedStatus & ) [inline],[virtual]`

<<cert>> Handles the [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#) communication status.

7.70.2.3 `virtual void DDSDataReaderListener::on_liveliness_changed ( DDSDataReader *, const DDS_LivelinessChangedStatus & ) [inline],[virtual]`

<<cert>> Handles the [DDS_LIVELINESS_CHANGED_STATUS](#) communication status.

7.70.2.4 `virtual void DDSDataReaderListener::on_requested_incompatible_qos ( DDSDataReader *, const DDS_RequestedIncompatibleQosStatus & ) [inline],[virtual]`

<<cert>> Handles the [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#) communication status.

7.70.2.5 `virtual void DDSDataReaderListener::on_sample_rejected ( DDSDataReader *, const DDS_SampleRejectedStatus & ) [inline],[virtual]`

<<cert>> Handles the [DDS_SAMPLE_REJECTED_STATUS](#) communication status.

7.70.2.6 `virtual void DDSDataReaderListener::on_subscription_matched ( DDSDataReader *, const DDS_SubscriptionMatchedStatus & ) [inline],[virtual]`

<<cert>> Handles the [DDS_SUBSCRIPTION_MATCHED_STATUS](#) communication status.

7.70.2.7 `virtual void DDSDataReaderListener::on_sample_lost ( DDSDataReader *, const DDS_SampleLostStatus & ) [inline],[virtual]`

<<cert>> Handles the [DDS_SAMPLE_LOST_STATUS](#) communication status.

7.70.2.8 `virtual void DDSDataReaderListener::on_instance_replaced ( DDSDataReader *, const DDS_DataReaderInstanceReplacedStatus & ) [inline],[virtual]`

<<cert>> Handles the [DDS_INSTANCE_REPLACED_STATUS](#) communication status.

This listener may only be called when the instance replacement policy is set to `DDS_REPLACE_OLDEST_INSTANCE_REPLACEMENT_QOS`. When a new instance is detected and resources are exhausted the oldest instance will be removed even if samples with a not read state exists for the instance. Thus, this is a forceful removal of samples and the instance. The freed instance resources will be re-used for the newly detected instance. If there are outstanding loans for the replaced instance the samples can not be freed until returned. However, when the loan is returned the samples are automatically freed. Note that the instance will not be rejected.

7.70.2.9 `virtual DDS_Boolean DDSDataReaderListener::on_before_sample_deserialize ( DDSDataReader *, NDDS_Type_Plugin *, CDR.Stream_t *, DDS_Boolean * ) [inline],[virtual]`

<<cert>> Callback to filter a received sample based on serialized data

When a DataReader receives a (serialized) sample, it will deserialize it before storing it in its queue. `On_before_sample_deserialize()` is called before deserialization. It allows the application to determine whether or not to drop the sample, before it is read or taken.

The callback controls whether the sample is filter out of the DataReader's queue, with the `sample_dropped` parameter. If user code within the callback determines that the sample should be dropped, the user should set `sample_dropped` to true; consequently, when the callback returns, the DataReader will drop the sample before it is committed to the queue. Otherwise, returning the callback with `sample_dropped` as false will allow the sample to be read or taken.

Compared to `on_before_sample_commit()`, the sample would be dropped with less processing (without deserialization) by the subscribing application, but with the added complexity of filtering on serialized data.

The callback is not associated with a DDS Status. It is enabled when the callback in the listener is assigned, to a non-NULL callback.

References RTI_TRUE.

7.70.2.10 `virtual DDS_Boolean DDSDataReaderListener::on_before_sample_commit ( DDSDataReader *, const void * const, const struct DDS_SampleInfo * const, DDS_Boolean * ) [inline],[virtual]`

<<cert>> Callback to filter a received sample based on deserialized data

When a DataReader receives a (serialized) sample, it will deserialize it before storing it in its queue. On `on_before_sample_commit()` is called after deserialization, but before the sample is stored into the DataReader's queue. It allows the application to determine whether or not to drop the sample, before it is read or taken.

The callback controls whether the sample is filter out of the DataReader's queue, with the `sample_dropped` parameter. If user code within the callback determines that the sample should be dropped, the user should set `sample_dropped` to true; consequently, when the callback returns, the DataReader will drop the sample before it is committed to the queue. Otherwise, returning the callback with `sample_dropped` as false will allow the sample to be read or taken.

Compared to `on_before_sample_deserialize()`, the sample would be dropped with more processing (with deserialization) by the subscribing application, but with the lesser complexity of filtering on deserialized data.

The callback is not associated with a DDS Status. It is enabled when the callback in the listener is assigned, to a non-NULL callback.

The following fields are valid in the `DDS_SampleInfo` structure:

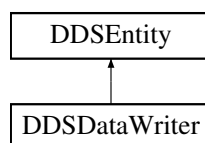
- `source_timestamp`
- `instance_handle`
- `publication_handle`
- `valid_data`
- `encapsulation_id`

References RTI_TRUE.

7.71 DDSDataWriter Class Reference

<<interface>> <<cert>> Allows an application to set the value of the data to be published under a given `DDS-Topic`.

Inheritance diagram for DDSDataWriter:



Public Member Functions

- virtual `DDS_ReturnCode_t write_untyped (const void *instance_data, const DDS_InstanceHandle_t &handle)=0`
 <<cert>> *Modifies the value of a data instance.*
- virtual `DDS_ReturnCode_t assert_liveliness ()=0`
 <<cert>> *This operation manually asserts the liveliness of this DDSDataWriter.*
- virtual `DDSTopic * get_topic ()=0`
 <<cert>> *This operation returns the DDSTopic associated with the DDSDataWriter.*
- virtual `DDSPublisher * get_publisher ()=0`

- <<cert>> This operation returns the *DDSPublisher* to which the *DDSDataWriter* belongs.
- virtual *DDS_ReturnCode_t* *get_qos* (*DDS_DataWriterQos* &qos)=0

Gets the writer QoS.
- virtual *DDS_ReturnCode_t* *set_qos* (const *DDS_DataWriterQos* &qos)=0

Sets the writer QoS.
- virtual *DDS_ReturnCode_t* *set_listener* (const *DDSDDataWriterListener* *listener, *DDS_StatusMask* mask)=0

Sets the writer listener.
- virtual *DDSDDataWriterListener* * *get_listener* ()=0

Get the writer listener.
- virtual *DDS_InstanceHandle_t* *register_instance_untyped* (const void *instance_data)=0

<<cert>> *Informs RTI Connex Micro that the application will be modifying a particular instance.*
- virtual *DDS_InstanceHandle_t* *register_instance_w_timestamp_untyped* (const void *instance_data, const *DDS_Time_t* &source_timestamp)=0

<<cert>> *Performs the same function as *DDSDDataWriter::register_instance_untyped* except that it also provides the value for the *source_timestamp*.*
- virtual *DDS_ReturnCode_t* *unregister_instance_untyped* (const void *instance_data, const *DDS_InstanceHandle_t* &handle)=0

<<cert>> *Reverses the action of *DDSDDataWriter::register_instance_untyped*.*
- virtual *DDS_ReturnCode_t* *unregister_instance_w_timestamp_untyped* (const void *instance_data, const *DDS_InstanceHandle_t* &handle, const *DDS_Time_t* &source_timestamp)=0

<<cert>> *Performs the same function as *DDSDDataWriter::unregister_instance_untyped* except that it also provides the value for the *source_timestamp*.*
- virtual *DDS_ReturnCode_t* *dispose_untyped* (const void *instance_data, const *DDS_InstanceHandle_t* &handle)=0

<<cert>> *Requests the middleware to delete the data.*
- virtual *DDS_ReturnCode_t* *dispose_w_timestamp_untyped* (const void *instance_data, const *DDS_InstanceHandle_t* &handle, const *DDS_Time_t* &source_timestamp)=0

<<cert>> *Performs the same function as *DDSDDataWriter::dispose_untyped* except that it also provides the value for the *source_timestamp*.*
- virtual *DDS_ReturnCode_t* *write_w_timestamp_untyped* (const void *instance_data, const *DDS_InstanceHandle_t* &handle, const *DDS_Time_t* &source_timestamp)=0

<<cert>> *Performs the same function as *DDSDDataWriter::write_untyped* except that it also provides the value for the *source_timestamp*.*
- virtual *DDS_ReturnCode_t* *get_publication_matched_status* (*DDS_PublicationMatchedStatus* &status)=0

<<cert>> *Accesses the *DDS_PUBLICATION_MATCHED_STATUS* communication status.*
- virtual *DDS_ReturnCode_t* *get_liveliness_lost_status* (*DDS_LivelinessLostStatus* &status)=0

<<cert>> *Accesses the *DDS_LIVELINESS_LOST_STATUS* communication status.*
- virtual *DDS_ReturnCode_t* *get_offered_deadline_missed_status* (*DDS_OfferedDeadlineMissedStatus* &status)=0

<<cert>> *Accesses the *DDS_OFFERED_DEADLINE_MISSED_STATUS* communication status.*
- virtual *DDS_ReturnCode_t* *get_offered_incompatible_qos_status* (*DDS_OfferedIncompatibleQosStatus* &status)=0

<<cert>> *Accesses the *DDS_OFFERED_INCOMPATIBLE_QOS_STATUS* communication status.*

7.7.1.1 Detailed Description

<<interface>> <<cert>> Allows an application to set the value of the data to be published under a given *DDS-Topic*.

QoS:

DDS_DataWriterQos

Status:

```
DDS_LIVELINESS_LOST_STATUS, DDS_LivelinessLostStatus;
DDS_OFFERED_DEADLINE_MISSED_STATUS, DDS_OfferedDeadlineMissedStatus;
DDS_OFFERED_INCOMPATIBLE_QOS_STATUS, DDS_OfferedIncompatibleQosStatus;
DDS_PUBLICATION_MATCHED_STATUS, DDS_PublicationMatchedStatus;
```

Listener:

```
DDSDDataWriterListener
```

A [DDSDDataWriter](#) is attached to exactly one [DDSPublisher](#), that acts as a factory for it.

A [DDSDDataWriter](#) is bound to exactly one [DDSTopic](#) and therefore to exactly one data type. The [DDSTopic](#) must exist prior to the [DDSDDataWriter](#)'s creation.

[DDSDDataWriter](#) is an abstract class. It must be specialized for each particular application data-type. The additional methods or functions that must be defined in the auto-generated class for a hypothetical application type `Foo` are specified in the example type [DDSDDataWriter](#).

The following operations may be called even if the [DDSDDataWriter](#) is not enabled. Other operations will fail with [DDS_RETCODE_NOT_ENABLED](#) if called on a disabled [DDSDDataWriter](#):

- The base-class operations [DDSEntity::enable](#), [DDSDDataWriter::get_qos](#), [DDSDDataWriter::get_listener](#), and [idref_DataWriter_set_qos](#), [DDSDDataWriter::set_listener](#)
- [DDSDDataWriter::get_liveliness_lost_status](#), [DDSDDataWriter::get_offered_deadline_missed_status](#), [DDSDDataWriter::get_offered_incompatible_qos_status](#), [DDSDDataWriter::get_publication_matched_status](#)

Several [DDSDDataWriter](#) may operate in different threads. If they share the same [DDSPublisher](#), the middleware guarantees that its operations are thread-safe.

7.71.2 Member Function Documentation

7.71.2.1 `virtual DDS_ReturnCode_t DDSDDataWriter::write_untyped ( const void * instance_data, const DDS_InstanceHandle_t & handle ) [pure virtual]`

<<cert>> Modifies the value of a data instance.

When this operation is used, RTI Connex Micro will automatically supply the value of the `source_timestamp` that is made available to [DDSDDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS_SampleInfo](#). (Refer to [DDS_SampleInfo](#) for details).

As a side effect, this operation asserts liveliness on the [DDSDDataWriter](#) itself, the [DDSPublisher](#) and the [DDS-DomainParticipant](#).

Note that the special value [DDS_HANDLE_NIL](#) can be used for the parameter `handle`. This indicates the identity of the instance should be automatically deduced from the `instance_data` (by means of the `key`).

If `handle` is any value other than [DDS_HANDLE_NIL](#), then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with [DDS_RETCODE_BAD_PARAMETER](#).

RTI Connex Micro will not detect the error when the `handle` is any value other than [DDS_HANDLE_NIL](#), corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the `key`). RTI Connex Micro will treat as if the `write()` operation is for the instance as indicated by the `handle`.

If there are no instance resources left, this operation may fail with [DDS_RETCODE_OUT_OF_RESOURCES](#). Calling [DDSDDataWriter::unregister_instance_untyped](#) may help freeing up some resources.

Parameters

<i>instance_data</i>	<< <i>in</i> >> The data to write.
<i>handle</i>	<< <i>in</i> >> Either the handle returned by a previous call to DDSDDataWriter::register_instance_untyped , or else the special value DDS_HANDLE_NIL . If the data <i>has</i> a key and <i>handle</i> is not DDS_HANDLE_NIL , <i>handle</i> must represent a registered instance of the data. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#), [DDS_RETCODE_OUT_OF_RESOURCES](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDDataReader](#)
[DDSDDataWriter::write_w_timestamp_untyped](#)

7.71.2.2 `virtual DDS_ReturnCode_t DDSDDataWriter::assert_liveliness ( ) [pure virtual]`

<<*cert*>> This operation manually asserts the liveliness of this [DDSDDataWriter](#).

This is used in combination with the [LIVELINESS](#) policy to indicate to RTI Connex Micro that the [DDSDDataWriter](#) remains active.

You only need to use this operation if the [LIVELINESS](#) setting is [DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS](#). Otherwise, it has no effect.

Note: writing data via the [FooDataWriter::write](#) or [FooDataWriter::write_w_timestamp](#) operation asserts liveliness on the [DDSDDataWriter](#) itself, and its [DDSDDomainParticipant](#). Consequently the use of [assert_liveliness\(\)](#) is only needed if the application is not writing data regularly.

Returns

One of the [Standard Return Codes](#) or [DDS_RETCODE_NOT_ENABLED](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDS_LivelinessQosPolicy](#)

7.71.2.3 `virtual DDSTopic* DDSDDataWriter::get_topic ( ) [pure virtual]`

<<*cert*>> This operation returns the [DDSTopic](#) associated with the [DDSDDataWriter](#).

This is the same [DDSTopic](#) that was used to create the [DDSDDataWriter](#).

Returns

[DDSTopic](#) that was used to create the [DDSDDataWriter](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.71.2.4 `virtual DDSPublisher* DDSDataWriter::get_publisher ( ) [pure virtual]`

<<cert>> This operation returns the [DDSPublisher](#) to which the [DDSDataWriter](#) belongs.

Returns

[DDSPublisher](#) to which the [DDSDataWriter](#) belongs.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.71.2.5 `virtual DDS_ReturnCode_t DDSDataWriter::get_qos ( DDS_DataWriterQos & qos ) [pure virtual]`

Gets the writer QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<code>qos</code>	<<inout>> The DDS_DataWriterQos to be filled up.
------------------	--

Returns

One of the [Standard Return Codes](#)

See also

[get_qos \(abstract\)](#)

7.71.2.6 `virtual DDS_ReturnCode_t DDSDataWriter::set_qos ( const DDS_DataWriterQos & qos ) [pure virtual]`

Sets the writer QoS.

This operation modifies the QoS of the [DDSDataWriter](#).

The [DDS_DataWriterQos::deadline](#) and [DDS_DataWriterQos::ownership_strength](#), can be changed. The other policies are immutable.

Parameters

<code>qos</code>	<<in>> The DDS_DataWriterQos to be set to. Policies must be consistent. Policies cannot be changed after DDSDataWriter is enabled. The special value DDS_DATAWRITER_QOS_DEFAULT can be used to indicate that the QoS of the DDSDataWriter should be changed to match the current default DDS_DataWriterQos set in the DDSPublisher .
------------------	--

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_IMMUTABLE_POLICY](#) or [DDS_RETCODE_INCONSISTENT_POLICY](#)

See also

[DDS_DataWriterQos](#) for rules on consistency among QoS
[set_qos \(abstract\)](#)

7.71.2.7 `virtual DDS_ReturnCode_t DDSDataWriter::set_listener ( const DDSDataWriterListener * listener, DDS_StatusMask mask ) [pure virtual]`

Sets the writer listener.

Parameters

<i>listener</i>	<<in>> DDSDataWriterListener to set to
<i>mask</i>	<<in>> DDS_StatusMask associated with the DDSDataWriterListener .

Returns

One of the [Standard Return Codes](#)

See also

[set_listener](#) (abstract)

7.71.2.8 `virtual DDSDataWriterListener* DDSDataWriter::get_listener ( ) [pure virtual]`

Get the writer listener.

Returns

[DDSDataWriterListener](#) of the [DDSDataWriter](#).

7.71.2.9 `virtual DDS_InstanceHandle_t DDSDataWriter::register_instance_untyped ( const void * instance_data ) [pure virtual]`

<<cert>> Informs RTI Connex Micro that the application will be modifying a particular instance.

This operation is only useful for keyed data types. Using it for non-keyed types causes no effect and returns [DDS_HANDLE_NIL](#). The operation takes as a parameter an instance (of which only the key value is examined) and returns a `handle` that can be used in successive `write()` or `dispose()` operations.

The operation gives RTI Connex Micro an opportunity to pre-configure itself to improve performance.

The use of this operation by an application is optional even for keyed types. If an instance has not been pre-registered, the application can use the special value [DDS_HANDLE_NIL](#) as the [DDS_InstanceHandle_t](#) parameter to the `write` or `dispose` operation and RTI Connex Micro will auto-register the instance.

For best performance, the operation should be invoked prior to calling any operation that modifies the instance, such as [DDSDataWriter::write_untyped](#), [DDSDataWriter::write_w_timestamp_untyped](#), [DDSDataWriter::dispose_untyped](#) and the handle used in conjunction with the data for those calls.

When this operation is used, RTI Connex Micro will automatically supply the value of the `source_timestamp` that is used.

This operation may fail and return [DDS_HANDLE_NIL](#) if [DDS_ResourceLimitsQosPolicy::max_instances](#) limit has been exceeded.

The operation is **idempotent**. If it is called for an already registered instance, it just returns the already allocated handle. This may be used to lookup and retrieve the handle allocated to a given instance.

This operation can only be called after [DDSDataWriter](#) has been enabled. Otherwise, [DDS_HANDLE_NIL](#) will be returned.

Parameters

<i>instance_data</i>	<<in>> The instance that should be registered. Of this instance, only the fields that represent the key are examined by the function. .
----------------------	---

Returns

For keyed data type, a handle that can be used in the calls that take a [DDS_InstanceHandle_t](#), such as `write`, `dispose`, `unregister_instance`, or return `DDS_HANDLE_NIL` on failure. If the `instance_data` is of a data type that has no keys, this function always return `DDS_HANDLE_NIL`.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDataWriter::unregister_instance_untyped](#)

7.71.2.10 `virtual DDS_InstanceHandle_t DDSDataWriter::register_instance_w_timestamp_untyped ( const void * instance_data, const DDS_Time_t & source_timestamp ) [pure virtual]`

<<cert>> Performs the same function as [DDSDataWriter::register_instance_untyped](#) except that it also provides the value for the `source_timestamp`.

Explicitly provides the timestamp that will be available to the [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS_SampleInfo](#). (Refer to [DDS_SampleInfo](#) for details)

The constraints on the values of the parameters and the corresponding error behavior are the same specified for the [DDSDataWriter::register_instance_untyped](#) operation.

Parameters

<i>instance_data</i>	<<in>> The instance that should be registered. Of this instance, only the fields that represent the key are examined by the function. .
<i>source_timestamp</i>	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the DDSDataReader objects by means of the <code>source_timestamp</code> attribute inside the DDS_SampleInfo .

Returns

For keyed data type, a handle that can be used in the calls that take a [DDS_InstanceHandle_t](#), such as `write`, `dispose`, `unregister_instance`, or return `DDS_HANDLE_NIL` on failure. If the `instance_data` is of a data type that has no keys, this function always return `DDS_HANDLE_NIL`.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDataWriter::unregister_instance_untyped](#)

7.71.2.11 `virtual DDS_ReturnCode_t DDSDataWriter::unregister_instance_untyped ( const void * instance_data, const DDS_InstanceHandle_t & handle ) [pure virtual]`

<<cert>> Reverses the action of [DDSDataWriter::register_instance_untyped](#).

This operation is useful only for keyed data types. Using it for non-keyed types causes no effect and reports no error. The operation takes as a parameter an instance (of which only the key value is examined) and a handle.

This operation should only be called on an instance that is currently registered. This includes instances that have been auto-registered by calling operations such as `write` or `dispose` as described in [DDSDDataWriter::register_instance_untyped](#). Otherwise, this operation may fail with [DDS_RETCODE_BAD_PARAMETER](#).

This only need be called just once per instance, regardless of how many times `register_instance` was called for that instance.

When this operation is used, RTI Connex Micro will automatically supply the value of the `source_timestamp` that is used.

This operation informs RTI Connex Micro that the [DDSDDataWriter](#) is no longer going to provide any information about the instance. This operation also indicates that RTI Connex Micro can locally remove all information regarding that instance. The application should not attempt to use the `handle` previously allocated to that instance after calling [DDSDDataWriter::unregister_instance_untyped\(\)](#).

The special value [DDS_HANDLE_NIL](#) can be used for the parameter `handle`. This indicates that the identity of the instance should be automatically deduced from the `instance_data` (by means of the `key`).

If `handle` is any value other than [DDS_HANDLE_NIL](#), then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with [DDS_RETCODE_BAD_PARAMETER](#).

RTI Connex Micro will not detect the error when the `handle` is any value other than [DDS_HANDLE_NIL](#), corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the `key`). RTI Connex Micro will treat as if the `unregister_instance()` operation is for the instance as indicated by the `handle`.

If after a [DDSDDataWriter::unregister_instance_untyped](#), the application wants to modify ([DDSDDataWriter::write_untyped](#) or [DDSDDataWriter::dispose_untyped](#)) an instance, it has to register it again, or else use the special `handle` value [DDS_HANDLE_NIL](#).

This operation does not indicate that the instance is deleted (that is the purpose of [DDSDDataWriter::dispose_untyped](#)). The operation [DDSDDataWriter::unregister_instance_untyped](#) just indicates that the [DDSDDataWriter](#) no longer has anything to say about the instance. [DDSDDataReader](#) entities that are reading the instance may receive a sample with [DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE](#) for the instance, unless there are other [DDSDDataWriter](#) objects writing that same instance.

This operation can affect the ownership of the data instance (see [OWNERSHIP](#)). If the [DDSDDataWriter](#) was the exclusive owner of the instance, then calling `unregister_instance()` will relinquish that ownership.

Parameters

<i>instance_data</i>	<<in>> The instance that should be unregistered. If the data <i>has</i> a key and <code>instance_handle</code> is DDS_HANDLE_NIL , only the fields that represent the key are examined by the function. Otherwise, <code>instance_data</code> is not used. If <code>instance_data</code> is used, it must represent an instance that has been registered. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .
<i>handle</i>	<<in>> represents the instance to be unregistered. If the data <i>has</i> a key and <code>handle</code> is DDS_HANDLE_NIL , <code>handle</code> is not used and <code>instance</code> is deduced from <code>instance_data</code> . If the data has no key, <code>handle</code> is not used. If <code>handle</code> is used, it must represent an instance that has been registered. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#) or [DDS_RETCODE_NOT_ENABLED](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDDataWriter::register_instance_untyped](#)

7.71.2.12 `virtual DDS_ReturnCode_t DDSDataWriter::unregister_instance_w_timestamp_untyped ( const void * instance_data, const DDS_InstanceHandle_t & handle, const DDS_Time_t & source_timestamp ) [pure virtual]`

<<cert>> Performs the same function as [DDSDataWriter::unregister_instance_untyped](#) except that it also provides the value for the `source_timestamp`.

Explicitly provides the timestamp that will be available to the [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS_SampleInfo](#). (Refer to [DDS_SampleInfo](#) for details)

The constraints on the values of the parameters and the corresponding error behavior are the same specified for the [DDSDataWriter::unregister_instance_untyped](#) operation.

Parameters

<i>instance_data</i>	<<in>> The instance that should be unregistered. If the data <i>has</i> a key and <code>instance_handle</code> is DDS_HANDLE_NIL , only the fields that represent the key are examined by the function. Otherwise, <code>instance_data</code> is not used. If <code>instance_data</code> is used, it must represent an instance that has been registered. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .
<i>handle</i>	<<in>> represents the instance to be unregistered. If the data <i>has</i> a key and <code>handle</code> is DDS_HANDLE_NIL , <code>handle</code> is not used and <code>instance</code> is deduced from <code>instance_data</code> . If the data has no key, <code>handle</code> is not used. If <code>handle</code> is used, it must represent an instance that has been registered. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .

Parameters

<i>source_timestamp</i>	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the DDSDataReader objects by means of the <code>source_timestamp</code> attribute inside the DDS_SampleInfo .
-------------------------	---

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#) or [DDS_RETCODE_NOT_ENABLED](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDataWriter::register_instance_untyped](#)

7.71.2.13 `virtual DDS_ReturnCode_t DDSDataWriter::dispose_untyped ( const void * instance_data, const DDS_InstanceHandle_t & handle ) [pure virtual]`

<<cert>> Requests the middleware to delete the data.

This operation is useful only for keyed data types. Using it for non-keyed types has no effect and reports no error.

The actual deletion is postponed until there is no more use for that data in the whole system.

Applications are made aware of the deletion by means of operations on the [DDSDataReader](#) objects that already knew that instance. [DDSDataReader](#) objects that didn't know the instance will never see it.

This operation does not modify the value of the instance. The `instance_data` parameter is passed just for the purposes of identifying the instance.

When this operation is used, RTI Connext Micro will automatically supply the value of the `source_timestamp` that is made available to `DDSDataReader` objects by means of the `source_timestamp` attribute inside the `DDS_SampleInfo`.

The constraints on the values of the handle parameter and the corresponding error behavior are the same specified for the `DDSDataWriter::unregister_instance_untyped` operation.

The special value `DDS_HANDLE_NIL` can be used for the parameter `instance_handle`. This indicates the identity of the instance should be automatically deduced from the `instance_data` (by means of the key).

If `handle` is any value other than `DDS_HANDLE_NIL`, then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with `DDS_RETCODE_BAD_PARAMETER`.

RTI Connext Micro will not detect the error when the `handle` is any value other than `DDS_HANDLE_NIL`, corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the key). RTI Connext Micro will treat as if the `dispose()` operation is for the instance as indicated by the `handle`.

This operation may block and time out (`DDS_RETCODE_TIMEOUT`) under the same circumstances described for `DDSDataWriter::write_untyped()`.

If there are no instance resources left, this operation may fail with `DDS_RETCODE_OUT_OF_RESOURCES`. Calling `DDSDataWriter::unregister_instance_untyped` may help freeing up some resources.

Parameters

<i>instance_data</i>	<<in>> The data to dispose. If the data <i>has</i> a key and <code>instance_handle</code> is <code>DDS_HANDLE_NIL</code> , only the fields that represent the key are examined by the function. Otherwise, <code>instance_data</code> is not used.
<i>handle</i>	<<in>> Either the handle returned by a previous call to <code>DDSDataWriter::register_instance_untyped</code> , or else the special value <code>DDS_HANDLE_NIL</code> . If the data <i>has</i> a key and <code>instance_handle</code> is <code>DDS_HANDLE_NIL</code> , <code>instance_handle</code> is not used and <code>instance</code> is deduced from <code>instance_data</code> . If the data has no key, <code>instance_handle</code> is not used. If <code>handle</code> is used, it must represent a registered instance of type the matching data type. Otherwise, this method fail with <code>DDS_RETCODE_BAD_PARAMETER</code> .

Returns

One of the [Standard Return Codes](#), `DDS_RETCODE_TIMEOUT`, `DDS_RETCODE_OUT_OF_RESOURCES` or `DDS_RETCODE_NOT_ENABLED`.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.71.2.14 `virtual DDS_ReturnCode_t DDSDataWriter::dispose_w_timestamp_untyped ( const void * instance_data, const DDS_InstanceHandle_t & handle, const DDS_Time_t & source_timestamp ) [pure virtual]`

<<cert>> Performs the same function as `DDSDataWriter::dispose_untyped` except that it also provides the value for the `source_timestamp`.

Explicitly provides the timestamp that will be available to the `DDSDataReader` objects by means of the `source_timestamp` attribute inside the `DDS_SampleInfo`. (Refer to `DDS_SampleInfo` for details)

The constraints on the values of the parameters and the corresponding error behavior are the same specified for the `DDSDataWriter::dispose_untyped` operation.

Parameters

<i>instance_data</i>	<<in>> The data to dispose. If the data <i>has</i> a key and <code>instance_handle</code> is <code>DDS_HANDLE_NIL</code> , only the fields that represent the key are examined by the function. Otherwise, <code>instance_data</code> is not used.
----------------------	--

<i>handle</i>	<<in>> Either the handle returned by a previous call to DDSDataWriter::register_instance_untyped , or else the special value DDS_HANDLE_NIL . If the data <i>has</i> a key and <i>instance_handle</i> is DDS_HANDLE_NIL , <i>instance_handle</i> is not used and <i>instance</i> is deduced from <i>instance_data</i> . If the data has no key, <i>instance_handle</i> is not used. If <i>handle</i> is used, it must represent a registered instance of type the matching data type. Otherwise, this method fail with DDS_RETCODE_BAD_PARAMETER .
<i>source_timestamp</i>	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the DDSDataReader objects by means of the <i>source_timestamp</i> attribute inside the DDS_SampleInfo .

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#), [DDS_RETCODE_OUT_OF_RESOURCES](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.71.2.15 `virtual DDS_ReturnCode_t DDSDataWriter::write_w_timestamp_untyped ( const void * instance_data, const DDS_InstanceHandle_t & handle, const DDS_Time_t & source_timestamp ) [pure virtual]`

<<cert>> Performs the same function as [DDSDataWriter::write_untyped](#) except that it also provides the value for the *source_timestamp*.

Explicitly provides the timestamp that will be available to the [DDSDataReader](#) objects by means of the *source_timestamp* attribute inside the [DDS_SampleInfo](#). (Refer to [DDS_SampleInfo](#) for details)

The constraints on the values of the *handle* parameter and the corresponding error behavior are the same specified for the [DDSDataWriter::write_untyped](#) operation.

If there are no instance resources left, this operation may fail with [DDS_RETCODE_OUT_OF_RESOURCES](#). Calling [DDSDataWriter::unregister_instance_untyped](#) may help free up some resources.

This operation may fail with [DDS_RETCODE_BAD_PARAMETER](#) under the same circumstances described for the *write* operation.

Parameters

<i>instance_data</i>	<<in>> The data to write.
<i>handle</i>	<<in>> Either the handle returned by a previous call to DDSDataWriter::register_instance_untyped , or else the special value DDS_HANDLE_NIL . If the data <i>has</i> a key and <i>handle</i> is not DDS_HANDLE_NIL , <i>handle</i> must represent a registered instance of the matching data type. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .
<i>source_timestamp</i>	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the DDSDataReader objects by means of the <i>source_timestamp</i> attribute inside the DDS_SampleInfo .

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#), [DDS_RETCODE_OUT_OF_RESOURCES](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDataWriter::write_untyped](#)
[DDSDataReader](#)

7.71.2.16 `virtual DDS_ReturnCode_t DDSDataWriter::get_publication_matched_status ( DDS_PublicationMatchedStatus & status ) [pure virtual]`

[<<cert>>](#) Accesses the [DDS_PUBLICATION_MATCHED_STATUS](#) communication status.

Parameters

<i>status</i>	<<inout>> DDS_PublicationMatchedStatus to be filled in.
---------------	---

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.71.2.17 `virtual DDS_ReturnCode_t DDSDataWriter::get_liveliness_lost_status ( DDS_LivelinessLostStatus & status ) [pure virtual]`

[<<cert>>](#) Accesses the [DDS_LIVELINESS_LOST_STATUS](#) communication status.

Parameters

<i>status</i>	<<inout>> DDS_LivelinessLostStatus to be filled in.
---------------	---

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.71.2.18 `virtual DDS_ReturnCode_t DDSDataWriter::get_offered_deadline_missed_status ( DDS_OfferedDeadlineMissedStatus & status ) [pure virtual]`

[<<cert>>](#) Accesses the [DDS_OFFERED_DEADLINE_MISSED_STATUS](#) communication status.

Parameters

<i>status</i>	<<inout>> DDS_OfferedDeadlineMissedStatus to be filled in.
---------------	--

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.71.2.19 virtual DDS_ReturnCode_t DDSDataWriter::get_offered_incompatible_qos_status (DDS_OfferedIncompatibleQosStatus & status) [pure virtual]

<<cert>> Accesses the [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#) communication status.

Parameters

status	<<inout>> DDS_OfferedIncompatibleQosStatus to be filled in.
--------	---

Returns

One of the [Standard Return Codes](#)

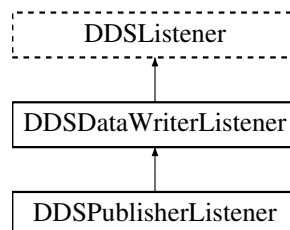
MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.72 DDSDataWriterListener Class Reference

<<interface>> <<cert>> [DDSListener](#) for writer status.

Inheritance diagram for DDSDataWriterListener:



Public Member Functions

- virtual void [on_offered_deadline_missed](#) (DDSDataWriter *writer, const [DDS_OfferedDeadlineMissedStatus](#) &status)
<<cert>> Handles the [DDS_OFFERED_DEADLINE_MISSED_STATUS](#) status.
- virtual void [on_liveliness_lost](#) (DDSDataWriter *writer, const [DDS_LivelinessLostStatus](#) &status)
<<cert>> Handles the [DDS_LIVELINESS_LOST_STATUS](#) status.
- virtual void [on_offered_incompatible_qos](#) (DDSDataWriter *writer, const [DDS_OfferedIncompatibleQosStatus](#) &status)
<<cert>> Handles the [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#) status.
- virtual void [on_publication_matched](#) (DDSDataWriter *writer, const [DDS_PublicationMatchedException](#) &status)
<<cert>> Handles the [DDS_PUBLICATION_MATCHED_STATUS](#) status.
- virtual void [on_reliable_reader_activity_changed](#) (DDSDataWriter *writer, const [DDS_ReliableReaderActivityChangedStatus](#) &status)

- <<eXtension>> <<cert>> A matched reliable reader has become active or become inactive.*
- virtual void `on_reliable_sample_unacknowledged` (`DDSDataWriter *writer`, const `DDS_ReliableSampleUnacknowledgedStatus &status`)
- <<eXtension>> <<cert>> A matched reliable reader has become active or become inactive.*

7.72.1 Detailed Description

<<interface>> <<cert>> `DDSListener` for writer status.

Entity:

`DDSDataWriter`

Status:

`DDS_LIVELINESS_LOST_STATUS`, `DDS_LivelinessLostStatus`;
`DDS_OFFERED_INCOMPATIBLE_QOS_STATUS`, `DDS_OfferedIncompatibleQosStatus`;
`DDS_PUBLICATION_MATCHED_STATUS`, `DDS_PublicationMatchedStatus`;

See also

[Status Kinds](#)

7.72.2 Member Function Documentation

7.72.2.1 virtual void `DDSDataWriterListener::on_offered_deadline_missed` (`DDSDataWriter * writer`, const `DDS_OfferedDeadlineMissedStatus & status`) `[inline]`, `[virtual]`

<<cert>> Handles the `DDS_OFFERED_DEADLINE_MISSED_STATUS` status.

This callback is called when the deadline that the `DDSDataWriter` has committed through its `DEADLINE` qos policy was not respected for a specific instance. This callback is called for each deadline period elapsed during which the `DDSDataWriter` failed to provide data for an instance.

Parameters

<i>writer</i>	<i><<out>></i> Locally created <code>DDSDataWriter</code> that triggers the listener callback
<i>status</i>	<i><<out>></i> Current deadline missed status of locally created <code>DDSDataWriter</code>

7.72.2.2 virtual void `DDSDataWriterListener::on_liveliness_lost` (`DDSDataWriter * writer`, const `DDS_LivelinessLostStatus & status`) `[inline]`, `[virtual]`

<<cert>> Handles the `DDS_LIVELINESS_LOST_STATUS` status.

This callback is called when the liveliness that the `DDSDataWriter` has committed through its `LIVELINESS` qos policy was not respected; this `DDSDataReader` entities will consider the `DDSDataWriter` as no longer "alive/active". This callback will not be called when an already not alive `DDSDataWriter` simply remains not alive for another liveliness period.

Parameters

<i>writer</i>	<i><<out>></i> Locally created <code>DDSDataWriter</code> that triggers the listener callback
<i>status</i>	<i><<out>></i> Current liveliness lost status of locally created <code>DDSDataWriter</code>

7.72.2.3 virtual void `DDSDataWriterListener::on_offered_incompatible_qos` (`DDSDataWriter * writer`, const `DDS_OfferedIncompatibleQosStatus & status`) `[inline]`, `[virtual]`

<<cert>> Handles the `DDS_OFFERED_INCOMPATIBLE_QOS_STATUS` status.

This callback is called when the [DDS_DataWriterQos](#) of the [DDSDDataWriter](#) was incompatible with what was requested by a [DDSDDataReader](#). This callback is called when a [DDSDDataWriter](#) has discovered a [DDSDDataReader](#) for the same [DDSTopic](#) and common partition, but with a requested QoS that is incompatible with that offered by the [DDSDDataWriter](#).

Parameters

<i>writer</i>	<<out>> Locally created DDSDDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current incompatible qos status of locally created DDSDDataWriter

7.72.2.4 `virtual void DDSDataWriterListener::on_publication_matched ( DDSDDataWriter * writer, const DDS\_PublicationMatchedStatus & status )` `[inline]`,`[virtual]`

<<cert>> Handles the [DDS_PUBLICATION_MATCHED_STATUS](#) status.

This callback is called when the [DDSDDataWriter](#) has found a [DDSDDataReader](#) that matches the [DDSTopic](#), has a common partition and compatible QoS, or has ceased to be matched with a [DDSDDataReader](#) that was previously considered to be matched.

Parameters

<i>writer</i>	<<out>> Locally created DDSDDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current publication match status of locally created DDSDDataWriter

7.72.2.5 `virtual void DDSDataWriterListener::on_reliable_reader_activity_changed ( DDSDDataWriter * writer, const DDS\_ReliableReaderActivityChangedStatus & status )` `[inline]`,`[virtual]`

<<eXtension>> <<cert>> A matched reliable reader has become active or become inactive.

A reliable writer considers a matched reliable reader to be active when it receives ACKNACK messages from the reader in response to its HEARTBEAT messages. The writer considers an active reader to have become inactive when it has sent a number of consecutive periodic HEARTBEATS equal to [DDS_RtpsReliableWriterProtocol_t::max_heartbeat_retries](#) without receiving an ACKNACK from the reader. An inactive reader becomes active once the writer receives an ACKNACK from it.

This callback is called when the transition from active to inactive, and vice versa, has occurred for a matched reliable reader.

Parameters

<i>writer</i>	<<out>> Locally created DDSDDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current reliable reader activity changed status of locally created DDSDDataWriter

7.72.2.6 `virtual void DDSDataWriterListener::on_reliable_sample_unacknowledged ( DDSDDataWriter * writer, const DDS\_ReliableSampleUnacknowledgedStatus & status )` `[inline]`,`[virtual]`

<<eXtension>> <<cert>> A matched reliable reader has become active or become inactive.

A reliable writer considers a matched reliable reader to be active when it receives ACKNACK messages from the reader in response to its HEARTBEAT messages. The writer considers an active reader to have become inactive when it has sent a number of consecutive periodic HEARTBEATS equal to [DDS_RtpsReliableWriterProtocol_t::max_heartbeat_retries](#) without receiving an ACKNACK from the reader. An inactive reader becomes active once the writer receives an ACKNACK from it.

This callback is called when the transition from active to inactive, and vice versa, has occurred for a matched reliable reader.

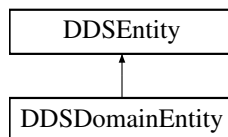
Parameters

<i>writer</i>	<<out>> Locally created DDSDDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current reliable reader activity changed status of locally created DDSDDataWriter

7.73 DDSDomainEntity Class Reference

<<interface>> Abstract base class for all DDS entities except for the [DDSDomainParticipant](#).

Inheritance diagram for DDSDomainEntity:



Additional Inherited Members

7.73.1 Detailed Description

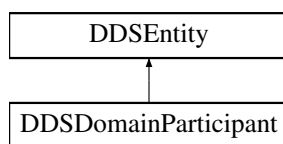
<<interface>> Abstract base class for all DDS entities except for the [DDSDomainParticipant](#).

Its sole purpose is to *conceptually* express that [DDSDomainParticipant](#) is a special kind of [DDSEntity](#) that acts as a container of all other [DDSEntity](#) but itself cannot contain other [DDSDomainParticipant](#).

7.74 DDSDomainParticipant Class Reference

<<interface>> <<cert>> Container for all [DDSDomainEntity](#) objects.

Inheritance diagram for DDSDomainParticipant:



Public Member Functions

- virtual [DDSEntity](#) * [as_entity](#) ()=0
 <<cert>> Access a [DDSDomainParticipant](#)'s supertype instance.
- virtual [DDSPublisher](#) * [create_publisher](#) (const [DDS_PublisherQos](#) &qos, [DDSPublisherListener](#) *listener, [DDS_StatusMask](#) mask)=0
 <<cert>> Creates a [DDSPublisher](#) with the desired QoS policies and attaches to it the specified [DDSPublisherListener](#).
- virtual [DDS_ReturnCode_t](#) [delete_publisher](#) ([DDSPublisher](#) *p)=0
 Deletes an existing [DDSPublisher](#).
- virtual [DDSSubscriber](#) * [create_subscriber](#) (const [DDS_SubscriberQos](#) &qos, [DDSSubscriberListener](#) *listener, [DDS_StatusMask](#) mask)=0
 <<cert>> Creates a [DDSSubscriber](#) with the desired QoS policies and attaches to it the specified [DDSSubscriberListener](#).
- virtual [DDS_ReturnCode_t](#) [delete_subscriber](#) ([DDSSubscriber](#) *s)=0
 Deletes an existing [DDSSubscriber](#).
- virtual [DDSTopic](#) * [create_topic](#) (const char *topic_name, const char *type_name, const [DDS_TopicQos](#) &qos, [DDSTopicListener](#) *listener, [DDS_StatusMask](#) mask)=0
 <<cert>> Creates a [DDSTopic](#) with the desired QoS policies
- virtual [DDS_ReturnCode_t](#) [delete_topic](#) ([DDSTopic](#) *topic)=0
 Deletes a [DDSTopic](#).

- virtual `DDSTopic * find_topic` (const char *topic_name, const `DDS_Duration_t` &timeout)=0
Finds an existing (or ready to exist) `DDSTopic`, based on its name.
- virtual `DDS_ReturnCode_t get_qos` (`DDS_DomainParticipantQos` &qos)=0
Get the participant QoS.
- virtual `DDS_ReturnCode_t set_qos` (const `DDS_DomainParticipantQos` &qos)=0
Change the QoS of this domain participant.
- virtual `DDS_ReturnCode_t get_default_publisher_qos` (`DDS_PublisherQos` &qos)=0
Copy the default `DDS_PublisherQos` values into the provided `DDS_PublisherQos` instance.
- virtual `DDS_ReturnCode_t set_default_publisher_qos` (const `DDS_PublisherQos` &qos)=0
Set the default `DDS_PublisherQos` values for this domain participant.
- virtual `DDS_ReturnCode_t get_default_subscriber_qos` (`DDS_SubscriberQos` &qos)=0
Copy the default `DDS_SubscriberQos` values into the provided `DDS_SubscriberQos` instance.
- virtual `DDS_ReturnCode_t set_default_subscriber_qos` (const `DDS_SubscriberQos` &qos)=0
Set the default `DDS_SubscriberQos` values for this domain participant.
- virtual `DDS_ReturnCode_t get_default_topic_qos` (`DDS_TopicQos` &qos)=0
Copies the default `DDS_TopicQos` values for this domain participant into the given `DDS_TopicQos` instance.
- virtual `DDS_ReturnCode_t set_default_topic_qos` (const `DDS_TopicQos` &qos)=0
Set the default `DDS_TopicQos` values for this domain participant.
- virtual `DDSTopicDescription * lookup_topicdescription` (const char *topic_name)=0
<<cert>> Lookup an existing locally-created `DDSTopicDescription`, based on its name.
- virtual `DDS_ReturnCode_t delete_contained_entities` ()=0
Delete all the entities that were created by means of the "create" operations on the `DDSDomainParticipant`.
- virtual `DDS_ReturnCode_t register_type` (const char *type_name, `NDDS_Type_Plugin` *plugin)=0
<<eXtension>> <<cert>> Allows an application to communicate to RTI Connex Micro the existence of a data type.
- virtual `NDDS_Type_Plugin * unregister_type` (const char *type_name)=0
Allows an application to unregister a data type from RTI Connex Micro. After calling `unregister_type`, no further communication using that type is possible.
- virtual `DDS_ReturnCode_t set_listener` (const `DDSDomainParticipantListener` *listener, `DDS_StatusMask` mask)=0
Sets the participant listener.
- virtual `DDSDomainParticipantListener * get_listener` ()=0
Get the participant listener.
- virtual `DDS_ReturnCode_t get_current_time` (`DDS_Time_t` ¤t_time)=0
<<cert>> Returns the current value of the time.
- virtual `DDS_ReturnCode_t add_peer` (const char *peer)=0
<<eXtension>> <<cert>> Attempt to contact one or more additional peer participants.

7.74.1 Detailed Description

<<interface>> <<cert>> Container for all `DDSDomainEntity` objects.

The `DomainParticipant` object plays several roles:

- It acts as a container for all other `DDSEntity` objects.
- It acts as *factory* for the `DDSPublisher`, `DDSSubscriber`, `DDSTopic` and `DDSEntity` objects.
- It represents the participation of the application on a communication plane that isolates applications running on the same set of physical computers from each other. A domain establishes a virtual network linking all applications that share the same `domainId` and isolating them from applications running on different domains. In this way, several independent distributed applications can coexist in the same physical network without interfering, or even being aware of each other.

A [DDSDomainParticipant](#) may be created in a disabled or enabled state; refer to [DDSEntity::enable](#) for details. The initial state of the [DDSDomainParticipant](#) is determined by [DDS_DomainParticipantFactoryQos::entity_factory](#).

The documentation for each operation on the [DDSDomainParticipant](#) includes any restrictions with respect to whether the operation is valid only for a disabled or enabled [DDSDomainParticipant](#).

The following operations may be called even if the [DDSDomainParticipant](#) is not enabled. Other operations will fail with the value [DDS_RETCODE_NOT_ENABLED](#) if called on a disabled DomainParticipant:

- Operations defined at the base-class level namely, [get_qos\(\)](#), [get_listener\(\)](#), and [enable\(\)](#), [set_qos\(\)](#), [set_listener\(\)](#)
- Factory operations: [create_topic\(\)](#), [create_publisher\(\)](#), [create_subscriber\(\)](#), [delete_topic\(\)](#), [delete_publisher\(\)](#), [delete_subscriber\(\)](#), [delete_contained_entities\(\)](#).

QoS:

[DDS_DomainParticipantQos](#)

Status:

[Status Kinds](#)

Listener:

[DDSDomainParticipantListener](#)

7.74.2 Member Function Documentation

7.74.2.1 `virtual DDSEntity* DDSDomainParticipant::as_entity ( ) [pure virtual]`

<<cert>> Access a [DDSDomainParticipant](#)'s supertype instance.

Returns

[DDSDomainParticipant](#)'s supertype [DDSEntity](#) instance

7.74.2.2 `virtual DDSPublisher* DDSDomainParticipant::create_publisher ( const DDS_PublisherQos & qos, DDSPublisherListener * listener, DDS_StatusMask mask ) [pure virtual]`

<<cert>> Creates a [DDSPublisher](#) with the desired QoS policies and attaches to it the specified [DDSPublisherListener](#).

Precondition

The specified QoS policies must be consistent, or the operation will fail and no [DDSPublisher](#) will be created.

MT Safety:

UNSAFE. If [DDS_PUBLISHER_QOS_DEFAULT](#) is used for `qos`, it is not safe to create the publisher while another thread may be simultaneously calling [DDSDomainParticipant::set_default_publisher_qos](#).

Parameters

<i>qos</i>	<<in>> QoS to be used for creating the new DDSPublisher . The special value DDS_PUBLISHER_QOS_DEFAULT can be used to indicate that the DDSPublisher should be created with the default DDS_PublisherQos set in the DDSDomainParticipant .
<i>listener</i>	<<in>> . Listener to be attached to the newly created DDSPublisher .
<i>mask</i>	<<in>> . Changes of communication status to be invoked on the listener.

Returns

newly created publisher object or NULL on failure.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

ARINC 653 API Restriction:

This function *must* only be called before a partition enters *NORMAL* mode.

See also

[Specifying QoS on entities](#) for information on setting QoS before entity creation
[DDS_PublisherQos](#) for rules on consistency among QoS
[DDS_PUBLISHER_QOS_DEFAULT](#)

7.74.2.3 `virtual DDS_ReturnCode_t DDSDomainParticipant::delete_publisher ( DDSPublisher * p ) [pure virtual]`

Deletes an existing [DDSPublisher](#).

Precondition

The [DDSPublisher](#) must not have any attached [DDSDataWriter](#) objects. If there are existing [DDSDataWriter](#) objects, it will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).
[DDSPublisher](#) must have been created by this [DDSDomainParticipant](#), or else it will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Postcondition

Listener installed on the [DDSPublisher](#) will not be called after this method completes successfully.

Parameters

<i>p</i>	<< <i>in</i> >> DDSPublisher to be deleted.
----------	---

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

7.74.2.4 `virtual DDSSubscriber* DDSDomainParticipant::create_subscriber ( const DDS_SubscriberQos & qos, DDSSubscriberListener * listener, DDS_StatusMask mask ) [pure virtual]`

<<*cert*>> Creates a [DDSSubscriber](#) with the desired QoS policies and attaches to it the specified [DDSSubscriberListener](#).

Precondition

The specified QoS policies must be consistent, or the operation will fail and no [DDSSubscriber](#) will be created.

MT Safety:

UNSAFE. If [DDS_SUBSCRIBER_QOS_DEFAULT](#) is used for *qos*, it is not safe to create the subscriber while another thread may be simultaneously calling [DDSDomainParticipant::set_default_subscriber_qos](#).

Parameters

<i>qos</i>	<<in>> QoS to be used for creating the new DDSSubscriber . The special value DDS_SUBSCRIBER_QOS_DEFAULT can be used to indicate that the DDSSubscriber should be created with the default DDS_SubscriberQos set in the DDSDomainParticipant .
<i>listener</i>	<<in>>. Listener to be attached to the newly created DDSSubscriber .
<i>mask</i>	<<in>>. Changes of communication status to be invoked on the listener.

Returns

newly created subscriber object or NULL on failure.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

ARINC 653 API Restriction:

This function *must* only be called before a partition enters *NORMAL* mode.

See also

[Specifying QoS on entities](#) for information on setting QoS before entity creation
[DDS_SubscriberQos](#) for rules on consistency among QoS
[DDS_SUBSCRIBER_QOS_DEFAULT](#)

7.74.2.5 `virtual DDS_ReturnCode_t DDSDomainParticipant::delete_subscriber ( DDSSubscriber * s ) [pure virtual]`

Deletes an existing [DDSSubscriber](#).

Precondition

The [DDSSubscriber](#) must not have any attached [DDSDataReader](#) objects. If there are existing [DDSDataReader](#) objects, it will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).
The [DDSSubscriber](#) must have been created by this [DDSDomainParticipant](#), or else it will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Postcondition

A Listener installed on the [DDSSubscriber](#) will not be called after this method completes successfully.

Parameters

<i>s</i>	<<in>> DDSSubscriber to be deleted.
----------	---

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

7.74.2.6 `virtual DDSTopic* DDSDomainParticipant::create_topic ( const char * topic_name, const char * type_name, const DDS_TopicQos & qos, DDSTopicListener * listener, DDS_StatusMask mask ) [pure virtual]`

<<cert>> Creates a [DDSTopic](#) with the desired QoS policies

Precondition

The application is not allowed to create two [DDSTopicDescription](#) objects with the same `topic_name` attached to the same [DDSDomainParticipant](#). If the application attempts this, this method will fail and return `NULL`.

The specified QoS policies must be consistent, or the operation will fail and no [DDSTopic](#) will be created.

Prior to creating a [DDSTopic](#), the type must have been registered with RTI Connex Micro. This is done using the [DDSDomainParticipant::register_type](#) operation on a derived class of the [NDDS_Type_Plugin](#) interface.

MT Safety:

UNSAFE. If [DDS_TOPIC_QOS_DEFAULT](#) is used for `qos`, it is not safe create the topic while another thread may be simultaneously calling [DDSDomainParticipant::set_default_topic_qos](#).

Parameters

<i>topic_name</i>	<<in>> Name for the new topic, must not exceed 255 characters. Cannot be <code>NULL</code> .
<i>type_name</i>	<<in>> The type to which the new DDSTopic will be bound. Cannot be <code>NULL</code> .
<i>qos</i>	<<in>> Must be DDS_TOPIC_QOS_DEFAULT .

Parameters

<i>listener</i>	<<in>>. Listener to be attached to the newly created DDSTopic .
<i>mask</i>	<<in>>. Changes of communication status to be invoked on the listener.

Returns

newly created topic, or `NULL` on failure

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

ARINC 653 API Restriction:

This function *must* only be called before a partition enters *NORMAL* mode.

See also

[Specifying QoS on entities](#) for information on setting QoS before entity creation

[DDS_TopicQos](#) for rules on consistency among QoS

[DDS_TOPIC_QOS_DEFAULT](#)

7.74.2.7 `virtual DDS_ReturnCode_t DDSDomainParticipant::delete_topic ( DDSTopic * topic ) [pure virtual]`

Deletes a [DDSTopic](#).

Precondition

If the [DDSTopic](#) does not belong to the application's [DDSDomainParticipant](#), this operation fails with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Make sure no objects are using the topic. More specifically, there must be no existing [DDSDataReader](#), or [DDS-DataWriter](#), objects belonging to the same [DDSDomainParticipant](#) that are using the [DDSTopic](#). If `delete_topic` is called on a [DDSTopic](#) with any of these existing objects attached to it, it will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Postcondition

Listener installed on the [DDSTopic](#) will not be called after this method completes successfully.

Parameters

<i>topic</i>	<< <i>in</i> >> DDSTopic to be deleted.
--------------	---

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#)

7.74.2.8 `virtual DDSTopic* DDSDomainParticipant::find_topic ( const char * topic_name, const DDS_Duration_t & timeout )` [pure virtual]

Finds an existing (or ready to exist) [DDSTopic](#), based on its name.

If a [DDSTopic](#) of the same name already exists, it gives access to it, otherwise it waits until either another mechanism creates it, or else completes when the specified `timeout` occurs.

[DDSTopic](#) obtained by [DDSDomainParticipant::find_topic](#) must also be deleted by means of [DDSDomainParticipant::delete_topic](#). If [DDSTopic](#) is obtained multiple times by means of [DDSDomainParticipant::find_topic](#) or [DDSDomainParticipant::create_topic](#), it must also be deleted that same number of times using [DDSDomainParticipant::delete_topic](#)

Parameters

<i>topic_name</i>	<< <i>in</i> >> Name of the DDSTopic to search for. Cannot be NULL.
<i>timeout</i>	<< <i>in</i> >> The time to wait if the DDSTopic doesn't exist already.

Returns

the topic, if it exists, or NULL

7.74.2.9 `virtual DDS_ReturnCode_t DDSDomainParticipant::get_qos ( DDS_DomainParticipantQos & qos )` [pure virtual]

Get the participant QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<i>qos</i>	<< <i>inout</i> >> QoS to be filled up.
------------	---

Returns

One of the [Standard Return Codes](#)

See also

[get_qos](#) (abstract)

7.74.2.10 `virtual DDS_ReturnCode_t DDSDomainParticipant::set_qos ( const DDS_DomainParticipantQos & qos )` [pure virtual]

Change the QoS of this domain participant.

The [DDS_DomainParticipantQos::entity_factory](#) can be changed. The other policies are immutable.

Parameters

<i>qos</i>	<< <i>in</i> >> Set of policies to be applied to DDSDomainParticipant . Policies must be consistent. Immutable policies cannot be changed after DDSDomainParticipant is enabled. The special value DDS_PARTICIPANT_QOS_DEFAULT can be used to indicate that the QoS of the DDSDomainParticipant should be changed to match the current default DDS_DomainParticipant-Qos set in the DDSDomainParticipantFactory .
------------	---

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_IMMUTABLE_POLICY](#) if immutable policy is changed, or [DDS_RETCODE_INCONSISTENT_POLICY](#) if policies are inconsistent

See also

[DDS_DomainParticipantQos](#) for rules on consistency among QoS
[set_qos](#) (abstract)

7.74.2.11 `virtual DDS_ReturnCode_t DDSDomainParticipant::get_default_publisher_qos ( DDS_PublisherQos & qos )`
 [pure virtual]

Copy the default [DDS_PublisherQos](#) values into the provided [DDS_PublisherQos](#) instance.

The retrieved *qos* will match the set of values specified on the last successful call to [DDSDomainParticipant::set_default_publisher_qos](#), or else, if the call was never made, the default values listed in [DDS_PublisherQos](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

If [DDS_PUBLISHER_QOS_DEFAULT](#) is specified as the *qos* parameter when [DDSDomainParticipant::create_topic](#) is called, the default value of the QoS set in the factory, equivalent to the value obtained by calling [DDS-DomainParticipant::get_default_publisher_qos](#), will be used to create the [DDSPublisher](#).

MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a domain participant while another thread may be simultaneously calling [DDSDomainParticipant::set_default_publisher_qos](#)

Parameters

<i>qos</i>	<< <i>inout</i> >> Qos to be filled up.
------------	---

Returns

One of the [Standard Return Codes](#)

See also

[DDS_PUBLISHER_QOS_DEFAULT](#)
[DDSDomainParticipant::create_publisher](#)

7.74.2.12 `virtual DDS_ReturnCode_t DDSDomainParticipant::set_default_publisher_qos ( const DDS_PublisherQos & qos )` [pure virtual]

Set the default [DDS_PublisherQos](#) values for this domain participant.

This default value will be used for newly created [DDSPublisher](#) if [DDS_PUBLISHER_QOS_DEFAULT](#) is specified as the *qos* parameter when [DDSDomainParticipant::create_publisher](#) is called.

Precondition

The specified QoS policies must be consistent, or else the operation will have no effect and fail with [DDS_RETCODE_INCONSISTENT_POLICY](#)

MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a domain participant while another thread may be simultaneously calling [DDSDomainParticipant::get_default_publisher_qos](#) or calling [DDSDomainParticipant::create_publisher](#) with [DDS_PUBLISHER_QOS_DEFAULT](#) as the `qos` parameter.

Parameters

<code>qos</code>	<< <i>in</i> >> Default qos to be set. The special value DDS_PUBLISHER_QOS_DEFAULT may be passed as <code>qos</code> to indicate that the default QoS should be reset back to the initial values the factory would use if DDSDomainParticipant::set_default_publisher_qos had never been called.
------------------	--

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#)

See also

[DDS_PUBLISHER_QOS_DEFAULT](#)
[DDSDomainParticipant::create_publisher](#)

7.74.2.13 `virtual DDS_ReturnCode_t DDSDomainParticipant::get_default_subscriber_qos ( DDS_SubscriberQos & qos ) [pure virtual]`

Copy the default [DDS_SubscriberQos](#) values into the provided [DDS_SubscriberQos](#) instance.

The retrieved `qos` will match the set of values specified on the last successful call to [DDSDomainParticipant::set_default_subscriber_qos](#), or else, if the call was never made, the default values listed in [DDS_SubscriberQos](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

If [DDS_SUBSCRIBER_QOS_DEFAULT](#) is specified as the `qos` parameter when [DDSDomainParticipant::create_subscriber](#) is called, the default value of the QoS set in the factory, equivalent to the value obtained by calling [DDSDomainParticipant::get_default_subscriber_qos](#), will be used to create the [DDSSubscriber](#).

MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a domain participant while another thread may be simultaneously calling [DDSDomainParticipant::set_default_subscriber_qos](#)

Parameters

<code>qos</code>	<< <i>inout</i> >> Qos to be filled up.
------------------	---

Returns

One of the [Standard Return Codes](#)

See also

[DDS_SUBSCRIBER_QOS_DEFAULT](#)
[DDSDomainParticipant::create_subscriber](#)

7.74.2.14 `virtual DDS_ReturnCode_t DDSDomainParticipant::set_default_subscriber_qos ( const DDS_SubscriberQos & qos ) [pure virtual]`

Set the default [DDS_SubscriberQos](#) values for this domain participant.

This default value will be used for newly created [DDSSubscriber](#) if [DDS_SUBSCRIBER_QOS_DEFAULT](#) is specified as the `qos` parameter when [DDSDomainParticipant::create_subscriber](#) is called.

Precondition

The specified QoS policies must be consistent, or else the operation will have no effect and fail with [DDS_RETCODE_INCONSISTENT_POLICY](#)

MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a domain participant while another thread may be simultaneously calling [DDSDomainParticipant::get_default_subscriber_qos](#) or calling [DDSDomainParticipant::create_subscriber](#) with [DDS_SUBSCRIBER_QOS_DEFAULT](#) as the `qos` parameter.

Parameters

<code>qos</code>	<< <i>in</i> >> Default qos to be set. The special value DDS_SUBSCRIBER_QOS_DEFAULT may be passed as <code>qos</code> to indicate that the default QoS should be reset back to the initial values the factory would use if DDSDomainParticipant::set_default_subscriber_qos had never been called.
------------------	--

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#)

7.74.2.15 `virtual DDS_ReturnCode_t DDSDomainParticipant::get_default_topic_qos ( DDS_TopicQos & qos ) [pure virtual]`

Copies the default [DDS_TopicQos](#) values for this domain participant into the given [DDS_TopicQos](#) instance.

The retrieved `qos` will match the set of values specified on the last successful call to [DDSDomainParticipant::set_default_topic_qos](#), or else, if the call was never made, the default values listed in [DDS_TopicQos](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a domain participant while another thread may be simultaneously calling [DDSDomainParticipant::get_default_topic_qos](#) or calling [DDSDomainParticipant::create_topic](#) with [DDS_TOPIC_QOS_DEFAULT](#) as the `qos` parameter.

Parameters

<code>qos</code>	<< <i>in</i> >> Default qos to be retrieved.
------------------	--

Returns

One of the [Standard Return Codes](#)

See also

[DDS_TOPIC_QOS_DEFAULT](#)
[DDSDomainParticipant::create_topic](#)

7.7.4.2.16 `virtual DDS_ReturnCode_t DDSDomainParticipant::set_default_topic_qos ( const DDS_TopicQos & qos )`
`[pure virtual]`

Set the default `DDS_TopicQos` values for this domain participant.

This default value will be used for newly created `DDSTopic` if `DDS_TOPIC_QOS_DEFAULT` is specified as the `qos` parameter when `DDSDomainParticipant::create_topic` is called.

Precondition

The specified QoS policies must be consistent, or else the operation will have no effect and fail with `DDS_RETCODE_INCONSISTENT_POLICY`

MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a domain participant while another thread may be simultaneously calling `DDSDomainParticipant::get_default_topic_qos` or calling `DDSDomainParticipant::create_topic` with `DDS_TOPIC_QOS_DEFAULT` as the `qos` parameter.

Parameters

<code>qos</code>	<<in>> Default qos to be set. The special value <code>DDS_TOPIC_QOS_DEFAULT</code> may be passed as <code>qos</code> to indicate that the default QoS should be reset back to the initial values the factory would have used if <code>DDSDomainParticipant::set_default_topic_qos</code> had never been called.
------------------	---

Returns

One of the [Standard Return Codes](#), or `DDS_RETCODE_INCONSISTENT_POLICY`

See also

`DDS_TOPIC_QOS_DEFAULT`
`DDSDomainParticipant::create_topic`

7.7.4.2.17 `virtual DDSTopicDescription* DDSDomainParticipant::lookup_topicdescription ( const char * topic_name )`
`[pure virtual]`

<<cert>> Lookup an existing locally-created `DDSTopicDescription`, based on its name.

The returned topic can either be enabled or disabled.

The operation `DDSDomainParticipant::lookup_topicdescription` searches among the locally created topics and returns a pointer to a `DDSTopicDescription`. It is possible to delete the `DDSTopicDescription` returned by `DDSDomainParticipant::lookup_topicdescription`, provided it has no readers or writers, but then it is really deleted and subsequent lookups will fail.

Parameters

<code>topic_name</code>	<<in>> Name of <code>DDSTopicDescription</code> to search for. This string must be no more than 255 characters. Cannot be NULL.
-------------------------	---

Returns

the topic description if it has already been created locally, or NULL otherwise.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.74.2.18 virtual DDS_ReturnCode_t DDSDomainParticipant::delete_contained_entities () [pure virtual]

Delete all the entities that were created by means of the "create" operations on the [DDSDomainParticipant](#).

This operation deletes all contained [DDSPublisher](#), [DDSSubscriber](#), and [DDSTopic](#).

Prior to deleting each contained entity, this operation will recursively call the corresponding delete_contained_entities operation on each contained entity (if applicable). This pattern is applied recursively. In this manner the operation [delete_contained_entities\(\)](#) on the [DDSDomainParticipant](#) will end up deleting all the entities recursively contained in the [DDSDomainParticipant](#), that is also the [DDSDataWriter](#), and [DDSDataReader](#).

The operation will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#) if any of the contained entities is in a state where it cannot be deleted.

If [delete_contained_entities\(\)](#) completes successfully, the application may delete the [DDSDomainParticipant](#) knowing that it has no contained entities.

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

7.74.2.19 virtual DDS_ReturnCode_t DDSDomainParticipant::register_type (const char * type_name, NDDS_Type_Plugin * plugin) [pure virtual]

<<eXtension>> <<cert>> Allows an application to communicate to RTI Connex Micro the existence of a data type.

The *generated* implementation of the operation embeds all the knowledge that has to be communicated to the middleware in order to make it able to manage the contents of data of that type. This includes in particular the key definition that will allow RTI Connex Micro to distinguish different instances of the same type.

The same [NDDS_Type_Plugin](#) can be registered multiple times with a [DDSDomainParticipant](#) using the same or different values for the `type_name`. If `register_type` is called multiple times on the same [NDDS_Type_Plugin](#) with the same [DDSDomainParticipant](#) and `type_name`, the second (and subsequent) registrations are ignored by the operation fails with [DDS_RETCODE_OK](#).

Precondition

Cannot use the same `type_name` to register two different [NDDS_Type_Plugin](#) with the same [DDSDomainParticipant](#), or else the operation will fail and [DDS_RETCODE_PRECONDITION_NOT_MET](#) will be returned.

Parameters

<i>type_name</i>	<<in>> the type name under with the data type <code>Foo</code> is registered with the participant; this type name is used when creating a new DDSTopic . (See DDSDomainParticipant::create_topic .) The name may not be NULL or longer than 255 characters.
<i>plugin</i>	<<in>> the <code>Foo</code> type plugin to register the data type with. Cannot be NULL.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#) or [DDS_RETCODE_OUT_OF_RESOURCES](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDomainParticipant::create_topic](#)

7.74.2.20 `virtual NDDS_Type_Plugin* DDSDomainParticipant::unregister_type ( const char * type_name ) [pure virtual]`

Allows an application to unregister a data type from RTI Connex Micro. After calling `unregister_type`, no further communication using that type is possible.

The generated implementation of the operation removes all the information about a type from RTI Connex Micro. No further communication using that type is possible.

Precondition

A type with `type_name` is registered with the participant and all [DDSTopic](#) objects referencing the type have been destroyed. If the type is not registered with the participant, or if any [DDSTopic](#) is associated with the type, the operation will fail with [DDS_RETCODE_ERROR](#).

Postcondition

All information about the type is removed from RTI Connex Micro. No further communication using this type is possible.

Parameters

<i>type_name</i>	<< <i>in</i> >> the type name under with the data type <code>Foo</code> is registered with the participant. The name should match a name that has been previously used to register a type with the participant. Cannot be NULL.
------------------	---

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_BAD_PARAMETER](#) or [DDS_RETCODE_ERROR](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDomainParticipant::register_type](#)

7.74.2.21 `virtual DDS_ReturnCode_t DDSDomainParticipant::set_listener ( const DDSDomainParticipantListener * listener, DDS_StatusMask mask ) [pure virtual]`

Sets the participant listener.

Parameters

<i>listener</i>	<< <i>in</i> >> Listener to be installed on entity.
<i>mask</i>	<< <i>in</i> >> Changes of communication status to be invoked on the listener.

MT Safety:

Unsafe. This method is not synchronized with the listener callbacks, so it is possible to set a new listener on a participant when the old listener is in a callback.

Care should therefore be taken not to delete any listener that has been set on an enabled participant unless some application-specific means are available of ensuring that the old listener cannot still be in use.

Returns

One of the [Standard Return Codes](#)

See also

[set_listener](#) (abstract)

7.74.2.22 `virtual DDSDomainParticipantListener* DDSDomainParticipant::get_listener ( ) [pure virtual]`

Get the participant listener.

Returns

Existing listener attached to the [DDSDomainParticipant](#).

7.74.2.23 `virtual DDS_ReturnCode_t DDSDomainParticipant::get_current_time ( DDS_Time_t & current_time ) [pure virtual]`

[<<cert>>](#) Returns the current value of the time.

The current value of the time that RTI Connex Micro uses to time-stamp [DDSDataWriter](#) and to set the reception-timestamp for the data updates that it receives. Note that the definition of the current time is dependent on the platform and may not be the actual `wall-clock` time. Additional information for specific platforms can be found in the User's Manual.

Parameters

<code>current_time</code>	<<inout>> Current time to be filled up.
---------------------------	---

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.74.2.24 `virtual DDS_ReturnCode_t DDSDomainParticipant::add_peer ( const char* peer ) [pure virtual]`

[<<eXtension>>](#) [<<cert>>](#) Attempt to contact one or more additional peer participants.

Add the given peer description to the list of peers with which this [DDSDomainParticipant](#) will try to communicate.

This method may be called at any time after this [DDSDomainParticipant](#) has been created, before or after it has been enabled. If it is called after [DDSEntity::enable](#), an attempt will be made to contact the new peer(s) immediately. If it is called before, the peer description will simply be added to the list that was populated by [DDS_DiscoveryQosPolicy::initial_peers](#); the first attempted contact will take place after this [DDSDomainParticipant](#) is enabled.

Adding a peer description with this method does not guarantee that any peer(s) discovered as a result will exactly correspond to those described:

- This [DDSDomainParticipant](#) will attempt to discover peer participants at the given locations, but may not succeed if no such participants are available. Such a situation will not result in an error result from this method, which will not wait for contact attempt(s) to be made.
- If remote participants such as are described by the given peer description are discovered, the distributed application is configured with asymmetric peer lists, and [DDS_DiscoveryQosPolicy::accept_unknown_peers](#) is set to [DDS_BOOLEAN_TRUE](#) this [DDSDomainParticipant](#) may actually discover *more* peers than are described in the given peer description.

Adding a peer description with this method has no effect on [DDS_DiscoveryQosPolicy::initial_peers](#).

Parameters

<i>peer</i>	<<in>> New peer descriptor to be added.
-------------	---

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDS_DiscoveryQosPolicy::initial_peers](#)
[DDS_PARTICIPANT_TOPIC_NAME](#)

7.75 DDSDomainParticipantFactory Class Reference

<<singleton>> <<interface>> <<cert>> Allows creation and destruction of [DDSDomainParticipant](#) objects.

Public Member Functions

- virtual [DDSDomainParticipant](#) * [create_participant](#) ([DDS_DomainId_t](#) domainId, const [DDS_DomainParticipantQos](#) &qos, [DDSDomainParticipantListener](#) *listener, [DDS_StatusMask](#) mask)=0
 <<cert>> Creates a new [DDSDomainParticipant](#) object.
- virtual [DDS_ReturnCode_t](#) [delete_participant](#) ([DDSDomainParticipant](#) *a_participant)=0
 Deletes an existing [DDSDomainParticipant](#).
- virtual [DDSDomainParticipant](#) * [lookup_participant](#) ([DDS_DomainId_t](#) domainId)=0
 <<cert>> Locates an existing [DDSDomainParticipant](#).
- virtual [DDS_ReturnCode_t](#) [set_default_participant_qos](#) (const [DDS_DomainParticipantQos](#) &qos)=0
 Sets the default [DDS_DomainParticipantQos](#) values for this domain participant factory.
- virtual [DDS_ReturnCode_t](#) [get_default_participant_qos](#) ([DDS_DomainParticipantQos](#) &qos)=0
 Initializes the [DDS_DomainParticipantQos](#) instance with default values.
- virtual [DDS_ReturnCode_t](#) [get_qos](#) ([DDS_DomainParticipantFactoryQos](#) &qos)=0
 <<cert>> Gets the value for participant factory QoS.
- virtual [DDS_ReturnCode_t](#) [set_qos](#) (const [DDS_DomainParticipantFactoryQos](#) &qos)=0
 <<cert>> Sets the value for a participant factory QoS.
- virtual [RTRegistry](#) * [get_registry](#) ()=0
 <<cert>> Retrieves the singleton registry instance associated with the factory.

Static Public Member Functions

- static
[DDSDomainParticipantFactory](#) * [get_instance](#) ()
 <<cert>> Gets the singleton instance of this class.
- static [DDS_ReturnCode_t](#) [finalize_instance](#) ()
 <<eXtension>> Destroys the singleton instance of this class.

7.75.1 Detailed Description

<<singleton>> <<interface>> <<cert>> Allows creation and destruction of [DDSDomainParticipant](#) objects.

The sole purpose of this class is to allow the creation and destruction of [DDSDomainParticipant](#) objects. This class itself is a <<singleton>>, and accessed via the [get_instance\(\)](#) method, and destroyed with [finalize_instance\(\)](#) method.

A single application can participate in multiple domains by instantiating multiple [DDSDomainParticipant](#) objects.

An application may even instantiate multiple participants in the same domain. Participants in the same domain exchange data in the same way regardless of whether they are in the same application or different applications or on the same node or different nodes; their location is transparent.

There are two important caveats:

- With multiple participants in the same domain on the same node (in the same application or different applications), each participant must be assigned its own receive ports. The receive ports are calculated based on the domain ID and the participant index set in the [DDS_WireProtocolQosPolicy](#). The default index (-1) causes the participant to automatically search for the next available index, starting at 0, when it is created. If an index larger or equal to 0 is assigned then only that value will be used. If another participant with the same index already exists the participant creation will fail.

NOTE: An exception to this rule is multicast ports. Multicast ports are shared between participants in the same domain on the same host). Thus, the index value is ignored.

- You cannot mix entities from different participants. For example, you cannot delete a topic on a different participant than you created it from, and you cannot ask a subscriber to create a reader for a topic created from a participant different than the subscriber's own participant. (Note that it is permissible for an application built on top of RTI Connex Micro to know about entities from different participants. For example, an application could keep references to a reader from one domain and a writer from another and then bridge the domains by writing the data received in the reader callback.)

See also

[DDSDomainParticipant](#)

7.75.2 Member Function Documentation

7.75.2.1 static [DDSDomainParticipantFactory](#)* [DDSDomainParticipantFactory::get_instance](#) () [static]

<<cert>> Gets the singleton instance of this class.

Returns

the [DDSDomainParticipantFactory](#) instance.

MT Safety:

UNSAFE on the FIRST call to [DDSDomainParticipantFactory::get_instance\(\)](#). It is not safe for two threads to simultaneously make the first call to get an instance of the factory. Subsequent calls are thread safe.

[DDSTheParticipantFactory](#) can be used as an alias for the singleton factory returned by this operation.

[DDSDomainParticipantFactory::get_instance](#) also initializes the RTI Connex Micro library and should be called before any other API, unless the API is explicitly mentioned as safe to call before [DDSDomainParticipantFactory::get_instance](#).

Returns

the singleton [DDSDomainParticipantFactory](#) instance.

ARINC 653 API Restriction:

This function *must* only be called before a partition enters *NORMAL* mode.

See also

[DDSTheParticipantFactory](#)

7.75.2.2 static DDS_ReturnCode_t DDSDomainParticipantFactory::finalize_instance () [static]

<<*eXtension*>> Destroys the singleton instance of this class.

Only necessary to explicitly reclaim resources used by the participant factory singleton. Note that on many OSes, these resources are automatically reclaimed by the OS when the program terminates. Some memory checker tools still flag these as unreclaimed however. So this method provides a way to clean up memory used by the participant factory.

Precondition

All participants created from the factory have been deleted.

Postcondition

All resources belonging to the factory reclaimed. Another call to [DDSDomainParticipantFactory::get_instance](#) will return a new lifecycle of the singleton.

MT Safety:

UNSAFE for two threads to simultaneously call [DDSDomainParticipantFactory::finalize_instance\(\)](#) or [DDS-DomainParticipantFactory::get_instance\(\)](#) at the same time.

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#)

7.75.2.3 virtual DDSDomainParticipant* DDSDomainParticipantFactory::create_participant (DDS_DomainId_t domainId, const DDS_DomainParticipantQos & qos, DDSDomainParticipantListener * listener, DDS_StatusMask mask) [pure virtual]

<<*cert*>> Creates a new [DDSDomainParticipant](#) object.

Precondition

The specified QoS policies must be consistent, or the operation will fail and no [DDSDomainParticipant](#) will be created.

If you want to create multiple participants on a given host in the same domain, make sure each one has a different participant index (set in the [DDS_WireProtocolQosPolicy](#)). This in turn will ensure each participant uses a different port number (since the unicast port numbers are calculated from the participant index and the domain ID).

Note that if there is a single participant per host in a given domain, the participant index can be left at the default value (-1).

Parameters

<i>domainId</i>	<<in>> ID of the domain that the application intends to join. [range] [≥ 0], and does not violate guidelines stated in DDS_RtpsWellKnownPorts_t .
<i>qos</i>	<<in>> the domain participant's QoS. The special value DDS_PARTICIPANT_QOS_DEFAULT can be used to indicate that the DDSDomainParticipant should be created with the default DDS_DomainParticipantQos set in the DDSDomainParticipantFactory .
<i>listener</i>	<<in>> the domain participant's listener.
<i>mask</i>	<<in>> Changes of communication status to be invoked on the listener.

Returns

domain participant or NULL on failure

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

ARINC 653 API Restriction:

This function *must* only be called before a partition enters *NORMAL* mode.

See also

[Specifying QoS on entities](#) for information on setting QoS before entity creation
[DDS_DomainParticipantQos](#) for rules on consistency among QoS
[DDS_PARTICIPANT_QOS_DEFAULT](#)

7.75.2.4 `virtual DDS_ReturnCode_t DDSDomainParticipantFactory::delete_participant ( DDSDomainParticipant * a_participant )` [pure virtual]

Deletes an existing [DDSDomainParticipant](#).

Precondition

All domain entities belonging to the participant must have already been deleted. Otherwise it fails with the error [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Postcondition

Listener installed on the [DDSDomainParticipant](#) will not be called after this method returns successfully.

Parameters

<i>a_participant</i>	<<in>> DDSDomainParticipant to be deleted.
----------------------	--

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

7.75.2.5 `virtual DDSDomainParticipant* DDSDomainParticipantFactory::lookup_participant ( DDS_DomainId_t domainId )` [pure virtual]

<<cert>> Locates an existing [DDSDomainParticipant](#).

If no such [DDSDomainParticipant](#) exists, the operation will return NULL value.

If multiple [DDSDomainParticipant](#) entities belonging to that domainId exist, then the operation will return one of them. It is not specified which one.

Parameters

<i>domainId</i>	<<in>> ID of the domain participant to lookup.
-----------------	--

Returns

domain participant if it exists, or NULL

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.75.2.6 `virtual DDS_ReturnCode_t DDSDomainParticipantFactory::set_default_participant_qos ( const DDS_DomainParticipantQos & qos ) [pure virtual]`

Sets the default [DDS_DomainParticipantQos](#) values for this domain participant factory.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<i>qos</i>	<<inout>> Qos to be used by the DDSDomainParticipant . The special value DDS_PARTICIPANT_QOS_DEFAULT may be passed as <i>qos</i> to indicate that the default QoS should be reset back to the initial values the factory would used if DDSDomainParticipantFactory::set_default_participant_qos had never been called.
------------	--

Returns

One of the [Standard Return Codes](#)

See also

[DDS_PARTICIPANT_QOS_DEFAULT](#)
[DDSDomainParticipantFactory::create_participant](#)

7.75.2.7 `virtual DDS_ReturnCode_t DDSDomainParticipantFactory::get_default_participant_qos ( DDS_DomainParticipantQos & qos ) [pure virtual]`

Initializes the [DDS_DomainParticipantQos](#) instance with default values.

The retrieved *qos* will match the set of values specified on the last successful call to [DDSDomainParticipantFactory::set_default_participant_qos](#), or else, if the call was never made, the default values listed in [DDS_DomainParticipantQos](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<i>qos</i>	<<out>> the domain participant's QoS
------------	--------------------------------------

Returns

One of the [Standard Return Codes](#)

See also

[DDS_PARTICIPANT_QOS_DEFAULT](#)
[DDSDomainParticipantFactory::create_participant](#)

7.75.2.8 `virtual DDS_ReturnCode_t DDSDomainParticipantFactory::get_qos ( DDS_DomainParticipantFactoryQos & qos ) [pure virtual]`

<<cert>> Gets the value for participant factory QoS.

Parameters

<code>qos</code>	<<inout>> QoS to be filled up.
------------------	--------------------------------

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.75.2.9 `virtual DDS_ReturnCode_t DDSDomainParticipantFactory::set_qos ( const DDS_DomainParticipantFactoryQos & qos ) [pure virtual]`

<<cert>> Sets the value for a participant factory QoS.

The [DDS_DomainParticipantFactoryQos::entity_factory](#) can be changed. The other policies are immutable.

Note that despite having QoS, the [DDSDomainParticipantFactory](#) is not an [DDSEntity](#).

Note that this function may cause RTI Connex Micro to free and reallocate memory, depending on the QoS policies that are changed.

Parameters

<code>qos</code>	<<in>> Set of policies to be applied to DDSDomainParticipantFactory . Policies must be consistent. Immutable policies can only be changed before calling any other RTI Connex Micro methods except for DDSDomainParticipantFactory::get_qos
------------------	---

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_IMMUTABLE_POLICY](#) if immutable policy is changed, or [DDS_RETCODE_INCONSISTENT_POLICY](#) if policies are inconsistent

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDS_DomainParticipantFactoryQos](#) for rules on consistency among QoS

7.75.2.10 `virtual RTRegistry* DDSDomainParticipantFactory::get_registry ( ) [pure virtual]`

<<cert>> Retrieves the singleton registry instance associated with the factory.

Returns

the registry instance or NULL on failure.

MT Safety:

This operation is thread safe.

7.76 DDSDomainParticipantListener Class Reference

<<interface>> <<cert>> Listener for participant status.

Public Member Functions

- virtual void `on_data_available` (DDSDataReader *)
 <<cert>> Handle the `DDS_DATA_AVAILABLE_STATUS` communication status.
- virtual void `on_requested_deadline_missed` (DDSDataReader *reader, const DDS_RequestedDeadlineMissedStatus &status)
 <<cert>> Handles the `DDS_REQUESTED_DEADLINE_MISSED_STATUS` communication status.
- virtual void `on_liveliness_changed` (DDSDataReader *reader, const DDS_LivelinessChangedStatus &status)
 <<cert>> Handles the `DDS_LIVELINESS_CHANGED_STATUS` communication status.
- virtual void `on_requested_incompatible_qos` (DDSDataReader *reader, const DDS_RequestedIncompatibleQosStatus &status)
 <<cert>> Handles the `DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS` communication status.
- virtual void `on_sample_rejected` (DDSDataReader *reader, const DDS_SampleRejectedStatus &status)
 <<cert>> Handles the `DDS_SAMPLE_REJECTED_STATUS` communication status.
- virtual void `on_subscription_matched` (DDSDataReader *reader, const DDS_SubscriptionMatchedStatus &status)
 <<cert>> Handles the `DDS_SUBSCRIPTION_MATCHED_STATUS` communication status.
- virtual void `on_sample_lost` (DDSDataReader *reader, const DDS_SampleLostStatus &status)
 <<cert>> Handles the `DDS_SAMPLE_LOST_STATUS` communication status.
- virtual void `on_instance_replaced` (DDSDataReader *reader, const DDS_DataReaderInstanceReplacedStatus &status)
 <<cert>> Handles the `DDS_INSTANCE_REPLACED_STATUS` communication status.
- virtual DDS_Boolean `on_before_sample_deserialize` (DDSDataReader *reader, NDDS_Type_Plugin *, CD-R_Stream_t *stream, DDS_Boolean *)
 <<cert>> Callback to filter a received sample based on serialized data
- virtual DDS_Boolean `on_before_sample_commit` (DDSDataReader *, const void *const, const struct DDS_SampleInfo *const, DDS_Boolean *)
 <<cert>> Callback to filter a received sample based on deserialized data
- virtual void `on_data_on_readers` (DDSSubscriber *)
 Handles the `DDS_DATA_ON_READERS_STATUS` communication status.
- virtual void `on_offered_deadline_missed` (DDSDataWriter *writer, const struct DDS_OfferedDeadlineMissedStatus &status)
 <<cert>> Handles the `DDS_OFFERED_DEADLINE_MISSED_STATUS` status.
- virtual void `on_liveliness_lost` (DDSDataWriter *writer, const struct DDS_LivelinessLostStatus &status)
 <<cert>> Handles the `DDS_LIVELINESS_LOST_STATUS` status.
- virtual void `on_offered_incompatible_qos` (DDSDataWriter *writer, const struct DDS_OfferedIncompatibleQosStatus &status)
 <<cert>> Handles the `DDS_OFFERED_INCOMPATIBLE_QOS_STATUS` status.
- virtual void `on_publication_matched` (DDSDataWriter *writer, const struct DDS_PublicationMatchedStatus &status)
 <<cert>> Handles the `DDS_PUBLICATION_MATCHED_STATUS` status.
- virtual void `on_reliable_reader_activity_changed` (DDSDataWriter *writer, const struct DDS_ReliableReaderActivityChangedStatus &status)
 <<eXtension>> <<cert>> A matched reliable reader has become active or become inactive.
- virtual void `on_inconsistent_topic` (DDSTopic *topic, const DDS_InconsistentTopicStatus &status)
 Handle the `DDS_INCONSISTENT_TOPIC_STATUS` status.

7.76.1 Detailed Description

<<interface>> <<cert>> Listener for participant status.

Entity:

[DDSDomainParticipant](#)

Status:

[Status Kinds](#)

This is the interface that can be implemented by an application-provided class and then registered with the [DDS-DomainParticipant](#) such that the application can be notified by RTI Connex Micro of relevant status changes.

The [DDSDomainParticipantListener](#) interface extends all other Listener interfaces and has no additional operation beyond the ones defined by the more general listeners.

The purpose of the [DDSDomainParticipantListener](#) is to be the listener of last resort that is notified of all status changes not captured by more specific listeners attached to the [DDSDomainEntity](#) objects. When a relevant status change occurs, RTI Connex Micro will first attempt to notify the listener attached to the concerned [DDS-DomainEntity](#) if one is installed. Otherwise, RTI Connex Micro will notify the Listener attached to the [DDSDomainParticipant](#).

Important: Because a [DDSDomainParticipantListener](#) may receive callbacks pertaining to many different entities, it is possible for the same listener to receive multiple callbacks simultaneously in different threads. (Such is not the case for listeners of other types.) It is therefore critical that users of this listener provide their own protection for any thread-unsafe activities undertaken in a [DDSDomainParticipantListener](#) callback.

See also

[DDSLListener](#)

[DDSDomainParticipant::set_listener](#) RTI Connex Micro C++ API avoids the use of multiple inheritance for certification purposes. For this reason, [DDSDomainParticipantListener](#) is not declared as a sub-class of [DDSPublisherListener](#), [DDSSubscriberListener](#), and [DDSTopicListener](#), but it explicitly declares all supported call-backs.

7.76.2 Member Function Documentation

7.76.2.1 `virtual void DDSDomainParticipantListener::on_data_available ( DDSDataReader * ) [inline], [virtual]`

<<cert>> Handle the [DDS_DATA_AVAILABLE_STATUS](#) communication status.

7.76.2.2 `virtual void DDSDomainParticipantListener::on_requested_deadline_missed ( DDSDataReader * reader, const DDS_RequestedDeadlineMissedStatus & status ) [inline], [virtual]`

<<cert>> Handles the [DDS_REQUESTED_DEADLINE_MISSED_STATUS](#) communication status.

7.76.2.3 `virtual void DDSDomainParticipantListener::on_liveliness_changed ( DDSDataReader * reader, const DDS_LivelinessChangedStatus & status ) [inline], [virtual]`

<<cert>> Handles the [DDS_LIVELINESS_CHANGED_STATUS](#) communication status.

7.76.2.4 `virtual void DDSDomainParticipantListener::on_requested_incompatible_qos ( DDSDataReader * reader, const DDS_RequestedIncompatibleQosStatus & status ) [inline], [virtual]`

<<cert>> Handles the [DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#) communication status.

7.76.2.5 `virtual void DDSDomainParticipantListener::on_sample_rejected ( DDSDataReader * reader, const DDS_SampleRejectedStatus & status ) [inline], [virtual]`

<<cert>> Handles the [DDS_SAMPLE_REJECTED_STATUS](#) communication status.

7.76.2.6 `virtual void DDSDomainParticipantListener::on_subscription_matched ( DDSDataReader * reader, const DDS_SubscriptionMatchedStatus & status ) [inline],[virtual]`

<<cert>> Handles the [DDS_SUBSCRIPTION_MATCHED_STATUS](#) communication status.

7.76.2.7 `virtual void DDSDomainParticipantListener::on_sample_lost ( DDSDataReader * reader, const DDS_SampleLostStatus & status ) [inline],[virtual]`

<<cert>> Handles the [DDS_SAMPLE_LOST_STATUS](#) communication status.

7.76.2.8 `virtual void DDSDomainParticipantListener::on_instance_replaced ( DDSDataReader * reader, const DDS_DataReaderInstanceReplacedStatus & status ) [inline],[virtual]`

<<cert>> Handles the [DDS_INSTANCE_REPLACED_STATUS](#) communication status.

This listener may only be called when the instance replacement policy is set to `DDS_REPLACE_OLDEST_INSTANCE_REPLACEMENT_QOS`. When a new instance is detected and resources are exhausted the oldest instance will be removed even if samples with a not read state exists for the instance. Thus, this is a forceful removal of samples and the instance. The freed instance resources will be re-used for the newly detected instance. If there are outstanding loans for the replaced instance the samples can not be freed until returned. However, when the loan is returned the samples are automatically freed. Note that the instance will not be rejected.

7.76.2.9 `virtual DDS_Boolean DDSDomainParticipantListener::on_before_sample_deserialize ( DDSDataReader * reader, NDDS_Type_Plugin *, CDR_Stream_t * stream, DDS_Boolean * ) [inline],[virtual]`

<<cert>> Callback to filter a received sample based on serialized data

When a DataReader receives a (serialized) sample, it will deserialize it before storing it in its queue. `On_before_sample_deserialize()` is called before deserialization. It allows the application to determine whether or not to drop the sample, before it is read or taken.

The callback controls whether the sample is filter out of the DataReader's queue, with the `sample_dropped` parameter. If user code within the callback determines that the sample should be dropped, the user should set `sample_dropped` to true; consequently, when the callback returns, the DataReader will drop the sample before it is committed to the queue. Otherwise, returning the callback with `sample_dropped` as false will allow the sample to be read or taken.

Compared to `on_before_sample_commit()`, the sample would be dropped with less processing (without deserialization) by the subscribing application, but with the added complexity of filtering on serialized data.

The callback is not associated with a DDS Status. It is enabled when the callback in the listener is assigned, to a non-NULL callback.

References `RTI_TRUE`.

7.76.2.10 `virtual DDS_Boolean DDSDomainParticipantListener::on_before_sample_commit ( DDSDataReader *, const void * const, const struct DDS_SampleInfo * const, DDS_Boolean * ) [inline],[virtual]`

<<cert>> Callback to filter a received sample based on deserialized data

When a DataReader receives a (serialized) sample, it will deserialize it before storing it in its queue. `On_before_sample_commit()` is called after deserialization, but before the sample is stored into the DataReader's queue. It allows the application to determine whether or not to drop the sample, before it is read or taken.

The callback controls whether the sample is filter out of the DataReader's queue, with the `sample_dropped` parameter. If user code within the callback determines that the sample should be dropped, the user should set `sample_dropped` to true; consequently, when the callback returns, the DataReader will drop the sample before it is committed to the queue. Otherwise, returning the callback with `sample_dropped` as false will allow the sample to be read or taken.

Compared to `on_before_sample_deserialize()`, the sample would be dropped with more processing (with deserialization) by the subscribing application, but with the lesser complexity of filtering on deserialized data.

The callback is not associated with a DDS Status. It is enabled when the callback in the listener is assigned, to a non-NULL callback.

The following fields are valid in the [DDS_SampleInfo](#) structure:

- source_timestamp
- instance_handle
- publication_handle
- valid_data
- encapsulation_id

References RTI_TRUE.

7.76.2.11 virtual void DDSDomainParticipantListener::on_data_on_readers (DDSSubscriber *) [inline],
[virtual]

Handles the [DDS_DATA_ON_READERS_STATUS](#) communication status.

7.76.2.12 virtual void DDSDomainParticipantListener::on_offered_deadline_missed (DDSDataWriter * writer, const struct
DDS_OfferedDeadlineMissedStatus & status) [inline], [virtual]

<<cert>> Handles the [DDS_OFFERED_DEADLINE_MISSED_STATUS](#) status.

This callback is called when the deadline that the [DDSDataWriter](#) has committed through its [DEADLINE](#) qos policy was not respected for a specific instance. This callback is called for each deadline period elapsed during which the [DDSDataWriter](#) failed to provide data for an instance.

Parameters

<i>writer</i>	<<out>> Locally created DDSDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current deadline missed status of locally created DDSDataWriter

7.76.2.13 virtual void DDSDomainParticipantListener::on_liveliness_lost (DDSDataWriter * writer, const struct
DDS_LivelinessLostStatus & status) [inline], [virtual]

<<cert>> Handles the [DDS_LIVELINESS_LOST_STATUS](#) status.

This callback is called when the liveliness that the [DDSDataWriter](#) has committed through its [LIVELINESS](#) qos policy was not respected; this [DDSDataReader](#) entities will consider the [DDSDataWriter](#) as no longer "alive/active". This callback will not be called when an already not alive [DDSDataWriter](#) simply remains not alive for another liveliness period.

Parameters

<i>writer</i>	<<out>> Locally created DDSDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current liveliness lost status of locally created DDSDataWriter

7.76.2.14 virtual void DDSDomainParticipantListener::on_offered_incompatible_qos (DDSDataWriter * writer, const struct
DDS_OfferedIncompatibleQosStatus & status) [inline], [virtual]

<<cert>> Handles the [DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#) status.

This callback is called when the [DDS_DataWriterQos](#) of the [DDSDataWriter](#) was incompatible with what was requested by a [DDSDataReader](#). This callback is called when a [DDSDataWriter](#) has discovered a [DDSDataReader](#) for the same [DDSTopic](#) and common partition, but with a requested QoS that is incompatible with that offered by the [DDSDataWriter](#).

Parameters

<i>writer</i>	<<out>> Locally created DDSDataWriter that triggers the listener callback
<i>status</i>	<<out>> Current incompatible qos status of locally created DDSDataWriter

7.76.2.15 `virtual void DDSDomainParticipantListener::on_publication_matched ( DDSDataWriter * writer, const struct DDS_PublicationMatchedStatus & status ) [inline],[virtual]`

<<cert>> Handles the `DDS_PUBLICATION_MATCHED_STATUS` status.

This callback is called when the `DDSDataWriter` has found a `DDSDataReader` that matches the `DDSTopic`, has a common partition and compatible QoS, or has ceased to be matched with a `DDSDataReader` that was previously considered to be matched.

Parameters

<i>writer</i>	<<out>> Locally created <code>DDSDataWriter</code> that triggers the listener callback
<i>status</i>	<<out>> Current publication match status of locally created <code>DDSDataWriter</code>

7.76.2.16 `virtual void DDSDomainParticipantListener::on_reliable_reader_activity_changed ( DDSDataWriter * writer, const struct DDS_ReliableReaderActivityChangedStatus & status ) [inline],[virtual]`

<<eXtension>> <<cert>> A matched reliable reader has become active or become inactive.

A reliable writer considers a matched reliable reader to be active when it receives ACKNACK messages from the reader in response to its HEARTBEAT messages. The writer considers an active reader to have become inactive when it has sent a number of consecutive periodic HEARTBEATs equal to `DDS_RtpsReliableWriterProtocol_t::max_heartbeat_retries` without receiving an ACKNACK from the reader. An inactive reader becomes active once the writer receives an ACKNACK from it.

This callback is called when the transition from active to inactive, and vice versa, has occurred for a matched reliable reader.

Parameters

<i>writer</i>	<<out>> Locally created <code>DDSDataWriter</code> that triggers the listener callback
<i>status</i>	<<out>> Current reliable reader activity changed status of locally created <code>DDSDataWriter</code>

7.76.2.17 `virtual void DDSDomainParticipantListener::on_inconsistent_topic ( DDSTopic * topic, const struct DDS_InconsistentTopicStatus & status ) [inline],[virtual]`

Handle the `DDS_INCONSISTENT_TOPIC_STATUS` status.

This callback is called when a remote `DDSTopic` is discovered but is inconsistent with the locally created `DDSTopic` of the same topic name.

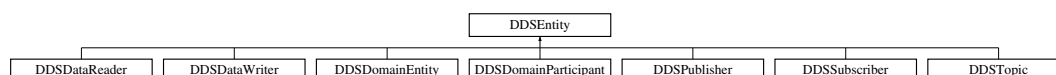
Parameters

<i>topic</i>	<<out>> Locally created <code>DDSTopic</code> that triggers the listener callback
<i>status</i>	<<out>> Current inconsistent status of locally created <code>DDSTopic</code>

7.77 DDSEntity Class Reference

<<interface>> Abstract base class for all the DDS objects that support QoS policies, and a listener.

Inheritance diagram for DDSEntity:



Public Member Functions

- `DDS_ReturnCode_t enable ()`

- `<<cert>>` Enables the `DDSEntity`.
- `DDSStatusCondition * get_statuscondition ()`
`<<cert>>` Return the `DDSStatusCondition` associated with a particular `DDSEntity`.
- `DDS_StatusMask get_status_changes ()`
`<<cert>>` Retrieves the list of communication statuses in the `DDSEntity` that are triggered.
- `DDS_InstanceHandle_t get_instance_handle ()`
`<<cert>>` Allows access to the `DDS_InstanceHandle_t` associated with the `DDSEntity`.

7.77.1 Detailed Description

`<<interface>>` Abstract base class for all the DDS objects that support QoS policies, and a listener.

All operations except for `set_qos()`, `get_qos()`, `set_listener()`, `get_listener()` and `enable()`, may return the value `DDS_RETCODE_NOT_ENABLED`.

QoS:

[QoS Policies](#)

Status:

[Status Kinds](#)

Listener:

[DDSListener](#)

7.77.2 Abstract operations

Each derived entity provides the following operations specific to its role in RTI Connext Micro.

7.77.2.1 `set_qos` (abstract)

This operation sets the QoS policies of the `DDSEntity`.

This operation must be provided by each of the derived `DDSEntity` classes (`DDSDomainParticipant`, `DDSTopic`, `DDSPublisher`, `DDSDataWriter`, `DDSSubscriber`, and `DDSDataReader`) so that the policies that are meaningful to each `DDSEntity` can be set.

Precondition

Certain policies are immutable (see [QoS Policies](#)): they can only be set at `DDSEntity` creation time or before the entity is enabled. If `set_qos()` is invoked after the `DDSEntity` is enabled and it attempts to change the value of an immutable policy, the operation will fail and return `DDS_RETCODE_IMMUTABLE_POLICY`.

Certain values of QoS policies can be incompatible with the settings of the other policies. The `set_qos()` operation will also fail if it specifies a set of values that, once combined with the existing values, would result in an inconsistent set of policies. In this case, the operation will fail and return `DDS_RETCODE_INCONSISTENT_POLICY`.

If the application supplies a non-default value for a QoS policy that is not supported by the implementation of the service, the `set_qos` operation will fail and return `DDS_RETCODE_UNSUPPORTED`.

Postcondition

The existing set of policies is only changed if the `set_qos()` operation succeeds. This is indicated by a return code of `DDS_RETCODE_OK`. In all other cases, none of the policies are modified.

Each derived `DDSEntity` class (`DDSDomainParticipant`, `DDSTopic`, `DDSPublisher`, `DDSDataWriter`, `DDSSubscriber`, `DDSDataReader`) has a corresponding special value of the QoS (`DDS_PARTICIPANT_QOS_DEFAULT`, `DDS_PUBLISHER_QOS_DEFAULT`, `DDS_SUBSCRIBER_QOS_DEFAULT`, `DDS_TOPIC_QOS_DEFAULT`,

`DDS_DATAWRITER_QOS_DEFAULT`, `DDS_DATAREADER_QOS_DEFAULT`). This special value may be used as a parameter to the `set_qos` operation to indicate that the QoS of the `DDSEntity` should be changed to match the current default QoS set in the `DDSEntity`'s factory. The operation `set_qos` cannot modify the immutable QoS, so a successful return of the operation indicates that the mutable QoS for the Entity has been modified to match the current default for the `DDSEntity`'s factory.

The set of policies specified in the `qos` parameter are applied on top of the existing QoS, replacing the values of any policies previously set.

Possible error codes returned in addition to [Standard Return Codes](#) : `DDS_RETCODE_IMMUTABLE_POLICY`, or `DDS_RETCODE_INCONSISTENT_POLICY`.

7.77.2.2 `get_qos` (abstract)

This operation allows access to the existing set of QoS policies for the `DDSEntity`. This operation must be provided by each of the derived `DDSEntity` classes (`DDSDomainParticipant`, `DDSTopic`, `DDSPublisher`, `DDSDataWriter`, `DDSSubscriber`, and `DDSDataReader`), so that the policies that are meaningful to each `DDSEntity` can be retrieved.

Possible error codes are [Standard Return Codes](#).

7.77.2.3 `set_listener` (abstract)

This operation installs a `DDSListener` on the `DDSEntity`. The listener will only be invoked on the changes of communication status indicated by the specified `mask`.

This operation must be provided by each of the derived `DDSEntity` classes (`DDSDomainParticipant`, `DDSTopic`, `DDSPublisher`, `DDSDataWriter`, `DDSSubscriber`, and `DDSDataReader`), so that the listener is of the concrete type suitable to the particular `DDSEntity`.

It is permitted to use NULL as the value of the listener. The NULL listener behaves as if the `mask` is `DDS_STATUS_MASK_NONE`.

Postcondition

Only one listener can be attached to each `DDSEntity`. If a listener was already set, the operation `set_listener()` will replace it with the new one. Consequently, if the value NULL is passed for the listener parameter to the `set_listener` operation, any existing listener will be removed.

7.77.2.4 `get_listener` (abstract)

This operation allows access to the existing `DDSListener` attached to the `DDSEntity`.

This operation must be provided by each of the derived `DDSEntity` classes (`DDSDomainParticipant`, `DDSTopic`, `DDSPublisher`, `DDSDataWriter`, `DDSSubscriber`, and `DDSDataReader`) so that the listener is of the concrete type suitable to the particular `DDSEntity`.

If no listener is installed on the `DDSEntity`, this operation will return NULL.

7.77.3 Member Function Documentation

7.77.3.1 `DDS_ReturnCode_t DDSEntity::enable ( )`

<<cert>> Enables the `DDSEntity`.

This operation enables the Entity. Entity objects can be created either enabled or disabled. This is controlled by the value of the `ENTITY_FACTORY` QoS policy on the corresponding factory for the `DDSEntity`.

By default, `ENTITY_FACTORY` is set so that it is not necessary to explicitly call `DDSEntity::enable` on newly created entities.

The `DDSEntity::enable` operation is idempotent. Calling `enable` on an already enabled Entity returns OK and has no effect.

If a `DDSEntity` has not yet been enabled, the following kinds of operations may be invoked on it:

- set or get the QoS policies (including default QoS policies) and listener
- 'factory' operations
- 'lookup' operations

Other operations may explicitly state that they may be called on disabled entities; those that do not will return the error [DDS_RETCODE_NOT_ENABLED](#).

It is legal to delete an [DDSEntity](#) that has not been enabled by calling the proper operation on its factory.

Entities created from a factory that is disabled are created disabled, regardless of the setting of the [DDS_EntityFactoryQosPolicy](#).

Calling enable on an Entity whose factory is not enabled will fail and return [DDS_RETCODE_PRECONDITION_NOT_MET](#).

If [DDS_EntityFactoryQosPolicy::autoenable_created_entities](#) is TRUE, the enable operation on a factory will automatically enable all entities created from that factory.

Listeners associated with an entity are not called until the entity is enabled.

Returns

One of the [Standard Return Codes](#), [Standard Return Codes](#) or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.77.3.2 DDSStatusCondition* DDSEntity::get_statuscondition ()

<<cert>> Return the [DDSStatusCondition](#) associated with a particular [DDSEntity](#).

Returns

The [DDSStatusCondition](#) that belongs to the specified [DDSEntity](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.77.3.3 DDS_StatusMask DDSEntity::get_status_changes ()

<<cert>> Retrieves the list of communication statuses in the [DDSEntity](#) that are triggered.

That is, the list of statuses whose value has changed since the last time the application read the status using the `get_*_status()` method.

When the entity is first created or if the entity is not enabled, all communication statuses are in the "untriggered" state so the list returned by the `get_status_changes` operation will be empty.

The list of statuses returned by the `get_status_changes` operation refers to the status that are triggered on the Entity itself and does not include statuses that apply to contained entities.

Returns

list of communication statuses in the [DDSEntity](#) that are triggered.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[Status Kinds](#)

7.77.3.4 DDS_InstanceHandle_t DDSEntity::get_instance_handle ()

<<cert>> Allows access to the [DDS_InstanceHandle_t](#) associated with the [DDSEntity](#).

This operation returns the [DDS_InstanceHandle_t](#) that represents the [DDSEntity](#).

Returns

the instance handle associated with this entity.

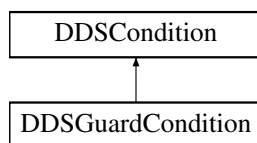
MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.78 DDSGuardCondition Class Reference

<<interface>> **<<cert>>** A specific [DDSCondition](#) whose `trigger_value` is completely under the control of the application.

Inheritance diagram for DDSGuardCondition:



Public Member Functions

- [DDS_ReturnCode_t set_trigger_value](#) (bool value)
<<cert>> Set the guard condition trigger value.

7.78.1 Detailed Description

<<interface>> **<<cert>>** A specific [DDSCondition](#) whose `trigger_value` is completely under the control of the application.

The [DDSGuardCondition](#) provides a way for an application to manually wake up a [DDSWaitSet](#). This is accomplished by attaching the [DDSGuardCondition](#) to the [DDSWaitSet](#) and then setting the `trigger_value` by means of the [DDSGuardCondition::set_trigger_value](#) operation.

See also

[DDSWaitSet](#)

7.78.2 Member Function Documentation

7.78.2.1 DDS_ReturnCode_t DDSGuardCondition::set_trigger_value (bool value)

<<cert>> Set the guard condition trigger value.

Parameters

<i>value</i>	<< <i>in</i> >> the new trigger value.
--------------	--

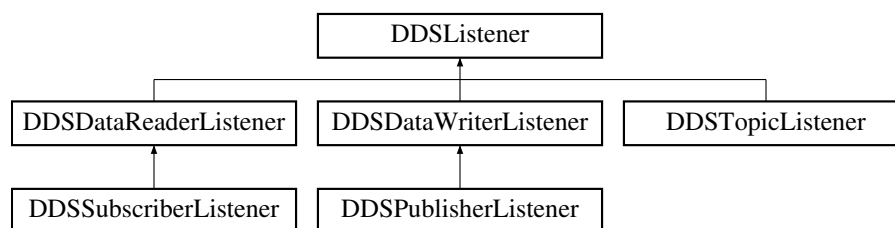
MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.79 DDSListener Class Reference

<<*interface*>> Abstract base class for all Listener interfaces.

Inheritance diagram for DDSListener:

**7.79.1 Detailed Description**

<<*interface*>> Abstract base class for all Listener interfaces.

Entity:

[DDSEntity](#)

QoS:

[QoS Policies](#)

Status:

[Status Kinds](#)

All the supported kinds of concrete [DDSListener](#) interfaces (one per concrete [DDSEntity](#) type) derive from this root and add methods whose prototype depends on the concrete Listener.

Listeners provide a way for RTI Connex Micro to asynchronously alert the application when there are relevant status changes.

Almost every application will have to implement listener interfaces.

Each dedicated listener presents a list of operations that correspond to the relevant communication status changes to which an application may respond.

The same [DDSListener](#) instance may be shared among multiple entities if you so desire. Consequently, the provided parameter contains a reference to the concerned [DDSEntity](#).

7.79.2 Access to Plain Communication Status

The general mapping between the plain communication statuses (see [Status Kinds](#)) and the listeners' operations is as follows:

- For each communication status, there is a corresponding operation whose name is `on_<communication_status>()`, which takes a parameter of type `<communication_status>` as listed in [Status Kinds](#).

- `on_<communication_status>` is available on the relevant [DDSEntity](#) as well as those that embed it, as expressed in the following figure:
- When the application attaches a listener on an entity, it must set a mask. The mask indicates to RTI Connex Micro which operations are enabled within the listener (cf. operation [DDSEntity set_listener\(\)](#)).
- When a plain communication status changes, RTI Connex Micro triggers the most specific relevant listener operation that is enabled. In case the most specific relevant listener operation corresponds to an application-installed 'nil' listener the operation will be considered handled by a NO-OP operation that does not reset the communication status.

This behavior allows the application to set a default behavior (e.g., in the listener associated with the [DDSDomainParticipant](#)) and to set dedicated behaviors only where needed.

7.79.3 Access to Read Communication Status

The two statuses related to data arrival are treated slightly differently. Since they constitute the core purpose of the Data Distribution Service, there is no need to provide a default mechanism (as is done for the plain communication statuses above).

The rule is as follows. Each time the read communication status changes:

- First, RTI Connex Micro tries to trigger the [DDSSubscriberListener::on_data_on_readers](#) with a parameter of the related [DDSSubscriber](#);
- If this does not succeed (there is no listener or the operation is not enabled), RTI Connex Micro tries to trigger [DDSDataReaderListener::on_data_available](#) on all the related [DDSDataReaderListener](#) objects, with a parameter of the related [DDSDataReader](#).

The rationale is that either the application is interested in relations among data arrivals and it must use the first option or it wants to treat each [DDSDataReader](#) independently and it may choose the second option (and then get the data by calling [FooDataReader::read_next_sample](#) or [FooDataReader::take_next_sample](#) on the related [DDSDataReader](#)).

Note that if [DDSSubscriberListener::on_data_on_readers](#) is called, RTI Connex Micro will *not* try to call [DDSDataReaderListener::on_data_available](#).

The operations that are allowed in [DDSListener](#) callbacks depend the [DDSEntity](#) to which the [DDSListener](#) is attached – or in the case of a [DDSDataWriter](#) or [DDSDataReader](#) listener, on the parent [DDSPublisher](#) or [DDSSubscriber](#).

The following operations are *not* allowed:

- Within any listener callback, deleting the entity to which the [DDSListener](#) is attached
- Within a [DDSTopic](#) listener callback, any operations on any subscribers, readers, publishers or writers

An attempt to call a disallowed method from within a callback will result in [DDS_RETCODE_ILLEGAL_OPERATION](#).

The following are *not* allowed:

- Within any listener callback, creating any entity
- Within any listener callback, deleting any entity
- Within any listener callback, enabling any entity
- Within any listener callback, setting the QoS of any entities

- Within a [DDSDataReader](#) or [DDSSubscriber](#) listener callback, invoking any operation on any other [DDSSubscriber](#) or on any [DDSDataReader](#) belonging to another [DDSSubscriber](#).
- Within a [DDSDataReader](#) or [DDSSubscriber](#) listener callback, invoking any operation on any [DDSPublisher](#) (or on any [DDSDataWriter](#) belonging to such a [DDSPublisher](#)).
- Within a [DDSDataWriter](#) of [DDSPublisher](#) listener callback, invoking any operation on another Publisher or on a [DDSDataWriter](#) belonging to another [DDSPublisher](#).
- Within a [DDSDataWriter](#) of [DDSPublisher](#) listener callback, invoking any operation on any [DDSSubscriber](#) or [DDSDataReader](#).

An attempt to call a disallowed method from within a callback will result in [DDS_RETCODE_ILLEGAL_OPERATION](#).

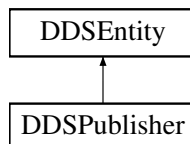
See also

[Status Kinds](#)

7.80 DDSPublisher Class Reference

[<<interface>>](#) [<<cert>>](#) A publisher is the object responsible for the actual dissemination of publications.

Inheritance diagram for DDSPublisher:



Public Member Functions

- virtual [DDSDataWriter](#) * [create_datawriter](#) ([DDSTopic](#) *topic, const [DDS_DataWriterQos](#) &qos, [DDSDataWriterListener](#) *listener, [DDS_StatusMask](#) mask)=0
[<<cert>>](#) Creates a [DDSDataWriter](#) that will be attached and belong to the [DDSPublisher](#).
- virtual [DDS_ReturnCode_t](#) [delete_datawriter](#) ([DDSDataWriter](#) *a_datawriter)=0
Deletes a [DDSDataWriter](#) that belongs to the [DDSPublisher](#).
- virtual [DDS_ReturnCode_t](#) [get_default_datawriter_qos](#) ([DDS_DataWriterQos](#) &qos)=0
Copies the default [DDS_DataWriterQos](#) values into the provided [DDS_DataWriterQos](#) instance.
- virtual [DDS_ReturnCode_t](#) [set_default_datawriter_qos](#) (const [DDS_DataWriterQos](#) &qos)=0
Sets the default [DDS_DataWriterQos](#) values for this publisher.
- virtual [DDSDataWriter](#) * [lookup_datawriter](#) (const char *topic_name)=0
[<<cert>>](#) Retrieves the [DDSDataWriter](#) for a specific [DDSTopic](#).
- virtual [DDSDomainParticipant](#) * [get_participant](#) ()=0
[<<cert>>](#) Returns the [DDSDomainParticipant](#) to which the [DDSPublisher](#) belongs.
- virtual [DDS_ReturnCode_t](#) [get_qos](#) ([DDS_PublisherQos](#) &qos)=0
Gets the publisher QoS.
- virtual [DDS_ReturnCode_t](#) [set_qos](#) (const [DDS_PublisherQos](#) &qos)=0
Sets the publisher QoS.
- virtual [DDS_ReturnCode_t](#) [delete_contained_entities](#) ()=0
Deletes all the entities that were created by means of the "create" operation on the [DDSPublisher](#).
- virtual [DDS_ReturnCode_t](#) [set_listener](#) (const [DDSPublisherListener](#) *listener, [DDS_StatusMask](#) mask)=0
Sets the publisher listener.
- virtual [DDSPublisherListener](#) * [get_listener](#) ()=0
Get the publisher listener.

7.80.1 Detailed Description

<<interface>> <<cert>> A publisher is the object responsible for the actual dissemination of publications.

QoS:

[DDS_PublisherQos](#)

Listener:

[DDSPublisherListener](#)

A publisher acts on the behalf of one or several [DDSDataWriter](#) objects that belong to it. When it is informed of a change to the data associated with one of its [DDSDataWriter](#) objects, it decides when it is appropriate to actually send the data-update message. In making this decision, it considers any extra information that goes with the data (timestamp, writer, etc.) as well as the QoS of the [DDSPublisher](#) and the [DDSDataWriter](#).

The following operations may be called even if the [DDSPublisher](#) is not enabled. Other operations will fail with the value [DDS_RETCODE_NOT_ENABLED](#) if called on a disabled [DDSPublisher](#):

- The base-class operations [DDSEntity::enable](#)
- [DDSPublisher::create_datawriter](#) , [DDSPublisher::delete_datawriter](#), [DDSPublisher::delete_contained_entities](#)

7.80.2 Member Function Documentation

7.80.2.1 **virtual [DDSDataWriter](#)* [DDSPublisher::create_datawriter](#) ([DDSTopic](#) * *topic*, const [DDS_DataWriterQos](#) & *qos*, [DDSDataWriterListener](#) * *listener*, [DDS_StatusMask](#) *mask*)** [pure virtual]

<<cert>> Creates a [DDSDataWriter](#) that will be attached and belong to the [DDSPublisher](#).

Precondition

If publisher is enabled, topic must have been enabled. Otherwise, this operation will fail and no [DDSDataWriter](#) will be created.

The given [DDSTopic](#) must have been created from the same participant as this publisher. If it was created from a different participant, this method will fail.

Parameters

<i>topic</i>	<<in>> The DDSTopic that the DDSDataWriter will be associated with. Cannot be NULL.
<i>qos</i>	<<in>> QoS to be used for creating the new DDSDataWriter . The special value DDS_DATAWRITER_QOS_DEFAULT can be used to indicate that the DDSDataWriter should be created with the default DDS_DataWriterQos set in the DDSPublisher .

Parameters

<i>listener</i>	<<in>> The listener of the DDSDataWriter .
<i>mask</i>	<<in>>. Changes of communication status to be invoked on the listener.

Returns

A [DDSDataWriter](#) of a derived class specific to the data type associated with the [DDSTopic](#) or NULL if an error occurred.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

ARINC 653 API Restriction:

This function *must* only be called before a partition enters *NORMAL* mode.

See also

[Specifying QoS on entities](#) for information on setting QoS before entity creation
[DDS_DataWriterQos](#) for rules on consistency among QoS
[DDS_DATAWRITER_QOS_DEFAULT](#)

7.80.2.2 `virtual DDS_ReturnCode_t DDSPublisher::delete_datawriter ( DDSDataWriter * a_datawriter ) [pure virtual]`

Deletes a [DDSDataWriter](#) that belongs to the [DDSPublisher](#).

The deletion of the [DDSDataWriter](#) will automatically unregister all instances.

Precondition

If the [DDSDataWriter](#) does not belong to the [DDSPublisher](#), the operation will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Postcondition

Listener installed on the [DDSDataWriter](#) will not be called after this method completes successfully.

Parameters

<i>a_datawriter</i>	<<in>> The DDSDataWriter to be deleted.
---------------------	---

Returns

One of the [Standard Return Codes](#) or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

7.80.2.3 `virtual DDS_ReturnCode_t DDSPublisher::get_default_datawriter_qos ( DDS_DataWriterQos & qos ) [pure virtual]`

Copies the default [DDS_DataWriterQos](#) values into the provided [DDS_DataWriterQos](#) instance.

The retrieved `qos` will match the set of values specified on the last successful call to [DDSPublisher::set_default_datawriter_qos](#) or else, if the call was never made, the default values from its owning [DDSDomainParticipant](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a [DDSPublisher](#) while another thread may be simultaneously calling [DDSPublisher::get_default_datawriter_qos](#) or calling [DDSPublisher::create_datawriter](#) with [DDS_DATAWRITER_QOS_DEFAULT](#) as the `qos` parameter.

Parameters

<i>qos</i>	<<inout>> DDS_DataWriterQos to be filled-up.
------------	--

Returns

One of the [Standard Return Codes](#)

See also

[DDS_DATAWRITER_QOS_DEFAULT](#)
[DDSPublisher::create_datawriter](#)

7.80.2.4 `virtual DDS_ReturnCode_t DDSPublisher::set_default_datawriter_qos ( const DDS_DataWriterQos & qos )`
 [pure virtual]

Sets the default [DDS_DataWriterQos](#) values for this publisher.

This call causes the default values inherited from the owning [DDSDomainParticipant](#) to be overridden.

This default value will be used for newly created [DDSDDataWriter](#) if [DDS_DATAWRITER_QOS_DEFAULT](#) is specified as the `qos` parameter when [DDSPublisher::create_datawriter](#) is called.

Precondition

The specified QoS policies must be consistent, or else the operation will have no effect and fail with [DDS_RETCODE_INCONSISTENT_POLICY](#)

MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a [DDSPublisher](#) while another thread may be simultaneously calling [DDSPublisher::get_default_datawriter_qos](#) or calling [DDSPublisher::create_datawriter](#) with [DDS_DATAWRITER_QOS_DEFAULT](#) as the `qos` parameter.

Parameters

<i>qos</i>	<<in>> Default qos to be set. The special value DDS_DATAREADER_QOS_DEFAULT may be passed as <code>qos</code> to indicate that the default QoS should be reset back to the initial values the factory would have used if DDSPublisher::set_default_datawriter_qos had never been called.
------------	---

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#)

7.80.2.5 `virtual DDSDDataWriter* DDSPublisher::lookup_datawriter ( const char * topic_name )` [pure virtual]

<<cert>> Retrieves the [DDSDDataWriter](#) for a specific [DDSTopic](#).

This returned [DDSDDataWriter](#) is either enabled or disabled.

Parameters

<i>topic_name</i>	<<in>> Name of the DDSTopic associated with the DDSDDataWriter that is to be looked up. Cannot be NULL.
-------------------	---

Returns

A [DDSDDataWriter](#) that belongs to the [DDSPublisher](#) attached to the [DDSTopic](#) with `topic_name`. If no such [DDSDDataWriter](#) exists, this operation returns NULL.

If more than one [DDSDDataWriter](#) is attached to the [DDSPublisher](#) with the same `topic_name`, then this operation may return any one of them.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.80.2.6 `virtual DDSDomainParticipant* DDSPublisher::get_participant ( ) [pure virtual]`

<<cert>> Returns the [DDSDomainParticipant](#) to which the [DDSPublisher](#) belongs.

Returns

the [DDSDomainParticipant](#) to which the [DDSPublisher](#) belongs.

MT Safety:

. This operation is thread safe on a valid [DDSDataWriter](#).

7.80.2.7 `virtual DDS_ReturnCode_t DDSPublisher::get_qos ( DDS_PublisherQos & qos ) [pure virtual]`

Gets the publisher QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<i>qos</i>	<<in>> DDS_PublisherQos to be filled in.
------------	--

Returns

One of the [Standard Return Codes](#)

See also

[get_qos \(abstract\)](#)

7.80.2.8 `virtual DDS_ReturnCode_t DDSPublisher::set_qos ( const DDS_PublisherQos & qos ) [pure virtual]`

Sets the publisher QoS.

This operation modifies the QoS of the [DDSPublisher](#).

The [DDS_PublisherQos::entity_factory](#) can be changed. The other policies are immutable.

Parameters

<i>qos</i>	<<in>> DDS_PublisherQos to be set to. Policies must be consistent. Policies cannot be changed after DDSPublisher is enabled. The special value DDS_PUBLISHER_QOS_DEFAULT can be used to indicate that the QoS of the DDSPublisher should be changed to match the current default DDS_PublisherQos set in the DDSDomainParticipant .
------------	---

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_IMMUTABLE_POLICY](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#).

See also

[DDS_PublisherQos](#) for rules on consistency among QoS
[set_qos \(abstract\)](#)

7.80.2.9 `virtual DDS_ReturnCode_t DDSPublisher::delete_contained_entities ( ) [pure virtual]`

Deletes all the entities that were created by means of the "create" operation on the [DDSPublisher](#).

Deletes all contained [DDSDDataWriter](#) objects. Once [DDSPublisher::delete_contained_entities](#) completes successfully, the application may delete the [DDSPublisher](#), knowing that it has no contained [DDSDDataWriter](#) objects.

The operation will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#) if any of the contained entities is in a state where it cannot be deleted.

Returns

One of the [Standard Return Codes](#) or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

7.80.2.10 `virtual DDS_ReturnCode_t DDSPublisher::set_listener ( const DDSPublisherListener * listener, DDS_StatusMask mask ) [pure virtual]`

Sets the publisher listener.

Parameters

<i>listener</i>	<<in>> DDSPublisherListener to set to.
<i>mask</i>	<<in>> DDS_StatusMask associated with the DDSPublisherListener .

Returns

One of the [Standard Return Codes](#)

See also

[set_listener](#) (abstract)

7.80.2.11 `virtual DDSPublisherListener* DDSPublisher::get_listener ( ) [pure virtual]`

Get the publisher listener.

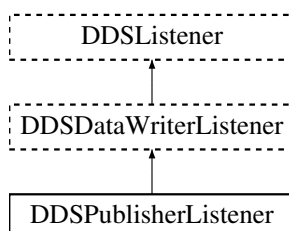
Returns

[DDSPublisherListener](#) of the [DDSPublisher](#).

7.81 DDSPublisherListener Class Reference

<<interface>> <<cert>> [DDSListener](#) for [DDSPublisher](#) status.

Inheritance diagram for [DDSPublisherListener](#):



Additional Inherited Members

7.81.1 Detailed Description

<<interface>> <<cert>> [DDSListener](#) for [DDSPublisher](#) status.

Entity:

[DDSPublisher](#)

Status:

[DDS_LIVELINESS_LOST_STATUS](#), [DDS_LivelinessLostStatus](#);
[DDS_OFFERED_DEADLINE_MISSED_STATUS](#), [DDS_OfferedDeadlineMissedStatus](#);
[DDS_OFFERED_INCOMPATIBLE_QOS_STATUS](#), [DDS_OfferedIncompatibleQosStatus](#);
[DDS_PUBLICATION_MATCHED_STATUS](#), [DDS_PublicationMatchedStatus](#);

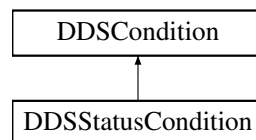
See also

[DDSListener](#)
[Status Kinds](#)

7.82 DDSStatusCondition Class Reference

<<interface>> *<<cert>>* A specific [DDSCondition](#) that is associated with each [DDSEntity](#).

Inheritance diagram for DDSStatusCondition:



Public Member Functions

- [DDS_StatusMask get_enabled_statuses \(\)](#)
<<cert>> Get the list of statuses enabled on an [DDSEntity](#).
- [DDS_ReturnCode_t set_enabled_statuses \(DDS_StatusMask mask\)](#)
<<cert>> This operation defines the list of communication statuses that determine the *trigger_value* of the [DDSStatusCondition](#).

Friends

- class [DDSEntity](#)

7.82.1 Detailed Description

<<interface>> *<<cert>>* A specific [DDSCondition](#) that is associated with each [DDSEntity](#).

The *trigger_value* of the [DDSStatusCondition](#) depends on the communication status of that entity (e.g., arrival of data, loss of information, etc.), 'filtered' by the set of *enabled_statuses* on the [DDSStatusCondition](#).

See also

[Status Kinds](#)
[DDSWaitSet](#), [DDSCondition](#)
[DDSListener](#)

7.82.2 Member Function Documentation

7.82.2.1 DDS_StatusMask DDSStatusCondition::get_enabled_statuses ()

<<cert>> Get the list of statuses enabled on an [DDSEntity](#).

Returns

list of enabled statuses.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.82.2.2 DDS_ReturnCode_t DDSStatusCondition::set_enabled_statuses (DDS_StatusMask mask)

<<cert>> This operation defines the list of communication statuses that determine the `trigger_value` of the [DDSStatusCondition](#).

This operation may change the `trigger_value` of the [DDSStatusCondition](#).

[DDSWaitSet](#) objects' behavior depends on the changes of the `trigger_value` of their attached conditions. Therefore, any [DDSWaitSet](#) to which the [DDSStatusCondition](#) is attached is potentially affected by this operation.

If this function is not invoked, the default list of enabled statuses includes all the statuses.

Parameters

<i>mask</i>	<<in>> the list of enables statuses (see Status Kinds)
-------------	---

Returns

One of the [Standard Return Codes](#)

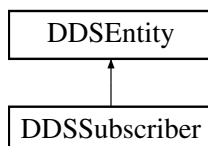
MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.83 DDSSubscriber Class Reference

<<interface>> A subscriber is the object responsible for actually receiving data from a subscription.

Inheritance diagram for DDSSubscriber:



Public Member Functions

- virtual [DDSDDataReader](#) * [create_datareader](#) ([DDSTopicDescription](#) *topic, const [DDS_DataReaderQos](#) &qos, [DDSDDataReaderListener](#) *listener, [DDS_StatusMask](#) mask)=0
 <<cert>> Creates a [DDSDDataReader](#) that will be attached and belong to the [DDSSubscriber](#).
- virtual [DDS_ReturnCode_t](#) [delete_datareader](#) ([DDSDDataReader](#) *a_datareader)=0
 Deletes a [DDSDDataReader](#) that belongs to the [DDSSubscriber](#).

- virtual `DDS_ReturnCode_t get_default_datareader_qos (DDS_DataReaderQos &qos)=0`
Copies the default `DDS_DataReaderQos` values into the provided `DDS_DataReaderQos` instance.
- virtual `DDS_ReturnCode_t set_default_datareader_qos (const DDS_DataReaderQos &qos)=0`
Sets the default `DDS_DataReaderQos` values for this subscriber.
- virtual `DDS_ReturnCode_t delete_contained_entities ()=0`
Deletes all the entities that were created by means of the "create" operation on the `DDSSubscriber`.
- virtual `DDSDataReader * lookup_datareader (const char *topic_name)=0`
<<cert>> Retrieves an existing `DDSDataReader`.
- virtual `DDSDomainParticipant * get_participant ()=0`
<<cert>> Returns the `DDSDomainParticipant` to which the `DDSSubscriber` belongs.
- virtual `DDS_ReturnCode_t set_qos (const DDS_SubscriberQos &qos)=0`
Sets the subscriber QoS.
- virtual `DDS_ReturnCode_t get_qos (DDS_SubscriberQos &qos)=0`
Gets the subscriber QoS.
- virtual `DDS_ReturnCode_t set_listener (const DDSSubscriberListener *listener, DDS_StatusMask mask)=0`
Sets the subscriber listener.
- virtual `DDSSubscriberListener * get_listener ()=0`
Get the subscriber listener.

7.83.1 Detailed Description

<<interface>> A subscriber is the object responsible for actually receiving data from a subscription.

QoS:

`DDS_SubscriberQos`

Status:

`DDS_DATA_ON_READERS_STATUS`

Listener:

`DDSSubscriberListener`

A subscriber acts on the behalf of one or several `DDSDataReader` objects that are related to it. When it receives data (from the other parts of the system), it builds the list of concerned `DDSDataReader` objects and then indicates to the application that data is available through its listener.

The application can access the list of concerned `DDSDataReader` objects through the operation `get_datareaders()` and then access the data available through operations on the `DDSDataReader`.

The following operations may be called even if the `DDSSubscriber` is not enabled. Other operations will the value `DDS_RETCODE_NOT_ENABLED` if called on a disabled `DDSSubscriber`:

- The base-class operations `DDSEntity::enable`
- `DDSSubscriber::create_datareader`., `DDSSubscriber::delete_datareader`, `DDSSubscriber::delete_contained_entities`.

All operations except for the base-class operations `enable()` and `create_datareader()` may fail with `DDS_RETCODE_NOT_ENABLED`.

7.83.2 Member Function Documentation

7.83.2.1 `virtual DDSDataReader* DDSSubscriber::create_datareader ( DDSTopicDescription * topic, const DDS_DataReaderQos & qos, DDSDataReaderListener * listener, DDS_StatusMask mask ) [pure virtual]`

<<cert>> Creates a [DDSDataReader](#) that will be attached and belong to the [DDSSubscriber](#).

Precondition

If subscriber is enabled, the topic must be enabled. If it is not, this operation will fail and no [DDSDataReader](#) will be created.

The given [DDSTopicDescription](#) must have been created from the same participant as this subscriber. If it was created from a different participant, this method will return NULL.

Parameters

<i>topic</i>	<<in>> The DDSTopicDescription that the DDSDataReader will be associated with. Cannot be NULL.
<i>qos</i>	<<in>> The qos of the DDSDataReader . The special value DDS_DATAREADER_QOS_DEFAULT can be used to indicate that the DDSDataReader should be created with the default DDS_DataReaderQos set in the DDSSubscriber .
<i>listener</i>	<<in>> The listener of the DDSDataReader .
<i>mask</i>	<<in>>. Changes of communication status to be invoked on the listener.

Returns

A [DDSDataReader](#) of a derived class specific to the data-type associated with the [DDSTopic](#) or NULL if an error occurred.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

ARINC 653 API Restriction:

This function *must* only be called before a partition enters *NORMAL* mode.

See also

[Specifying QoS on entities](#) for information on setting QoS before entity creation

[DDS_DataReaderQos](#) for rules on consistency among QoS

7.83.2.2 `virtual DDS_ReturnCode_t DDSSubscriber::delete_datareader ( DDSDataReader * a_datareader ) [pure virtual]`

Deletes a [DDSDataReader](#) that belongs to the [DDSSubscriber](#).

Precondition

If the [DDSDataReader](#) does not belong to the [DDSSubscriber](#), or if there are any existing objects that are attached to the [DDSDataReader](#), or if there are outstanding loans on samples (as a result of a call to `read()`, `take()`, or one of the variants thereof), the operation fails with the error [DDS_RETCODE_PRECONDITION_NOT_MET](#).

Postcondition

Listener installed on the [DDSDataReader](#) will not be called after this method completes successfully.

Parameters

<i>a_datareader</i>	<<in>> The DDSDataReader to be deleted.
---------------------	---

Returns

One of the [Standard Return Codes](#) or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

7.83.2.3 `virtual DDS_ReturnCode_t DDSSubscriber::get_default_datareader_qos ( DDS_DataReaderQos & qos )`
 [pure virtual]

Copies the default [DDS_DataReaderQos](#) values into the provided [DDS_DataReaderQos](#) instance.

The retrieved `qos` will match the set of values specified on the last successful call to [DDSSubscriber::set_default_datareader_qos](#), or else, if the call was never made, the default values from its owning [DDSDomainParticipant](#).

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a subscriber while another thread may be simultaneously calling [DDSSubscriber::get_default_datareader_qos](#) or calling [DDSSubscriber::create_datareader](#) with [DDS_DATAREADER_QOS_DEFAULT](#) as the `qos` parameter.

Parameters

<i>qos</i>	<<inout>> DDS_DataReaderQos to be filled-up.
------------	--

Returns

One of the [Standard Return Codes](#)

See also

[DDS_DATAREADER_QOS_DEFAULT](#)
[DDSSubscriber::create_datareader](#)

7.83.2.4 `virtual DDS_ReturnCode_t DDSSubscriber::set_default_datareader_qos ( const DDS_DataReaderQos & qos )`
 [pure virtual]

Sets the default [DDS_DataReaderQos](#) values for this subscriber.

This call causes the default values inherited from the owning [DDSDomainParticipant](#) to be overridden.

This default value will be used for newly created [DDSDataReader](#) if [DDS_DATAREADER_QOS_DEFAULT](#) is specified as the `qos` parameter when [DDSSubscriber::create_datareader](#) is called.

Precondition

The specified QoS policies must be consistent, or else the operation will have no effect and fail with [DDS_RETCODE_INCONSISTENT_POLICY](#)

MT Safety:

UNSAFE. It is not safe to retrieve the default QoS value from a subscriber while another thread may be simultaneously calling [DDSSubscriber::get_default_datareader_qos](#) or calling [DDSSubscriber::create_datareader](#) with [DDS_DATAREADER_QOS_DEFAULT](#) as the `qos` parameter.

Parameters

<i>qos</i>	<<in>> The default DDS_DataReaderQos to be set to. The special value DDS_DATAREADER_QOS_DEFAULT may be passed as <i>qos</i> to indicate that the default QoS should be reset back to the initial values the factory would have used if DDSSubscriber::set_default_datareader_qos had never been called.
------------	---

Returns

One of the [Standard Return Codes](#), or or [DDS_RETCODE_INCONSISTENT_POLICY](#)

7.83.2.5 `virtual DDS_ReturnCode_t DDSSubscriber::delete_contained_entities ( ) [pure virtual]`

Deletes all the entities that were created by means of the "create" operation on the [DDSSubscriber](#).

Deletes all contained [DDSDDataReader](#) objects. This pattern is applied recursively. In this manner, the operation [DDSSubscriber::delete_contained_entities](#) on the [DDSSubscriber](#) will end up deleting all the entities recursively contained in the [DDSSubscriber](#), that is also the objects belonging to the contained [DDSDDataReader](#).

The operation will fail with [DDS_RETCODE_PRECONDITION_NOT_MET](#) if any of the contained entities is in a state where it cannot be deleted. This will occur, for example, if a contained [DDSDDataReader](#) cannot be deleted because the application has called a [FooDataReader::read](#) or [FooDataReader::take](#) operation and has not called the corresponding [FooDataReader::return_loan](#) operation to return the loaned samples.

Once [DDSSubscriber::delete_contained_entities](#) completes successfully, the application may delete the [DDSSubscriber](#), knowing that it has no contained [DDSDDataReader](#) objects.

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#)

7.83.2.6 `virtual DDSDDataReader* DDSSubscriber::lookup_datareader ( const char * topic_name ) [pure virtual]`

<<cert>> Retrieves an existing [DDSDDataReader](#).

Use this operation on the built-in [DDSSubscriber](#) ([Discovery](#)) to access the built-in [DDSDDataReader](#) entities for the built-in topics.

The built-in [DDSDDataReader](#) is created when this operation is called on a built-in topic for the first time. The built-in [DDSDDataReader](#) is deleted automatically when the [DDSDomainParticipant](#) is deleted.

To ensure that builtin [DDSDDataReader](#) entities receive all the discovery traffic, it is suggested that you lookup the builtin [DDSDDataReader](#) before the [DDSDomainParticipant](#) is enabled. Looking up builtin [DDSDDataReader](#) may implicitly register builtin transports due to creation of [DDSDDataReader](#). Therefore, if you are want to modify builtin transport properties, do so *before* using this operation.

Therefore the suggested sequence when looking up builtin DataReaders is:

- Create a disabled [DDSDomainParticipant](#).
- (optional) Modify builtin transport properties
- Call [DDS_DomainParticipant_get_builtin_subscriber\(\)](#).
- Call [lookup_datareader\(\)](#).
- Call [enable\(\)](#) on the [DomainParticipant](#).

Parameters

<i>topic_name</i>	<<in>> Name of the DDSTopicDescription that the retrieved DDSDDataReader is attached to. Cannot be NULL.
-------------------	--

Returns

A [DDSDDataReader](#) that belongs to the [DDSSubscriber](#) attached to the [DDSTopicDescription](#) with `topic_name`. If no such [DDSDDataReader](#) exists, this operation returns `NULL`.

The returned [DDSDDataReader](#) may be enabled or disabled.

If more than one [DDSDDataReader](#) is attached to the [DDSSubscriber](#), this operation may return any one of them.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.83.2.7 `virtual DDSDomainParticipant* DDSSubscriber::get_participant ( ) [pure virtual]`

`<<cert>>` Returns the [DDSDomainParticipant](#) to which the [DDSSubscriber](#) belongs.

Returns

the [DDSDomainParticipant](#) to which the [DDSSubscriber](#) belongs.

MT Safety:

This operation is thread safe.

7.83.2.8 `virtual DDS_ReturnCode_t DDSSubscriber::set_qos ( const DDS_SubscriberQos & qos ) [pure virtual]`

Sets the subscriber QoS.

This operation modifies the QoS of the [DDSSubscriber](#).

The [DDS_SubscriberQos::entity_factory](#) can be changed. The other policies are immutable.

Parameters

<i>qos</i>	<code><<in>></code> DDS_SubscriberQos to be set to. Policies must be consistent. Policies cannot be changed after DDSSubscriber is enabled. The special value DDS_SUBSCRIBER_QOS_D-EFAULT can be used to indicate that the QoS of the DDSSubscriber should be changed to match the current default DDS_SubscriberQos set in the DDSDomainParticipant .
------------	--

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_IMMUTABLE_POLICY](#), or [DDS_RETCODE_INCONSISTENT_POLICY](#).

See also

[DDS_SubscriberQos](#) for rules on consistency among QoS
[set_qos](#) (abstract)

7.83.2.9 `virtual DDS_ReturnCode_t DDSSubscriber::get_qos ( DDS_SubscriberQos & qos ) [pure virtual]`

Gets the subscriber QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<i>qos</i>	<<in>> DDS_SubscriberQos to be filled in.
------------	---

Returns

One of the [Standard Return Codes](#)

See also

[get_qos \(abstract\)](#)

7.83.2.10 `virtual DDS_ReturnCode_t DDSSubscriber::set_listener ( const DDSSubscriberListener * listener, DDS_StatusMask mask ) [pure virtual]`

Sets the subscriber listener.

Parameters

<i>listener</i>	<<in>> DDSSubscriberListener to set to.
<i>mask</i>	<<in>> DDS_StatusMask associated with the DDSSubscriberListener .

Returns

One of the [Standard Return Codes](#)

See also

[set_listener \(abstract\)](#)

7.83.2.11 `virtual DDSSubscriberListener* DDSSubscriber::get_listener ( ) [pure virtual]`

Get the subscriber listener.

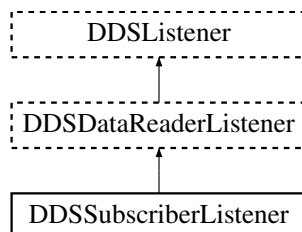
Returns

[DDSSubscriberListener](#) of the [DDSSubscriber](#).

7.84 DDSSubscriberListener Class Reference

<<interface>> <<cert>> [DDSListener](#) for status about a subscriber.

Inheritance diagram for [DDSSubscriberListener](#):



Public Member Functions

- virtual void [on_data_on_readers](#) ([DDSSubscriber](#) *)
Handles the [DDS_DATA_ON_READERS_STATUS](#) communication status.

7.84.1 Detailed Description

<<interface>> <<cert>> [DDSTopicListener](#) for status about a subscriber.

Entity:

[DDSSubscriber](#)

Status:

[DDS_DATA_AVAILABLE_STATUS](#);
[DDS_DATA_ON_READERS_STATUS](#);
[DDS_LIVELINESS_CHANGED_STATUS](#), [DDS_LivelinessChangedStatus](#);
[DDS_REQUESTED_DEADLINE_MISSED_STATUS](#), [DDS_RequestedDeadlineMissedStatus](#);
[DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS](#), [DDS_RequestedIncompatibleQosStatus](#);
[DDS_SAMPLE_LOST_STATUS](#), [DDS_SampleLostStatus](#);
[DDS_SAMPLE_REJECTED_STATUS](#), [DDS_SampleRejectedStatus](#);
[DDS_SUBSCRIPTION_MATCHED_STATUS](#), [DDS_SubscriptionMatchedStatus](#);

See also

[DDSTopicListener](#)
[Status Kinds](#)

7.84.2 Member Function Documentation

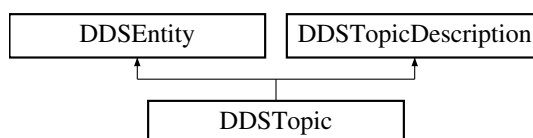
7.84.2.1 `virtual void DDSSubscriberListener::on_data_on_readers ( DDSSubscriber * ) [inline], [virtual]`

Handles the [DDS_DATA_ON_READERS_STATUS](#) communication status.

7.85 DDSTopic Class Reference

<<interface>> The most basic description of the data to be published and subscribed.

Inheritance diagram for DDSTopic:



Public Member Functions

- virtual [DDSTopicDescription](#) * [as_topicdescription](#) ()=0
 <<cert>> Access a [DDSTopic](#)'s [DDSTopicDescription](#) supertype instance.
- virtual [DDSEntity](#) * [as_entity](#) ()=0
 <<cert>> Access a [DDSTopic](#)'s [DDSEntity](#) supertype instance.
- virtual [DDS_ReturnCode_t](#) [set_qos](#) (const [DDS_TopicQos](#) &qos)=0
 Set the topic QoS.
- virtual [DDS_ReturnCode_t](#) [get_qos](#) ([DDS_TopicQos](#) &qos)=0
 Get the topic QoS.
- virtual [DDS_ReturnCode_t](#) [get_inconsistent_topic_status](#) ([DDS_InconsistentTopicStatus](#) &status)=0
 <<cert>> Allows the application to retrieve the [DDS_INCONSISTENT_TOPIC_STATUS](#) status of a [DDSTopic](#).
- virtual [DDS_ReturnCode_t](#) [set_listener](#) (const [DDSTopicListener](#) *listener, [DDS_StatusMask](#) mask)=0

Set the topic listener.

- virtual `DDSTopicListener * get_listener ()=0`

Get the topic listener.

- virtual const char * `get_type_name ()=0`

<<cert>> Get the associated `type_name`.

- virtual const char * `get_name ()=0`

<<cert>> Get the name used to create this `DDSTopicDescription`.

- virtual `DDSDomainParticipant * get_participant ()=0`

<<cert>> Get the `DDSDomainParticipant` to which the `DDSTopicDescription` belongs.

Static Public Member Functions

- static `DDSTopic * narrow (DDSTopicDescription *topic_description)`

<<cert>> Narrow the given `DDSTopicDescription` pointer to a `DDSTopic` pointer.

7.85.1 Detailed Description

<<interface>> The most basic description of the data to be published and subscribed.

QoS:

`DDS_TopicQos`

Status:

`DDS_INCONSISTENT_TOPIC_STATUS, DDS_InconsistentTopicStatus`

Listener:

`DDSTopicListener`

A `DDSTopic` is identified by its name, which must be unique in the whole domain. In addition (by virtue of extending `DDSTopicDescription`) it fully specifies the type of the data that can be communicated when publishing or subscribing to the `DDSTopic`.

`DDSTopic` is the only `DDSTopicDescription` that can be used for publications and therefore associated with a `DDS-DataWriter`.

The following operations may be called even if the `DDSTopic` is not enabled. Other operations will fail with the value `DDS_RETCODE_NOT_ENABLED` if called on a disabled `DDSTopic`:

- `get_inconsistent_topic_status()`

7.85.2 Member Function Documentation

7.85.2.1 `static DDSTopic* DDSTopic::narrow ( DDSTopicDescription * topic_description ) [static]`

<<cert>> Narrow the given `DDSTopicDescription` pointer to a `DDSTopic` pointer.

Returns

`DDSTopic` if this `DDSTopicDescription` is a `DDSTopic`. Otherwise, return `NULL`.

7.85.2.2 `virtual DDSTopicDescription* DDSTopic::as_topicdescription ( ) [pure virtual]`

<<cert>> Access a `DDSTopic`'s `DDSTopicDescription` supertype instance.

Returns

`DDSTopic`'s supertype `DDSTopicDescription` instance

7.85.2.3 `virtual DDSEntity* DDSTopic::as_entity ( ) [pure virtual]`

<<cert>> Access a [DDSTopic](#)'s [DDSEntity](#) supertype instance.

Returns

[DDSTopic](#)'s supertype [DDSEntity](#) instance

7.85.2.4 `virtual DDS_ReturnCode_t DDSTopic::set_qos ( const DDS_TopicQos & qos ) [pure virtual]`

Set the topic QoS.

Parameters

<i>qos</i>	<<in>> Set of policies to be applied to DDSTopic .
------------	--

Policies must be consistent. Policies cannot be changed after [DDSTopic](#) is enabled. The special value [DDS_TOPIC_QOS_DEFAULT](#) can be used to indicate that the QoS of the [DDSTopic](#) should be changed to match the current default [DDS_TopicQos](#) set in the [DDSDomainParticipant](#).

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_IMMUTABLE_POLICY](#) if immutable policy is changed, or [DDS_RETCODE_INCONSISTENT_POLICY](#) if policies are inconsistent

See also

[DDS_TopicQos](#) for rules on consistency among QoS
[set_qos](#) (abstract)

7.85.2.5 `virtual DDS_ReturnCode_t DDSTopic::get_qos ( DDS_TopicQos & qos ) [pure virtual]`

Get the topic QoS.

This method may potentially allocate memory depending on the sequences contained in some QoS policies.

Parameters

<i>qos</i>	<<inout>> QoS to be filled up.
------------	--------------------------------

Returns

One of the [Standard Return Codes](#)

See also

[get_qos](#) (abstract)

7.85.2.6 `virtual DDS_ReturnCode_t DDSTopic::get_inconsistent_topic_status ( DDS_InconsistentTopicStatus & status ) [pure virtual]`

<<cert>> Allows the application to retrieve the [DDS_INCONSISTENT_TOPIC_STATUS](#) status of a [DDSTopic](#).

Retrieve the current [DDS_InconsistentTopicStatus](#)

Parameters

<i>status</i>	<<inout>> Status to be retrieved.
---------------	-----------------------------------

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDS_InconsistentTopicStatus](#)

7.85.2.7 `virtual DDS_ReturnCode_t DDSTopic::set_listener ( const DDSTopicListener * listener, DDS_StatusMask mask ) [pure virtual]`

Set the topic listener.

Parameters

<i>listener</i>	<<in>> Listener to be installed on entity.
<i>mask</i>	<<in>> Changes of communication status to be invoked on the listener.

Returns

One of the [Standard Return Codes](#)

7.85.2.8 `virtual DDSTopicListener* DDSTopic::get_listener ( ) [pure virtual]`

Get the topic listener.

Returns

Existing listener attached to the [DDSTopic](#).

7.85.2.9 `virtual const char* DDSTopic::get_type_name ( ) [pure virtual]`

<<cert>> Get the associated `type_name`.

The type name defines a locally unique type for the publication or the subscription.

The `type_name` corresponds to a unique string used to register a type via the [DDSDomainParticipant::register_type](#) method.

Thus, the `type_name` implies an association with a corresponding type and this [DDSTopicDescription](#).

Returns

the type name. The returned type name is valid until the [DDSTopicDescription](#) is deleted.

MT Safety:

This operation is thread safe.

Postcondition

The result is NULL if self is NULL, otherwise result is non-NULL.

Implements [DDSTopicDescription](#).

7.85.2.10 `virtual const char* DDSTopic::get_name ( ) [pure virtual]`

<<cert>> Get the name used to create this [DDSTopicDescription](#).

Returns

the name used to create this [DDSTopicDescription](#). The returned topic name is valid until the [DDSTopicDescription](#) is deleted.

MT Safety:

This operation is thread safe.

Postcondition

The result is NULL if self is NULL, otherwise result is non-NULL.

Implements [DDSTopicDescription](#).

7.85.2.11 `virtual DDSDomainParticipant* DDSTopic::get_participant ( ) [pure virtual]`

<<cert>> Get the [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.

Returns

The [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.

MT Safety:

This operation is thread safe.

Postcondition

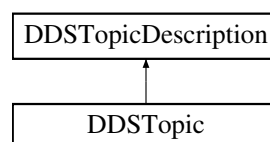
The result is NULL if self is NULL, otherwise result is non-NULL.

Implements [DDSTopicDescription](#).

7.86 DDSTopicDescription Class Reference

<<interface>> Base class for [DDSTopic](#).

Inheritance diagram for [DDSTopicDescription](#):



Public Member Functions

- `virtual const char * get_type_name ()=0`
<<cert>> Get the associated `type_name`.
- `virtual const char * get_name ()=0`
<<cert>> Get the name used to create this [DDSTopicDescription](#).
- `virtual DDSDomainParticipant * get_participant ()=0`
<<cert>> Get the [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.

7.86.1 Detailed Description

<<*interface*>> Base class for [DDSTopic](#).

[DDSTopicDescription](#) represents the fact that both publications and subscriptions are tied to a single data-type. Its attribute `type_name` defines a unique resulting type for the publication or the subscription and therefore creates an implicit association with a type.

[DDSTopicDescription](#) has also a `name` that allows it to be retrieved locally.

7.86.2 Member Function Documentation

7.86.2.1 `virtual const char* DDSTopicDescription::get_type_name ( ) [pure virtual]`

<<*cert*>> Get the associated `type_name`.

The type name defines a locally unique type for the publication or the subscription.

The `type_name` corresponds to a unique string used to register a type via the [DDSDomainParticipant::register_type](#) method.

Thus, the `type_name` implies an association with a corresponding type and this [DDSTopicDescription](#).

Returns

the type name. The returned type name is valid until the [DDSTopicDescription](#) is deleted.

MT Safety:

This operation is thread safe.

Postcondition

The result is NULL if self is NULL, otherwise result is non-NULL.

Implemented in [DDSTopic](#).

7.86.2.2 `virtual const char* DDSTopicDescription::get_name ( ) [pure virtual]`

<<*cert*>> Get the name used to create this [DDSTopicDescription](#).

Returns

the name used to create this [DDSTopicDescription](#). The returned topic name is valid until the [DDSTopicDescription](#) is deleted.

MT Safety:

This operation is thread safe.

Postcondition

The result is NULL if self is NULL, otherwise result is non-NULL.

Implemented in [DDSTopic](#).

7.86.2.3 `virtual DDSDomainParticipant* DDSTopicDescription::get_participant ( ) [pure virtual]`

<<*cert*>> Get the [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.

Returns

The [DDSDomainParticipant](#) to which the [DDSTopicDescription](#) belongs.

MT Safety:

This operation is thread safe.

Postcondition

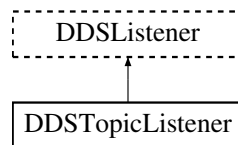
The result is NULL if self is NULL, otherwise result is non-NULL.

Implemented in [DDSTopic](#).

7.87 DDSTopicListener Class Reference

<<*interface*>> [DDSListener](#) for [DDSTopic](#) entities.

Inheritance diagram for DDSTopicListener:

**Public Member Functions**

- virtual void [on_inconsistent_topic](#) ([DDSTopic](#) *topic, const [DDS_InconsistentTopicStatus](#) &status)
Handle the [DDS_INCONSISTENT_TOPIC_STATUS](#) status.

7.87.1 Detailed Description

<<*interface*>> [DDSListener](#) for [DDSTopic](#) entities.

Entity:

[DDSTopic](#)

Status:

[DDS_INCONSISTENT_TOPIC_STATUS](#), [DDS_InconsistentTopicStatus](#)

This is the interface that can be implemented by an application-provided class and then registered with the [DDS-Topic](#) such that the application can be notified by RTI Connex Micro of relevant status changes.

See also

[Status Kinds](#)
[DDSListener](#)

7.87.2 Member Function Documentation

7.87.2.1 virtual void [DDSTopicListener::on_inconsistent_topic](#) ([DDSTopic](#) * topic, const [DDS_InconsistentTopicStatus](#) & status) [inline], [virtual]

Handle the [DDS_INCONSISTENT_TOPIC_STATUS](#) status.

This callback is called when a remote [DDSTopic](#) is discovered but is inconsistent with the locally created [DDSTopic](#) of the same topic name.

Parameters

<i>topic</i>	<<out>> Locally created DDSTopic that triggers the listener callback
<i>status</i>	<<out>> Current inconsistent status of locally created DDSTopic

7.88 DDSWaitSet Class Reference

<<interface>> <<cert>> Allows an application to wait until one or more of the attached [DDSCondition](#) objects has a `trigger_value` of [DDS_BOOLEAN_TRUE](#) or else until the timeout expires.

Public Member Functions

- [DDS_ReturnCode_t wait](#) ([DDSConditionSeq](#) &active_conditions, const [DDS_Duration_t](#) &timeout)
<<cert>> Allows an application thread to wait for the occurrence of certain conditions.
- [DDS_ReturnCode_t attach_condition](#) ([DDSCondition](#) *cond)
<<cert>> Attaches a [DDSCondition](#) to the [DDSWaitSet](#).
- [DDS_ReturnCode_t detach_condition](#) ([DDSCondition](#) *cond)
Detaches a [DDSCondition](#) from the [DDSWaitSet](#).
- [DDS_ReturnCode_t get_conditions](#) ([DDSConditionSeq](#) &attached_conditions)
<<cert>> Retrieves the list of attached [DDSCondition](#) (s).

7.88.1 Detailed Description

<<interface>> <<cert>> Allows an application to wait until one or more of the attached [DDSCondition](#) objects has a `trigger_value` of [DDS_BOOLEAN_TRUE](#) or else until the timeout expires.

7.88.2 Usage

[DDSCondition](#) (s) (in conjunction with wait-sets) provide an alternative mechanism to allow the middleware to communicate communication status changes (including arrival of data) to the application.

This mechanism is wait-based. Its general use pattern is as follows:

- The application indicates which relevant information it wants to get by creating [DDSCondition](#) objects ([DDS-StatusCondition](#)) and attaching them to a [DDSWaitSet](#).
- It then waits on that [DDSWaitSet](#) until the `trigger_value` of one or several [DDSCondition](#) objects become [DDS_BOOLEAN_TRUE](#).
- It then uses the result of the wait (i.e., `active_conditions`, the list of [DDSCondition](#) objects with `trigger_value == DDS_BOOLEAN_TRUE`) to actually get the information:
 - by calling [DDSEntity::get_status_changes](#) and then `get_<communication_status>()` on the relevant [DDSEntity](#), if the condition is a [DDSStatusCondition](#) and the status changes, refer to plain communication status;
 - by calling [DDSEntity::get_status_changes](#) and then [FooDataReader::read\(\)](#) or [FooDataReader::take](#) on the relevant [DDSDataReader](#), if the condition is a [DDSStatusCondition](#) and the status changes refers to [DDS_DATA_AVAILABLE_STATUS](#);

Usually the first step is done in an initialization phase, while the others are put in the application main loop.

As there is no extra information passed from the middleware to the application when a wait returns (only the list of triggered [DDSCondition](#) objects), [DDSCondition](#) objects are meant to embed all that is needed to react properly when enabled. In particular, [DDSEntity](#)-related conditions are related to exactly one [DDSEntity](#) and cannot be shared.

The result of a `DDSWaitSet::wait` operation depends on the state of the `DDSWaitSet`, which in turn depends on whether at least one attached `DDSCondition` has a `trigger_value` of `DDS_BOOLEAN_TRUE`. If the wait operation is called on `DDSWaitSet` with state `BLOCKED`, it will block the calling thread. If wait is called on a `DDSWaitSet` with state `UNBLOCKED`, it will return immediately. In addition, when the `DDSWaitSet` transitions from `BLOCKED` to `UNBLOCKED` it wakes up any threads that had called wait on it.

A key aspect of the `DDSCondition/DDSWaitSet` mechanism is the setting of the `trigger_value` of each `DDSCondition`.

7.88.3 Trigger State of a `DDSCondition`

The `trigger_value` of a `DDSCondition` is the boolean OR of the `ChangedStatusFlag` of all the communication statuses (see [Status Kinds](#)) to which it is sensitive. That is, `trigger_value == DDS_BOOLEAN_FALSE` only if all the values of the `ChangedStatusFlags` are `DDS_BOOLEAN_FALSE`.

The sensitivity of the `DDSCondition` to a particular communication status is controlled by the list of `enabled_statuses` set on the condition by means of the `DDSCondition::set_enabled_statuses` operation.

7.88.4 Trigger State of a `DDSGuardCondition`

The `trigger_value` of a `DDSGuardCondition` is completely controlled by the application via the operation `DDSGuardCondition::set_trigger_value`.

See also

[Status Kinds](#)
[DDSCondition](#), [DDSGuardCondition](#)
[DDSListener](#)

7.88.5 Member Function Documentation

7.88.5.1 `DDS_ReturnCode_t DDSWaitSet::wait ( DDSConditionSeq & active_conditions, const DDS_Duration_t & timeout )`

<<cert>> Allows an application thread to wait for the occurrence of certain conditions.

If none of the conditions attached to the `DDSWaitSet` have a `trigger_value` of `DDS_BOOLEAN_TRUE`, the wait operation will block suspending the calling thread.

The result of the wait operation is the list of all the attached conditions that have a `trigger_value` of `DDS_BOOLEAN_TRUE` (i.e., the conditions that unblocked the wait).

Note: The resolution of the `timeout` period is constrained by the resolution of the system clock.

The wait operation takes a `timeout` argument that specifies the maximum duration for the wait. If this duration is exceeded and none of the attached `DDSCondition` objects is `DDS_BOOLEAN_TRUE`, wait will return with the return code `DDS_RETCODE_TIMEOUT`. In this case, the resulting list of conditions will be empty.

Note: The resolution of the `timeout` period is constrained by the resolution of the system clock.

`DDS_RETCODE_TIMEOUT` will *not* be returned when the `timeout` duration is exceeded if attached `DDSCondition` objects are `DDS_BOOLEAN_TRUE`, or in the case of a `DDSWaitSet` waiting for more than one trigger event, if one or more trigger events have occurred.

It is not allowable for more than one application thread to be waiting on the same `DDSWaitSet`. If the wait operation is invoked on a `DDSWaitSet` that already has a thread blocking on it, the operation will return immediately with the value `DDS_RETCODE_PRECONDITION_NOT_MET`.

Parameters

<i>active_conditions</i>	<<inout>> a valid non-NULL DDSConditionSeq object. Note that RTI Connex Micro will not allocate a new object if <i>active_conditions</i> is NULL; the method will return DDS_RETCODE_PRECONDITION_NOT_MET .
--------------------------	---

Parameters

<i>timeout</i>	<<in>> a wait timeout
----------------	-----------------------

Returns

One of the [Standard Return Codes](#) or [DDS_RETCODE_PRECONDITION_NOT_MET](#) or [DDS_RETCODE_TIMEOUT](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.88.5.2 [DDS_ReturnCode_t](#) [DDSWaitSet::attach_condition](#) ([DDSCondition](#) * *cond*)

<<cert>> Attaches a [DDSCondition](#) to the [DDSWaitSet](#).

It is possible to attach a [DDSCondition](#) on a [DDSWaitSet](#) that is currently being waited upon (via the wait operation). In this case, if the [DDSCondition](#) has a *trigger_value* of [DDS_BOOLEAN_TRUE](#), then attaching the condition will unblock the [DDSWaitSet](#).

Parameters

<i>cond</i>	<<in>> Contition to be attached.
-------------	----------------------------------

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_OUT_OF_RESOURCES](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.88.5.3 [DDS_ReturnCode_t](#) [DDSWaitSet::detach_condition](#) ([DDSCondition](#) * *cond*)

Detaches a [DDSCondition](#) from the [DDSWaitSet](#).

If the [DDSCondition](#) was not attached to the [DDSWaitSet](#) the operation will return [DDS_RETCODE_BAD_PARAMETER](#).

Parameters

<i>cond</i>	<<in>> Condition to be detached.
-------------	----------------------------------

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_PRECONDITION_NOT_MET](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.88.5.4 DDS_ReturnCode_t DDSWaitSet::get_conditions (DDSConditionSeq & attached_conditions)

<<cert>> Retrieves the list of attached [DDSCondition](#) (s).

Parameters

<i>attached_conditions</i>	<<inout>> a DDSConditionSeq object where the list of attached conditions will be returned
----------------------------	---

Returns

One of the [Standard Return Codes](#), or [DDS_RETCODE_OUT_OF_RESOURCES](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.89 DPDE_DiscoveryPluginProperty Struct Reference

<<eXtension>> Properties for the Dynamic Participant/Dynamic Endpoint (DPDE) discovery plugin. This includes all discovery timing properties for participant discovery.

Public Attributes

- struct [DDS_Duration_t participant_liveliness_assert_period](#)
The period to assert liveliness for the participant.
- struct [DDS_Duration_t participant_liveliness_lease_duration](#)
The liveliness lease duration for the participant.
- [DDS_Long initial_participant_announcements](#)
The number of initial announcements sent when a participant is first enabled or when a remote participant is newly discovered.
- struct [DDS_Duration_t initial_participant_announcement_period](#)
The period between initial announcements when a participant is first enabled or when a remote participant is newly discovered.
- [DDS_Boolean cache_serialized_samples](#)
Cache serialized discovery samples for locally created data readers and data writers.
- [DDS_Long max_participant_locators](#)
The maximum number of locators the RTI Connex Micro can send to to discover other participants in the same domain.
- [DDS_Long max_locators_per_discovered_participant](#)
The maximum number of locators the RTI Connex Micro can keep track of per discovered participant.
- [DDS_Long max_samples_per_builtin_endpoint_reader](#)
The maximum number of samples allocated per builtin endpoint DataReader.
- [DDS_Long max_samples_per_remote_builtin_endpoint_writer](#)
The maximum number of samples allocated per remote builtin writer.
- [DDS_Long builtin_writer_max_heartbeat_retries](#)
Maximum number of heartbeat retries for built-in endpoints.
- struct [DDS_Duration_t builtin_writer_heartbeat_period](#)
Heartbeat period for built-in endpoints.
- [DDS_Long builtin_writer_heartbeats_per_max_samples](#)
Number of heartbeats per max_samples built-in endpoints.
- struct [DDS_Duration_t builtin_endpoint_reader_nack_period](#)
The period at which NACKs are sent for the builtin endpoint readers.

7.89.1 Detailed Description

<<*eXtension*>> Properties for the Dynamic Participant/Dynamic Endpoint (DPDE) discovery plugin. This includes all discovery timing properties for participant discovery.

7.89.2 Member Data Documentation

7.89.2.1 struct DDS_Duration_t DPDE_DiscoveryPluginProperty::participant_liveliness_assert_period

The period to assert liveliness for the participant.

Specifies how often the DomainParticipant's discovery DataWriter will assert the liveliness of the DomainParticipant to its peers.

Should be strictly less than participant_liveliness_lease_duration.

[default] 30 seconds

[range] [1 nanosec, 1 year], < participant_liveliness_lease_duration

7.89.2.2 struct DDS_Duration_t DPDE_DiscoveryPluginProperty::participant_liveliness_lease_duration

The liveliness lease duration for the participant.

This is the same as the expiration time of the DomainParticipant as defined in the RTPS protocol.

If the participant has not refreshed its own liveliness to other participants at least once within this period, it may be considered as stale by other participants in the network.

Should be strictly greater than participant_liveliness_assert_period.

[default] 100 seconds

[range] [1 nanosec, 1 year], > participant_liveliness_assert_period

7.89.2.3 DDS_Long DPDE_DiscoveryPluginProperty::initial_participant_announcements

The number of initial announcements sent when a participant is first enabled or when a remote participant is newly discovered.

The number of initial announcements sent when a participant is first enabled or when a remote participant is newly discovered.

Also, when a new remote participant appears, the local participant will announce itself to the peers a number of times controlled by this parameter.

[default] 5

7.89.2.4 struct DDS_Duration_t DPDE_DiscoveryPluginProperty::initial_participant_announcement_period

The period between initial announcements when a participant is first enabled or when a remote participant is newly discovered.

[default] 1 second

7.89.2.5 DDS_Boolean DPDE_DiscoveryPluginProperty::cache_serialized_samples

Cache serialized discovery samples for locally created data readers and data writers.

When a data reader or data writer is enabled locally, the discovery plugin announces this to other participants in its domain. When this value is set to `DDS_BOOLEAN_TRUE` the serialized sample is stored in case a new participant is discovered and the information must be sent again. This may improve discovery performance, but increases memory usage. The default value is `DDS_BOOLEAN_FALSE` which uses less memory.

[default] `DDS_BOOLEAN_FALSE`

7.89.2.6 DDS_Long DPDE_DiscoveryPluginProperty::max_participant_locators

The maximum number of locators the RTI Connex Micro can send to to discover other participants in the same domain.

The discovery plugin sends participant announcements to a number of locators as specified in the initial peer list. The maximum number of locators to send too is specified here.

The value DDS_LENGTH_AUTO calculates the maximum number of locators as $\text{DomainParticipantQos.resource_limits.remote_participant_allocation} + (\text{DPDE_MAX_ANON_PARTICIPANT} * (\text{initial_peers length} + 1))$

The DPDE builtin participant topic [DDSDDataWriter](#) allocates max_participant_locators to send participant announcements to. The default value DDS_LENGTH_AUTO calculates max_participant_locators as $\text{remote_participant_allocation} + (\text{DPDE_MAX_ANON_PARTICIPANT} * (\text{peer_length} + 1))$ where the constant DPDE_MAX_ANON_PARTICIPANT = 6.

A participant locator is defined as a unique index@address to send to where index is an integer ≥ 0 and only applies to unicast addresses. For multicast address the index is 0. This index is the highest participant_id that will receive participant announcements at the address.

The maximum number of addresses to send to is defined by the peer list (either specified in [DDS_DiscoveryQos-Policy::initial_peers](#) or by calling [DDSDomainParticipant::add_peer](#). The maximum number of addresses needed is determined by adding all indices for all unicast addresses and adding one for each multicast address.

If memory is limited it is advised to set this value as low as possible.

[default] DDS_LENGTH_AUTO

7.89.2.7 DDS_Long DPDE_DiscoveryPluginProperty::max_locators_per_discovered_participant

The maximum number of locators the RTI Connex Micro can keep track of per discovered participant.

When a participant discover another participant, it must keep track of which addresses to respond to for each participant. NOTE: If two participants respond with the same addresses, for example multicast, only one address is saved.

[default] 4

7.89.2.8 DDS_Long DPDE_DiscoveryPluginProperty::max_samples_per_builtin_endpoint_reader

The maximum number of samples allocated per builtin endpoint DataReader.

The builtin endpoint datareaders are reliable and may cache samples received out-of-order to improve discovery performance. This resource limit determines the maximum number of outstanding samples at any given time.

[default] 8

[range] [1, 100000000]

7.89.2.9 DDS_Long DPDE_DiscoveryPluginProperty::max_samples_per_remote_builtin_endpoint_writer

The maximum number of samples allocated per remote builtin writer.

The builtin endpoint datareaders are reliable and may cache samples received out-of-order to improve discovery performance. This resource limit determines the maximum number of outstanding samples at any given time per remote builtin endpoint DataWriter. Note that this limit is per builtin endpoint DataReader.

The default value DDS_MAX_UNLIMITED allows a remote builtin endpoint DataWriter to potentially use all samples while missing samples are being resent. This may cause samples received by other remote builtin endpoint datawriters to be rejected until the remote builtin endpoint DataWriter using all the samples frees up resources.

The following values are supported for this member:

- **DDS_MAX_UNLIMITED**. This allows each remote builtin endpoint DataWriter to use up to [DPDE_DiscoveryPluginProperty::max_samples_per_builtin_endpoint_reader](#) samples.
- [1, 256]. When a positive value is specified [DPDE_DiscoveryPluginProperty::max_samples_per_builtin_](#)

`endpoint_reader` is automatically adjusted to `DPDE_DiscoveryPluginProperty::max_samples_per_remote_built_in_endpoint_writer * DDS_DomainParticipantResourceLimitsQosPolicy::remote_participant_allocation`.

Note that the value of `DPDE_DiscoveryPluginProperty::max_samples_per_built_in_endpoint_reader` is still limited to [1, 100000000] and a run-time error will occur if the value is exceeded.

[default] DDS_MAX_UNLIMITED

[range] DDS_MAX_UNLIMITED, [1, 256]

7.89.2.10 DDS_Long DPDE_DiscoveryPluginProperty::builtin_writer_max_heartbeat_retries

Maximum number of heartbeat retries for built-in endpoints.

[default] DDS_LENGTH_UNLIMITED

[range] [1, 100000000] or DDS_LENGTH_UNLIMITED.

7.89.2.11 struct DDS_Duration_t DPDE_DiscoveryPluginProperty::builtin_writer_heartbeat_period

Heartbeat period for built-in endpoints.

[default] 100 ms.

[range] [1 nanosec, 1 year]

7.89.2.12 DDS_Long DPDE_DiscoveryPluginProperty::builtin_writer_heartbeats_per_max_samples

Number of heartbeats per max_samples built-in endpoints.

[default] DDS_LENGTH_UNLIMITED

[range] [1, local_writer_allocation] or DDS_LENGTH_UNLIMITED.

7.89.2.13 struct DDS_Duration_t DPDE_DiscoveryPluginProperty::builtin_endpoint_reader_nack_period

The period at which NACKs are sent for the builtin endpoint readers.

Please refer to `DDS_RtpsReliableReaderProtocol_t::nack_period` for a detailed description.

7.90 DPDEDiscoveryFactory Class Reference

<<eXtension>> Discovery plug-in used for asserting remote entities.

Static Public Member Functions

- static struct RT_ComponentFactoryI * `get_interface ()`
 <<cert>> Gets the component factory necessary when registering a DPDE component with RTI Connex Micro

7.90.1 Detailed Description

<<eXtension>> Discovery plug-in used for asserting remote entities.

7.90.2 Member Function Documentation

7.90.2.1 static struct RT_ComponentFactoryI* DPDEDiscoveryFactory::get_interface () [static], [read]

<<cert>> Gets the component factory necessary when registering a DPDE component with RTI Connex Micro

7.91 DPSE_DiscoveryPluginProperty Struct Reference

`<<eXtension>>` `<<cert>>` Properties for the Dynamic Participant/Static Endpoint (DPSE) discovery plugin. This includes all discovery timing properties for participant discovery.

Public Attributes

- struct `DDS_Duration_t participant_liveliness_assert_period`
The period to assert liveliness for the participant.
- struct `DDS_Duration_t participant_liveliness_lease_duration`
The liveliness lease duration for the participant.
- `DDS_Long initial_participant_announcements`
The number of initial announcements sent when a participant is first enabled or when a remote participant is newly discovered.
- struct `DDS_Duration_t initial_participant_announcement_period`
The period between initial announcements when a participant is first enabled or when a remote participant is newly discovered.
- `DDS_Long max_participant_locators`
The maximum number of locators RTI Connex Micro can send to to discover other participants in the same domain.
- `DDS_Long max_locators_per_discovered_participant`
The maximum number of locators RTI Connex Micro can keep track of per discovered participant.

7.91.1 Detailed Description

`<<eXtension>>` `<<cert>>` Properties for the Dynamic Participant/Static Endpoint (DPSE) discovery plugin. This includes all discovery timing properties for participant discovery.

7.91.2 Member Data Documentation

7.91.2.1 struct `DDS_Duration_t DPSE_DiscoveryPluginProperty::participant_liveliness_assert_period`

The period to assert liveliness for the participant.

Specifies how often the DomainParticipant's discovery DataWriter will assert the liveliness of the DomainParticipant to its peers.

Should be strictly less than `participant_liveliness_lease_duration`.

[default] 30 seconds

[range] [1 nanosec, 1 year], < `participant_liveliness_lease_duration`

7.91.2.2 struct `DDS_Duration_t DPSE_DiscoveryPluginProperty::participant_liveliness_lease_duration`

The liveliness lease duration for the participant.

This is the same as the expiration time of the DomainParticipant as defined in the RTPS protocol.

If the participant has not refreshed its own liveliness to other participants at least once within this period, it may be considered as stale by other participants in the network.

Should be strictly greater than `participant_liveliness_assert_period`.

[default] 100 seconds

[range] [1 nanosec, 1 year], > `participant_liveliness_assert_period`

7.91.2.3 DDS_Long DPSE_DiscoveryPluginProperty::initial_participant_announcements

The number of initial announcements sent when a participant is first enabled or when a remote participant is newly discovered.

The number of initial announcements sent when a participant is first enabled or when a remote participant is newly discovered.

Also, when a new remote participant appears, the local participant will announce itself to the peers a number of times controlled by this parameter.

[default] 5

7.91.2.4 struct DDS_Duration_t DPSE_DiscoveryPluginProperty::initial_participant_announcement_period

The period between initial announcements when a participant is first enabled or when a remote participant is newly discovered.

[default] 1 second

7.91.2.5 DDS_Long DPSE_DiscoveryPluginProperty::max_participant_locators

The maximum number of locators RTI Connex Micro can send to to discover other participants in the same domain.

The DPSE builtin participant topic [DDSDDataWriter](#) allocates max_participant_locators to send participant announcements to. The default value DDS_LENGTH_AUTO calculates max_participant_locators as remote_participant_allocation + (DPSE_MAX_ANON_PARTICIPANT * (peer_length + 1)) where the constant DPSE_MAX_ANON_PARTICIPANT = 6.

A participant locator is defined as a unique index@address to send to where index is an integer ≥ 0 and only applies to unicast addresses. For multicast address the index is 0. This index is the highest participant_id that will receive participant announcements at the address.

The maximum number of addresses to send to is defined by the peer list (either specified in [DDS_DiscoveryQosPolicy::initial_peers](#) or by calling [DDSDomainParticipant::add_peer](#). The maximum number of addresses needed is determined by adding all indices for all unicast addresses and adding one for each multicast address.

If memory is limited it is advised to set this value as low as possible.

[default] DDS_LENGTH_AUTO;

7.91.2.6 DDS_Long DPSE_DiscoveryPluginProperty::max_locators_per_discovered_participant

The maximum number of locators RTI Connex Micro can keep track of per discovered participant.

When a participant discover another participant, it must keep track of which addresses to respond to for each participant. NOTE: If two participants respond with the same addresses, for example multicast, only one address is saved.

[default] 4;

7.92 DPSEDiscoveryFactory Class Reference

[`<<eXtension>>`](#) [`<<cert>>`](#) A discovery factory that will be used by the [DDSDomainParticipantFactory](#) to create a discovery plugin.

Static Public Member Functions

- static struct
RT_ComponentFactoryI * [get_interface](#) ()

Gets the singleton instance of the DPSE discovery plugin factory. The singleton instance of the factory is used when creating a domain participant.

7.92.1 Detailed Description

<<eXtension>> <<cert>> A discovery factory that will be used by the [DDSDomainParticipantFactory](#) to create a discovery plugin.

7.92.2 Member Function Documentation

7.92.2.1 static struct RT_ComponentFactoryI* DPSEDiscoveryFactory::get_interface () [static],[read]

Gets the singleton instance of the DPSE discovery plugin factory. The singleton instance of the factory is used when creating a domain participant.

This function gets the single singleton instance of the DPSE discovery plugin factory that is used by the middleware to create a DPSE plugin. In the future, as more discovery plug-ins are supported, they will be registered with the middleware by installing different discovery factory instances with the [DDSDomainParticipantFactory](#). The singleton instance of the factory is registered with a name, and that name is specified in the [DDSDomainParticipant DDS_-DiscoveryQosPolicy](#). This is used to create a discovery plug-in instance when creating a domain participant.

Returns

Pointer to DPSE discovery plugin factory

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDS_DiscoveryQosPolicy](#)

7.93 DPSEDiscoveryPlugin Class Reference

<<eXtension>> <<cert>> Discovery plug-in used for asserting remote entities.

Static Public Member Functions

- static [DDS_ReturnCode_t RemoteParticipant_assert](#) ([DDSDomainParticipant](#) *const participant, const char *rem_participant_name)
 <<cert>> Assert remote participant by passing in the name of that participant.
- static [DDS_ReturnCode_t RemotePublication_assert](#) ([DDSDomainParticipant](#) *const participant, const char *const rem_participant_name, const struct [DDS_PublicationBuiltinTopicData](#) *const data, NDDS_TypePluginKeyKind key_kind)
 <<cert>> Assert remote publication by passing in the builtin data that describes the type, topic, and QoS of that remote publication.
- static [DDS_ReturnCode_t RemoteSubscription_assert](#) ([DDSDomainParticipant](#) *const participant, const char *const rem_participant_name, const struct [DDS_SubscriptionBuiltinTopicData](#) *const data, NDDS_TypePluginKeyKind key_kind)
 <<cert>> Assert remote subscription by passing in the builtin data that describes the type, topic, and QoS of that remote subscription.

7.93.1 Detailed Description

<<eXtension>> <<cert>> Discovery plug-in used for asserting remote entities.

7.93.2 Member Function Documentation

7.93.2.1 `static DDS_ReturnCode_t DPSEDiscoveryPlugin::RemoteParticipant_assert ( DDSDomainParticipant *const participant, const char * rem_participant_name ) [static]`

<<cert>> Assert remote participant by passing in the name of that participant.

DPSE discovery requires that you pre-configure your application with information about any remote participants that you intend to discover. The only information that needs to be pre-configured is the name of the remote [DDSDomainParticipant](#) that your application intends to discover.

Parameters

<i>participant</i>	<<in>> The DDSDomainParticipant to assert the remote participant <i>rem_participant_name</i> into.
<i>rem_participant_name</i>	<<in>> The name of the remote participant as specified in DDS_ParticipantBuiltinTopicData::participant_name .

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.93.2.2 `static DDS_ReturnCode_t DPSEDiscoveryPlugin::RemotePublication_assert ( DDSDomainParticipant *const participant, const char *const rem_participant_name, const struct DDS_PublicationBuiltinTopicData *const data, NDDS_TypePluginKeyKind key_kind ) [static]`

<<cert>> Assert remote publication by passing in the builtin data that describes the type, topic, and QoS of that remote publication.

DPSE discovery requires that you pre-configure your application with information about any remote publications that you intend to discover and receive data from. The [DDS_PublicationBuiltinTopicData](#) field must include the topic name, type name, and QoS of the remote [DDSDataWriter](#) that you intend to receive communications from.

Additionally, this method takes a *key_kind* argument that must match the key kind returned by the type plugin.

Parameters

<i>participant</i>	<<in>> The DDSDomainParticipant to assert the remote publication defined by DDS_PublicationBuiltinTopicData into.
<i>rem_participant_name</i>	<<in>> The name of the remote participant as specified in DDS_ParticipantBuiltinTopicData::participant_name .
<i>data</i>	<<in>> The DDS_PublicationBuiltinTopicData defining the remote publication.
<i>key_kind</i>	<<in>> The IDL key type.

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.93.2.3 **static** `DDS_ReturnCode_t DPSEDiscoveryPlugin::RemoteSubscription::assert ( DDSDomainParticipant *const participant, const char *const rem_participant_name, const struct DDS_SubscriptionBuiltinTopicData *const data, NDDS_TypePluginKeyKind key_kind ) [static]`

<<cert>> Assert remote subscription by passing in the builtin data that describes the type, topic, and QoS of that remote subscription.

DPSE discovery requires that you pre-configure your application with information about any remote subscriptions that you intend to discover and send data to. The `DDS_SubscriptionBuiltinTopicData` parameter must include the topic name, type name, and QoS of the remote `DDSDataReader` that your application intends to send data to. Additionally, if the remote `DDSDataReader` is listening on a non-default port, you must set the remote locator (including the non-default port) in the `DDS_SubscriptionBuiltinTopicData` that you intend to use to contact the remote `DDSDataReader`.

Additionally, this method takes a `key_kind` argument that must match the key kind returned by the type plugin.

Parameters

<i>participant</i>	<<in>> The <code>DDSDomainParticipant</code> to assert the remote subscription defined by <code>DDS_SubscriptionBuiltinTopicData</code> into.
<i>rem_participant_name</i>	<<in>> The name of the remote participant as specified in <code>DDS_ParticipantBuiltinTopicData::participant_name</code> .
<i>data</i>	<<in>> The <code>DDS_SubscriptionBuiltinTopicData</code> defining the remote subscription.
<i>key_kind</i>	<<in>> The IDL key type.

Returns

One of the [Standard Return Codes](#)

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.94 FooDataReader Struct Reference

Declares the interface required to support a user data type-specific data reader.

Inherits `DDSDataReader_impl`.

Public Member Functions

- `DDS_ReturnCode_t return_loan (FooSeq &received_data, DDS_SampleInfoSeq &info_seq)`
Indicates to the `DDSDataReader` that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of `read` or `take` on the `DDSDataReader`.
- `DDS_ReturnCode_t read_next_sample (Foo &received_data, DDS_SampleInfo &sample_info)`
Copies the next not-previously-accessed data value from the `DDSDataReader`.
- `DDS_ReturnCode_t take_next_sample (Foo &received_data, DDS_SampleInfo &sample_info)`
Copies the next not-previously-accessed data value from the `DDSDataReader`.
- `DDS_ReturnCode_t read (FooSeq &received_data, DDS_SampleInfoSeq &info_seq, DDS_Long max_samples, DDS_SampleStateMask sample_states, DDS_ViewStateMask view_states, DDS_InstanceStateMask instance_states)`
Access a collection of data samples from the `DDSDataReader`.
- `DDS_ReturnCode_t take (FooSeq &received_data, DDS_SampleInfoSeq &info_seq, DDS_Long max_samples, DDS_SampleStateMask sample_states, DDS_ViewStateMask view_states, DDS_InstanceStateMask instance_states)`
Access a collection of data-samples from the `DDSDataReader`.

- `DDS_ReturnCode_t read_instance` (`FooSeq` &received_data, `DDS_SampleInfoSeq` &info_seq, `DDS_Long` max_samples, const `DDS_InstanceHandle_t` &a_handle, `DDS_SampleStateMask` sample_states, `DDS_ViewStateMask` view_states, `DDS_InstanceStateMask` instance_states)

Access a collection of data samples from the `DDSDataReader`.

- `DDS_ReturnCode_t take_instance` (`FooSeq` &received_data, `DDS_SampleInfoSeq` &info_seq, `DDS_Long` max_samples, const `DDS_InstanceHandle_t` &a_handle, `DDS_SampleStateMask` sample_states, `DDS_ViewStateMask` view_states, `DDS_InstanceStateMask` instance_states)

Access a collection of data samples from the `DDSDataReader`.

Static Public Member Functions

- static `FooDataReader * narrow` (`DDSDataReader * reader`)

Narrow the given `DDSDataReader` pointer to a `FooDataReader` pointer.

7.94.1 Detailed Description

Declares the interface required to support a user data type-specific data reader.

7.94.2 Member Function Documentation

7.94.2.1 `static FooDataReader* FooDataReader::narrow ( DDSDataReader * reader ) [static]`

Narrow the given `DDSDataReader` pointer to a `FooDataReader` pointer.

7.94.2.2 `DDS_ReturnCode_t FooDataReader::return_loan ( FooSeq & received_data, DDS_SampleInfoSeq & info_seq )`

Indicates to the `DDSDataReader` that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of `read` or `take` on the `DDSDataReader`.

This operation indicates to the `DDSDataReader` that the application is done accessing the collection of `received_data` and `info_seq` obtained by some earlier invocation of `read` or `take` on the `DDSDataReader`.

The `received_data` and `info_seq` must belong to a single related "pair"; that is, they should correspond to a pair returned from a single call to `read` or `take`. The `received_data` and `info_seq` must also have been obtained from the same `DDSDataReader` to which they are returned. If either of these conditions is not met, the operation will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`.

The operation `FooDataReader::return_loan` allows implementations of the `read` and `take` operations to "loan" buffers from the `DDSDataReader` to the application and in this manner provide "zerocopy" access to the data. During the loan, the `DDSDataReader` will guarantee that the data and sample-information are not modified.

It is not necessary for an application to return the loans immediately after the `read` or `take` calls. However, as these buffers correspond to internal resources inside the `DDSDataReader`, the application should not retain them indefinitely.

The use of `FooDataReader::return_loan` is only necessary if the `read` or `take` calls "loaned" buffers to the application. This only occurs if the `received_data` and `info_seq` collections had `max_len=0` at the time `read` or `take` was called.

The application may also examine the "owns" property of the collection to determine where there is an outstanding loan. However, calling `FooDataReader::return_loan` on a collection that does not have a loan is safe and has no side effects.

If the collections had a loan, upon completion of `FooDataReader::return_loan`, the collections will have `max_len=0`.

Similar to `read`, this operation must be provided on the specialized class that is generated for the particular application data-type that is being taken.

Parameters

<i>received_data</i>	<< <i>in</i> >> user data type-specific FooSeq object where the received data samples was obtained from earlier invocation of read or take on the DDSDDataReader .
----------------------	--

Parameters

<i>info_seq</i>	<< <i>in</i> >> a DDS_SampleInfoSeq object where the received sample info was obtained from earlier invocation of read or take on the DDSDDataReader .
-----------------	--

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

7.94.2.3 DDS_ReturnCode_t FooDataReader::read_next_sample (Foo & *received_data*, DDS_SampleInfo & *sample_info*)

Copies the next not-previously-accessed data value from the [DDSDDataReader](#).

This operation copies the next not-previously-accessed data value from the [DDSDDataReader](#). This operation also copies the corresponding [DDS_SampleInfo](#). The implied order among the samples stored in the [DDSDDataReader](#) is the same as for the [FooDataReader::read](#) operation.

The [FooDataReader::read_next_sample](#) operation is semantically equivalent to the [FooDataReader::read](#) operation, where the input data sequences has `max_len=1`, the `sample_states=NOT_READ`, the `view_states=ANY_VIEW_STATE`, and the `instance_states=ANY_INSTANCE_STATE`.

The [FooDataReader::read_next_sample](#) operation provides a simplified API to 'read' samples, avoiding the need for the application to manage sequences and specify states.

If there is no unread data in the [DDSDDataReader](#), the operation will fail with [DDS_RETCODE_NO_DATA](#) and nothing is copied.

Parameters

<i>received_data</i>	<< <i>inout</i> >> user data type-specific Foo object where the next received data sample will be returned. The <code>received_data</code> must have been fully allocated. Otherwise, this operation may fail.
<i>sample_info</i>	<< <i>inout</i> >> a DDS_SampleInfo object where the next received sample info will be returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataReader::read](#)

7.94.2.4 **DDS_ReturnCode_t** FooDataReader::take_next_sample (Foo & received_data, DDS_SampleInfo & sample_info)

Copies the next not-previously-accessed data value from the [DDSDDataReader](#).

This operation copies the next not-previously-accessed data value from the [DDSDDataReader](#) and 'removes' it from the [DDSDDataReader](#) so that it is no longer accessible. This operation also copies the corresponding [DDS_SampleInfo](#). This operation is analogous to the [FooDataReader::read_next_sample](#) except for the fact that the sample is removed from the [DDSDDataReader](#).

The [FooDataReader::take_next_sample](#) operation is semantically equivalent to the [FooDataReader::take](#) operation, where the input data sequences has max_len=1, the sample_states=NOT_READ, the view_states=ANY_VIEW_STATE, and the instance_states=ANY_INSTANCE_STATE.

The [FooDataReader::read_next_sample](#) operation provides a simplified API to 'take' samples, avoiding the need for the application to manage sequences and specify states.

If there is no unread data in the [DDSDDataReader](#), the operation will fail with [DDS_RETCODE_NO_DATA](#) and nothing is copied.

Parameters

<i>received_data</i>	<<inout>> user data type-specific Foo object where the next received data sample will be returned. The received_data must have been fully allocated. Otherwise, this operation may fail.
<i>sample_info</i>	<<inout>> a DDS_SampleInfo object where the next received sample info will be returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataReader::take](#)

7.94.2.5 **DDS_ReturnCode_t** FooDataReader::read (FooSeq & received_data, DDS_SampleInfoSeq & info_seq, DDS_Long max_samples, DDS_SampleStateMask sample_states, DDS_ViewStateMask view_states, DDS_InstanceStateMask instance_states)

Access a collection of data samples from the [DDSDDataReader](#).

This operation offers the same functionality and API as [FooDataReader::take](#) except that the samples returned remain in the [DDSDDataReader](#) such that they can be retrieved again by means of a read or take operation.

Please refer to the documentation of [FooDataReader::take\(\)](#) for details on the number of samples returned within the received_data and info_seq as well as the order in which the samples appear in these sequences.

The act of reading a sample changes its sample_state to [DDS_READ_SAMPLE_STATE](#). If the sample belongs to the most recent generation of the instance, it will also set the view_state of the instance to be [DDS_NOT_NEW_VIEW_STATE](#). It will not affect the instance_state of the instance.

Important: If the samples "returned" by this method are loaned from RTI Connex Micro (see [FooDataReader::take](#) for more information on memory loaning), it is important that their contents not be changed. Because the memory in which the data is stored belongs to the middleware, any modifications made to the data will be seen the next time the same samples are read or taken; the samples will no longer reflect the state that was received from the network.

Parameters

<i>received_data</i>	<<inout>> User data type-specific FooSeq object where the received data samples will be returned.
<i>info_seq</i>	<<inout>> A DDS_SampleInfoSeq object where the received sample info will be returned.
<i>max_samples</i>	<<in>> The maximum number of samples to be returned. If the special value DDS_LENGTH_UNLIMITED is provided, as many samples will be returned as are available.
<i>sample_states</i>	<<in>> Data samples matching one of these <i>sample_states</i> are returned.
<i>view_states</i>	<<in>> Data samples matching one of these <i>view_state</i> are returned.
<i>instance_states</i>	<<in>> Data samples matching ones of these <i>instance_state</i> are returned.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataReader::take](#),
[DDS_LENGTH_UNLIMITED](#)

7.94.2.6 [DDS_ReturnCode_t](#) [FooDataReader::take](#) ([FooSeq](#) & *received_data*, [DDS_SampleInfoSeq](#) & *info_seq*, [DDS_Long](#) *max_samples*, [DDS_SampleStateMask](#) *sample_states*, [DDS_ViewStateMask](#) *view_states*, [DDS_InstanceStateMask](#) *instance_states*)

Access a collection of data-samples from the [DDSDataReader](#).

The operation will return the list of samples received by the [DDSDataReader](#) since the last [FooDataReader::take](#) operation that match the specified [DDS_SampleStateMask](#), [DDS_ViewStateMask](#) and [DDS_InstanceStateMask](#).

- In the case where the [DDS_HistoryQosPolicy::kind](#) is [DDS_KEEP_LAST_HISTORY_QOS](#) (currently the only supported configuration), the call will return at most [DDS_HistoryQosPolicy::depth](#) samples per instance.
- The maximum number of samples returned is limited by [DDS_ResourceLimitsQosPolicy::max_samples](#)
- For multiple instances, the number of samples returned is additionally limited by the product ([DDS_ResourceLimitsQosPolicy::max_samples_per_instance](#) * [DDS_ResourceLimitsQosPolicy::max_instances](#))

If the read or take succeeds and the number of samples returned has been limited (by means of a maximum limit, as listed above, or insufficient [DDS_SampleInfo](#) resources), the call will complete successfully and provide those samples the reader is able to return. The user may need to make additional calls, or return outstanding loaned buffers in the case of insufficient resources, in order to access remaining samples.

Note that in the case where the [DDSTopic](#) associated with the [DDSDataReader](#) is bound to a data-type that has no key definition, then there will be at most one instance in the [DDSDataReader](#). So the per-sample limits will apply.

The act of *taking* a sample removes it from RTI Connex Micro so it cannot be read or taken again. If the sample belongs to the most recent generation of the instance, it will also set the *view_state* of the sample's instance to [DDS_NOT_NEW_VIEW_STATE](#). It will not affect the *instance_state* of the sample's instance.

After [FooDataReader::take](#) completes, *received_data* and *info_seq* will be of the same length and contain the received data.

If the sequences are empty (maximum size equal 0) when the [FooDataReader::take](#) is called, the samples returned in the *received_data* and the corresponding *info_seq* are 'loaned' to the application from buffers provided by the [DDSDataReader](#). The application can use them as desired and has guaranteed exclusive access to them.

Once the application completes its use of the samples it must 'return the loan' to the `DDSDataReader` by calling the `FooDataReader::return_loan` operation.

Note: While you must call `FooDataReader::return_loan` at some point, you do *not* have to do so before the next `FooDataReader::take` call. However, failure to return the loan will eventually deplete the `DDSDataReader` of the buffers it needs to receive new samples and eventually samples will start to be lost. The total number of buffers available to the `DDSDataReader` is specified by the `DDS_ResourceLimitsQosPolicy`.

If the sequences are not empty (maximum size not equal 0) when the `FooDataReader::take` is called, samples are copied to `received_data` and `info_seq`. The application will not need to call `FooDataReader::return_loan`.

The samples returned to the caller is a list where samples belonging to the same data instance are consecutive (i.e. instance access scope).

Samples belonging to the same instance will appear in the relative order in which there were received (i.e. reception timestamp destination order, where earlier samples are ahead of later samples).

If the `DDSDataReader` has no samples that meet the constraints, the method will fail with `DDS_RETCODE_NO_DATA`.

In addition to the collection of samples, the read and take operations also use a collection of `DDS_SampleInfo` structures.

7.94.3 SEQUENCES USAGE IN TAKE AND READ

The initial (input) properties of the `received_data` and `info_seq` collections will determine the precise behavior of the read or take operation. For the purposes of this description, the collections are modeled as having these properties:

- whether the collection container owns the memory of the elements within (`owns`)
- the current-length (`len`, see `FooSeq::length()`)
- the maximum length (`max_len`, see `FooSeq::maximum()`)

The initial values of the `owns`, `len` and `max_len` properties for the `received_data` and `info_seq` collections govern the behavior of the read and take operations as specified by the following rules:

1. The values of `owns`, `len` and `max_len` properties for the two collections must be identical. Otherwise read/take will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`.
2. On successful output, the values of `owns`, `len` and `max_len` will be the same for both collections.
3. If the initial `max_len==0`, then the `received_data` and `info_seq` collections will be filled with elements that are loaned by the `DDSDataReader`. On output, `owns` will be `FALSE`, `len` will be set to the number of values returned, and `max_len` will be set to a value verifying `max_len >= len`. The use of this variant allows for zero-copy access to the data and the application will need to return the loan to the `DDSDataWriter` using `FooDataReader::return_loan`.
4. If the initial `max_len>0` and `owns==FALSE`, then the read or take operation will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`. This avoids the potential hard-to-detect memory leaks caused by an application forgetting to return the loan.
5. If initial `max_len>0` and `owns==TRUE`, then the read or take operation will copy the `received_data` values and `DDS_SampleInfo` values into the elements already inside the collections. On output, `owns` will be `TRUE`, `len` will be set to the number of values copied and `max_len` will remain unchanged. The use of this variant forces a copy but the application can control where the copy is placed and the application will not need to return the loan. The number of samples copied depends on the relative values of `max_len` and `max_samples`:
 - If `max_samples == LENGTH_UNLIMITED`, then at most `max_len` values will be copied. The use of this variant lets the application limit the number of samples returned to what the sequence can accommodate.

- If `max_samples ≤ max_len`, then at most `max_samples` values will be copied. The use of this variant lets the application limit the number of samples returned to fewer than what the sequence can accommodate.
- If `max_samples > max_len`, then the read or take operation will fail with `DDS_RETCODE_PRECONDITION_NOT_MET`. This avoids the potential confusion where the application expects to be able to access up to `max_samples`, but that number can never be returned, even if they are available in the `DDSDataReader`, because the output sequence cannot accommodate them.

As described above, upon completion, the `received_data` and `info_seq` collections may contain elements loaned from the `DDSDataReader`. If this is the case, the application will need to use `FooDataReader::return_loan` to return the loan once it is no longer using the `received_data` in the collection. When `FooDataReader::return_loan` completes, the collection will have `owns=FALSE` and `max_len=0`. The application can determine whether it is necessary to return the loan or not based on how the state of the collections when the read/take operation was called or by accessing the `owns` property. However, in many cases it may be simpler to always call `FooDataReader::return_loan`, as this operation is harmless (i.e., it leaves all elements unchanged) if the collection does not have a loan.

To avoid potential memory leaks, the implementation of the `Foo` and `DDS_SampleInfo` collections should disallow changing the length of a collection for which `owns==FALSE`. Furthermore, deleting a collection for which `owns==FALSE` should be considered an error.

On output, the collection of `Foo` values and the collection of `DDS_SampleInfo` structures are of the same length and are in a one-to-one correspondence. Each `DDS_SampleInfo` provides information, such as the `source_timestamp`, the `sample_state`, `view_state`, and `instance_state`, etc., about the corresponding sample.

Some elements in the returned collection may not have valid data. If the `instance_state` in the `DDS_SampleInfo` is `DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE` or `DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE`, then the last sample for that instance in the collection (that is, the one whose `DDS_SampleInfo` has `sample_rank==0`) does not contain valid data.

Samples that contain no data do not count towards the limits imposed by the `DDS_ResourceLimitsQosPolicy`. The act of reading/taking a sample sets its `sample_state` to `DDS_READ_SAMPLE_STATE`.

If the sample belongs to the most recent generation of the instance, it will also set the `view_state` of the instance to `DDS_NOT_NEW_VIEW_STATE`. It will not affect the `instance_state` of the instance.

This operation must be provided on the specialized class that is generated for the particular application data-type that is being read (`Foo`). If the `DDSDataReader` has no samples that meet the constraints, the operation fails with `DDS_RETCODE_NO_DATA`.

Parameters

<i>received_data</i>	<<inout>> User data type-specific <code>FooSeq</code> object where the received data samples will be returned.
----------------------	--

Parameters

<i>info_seq</i>	<<inout>> A <code>DDS_SampleInfoSeq</code> object where the received sample info will be returned.
<i>max_samples</i>	<<in>> The maximum number of samples to be returned. If the special value <code>DDS_LENGTH_UNLIMITED</code> is provided, as many samples will be returned as are available.
<i>sample_states</i>	<<in>> Data samples matching one of these <code>sample_states</code> are returned.
<i>view_states</i>	<<in>> Data samples matching one of these <code>view_state</code> are returned.
<i>instance_states</i>	<<in>> Data samples matching one of these <code>instance_state</code> are returned.

Returns

One of the [Standard Return Codes](#), `DDS_RETCODE_PRECONDITION_NOT_MET`, `DDS_RETCODE_NO_DATA` or `DDS_RETCODE_NOT_ENABLED`.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other

threads and must not be modified until the call returns.

See also

[FooDataReader::read](#)
[DDS_LENGTH_UNLIMITED](#)

7.94.3.1 **DDS_ReturnCode_t** [FooDataReader::read_instance](#) ([FooSeq](#) & *received_data*, [DDS_SampleInfoSeq](#) & *info_seq*, [DDS_Long](#) *max_samples*, const [DDS_InstanceHandle_t](#) & *a_handle*, [DDS_SampleStateMask](#) *sample_states*, [DDS_ViewStateMask](#) *view_states*, [DDS_InstanceStateMask](#) *instance_states*)

Access a collection of data samples from the [DDSDDataReader](#).

This operation accesses a collection of data values from the [DDSDDataReader](#). The behavior is identical to [FooDataReader::read](#), except that all samples returned belong to the single specified instance whose handle is *a_handle*.

Upon successful completion, the data collection will contain samples all belonging to the same instance. The corresponding [DDS_SampleInfo](#) verifies [DDS_SampleInfo::instance_handle](#) == *a_handle*.

The [FooDataReader::read_instance](#) operation is semantically equivalent to the [FooDataReader::read](#) operation, except in building the collection, the [DDSDDataReader](#) will check that the sample belongs to the specified instance and otherwise it will not place the sample in the returned collection.

The behavior of the [FooDataReader::read_instance](#) operation follows the same rules as the [FooDataReader::read](#) operation regarding the pre-conditions and post-conditions for the *received_data* and *sample_info*. Similar to the [FooDataReader::read](#), the [FooDataReader::read_instance](#) operation may 'loan' elements to the output collections, which must then be returned by means of [FooDataReader::return_loan](#).

Similar to the [FooDataReader::read](#), this operation must be provided on the specialized class that is generated for the particular application data-type that is being taken.

If the [DDSDDataReader](#) has no samples that meet the constraints, the method will fail with [DDS_RETCODE_NO_DATA](#).

This operation may fail with [DDS_RETCODE_BAD_PARAMETER](#) if the [DDS_InstanceHandle_t](#) *a_handle* does not correspond to an existing data-object known to the [DDSDDataReader](#).

Parameters

<i>received_data</i>	<<inout>> user data type-specific FooSeq object where the received data samples will be returned.
<i>info_seq</i>	<<inout>> a DDS_SampleInfoSeq object where the received sample info will be returned.
<i>max_samples</i>	<<in>> The maximum number of samples to be returned. If the special value DDS_LENGTH_UNLIMITED is provided, as many samples will be returned as are available.
<i>a_handle</i>	<<in>> The specified instance to return samples for. The method will fail with DDS_RETCODE_BAD_PARAMETER if the <i>handle</i> does not correspond to an existing data-object known to the DDSDDataReader .
<i>sample_states</i>	<<in>> data samples matching ones of these <i>sample_states</i> are returned
<i>view_states</i>	<<in>> data samples matching ones of these <i>view_state</i> are returned
<i>instance_states</i>	<<in>> data samples matching ones of these <i>instance_state</i> are returned

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataReader::read](#)
[DDS_LENGTH_UNLIMITED](#)

7.94.3.2 **DDS_ReturnCode_t** FooDataReader::take_instance (FooSeq & *received_data*, DDS_SampleInfoSeq & *info_seq*, DDS_Long *max_samples*, const DDS_InstanceHandle_t & *a_handle*, DDS_SampleStateMask *sample_states*, DDS_ViewStateMask *view_states*, DDS_InstanceStateMask *instance_states*)

Access a collection of data samples from the [DDSDDataReader](#).

This operation accesses a collection of data values from the [DDSDDataReader](#). The behavior is identical to [FooDataReader::take](#), except for that all samples returned belong to the single specified instance whose handle is *a_handle*.

The semantics are the same for the [FooDataReader::take](#) operation, except in building the collection, the [DDSDDataReader](#) will check that the sample belongs to the specified instance, and otherwise it will not place the sample in the returned collection.

The behavior of the [FooDataReader::take_instance](#) operation follows the same rules as the [FooDataReader::read](#) operation regarding the pre-conditions and post-conditions for the *received_data* and *sample_info*. Similar to the [FooDataReader::read](#), the [FooDataReader::take_instance](#) operation may 'loan' elements to the output collections, which must then be returned by means of [FooDataReader::return_loan](#).

Similar to the [FooDataReader::read](#), this operation must be provided on the specialized class that is generated for the particular application data-type that is being taken.

If the [DDSDDataReader](#) has no samples that meet the constraints, the method fails with [DDS_RETCODE_NO_DATA](#).

This operation may fail with [DDS_RETCODE_BAD_PARAMETER](#) if the [DDS_InstanceHandle_t](#) *a_handle* does not correspond to an existing data-object known to the [DDSDDataReader](#).

Parameters

<i>received_data</i>	<<inout>> user data type-specific FooSeq object where the received data samples will be returned.
<i>info_seq</i>	<<inout>> a DDS_SampleInfoSeq object where the received sample info will be returned.
<i>max_samples</i>	<<in>> The maximum number of samples to be returned. If the special value DDS_LENGTH_UNLIMITED is provided, as many samples will be returned as are available.
<i>a_handle</i>	<<in>> The specified instance to return samples for. The method will fail with DDS_RETCODE_BAD_PARAMETER if the <i>handle</i> does not correspond to an existing data-object known to the DDSDDataReader .
<i>sample_states</i>	<<in>> data samples matching ones of these <i>sample_states</i> are returned
<i>view_states</i>	<<in>> data samples matching ones of these <i>view_state</i> are returned
<i>instance_states</i>	<<in>> data samples matching ones of these <i>instance_state</i> are returned

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#), [DDS_RETCODE_NO_DATA](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataReader::take](#)
[DDS_LENGTH_UNLIMITED](#)

7.95 FooDataWriter Struct Reference

Declares the interface required to support a user data type-specific data writer.

Inherits DDSDataWriter_impl.

Public Member Functions

- [DDS_ReturnCode_t write](#) (const Foo &instance_data, const [DDS_InstanceHandle_t](#) &handle)
<<cert>> Modifies the value of a data instance.
- [DDS_InstanceHandle_t register_instance](#) (const Foo &instance_data)
<<cert>> Informs RTI Connex Micro that the application will be modifying a particular instance.
- [DDS_InstanceHandle_t register_instance_w_timestamp](#) (const Foo &instance_data, const [DDS_Time_t](#) &source_timestamp)
<<cert>> Performs the same functions as register_instance except that the application provides the value for the source_timestamp.
- [DDS_ReturnCode_t unregister_instance](#) (const Foo &instance_data, const [DDS_InstanceHandle_t](#) &handle)
<<cert>> Reverses the action of FooDataWriter::register_instance.
- [DDS_ReturnCode_t unregister_instance_w_timestamp](#) (const Foo &instance_data, const [DDS_InstanceHandle_t](#) &handle, const [DDS_Time_t](#) &source_timestamp)
<<cert>> Performs the same function as FooDataWriter::unregister_instance except that it also provides the value for the source_timestamp.
- [DDS_ReturnCode_t dispose](#) (const Foo &instance_data, const [DDS_InstanceHandle_t](#) &handle)
Requests the middleware to delete the data.
- [DDS_ReturnCode_t dispose_w_timestamp](#) (const Foo &instance_data, const [DDS_InstanceHandle_t](#) &handle, const [DDS_Time_t](#) &source_timestamp)
<<cert>> Performs the same functions as dispose except that the application provides the value for the source_timestamp that is made available to DDSDataReader objects by means of the source_timestamp attribute inside the DDS_SampleInfo.
- [DDS_ReturnCode_t write_w_timestamp](#) (const Foo &instance_data, const [DDS_InstanceHandle_t](#) &handle, const [DDS_Time_t](#) &source_timestamp)
<<cert>> Performs the same function as FooDataWriter::write except that it also provides the value for the source_timestamp.

Static Public Member Functions

- static [FooDataWriter](#) * [narrow](#) ([DDSDataWriter](#) *writer)
Narrow the given DDSDataWriter pointer to a FooDataWriter pointer.

7.95.1 Detailed Description

Declares the interface required to support a user data type-specific data writer.

7.95.2 Member Function Documentation

7.95.2.1 static FooDataWriter* FooDataWriter::narrow (DDSDataWriter * writer) [static]

Narrow the given [DDSDataWriter](#) pointer to a [FooDataWriter](#) pointer.

Check if the given `writer` is of type [FooDataWriter](#).

Parameters

<i>writer</i>	<<in>> Base-class DDSDataWriter to be converted to the auto-generated class FooDataWriter that extends DDSDataWriter .
---------------	--

Returns

[FooDataWriter](#) if *writer* is of type Foo. Return NULL otherwise.

7.95.2.2 DDS_ReturnCode_t FooDataWriter::write (const Foo & *instance_data*, const DDS_InstanceHandle_t & *handle*)

<<cert>> Modifies the value of a data instance.

When this operation is used, RTI Connex Micro will automatically supply the value of the *source_timestamp* that is made available to [DDSDataReader](#) objects by means of the *source_timestamp* attribute inside the [DDS_SampleInfo](#). (Refer to [DDS_SampleInfo](#) details).

As a side effect, this operation asserts liveness on the [DDSDataWriter](#) itself, the [DDSPublisher](#) and the [DDS-DomainParticipant](#).

Note that the special value [DDS_HANDLE_NIL](#) can be used for the parameter *handle*. This indicates the identity of the instance should be automatically deduced from the *instance_data* (by means of the *key*).

If *handle* is any value other than [DDS_HANDLE_NIL](#), then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with [DDS_RETCODE_BAD_PARAMETER](#).

RTI Connex Micro will not detect the error when the *handle* is any value other than [DDS_HANDLE_NIL](#), corresponds to an instance that has been registered, but does not correspond to the instance deduced from the *instance_data* (by means of the *key*). RTI Connex Micro will treat as if the *write()* operation is for the instance as indicated by the *handle*.

This operation may block if the [RELIABILITY](#) kind is set to [DDS_RELIABLE_RELIABILITY_QOS](#) and the modification would cause data to be lost or else cause one of the limits specified in the [RESOURCE_LIMITS](#) to be exceeded.

Specifically, this operation may block in the following situations (note that the list may not be exhaustive), even if its [DDS_HistoryQosPolicyKind](#) is [DDS_KEEP_LAST_HISTORY_QOS](#):

- If ([DDS_ResourceLimitsQosPolicy::max_samples](#) < [DDS_ResourceLimitsQosPolicy::max_instances](#) * [DDS_HistoryQosPolicy::depth](#)), then in the situation where the *max_samples* resource limit is exhausted, RTI Connex Micro is allowed to discard samples of some other instance, as long as at least one sample remains for such an instance. If it is still not possible to make space available to store the modification, the writer is allowed to block.
- If ([DDS_ResourceLimitsQosPolicy::max_samples](#) < [DDS_ResourceLimitsQosPolicy::max_instances](#)), then the DataWriter may block regardless of the [DDS_HistoryQosPolicy::depth](#).

If this operation *does* block for any of the above reasons, the [RELIABILITY](#) *max_blocking_time* configures the maximum time the write operation may block (waiting for space to become available). If *max_blocking_time* elapses before the [DDSDataWriter](#) is able to store the modification without exceeding the limits, the operation will time out ([DDS_RETCODE_TIMEOUT](#)).

If there are no instance resources left, this operation may fail with [DDS_RETCODE_OUT_OF_RESOURCES](#). Calling [FooDataWriter::unregister_instance](#) may help freeing up some resources.

Parameters

<i>instance_data</i>	<<in>> The data to write.
<i>handle</i>	<<in>> Either the handle returned by a previous call to FooDataWriter::register_instance , or else the special value DDS_HANDLE_NIL . If Foo has a key and <i>handle</i> is not DDS_HANDLE_NIL , <i>handle</i> must represent a registered instance of type Foo. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#), [DDS_RETCODE_OUT_OF_RESOURCES](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDataReader](#)
[FooDataWriter::write_w_timestamp](#)

7.95.2.3 [DDS_InstanceHandle_t](#) [FooDataWriter::register_instance](#) ([const Foo & instance_data](#))

[<<cert>>](#) Informs RTI Connex Micro that the application will be modifying a particular instance.

This operation is only useful for keyed data types. Using it for non-keyed types causes no effect and returns [DDS_HANDLE_NIL](#). The operation takes as a parameter an instance (of which only the key value is examined) and returns a `handle` that can be used in successive [write\(\)](#) or [dispose\(\)](#) operations.

The operation gives RTI Connex Micro an opportunity to pre-configure itself to improve performance.

The use of this operation by an application is optional even for keyed types. If an instance has not been pre-registered, the application can use the special value [DDS_HANDLE_NIL](#) as the [DDS_InstanceHandle_t](#) parameter to the write or dispose operation and RTI Connex Micro will auto-register the instance.

For best performance, the operation should be invoked prior to calling any operation that modifies the instance, such as [FooDataWriter::write](#), [FooDataWriter::write_w_timestamp](#), [FooDataWriter::dispose](#) and [FooDataWriter::dispose_w_timestamp](#) and the handle used in conjunction with the data for those calls.

When this operation is used, RTI Connex Micro will automatically supply the value of the `source_timestamp` that is used.

This operation may fail and return [DDS_HANDLE_NIL](#) if [DDS_ResourceLimitsQosPolicy::max_instances](#) limit has been exceeded.

The operation is **idempotent**. If it is called for an already registered instance, it just returns the already allocated handle. This may be used to lookup and retrieve the handle allocated to a given instance.

This operation can only be called after [DDSDataWriter](#) has been enabled. Otherwise, [DDS_HANDLE_NIL](#) will be returned.

Parameters

<i>instance_data</i>	<<in>> The instance that should be registered. Of this instance, only the fields that represent the key are examined by the function. .
----------------------	---

Returns

For keyed data type, a handle that can be used in the calls that take a [DDS_InstanceHandle_t](#), such as [write](#), [dispose](#), [unregister_instance](#), or return [DDS_HANDLE_NIL](#) on failure. If the `instance_data` is of a data type that has no keys, this function always return [DDS_HANDLE_NIL](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataWriter::unregister_instance](#)

7.95.2.4 DDS_InstanceHandle_t FooDataWriter::register_instance_w_timestamp (const Foo & instance_data, const DDS_Time_t & source_timestamp)

<<cert>> Performs the same functions as register_instance except that the application provides the value for the source_timestamp.

The provided source_timestamp potentially affects the relative order in which readers observe events from multiple writers.

This operation may fail and return DDS_HANDLE_NIL if DDS_ResourceLimitsQosPolicy::max_instances limit has been exceeded.

This operation can only be called after DDSDataWriter has been enabled. Otherwise, DDS_HANDLE_NIL will be returned.

Parameters

instance_data	<<in>> The instance that should be registered. Of this instance, only the fields that represent the key are examined by the function.
source_timestamp	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a register, unregister, dispose, or write, with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers.

Returns

For keyed data type, return a handle that can be used in the calls that take a DDS_InstanceHandle_t, such as write, dispose, unregister_instance, or return DDS_HANDLE_NIL on failure. If the instance_data is of a data type that has no keys, this function always return DDS_HANDLE_NIL.

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataWriter::unregister_instance](#)

7.95.2.5 DDS_ReturnCode_t FooDataWriter::unregister_instance (const Foo & instance_data, const DDS_InstanceHandle_t & handle)

<<cert>> Reverses the action of [FooDataWriter::register_instance](#).

This operation is useful only for keyed data types. Using it for non-keyed types causes no effect and reports no error. The operation takes as a parameter an instance (of which only the key value is examined) and a handle.

This operation should only be called on an instance that is currently registered. This includes instances that have been auto-registered by calling operations such as write or dispose as described in [FooDataWriter::register_instance](#). Otherwise, this operation may fail with DDS_RETCODE_BAD_PARAMETER.

This only need be called just once per instance, regardless of how many times register_instance was called for that instance.

When this operation is used, RTI Connex Micro will automatically supply the value of the source_timestamp that is used.

This operation informs RTI Connex Micro that the DDSDataWriter is no longer going to provide any information about the instance. This operation also indicates that RTI Connex Micro can locally remove all information regarding that instance. The application should not attempt to use the handle previously allocated to that instance after calling [FooDataWriter::unregister_instance\(\)](#).

The special value DDS_HANDLE_NIL can be used for the parameter handle. This indicates that the identity of the instance should be automatically deduced from the instance_data (by means of the key).

If `handle` is any value other than `DDS_HANDLE_NIL`, then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with `DDS_RETCODE_BAD_PARAMETER`.

RTI Connex Micro will not detect the error when the `handle` is any value other than `DDS_HANDLE_NIL`, corresponds to an instance that has been registered, but does not correspond to the instance deduced from the `instance_data` (by means of the key). RTI Connex Micro will treat as if the `unregister_instance()` operation is for the instance as indicated by the `handle`.

If after a `FooDataWriter::unregister_instance`, the application wants to modify (`FooDataWriter::write` or `FooDataWriter::dispose`) an instance, it has to register it again, or else use the special `handle` value `DDS_HANDLE_NIL`.

This operation does not indicate that the instance is deleted (that is the purpose of `FooDataWriter::dispose`). The operation `FooDataWriter::unregister_instance` just indicates that the `DDSDataWriter` no longer has anything to say about the instance. `DDSDataReader` entities that are reading the instance may receive a sample with `DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE` for the instance, unless there are other `DDSDataWriter` objects writing that same instance.

This operation can affect the ownership of the data instance (see `OWNERSHIP`). If the `DDSDataWriter` was the exclusive owner of the instance, then calling `unregister_instance()` will relinquish that ownership.

If `DDS_ReliabilityQosPolicy::kind` is set to `DDS_RELIABLE_RELIABILITY_QOS` and the unregistration would overflow the resource limits of this writer or of a reader, this operation may block for up to `DDS_ReliabilityQosPolicy::max_blocking_time`; if this writer is still unable to unregister after that period, this method will fail with `DDS_RETCODE_TIMEOUT`.

Parameters

<i>instance_data</i>	<<in>> The instance that should be unregistered. If <i>Foo</i> has a key and <i>instance_handle</i> is <code>DDS_HANDLE_NIL</code> , only the fields that represent the key are examined by the function. Otherwise, <i>instance_data</i> is not used. If <i>instance_data</i> is used, it must represent an instance that has been registered. Otherwise, this method may fail with <code>DDS_RETCODE_BAD_PARAMETER</code> .
<i>handle</i>	<<in>> represents the <i>instance</i> to be unregistered. If <i>Foo</i> has a key and <i>handle</i> is <code>DDS_HANDLE_NIL</code> , <i>handle</i> is not used and <i>instance</i> is deduced from <i>instance_data</i> . If <i>Foo</i> has no key, <i>handle</i> is not used. If <i>handle</i> is used, it must represent an instance that has been registered. Otherwise, this method may fail with <code>DDS_RETCODE_BAD_PARAMETER</code> .

Returns

One of the [Standard Return Codes](#), `DDS_RETCODE_TIMEOUT` or `DDS_RETCODE_NOT_ENABLED`

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataWriter::register_instance](#)
[FooDataWriter::unregister_instance_w_timestamp](#)

7.95.2.6 `DDS_ReturnCode_t FooDataWriter::unregister_instance_w_timestamp ( const Foo & instance_data, const DDS_InstanceHandle_t & handle, const DDS_Time_t & source_timestamp )`

<<cert>> Performs the same function as `FooDataWriter::unregister_instance` except that it also provides the value for the `source_timestamp`.

The provided `source_timestamp` potentially affects the relative order in which readers observe events from multiple writers.

The constraints on the values of the `handle` parameter and the corresponding error behavior are the same specified for the `FooDataWriter::unregister_instance` operation.

This operation may block and may time out ([DDS_RETCODE_TIMEOUT](#)) under the same circumstances described for the `unregister_instance` operation.

Parameters

<i>instance_data</i>	<<in>> The instance that should be unregistered. If Foo has a key and <code>instance_handle</code> is DDS_HANDLE_NIL , only the fields that represent the key are examined by the function. Otherwise, <code>instance_data</code> is not used. If <code>instance_data</code> is used, it must represent an instance that has been registered. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .
<i>handle</i>	<<in>> represents the <code>instance</code> to be unregistered. If Foo has a key and <code>handle</code> is DDS_HANDLE_NIL , <code>handle</code> is not used and <code>instance</code> is deduced from <code>instance_data</code> . If Foo has no key, <code>handle</code> is not used. If <code>handle</code> is used, it must represent an instance that has been registered. Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .
<i>source_timestamp</i>	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <code>register</code> , <code>unregister</code> , <code>dispose</code> , or <code>write</code> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataWriter::register_instance](#)
[FooDataWriter::unregister_instance](#)

7.95.2.7 [DDS_ReturnCode_t](#) FooDataWriter::dispose (const Foo & *instance_data*, const [DDS_InstanceHandle_t](#) & *handle*)

Requests the middleware to delete the data.

This operation is useful only for keyed data types. Using it for non-keyed types has no effect and reports no error.

The actual deletion is postponed until there is no more use for that data in the whole system.

Applications are made aware of the deletion by means of operations on the [DDSDataReader](#) objects that already knew that instance. [DDSDataReader](#) objects that didn't know the instance will never see it.

This operation does not modify the value of the instance. The `instance_data` parameter is passed just for the purposes of identifying the instance.

When this operation is used, RTI Connex Micro will automatically supply the value of the `source_timestamp` that is made available to [DDSDataReader](#) objects by means of the `source_timestamp` attribute inside the [DDS_SampleInfo](#).

The constraints on the values of the `handle` parameter and the corresponding error behavior are the same specified for the [FooDataWriter::unregister_instance](#) operation.

The special value [DDS_HANDLE_NIL](#) can be used for the parameter `instance_handle`. This indicates the identity of the instance should be automatically deduced from the `instance_data` (by means of the key).

If `handle` is any value other than [DDS_HANDLE_NIL](#), then it must correspond to an instance that has been registered. If there is no correspondence, the operation will fail with [DDS_RETCODE_BAD_PARAMETER](#).

RTI Connex Micro will not detect the error when the `handle` is any value other than [DDS_HANDLE_NIL](#), corresponds to an instance that has been registered, but does not correspond to the instance deduced from the

`instance_data` (by means of the `key`). RTI Connex Micro will treat as if the `dispose()` operation is for the instance as indicated by the `handle`.

This operation may block and time out (`DDS_RETCODE_TIMEOUT`) under the same circumstances described for `FooDataWriter::write()`.

If there are no instance resources left, this operation may fail with `DDS_RETCODE_OUT_OF_RESOURCES`. Calling `FooDataWriter::unregister_instance` may help freeing up some resources.

Parameters

<i>instance_data</i>	<<in>> The data to dispose. If <i>Foo</i> has a key and <code>instance_handle</code> is <code>DDS_HANDLE_NIL</code> , only the fields that represent the key are examined by the function. Otherwise, <code>instance_data</code> is not used. If <i>Foo</i> has a key, <code>instance_data</code> can be <code>NULL</code> only if <code>instance_handle</code> is not <code>DDS_HANDLE_NIL</code> . Otherwise, this method will fail with <code>DDS_RETCODE_BAD_PARAMETER</code> .
<i>handle</i>	<<in>> Either the handle returned by a previous call to <code>FooDataWriter::register_instance</code> , or else the special value <code>DDS_HANDLE_NIL</code> . If <i>Foo</i> has a key and <code>instance_handle</code> is <code>DDS_HANDLE_NIL</code> , <code>instance_handle</code> is not used and <code>instance</code> is deduced from <code>instance_data</code> . If <i>Foo</i> has no key, <code>instance_handle</code> is not used. If <code>handle</code> is used, it must represent a registered instance of type <i>Foo</i> . Otherwise, this method fail with <code>DDS_RETCODE_BAD_PARAMETER</code> .

Returns

One of the [Standard Return Codes](#), `DDS_RETCODE_TIMEOUT`, `DDS_RETCODE_OUT_OF_RESOURCES` or `DDS_RETCODE_NOT_ENABLED`.

See also

[FooDataWriter::dispose_w_timestamp](#)

7.95.2.8 `DDS_ReturnCode_t FooDataWriter::dispose_w_timestamp ( const Foo & instance_data, const DDS_InstanceHandle_t & handle, const DDS_Time_t & source_timestamp )`

<<cert>> Performs the same functions as `dispose` except that the application provides the value for the `source_timestamp` that is made available to `DDSDataReader` objects by means of the `source_timestamp` attribute inside the `DDS_SampleInfo`.

The constraints on the values of the `handle` parameter and the corresponding error behavior are the same specified for the `FooDataWriter::dispose` operation.

This operation may block and time out (`DDS_RETCODE_TIMEOUT`) under the same circumstances described for `FooDataWriter::write`.

If there are no instance resources left, this operation may fail with `DDS_RETCODE_OUT_OF_RESOURCES`. Calling `FooDataWriter::unregister_instance` may help freeing up some resources.

Parameters

<i>instance_data</i>	<<in>> The data to dispose. If <i>Foo</i> has a key and <code>instance_handle</code> is <code>DDS_HANDLE_NIL</code> , only the fields that represent the key are examined by the function. Otherwise, <code>instance_data</code> is not used.
<i>handle</i>	<<in>> Either the handle returned by a previous call to <code>FooDataWriter::register_instance</code> , or else the special value <code>DDS_HANDLE_NIL</code> . If <i>Foo</i> has a key and <code>instance_handle</code> is <code>DDS_HANDLE_NIL</code> , <code>instance_handle</code> is not used and <code>instance</code> is deduced from <code>instance_data</code> . If <i>Foo</i> has no key, <code>instance_handle</code> is not used. If <code>handle</code> is used, it must represent a registered instance of type <i>Foo</i> . Otherwise, this method may fail with <code>DDS_RETCODE_BAD_PARAMETER</code>

<i>source_timestamp</i>	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the DDSDDataReader objects by means of the <i>source_timestamp</i> attribute inside the DDS_SampleInfo .
-------------------------	---

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#), [DDS_RETCODE_OUT_OF_RESOURCES](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataWriter::dispose](#)

7.95.2.9 DDS_ReturnCode_t FooDataWriter::write_w_timestamp (const Foo & *instance_data*, const DDS_InstanceHandle_t & *handle*, const DDS_Time_t & *source_timestamp*)

<<cert>> Performs the same function as [FooDataWriter::write](#) except that it also provides the value for the *source_timestamp*.

Explicitly provides the timestamp that will be available to the [DDSDDataReader](#) objects by means of the *source_timestamp* attribute inside the [DDS_SampleInfo](#). (Refer to [DDS_SampleInfo](#) details)

The constraints on the values of the *handle* parameter and the corresponding error behavior are the same specified for the [FooDataWriter::write](#) operation.

This operation may block and time out ([DDS_RETCODE_TIMEOUT](#)) under the same circumstances described for [FooDataWriter::write](#).

If there are no instance resources left, this operation may fail with [DDS_RETCODE_OUT_OF_RESOURCES](#). Calling [FooDataWriter::unregister_instance](#) may help free up some resources.

This operation may fail with [DDS_RETCODE_BAD_PARAMETER](#) under the same circumstances described for the write operation.

Parameters

<i>instance_data</i>	<<in>> The data to write.
<i>handle</i>	<<in>> Either the handle returned by a previous call to FooDataWriter::register_instance , or else the special value DDS_HANDLE_NIL . If <i>Foo</i> has a key and <i>handle</i> is not DDS_HANDLE_NIL , <i>handle</i> must represent a registered instance of type <i>Foo</i> . Otherwise, this method may fail with DDS_RETCODE_BAD_PARAMETER .
<i>source_timestamp</i>	<<in>> The timestamp value must be greater than or equal to the timestamp value used in the last writer operation (used in a <i>register</i> , <i>unregister</i> , <i>dispose</i> , or <i>write</i> , with either the automatically supplied timestamp or the application provided timestamp). This timestamp may potentially affect the order in which readers observe events from multiple writers. This timestamp will be available to the DDSDDataReader objects by means of the <i>source_timestamp</i> attribute inside the DDS_SampleInfo .

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_TIMEOUT](#), [DDS_RETCODE_OUT_OF_RESOURCES](#) or [DDS_RETCODE_NOT_ENABLED](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[FooDataWriter::write](#)
[DDSDataReader](#)

7.96 FooSeq Struct Reference

`<<interface>>` `<<generic>>` `<<cert>>` A type-safe, ordered collection of elements. The type of these elements is referred to in this documentation as "Foo".

Public Member Functions

- [FooSeq & operator=](#) (const struct [FooSeq](#) &src_seq)
Copy elements from another sequence, resizing the sequence if necessary.
- bool [copy_no_alloc](#) (const struct [FooSeq](#) &src_seq)
Copy elements from another sequence, only if the destination sequence has enough capacity.
- bool [from_array](#) (const Foo array[], [DDS_Long](#) length)
Copy elements from an array of elements, resizing the sequence if necessary. The original contents of the sequence (if any) are replaced.
- bool [to_array](#) (Foo array[], [DDS_Long](#) length)
Copy elements to an array of elements. The original contents of the array (if any) are replaced.
- Foo & [operator\[\]](#) ([DDS_Long](#) i)
Get the i-th element of the sequence.
- const Foo & [operator\[\]](#) ([DDS_Long](#) i) const
Get the i-th element for a const sequence.
- [DDS_Long](#) [length](#) () const
`<<cert>>` *Get the logical length of this sequence.*
- bool [length](#) ([DDS_Long](#) new_length)
`<<cert>>` *Change the length of this sequence.*
- bool [ensure_length](#) ([DDS_Long](#) length, [DDS_Long](#) max)
Set the sequence to the desired length, and resize the sequence if necessary.
- [DDS_Long](#) [maximum](#) () const
`<<cert>>` *Get the current maximum number of elements that can be stored in this sequence.*
- bool [maximum](#) ([DDS_Long](#) new_max)
`<<cert>>` *Resize this sequence to a new desired maximum.*
- bool [loan_contiguous](#) (Foo *buffer, [DDS_Long](#) new_length, [DDS_Long](#) new_max)
Loan a contiguous buffer to this sequence.
- bool [unloan](#) ()
Return the loaned buffer in the sequence and set the maximum to 0.
- Foo * [get_contiguous_buffer](#) () const
Access the internal buffer of a sequence.
- bool [has_ownership](#) ()
Return the value of the owned flag.
- [~FooSeq](#) ()
Deallocate this sequence's buffer.
- [FooSeq](#) ([DDS_Long](#) new_max=0)
Create a sequence with the given maximum.
- [FooSeq](#) (const struct [FooSeq](#) &foo_seq)
Create a sequence by copying from an existing sequence.

Public Attributes

- [Foo * _contiguous_buffer](#)
Pointer to array of contiguous or discontinuous data.
- [DDS_UnsignedLong _maximum](#)
Maximum size of the sequence.
- [DDS_UnsignedLong _length](#)
Actual length of the sequence that contains data.
- [DDS_Long _element_size](#)
Size of data element in the sequence.
- [void * _token1](#)
Reserved for internal use.
- [void * _token2](#)
Reserved for internal use.
- [DDS_UnsignedLong max_alloc_size](#)
The maximum allocation for pointer elements (string, wstring) Only in debug libraraires.
- [DDS_Char _flags](#)
Reserved for internal use.

7.96.1 Detailed Description

[<<interface>>](#) [<<generic>>](#) [<<cert>>](#) A type-safe, ordered collection of elements. The type of these elements is referred to in this documentation as "Foo".

The sequence offers a subset of the methods defined by the standard OMG IDL to C++ mapping for sequences. We refer to an IDL `sequence<Foo>` as [FooSeq](#).

The state of a sequence is described by the properties 'maximum', and 'length'.

- The 'maximum' represents the size of the underlying buffer; this is the maximum number of elements it can possibly hold. It is returned by the [FooSeq::maximum\(\)](#) operation.
- The 'length' represents the actual number of elements it currently holds. It is returned by the [FooSeq::length\(\)](#) operation.

7.96.2 Constructor & Destructor Documentation

7.96.2.1 [FooSeq::~FooSeq \(\)](#)

Deallocate this sequence's buffer.

Precondition

(owned == [DDS_BOOLEAN_TRUE](#)). If this precondition is not met, no memory will be freed and an error will be logged.

Postcondition

maximum == 0 and the underlying buffer is freed.

See also

[FooSeq::maximum\(\)](#), [FooSeq::unloan](#)

7.96.2.2 FooSeq::FooSeq (DDS_Long new_max = 0)

Create a sequence with the given maximum.

This is a constructor for the sequence. The constructor will automatically allocate memory to hold new_max elements of type Foo.

This constructor will be used when the application creates a sequence using one of the following:

```
FooSeq mySeq(5);
// or
FooSeq mySeq;
// or
FooSeq* mySeqPtr = new FooSeq(5);
```

Postcondition

```
maximum == new_max
length == 0
owned == DDS_BOOLEAN_TRUE,
```

Parameters

<i>new_max</i>	Must be ≥ 0 . Otherwise the sequence will be initialized to a new_max=0.
----------------	---

7.96.2.3 FooSeq::FooSeq (const struct FooSeq & foo_seq)

Create a sequence by copying from an existing sequence.

This is a constructor for the sequence. The constructor will automatically allocate memory to hold foo_seq::maximum() elements of type Foo and will copy the current contents of foo_seq into the new sequence.

This constructor will be used when the application creates a sequence using one of the following:

```
FooSeq mySeq(foo_seq);
// or
FooSeq mySeq = foo_seq;
// or
FooSeq *mySeqPtr = new FooSeq(foo_seq);
```

Postcondition

```
this::maximum == foo_seq::maximum
this::length == foo_seq::length
this[i] == foo_seq[i] for  $0 \leq i < \text{foo\_seq::length}$ 
this::owned == DDS_BOOLEAN_TRUE
```

Note

If the pre-conditions are not met, the constructor will initialize the new sequence to a maximum of zero.

7.96.3 Member Function Documentation

7.96.3.1 FooSeq& FooSeq::operator= (const struct FooSeq & src_seq)

Copy elements from another sequence, resizing the sequence if necessary.

This method invokes copies the content of the specified sequence, after ensuring that this sequence has enough capacity to hold the elements to be copied.

It is important to know that no check for overlapping memory regions is performed, and copying sequences with overlapping memory regions has undefined behavior. For example, copying two string sequences where the strings are overlapping has undefined behavior.

RTI Connex Micro does not create overlapping memory regions.

NOTE: If this method is called on a sequence that has been created with `set_maximum_w_max` or `ensure_length_w_max`, it will automatically call `copy_w_max()`.

This operator is invoked when the following expression appears in the code:

```
target_seq = src_seq
```

Important: This method *will* allocate memory if `this::maximum < src_seq::length`.

Therefore, to programatically detect the successful completion of the operator it is recommended that the application first sets the length of this sequence to zero, makes the assignment, and then checks that the length of this sequence matches that of `src_seq`.

Parameters

<code>src_seq</code>	<< <i>in</i> >> the sequence from which to copy
----------------------	---

MT Safety:

This operation is not thread safe.

API Restriction:

This function *must* only be called after `DDSDomainParticipantFactory::get_instance`.

7.96.3.2 bool FooSeq::copy_no_alloc (const struct FooSeq & src_seq)

Copy elements from another sequence, only if the destination sequence has enough capacity.

Fill the elements in this sequence by copying the corresponding elements in `src_seq`. The original contents in this sequence are replaced via the element assignment operation (`Foo_copy()` function). By default, elements are discarded; 'delete' is not invoked on the discarded elements.

Precondition

```
this::maximum >= src_seq::length
this::owned == DDS_BOOLEAN_TRUE
```

Postcondition

```
this::length == src_seq::length
this[i] == src_seq[i] for 0 <= i < target_seq::length
this::owned == DDS_BOOLEAN_TRUE
```

Parameters

<code>src_seq</code>	<< <i>in</i> >> the sequence from which to copy
----------------------	---

Returns

`DDS_BOOLEAN_TRUE` if the sequence was successfully copied; `DDS_BOOLEAN_FALSE` otherwise.

Note

If the pre-conditions are not met, the operator will print a message to stdout and leave this sequence unchanged.

See also

`FooSeq::operator=`

7.96.3.3 `bool FooSeq::from_array ( const Foo array[], DDS_Long length )`

Copy elements from an array of elements, resizing the sequence if necessary. The original contents of the sequence (if any) are replaced.

Fill the elements in this sequence by copying the corresponding elements in `array`. The original contents in this sequence are replaced via the element assignment operation (`Foo_copy()` function). By default, elements are discarded; 'delete' is not invoked on the discarded elements.

Precondition

`this::owned == DDS_BOOLEAN_TRUE`

Postcondition

`this::length == length`
`this[i] == array[i] for 0 <= i < length`
`this::owned == DDS_BOOLEAN_TRUE`

Parameters

<i>array</i>	<<in>> The array of elements to be copy elements from
<i>length</i>	<<in>> The length of the array.

Returns

`DDS_BOOLEAN_TRUE` if the array was successfully copied; `DDS_BOOLEAN_FALSE` otherwise.

7.96.3.4 `bool FooSeq::to_array ( Foo array[], DDS_Long length )`

Copy elements to an array of elements. The original contents of the array (if any) are replaced.

Copy the elements of this sequence to the corresponding elements in the array. The original contents of the array are replaced via the element assignment operation (`Foo_copy()` function). By default, elements are discarded; 'delete' is not invoked on the discarded elements.

Parameters

<i>array</i>	<<in>> The array of elements to be filled with elements from this sequence
<i>length</i>	<<in>> The number of elements to be copied.

Returns

`DDS_BOOLEAN_TRUE` if the elements of the sequence were successfully copied; `DDS_BOOLEAN_FALSE` otherwise.

7.96.3.5 `Foo& FooSeq::operator[] ( DDS_Long i )`

Get the `i`-th element of the sequence.

This is the operator that is invoked when the application indexes into a non-const sequence:

```
myElement = mySequence[i];
mySequence[i] = myElement;
```

Parameters

<i>i</i>	index of element to access, must be ≥ 0 and less than <code>FooSeq::length()</code>
----------	--

Returns

the `i`-th element

MT Safety:

This operation is not thread safe.

7.96.3.6 `const Foo& FooSeq::operator[]( DDS_Long i ) const`

Get the `i`-th element for a `const` sequence.

This is the operator that is invoked when the application indexes into a `const` sequence:

```
myElement = mySequence[i];
```

Parameters

<code>i</code>	index of element to access, must be ≥ 0 and less than <code>FooSeq::length()</code>
----------------	--

Returns

the `i`-th element

MT Safety:

This operation is not thread safe.

7.96.3.7 `DDS_Long FooSeq::length ( ) const`

<<cert>> Get the logical length of this sequence.

Get the length that was last set, or zero if the length has never been set.

Returns

the length of the sequence

MT Safety:

This operation is not thread safe.

7.96.3.8 `bool FooSeq::length ( DDS_Long new_length )`

<<cert>> Change the length of this sequence.

This method does not allocate/deallocate memory.

The new length must not exceed the maximum of this sequence as returned by the `FooSeq::maximum()` operation. (Note that, if necessary, the maximum of this sequence can be increased manually by using the `FooSeq::maximum(-DDS_Long)` operation.)

The elements of the sequence are not modified by this operation. If the new length is larger than the original length, the new elements will be uninitialized; if the length is decreased, the old elements that are beyond the new length will physically remain in the sequence but will not be accessible.

Postcondition

`length = new_length.`

Parameters

<i>new_length</i>	the new desired length. This value must be non-negative and cannot exceed maximum of the sequence. In other words $0 \leq \text{new_length} \leq \text{maximum}$
-------------------	---

Returns

[DDS_BOOLEAN_TRUE](#) on success or [DDS_BOOLEAN_FALSE](#) on failure

MT Safety:

This operation is not thread safe.

7.96.3.9 bool FooSeq::ensure_length (DDS_Long length, DDS_Long max)

Set the sequence to the desired length, and resize the sequence if necessary.

If the current maximum is greater than the desired length, then sequence is not resized.

Otherwise if this sequence owns its buffer, the sequence is resized to the new maximum by freeing and re-allocating the buffer. However, if the sequence does not own its buffer, this operation will fail.

This function allows user to avoid unnecessary buffer re-allocation.

Precondition

$\text{length} \leq \text{max}$

$\text{owned} == \text{DDS_BOOLEAN_TRUE}$ if sequence needs to be resized

Postcondition

$\text{length} == \text{length}$

$\text{maximum} == \text{max}$ if resized

Parameters

<i>length</i>	<<in>> The new length that should be set. Must be ≥ 0 .
<i>max</i>	<<in>> If sequence need to be resized, this is the maximum that should be set. $\text{max} \geq \text{length}$

Returns

[DDS_BOOLEAN_TRUE](#) on success, [DDS_BOOLEAN_FALSE](#) if the preconditions are not met. In that case the sequence is not modified.

7.96.3.10 DDS_Long FooSeq::maximum () const

<<cert>> Get the current maximum number of elements that can be stored in this sequence.

The `maximum` of the sequence represents the maximum number of elements that the underlying buffer can hold. It does not represent the current number of elements.

The `maximum` is a non-negative number. It is initialized when the sequence is first created and can only be changed by mean of the `FooSeq::maximum(DDS_Long)` operation.

Returns

the current maximum of the sequence.

MT Safety:

This operation is not thread safe.

See also

[FooSeq::length\(\)](#)

7.96.3.11 bool FooSeq::maximum (DDS_Long new_max)

[<<cert>>](#) Resize this sequence to a new desired maximum.

This operation does nothing if the new desired maximum matches the current maximum.

If this sequence owns its buffer and the new maximum is not equal to the old maximum, then the existing buffer will be freed and re-allocated.

Precondition

owned == [DDS_BOOLEAN_TRUE](#)

Postcondition

owned == [DDS_BOOLEAN_TRUE](#)
length == MINIMUM(original length, new_max)

Parameters

<i>new_max</i>	Must be ≥ 0 .
----------------	--------------------

Returns

[DDS_BOOLEAN_TRUE](#) on success, [DDS_BOOLEAN_FALSE](#) if the preconditions are not met. In that case the sequence is not modified.

MT Safety:

This operation is not thread safe.

API Restriction:

This function *must* only be called after [DDSDomainParticipantFactory::get_instance](#).

7.96.3.12 bool FooSeq::loan_contiguous (Foo * buffer, DDS_Long new_length, DDS_Long new_max)

Loan a contiguous buffer to this sequence.

This operation changes the `owned` flag of the sequence to [DDS_BOOLEAN_FALSE](#) and also sets the underlying buffer used by the sequence. See the user's manual for more information about sequences and memory ownership.

Use this method if you want to manage the memory used by the sequence yourself. You must provide an array of elements and integers indicating how many elements are allocated in that array (i.e. the maximum) and how many elements are valid (i.e. the length). The sequence will subsequently use the memory you provide and will not permit it to be freed by a call to [FooSeq::maximum\(DDS_Long\)](#).

Once you have loaned a buffer to a sequence, make sure that you don't free it before calling [FooSeq::unloan](#): the next time you access the sequence, you will be accessing freed memory!

You can use this method to wrap stack memory with a sequence interface, thereby avoiding dynamic memory allocation. Create a [FooSeq](#) and an array of type Foo and then loan the array to the sequence:

```
Foo fooArray[10];
::FooSeq fooSeq;
fooSeq.loan_contiguous(fooArray, 0, 10);
```

By default, a sequence you create owns its memory unless you explicitly loan memory of your own to it. In a very few cases, RTI Connex Micro will return a sequence to you that has a loan; those cases are documented as

such. For example, if you call `FooDataReader::read` or `FooDataReader::take` and pass in sequences with no loan and no memory allocated, RTI Connex Micro will loan memory to your sequences which must be unloaned with `FooDataReader::return_loan`. See the documentation of those methods for more information.

Precondition

`FooSeq::maximum() == 0`; i.e. the sequence has no memory allocated to it.
`FooSeq::has_ownership == DDS_BOOLEAN_TRUE`; i.e. the sequence does not already have an outstanding loan

Postcondition

The sequence will store its elements in the buffer provided.

`FooSeq::has_ownership == DDS_BOOLEAN_FALSE`
`FooSeq::length() == new_length`
`FooSeq::maximum() == new_max`

Parameters

<i>buffer</i>	The new buffer that the sequence will use. Must point to enough memory to hold <code>new_max</code> elements of type <code>Foo</code> . It may be <code>NULL</code> if <code>new_max == 0</code> .
<i>new_length</i>	The desired new length for the sequence. It must be the case that that <code>0 <= new_length <= new_max</code> .
<i>new_max</i>	The allocated number of elements that could fit in the loaned buffer.

Returns

`DDS_BOOLEAN_TRUE` if `buffer` is successfully loaned to this sequence or `DDS_BOOLEAN_FALSE` otherwise. Failure only occurs due to failing to meet the pre-conditions. Upon failure the sequence remains unmodified.

See also

`FooSeq::unloan`

7.96.3.13 `bool FooSeq::unloan ( )`

Return the loaned buffer in the sequence and set the maximum to 0.

This method affects only the state of this sequence; it does not change the contents of the buffer in any way.

Only the user who originally loaned a buffer should return that loan, as the user may have dependencies on that memory known only to them. Unloaning someone else's buffer may cause unspecified problems. For example, suppose a sequence is loaning memory from a custom memory pool. A user of the sequence likely has no way to release the memory back into the pool, so unloaning the sequence buffer would result in a resource leak. If the user were to then re-loan a different buffer, the original creator of the sequence would have no way to discover, when freeing the sequence, that the loan no longer referred to its own memory and would thus not free the user's memory properly, exacerbating the situation and leading to undefined behavior.

Precondition

`owned == DDS_BOOLEAN_FALSE`

Postcondition

`owned == DDS_BOOLEAN_TRUE`
`maximum == 0`

Returns

[DDS_BOOLEAN_TRUE](#) if the preconditions were met. Otherwise [DDS_BOOLEAN_FALSE](#). The function only fails if the pre-conditions are not met, in which case it leaves the sequence unmodified.

See also

[FooSeq::loan_contiguous](#) [FooSeq::maximum\(DDS_Long\)](#)

7.96.3.14 Foo* FooSeq::get_contiguous_buffer () const

Access the internal buffer of a sequence.

Use this function to access the memory buffer that a sequence uses to store the values of its elements.

7.96.3.15 bool FooSeq::has_ownership ()

Return the value of the owned flag.

Returns

[DDS_BOOLEAN_TRUE](#) if sequence owns the underlying buffer, or [DDS_BOOLEAN_FALSE](#) if it has an outstanding loan.

7.96.4 Member Data Documentation

7.96.4.1 Foo* FooSeq::_contiguous_buffer

Pointer to array of contiguous or discontinuous data.

7.96.4.2 DDS_UnsignedLong FooSeq::_maximum

Maximum size of the sequence.

The allocated length of this sequence. It applies to whichever of the above buffers is non-NULL, if any. If both a NULL, its value must be 0.

If `_maximum == 0`, `_owned == true`.

7.96.4.3 DDS_UnsignedLong FooSeq::_length

Actual length of the sequence that contains data.

The current logical length of this sequence, i.e. the number of valid elements it contains. It applies to whichever of the above buffers is non-null, if any. If both are NULL, its value must be 0.

7.96.4.4 DDS_Long FooSeq::_element_size

Size of data element in the sequence.

Each element in the sequence has this size.

7.96.4.5 void* FooSeq::_token1

Reserved for internal use.

7.96.4.6 void* FooSeq::_token2

Reserved for internal use.

7.96.4.7 DDS_UnsignedLong FooSeq::max_alloc_size

The maximum allocation for pointer elements (string,wstring) Only in debug libraraires.

7.96.4.8 DDS_Char FooSeq::flags

Reserved for internal use.

7.97 FooTypeSupport Struct Reference

[`<<interface>>`](#) [`<<generic>>`](#) User data type specific interface.

Static Public Member Functions

- static bool [`initialize\_data`](#) (Foo *sample)
[`<<eXtension>>`](#) Initialize data type.
- static bool [`finalize\_data`](#) (Foo *sample)
[`<<eXtension>>`](#) Finalize data type.
- static Foo * [`create\_data`](#) ()
[`<<cert>>`](#) [`<<eXtension>>`](#) Create a data type and initialize it.
- static void [`delete\_data`](#) (Foo *sample)
[`<<eXtension>>`](#) Destroy a user data type instance.
- static bool [`copy\_data`](#) (Foo *dst_data, Foo *src_data)
[`<<eXtension>>`](#) Copy data type.
- static [`DDS\_ReturnCode\_t`](#) [`register\_type`](#) ([`DDSDomainParticipant`](#) *participant, const char *type_name)
[`<<cert>>`](#) Allows an application to communicate to RTI Connex Micro the existence of a data type.
- static [`DDS\_ReturnCode\_t`](#) [`unregister\_type`](#) ([`DDSDomainParticipant`](#) *participant, const char *type_name)
Allows an application to unregister a data type from RTI Connex Micro. After calling `unregister_type`, no further communication using that type is possible.

7.97.1 Detailed Description

[`<<interface>>`](#) [`<<generic>>`](#) User data type specific interface.

7.97.2 Member Function Documentation

7.97.2.1 static bool FooTypeSupport::initialize_data (Foo * sample) [static]

[`<<eXtension>>`](#) Initialize data type.

The *generated* implementation of the operation knows how to initialize a data type. This method is typically called to initialize a data type that is allocated on the stack.

Parameters

<code>sample</code>	<code><<inout>></code> Cannot be NULL.
-------------------------------------	--

Returns

One of the [Standard Return Codes](#)

See also

[`FooTypeSupport::finalize\_data`](#)

7.97.2.2 static bool FooTypeSupport::finalize_data (Foo * sample) [static]

[`<<eXtension>>`](#) Finalize data type.

The *generated* implementation of the operation knows how to finalize a data type. This method is typically called to finalize a data type that has previously been initialized.

Parameters

<i>sample</i>	<< <i>in</i> >> Cannot be NULL.
---------------	---------------------------------

Returns

One of the [Standard Return Codes](#)

See also

[FooTypeSupport::initialize_data](#)

7.97.2.3 static Foo* FooTypeSupport::create_data () [static]

<<*cert*>> <<*eXtension*>> Create a data type and initialize it.

Memory for a data type is allocated from memory using internal RTI Connex Micro memory allocation and deallocation APIs. The implementation of these APIs are specific to a particular platform. For this reason it is of the utmost importance that [FooTypeSupport::create_data](#) is not called until [DDSDomainParticipantFactory::get_instance](#) has been called successfully. The [DDSDomainParticipantFactory](#) initializes the RTI Connex Micro system, including any memory management initialization.

The *generated* implementation of the operation knows how to instantiate a data type and initialize it properly.

All memory for the type is deeply allocated.

Returns

newly created data type

MT Safety:

This operation is thread safe.

API Restriction:

This function *must* only be called after [DDSDomainParticipantFactory::get_instance](#).

See also

[FooTypeSupport::delete_data](#)

7.97.2.4 static void FooTypeSupport::delete_data (Foo * *sample*) [static]

<<*eXtension*>> Destroy a user data type instance.

The *generated* implementation of the operation knows how to destroy a data type and return all resources.

Parameters

<i>sample</i>	<< <i>in</i> >> Cannot be NULL.
---------------	---------------------------------

Returns

One of the [Standard Return Codes](#)

See also

[FooTypeSupport::create_data](#)

7.97.2.5 `static bool FooTypeSupport::copy_data ( Foo * dst_data, Foo * src_data ) [static]`

<<*eXtension*>> Copy data type.

The *generated* implementation of the operation knows how to copy value of a data type.

Parameters

<i>dst_data</i>	<< <i>inout</i> >> Data type to copy value to. Cannot be NULL.
<i>src_data</i>	<< <i>in</i> >> Data type to copy value from. Cannot be NULL.

Returns

One of the [Standard Return Codes](#)

7.97.2.6 `static DDS_ReturnCode_t FooTypeSupport::register_type ( DDSDomainParticipant * participant, const char * type_name ) [static]`

<<*cert*>> Allows an application to communicate to RTI Connex Micro the existence of a data type.

The *generated* implementation of the operation embeds all the knowledge that has to be communicated to the middleware in order to make it able to manage the contents of data of that type. This includes in particular the key definition that will allow RTI Connex Micro to distinguish different instances of the same type.

The same [DDS_TYPESUPPORT_CPP](#) can be registered multiple times with a [DDSDomainParticipant](#) using the same or different values for the *type_name*. If *register_type* is called multiple times on the same [DDS_TYPESUPPORT_CPP](#) with the same [DDSDomainParticipant](#) and *type_name*, the second (and subsequent) registrations are ignored by the operation fails with [DDS_RETCODE_OK](#).

Precondition

Cannot use the same *type_name* to register two different [DDS_TYPESUPPORT_CPP](#) with the same [DDSDomainParticipant](#), or else the operation will fail and [DDS_RETCODE_PRECONDITION_NOT_MET](#) will be returned.

Parameters

<i>participant</i>	<< <i>in</i> >> the DDSDomainParticipant to register the data type <code>Foo</code> with. Cannot be NULL.
<i>type_name</i>	<< <i>in</i> >> the type name under with the data type <code>Foo</code> is registered with the participant; this type name is used when creating a new DDSTopic . (See DDSDomainParticipant::create_topic .) The name may not be NULL or longer than 255 characters.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_PRECONDITION_NOT_MET](#) or [DDS_RETCODE_OUT_OF_RESOURCES](#).

MT Safety:

This operation is thread safe. However, note that the arguments are not protected from being modified by other threads and must not be modified until the call returns.

See also

[DDSDomainParticipant::create_topic](#)

7.97.2.7 `static DDS_ReturnCode_t FooTypeSupport::unregister_type ( DDSDomainParticipant * participant, const char * type_name ) [static]`

Allows an application to unregister a data type from RTI Connex Micro. After calling `unregister_type`, no further communication using that type is possible.

The *generated* implementation of the operation removes all the information about a type from RTI Connex Micro. No further communication using that type is possible.

Precondition

A type with `type_name` is registered with the participant and all [DDSTopic](#) objects referencing the type have been destroyed. If the type is not registered with the participant, or if any [DDSTopic](#) is associated with the type, the operation will fail with [DDS_RETCODE_ERROR](#).

Postcondition

All information about the type is removed from RTI Connex Micro. No further communication using this type is possible.

Parameters

<i>participant</i>	<< <i>in</i> >> the DDSDomainParticipant to unregister the data type <code>Foo</code> from. Cannot be NULL.
<i>type_name</i>	<< <i>in</i> >> the type name under with the data type <code>Foo</code> is registered with the participant. The name should match a name that has been previously used to register a type with the participant. Cannot be NULL.

Returns

One of the [Standard Return Codes](#), [DDS_RETCODE_BAD_PARAMETER](#) or [DDS_RETCODE_ERROR](#)

MT Safety:

SAFE.

See also

[DDSDomainParticipant::register_type](#)

7.98 NDDS_Discovery_Property Struct Reference

Specifies internal information used to store discovery information about a [DDSDomainParticipant](#) in the registry.

7.98.1 Detailed Description

Specifies internal information used to store discovery information about a [DDSDomainParticipant](#) in the registry.

7.99 NDDS_Type_Plugin Struct Reference

<<*interface*>> Type plugin interface.

7.99.1 Detailed Description

<<*interface*>> Type plugin interface.

7.100 `NDDS_Type_PluginVersion` Struct Reference

Data type used to keep track of versioning information about a [NDDS_Type_Plugin](#).

Public Attributes

- `char majorRev`
Major version number.
- `char minorRev`
Minor version number.

7.100.1 Detailed Description

Data type used to keep track of versioning information about a [NDDS_Type_Plugin](#).

7.100.2 Member Data Documentation

7.100.2.1 `char NDDS_Type_PluginVersion::majorRev`

Major version number.

7.100.2.2 `char NDDS_Type_PluginVersion::minorRev`

Minor version number.

7.101 `OSAPI_LogProperty` Struct Reference

Configuration of logging functionality.

Public Attributes

- `RTI_SIZE_T max_buffer_size`
The maximum number of bytes allocated to the log buffer.
- `RTI_UINT32 log_detail`
Bitmap to control the fidelity of what is stored in the log entry.
- `OSAPI_Log_write_buffer_T write_buffer`
Function pointer to output a log/trace buffer.

7.101.1 Detailed Description

Configuration of logging functionality.

7.101.2 Member Data Documentation

7.101.2.1 `RTI_SIZE_T OSAPI_LogProperty::max_buffer_size`

The maximum number of bytes allocated to the log buffer.

Log entries are of variable length. When this limit has been reached, no more log entries can be stored unless the buffer is cleared.

7.101.2.2 RTI_UINT32 OSAPI_LogProperty::log_detail

Bitmap to control the fidelity of what is stored in the log entry.

7.101.2.3 OSAPI_Log_write_buffer_T OSAPI_LogProperty::write_buffer

Function pointer to output a log/trace buffer.

7.102 OSAPI_MutexBase Struct Reference

7.102.1 Detailed Description

The Mutex base-class

7.103 OSAPI_PortProperty Struct Reference

AutoSAR System properties.

Public Attributes

- uint32 [timer_resolution_ms](#)
Timer resolution in ms. Frequency at which the timer task is executed.
- uint32 [number_of_heap_areas](#)
Heap used by RTI Connex Micro can span across a number of different areas. This field indicates the number of areas in the heap memory. Minimum valid value is 1.
- uint32 const * [heap_area_size](#)
Pointer to size of each heap area. As many elements as indicated in field 'number_of_heap_areas'.
- char *const * [heap_area](#)
Pointer to heap areas. As many elements as indicated in field 'number_of_heap_areas'.
- [OSAPI_Autosar_SyncKind_T](#) [sync_type](#)
Kind of synchronization used.
- ResourceType [first_resource_id](#)
First resource id that RTI Connex Micro uses to synchronize the concurrent access to shared resources. This is only used in case Resources are configured as synchronization method.
- ResourceType [last_resource_id](#)
Last resource id that RTI Connex Micro uses to synchronize the concurrent access to shared resources. This is only used in case Resources are configured as synchronization method.
- uint16 [spinlock_id](#)
Spinlock id used in case sync_type is OSAPI_AUTOSAR_SYNCKIND_SPINLOCK. Type SpinlockIdType could be used for this field but some AutoSAR implementations don't include this type in case spinlock module is not added. To avoid ifdef we use uint16.
- uint32 [semaphore_max_count](#)
The maximum number of semaphores.
- EventMaskType [first_give_event](#)
First give event id used for semaphore implementation.
- EventMaskType [first_timeout_event](#)
First timeout event id used for semaphore implementation.
- AlarmType [first_alarm](#)
First alarm id used for semaphore implementation.
- boolean [use_socket_owner](#)
TRUE if SocketOwner is configured in AutoSAR Tcplp module configuration. In that case RTI Connex Micro will use sockets to send and receive UDP packets. Otherwise a function pointer will be used to send packets.
- uint8 [max_receive_sockets](#)

- Maximum number of sockets that RTI Connex Micro can use to receive UDP packets.*

 - uint8 [number_of_rcv_buffers](#)

Number of receive buffers of size specified in field 'rcv_buffer_size'.

 - uint32 [rcv_buffer_size](#)
- Size of each receive buffer.*
- [NETIO_Autosar_Tcplp_get_socket](#) [get_socket](#)
- Pointer to the function that can create a socket.*
- [NETIO_Autosar_Tcplp_send_dyn_data](#) [send_data](#)
- Pointer to the function that can send data.*
- Tcplp_LocalAddrIdType [max_local_addr_id](#)
- Maximum configured IP address identifier.*
- boolean [use_udp_thread](#)
- Set to TRUE to use a thread to receive data.*
- TaskType [udp_receive_task_id](#)
- The UDP receive task id.*
- EventMaskType [udp_packet_received_event](#)
- Event used to notify the UDP task that an incoming packet has been received.*
- uint32 [log_dst_address](#)
 - uint16 [log_dst_port](#)

7.103.1 Detailed Description

AutoSAR System properties.

7.103.2 Member Data Documentation

7.103.2.1 uint32 OSAPI_PortProperty::timer_resolution_ms

Timer resolution in ms. Frequency at which the timer task is executed.

7.103.2.2 uint32 OSAPI_PortProperty::number_of_heap_areas

Heap used by RTI Connex Micro can span across a number of different areas. This field indicates the number of areas in the heap memory. Minimum valid value is 1.

7.103.2.3 uint32 const* OSAPI_PortProperty::heap_area_size

Pointer to size of each heap area. As many elements as indicated in field 'number_of_heap_areas'.

7.103.2.4 char* const* OSAPI_PortProperty::heap_area

Pointer to heap areas. As many elements as indicated in field 'number_of_heap_areas'.

7.103.2.5 OSAPI_Autosar_SyncKind_T OSAPI_PortProperty::sync_type

Kind of synchronization used.

7.103.2.6 ResourceType OSAPI_PortProperty::first_resource_id

First resource id that RTI Connex Micro uses to synchronize the concurrent access to shared resources. This is only used in case Resources are configured as synchronization method.

7.103.2.7 ResourceType OSAPI_PortProperty::last_resource_id

Last resource id that RTI Connex Micro uses to synchronize the concurrent access to shared resources. This is only used in case Resources are configured as synchronization method.

7.103.2.8 uint16 OSAPI_PortProperty::spinlock_id

Spinlock id used in case sync_type is OSAPI_AUTOSAR_SYNCKIND_SPINLOCK. Type SpinlockIdType could be used for this field but some AutoSAR implementations don't include this type in case spinlock module is not added. To avoid ifdef we use uint16.

7.103.2.9 uint32 OSAPI_PortProperty::semaphore_max_count

The maximum number of semaphores.

Maximum number of semaphores that can be created. As many give and timeout events and alarms as indicated by this field must be configured in Autosar. Give events shall have 'consecutive' event IDs, starting at first_give_event (e.g. 1, 2, 4). Timeout events shall have 'consecutive' event IDs starting at first_timeout_event (e.g. 8, 16, 32). Alarms shall have 'consecutive' IDs starting at first_alarm (e.g. 1, 2, 3).

A Waitset object needs two semaphores. In case Waitsets are not used this field can be set to 0.

Semaphore implementation uses two events and one alarm for each semaphore that is created. One event, the give event, is set in the semaphore give() method. The other event, the timeout event, is used to signal a timeout in the semaphore. The alarm must be configured to set the timeout event. The semaphore take() operation starts the alarm and waits for any of the events.

The alarm must be configured as 'RELATIVE' and to set the timeout event when it expires. The counter used to trigger the alarm must be configured with one tick per millisecond and in case this is not done properly the semaphore timeout will occur sooner or later than expected.

The semaphore take() method in the semaphore can be used only from one task.

In case semaphores are not needed it is possible to just set fields first_give_event, first_timeout_event, first_alarm and semaphore_max_count to 0.

7.103.2.10 EventMaskType OSAPI_PortProperty::first_give_event

First give event id used for semaphore implementation.

7.103.2.11 EventMaskType OSAPI_PortProperty::first_timeout_event

First timeout event id used for semaphore implementation.

7.103.2.12 AlarmType OSAPI_PortProperty::first_alarm

First alarm id used for semaphore implementation.

7.103.2.13 boolean OSAPI_PortProperty::use_socket_owner

TRUE if SocketOwner is configured in AutoSAR TcpIp module configuration. In that case RTI Connex Micro will use sockets to send and receive UDP packets. Otherwise a function pointer will be used to send packets.

If set to 'TRUE' fields 'number_of_socket' and 'get_socket' must be configured properly (can't be NULL_PTR).

If set to 'FALSE' field 'send_data' needs to be configured properly (can't be NULL_PTR).

7.103.2.14 uint8 OSAPI_PortProperty::max_receive_sockets

Maximum number of sockets that RTI Connex Micro can use to receive UDP packets.

Maximum number of sockets that RTI Connex Micro can use to receive UDP packets. This number depends on the number of participants created and the UDP transport configuration.

If SocketOwner is used for each participant 2 sockets are normally needed to receive UDP packets: - Socket to receive unicast discovery data. - Socket to receive unicast user data.

If SocketOwner is not used sockets are not used but this field must have at least a value of 1.

7.103.2.15 uint8 OSAPI_PortProperty::number_of_rcv_buffers

Number of receive buffers of size specified in field 'rcv_buffer_size'.

RTI Connex Micro can use internal socket buffers. This is normally needed in case the received callback indication is called in the context of an ISR. Number of receive buffers of size specified in field 'rcv_buffer_size'. This is only used when field 'use_udp_thread' is TRUE.

7.103.2.16 uint32 OSAPI_PortProperty::rcv_buffer_size

Size of each receive buffer.

7.103.2.17 NETIO_Autosar_Tcplp_get_socket OSAPI_PortProperty::get_socket

Pointer to the function that can create a socket.

Pointer to the function that can create a socket. This function is used to create sockets to send and receive UDP packets. This field is used only in case a SocketOwner is configured, field 'use_socket_owner' is set to TRUE. Otherwise this field can be NULL_PTR.

7.103.2.18 NETIO_Autosar_Tcplp_send_dyn_data OSAPI_PortProperty::send_data

Pointer to the function that can send data.

Pointer to the function that can send data. This field is used only in case a SocketOwner is not configured, when field 'use_socket_owner' is FALSE. Otherwise this field can be NULL_PTR.

7.103.2.19 Tcplp_LocalAddrIdType OSAPI_PortProperty::max_local_addr_id

Maximum configured IP address identifier.

Maximum configured IP address identifier representing the range for the configured IP addresses. The maximum local address id configured in the Tcplp Basic Software Module. The Autosar port uses this information to locate the local addresses that are used for DDS communications.

```
Maximum configured IP address identifier representing the range top for
the configured IP addresses [0..max_local_addr_id]
```

7.103.2.20 boolean OSAPI_PortProperty::use_udp_thread

Set to TRUE to use a thread to receive data.

Set to TRUE to use a thread to receive data. If this field is set to FALSE, the incoming UDP packets will be processed in the callback notification function. If this field is set to TRUE, the callback notification function will copy the UDP packet in an internal buffer and set an event for the UDP thread to process the packet.

```
Set to TRUE to use a thread to receive data. If this field is set to
FALSE, the incoming UDP packets will be processed in the callback
notification function. If this field is set to TRUE, the callback
notification function will copy the UDP packet in an internal buffer and
set an event for the UDP thread to process the packet.
```

7.103.2.21 TaskType OSAPI_PortProperty::udp_receive_task_id

The UDP receive task id.

UDP receive task id. This must be configured with the ID of the task NETIO_Autosar_udp_receive_task. This field must be configured only in case field 'use_udp_thread' is set to TRUE.

7.103.2.22 EventMaskType OSAPI_PortProperty::udp_packet_received_event

Event used to notify the UDP task that an incoming packet has been received.

Event used to notify the UDP task that an incoming packet has been received. This event is used only in case UDP task is used, that is when field 'use_udp_thread' is set to TRUE.

7.104 OSAPI_System Struct Reference

<<cert>> System API.

Public Attributes

- OSAPI_System_start_timer_T [start_timer](#)
This field must not be modified.
- OSAPI_System_stop_timer_T [stop_timer](#)
This field must not be modified.
- OSAPI_System_get_timer_resolution_T [get_timer_resolution](#)
This field must not be modified.
- OSAPI_System_get_time_T [get_time](#)
This field must not be modified.
- OSAPI_System_initialize_T [initialize](#)
This field must not be modified.
- OSAPI_System_finalize_T [finalize](#)
This field must not be modified.
- OSAPI_System_generate_uuid_T [generate_uuid](#)
This field must not be modified.
- OSAPI_System_get_hostname_T [get_hostname](#)
This field must not be modified.
- OSAPI_System_get_ticktime_T [get_ticktime](#)
This field must not be modified.

7.104.1 Detailed Description

<<cert>> System API.

The methods in this interface are called by the corresponding OSAPI_System function. For example, OSAPI_System_get_time() calls system->[get_time\(\)](#).

7.104.2 Member Data Documentation

7.104.2.1 OSAPI_System_start_timer_T OSAPI_System::start_timer

This field must not be modified.

7.104.2.2 OSAPI_System_stop_timer_T OSAPI_System::stop_timer

This field must not be modified.

7.104.2.3 OSAPI_System_get_timer_resolution_T OSAPI_System::get_timer_resolution

This field must not be modified.

7.104.2.4 OSAPI_System_get_time_T OSAPI_System::get_time

This field must not be modified.

7.104.2.5 OSAPI_System_initialize_T OSAPI_System::initialize

This field must not be modified.

7.104.2.6 OSAPI_System_finalize_T OSAPI_SystemI::finalize

This field must not be modified.

7.104.2.7 OSAPI_System_generate_uuid_T OSAPI_SystemI::generate_uuid

This field must not be modified.

7.104.2.8 OSAPI_System_get_hostname_T OSAPI_SystemI::get_hostname

This field must not be modified.

7.104.2.9 OSAPI_System_get_ticktime_T OSAPI_SystemI::get_ticktime

This field must not be modified.

7.105 OSAPI_SystemProperty Struct Reference

<<cert>> The System Properties.

Public Attributes

- struct [OSAPI_TimerProperty](#) timer_property
- char [hostname](#) [OSAPI_SYSTEM_MAX_HOSTNAME]
The hostname for the node.
- struct [OSAPI_PortProperty](#) port_property
Port specific properties.

7.105.1 Detailed Description

<<cert>> The System Properties.

System properties

7.105.2 Member Data Documentation

7.105.2.1 struct OSAPI_TimerProperty OSAPI_SystemProperty::timer_property

The timer properties control the time thread running the internal RTI Connex Micro timer tick. Note that each port of RTI Connex Micro may choose to implement the timer tick differently. Please refer to platform specific information for details.

7.105.2.2 char OSAPI_SystemProperty::hostname[OSAPI_SYSTEM_MAX_HOSTNAME]

The hostname for the node.

RTI Connex Micro uses this name when it announces itself on the network. This name is used by other RTI products. The hostname must not exceed OSAPI_SYSTEM_MAX_HOSTNAME, including the \0 character.

7.105.2.3 struct OSAPI_PortProperty OSAPI_SystemProperty::port_property

Port specific properties.

A port of RTI Connex Micro may add additional properties beyond the standard properties defined by RTI Connex Micro. These properties are set in this attribute. Note that each port defines their own properties. Please refer to platform specific documentation for the port properties.

7.106 OSAPI_ThreadProperty Struct Reference

Properties for thread creation.

Public Attributes

- [RTI_UINT32 stack_size](#)
Hint on required stack-size in bytes. The special value OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE means do not set the stack-size and rely on the OS defaults.
- [RTI_INT32 priority](#)
Hint on required priority. A value ≥ 0 is used as is, and negative value is mapped according to the platform documentation.
- [OSAPI_ThreadOptions options](#)
Hint for thread creation to the OS.

7.106.1 Detailed Description

Properties for thread creation.

7.106.2 Member Data Documentation

7.106.2.1 RTI_UINT32 OSAPI_ThreadProperty::stack_size

Hint on required stack-size in bytes. The special value OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE means do not set the stack-size and rely on the OS defaults.

7.106.2.2 RTI_INT32 OSAPI_ThreadProperty::priority

Hint on required priority. A value ≥ 0 is used as is, and negative value is mapped according to the platform documentation.

7.106.2.3 OSAPI_ThreadOptions OSAPI_ThreadProperty::options

Hint for thread creation to the OS.

7.107 OSAPI_TimerProperty Struct Reference

Timer properies.

Public Attributes

- struct [OSAPI_ThreadProperty thread](#)
The thread settings for the timer thread, if a thread is created.

7.107.1 Detailed Description

Timer properies.

7.107.2 Member Data Documentation

7.107.2.1 struct OSAPI_ThreadProperty OSAPI_TimerProperty::thread

The thread settings for the timer thread, if a thread is created.

7.108 RHSMHistoryFactory Class Reference

[`<<eXtension>>`](#) [`<<cert>>`](#) A reader history factory that will be used by the DataReader to create a history cache.

Static Public Member Functions

- static struct
RT_ComponentFactoryI * [`get\_interface`](#) ()
Gets the singleton instance of the RHSM reader history factory.

7.108.1 Detailed Description

[`<<eXtension>>`](#) [`<<cert>>`](#) A reader history factory that will be used by the DataReader to create a history cache.

7.108.2 Member Function Documentation

7.108.2.1 static struct RT_ComponentFactoryI* RHSMHistoryFactory::get_interface () [static],[read]

Gets the singleton instance of the RHSM reader history factory.

This function gets the singleton instance of the RHSM history plugin factory that is used by the middleware to create a ReaderHistory plugin. The singleton instance of the factory must be registered with the name "rh" in the DomainParticipantFactory.

Returns

Pointer to RHSM history plugin factory

MT Safety:

This operation is thread safe.

7.109 RTI_ManagementQosPolicy Struct Reference

[`<<cert>>`](#) Policy used determine how [DDSDDataReader](#), [DDSDDataWriter](#), [DDSPublisher](#), and [DDSSubscriber](#) is managed.

Public Attributes

- [`DDS\_Boolean is\_anonymous`](#)
Whether the entity is an anonymous entity or not.

7.109.1 Detailed Description

[`<<cert>>`](#) Policy used determine how [DDSDDataReader](#), [DDSDDataWriter](#), [DDSPublisher](#), and [DDSSubscriber](#) is managed.

7.109.2 Member Data Documentation

7.109.2.1 DDS_Boolean RTI_ManagementQosPolicy::is_anonymous

Whether the entity is an anonymous entity or not.

Properties:

[Changeable](#) = NO

If [DDS_BOOLEAN_TRUE](#), the created entity is anonymous and state, such as which sequence number to send with, is not maintained. This option should only be used to create a built-in ParticipantBuiltTopicDataWriter.

[default] [DDS_BOOLEAN_FALSE](#)

7.110 RTPS_Checksum Union Reference

[<<eXtension>>](#) [<<cert>>](#) RTPS checksum type

Public Attributes

- [RTI_UINT32 checksum32](#)
A 32-bit unsigned checksum in host endianness order.
- [RTI_UINT64 checksum64](#)
A 64-bit unsigned integer checksum in host endianness order.
- [RTI_UINT8 checksum128](#) [16]
A 128-bit checksum as a 16-byte array.

7.110.1 Detailed Description

[<<eXtension>>](#) [<<cert>>](#) RTPS checksum type

7.111 RTPS_ChecksumClass Struct Reference

[<<eXtension>>](#) [<<cert>>](#) Definition of a checksum function

Public Attributes

- RTPS_ChecksumClassId_T [class_id](#)
The checksum class ID.
- void * [context](#)
User defined context.
- [RTPS_ChecksumCalculate_T checksum_calculate](#)
User defined function to calculate a checksum.

7.111.1 Detailed Description

[<<eXtension>>](#) [<<cert>>](#) Definition of a checksum function

7.112 RTPS_ChecksumProperty Struct Reference

[<<eXtension>>](#) [<<cert>>](#) Checksum properties for the RTPS plugin

Public Attributes

- [RTPS_ChecksumClass_T builtin_checksum32_class](#)
The builtin 32-bit checksum class.
- [RTPS_ChecksumClass_T builtin_checksum64_class](#)
The builtin 64-bit checksum class.
- [RTPS_ChecksumClass_T builtin_checksum128_class](#)
The builtin 128-bit checksum class.
- [RTPS_ChecksumTxMode_T checksum_tx_mode](#)
The transmit mode for checksums.
- [RTI_BOOL allow_builtin_override](#)
If TRUE, allow overriding the builtin checksum functions.

7.112.1 Detailed Description

<<[eXtension](#)>> <<[cert](#)>> Checksum properties for the RTPS plugin

7.113 RTPS_InterfaceFactoryProperty Struct Reference

<<[eXtension](#)>> <<[cert](#)>> Properties for the RTPS Transport.

Public Attributes

- struct [RTPS_ChecksumProperty checksum](#)
The checksum properties for RTPS.

7.113.1 Detailed Description

<<[eXtension](#)>> <<[cert](#)>> Properties for the RTPS Transport.

7.114 RTRegistry Class Reference

<<[eXtension](#)>> A run-time component factory.

Public Member Functions

- bool [register_component](#) (const char *name, RT_ComponentFactoryI *intf, RT_ComponentFactoryProperty *property, RT_ComponentFactoryListener *listener)
<<[cert](#)>> *Register a factory of RT components*
- bool [unregister](#) (const char *name, RT_ComponentFactoryProperty **property, RT_ComponentFactoryListener **listener)
<<[cert](#)>> *Remove a component factory from the registry*

7.114.1 Detailed Description

<<[eXtension](#)>> A run-time component factory.

7.114.2 Member Function Documentation

7.114.2.1 `bool RTRegistry::register_component ( const char * name, RT_ComponentFactoryI * intf, RT_ComponentFactoryProperty * property, RT_ComponentFactoryListener * listener )`

<<cert>> Register a factory of RT components

In order to make a component available, it is necessary to register a factory for the components. The factory is responsible for creating and deleting components.

The name of the factory must be unique. Otherwise, if a factory already exists with the the same name, the registration will fail.

Parameters

<i>name</i>	<<in>> Name of the component factory
<i>intf</i>	<<in>> Pointer to the implementation of the factory interface
<i>property</i>	<<in>> Properties with which to create the factory
<i>listener</i>	<<in>> The listener to install with the factory

Returns

RTI_TRUE on success, RTI_FALSE on failure

MT Safety:

This operation is not thread safe.

API Restriction:

This function *must* only be called after [DDSDomainParticipantFactory::get_instance](#).

See also

[RTRegistry::unregister](#)

7.114.2.2 `bool RTRegistry::unregister ( const char * name, RT_ComponentFactoryProperty ** property, RT_ComponentFactoryListener ** listener )`

<<cert>> Remove a component factory from the registry

After a component factory is unregistered, it will no longer be possible to create or delete components with the factory.

Parameters

<i>name</i>	<<in>> Name of the component factory
<i>property</i>	<<in>> Properties with which the factory was created
<i>listener</i>	<<in>> The listener installed with the factory

Returns

RTI_TRUE on success, RTI_FALSE on failure

MT Safety:

This operation is not thread safe.

See also

[RTRegistry::register_component](#)

7.115 UDP_InterfaceFactoryProperty Struct Reference

<<eXtension>> <<cert>> Properties for the UDP Transport.

Public Attributes

- struct REDA_StringSeq [allow_interface](#)
Sequence of allowed interface names.
- struct REDA_StringSeq [deny_interface](#)
Sequence of denied interface names. This list is checked after the allow_interface list.
- [RTI_INT32 max_send_buffer_size](#)
The size of the send socket buffer.
- [RTI_INT32 max_receive_buffer_size](#)
The size of the receive socket buffer.
- [RTI_INT32 max_message_size](#)
The maximum size of the message which can be sent or received.
- [RTI_INT32 max_send_message_size](#)
The maximum size of the message which can be sent. This is only a hint and not enforced.
- [RTI_INT32 multicast_ttl](#)
The maximum TTL.
- struct [UDP_NatEntrySeq nat](#)
Configure network address translation (NAT).
- struct [UDP_InterfaceTableEntrySeq if_table](#)
The interface table if interfaces are added manually.
- REDA_String_T [multicast_interface](#)
The network interface to use to send to multicast.
- [RTI_BOOL is_default_interface](#)
If this should be considered the default UDP interfaces if no other UDP interface is found to handle a route.
- struct [OSAPI_ThreadProperty recv_thread](#)
Thread properties for each receive thread created by this NETIO interface.
- struct [UDP_TransformRuleSeq source_rules](#)
Rules for how to transform received UDP payloads based on the source address.
- struct [UDP_TransformRuleSeq destination_rules](#)
Rules for how to transform sent UDP payloads based on the destination address.
- [UDP_TransformUdpMode_T transform_udp_mode](#)
Determines how regular UDP is supported when transformations are supported.
- [RTI_INT32 transform_locator_kind](#)
The locator to use for locators that have transformations.

7.115.1 Detailed Description

<<eXtension>> <<cert>> Properties for the UDP Transport.

7.115.2 Member Data Documentation

7.115.2.1 struct REDA_StringSeq UDP_InterfaceFactoryProperty::allow_interface

Sequence of allowed interface names.

7.115.2.2 struct REDA_StringSeq UDP_InterfaceFactoryProperty::deny_interface

Sequence of denied interface names. This list is checked after the allow_interface list.

7.115.2.3 RTI_INT32 UDP_InterfaceFactoryProperty::max_send_buffer_size

The size of the send socket buffer.

7.115.2.4 RTI_INT32 UDP_InterfaceFactoryProperty::max_receive_buffer_size

The size of the receive socket buffer.

7.115.2.5 RTI_INT32 UDP_InterfaceFactoryProperty::max_message_size

The maximum size of the message which can be sent or received.

The [UDP_InterfaceFactoryProperty::max_message_size](#) resource limit limits the size of the maximum message that can be received.

RTI Connex Micro does not support RTPS fragmentation. Thus, this resource-limit must be at least large enough to receive the largest serialized data-type plus overhead. A reasonable rule-of-thumb for overhead is 256 bytes per sample.

The UDP transport allocated on buffer for each resource in [DDS_DomainParticipantResourceLimitsQosPolicy::max_receive_ports](#). Thus, the total amount of memory is (max_message_size * max_receive_ports)

7.115.2.6 RTI_INT32 UDP_InterfaceFactoryProperty::max_send_message_size

The maximum size of the message which can be sent. This is only a hint and not enforced.

This parameters is useful to UDP transformation functions which must allocate a buffer to put the result in.

7.115.2.7 RTI_INT32 UDP_InterfaceFactoryProperty::multicast_ttl

The maximum TTL.

7.115.2.8 struct UDP_NatEntrySeq UDP_InterfaceFactoryProperty::nat

Configure network address translation (NAT).

Configure network address translation (NAT). This feature only supports translation between a private and public IP address only; UDP ports are not translated. Does not support any hole punching technique or WAN server, thus is only useful when the private and public address mapping is static. Please refer to [UDP_NatEntrySeq](#) for the definition of the sequence element.

7.115.2.9 struct UDP_InterfaceTableEntrySeq UDP_InterfaceFactoryProperty::if_table

The interface table if interfaces are added manually.

7.115.2.10 REDA_String_T UDP_InterfaceFactoryProperty::multicast_interface

The network interface to use to send to multicast.

7.115.2.11 RTI_BOOL UDP_InterfaceFactoryProperty::is_default_interface

If this should be considered the default UDP interfaces if no other UDP interface is found to handle a route.

7.115.2.12 struct OSAPI_ThreadProperty UDP_InterfaceFactoryProperty::recv_thread

Thread properties for each receive thread created by this NETIO interface.

7.115.2.13 struct UDP_TransformRuleSeq UDP_InterfaceFactoryProperty::source_rules

Rules for how to transform received UDP payloads based on the source address.

7.115.2.14 struct UDP_TransformRuleSeq UDP_InterfaceFactoryProperty::destination_rules

Rules for how to transform sent UDP payloads based on the destination address.

7.115.2.15 UDP_TransformUdpMode.T UDP_InterfaceFactoryProperty::transform_udp_mode

Determines how regular UDP is supported when transformations are supported.

7.115.2.16 RTI_UINT32 UDP_InterfaceFactoryProperty::transform_locator_kind

The locator to use for locators that have transformations.

When transformation rules have been enabled they are announced as a vendor specific locator. This property overrides this value. NOTE: Changing this value may prevent communication. NOTE : must be greater or equal than NETIO_ADDRESS_KIND_USER and not equal to NETIO_ADDRESS_KIND_SHMEM or NETIO_ADDRESS_KIND_INTRA.

7.116 UDP_InterfaceTableEntry Struct Reference

Generic structure to describe a network interface.

Public Attributes

- [RTI_UINT32 flags](#)
Flags to indicate if the interface is up etc.
- [RTI_UINT32 address](#)
The address of the interface as configured by the OS.
- [RTI_UINT32 netmask](#)
The netmask of the interface as configured by the OS.
- char ifname [UDP_INTERFACE_MAX_IFNAME]
The name of the interface as configured by the OS.

7.116.1 Detailed Description

Generic structure to describe a network interface.

7.116.2 Member Data Documentation

7.116.2.1 RTI_UINT32 UDP_InterfaceTableEntry::flags

Flags to indicate if the interface is up etc.

7.116.2.2 RTI_UINT32 UDP_InterfaceTableEntry::address

The address of the interface as configured by the OS.

7.116.2.3 RTI_UINT32 UDP_InterfaceTableEntry::netmask

The netmask of the interface as configured by the OS.

7.116.2.4 char UDP_InterfaceTableEntry::ifname[UDP_INTERFACE_MAX_IFNAME]

The name of the interface as configured by the OS.

7.117 UDP_InterfaceTableEntrySeq Struct Reference

<<eXtension>> Declares IDL sequence<UDP_InterfaceTableEntry>

7.117.1 Detailed Description

<<[eXtension](#)>> Declares IDL sequence<[UDP_InterfaceTableEntry](#)>

7.118 UDP_NatEntry Struct Reference

<<[eXtension](#)>> Properties for a Network Address Translation entry

7.118.1 Detailed Description

<<[eXtension](#)>> Properties for a Network Address Translation entry

Adding [UDP_NatEntry](#) entries to the [UDP_InterfaceFactoryProperty::nat](#) sequence enables translation between local and public network addresses. Please refer to [Sequence Support](#) for more information on how to manages sequences in RTI Connex Micro.

7.119 UDP_NatEntrySeq Struct Reference

<<[eXtension](#)>> Declares IDL sequence<[UDP_NatEntry](#)>

7.119.1 Detailed Description

<<[eXtension](#)>> Declares IDL sequence<[UDP_NatEntry](#)>

7.120 UDPInterfaceFactory Class Reference

<<[eXtension](#)>> <<[cert](#)>> A UDP transport factory that will be used by a DomainParticipant to create a UDP interface. Only one UDP interface can be registered with the DomainParticipantFactory.

Static Public Member Functions

- static struct
RT_ComponentFactoryI * [get_interface](#) ()
Gets the singleton instance of the UDP interface factory.

7.120.1 Detailed Description

<<[eXtension](#)>> <<[cert](#)>> A UDP transport factory that will be used by a DomainParticipant to create a UDP interface. Only one UDP interface can be registered with the DomainParticipantFactory.

7.120.2 Member Function Documentation

7.120.2.1 static struct RT_ComponentFactoryI* [UDPInterfaceFactory::get_interface](#) () [static],[read]

Gets the singleton instance of the UDP interface factory.

This function gets the singleton instance of the UDP factory that is used by the middleware to create a UDP transport instance. [UDP_InterfaceFactoryProperty](#)

Returns

Pointer to UDP factory instance

MT Safety:

This operation is thread safe.()

7.121 UDPInterfaceTable Class Reference**Static Public Member Functions**

- static bool `add_entry` (struct `UDP_InterfaceTableEntrySeq` *seq, `RTI_UINT32` address, `RTI_UINT32` net-mask, const char *ifname, `RTI_UINT32` flags)

Add an entry to the list of available interfaces.

7.121.1 Detailed Description**7.122 WHSMHistoryFactory Class Reference**

<<*eXtension*>> <<*cert*>> A writer history factory that will be used by the DataWriter to create a history cache.

Static Public Member Functions

- static struct `RT_ComponentFactoryI` * `get_interface` ()

Gets the singleton instance of the WHSM writer history factory.

7.122.1 Detailed Description

<<*eXtension*>> <<*cert*>> A writer history factory that will be used by the DataWriter to create a history cache.

7.122.2 Member Function Documentation**7.122.2.1 static struct `RT_ComponentFactoryI`* `WHSMHistoryFactory::get_interface` () [static],[read]**

Gets the singleton instance of the WHSM writer history factory.

This function gets the singleton instance of the WHSM history plugin factory that is used by the middleware to create a `WriterHistory` plugin. The singleton instance of the factory must be registered with the name "wh" in the `DomainParticipantFactory`.

Returns

Pointer to WHSM history plugin factory

MT Safety:

This operation is thread safe.

8 File Documentation**8.1 `cdr_log.h` File Reference**

Macros

- #define [CDR_LOG_STREAM_ALLOC_EC](#) (CDR_LOG_BASE + 1)
Failed to allocate stream or stream buffer.
- #define [CDR_LOG_SET_OFFSET_EC](#) (CDR_LOG_BASE + 2)
Failed to set stream's current offset.
- #define [CDR_LOG_INCR_OFFSET_EC](#) (CDR_LOG_BASE + 3)
Failed to increment stream's current offset.
- #define [CDR_LOG_INVALID_ENUM_EC](#) (CDR_LOG_BASE + 4)
Invalid enum size.
- #define [CDR_LOG_UNSUPPORTED_OPTIONS_EC](#) (CDR_LOG_BASE + 5)
Unsupported CDR encapsulations were received. The sample is dropped.

8.1.1 Detailed Description

8.2 db_log.h File Reference

DB module log codes.

Macros

- #define [DB_LOG_SORTED_ALLOC_EC](#) (DB_LOG_BASE + 1)
Not sufficient memory to allocate sorted list for an index.
- #define [DB_LOG_NAME_TOO_LONG_EC](#) (DB_LOG_BASE + 2)
The specified database name is too long.
- #define [DB_LOG_ILLEGAL_TABLE_SIZE_EC](#) (DB_LOG_BASE + 3)
Illegal table size specified.
- #define [DB_LOG_ILLEGAL_LOCK_MODE_EC](#) (DB_LOG_BASE + 4)
Illegal combination of lock mode and mutex given.
- #define [DB_LOG_MUTEX_ALLOC_EC](#) (DB_LOG_BASE + 5)
Failed to allocate database mutex.
- #define [DB_LOG_ALLOC_TABLE_POOL_EC](#) (DB_LOG_BASE + 6)
Failed to allocate buffer pool.
- #define [DB_LOG_TABLES_INUSE_EC](#) (DB_LOG_BASE + 7)
Table is in use.
- #define [DB_LOG_TABLE_NAME_TOO_LONG_EC](#) (DB_LOG_BASE + 8)
Specified table name too long.
- #define [DB_LOG_ILLEGAL_RECORD_COUNT_EC](#) (DB_LOG_BASE + 9)
Illegal record count specified.
- #define [DB_LOG_TABLE_EXISTS_EC](#) (DB_LOG_BASE + 10)
Specified table already exists.
- #define [DB_LOG_OUT_OF_TABLES_EC](#) (DB_LOG_BASE + 11)
Table resources exceeded.
- #define [DB_LOG_OUT_OF_RECORDS_EC](#) (DB_LOG_BASE + 12)
Table record resources exceeded.
- #define [DB_LOG_OUT_OF_INDICES_EC](#) (DB_LOG_BASE + 13)
Table index resources exceeded.
- #define [DB_LOG_OUT_OF_CURSORS_EC](#) (DB_LOG_BASE + 14)
Table cursor resources exceeded.
- #define [DB_LOG_RECORDS_INUSE_EC](#) (DB_LOG_BASE + 15)
Cannot delete table, records are still in table.

- #define `DB_LOG_CURSORS_INUSE_EC` (`DB_LOG_BASE + 16`)
Cannot delete table, cursors are still in use.
- #define `DB_LOG_INDEX_INUSE_EC` (`DB_LOG_BASE + 17`)
Cannot delete table, indices are still in use.
- #define `DB_LOG_ALLOC_CURSOR_POOL_EC` (`DB_LOG_BASE + 18`)
Failed to allocate cursor buffer pool.
- #define `DB_LOG_ALLOC_INDEX_POOL_EC` (`DB_LOG_BASE + 19`)
Failed to allocate index buffer pool.
- #define `DB_LOG_ALLOC_RECORD_POOL_EC` (`DB_LOG_BASE + 20`)
Failed to allocate record buffer pool.
- #define `DB_LOG_ALLOC_DATABASE_EC` (`DB_LOG_BASE + 21`)
Failed to allocate database.
- #define `DB_LOG_TABLE_NOT_INUSE_EC` (`DB_LOG_BASE + 22`)
An attempt was made to delete a table not in use.
- #define `DB_LOG_RECORD_ALREADY_EXISTS_EC` (`DB_LOG_BASE + 23`)
An attempt was made to insert an already existing record.
- #define `DB_LOG_RECORD_DOES_NOT_EXIST_EC` (`DB_LOG_BASE + 24`)
An attempt was made to delete a record that does not exist.
- #define `DB_LOG_CURSOR_INVALIDATED_EC` (`DB_LOG_BASE + 25`)
An attempt was made to use a cursor that has been invalidated.
- #define `DB_LOG_LOCK_FAILURE_EC` (`DB_LOG_BASE + 26`)
Failed to lock the database.
- #define `DB_LOG_UNLOCK_FAILURE_EC` (`DB_LOG_BASE + 27`)
Failed to unlock the database.

8.2.1 Detailed Description

DB module log codes. This file defines log codes for all messages logged by the database modules. All the log codes in this file are publicly documented.

8.3 `dds_c_config.h` File Reference

DDS C configuration file.

8.3.1 Detailed Description

DDS C configuration file.

8.4 `dds_c_log.h` File Reference

DDS_C module log codes.

Macros

- #define `DDSC_LOG_INVALID_DURATION_EC` (`DDSC_LOG_BASE + 1`)
The specified duration is not valid.
- #define `DDSC_LOG_INVALID_PARTICIPANT_NAME_EC` (`DDSC_LOG_BASE + 2`)
An invalid participant name was specified with an unknown GUID.
- #define `DDSC_LOG_INVALID_PARTICIPANT_GUID_PREFIX_EC` (`DDSC_LOG_BASE + 3`)

An invalid participant GUID prefix was specified, typically the GUID prefix does not match an already detected participant.

- #define `DDSC_LOG_SYS_GETTIME_EC` (DDSC_LOG_BASE + 4)
A call to OSAPI_System_get_time failed.
- #define `DDSC_LOG_GET_NEXT_OBJECT_ID_EC` (DDSC_LOG_BASE + 5)
Failed to get the next automatically generated object ID for an entity's GUID. This typically means the object id pool has been exhausted.
- #define `DDSC_LOG_SET_ENTITY_NAME_EC` (DDSC_LOG_BASE + 6)
Failed to set name string for a DDS entity in the DomainParticipantQos entity_name policy.
- #define `DDSC_LOG_SYS_GET_HOSTNAME_EC` (DDSC_LOG_BASE + 7)
A call to OSAPI_System_get_hostname failed.
- #define `DDSC_LOG_IO_SNPRINTF_FAILED_EC` (DDSC_LOG_BASE + 8)
A call to OSAPI_Stdio_snprintf failed. Typically this means the destination buffer was too small.
- #define `DDSC_LOG_TOPIC_FIND_EC` (DDSC_LOG_BASE + 9)
Failed to find a topic created by a DomainParticipant.
- #define `DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_NAME_EC` (DDSC_LOG_BASE + 10)
Endpoint discovery failed because the name of the remote participant parent for an endpoint was not found.
- #define `DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_KEY_EC` (DDSC_LOG_BASE + 11)
Endpoint discovery failed because the key of the remote participant parent for an endpoint was not found. Note that the key is logged as 4 integers in host endianness format.
- #define `DDSC_LOG_REMOTE_PARTICIPANT_KEY_NOT_EQUAL_EC` (DDSC_LOG_BASE + 12)
Failed endpoint discovery when key does not match the remote participant.
- #define `DDSC_LOG_INVALID_ENDPOINT_GUID_EC` (DDSC_LOG_BASE + 13)
Failed endpoint discovery due to an invalid or unknown endpoint GUID.
- #define `DDSC_LOG_ENDPOINT_NOT_CHILD_OF_PARTICIPANT_EC` (DDSC_LOG_BASE + 14)
Failed endpoint discovery when an endpoint is determined to belong to a different remote participant.
- #define `DDSC_LOG_PARTICIPANT_DOES_NOT_EXIST_EC` (DDSC_LOG_BASE + 15)
Failed participant discovery because a remote participant that should have been already asserted locally was not found.
- #define `DDSC_LOG_REFRESH_REM_PARTICIPANT_EC` (DDSC_LOG_BASE + 16)
Did not find a remote participant when asserting participant liveliness for it. Note that the key is logged as 4 integers in host endianness format.
- #define `DDSC_LOG_REFRESH_REM_PARTICIPANT_TIMEOUT_EC` (DDSC_LOG_BASE + 17)
Failed to assert participant liveliness to a remote participant.
- #define `DDSC_LOG_PARTICIPANT_LOOKUP_EC` (DDSC_LOG_BASE + 18)
Failed to find a remote participant as previously discovered.
- #define `DDSC_LOG_REMOVE_PUBLICATION_EC` (DDSC_LOG_BASE + 19)
Failed to remove resources for a remote publication.
- #define `DDSC_LOG_REMOVE_SUBSCRIPTION_EC` (DDSC_LOG_BASE + 20)
Failed to remove resources for a remote subscription.
- #define `DDSC_LOG_FIND_PUBLICATION_PARENT_EC` (DDSC_LOG_BASE + 21)
Cannot determine the participant of a remote publication.
- #define `DDSC_LOG_FIND_SUBSCRIPTION_PARENT_EC` (DDSC_LOG_BASE + 22)
Cannot determine the participant of a remote subscription.
- #define `DDSC_LOG_MAX_PARTICIPANT_ID_REACHED_EC` (DDSC_LOG_BASE + 23)
Failed to create DomainParticipant due to running out of participant IDs. This is typically caused by all UDP ports being used.
- #define `DDSC_LOG_RESERVE_LOCATORS_EC` (DDSC_LOG_BASE + 24)
Failed to reserve endpoint locators.
- #define `DDSC_LOG_TIMER_CREATE_TIMEOUT_EC` (DDSC_LOG_BASE + 25)
Failed to create a timeout.
- #define `DDSC_LOG_ILLEGAL_OBJECTID_EC` (DDSC_LOG_BASE + 26)

- Illegal object id specified.*
- #define [DDSC_LOG_TOPIC_NARROW_EC](#) (DDSC_LOG_BASE + 27)
Failed to narrow a TopicDescription to the named Topic.
- #define [DDSC_LOG_FAILED_UPDATE_STATUS_CONDITION_EC](#) (DDSC_LOG_BASE + 28)
Failed to update state of StatusCondition.
- #define [DDSC_LOG_WS_REMOVE_COND_REFERENCE_EC](#) (DDSC_LOG_BASE + 29)
Failed to remove a condition reference from a waitset.
- #define [DDSC_LOG_WS_ADD_COND_REFERENCE_EC](#) (DDSC_LOG_BASE + 30)
Failed to add a condition reference to a waitset.
- #define [DDSC_LOG_RELEASE_META_MC_EC](#) (DDSC_LOG_BASE + 31)
Failed to release resources for multicast discovery locators.
- #define [DDSC_LOG_RELEASE_META_UC_EC](#) (DDSC_LOG_BASE + 32)
Failed to release resources for unicast discovery locators.
- #define [DDSC_LOG_RELEASE_USER_MC_EC](#) (DDSC_LOG_BASE + 33)
Failed to release resources for multicast user locators.
- #define [DDSC_LOG_RELEASE_USER_UC_EC](#) (DDSC_LOG_BASE + 34)
Failed to release resources for unicast user locators.
- #define [DDSC_LOG_INVALID_DOMAINID_EC](#) (DDSC_LOG_BASE + 35)
The domain ID specified exceeds what is allowed based on the parameters specified in dp_qos.protocol.rtps_well_known_ports.
- #define [DDSC_LOG_DATABASE_CREATE_EC](#) (DDSC_LOG_BASE + 100)
Failed to create database.
- #define [DDSC_LOG_DATABASE_DELETE_EC](#) (DDSC_LOG_BASE + 101)
Failed to delete database.
- #define [DDSC_LOG_TABLE_CREATE_EC](#) (DDSC_LOG_BASE + 102)
Failed to create database table of the specified name.
- #define [DDSC_LOG_TABLE_INUSE_EC](#) (DDSC_LOG_BASE + 103)
Failed to delete a database table because it is not empty.
- #define [DDSC_LOG_TABLE_DELETE_EC](#) (DDSC_LOG_BASE + 104)
Failed to delete a database table.
- #define [DDSC_LOG_TABLE_SELECT_EC](#) (DDSC_LOG_BASE + 105)
A selection operation failed on the specified database table.
- #define [DDSC_LOG_CREATE_INDEX_EC](#) (DDSC_LOG_BASE + 106)
Failed to create an index on a database table.
- #define [DDSC_LOG_DELETE_INDEX_EC](#) (DDSC_LOG_BASE + 107)
Failed to delete an index on a database table.
- #define [DDSC_LOG_DB_CURSOR_INVALIDATED_EC](#) (DDSC_LOG_BASE + 108)
A database table cursor was invalidated while in use.
- #define [DDSC_LOG_SUBSCRIPTION_RECORD](#) 1
Database record for a Subscription.
- #define [DDSC_LOG_PUBLICATION_RECORD](#) 2
Database record for a Publication.
- #define [DDSC_LOG_PARTICIPANT_RECORD](#) 3
Database record for a Remote Participant.
- #define [DDSC_LOG_TOPIC_RECORD](#) 4
Database record for a Topic.
- #define [DDSC_LOG_DATAWRITER_RECORD](#) 5
Database record for a DataWriter.
- #define [DDSC_LOG_DATAREADER_RECORD](#) 6
Database record for a DataReader.
- #define [DDSC_LOG_PUBLISHER_RECORD](#) 7

- Database record for a Publisher.*

 - #define [DDSC_LOG_SUBSCRIBER_RECORD](#) 8
- Database record for a Subscriber.*

 - #define [DDSC_LOG_ROUTE_RECORD](#) 9
- Database record for a route record.*

 - #define [DDSC_LOG_BIND_RECORD](#) 10
- Database record for a bind record.*

 - #define [DDSC_LOG_TYPE_RECORD](#) 11
- Database record for a type plugin.*

 - #define [DDSC_LOG_STRING_RECORD](#) 14
- String record.*

 - #define [DDSC_LOG_RECORD_CREATE_EC](#) (DDSC_LOG_BASE + 109)
- Failed to create a database record of the specified kind.*

 - #define [DDSC_LOG_RECORD_DELETE_EC](#) (DDSC_LOG_BASE + 110)
- Failed to delete a database record of the specified kind.*

 - #define [DDSC_LOG_RECORD_INSERT_EC](#) (DDSC_LOG_BASE + 111)
- Failed to insert a database record of the specified kind.*

 - #define [DDSC_LOG_RECORD_ERROR_EC](#) (DDSC_LOG_BASE + 112)
- Unknown error for database record of the specified kind.*

 - #define [DDSC_LOG_RECORD_EXISTS_EC](#) (DDSC_LOG_BASE + 113)
- A database record of the specified kind already exists.*

 - #define [DDSC_LOG_RECORD_LOOKUP_EC](#) (DDSC_LOG_BASE + 114)
- A lookup of a database record of the specified kind failed.*

 - #define [DDSC_LOG_RECORD_NOT_EXISTS_EC](#) (DDSC_LOG_BASE + 115)
- A database record of the specified kind does not exist.*

 - #define [DDSC_LOG_RECORD_SELECT_EC](#) (DDSC_LOG_BASE + 116)
- A database select on the specified record kind failed.*

 - #define [DDSC_LOG_RECORD_REMOVE_EC](#) (DDSC_LOG_BASE + 117)
- Removal a database record of the specified kind failed.*

 - #define [DDSC_LOG_RECORD_INITIALIZE_EC](#) (DDSC_LOG_BASE + 118)
- A database record of the specified kind could not be initialized.*

 - #define [DDSC_LOG_RECORD_FINALIZE_EC](#) (DDSC_LOG_BASE + 119)
- A database record of the specified kind could not be finalized.*

 - #define [DDSC_LOG_PUBLICATIONDATA_OBJECT](#) 1
- DDS PublicationBuiltinTopicData object.*

 - #define [DDSC_LOG_SUBSCRIPTIONDATA_OBJECT](#) 2
- DDS SubscriptionBuiltinTopicData object.*

 - #define [DDSC_LOG_PARTICIPANTDATA_OBJECT](#) 3
- DDS ParticipantBuiltinTopicData object.*

 - #define [DDSC_LOG_DATAREADERQOS_OBJECT](#) 4
- DDS DataReaderQos object.*

 - #define [DDSC_LOG_DATAWRITERQOS_OBJECT](#) 5
- DDS DataWriterQos object.*

 - #define [DDSC_LOG_TOPICQOS_OBJECT](#) 6
- DDS TopicQos object.*

 - #define [DDSC_LOG_PUBLISHERQOS_OBJECT](#) 7
- DDS PublisherQos object.*

 - #define [DDSC_LOG_SUBSCRIBERQOS_OBJECT](#) 8
- DDS SubscriberQos object.*

 - #define [DDSC_LOG_PARTICIPANTQOS_OBJECT](#) 9
- DDS DomainParticipantQos object.*

- #define [DDSC_LOG_ENTITY_OBJECT](#) 10
DDS Entity object.
- #define [DDSC_LOG_PARTICIPANTFACTORYQOS_OBJECT](#) 11
DDS DomainParticipantFactoryQos object.
- #define [DDSC_LOG_TYPE_OBJECT](#) 12
A DDS Type object.
- #define [DDSC_LOG_BINDRESOLVER_OBJECT](#) 13
A NETIO BindResolver object.
- #define [DDSC_LOG_ROUTERESOLVER_OBJECT](#) 14
A NETIO RouteResolver object.
- #define [DDSC_LOG_ADDRESSRESOLVER_OBJECT](#) 15
A NETIO AddressResolver object.
- #define [DDSC_LOG_DATAWRITERIO_OBJECT](#) 16
A NETIO DataWriterInterface object.
- #define [DDSC_LOG_DATAREADERIO_OBJECT](#) 17
A NETIO DataReaderInterface object.
- #define [DDSC_LOG_LOG_OBJECT](#) 18
A OSAPI Log object.
- #define [DDSC_LOG_SYSTEM_OBJECT](#) 19
A OSAPI system object.
- #define [DDSC_LOG_MUTEX_OBJECT](#) 20
A OSAPI mutex object.
- #define [DDSC_LOG_CONDREF_OBJECT](#) 21
A DDS Waitset condition reference object.
- #define [DDSC_LOG_WSREF_OBJECT](#) 22
A DDS Condition waitset reference object.
- #define [DDSC_LOG_MD5STREAM_OBJECT](#) 23
A MD5 stream object.
- #define [DDSC_LOG_CONDITION_OBJECT](#) 24
A DDS Condition object.
- #define [DDSC_LOG_RT_OBJECT](#) 25
A RT Condition object.
- #define [DDSC_LOG_PARTICIPANT_POOL_OBJECT](#) 26
A Participant pool object.
- #define [DDSC_LOG_TIMER_OBJECT](#) 27
A OSAPI_Timer object.
- #define [DDSC_LOG_TIMEROUT_OBJECT](#) 28
A OSAPI_Timer timeout object.
- #define [DDSC_LOG_TOPIC_OBJECT](#) 29
A DDS Topic object.
- #define [DDSC_LOG_WAITSET_OBJECT](#) 30
A DDS WaitSet object.
- #define [DDSC_LOG_UUID_OBJECT](#) 31
A UUID object.
- #define [DDSC_LOG_STRING_OBJECT](#) 40
A String object.
- #define [DDSC_LOG_STRINGMANAGER_OBJECT](#) 43
A String Manager object.
- #define [DDSC_LOG_OBJECT_INITIALIZE_EC](#) (DDSC_LOG_BASE + 200)
Out of resources to initialize object of the specified kind.
- #define [DDSC_LOG_OBJECT_ALLOCATE_EC](#) (DDSC_LOG_BASE + 201)

- Out of resources to allocate an object of the specified kind.*
- #define [DDSC_LOG_OBJECT_FINALIZE_EC](#) (DDSC_LOG_BASE + 202)
Failed to finalize object of specified kind.
 - #define [DDSC_LOG_OBJECT_DELETE_EC](#) (DDSC_LOG_BASE + 203)
Failed to delete object of specified kind.
 - #define [DDSC_LOG_OBJECT_COPY_EC](#) (DDSC_LOG_BASE + 204)
Failed to copy object of specified kind.
 - #define [DDSC_LOG_OBJECT_REFCOUNT_EC](#) (DDSC_LOG_BASE + 205)
Failed to delete/finalize an object because other objects are referencing it.
 - #define [DDSC_LOG_OBJECT_GET_PROPERTY_EC](#) (DDSC_LOG_BASE + 206)
Failed to get the object properties.
 - #define [DDSC_LOG_OBJECT_SET_PROPERTY_EC](#) (DDSC_LOG_BASE + 207)
Failed to set the object properties.
 - #define [DDSC_LOG_OBJECT_EMPTY_EC](#) (DDSC_LOG_BASE + 208)
An object is empty, typically applies only to buffer-pool objects.
 - #define [DDSC_LOG_OBJECT_INUSECOUNT_EC](#) (DDSC_LOG_BASE + 209)
Failed to delete/finalize an object because other objects are using it.
 - #define [DDSC_LOG_NETMASK_SEQUENCE](#) 1
A NETIO Netmask sequence.
 - #define [DDSC_LOG_ROUTE_SEQUENCE](#) 2
A NETIO Route sequence.
 - #define [DDSC_LOG_RESERVED_SEQUENCE](#) 3
A NETIO Reserved address sequence.
 - #define [DDSC_LOG_ENABLED_USER_TRANSPORT_SEQUENCE](#) 4
A DDS sequence of enabled user NETIO transports.
 - #define [DDSC_LOG_ENABLED_TRANSPORT_SEQUENCE](#) 5
A DDS sequence of enabled transports.
 - #define [DDSC_LOG_ENABLED_DISCOVERY_TRANSPORT_SEQUENCE](#) 6
A DDS sequence of enabled discovery transports.
 - #define [DDSC_LOG_INITIAL_PEER_SEQUENCE](#) 7
A DDS sequence of initial peers.
 - #define [DDSC_LOG_DESTINATION_SEQUENCE](#) 8
A NETIO sequence of destinations to send to.
 - #define [DDSC_LOG_METAUNICAST_SEQUENCE](#) 9
A DDS sequence of meta-traffic unicast locators.
 - #define [DDSC_LOG_METAMULTICAST_SEQUENCE](#) 10
A DDS sequence of meta-traffic multicast locators.
 - #define [DDSC_LOG_USERUNICAST_SEQUENCE](#) 11
A DDS sequence of user-traffic unicast locators.
 - #define [DDSC_LOG_USERMULTICAST_SEQUENCE](#) 12
A DDS sequence of user-traffic multicast locators.
 - #define [DDSC_LOG_READTAKE_SEQUENCE](#) 13
A DDS sequence used in a read/take call.
 - #define [DDSC_LOG_SEQUENCE_SETMAX_EC](#) (DDSC_LOG_BASE + 300)
Failed to set the maximum length of a sequence of the specified kind.
 - #define [DDSC_LOG_SEQUENCE_SETLENGTH_EC](#) (DDSC_LOG_BASE + 301)
Failed to set the length of a sequence of the specified kind.
 - #define [DDSC_LOG_SEQUENCE_GETREF_EC](#) (DDSC_LOG_BASE + 302)
Failed to get a reference at the specified index for a sequence of the specified kind.
 - #define [DDSC_LOG_SEQUENCE_INITIALIZE_EC](#) (DDSC_LOG_BASE + 303)
Failed to initialize a sequence of the specified kind.

- `#define DDSC_LOG_SEQUENCE_FINALIZE_EC` (DDSC_LOG_BASE + 304)
Failed to finalize a sequence of the specified kind.
- `#define DDSC_LOG_SEQUENCE_COPY_EC` (DDSC_LOG_BASE + 305)
Failed to copy a sequence of the specified kind.
- `#define DDSC_LOG_SEQUENCE_INVALID_EC` (DDSC_LOG_BASE + 306)
The sequence of the specified kind was invalid in the context it is used.
- `#define DDSC_LOG_DISCOVERY_COMPONENT` 1
A component of discovery kind.
- `#define DDSC_LOG_RTPS_COMPONENT` 2
A component of RTPS kind.
- `#define DDSC_LOG_READERHISTORY_COMPONENT` 3
A component of reader-history kind.
- `#define DDSC_LOG_WRITERHISTORY_COMPONENT` 4
A component of writer-history kind.
- `#define DDSC_LOG_DATAREADERIO_COMPONENT` 5
A component of DataReaderInterface kind.
- `#define DDSC_LOG_DATAWRITERIO_COMPONENT` 6
A component of DataWriterInterface kind.
- `#define DDSC_LOG_NETIO_COMPONENT` 7
A component of NETIO kind.
- `#define DDSC_LOG_TRANSPORT_COMPONENT` 8
A component of transport kind.
- `#define DDSC_LOG_COMPONENT_LOOKUP_EC` (DDSC_LOG_BASE + 400)
Did not find a component factory with the given name in the registry.
- `#define DDSC_LOG_COMPONENT_CREATE_EC` (DDSC_LOG_BASE + 401)
Could not create a component of the specified kind using the specified factory.
- `#define DDSC_LOG_COMPONENT_DELETE_EC` (DDSC_LOG_BASE + 402)
Could not delete a component of the specified kind using the specified factory.
- `#define DDSC_LOG_HISTORY_RESOURCE` 1
Exceeded resource limits for writer history queue.
- `#define DDSC_LOG_SAMPLE_RESOURCES` 2
Failed to get resource for new sample due to resource limit.
- `#define DDSC_LOG_PARTICIPANT_RESOURCES` 3
Failed to allocate a new participant.
- `#define DDSC_LOG_RESOURCE_EXCEEDED_EC` (DDSC_LOG_BASE + 500)
Could not allocate a resource of the specified kind.
- `#define DDSC_LOG_TOPIC_QOS` 1
The TopicQos kind.
- `#define DDSC_LOG_DATAREADER_QOS` 2
The DataReaderQos kind.
- `#define DDSC_LOG_DATAWRITER_QOS` 3
The DataWriterQos kind.
- `#define DDSC_LOG_SUBSCRIBER_QOS` 4
The SubscriberQos kind.
- `#define DDSC_LOG_PUBLISHER_QOS` 5
The PublisherQos kind.
- `#define DDSC_LOG_PARTICIPANT_QOS` 6
The DomainParticipantQos kind.
- `#define DDSC_LOG_PARTICIPANTFACTORY_QOS` 7
The DomainParticipantFactoryQos kind.
- `#define DDSC_LOG_DEFAULTTOPIC_QOS` 8

- The default TopicQos kind.*

 - #define [DDSC_LOG_DEFAULTDATAREADER_QOS](#) 9
- The default DataReaderQos kind.*

 - #define [DDSC_LOG_DEFAULTDATAWRITER_QOS](#) 10
- The default DataWriterQos kind.*

 - #define [DDSC_LOG_DEFAULTSUBSCRIBER_QOS](#) 11
- The default SubscriberQos kind.*

 - #define [DDSC_LOG_DEFAULTPUBLISHER_QOS](#) 12
- The default PublisherQos kind.*

 - #define [DDSC_LOG_DEFAULTPARTICIPANT_QOS](#) 13
- The default PublisherQos kind.*

 - #define [DDSC_LOG_DEADLINE_QOS_POLICY](#) 14
- The deadline qos policy kind.*

 - #define [DDSC_LOG_LIVELINESS_QOS_POLICY](#) 15
- The liveliness qos policy kind.*

 - #define [DDSC_LOG_HISTORY_QOS_POLICY](#) 16
- The history qos policy kind.*

 - #define [DDSC_LOG_RESOURCE_LIMIT_QOS_POLICY](#) 17
- The resource limits qos policy kind.*

 - #define [DDSC_LOG_PROTOCOL_QOS_POLICY](#) 18
- The protocol qos policy kind.*

 - #define [DDSC_LOG_TYPE_SUPPORT_QOS_POLICY](#) 19
- The type-support qos policy kind.*

 - #define [DDSC_LOG_RELIABILITY_QOS_POLICY](#) 20
- The reliability qos policy kind.*

 - #define [DDSC_LOG_DURABILITY_QOS_POLICY](#) 21
- The durability qos policy kind.*

 - #define [DDSC_LOG_OWNERSHIP_QOS_POLICY](#) 22
- The ownership qos policy kind.*

 - #define [DDSC_LOG_OWNERSHIP_STRENGTH_QOS_POLICY](#) 23
- The ownership-strength qos policy kind.*

 - #define [DDSC_LOG_TRANSPORT_QOS_POLICY](#) 24
- The transport qos policy kind.*

 - #define [DDSC_LOG_PARTICIPANT_ID_QOS_POLICY](#) 25
- The participant id qos policy kind.*

 - #define [DDSC_LOG_HEARTBEATS_QOS_POLICY](#) 26
- The heartbeat qos policy kind.*

 - #define [DDSC_LOG_DATAWRITER_RESOURCE_QOS_POLICY](#) 27
- The DataWriterQos writer_resource_limits policy.*

 - #define [DDSC_LOG_DATAREADER_RESOURCE_QOS_POLICY](#) 28
- The DataReaderQos reader_resource_limits policy.*

 - #define [DDSC_LOG_DESTINATION_ORDER_POLICY](#) 29
- The DestinationOrderQos destination_order policy.*

 - #define [DDSC_LOG_USER_TRAFFIC_QOS_POLICY](#) 30
- The UserTraffic policy.*

 - #define [DDSC_LOG_DISCOVERY_TRAFFIC_QOS_POLICY](#) 31
- The Discovery.enabled_transport policy.*

 - #define [DDSC_LOG_QOS_INCONSISTENT_POLICY_EC](#) (DDSC_LOG_BASE + 501)
- An inconsistent Qos policy for the specified Qos kind was found.*

 - #define [DDSC_LOG_QOS_INCONSISTENT_POLICIES_EC](#) (DDSC_LOG_BASE + 502)
- Inconsistency between two Qos policies for the specified Qos kind was found.*

- #define [DDSC_LOG_QOS_INCONSISTENT_EC](#) (DDSC_LOG_BASE + 503)
Failed to create an entity or set a qos due to inconsistent policy.
- #define [DDSC_LOG_QOS_COPY_EC](#) (DDSC_LOG_BASE + 504)
Failed to copy a Qos of the specified kind.
- #define [DDSC_LOG_QOS_INITIALIZE_EC](#) (DDSC_LOG_BASE + 505)
Failed to initialize a Qos of the specified kind.
- #define [DDSC_LOG_QOS_FINALIZE_EC](#) (DDSC_LOG_BASE + 506)
Failed to finalize a Qos of the specified kind.
- #define [DDSC_LOG_QOS_SET_EC](#) (DDSC_LOG_BASE + 507)
Failed to set a Qos of the specified kind.
- #define [DDSC_LOG_QOS_SET_ON_ENABLED_EC](#) (DDSC_LOG_BASE + 508)
Failed to set a Qos of the specified kind because the entity is already enabled.
- #define [DDSC_LOG_QOS_IMMUTABLE_EC](#) (DDSC_LOG_BASE + 509)
Failed to set a Qos of the specified kind the immutable Qos policies have been changed.
- #define [DDSC_LOG_QOS_CHANGED_EC](#) (DDSC_LOG_BASE + 510)
A discovered Qos changed (the entity already existed)
- #define [DDSC_LOG_QOS_GET_EC](#) (DDSC_LOG_BASE + 511)
Failed to get a Qos of the specified kind.
- #define [DDSC_LOG_TOPIC_LISTENER](#) 1
The topic listener kind.
- #define [DDSC_LOG_DATAREADER_LISTENER](#) 2
The datareader listener kind.
- #define [DDSC_LOG_DATAWRITER_LISTENER](#) 3
The datawriter listener kind.
- #define [DDSC_LOG_SUBSCRIBER_LISTENER](#) 4
The subscriber listener kind.
- #define [DDSC_LOG_PUBLISHER_LISTENER](#) 5
The publisher listener kind.
- #define [DDSC_LOG_PARTICIPANT_LISTENER](#) 6
The participant listener kind.
- #define [DDSC_LOG_PARTICIPANTFACTORY_LISTENER](#) 7
The participant-factory listener kind.
- #define [DDSC_LOG_LISTENER_INCONSISTENT_EC](#) (DDSC_LOG_BASE + 512)
Failed to create an entity due to inconsistent listener and status mask.
- #define [DDSC_LOG_LISTENER_SET_EC](#) (DDSC_LOG_BASE + 513)
Failed to set the listener of the specified kind.
- #define [DDSC_LOG_LISTENER_GET_EC](#) (DDSC_LOG_BASE + 514)
Failed to get the listener of the specified kind.
- #define [DDSC_LOG_LISTENER_SET_ILLEGAL_NULL_EC](#) (DDSC_LOG_BASE + 515)
Illegal combination of NULL listener and non-NONE status mask when setting a listener for an Entity.
- #define [DDSC_LOG_TOPIC_ENTITY](#) 1
The topic entity kind.
- #define [DDSC_LOG_DATAWRITER_ENTITY](#) 2
The datawriter entity kind.
- #define [DDSC_LOG_DATAREADER_ENTITY](#) 3
The datareader entity kind.
- #define [DDSC_LOG_PUBLISHER_ENTITY](#) 4
The publisher entity kind.
- #define [DDSC_LOG_SUBSCRIBER_ENTITY](#) 5
The subscriber entity kind.
- #define [DDSC_LOG_PARTICIPANT_ENTITY](#) 6

- The participant entity kind.*

 - #define `DDSC_LOG_PUBLICATION_ENTITY` 7
- The publication entity kind.*

 - #define `DDSC_LOG_SUBSCRIPTION_ENTITY` 8
- The subscription entity kind.*

 - #define `DDSC_LOG_ENTITY_ENABLE_EC` (DDSC_LOG_BASE + 600)

Failed to enable an entity of the specified kind.

 - #define `DDSC_LOG_ENTITY_NOT_EMPTY_EC` (DDSC_LOG_BASE + 601)

Failed to an delete/finalize an entity of the specified kind because it is not empty.

 - #define `DDSC_LOG_ENTITY_FINALIZE_EC` (DDSC_LOG_BASE + 602)

Failed to finalize an entity of the specified kind.

 - #define `DDSC_LOG_ENTITY_INITIALIZE_EC` (DDSC_LOG_BASE + 603)

Failed to initialize an entity of the specified kind.

 - #define `DDSC_LOG_ENTITY_NOT_ENABLED_EC` (DDSC_LOG_BASE + 604)

An operation was attempted on an entity that is not enabled.

 - #define `DDSC_LOG_ENTITY_DIFFERENT_FACTORY_EC` (DDSC_LOG_BASE + 605)

Entities are in different factories.

 - #define `DDSC_LOG_DATAWRITER_CDR` 1

The datawriter CDR kind.

 - #define `DDSC_LOG_DATAREADER_CDR` 2

The datareader CDR kind.

 - #define `DDSC_LOG_CDR_POOL_ALLOC_EC` (DDSC_LOG_BASE + 700)

Failed to allocate a pool of the specified kind.

 - #define `DDSC_LOG_CDR_BUFFER_SET_EC` (DDSC_LOG_BASE + 701)

Failed to set the CDR buffer for a packet.

 - #define `DDSC_LOG_CDR_POOL_DELETE_EC` (DDSC_LOG_BASE + 702)

Failed to delete the CDR pool.

 - #define `DDSC_LOG_CDR_SERIALIZE_PID_EC` (DDSC_LOG_BASE + 703)

Failed to serialize a parameter ID.

 - #define `DDSC_LOG_CDR_SERIALIZE_PID_LENGTH_EC` (DDSC_LOG_BASE + 704)

Failed to serialize a parameter length.

 - #define `DDSC_LOG_CDR_SERIALIZE_KEYHASH_EC` (DDSC_LOG_BASE + 705)

Failed to serialize a key-hash.

 - #define `DDSC_LOG_CDR_SERIALIZE_DATA_EC` (DDSC_LOG_BASE + 706)

Failed to serialize payload data.

 - #define `DDSC_LOG_DESERIALIZE_BAD_PID_LENGTH_EC` (DDSC_LOG_BASE + 707)

Deserialized an invalid parameter length for a specific parameter ID.

 - #define `DDSC_LOG_CDR_DESERIALIZE_PID_EC` (DDSC_LOG_BASE + 708)

Failed to deserialize the ID of an inline parameter.

 - #define `DDSC_LOG_CDR_DESERIALIZE_PID_LENGTH_EC` (DDSC_LOG_BASE + 709)

Failed to deserialize the length of an inline parameter.

 - #define `DDSC_LOG_CDR_INCREMENT_POS_EC` (DDSC_LOG_BASE + 710)

Failed to increment to the position of the next inline parameter.

 - #define `DDSC_LOG_CDR_SET_POS_EC` (DDSC_LOG_BASE + 711)

Failed to set the reception stream position.

 - #define `DDSC_LOG_CDR_DESERIALIZE_HEADER_EC` (DDSC_LOG_BASE + 712)

Failed to deserialize the encapsulation header.

 - #define `DDSC_LOG_CDR_DESERIALIZE_DATA_EC` (DDSC_LOG_BASE + 713)

Failed to deserialize CDR payload data.

 - #define `DDSC_LOG_CDR_INITIALIZE_SAMPLE_EC` (DDSC_LOG_BASE + 714)

Failed to initialize CDR sample.

- #define [DDSC_LOG_CDR_FINALIZE_SAMPLE_EC](#) (DDSC_LOG_BASE + 715)
Failed to finalize sample.
- #define [DDSC_LOG_CDR_SERIALIZE_STATUS_INFO_EC](#) (DDSC_LOG_BASE + 716)
Failed to serialize the status info parameter.
- #define [DDSC_LOG_CDR_DESERIALIZE_KEYHASH_EC](#) (DDSC_LOG_BASE + 717)
Failed to deserialize a key-hash.
- #define [DDSC_LOG_CDR_DESERIALIZE_KEY_EC](#) (DDSC_LOG_BASE + 718)
Failed to deserialize CDR payload key.
- #define [DDSC_LOG_NETIO_ADD_ANON_TOPIC_ROUTE_EC](#) (DDSC_LOG_BASE + 800)
Failed to add a route to an anonymous participant discovery datawriter.
- #define [DDSC_LOG_NETIO_ADD_TOPIC_ROUTE_EC](#) (DDSC_LOG_BASE + 801)
Failed to add a route to a topic from a datawriter.
- #define [DDSC_LOG_NETIO_DELETE_TOPIC_ROUTE_EC](#) (DDSC_LOG_BASE + 802)
Failed to delete a route to a topic.
- #define [DDSC_LOG_NETIO_FORWARD_TOPIC_EC](#) (DDSC_LOG_BASE + 803)
Failed to forward a topic.
- #define [DDSC_LOG_NETIO_NETIO_KIND](#) 1
Any NETIO interface kind, typically UDP.
- #define [DDSC_LOG_INTRA_NETIO_KIND](#) 2
The intra interface kind.
- #define [DDSC_LOG_RTPS_NETIO_KIND](#) 3
The RTPS interface kind.
- #define [DDSC_LOG_DATAREADER_NETIO_KIND](#) 4
The DataReader interface kind.
- #define [DDSC_LOG_DATAWRITER_NETIO_KIND](#) 5
The DataWriter interface kind.
- #define [DDSC_LOG_NETIO_BIND_EXTERNAL_EC](#) (DDSC_LOG_BASE + 804)
Failed to bind two external interface of the specified kind.
- #define [DDSC_LOG_NETIO_UNBIND_EXTERNAL_EC](#) (DDSC_LOG_BASE + 805)
Failed to unbind two external interfaces of the specified kind.
- #define [DDSC_LOG_NETIO_BIND_EC](#) (DDSC_LOG_BASE + 806)
Failed to bind an interface to a peer interface.
- #define [DDSC_LOG_NETIO_UNBIND_EC](#) (DDSC_LOG_BASE + 807)
Failed to unbind an interface from a peer interface.
- #define [DDSC_LOG_NETIO_ADD_ROUTE_EC](#) (DDSC_LOG_BASE + 808)
Failed to add a route from an interface to a peer interface.
- #define [DDSC_LOG_NETIO_DELETE_ROUTE_EC](#) (DDSC_LOG_BASE + 809)
Failed to delete a route from an interface to a peer interface.
- #define [DDSC_LOG_NETIO_GET_EXTERNAL_INTF_EC](#) (DDSC_LOG_BASE + 810)
Failed to get an external interface for the specified interface kind.
- #define [DDSC_LOG_NETIO_NO_ROUTE_EC](#) (DDSC_LOG_BASE + 811)
A DataReader failed a bind due to no existing route.
- #define [DDSC_LOG_NETIO_ROUTE_LOOKUP_FAILED_EC](#) (DDSC_LOG_BASE + 812)
Lookup a route to a destination failed.
- #define [DDSC_LOG_NETIO_GET_ROUTE_TABLE_FAILED_EC](#) (DDSC_LOG_BASE + 813)
Failed to get the route table for an interface.
- #define [DDSC_LOG_NETIO_SEND_FAILED_EC](#) (DDSC_LOG_BASE + 814)
Failure when sending on an interface.
- #define [DDSC_LOG_NETIO_SET_STATE_EC](#) (DDSC_LOG_BASE + 815)
Failed to set an interface state.
- #define [DDSC_LOG_NETIO_PEER_LOOKUP_EC](#) (DDSC_LOG_BASE + 816)

- Datawriter did not find a peer.*

 - #define [DDSC_LOG_NETIO_FORCED_REMOVE_EC](#) (DDSC_LOG_BASE + 817)

Forced removal of sample downstream failed.
- #define [DDSC_LOG_PACKET_INIT_EC](#) (DDSC_LOG_BASE + 818)

Failed to initialize a packet.
- #define [DDSC_LOG_PACKET_SET_HEAD_EC](#) (DDSC_LOG_BASE + 819)

Failed to set the head of a packet.
- #define [DDSC_LOG_PACKET_SET_TAIL_EC](#) (DDSC_LOG_BASE + 820)

Failed to set the tail of a packet.
- #define [DDSC_LOG_DW_ACKNACK_FAILED_EC](#) (DDSC_LOG_BASE + 900)

Failed to ACKNACK sample in the writer history.
- #define [DDSC_LOG_DW_COMMIT_EC](#) (DDSC_LOG_BASE + 901)

Failed to commit a sample to the writer queue.
- #define [DDSC_LOG_DW_KEYHASH_CREATE_EC](#) (DDSC_LOG_BASE + 902)

Failed to create keyhash of instance handle.
- #define [DDSC_LOG_DW_ILLEGAL_KEY_KIND_EC](#) (DDSC_LOG_BASE + 903)

Failed a write due to an invalid key kind for the type being written.
- #define [DDSC_LOG_DW_CREATE_TYPED_WRITER_EC](#) (DDSC_LOG_BASE + 904)

Failed to create a typed writer.
- #define [DDSC_LOG_DW_HISTORY_REGISTER_KEY_EC](#) (DDSC_LOG_BASE + 905)

Failed to register the key of an instance.
- #define [DDSC_LOG_DR_CREATE_TYPED_READER_EC](#) (DDSC_LOG_BASE + 1000)

Failed to create a typed datareader.
- #define [DDSC_LOG_DR_COPY_DATA_SAMPLE_EC](#) (DDSC_LOG_BASE + 1001)

Failed to copy a sample upon reception, read, or take.
- #define [DDSC_LOG_DR_COMMIT_SAMPLE_EC](#) (DDSC_LOG_BASE + 1002)

Failed to commit a sample to be made available to be read or taken.
- #define [DDSC_LOG_DR_FILTER_ERROR_EC](#) (DDSC_LOG_BASE + 1003)

A datareader filter function failed.
- #define [DDSC_LOG_DR_DESERIALIZE_KEYHASH_EC](#) (DDSC_LOG_BASE + 1004)

Failed to deserialize a key-hash parameter.
- #define [DDSC_LOG_DR_GET_ENTRY_FAILED_EC](#) (DDSC_LOG_BASE + 1005)

Failed to get a Reader History entry for a received sample.
- #define [DDSC_LOG_DR_COMMIT_ENTRY_EC](#) (DDSC_LOG_BASE + 1006)

Failed to commit a receive sample to Reader History to be read or taken.
- #define [DDSC_LOG_DR_UNREGISTER_KEY_EC](#) (DDSC_LOG_BASE + 1007)

A DataReader failed to unregister an instance.
- #define [DDSC_LOG_DR_DISPOSE_KEY_EC](#) (DDSC_LOG_BASE + 1008)

A DataReader failed to dispose an instance.
- #define [DDSC_LOG_DR_READ_TAKE_FAILURE_EC](#) (DDSC_LOG_BASE + 1009)

A call to a reader/take function failed.
- #define [DDSC_LOG_DR_INSTANCE_TO_KEYHASH_EC](#) (DDSC_LOG_BASE + 1010)

Failed to create keyhash of instance handle.
- #define [DDSC_LOG_TYPE_NAME_CMP_EC](#) (DDSC_LOG_BASE + 1100)

Two type names are incompatible.
- #define [DDSC_LOG_TOPIC_NAME_CMP_EC](#) (DDSC_LOG_BASE + 1101)

Two topic names are incompatible.
- #define [DDSC_LOG_TYPE_FUNCTION_GET_SERIALIZED_SAMPLE_MAX_SIZE](#) 1

The `get_serialized_sample_max_size` function kind.
- #define [DDSC_LOG_TYPE_FUNCTION_SERIALIZE_DATA](#) 2

The `serialize_data` function kind.

- `#define DDSC_LOG_TYPE_FUNCTION_DESERIALIZE_DATA 3`
The deserialize_data function kind.
- `#define DDSC_LOG_TYPE_FUNCTION_CREATE_SAMPLE 4`
The create_sample function kind.
- `#define DDSC_LOG_TYPE_FUNCTION_COPY_SAMPLE 5`
The copy_sample function kind.
- `#define DDSC_LOG_TYPE_FUNCTION_DELETE_SAMPLE 6`
The delete_sample function kind.
- `#define DDSC_LOG_TYPE_FUNCTION_GET_KEY_KIND 7`
The get_key_kind function kind.
- `#define DDSC_LOG_TYPE_FUNCTION_INSTANCE_TO_KEYHASH 8`
The instance_to_keyhash function kind.
- `#define DDSC_LOG_TYPE_FUNCTION_NULL_EC (DDSC_LOG_BASE + 1102)`
Invalid type plugin, The specified function pointer is NULL.
- `#define DDSC_LOG_TOPIC_TOO_LONG_EC (DDSC_LOG_BASE + 1103)`
Failed to create a topic because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)
- `#define DDSC_LOG_TYPE_TOO_LONG_EC (DDSC_LOG_BASE + 1104)`
Failed to create a type because the name exceeded the maximum length of 255 octets (excluding the terminating NUL)
- `#define DDSC_LOG_LOOKUP_TYPE_PLUGIN_EC (DDSC_LOG_BASE + 1105)`
A type-plugin for the given type could not be found.
- `#define DDSC_LOG_TYPE_KEY_TYPE_EC (DDSC_LOG_BASE + 1106)`
Two types have incompatible keys.
- `#define DDSC_LOG_DUPLICATE_GUID_PREFIX_EC (DDSC_LOG_BASE + 1107)`
A DomainParticipant with the specified GUID already exists in the DomainParticipant factory.
- `#define DDSC_LOG_DISC_LOCAL_PARTICIPANT_ENABLED_EC (DDSC_LOG_BASE + 1200)`
Discovery plugin failed its update after a local DomainParticipant was enabled.
- `#define DDSC_LOG_DISC_BEFORE_LOCAL_PARTICIPANT_CREATED_EC (DDSC_LOG_BASE + 1201)`
Discovery plugin failed its update before a local DomainParticipant was created.
- `#define DDSC_LOG_DISC_AFTER_LOCAL_PARTICIPANT_CREATED_EC (DDSC_LOG_BASE + 1202)`
Discovery plugin failed its update after a local DomainParticipant was created.
- `#define DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_ENABLED_EC (DDSC_LOG_BASE + 1203)`
Discovery plugin failed its update after a local DataReader was enabled.
- `#define DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_DELETED_EC (DDSC_LOG_BASE + 1204)`
Discovery plugin failed its update after a local DataReader was deleted.
- `#define DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_ENABLED_EC (DDSC_LOG_BASE + 1205)`
Discovery plugin failed its update after a local DataWriter was enabled.
- `#define DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_DELETED_EC (DDSC_LOG_BASE + 1206)`
Discovery plugin failed its update after a local DataWriter was deleted.
- `#define DDSC_LOG_DISC_REMOTE_PARTICIPANT_EXPIRED_EC (DDSC_LOG_BASE + 1207)`
Discovery plugin failed its update after a remote DomainParticipant's liveliness expired.
- `#define DDSC_LOG_DISC_ADD_PEER_EC (DDSC_LOG_BASE + 1208)`
Failed to add a peer with discovery plugin.
- `#define DDSC_LOG_CHECKSUM_REQUIRED_EC (DDSC_LOG_BASE + 1301)`
A participant requires checksum, but the discovered participant does not send a checksum.
- `#define DDSC_LOG_CHECKSUM_INCOMPATIBLE_EC (DDSC_LOG_BASE + 1302)`
A participant does not support checksum's sent by a discovered participant.
- `#define DDSC_LOG_CHECKSUM_INCONSISTENT_CLASS_EC (DDSC_LOG_BASE + 1303)`

The checksum class id for a custom checksum does match.

- #define `DDSC_LOG_CHECKSUM_INCONSISTENT_COMPUTE_EC` (DDSC_LOG_BASE + 1304)

The computed_crc_kind is inconsistent with what is allowed or supported.

- #define `DDSC_LOG_CHECKSUM_INCONSISTENT_ALLOW_EC` (DDSC_LOG_BASE + 1305)

The allowed_crc_mask is inconsistent with what is supported.

8.4.1 Detailed Description

DDS_C module log codes.

8.5 disc_dpde_log.h File Reference

Macros

- #define `DPDE_LOG_CDR_SET_OFFSET_EC` (DPDE_LOG_BASE + 1)
Failed to set current offset of stream.
- #define `DPDE_LOG_SERIALIZE_ENTITY_NAME_EC` (DPDE_LOG_BASE + 2)
Failed to serialize Entity Name parameter.
- #define `DPDE_LOG_SERIALIZE_TOPIC_NAME_EC` (DPDE_LOG_BASE + 3)
Failed to serialize Topic Name parameter.
- #define `DPDE_LOG_SERIALIZE_TYPE_NAME_EC` (DPDE_LOG_BASE + 4)
Failed to serialize Type Name parameter.
- #define `DPDE_LOG_SERIALIZE_GUID_EC` (DPDE_LOG_BASE + 5)
Failed to serialize GUID key.
- #define `DPDE_LOG_SERIALIZE_DEFAULT_UNICAST_EC` (DPDE_LOG_BASE + 6)
Failed to serialize Default Unicast Locator parameter.
- #define `DPDE_LOG_SERIALIZE_PROTOCOL_VERSION_EC` (DPDE_LOG_BASE + 7)
Failed to serialize Protocol Version parameter.
- #define `DPDE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC` (DPDE_LOG_BASE + 8)
Failed to deserialize Protocol Version parameter.
- #define `DPDE_LOG_DESERIALIZE_VENDOR_ID_EC` (DPDE_LOG_BASE + 9)
Failed to deserialize Vendor ID parameter.
- #define `DPDE_LOG_SERIALIZE_VENDOR_ID_EC` (DPDE_LOG_BASE + 10)
Failed to serialize Vendor ID parameter.
- #define `DPDE_LOG_SERIALIZE_PRODUCT_VERSION_EC` (DPDE_LOG_BASE + 11)
Failed to serialize Product Version parameter.
- #define `DPDE_LOG_SERIALIZE_LEASE_DURATION_EC` (DPDE_LOG_BASE + 12)
Failed to serialize Lease Duration parameter.
- #define `DPDE_LOG_CREATE_PUBLISHER_EC` (DPDE_LOG_BASE + 13)
Failed to create Publisher for built-in endpoint DataWriters.
- #define `DPDE_LOG_CREATE_SUBSCRIBER_EC` (DPDE_LOG_BASE + 14)
Failed to create Subscriber for built-in endpoint DataReaders.
- #define `DPDE_LOG_CREATE_PARTICIPANT_DISCOVERY_EC` (DPDE_LOG_BASE + 15)
Failed to create built-in endpoint for participant discovery after creating local user DomainParticipant.
- #define `DPDE_LOG_CREATE_PUBLICATION_DISCOVERY_EC` (DPDE_LOG_BASE + 16)
Failed to create built-in publication endpoint after creating local user DomainParticipant.
- #define `DPDE_LOG_CREATE_SUBSCRIPTION_DISCOVERY_EC` (DPDE_LOG_BASE + 17)
Failed to create built-in subscription endpoint after creating local DomainParticipant.
- #define `DPDE_LOG_DELETE_SUBSCRIPTION_TOPIC_EC` (DPDE_LOG_BASE + 18)
Failed to delete subscription built-in topic.

- `#define DPDE_LOG_DELETE_PUBLICATION_TOPIC_EC` (DPDE_LOG_BASE + 19)
Failed to delete publication built-in topic.
- `#define DPDE_LOG_DELETE_PARTICIPANT_TOPIC_EC` (DPDE_LOG_BASE + 20)
Failed to delete participant built-in topic.
- `#define DPDE_LOG_DISPOSE_PARTICIPANT_EC` (DPDE_LOG_BASE + 21)
Failed to dispose built-in participant when deleting DomainParticipant.
- `#define DPDE_LOG_ALLOCATE_DPDE_EC` (DPDE_LOG_BASE + 22)
Out of memory to allocate discovery plugin.
- `#define DPDE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC` (DPDE_LOG_BASE + 23)
Failed to serialize Builtin Endpoint Mask parameter.
- `#define DPDE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC` (DPDE_LOG_BASE + 24)
Failed to deserialize Builtin Endpoint Mask parameter.
- `#define DPDE_LOG_DESERIALIZE_UNKNOWN_PID_EC` (DPDE_LOG_BASE + 25)
Deserialized a parameter of unknown ID.
- `#define DPDE_LOG_ANNOUNCEMENT_EC` (DPDE_LOG_BASE + 26)
Failed to write a dynamic participant discovery message.
- `#define DPDE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC` (DPDE_LOG_BASE + 27)
Failed to schedule an event to assert the next participant discovery announcement.
- `#define DPDE_LOG_ADVANCE_SN_EC` (DPDE_LOG_BASE + 28)
Failed to advance the sequence number of a participant discovery announcement.
- `#define DPDE_LOG_ANNOUNCE_WRITE_EC` (DPDE_LOG_BASE + 29)
Failed to write initial participant discovery announcement.
- `#define DPDE_LOG_SCHEDULE_FAST_ASSERTION_EC` (DPDE_LOG_BASE + 30)
Failed to schedule event for asserting participant discovery announcements.
- `#define DPDE_LOG_REGISTER_TYPE_EC` (DPDE_LOG_BASE + 32)
Failed to register a built-in type for discovery.
- `#define DPDE_LOG_CREATE_TOPIC_EC` (DPDE_LOG_BASE + 33)
Failed to create a topic for discovery.
- `#define DPDE_LOG_CREATE_WRITER_EC` (DPDE_LOG_BASE + 34)
Failed to create a built-in DataWriter for discovery.
- `#define DPDE_LOG_CREATE_READER_EC` (DPDE_LOG_BASE + 35)
Failed to create a built-in DataReader for discovery.
- `#define DPDE_LOG_ASSERT_REMOTE_PARTICIPANT_EC` (DPDE_LOG_BASE + 36)
Failed to assert and complete discovery of a remote participant.
- `#define DPDE_LOG_ENABLE_REMOTE_PARTICIPANT_EC` (DPDE_LOG_BASE + 37)
Failed to enable and complete discovery of a remote participant.
- `#define DPDE_LOG_REFRESH_REMOTE_PARTICIPANT_EC` (DPDE_LOG_BASE + 38)
Failed to refresh liveliness for a discovered remote participant.
- `#define DPDE_LOG_TAKE_PARTICIPANT_SAMPLE_EC` (DPDE_LOG_BASE + 39)
Failed to take a sample from a participant discovery DataReader.
- `#define DPDE_LOG_INVALID_PARTICIPANT_SAMPLE_EC` (DPDE_LOG_BASE + 40)
Received a participant discovery announcement with invalid state.
- `#define DPDE_LOG_RETURN_PARTICIPANT_SAMPLE_EC` (DPDE_LOG_BASE + 41)
Failed to return a loan on a participant discovery sample.
- `#define DPDE_LOG_DISPOSE_PUBLICATION_EC` (DPDE_LOG_BASE + 42)
Failed to dispose an instance for a publication.
- `#define DPDE_LOG_DISPOSE_SUBSCRIPTION_EC` (DPDE_LOG_BASE + 43)
Failed to dispose an instance of a subscription.
- `#define DPDE_LOG_ASSERT_REMOTE_PUBLICATION_EC` (DPDE_LOG_BASE + 44)
Failed to assert and complete discovery of a remote publication.
- `#define DPDE_LOG_ASSERT_REMOTE_SUBSCRIPTION_EC` (DPDE_LOG_BASE + 45)

- Failed to assert and complete discovery of a remote subscription.*
- #define `DPDE_LOG_TAKE_PUBLICATION_SAMPLE_EC` (DPDE_LOG_BASE + 46)
Failed to take a sample from a publication discovery DataReader.
- #define `DPDE_LOG_REMOVE_REMOTE_PUBLICATION_EC` (DPDE_LOG_BASE + 47)
Failed to dispose or unregister a remote publication instance.
- #define `DPDE_LOG_INVALID_PUBLICATION_SAMPLE_EC` (DPDE_LOG_BASE + 48)
Received a publication discovery announcement with invalid state.
- #define `DPDE_LOG_RETURN_PUBLICATION_SAMPLE_EC` (DPDE_LOG_BASE + 49)
Failed to return a loan on a publication discovery sample.
- #define `DPDE_LOG_TAKE_SUBSCRIPTION_SAMPLE_EC` (DPDE_LOG_BASE + 51)
Failed to take a sample from a subscription discovery DataReader.
- #define `DPDE_LOG_INVALID_SUBSCRIPTION_SAMPLE_EC` (DPDE_LOG_BASE + 52)
Received a subscription discovery sample with invalid state.
- #define `DPDE_LOG_RETURN_SUBSCRIPTION_SAMPLE_EC` (DPDE_LOG_BASE + 53)
Failed to return a loan on a subscription discovery sample.
- #define `DPDE_LOG_REMOVE_REMOTE_SUBSCRIPTION_EC` (DPDE_LOG_BASE + 54)
Failed to dispose or unregister a remote subscription instance.
- #define `DPDE_LOG_LOCATORS_FULL_EC` (DPDE_LOG_BASE + 55)
A locator sequence is full, the remaining locators of the specified kind are dropped.
- #define `DPDE_LOG_DESERIALIZE_LOCATOR_EC` (DPDE_LOG_BASE + 56)
Failed to deserialize a locator of the specified kind.
- #define `DPDE_LOG_UNSUPPORTED_LOCATOR_EC` (DPDE_LOG_BASE + 57)
The locator kind is not supported by any transport registered with the domain participant and is dropped.
- #define `DPDE_LOG_ADD_PEER_EC` (DPDE_LOG_BASE + 58)
Failed to add the specified entity as a peer.
- #define `DPDE_LOG_REMOVE_PEER_EC` (DPDE_LOG_BASE + 59)
Failed to add the specified entity as a peer.
- #define `DPDE_LOG_SERIALIZE_PRESENTATION_EC` (DPDE_LOG_BASE + 60)
Failed to serialize Presentation parameter.
- #define `DPDE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC` (DPDE_LOG_BASE + 61)
A participant with incompatible checksum was discovered and ignored.

8.5.1 Detailed Description

8.6 disc_dpse_log.h File Reference

Macros

- #define `DPSE_LOG_INVALID_PEER_ADDRESS_EC` (DPSE_LOG_BASE + 1)
A peer address string specifies an invalid address.
- #define `DPSE_LOG_CREATE_DISCOVERY_PUBLISHER_EC` (DPSE_LOG_BASE + 2)
Failed to create the publisher for a participant discovery writer.
- #define `DPSE_LOG_CREATE_DISCOVERY_SUBSCRIBER_EC` (DPSE_LOG_BASE + 3)
Failed to create the subscriber for a participant discovery datareader.
- #define `DPSE_LOG_REGISTER_TYPE_EC` (DPSE_LOG_BASE + 4)
Failed to register a built-in participant discovery type.
- #define `DPSE_LOG_CREATE_TOPIC_EC` (DPSE_LOG_BASE + 5)
Failed to create a topic for the built-in participant discovery topic.
- #define `DPSE_LOG_CREATE_WRITER_EC` (DPSE_LOG_BASE + 6)
Failed to create a DataWriter for participant discovery.

- `#define DPSE_LOG_CREATE_READER_EC` (DPSE_LOG_BASE + 7)
Failed to create a DataReader for participant discovery.
- `#define DPSE_LOG_DISPOSE_EC` (DPSE_LOG_BASE + 8)
Failed to dispose a participant discovery instance.
- `#define DPSE_LOG_ANNOUNCE_LOCAL_PARTICIPANT_DELETION_EC` (DPSE_LOG_BASE + 9)
Failed to dispose a participant discovery instance upon deletion.
- `#define DPSE_LOG_DELETE_WRITER_EC` (DPSE_LOG_BASE + 10)
Failed to delete a participant discovery DataWriter.
- `#define DPSE_LOG_DELETE_PUBLISHER_EC` (DPSE_LOG_BASE + 11)
Failed to delete a participant discovery Publisher.
- `#define DPSE_LOG_DELETE_READER_EC` (DPSE_LOG_BASE + 12)
Failed to delete a participant discovery DataReader.
- `#define DPSE_LOG_DELETE_TOPIC_EC` (DPSE_LOG_BASE + 13)
Failed to delete a participant discovery Topic.
- `#define DPSE_LOG_DELETE_SUBSCRIBER_EC` (DPSE_LOG_BASE + 14)
Failed to delete a participant discovery Subscriber.
- `#define DPSE_KEYLIST_OBJECT` (1)
The list keeping track of asserted participant keys.
- `#define DPSE_PARTICIPANTNAMES_OBJECT` (2)
The list keeping track of asserted participant names.
- `#define DPSE_PARTICIPANTNAMEINDEX_OBJECT` (3)
The index keeping track of asserted participant names.
- `#define DPSE_PARTICIPANTANNOUNCEMENT_OBJECT` (4)
The participant announcement timeout object.
- `#define DPSE_DISCOVERYPLUGIN_OBJECT` (5)
A discovery plugin object.
- `#define DPSE_PARTICIPANTBUILTINTOPICDATA_OBJECT` (6)
ParticipantBuiltinTopicData.
- `#define DPSE_PARTICIPANTENTITYNAME_OBJECT` (7)
ParticipantBuiltinTopicData entity-name object.
- `#define DPSE_ENABLEDPARTICIPANTINDEX_OBJECT` (8)
The index keeping track of enabled participant names.
- `#define DPSE_PARTICIPANTPOOL_OBJECT` (10)
The pool of asserted participants.
- `#define DPSE_LOG_OBJECT_ALLOCATE_EC` (DPSE_LOG_BASE + 15)
Failed to allocate an object of the specified kind.
- `#define DPSE_LOG_OBJECT_INITIALIZE_EC` (DPSE_LOG_BASE + 16)
Failed to initialize an object of the specified kind.
- `#define DPSE_LOG_OBJECT_FINALIZE_EC` (DPSE_LOG_BASE + 17)
Failed to finalize an object of the specified kind.
- `#define DPSE_LOG_OBJECT_DELETE_EC` (DPSE_LOG_BASE + 18)
Failed to delete an object of the specified kind.
- `#define DPSE_LOG_OBJECT_INVALID_EC` (DPSE_LOG_BASE + 19)
The object was invalid in the context it was used.
- `#define DPSE_LOG_CDR_SET_POSITION_EC` (DPSE_LOG_BASE + 20)
Failed to set the CDR stream position.
- `#define DPSE_LOG_CDR_LONG_KIND` 1
CDR Long kind.
- `#define DPSE_LOG_CDR_PID_KIND` 2
CDR PID kind, the pid argument specifies which pid.
- `#define DPSE_LOG_CDR_SERIALIZE_EC` (DPSE_LOG_BASE + 21)

- Failed to serialize the specified kind.*

 - #define [DPSE_LOG_CDR_DESERIALIZE_EC](#) (DPSE_LOG_BASE + 22)
- Failed to deserialize the specified kind.*

 - #define [DPSE_LOG_SERIALIZE_GUID_EC](#) (DPSE_LOG_BASE + 23)
- Failed to serialize a GUID key parameter.*

 - #define [DPSE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC](#) (DPSE_LOG_BASE + 24)
- Failed to serialize a Builtin Endpoint Mask parameter.*

 - #define [DPSE_LOG_SERIALIZE_PROTOCOL_VERSION_EC](#) (DPSE_LOG_BASE + 25)
- Failed to serialize a Protocol Version parameter.*

 - #define [DPSE_LOG_SERIALIZE_VENDOR_ID_EC](#) (DPSE_LOG_BASE + 26)
- Failed to serialize a Vendor ID parameter.*

 - #define [DPSE_LOG_SERIALIZE_DEFAULT_UNICAST_EC](#) (DPSE_LOG_BASE + 27)
- Failed to serialize a Default Unicast Locator parameter.*

 - #define [DPSE_LOG_SERIALIZE_META_UNICAST_EC](#) (DPSE_LOG_BASE + 28)
- Failed to serialize a Meta Unicast Locator parameter.*

 - #define [DPSE_LOG_SERIALIZE_META_MULTICAST_EC](#) (DPSE_LOG_BASE + 29)
- Failed to serialize a Meta Multicast Locator parameter.*

 - #define [DPSE_LOG_SERIALIZE_LEASE_DURATION_EC](#) (DPSE_LOG_BASE + 30)
- Failed to serialize a Lease Duration parameter.*

 - #define [DPSE_LOG_SERIALIZE_PRODUCT_VERSION_EC](#) (DPSE_LOG_BASE + 31)
- Failed to serialize a Product Version parameter.*

 - #define [DPSE_LOG_DESERIALIZE_PRODUCT_VERSION_EC](#) (DPSE_LOG_BASE + 32)
- Failed to serialize a Product Version parameter.*

 - #define [DPSE_LOG_SERIALIZE_PARTICIPANT_NAME_EC](#) (DPSE_LOG_BASE + 33)
- Failed to serialize a Participant Name parameter.*

 - #define [DPSE_LOG_DESERIALIZE_GUID_EC](#) (DPSE_LOG_BASE + 34)
- Failed to deserialize a GUID key parameter.*

 - #define [DPSE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC](#) (DPSE_LOG_BASE + 35)
- Failed to deserialize a Builtin Endpoint Mask parameter.*

 - #define [DPSE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC](#) (DPSE_LOG_BASE + 36)
- Failed to deserialize a Protocol Version parameter.*

 - #define [DPSE_LOG_DESERIALIZE_VENDOR_ID_EC](#) (DPSE_LOG_BASE + 37)
- Failed to deserialize a Vendor ID parameter.*

 - #define [DPSE_LOG_DESERIALIZE_TOO_MANY_LOCATORS_EC](#) (DPSE_LOG_BASE + 38)
- Cannot deserialize another locator parameter, having reached maximum of 4 unicast or 4 multicast locators.*

 - #define [DPSE_LOG_DESERIALIZE_PARTICIPANT_NAME_EC](#) (DPSE_LOG_BASE + 39)
- Failed to deserialize a Participant Name parameter.*

 - #define [DPSE_LOG_DESERIALIZE_UNKNOWN_PID_EC](#) (DPSE_LOG_BASE + 40)
- Failed to deserialize a parameter with an unknown ID.*

 - #define [DPSE_LOG_ANNOUNCEMENT_EC](#) (DPSE_LOG_BASE + 41)
- Failed to send a participant discovery announcement.*

 - #define [DPSE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC](#) (DPSE_LOG_BASE + 42)
- Failed to schedule an event to send the next participant discovery announcement.*

 - #define [DPSE_LOG_ADVANCE_SN_EC](#) (DPSE_LOG_BASE + 43)
- Failed to advance the sequence number of a participant discovery announcement.*

 - #define [DPSE_LOG_ANNOUNCE_WRITE_EC](#) (DPSE_LOG_BASE + 44)
- Failed to write a participant discovery announcement message.*

 - #define [DPSE_LOG_SCHEDULE_FAST_ASSERTION_EC](#) (DPSE_LOG_BASE + 45)
- Failed to schedule an event to send the next participant discovery announcement.*

 - #define [DPSE_LOG_PARTICIPANT_TAKE_EC](#) (DPSE_LOG_BASE + 46)
- Failed to take a sample from a participant discovery DataReader.*

- `#define DPSE_LOG_ASSERT_REMOTE_PARTICIPANT_EC` (DPSE_LOG_BASE + 47)
Failed to assert remote participant.
- `#define DPSE_LOG_ON_ASSERT_REMOTE_PARTICIPANT_EC` (DPSE_LOG_BASE + 48)
Failed to assert and complete discovery of a remote participant.
- `#define DPSE_LOG_ADD_ANONYMOUS_ROUTE_EC` (DPSE_LOG_BASE + 49)
Failed to add an anonymous route.
- `#define DPSE_LOG_DELETE_ANONYMOUS_ROUTE_EC` (DPSE_LOG_BASE + 50)
Failed to delete an anonymous route.
- `#define DPSE_LOG_MAX_REMOTE_PARTICIPANT_EC` (DPSE_LOG_BASE + 51)
Exceeded resource limit, remote_participant_allocation.
- `#define DPSE_LOG_REFRESH_REMOTE_PARTICIPANT_EC` (DPSE_LOG_BASE + 52)
Failed to refresh liveliness for a remote participant.
- `#define DPSE_LOG_RESET_REMOTE_PARTICIPANT_EC` (DPSE_LOG_BASE + 53)
Failed to reset liveliness for a remote participant.
- `#define DPSE_LOG_INVALID_DISCOVERY_SAMPLE_EC` (DPSE_LOG_BASE + 54)
Received a participant discovery announcement with invalid state.
- `#define DPSE_LOG_RETURN_DISCOVERY_SAMPLE_EC` (DPSE_LOG_BASE + 55)
Failed to return a loan on a participant discovery announcement.
- `#define DPSE_LOG_SERIALIZE_MULTICAST_EC` (DPSE_LOG_BASE + 56)
Failed to serialize a Multicast Locator parameter.
- `#define DPSE_LOG_GET_DDS_PROPERTIES_EC` (DPSE_LOG_BASE + 57)
Failed to serialize a Multicast Locator parameter.
- `#define DPSE_TOPIC_ENTITY` 1
A DDS Topic entity.
- `#define DPSE_PUBLISHER_ENTITY` 2
A DDS Publisher entity.
- `#define DPSE_SUBSCRIBER_ENTITY` 3
A DDS Subscriber entity.
- `#define DPSE_LOG_ENTITY_ENABLE_EC` (DPSE_LOG_BASE + 58)
Failed to enable the specified entity kind.
- `#define DPSE_LOG_DEFAULT_UNICAST_LOCATOR_SEQUENCE` 1
The default unicast locator sequence.
- `#define DPSE_LOG_DEFAULT_MULTICAST_LOCATOR_SEQUENCE` 2
The default multicast locator sequence.
- `#define DPSE_LOG_META_UNICAST_LOCATOR_SEQUENCE` 3
The meta unicast locator sequence.
- `#define DPSE_LOG_META_MULTICAST_LOCATOR_SEQUENCE` 4
The meta multicast locator sequence.
- `#define DPSE_LOG_SEQUENCE_SETMAX_EC` (DPSE_LOG_BASE + 59)
Failed to set the maximum length of a sequence of the specified kind.
- `#define DPSE_LOG_SEQUENCE_SETLENGTH_EC` (DPSE_LOG_BASE + 60)
Failed to set the length of a sequence of the specified kind.
- `#define DPSE_LOG_SEQUENCE_GETREF_EC` (DPSE_LOG_BASE + 61)
Failed to get a reference at the specified index for a sequence of the specified kind.
- `#define DPSE_LOG_SEQUENCE_INITIALIZE_EC` (DPSE_LOG_BASE + 62)
Failed to initialize a sequence of the specified kind.
- `#define DPSE_LOG_SEQUENCE_FINALIZE_EC` (DPSE_LOG_BASE + 63)
Failed to finalize a sequence of the specified kind.
- `#define DPSE_LOG_SEQUENCE_COPY_EC` (DPSE_LOG_BASE + 64)
Failed to copy a sequence of the specified kind.
- `#define DPSE_LOG_GET_TIMER_EC` (DPSE_LOG_BASE + 65)

Failed to get timer.

- `#define DPSE_LOG_UNSUPPORTED_LOCATOR_EC (DPSE_LOG_BASE + 66)`

The locator kind is not supported by any transport registered with the domain participant and is dropped.

- `#define DPSE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC (DPSE_LOG_BASE + 67)`

A participant with incompatible checksum was discovered and ignored.

8.6.1 Detailed Description

8.6.2 Macro Definition Documentation

8.6.2.1 `#define DPSE_KEYLIST_OBJECT (1)`

The list keeping track of asserted participant keys.

8.6.2.2 `#define DPSE_PARTICIPANTNAMES_OBJECT (2)`

The list keeping track of asserted participant names.

8.6.2.3 `#define DPSE_PARTICIPANTNAMEINDEX_OBJECT (3)`

The index keeping track of asserted participant names.

8.6.2.4 `#define DPSE_PARTICIPANTANNOUNCEMENT_OBJECT (4)`

The participant announcement timeout object.

8.6.2.5 `#define DPSE_DISCOVERYPLUGIN_OBJECT (5)`

A discovery plugin object.

8.6.2.6 `#define DPSE_PARTICIPANTBUILTINTOPICDATA_OBJECT (6)`

ParticipantBuiltinTopicData.

8.6.2.7 `#define DPSE_PARTICIPANTENTITYNAME_OBJECT (7)`

ParticipantBuiltinTopicData entity-name object.

8.6.2.8 `#define DPSE_ENABLEDPARTICIPANTINDEX_OBJECT (8)`

The index keeping track of enabled participant names.

8.6.2.9 `#define DPSE_PARTICIPANTPOOL_OBJECT (10)`

The pool of asserted participants.

8.6.2.10 `#define DPSE_LOG_CDR_LONG_KIND 1`

CDR Long kind.

8.6.2.11 `#define DPSE_LOG_CDR_PID_KIND 2`

CDR PID kind, the pid argument specifies which pid.

8.6.2.12 `#define DPSE_TOPIC_ENTITY 1`

A DDS Topic entity.

8.6.2.13 #define DPSE_PUBLISHER_ENTITY 2

A DDS Publisher entity.

8.6.2.14 #define DPSE_SUBSCRIBER_ENTITY 3

A DDS Subscriber entity.

8.6.2.15 #define DPSE_LOG_DEFAULT_UNICAST_LOCATOR_SEQUENCE 1

The default unicast locator sequence.

8.6.2.16 #define DPSE_LOG_DEFAULT_MULTICAST_LOCATOR_SEQUENCE 2

The default multicast locator sequence.

8.6.2.17 #define DPSE_LOG_META_UNICAST_LOCATOR_SEQUENCE 3

The meta unicast locator sequence.

8.6.2.18 #define DPSE_LOG_META_MULTICAST_LOCATOR_SEQUENCE 4

The meta multicast locator sequence.

8.7 netio_log.h File Reference

NETIO module log codes.

Macros

- #define [NETIO_LOG_GETHOST_BYNAME_EC](#) (NETIO_LOG_BASE + 1)
Failed to get host address from a string representation.
- #define [NETIO_LOG_BIND_NEW_EC](#) (NETIO_LOG_BASE + 2)
Failed to allocate memory for a bind-resolver.
- #define [NETIO_LOG_BIND_NEW_RTABLE_EC](#) (NETIO_LOG_BASE + 3)
Failed to create a database table for routes of a bind-resolver.
- #define [NETIO_LOG_BIND_CREATE_RTABLE_EC](#) (NETIO_LOG_BASE + 4)
Failed to create route record.
- #define [NETIO_LOG_BIND_DELETE_RTABLE_EC](#) (NETIO_LOG_BASE + 5)
Failed to delete a database table for routes of a bind_resolver.
- #define [NETIO_LOG_BIND_SET_LENGTH_EC](#) (NETIO_LOG_BASE + 6)
Failed to set the length for a sequence of resolved addresses.
- #define [NETIO_LOG_BIND_SET_MAX_EC](#) (NETIO_LOG_BASE + 7)
Failed to set the maximum length of an internal sequence of addresses.
- #define [NETIO_LOG_BIND_EXTERNAL_FAILED_EC](#) (NETIO_LOG_BASE + 8)
An interface failed on bind_external.
- #define [NETIO_LOG_UNBIND_EXTERNAL_FAILED_EC](#) (NETIO_LOG_BASE + 10)
An interface failed on unbind_external.
- #define [NETIO_LOG_ROUTE_RTABLE_ALLOC_EC](#) (NETIO_LOG_BASE + 11)
Failed to allocate memory for a route resolver.
- #define [NETIO_LOG_ROUTE_RTABLE_CREATE_EC](#) (NETIO_LOG_BASE + 12)
Failed to create a database table for routes of a route-resolver.
- #define [NETIO_LOG_ROUTE_RTABLE_DELETE_EC](#) (NETIO_LOG_BASE + 13)
Failed to delete a database table for routes of a route-resolver.

- #define [NETIO_LOG_ROUTE_RTABLE_ADD_EC](#) (NETIO_LOG_BASE + 14)
Failed to create route resolver record.
- #define [NETIO_LOG_ROUTE_RTABLE_UPDATE_EC](#) (NETIO_LOG_BASE + 15)
Failed to find routes to update for a route-resolver.
- #define [NETIO_LOG_AR_ALLOC_FAILED_EC](#) (NETIO_LOG_BASE + 16)
Failed to allocate memory for address resolver.
- #define [NETIO_LOG_AR_ADD_INVALID_INTERFACE_EC](#) (NETIO_LOG_BASE + 17)
An attempt was made to add an existing interface with a different port resolver to the address resolver.
- #define [NETIO_LOG_AR_ADDRESS_RESOLVE_FAILED_EC](#) (NETIO_LOG_BASE + 18)
An interface failed to resolve an address.
- #define [NETIO_LOG_AR_PORT_RESOLVE_FAILED_EC](#) (NETIO_LOG_BASE + 19)
An interface failed to resolve a port.
- #define [NETIO_LOG_AR_CONTEXT_UNINITIALIZED_EC](#) (NETIO_LOG_BASE + 21)
NETIO_AddressResolver_resolve/get_next was called with an uninitialized context.
- #define [NETIO_LOG_AR_INVALID_TOKEN_EC](#) (NETIO_LOG_BASE + 22)
An invalid token was encountered in the address string.
- #define [NETIO_LOG_AR_ADDRESS_STRING_EXCEEDED_EC](#) (NETIO_LOG_BASE + 23)
The length of the address exceeded the address buffer passed in.
- #define [NETIO_LOG_INVALID_ADDRESS_INDEX_EC](#) (NETIO_LOG_BASE + 24)
An address index was not parsed correctly.
- #define [NETIO_LOG_ILLEGAL_ROUTE_OPERATION_EC](#) (NETIO_LOG_BASE + 25)
Illegal route operation attempted.
- #define [NETIO_LOG_SET_NAME_EC](#) (UDP_LOG_BASE + 26)
Failed to set the name of a runtime component interface.
- #define [NETIO_LOG_PACKET_SET_HEAD_EC](#) (UDP_LOG_BASE + 27)
Failed to set packet's head.
- #define [NETIO_LOG_PACKET_SET_TAIL_EC](#) (UDP_LOG_BASE + 28)
Failed to set packet's tail.
- #define [NETIO_LOG_PACKET_INITIALIZE_EC](#) (UDP_LOG_BASE + 29)
Failed to initialize packet.
- #define [NETIO_LOG_LOOP_DELETE_TABLE_EC](#) (NETIO_LOG_BASE + 100)
Failed to delete a database table for a NETIO loopback interface.
- #define [NETIO_LOG_LOOP_CREATE_TABLE_EC](#) (NETIO_LOG_BASE + 101)
Failed to create a database table for a NETIO loopback interface.
- #define [NETIO_LOG_LOOP_SELECT_TABLE_EC](#) (NETIO_LOG_BASE + 102)
Failed to select a valid record when sending with the NETIO loopback interface.
- #define [NETIO_LOG_LOOP_FWD_EC](#) (NETIO_LOG_BASE + 103)
Failed to send forward with the NETIO loopback interface.
- #define [NETIO_LOG_LOOP_BINDX_DB_EC](#) (NETIO_LOG_BASE + 104)
Failed to bind an external interface with the NETIO loopback interface.
- #define [NETIO_LOG_LOOP_UNBINDX_DB_EC](#) (NETIO_LOG_BASE + 105)
Failed to unbind an external interface with the NETIO loopback interface.
- #define [NETIO_LOG_LOOP_SET_LENGTH_EC](#) (NETIO_LOG_BASE + 106)
Failed to set the length of an address sequence.
- #define [NETIO_LOG_LOOP_INVALID_PROPERTY_EC](#) (NETIO_LOG_BASE + 107)
Invalid property passed in when creating a loopback interface.
- #define [NETIO_LOG_LOOP_INITIALIZE_FAILED_EC](#) (NETIO_LOG_BASE + 108)
Failed to initialize the loopback interface.
- #define [NETIO_LOG_LOOP_CURSOR_ERROR_EC](#) (NETIO_LOG_BASE + 109)
An error occurred with a cursor while iterating over a table.
- #define [NETIO_LOG_LOOP_INVALID_FACTORY_EC](#) (NETIO_LOG_BASE + 110)

- An invalid factory was passed in to create a loopback interface.*

 - #define [NETIO_LOG_LOOP_INVALID_COMPONENT_EC](#) (NETIO_LOG_BASE + 111)
- An invalid component was passed in to delete a loopback interface.*

 - #define [UDP_LOG_SOCKET_INIT_EC](#) (UDP_LOG_BASE + 200)
- Failed to initialize use of sockets.*

 - #define [UDP_LOG_GETHOSTNAME_EC](#) (UDP_LOG_BASE + 201)
- Failed to get the host address given the host name.*

 - #define [UDP_LOG_SOCKET_SET_MCAST_EC](#) (UDP_LOG_BASE + 202)
- Failed to set a socket for multicast loopback.*

 - #define [UDP_LOG_SOCKET_SET_MCASTIF_EC](#) (UDP_LOG_BASE + 203)
- Failed to set a socket for multicast bound to a specific interface.*

 - #define [UDP_LOG_SOCKET_CREATE_EC](#) (UDP_LOG_BASE + 204)
- Failed to create a new socket, the sysrc code is the OS return code and reason for failure.*

 - #define [UDP_LOG_SOCKET_SETFD_EC](#) (UDP_LOG_BASE + 205)
- Failed to set the close-on-exec flag for a socket.*

 - #define [UDP_LOG_SOCKET_SNDBUF_EC](#) (UDP_LOG_BASE + 206)
- Failed to set the socket send buffer size.*

 - #define [UDP_LOG_SOCKET_TTL_EC](#) (UDP_LOG_BASE + 207)
- Failed to set multicast TTL for a socket.*

 - #define [UDP_LOG_PACKET_INIT_EC](#) (UDP_LOG_BASE + 208)
- Failed to initialize a receive packet.*

 - #define [UDP_LOG_PACKET_HEAD_EC](#) (UDP_LOG_BASE + 209)
- Failed to set the head of a receive packet.*

 - #define [UDP_LOG_PACKET_FWD_EC](#) (UDP_LOG_BASE + 210)
- Failed reception upstream for a received packet.*

 - #define [UDP_LOG_NAT_DELETE_EC](#) (UDP_LOG_BASE + 211)
- Failed to delete a database record or index from an internal NAT.*

 - #define [UDP_LOG_FINALIZE_EC](#) (UDP_LOG_BASE + 212)
- Failed to finalize the parent NETIO interface.*

 - #define [UDP_LOG_SOCKET_REUSE_PORT_EC](#) (UDP_LOG_BASE + 213)
- Failed to set socket to reuse a port or address.*

 - #define [UDP_LOG_SOCKET_BIND_EC](#) (UDP_LOG_BASE + 214)
- Failed a socket bind.*

 - #define [UDP_LOG_SOCKET_ADDGRP_EC](#) (UDP_LOG_BASE + 215)
- Failed to add membership to a multicast group for a socket.*

 - #define [UDP_LOG_PORT_ENTRY_EC](#) (UDP_LOG_BASE + 216)
- Failed to create a bind entry for a receive port.*

 - #define [UDP_LOG_PORT_ALLOC_EC](#) (UDP_LOG_BASE + 217)
- Failed to allocate memory for a receive buffer of a specific port.*

 - #define [UDP_LOG_SOCKET_RXBUF_EC](#) (UDP_LOG_BASE + 218)
- Failed to set the receive buffer size of a socket.*

 - #define [UDP_LOG_PORT_ADD_EC](#) (UDP_LOG_BASE + 219)
- Failed to add a port entry record to a database table.*

 - #define [UDP_LOG_PORT_THREAD_EC](#) (UDP_LOG_BASE + 220)
- Failed to create a receive thread for a receive port.*

 - #define [UDP_LOG_SOCKET_IFLIST_EC](#) (UDP_LOG_BASE + 221)
- Failed to get a list of interface addresses.*

 - #define [UDP_LOG_SOCKET_IFFLAGS_EC](#) (UDP_LOG_BASE + 222)
- Failed to get the network interface flags or mask of a socket.*

 - #define [UDP_LOG_SOCKET_CLOSE_EC](#) (UDP_LOG_BASE + 223)
- Failed to close a socket.*

- `#define UDP_LOG_LOADLIB_EC (UDP_LOG_BASE + 224)`
Failed to load a dynamic library.
- `#define UDP_LOG_GET_BUF_SIZE_EC (UDP_LOG_BASE + 225)`
Failed to get interface buffer size.
- `#define UDP_LOG_GET_BUF_ALLOC_EC (UDP_LOG_BASE + 226)`
Out of memory to allocate interface buffer.
- `#define UDP_LOG_GET_IFLIST_EC (UDP_LOG_BASE + 227)`
Failed to get an interface list.
- `#define UDP_LOG_GET_NAMEINFO_EC (UDP_LOG_BASE + 228)`
Failed getnameinfo()
- `#define UDP_LOG_MULTICAST_ENABLE_EC (UDP_LOG_BASE + 229)`
Failed to enable multicast loopback.
- `#define UDP_LOG_UDP_CREATE_TABLE_EC (UDP_LOG_BASE + 230)`
Failed to create a database table for a UDP interface.
- `#define UDP_LOG_UDP_CREATE_INDEX_EC (UDP_LOG_BASE + 231)`
Failed to create an index for a UDP interface.
- `#define UDP_LOG_RECORD_EC (UDP_LOG_BASE + 232)`
Failed to create or insert a database record for a UDP interface.
- `#define UDP_LOG_PROPERTY_FINALIZE_EC (UDP_LOG_BASE + 233)`
Failed to finalize UDP interface factory property.
- `#define UDP_LOG_ALLOC_EC (UDP_LOG_BASE + 234)`
Failed to allocate memory for a UDP interface.
- `#define UDP_LOG_SEND_PING_EC (UDP_LOG_BASE + 235)`
Failed sending a ping message, upon adding a route or waking up a receive thread.
- `#define UDP_LOG_DROPGRP_EC (UDP_LOG_BASE + 236)`
Failed to drop membership from a multicast group.
- `#define UDP_LOG_GET_LENGTH_EC (UDP_LOG_BASE + 237)`
Length of address sequence exceeds its maximum.
- `#define UDP_LOG_SET_LENGTH_EC (UDP_LOG_BASE + 240)`
Failed to set the length of an address sequence.
- `#define UDP_LOG_WSA_STARTUP_EC (UDP_LOG_BASE + 241)`
Failed WSStartup()
- `#define UDP_LOG_WINSOCK_INCOMPATIBLE_EC (UDP_LOG_BASE + 242)`
Incompatible version of Winsock, must not be older than 2.0.
- `#define UDP_LOG_INCONSISTENT_MAX_MESSAGE_SIZE_EC (UDP_LOG_BASE + 243)`
Inconsistent max_message_size.
- `#define UDP_LOG_INITIALIZE_FAILED_EC (UDP_LOG_BASE + 244)`
Failed to initialize the loopback interface.
- `#define UDP_LOG_PORT_NOT_FOUND_EC (UDP_LOG_BASE + 245)`
Did not find the specified thread port.
- `#define UDP_LOG_CURSOR_ERROR_EC (UDP_LOG_BASE + 246)`
An error occurred while iterating over a cursor.
- `#define UDP_LOG_SEND_ERROR_EC (UDP_LOG_BASE + 247)`
Failed to send message.
- `#define UDP_LOG_MULTICAST_NOT_ENABLED_EC (UDP_LOG_BASE + 248)`
Multicast address ignored because multicast supported is not compiled in.
- `#define UDP_LOG_TRANSPORT_NO_INTERFACES_EC (UDP_LOG_BASE + 249)`
Transport does not have any interfaces.
- `#define UDP_LOG_RECV_ERROR_EC (UDP_LOG_BASE + 250)`
Failed to receive message.
- `#define UDP_TRANSFORM_LOG_KIND_RULE (1)`

- Rule object.*
- `#define UDP_TRANSFORM_LOG_KIND_ENTRY (2)`
- Entry object.*
- `#define UDP_TRANSFORM_LOG_KIND_FACTORY (3)`
- Factory object.*
- `#define UDP_TRANSFORM_LOG_KIND_FACTORY_INDEX (4)`
- Factory object.*
- `#define UDP_TRANSFORM_LOG_KIND_DST_INDEX (5)`
- Factory object.*
- `#define UDP_TRANSFORM_LOG_KIND_SRC_INDEX (6)`
- Factory object.*
- `#define UDP_TRANSFORM_LOG_KIND_TRANSFORM_SOURCE (7)`
- Rule object.*
- `#define UDP_TRANSFORM_LOG_KIND_TRANSFORM_DESTINATION (8)`
- Entry object.*
- `#define UDP_TRANSFORM_LOG_KIND_TRANSFORM_NETMASK_INDEX (9)`
- Mask object.*
- `#define UDP_TRANSFORM_LOG_INVALID_FACTORY_REFERENCE_EC (UDP_LOG_BASE + 300)`
- The returned factory reference is NULL.*
- `#define UDP_TRANSFORM_LOG_FACTORY_NOT_FOUND_EC (UDP_LOG_BASE + 301)`
- Could not find the factory used as part of a transform rule.*
- `#define UDP_TRANSFORM_LOG_ALLOC_EC (UDP_LOG_BASE + 302)`
- Failed to allocate entry of the specified kind.*
- `#define UDP_TRANSFORM_LOG_CREATE_EC (UDP_LOG_BASE + 303)`
- Failed to create a transformation of the specified kind.*
- `#define UDP_TRANSFORM_LOG_ADD_EC (UDP_LOG_BASE + 304)`
- Failed to add object of specified kind.*
- `#define UDP_TRANSFORM_LOG_FIND_EC (UDP_LOG_BASE + 305)`
- Failed to find a transformation.*
- `#define UDP_TRANSFORM_LOG_DUPLICATE_EC (UDP_LOG_BASE + 306)`
- Duplicate object.*
- `#define UDP_TRANSFORM_LOG_TRANSFORM_EC (UDP_LOG_BASE + 307)`
- Source or destination transformation function failed.*
- `#define UDP_TRANSFORM_LOG_INVALID_UDP_MODE_EC (UDP_LOG_BASE + 308)`
- Invalid UDP mode specified.*
- `#define UDP_TRANSFORM_LOG_INVALID_TUDP_LOCATOR_KIND_EC (UDP_LOG_BASE + 309)`
- Invalid transformation UDP locator kind specified.*

8.7.1 Detailed Description

NETIO module log codes.

8.7.2 Macro Definition Documentation

8.7.2.1 `#define UDP_TRANSFORM_LOG_KIND_RULE (1)`

Rule object.

8.7.2.2 `#define UDP_TRANSFORM_LOG_KIND_ENTRY (2)`

Entry object.

8.7.2.3 #define UDP_TRANSFORM_LOG_KIND_FACTORY (3)

Factory object.

8.7.2.4 #define UDP_TRANSFORM_LOG_KIND_FACTORY_INDEX (4)

Factory object.

8.7.2.5 #define UDP_TRANSFORM_LOG_KIND_DST_INDEX (5)

Factory object.

8.7.2.6 #define UDP_TRANSFORM_LOG_KIND_SRC_INDEX (6)

Factory object.

8.7.2.7 #define UDP_TRANSFORM_LOG_KIND_TRANSFORM_SOURCE (7)

Rule object.

8.7.2.8 #define UDP_TRANSFORM_LOG_KIND_TRANSFORM_DESTINATION (8)

Entry object.

8.7.2.9 #define UDP_TRANSFORM_LOG_KIND_TRANSFORM_NETMASK_INDEX (9)

Mask object.

8.8 osapi_semaphore.h File Reference

Semaphore interface definition.

Macros

- #define [OSAPI_SEMAPHORE_TIMEOUT_INFINITE](#) -1
- #define [OSAPI_SEMAPHORE_RESULT_OK](#) 0
- #define [OSAPI_SEMAPHORE_RESULT_TIMEOUT](#) 1
- #define [OSAPI_SEMAPHORE_RESULT_ERROR](#) 2

Typedefs

- typedef struct OSAPI_Semaphore [OSAPI_Semaphore_T](#)
Abstract semaphore object.

Functions

- [OSAPI_Semaphore_T * OSAPI_Semaphore_new](#) (void)
Create a semaphore.
- [RTI_BOOL OSAPI_Semaphore_delete](#) ([OSAPI_Semaphore_T *self](#))
Delete a semaphore.
- [RTI_BOOL OSAPI_Semaphore_take](#) ([OSAPI_Semaphore_T *self](#), [RTI_INT32](#) timeout_ms, [RTI_INT32 *fail_reason](#))
Take a semaphore.
- [RTI_BOOL OSAPI_Semaphore_give](#) ([OSAPI_Semaphore_T *self](#))
Give a semaphore.

8.8.1 Detailed Description

Semaphore interface definition.

8.8.2 Macro Definition Documentation

8.8.2.1 `#define OSAPI_SEMAPHORE_TIMEOUT_INFINITE -1`

Infinite timeout

8.8.2.2 `#define OSAPI_SEMAPHORE_RESULT_OK 0`

Semaphore ok

8.8.2.3 `#define OSAPI_SEMAPHORE_RESULT_TIMEOUT 1`

Semaphore timed out

8.8.2.4 `#define OSAPI_SEMAPHORE_RESULT_ERROR 2`

Semaphore error

8.9 `osapi_stdio.h` File Reference

Standard I/O interface definition.

8.9.1 Detailed Description

Standard I/O interface definition.

8.10 `osapi_time.h` File Reference

Time API definition.

8.10.1 Detailed Description

Time API definition. This file implements function to convert between the NtpTime format and other formats.

8.11 `reda_log.h` File Reference

REDA module log codes.

Macros

- `#define REDA_LOG_BUFFERPOOL_OUT_OF_RESOURCES_EC (REDA_LOG_BASE + 1)`
Not sufficient memory to allocate buffer-pool.
- `#define REDA_LOG_BUFFERPOOL_BUFFER_INITIALIZATION_FAILED_EC (REDA_LOG_BASE + 2)`
The buffer initialization method failed.
- `#define REDA_LOG_BUFFERPOOL_NOT_EMPTY_EC (REDA_LOG_BASE + 3)`
Cannot delete buffer-pool due to buffer(s) not returned to pool.
- `#define REDA_LOG_BUFFERPOOL_DOUBLE_FREE_EC (REDA_LOG_BASE + 4)`
A buffer was freed more than once.

- #define REDA_LOG_BUFFERPOOL_NULL_POINTER_EC (REDA_LOG_BASE + 5)
A pointer in a pool's linked list was NULL.
- #define REDA_LOG_SEQUENCE_COPY_FAILED_EC (REDA_LOG_BASE + 100)
An error occurred while copying a sequence.
- #define REDA_LOG_SEQUENCE_ALLOC_FAILED_EC (REDA_LOG_BASE + 101)
An error occurred while allocating space for a sequence.
- #define REDA_LOG_SEQUENCE_SET_MAX_FAILED_EC (REDA_LOG_BASE + 102)
An error occurred while setting the maximum sequence size.
- #define REDA_LOG_SEQUENCE_REALLOCATION_EC (REDA_LOG_BASE + 103)
An attempt was made to resize an already allocated sequence.
- #define REDA_LOG_SEQUENCE_INVALID_OPERATION_EC (REDA_LOG_BASE + 104)
An error occurred while operating on two sequences (copy/compare)
- #define REDA_LOG_SEQUENCE_INVALID_LENGTH_EC (REDA_LOG_BASE + 105)
An attempt was made to set an illegal length (length < 0 or length > max_length)
- #define REDA_LOG_SEQUENCE_INDEX_OUT_OF_BOUNDS_EC (REDA_LOG_BASE + 106)
An illegal sequence index was specified (index < 0 or index > length)
- #define REDA_LOG_STRING_ALLOC_FAILED_EC (REDA_LOG_BASE + 200)
An error occurred while allocating memory for a string.
- #define REDA_LOG_INDEX_FULL_EC (REDA_LOG_BASE + 300)
An attempt was made to add an element to a full index.
- #define REDA_LOG_INDEX_ENTRY_EXISTS_EC (REDA_LOG_BASE + 301)
An attempt was made to add an existing element to an index.
- #define REDA_LOG_INVALID_LIST_NODE_EC (REDA_LOG_BASE + 302)
An attempt was made to add an existing element to an index.

8.11.1 Detailed Description

REDA module log codes.

8.12 rh_sm_log.h File Reference

RH module log codes.

Macros

- #define RHSM_LOG_OUTSTANDING_SAMPLE_EC (RHSM_LOG_BASE + 1)
Failed to delete Reader History due to outstanding, unremoved sample.
- #define RHSM_LOG_NO_PROPERTY_QOS_EC (RHSM_LOG_BASE + 2)
Failed to create Reader History due to unspecified property Qos.
- #define RHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC (RHSM_LOG_BASE + 3)
KEEP_ALL History kind is not supported.
- #define RHSM_LOG_MAX_SAMPLE_TOO_SMALL_EC (RHSM_LOG_BASE + 4)
DataReaderQos.resource_limits.max_samples set too small.
- #define RHSM_LOG_TBF_NOT_SUPPORTED_EC (RHSM_LOG_BASE + 5)
Time-based filtering is not supported.
- #define RHSM_LOG_SAMPLE_INFO_POOL_EC (RHSM_LOG_BASE + 6)
Failed to allocate a buffer pool for sample info.
- #define RHSM_LOG_SAMPLE_POOL_EC (RHSM_LOG_BASE + 7)
Failed to allocate a buffer pool for sample pointers.
- #define RHSM_LOG_SAMPLE_PTR_ARRAY_EC (RHSM_LOG_BASE + 8)

- Failed to get sample buffer.*

 - #define [RHSM_LOG_INFO_ARRAY_EC](#) (RHSM_LOG_BASE + 9)
- Failed to get sample info buffer.*

 - #define [RHSM_LOG_GET_RECEPTION_TIMESTAMP_EC](#) (RHSM_LOG_BASE + 10)
- Failed to get time for reception timestamp.*

 - #define [RHSM_LOG_INVALID_INSTANCE_REPLACEMENT_EC](#) (RHSM_LOG_BASE + 11)
- An invalid instance replacement policy was specified.*

 - #define [RHSM_LOG_FAILED_TO_REMOVE_OLDEST_EC](#) (RHSM_LOG_BASE + 12)
- Failed to remove the oldest sample.*

 - #define [RHSM_LOG_SAMPLE_POOL_EMPTY_EC](#) (RHSM_LOG_BASE + 13)
- Sample pool empty, may have exceeded.*

 - #define [RHSM_LOG_RW_PRUNE_FAILED_EC](#) (RHSM_LOG_BASE + 14)
- Failed to prune remote writer entry.*

 - #define [RHSM_LOG_ENTRY_RESERVATION_FAILED_EC](#) (RHSM_LOG_BASE + 15)
- Failed to reserve an entry.*

 - #define [RHSM_LOG_RETURN_SAMPLE_EC](#) (RHSM_LOG_BASE + 16)
- Failed to return a sample.*

 - #define [RHSM_LOG_HISTORY_UPDATE_EC](#) (RHSM_LOG_BASE + 17)
- Failed to update the history queue.*

 - #define [RHSM_LOG_GET_TIME_EC](#) (RHSM_LOG_BASE + 18)
- Failed to get the system time.*

 - #define [RHSM_LOG_HISTORY_OBJECT](#) 1
- A history object.*

 - #define [RHSM_LOG_REMOVELIST_OBJECT](#) 2
- A removal list object.*

 - #define [RHSM_LOG_RWPOOL_OBJECT](#) 3
- A remote writer pool object.*

 - #define [RHSM_LOG_RWINDEX_OBJECT](#) 4
- A remote writer index object.*

 - #define [RHSM_LOG_KEY_OBJECT](#) 5
- A Key object.*

 - #define [RHSM_LOG_RW_OBJECT](#) 6
- A Key object.*

 - #define [RHSM_LOG_KEYPOOL_OBJECT](#) 7
- A Key pool object.*

 - #define [RHSM_LOG_OWNER_OBJECT](#) 8
- A key owner object.*

 - #define [RHSM_LOG_SAMPLEPOOL_OBJECT](#) 9
- A sample pool object.*

 - #define [RHSM_LOG_KEYINDEXPOOL_OBJECT](#) 10
- A key index pool.*

 - #define [RHSM_LOG_SAMPLEPTPOOL_OBJECT](#) 11
- A sample ptr pool object.*

 - #define [RHSM_LOG_SAMPLEINFO_OBJECT](#) 12
- A sample info pool.*

 - #define [RHSM_LOG_INSTANCEHANDLE_OBJECT](#) 13
- A sample info pool.*

 - #define [RHSM_LOG_OBJECT_ALLOCATE_EC](#) (RHSM_LOG_BASE + 19)
- Failed to allocate object of the specified kind.*

 - #define [RHSM_LOG_OBJECT_DELETE_EC](#) (RHSM_LOG_BASE + 20)
- Failed to delete object of the specified kind.*

- #define `RHSM_LOG_OBJECT_INDEX_EC` (`RHSM_LOG_BASE + 21`)
Failed to index an object of the specified kind.
- #define `RHSM_LOG_OBJECT_INVALID_EC` (`RHSM_LOG_BASE + 22`)
Invalid object specified.
- #define `RHSM_LOG_NO_PROPERTY_EC` (`RHSM_LOG_BASE + 23`)
No property when creating a reader history instance.
- #define `RHSM_LOG_DESTINATION_BY_SOURCE_NOT_SUPPORTED_EC` (`RHSM_LOG_BASE + 24`)
Destination ordering by `SOURCE_TIMESTAMP` ordering is not supported.
- #define `RHSM_LOG_RECOMMIT_FAILURE_EC` (`RHSM_LOG_BASE + 25`)
Failed to recommit sample.

8.12.1 Detailed Description

RH module log codes.

8.12.2 Macro Definition Documentation

8.12.2.1 #define RHSM_LOG_HISTORY_OBJECT 1

A history object.

8.12.2.2 #define RHSM_LOG_REMOVELIST_OBJECT 2

A removal list object.

8.12.2.3 #define RHSM_LOG_RWPOOL_OBJECT 3

A remote writer pool object.

8.12.2.4 #define RHSM_LOG_RWINDEX_OBJECT 4

A remote writer index object.

8.12.2.5 #define RHSM_LOG_KEY_OBJECT 5

A Key object.

8.12.2.6 #define RHSM_LOG_RW_OBJECT 6

A Key object.

8.12.2.7 #define RHSM_LOG_KEYPOOL_OBJECT 7

A Key pool object.

8.12.2.8 #define RHSM_LOG_OWNER_OBJECT 8

A key owner object.

8.12.2.9 #define RHSM_LOG_SAMPLEPOOL_OBJECT 9

A sample pool object.

8.12.2.10 #define RHSM_LOG_KEYINDEXPOOL_OBJECT 10

A key index pool.

8.12.2.11 `#define RHSM_LOG_SAMPLEPTPOOL_OBJECT 11`

A sample ptr pool object.

8.12.2.12 `#define RHSM_LOG_SAMPLEINFO_OBJECT 12`

A sample info pool.

8.12.2.13 `#define RHSM_LOG_INSTANCEHANDLE_OBJECT 13`

A sample info pool.

8.13 `rt_log.h` File Reference

Macros

- `#define RT_LOG_SET_IMMUTABLE_PROPERTY_EC (RT_LOG_BASE + 1)`
Failed to set registry property due to registry already being enabled.
- `#define RT_LOG_REGISTRY_INIT_FAILURE_EC (RT_LOG_BASE + 2)`
Failed to initialize registry due to failed table creation.
- `#define RT_LOG_REGISTRY_FINALIZE_EC (RT_LOG_BASE + 3)`
Failed to finalize registry due to failed table deletion.
- `#define RT_LOG_REGISTRY_EXISTS_EC (RT_LOG_BASE + 4)`
An attempt was made to register a factory that already existed.
- `#define RT_LOG_REGISTRY_REGISTER_EC (RT_LOG_BASE + 5)`
Error registering a component factory. May have exceeded RT_RegistryProperty.max_factories.
- `#define RT_LOG_REGISTRY_INIT_FACTORY_EC (RT_LOG_BASE + 7)`
A registered factory failed to initialize.
- `#define RT_LOG_REGISTRY_NAME_TOO_LONG_EC (RT_LOG_BASE + 8)`
Factory name longer than maximum length of 8.
- `#define RT_LOG_REGISTRY_INCONSISTENT_CID_EC (RT_LOG_BASE + 9)`
Factory name exists, but the class ID is not of the requested type.
- `#define RT_LOG_REGISTRY_NOT_INITIALIZED_EC (RT_LOG_BASE + 10)`
An attempt was made to operate on an uninitialized registry.

8.13.1 Detailed Description

8.14 `rtps_config.h` File Reference

RTPS Configuration.

8.14.1 Detailed Description

RTPS Configuration. Configurable build flags for RTPS functionality

8.15 `rtps_dll.h` File Reference

RTPS Configuration.

8.15.1 Detailed Description

RTPS Configuration. Configurable build flags for RTPS functionality

8.16 rtps_log.h File Reference

RTPS module log codes.

Macros

- #define `RTPS_LOG_INITIALIZE_INTERFACE_EC` (RTPS_LOG_BASE + 1)
Failed to initialize an RTPS interface.
- #define `RTPS_LOG_ALLOCATE_EC` (RTPS_LOG_BASE + 2)
Failed to allocate heap memory for internal resources.
- #define `RTPS_LOG_CREATE_ROUTE_TABLE_EC` (RTPS_LOG_BASE + 3)
Failed to create a database table for storing RTPS route info.
- #define `RTPS_LOG_DB_CREATE_ROUTE_PEER_INDEX_EC` (RTPS_LOG_BASE + 4)
Failed to create database index for route-peer info.
- #define `RTPS_LOG_DB_CREATE_ROUTE_XPORT_INDEX_EC` (RTPS_LOG_BASE + 5)
Failed to create database index for route-transport info.
- #define `RTPS_LOG_DB_CREATE_BIND_PEER_INDEX_EC` (RTPS_LOG_BASE + 6)
Failed to create database index for bind-peer info.
- #define `RTPS_LOG_INDEX_ADD_ENTRY_EC` (RTPS_LOG_BASE + 8)
Failed to add an entry to a database index.
- #define `RTPS_LOG_DB_SELECT_ALL_EC` (RTPS_LOG_BASE + 9)
Failed to select all entries of a database table.
- #define `RTPS_LOG_DB_SELECT_MATCH_EC` (RTPS_LOG_BASE + 10)
Matching entry not found in a database table.
- #define `RTPS_LOG_DB_SELECT_RANGE_EC` (RTPS_LOG_BASE + 11)
No matching entries found in a database table.
- #define `RTPS_LOG_DB_CREATE_ROUTE_ENTRY_EC` (RTPS_LOG_BASE + 12)
Failed to create database entry for route.
- #define `RTPS_LOG_DB_INSERT_ROUTE_ENTRY_EC` (RTPS_LOG_BASE + 13)
Failed to insert entry into route table.
- #define `RTPS_LOG_DB_REMOVE_ROUTE_ENTRY_EC` (RTPS_LOG_BASE + 14)
Failed to remove entry from route table.
- #define `RTPS_LOG_DB_DELETE_ROUTE_ENTRY_EC` (RTPS_LOG_BASE + 15)
Failed to delete entry from route table.
- #define `RTPS_LOG_DB_CREATE_BIND_ENTRY_EC` (RTPS_LOG_BASE + 16)
Failed to create database entry for bind.
- #define `RTPS_LOG_DB_INSERT_BIND_ENTRY_EC` (RTPS_LOG_BASE + 17)
Failed to insert an entry into the bind table.
- #define `RTPS_LOG_DB_REMOVE_BIND_ENTRY_EC` (RTPS_LOG_BASE + 18)
Failed to remove an entry from the bind table.
- #define `RTPS_LOG_DB_REMOVE_EXTERNAL_BIND_ENTRY_EC` (RTPS_LOG_BASE + 19)
Failed to remove an entry from the external bind table.
- #define `RTPS_LOG_DB_DELETE_EXTERNAL_BIND_ENTRY_EC` (RTPS_LOG_BASE + 20)
Failed to delete entry from external bind table.
- #define `RTPS_LOG_SEND_EC` (RTPS_LOG_BASE + 21)
Failed to send a packet due to a lower module failing to send the packet.
- #define `RTPS_LOG_RECEIVE_EC` (RTPS_LOG_BASE + 22)
Failed to receive a packet due to a higher module failing to receive the packet.
- #define `RTPS_LOG_ROUTE_PACKET_EC` (RTPS_LOG_BASE + 23)
Failed to send a packet, when routing to a lower module or peer.
- #define `RTPS_LOG_FORWARD_UPSTREAM_EC` (RTPS_LOG_BASE + 24)

- Failed to receive a packet, when forwarding to a higher upstream module.*

 - #define `RTPS_LOG_BAD_PARAMETER_EC` (RTPS_LOG_BASE + 25)

Bad parameter to an RTPS interface function.
- #define `RTPS_LOG_NOT_ENABLED_EC` (RTPS_LOG_BASE + 26)

Failed because interface is not enabled.
- #define `RTPS_LOG_READER_UNSUPPORTED_EC` (RTPS_LOG_BASE + 27)

RTPS reader does not support send.
- #define `RTPS_LOG_INTERFACE_MISMATCH_EC` (RTPS_LOG_BASE + 28)

Upstream interface does not match source or destination of packet being sent or received.
- #define `RTPS_LOG_FULL_SEND_WINDOW_EC` (RTPS_LOG_BASE + 29)

Failed to send due to reaching maximum send window size.
- #define `RTPS_LOG_NETIO_PACKET_SET_TAIL_EC` (RTPS_LOG_BASE + 31)

Failed to set tail of packet to send.
- #define `RTPS_LOG_FUNC_UNSUPPORTED_EC` (RTPS_LOG_BASE + 32)

RTPS interface does not support this function.
- #define `RTPS_LOG_EXCEEDED_LIMIT_TRANSPORTS_EC` (RTPS_LOG_BASE + 33)

Out of transport entries.
- #define `RTPS_LOG_EXCEEDED_LIMIT_PEERS_EC` (RTPS_LOG_BASE + 34)

Out of peer entries.
- #define `RTPS_LOG_ASSERT_PEER_EC` (RTPS_LOG_BASE + 35)

Failed to assert a remote writer or reader.
- #define `RTPS_LOG_ASSERT_TRANSPORT_EC` (RTPS_LOG_BASE + 36)

Failed to assert a downstream transport.
- #define `RTPS_LOG_FIND_EXTERNAL_INTERFACE_EC` (RTPS_LOG_BASE + 37)

Failed to lookup an external interface.
- #define `RTPS_LOG_NONEXISTENT_ROUTE_EC` (RTPS_LOG_BASE + 38)

Failed to delete a nonexistent route.
- #define `RTPS_LOG_NONEXISTENT_BIND_EC` (RTPS_LOG_BASE + 39)

Failed to remove a nonexistent bind entry.
- #define `RTPS_LOG_NONEXISTENT_EXTERNAL_BIND_EC` (RTPS_LOG_BASE + 40)

Failed to remove a nonexistent external bind entry.
- #define `RTPS_LOG_STALE_ACK_EPOCH_EC` (RTPS_LOG_BASE + 41)

Dropped an ACKNACK submessage with an old epoch.
- #define `RTPS_LOG_STALE_HB_EPOCH_EC` (RTPS_LOG_BASE + 42)

Dropped a HEARTBEAT submessage with an old epoch.
- #define `RTPS_LOG_ACK_EC` (RTPS_LOG_BASE + 43)

Failed an acknack() upstream.
- #define `RTPS_LOG_REQUEST_EC` (RTPS_LOG_BASE + 45)

Failed a request() upstream.
- #define `RTPS_LOG_RETURN_LOAN_EC` (RTPS_LOG_BASE + 46)

Failed a return_loan() upstream.
- #define `RTPS_LOG_NETIO_PACKET_SET_HEAD_EC` (RTPS_LOG_BASE + 47)

Failed to set the head of a packet.
- #define `RTPS_LOG_SEND_HEARTBEAT_EC` (RTPS_LOG_BASE + 48)

Failed to send a HEARTBEAT submessage.
- #define `RTPS_LOG_SHIFT_BITMAP_EC` (RTPS_LOG_BASE + 49)

Failed to shift a bitmap.
- #define `RTPS_LOG_FULLY_ACKED_READER_EC` (RTPS_LOG_BASE + 50)

A Reader is fully acknowledged and does not need to respond to a final HEARTBEAT.
- #define `RTPS_LOG_DATA_OUT_OF_RANGE_EC` (RTPS_LOG_BASE + 51)

Dropping a DATA submessage whose sequence number is outside of a Reader's receive window.

- #define [RTPS_LOG_DATA_ALREADY_RECEIVED_EC](#) (RTPS_LOG_BASE + 52)
A DATA submessage whose sequence number was previously received was discarded. This is not an error and may happen for example when a message is retransmitted.
- #define [RTPS_LOG_INTERFACE_NOT_ENABLED_EC](#) (RTPS_LOG_BASE + 53)
Failed to receive a message because interface is not enabled.
- #define [RTPS_LOG_INVALID_PACKET_EC](#) (RTPS_LOG_BASE + 54)
Dropped a message with an invalid packet header.
- #define [RTPS_LOG_UNSUPPORTED_INTERFACE_EC](#) (RTPS_LOG_BASE + 55)
Received a message on an unsupported interface.
- #define [RTPS_LOG_UNKNOWN_SUBMESSAGE_EC](#) (RTPS_LOG_BASE + 56)
Received a submessage with an unknown ID.
- #define [RTPS_LOG_PROCESS_ACKNACK_EC](#) (RTPS_LOG_BASE + 57)
Failed to process received ACKNACK submessage.
- #define [RTPS_LOG_PROCESS_DATA_EC](#) (RTPS_LOG_BASE + 58)
Failed to process received DATA submessage.
- #define [RTPS_LOG_PROCESS_GAP_EC](#) (RTPS_LOG_BASE + 59)
Failed to process received GAP submessage.
- #define [RTPS_LOG_PROCESS_HEARTBEAT_EC](#) (RTPS_LOG_BASE + 60)
Failed to process received HEARTBEAT submessage.
- #define [RTPS_LOG_CREATE_HB_EVENT_EC](#) (RTPS_LOG_BASE + 61)
Failed to create periodic HEARTBEAT event.
- #define [RTPS_LOG_CREATE_EVENT_TIMER_EC](#) (RTPS_LOG_BASE + 62)
Failed to create periodic HEARTBEAT event timer.
- #define [RTPS_LOG_SEQ_NUM_OUT_OF_RANGE_EC](#) (RTPS_LOG_BASE + 63)
Failed to set bitmap bit due to out-of-range sequence number.
- #define [RTPS_LOG_FIRST_SN_GREATER_LAST_SN_EC](#) (RTPS_LOG_BASE + 65)
Invalid range of sequence numbers to fill in bitmap.
- #define [RTPS_LOG_BITCOUNT_OUT_OF_BOUNDS_EC](#) (RTPS_LOG_BASE + 66)
Deserialized an invalid out-of-bounds bitcount for a bitmap.
- #define [RTPS_LOG_SHIFT_SN_OUT_OF_RANGE_EC](#) (RTPS_LOG_BASE + 67)
Failed to shift bitmap due to invalid starting sequence number.
- #define [RTPS_LOG_SERIALIZE_HOST_ID_EC](#) (RTPS_LOG_BASE + 68)
Failed to serialize host ID of GUID.
- #define [RTPS_LOG_SERIALIZE_APP_ID_EC](#) (RTPS_LOG_BASE + 69)
Failed to serialize app ID of GUID.
- #define [RTPS_LOG_SERIALIZE_INSTANCE_ID_EC](#) (RTPS_LOG_BASE + 70)
Failed to serialized instance ID of GUID.
- #define [RTPS_LOG_SERIALIZE_OBJECT_ID_EC](#) (RTPS_LOG_BASE + 71)
Failed to serialize object ID of GUID.
- #define [RTPS_LOG_DESERIALIZE_HOST_ID_EC](#) (RTPS_LOG_BASE + 72)
Failed to deserialize host ID of GUID.
- #define [RTPS_LOG_DESERIALIZE_APP_ID_EC](#) (RTPS_LOG_BASE + 73)
Failed to deserialize app ID of GUID.
- #define [RTPS_LOG_DESERIALIZE_INSTANCE_ID_EC](#) (RTPS_LOG_BASE + 74)
Failed to deserialize instance ID of GUID.
- #define [RTPS_LOG_DESERIALIZE_OBJECT_ID_EC](#) (RTPS_LOG_BASE + 75)
Failed to deserialize object ID of GUID.
- #define [RTPS_LOG_DB_CREATE_EXT_BIND_ENTRY_EC](#) (RTPS_LOG_BASE + 76)
Failed to create database entry for external bind table.
- #define [RTPS_LOG_BITMAP_SET_BIT_EC](#) (RTPS_LOG_BASE + 77)
Failed to set bit in bitmap.

- #define `RTPS_LOG_UNSUPPORTED_INFO_REPLY_EC` (`RTPS_LOG_BASE + 78`)
Received and dropped currently unsupported INFO_REPLY submessage.
- #define `RTPS_LOG_UNSUPPORTED_INFO_REPLY_IP4_EC` (`RTPS_LOG_BASE + 79`)
Received and dropped currently unsupported INFO_REPLY_IP4 submessage.
- #define `RTPS_LOG_UNSUPPORTED_INFO_SRC_EC` (`RTPS_LOG_BASE + 80`)
Received and dropped currently unsupported INFO_SRC submessage.
- #define `RTPS_LOG_DELETE_INDEX_EC` (`RTPS_LOG_BASE + 81`)
Failed to delete index of a database table.
- #define `RTPS_LOG_DELETE_TABLE_EC` (`RTPS_LOG_BASE + 82`)
Failed to delete a database table.
- #define `RTPS_LOG_DB_LOCK_EC` (`RTPS_LOG_BASE + 83`)
Failed to take database lock.
- #define `RTPS_LOG_DB_UNLOCK_EC` (`RTPS_LOG_BASE + 84`)
Failed to give database lock.
- #define `RTPS_LOG_CREATE_BIND_TABLE_EC` (`RTPS_LOG_BASE + 85`)
Failed to create a database table for storing RTPS bind info.
- #define `RTPS_LOG_STATUS_CHANGE_EC` (`RTPS_LOG_BASE + 86`)
Failed to update status for reliable reader activity changed.
- #define `RTPS_LOG_SET_GAP_EC` (`RTPS_LOG_BASE + 87`)
Failed to update status for reliable reader activity changed.
- #define `RTPS_LOG_WINDOW_POOL_ALLOC_EC` (`RTPS_LOG_BASE + 88`)
Out of memory to allocate send window pool.
- #define `RTPS_LOG_GET_WINDOW_BUFFER_EC` (`RTPS_LOG_BASE + 89`)
Failed to get buffer from send window pool.
- #define `RTPS_LOG_INSERT_WINDOW_FULL_EC` (`RTPS_LOG_BASE + 90`)
Cannot insert sample into full window.
- #define `RTPS_LOG_WINDOW_INSERT_EC` (`RTPS_LOG_BASE + 91`)
Failed to insert sample into send window.
- #define `RTPS_LOG_TIMER_DELETE_TIMEOUT_EC` (`RTPS_LOG_BASE + 92`)
Failed to delete a timeout event.
- #define `RTPS_LOG_BUFFERPOOL_DELETE_EC` (`RTPS_LOG_BASE + 93`)
Failed to delete a buffer pool.
- #define `RTPS_LOG_DIRECT_SEND_EC` (`RTPS_LOG_BASE + 94`)
Failed to send a message to a specific peer.
- #define `RTPS_LOG_PACKET_INITIALIZE_EC` (`RTPS_LOG_BASE + 95`)
RTPS writer failed to initialize a packet.
- #define `RTPS_LOG_WRITER_REQUEST_SENT_HISTORY_EC` (`RTPS_LOG_BASE + 96`)
RTPS reliable writer failed to request and resend history.
- #define `RTPS_LOG_BITMAP_SHIFT_EC` (`RTPS_LOG_BASE + 97`)
Failed to shift bitmap to new lead sequence number.
- #define `RTPS_LOG_WINDOW_ADVANCE_EC` (`RTPS_LOG_BASE + 98`)
Send window of a remote reader failed to advance.
- #define `RTPS_LOG_WINDOW_ADVANCE_UNACKED_EC` (`RTPS_LOG_BASE + 99`)
Failed when advancing window ahead of unacknowledged sequence number.
- #define `RTPS_LOG_LOOKUP_PEER_EC` (`RTPS_LOG_BASE + 100`)
Failed to find peer that should exist.
- #define `RTPS_LOG_READER_SEND_ACKNACK_EC` (`RTPS_LOG_BASE + 101`)
RTPS reader failed to send an ACKNACK message.
- #define `RTPS_LOG_FINALIZE_INTERFACE_EC` (`RTPS_LOG_BASE + 102`)
Failed to finalize an RTPS interface.
- #define `RTPS_LOG_INDEXER_REMOVE_ENTRY_EC` (`RTPS_LOG_BASE + 103`)

- Failed to remove index entry for external interface upon deleting RTPS interface.*

 - #define [RTPS_LOG_LOOKUP_EXT_BIND_ENTRY_EC](#) (RTPS_LOG_BASE + 104)
- Failed when looking up external bind entry from database.*

 - #define [RTPS_LOG_SEQ_INIT_EC](#) (RTPS_LOG_BASE + 105)
- Failed to initialize a local sequence.*

 - #define [RTPS_LOG_SEQ_MAX_EC](#) (RTPS_LOG_BASE + 106)
- Failed to set maximum of a local sequence.*

 - #define [RTPS_LOG_SEQ_LEN_EC](#) (RTPS_LOG_BASE + 107)
- Failed to set length of a local sequence.*

 - #define [RTPS_LOG_INTF_MODE_UNDEF_EC](#) (RTPS_LOG_BASE + 108)
- Initializing an RTPS interface with undefined mode.*

 - #define [RTPS_LOG_DELETE_BUFFER_POOL_EC](#) (RTPS_LOG_BASE + 109)
- Failed to delete buffer pool.*

 - #define [RTPS_LOG_CREATE_BUFFER_POOL_EC](#) (RTPS_LOG_BASE + 110)
- Failed to create buffer pool.*

 - #define [RTPS_LOG_CREATE_PEERS_INDEX_EC](#) (RTPS_LOG_BASE + 111)
- Failed to create index of peers.*

 - #define [RTPS_LOG_DELETE_INDEXER_EC](#) (RTPS_LOG_BASE + 112)
- Failed to delete indexer.*

 - #define [RTPS_LOG_GET_PEER_BUFFER_EC](#) (RTPS_LOG_BASE + 113)
- Failed to get buffer from peer buffer pool.*

 - #define [RTPS_LOG_DB_CREATE_DIRECT_INDEX_EC](#) (RTPS_LOG_BASE + 114)
- Failed to create database index for direct sends.*

 - #define [RTPS_LOG_DB_CREATE_GROUP_INDEX_EC](#) (RTPS_LOG_BASE + 115)
- Failed to create database index for group sends.*

 - #define [RTPS_LOG_NETIO_PACKET_INSUFFICIENT_BUFFER_EC](#) (RTPS_LOG_BASE + 116)
- Insufficient buffer in packet.*

 - #define [RTPS_LOG_PROCESS_DATA_BATCH_EC](#) (RTPS_LOG_BASE + 117)
- Failed to process received DATA BATCH submessage.*

 - #define [RTPS_LOG_PROCESS_HEARTBEAT_BATCH_EC](#) (RTPS_LOG_BASE + 118)
- Failed to process received HEARTBEAT_BATCH submessage.*

 - #define [RTPS_LOG_NEGATIVE_RESERVATION_COUNT_EC](#) (RTPS_LOG_BASE + 119)
- The reservation count is zero.*

 - #define [RTPS_LOG_DB_CREATE_SELECTED_GROUP_INDEX_EC](#) (RTPS_LOG_BASE + 148)
- Failed to create database index for the selected route group.*

 - #define [RTPS_LOG_CHECKSUM_MISSING_CHECKSUM_LENGTH_EC](#) (RTPS_LOG_BASE + 149)
- A header extension was received, but the checksum length is not present.*

 - #define [RTPS_LOG_CHECKSUM_UNSUPPORTED_EC](#) (RTPS_LOG_BASE + 150)
- Unsupported checksum bits received, message dropped.*

 - #define [RTPS_LOG_CHECKSUM_LENGTH_DESERIALIZE_ERROR_EC](#) (RTPS_LOG_BASE + 151)
- Checksum length could not be deserialized.*

 - #define [RTPS_LOG_CHECKSUM_INCONSISTENT_LENGTH_EC](#) (RTPS_LOG_BASE + 152)
- The payload length is less than the deserialized checksum length.*

 - #define [RTPS_LOG_CHECKSUM_CHECKSUM_DESERIALIZE_ERROR_EC](#) (RTPS_LOG_BASE + 153)
- Checksum length could not be deserialized.*

 - #define [RTPS_LOG_CHECKSUM_CHECKSUM_ERROR_EC](#) (RTPS_LOG_BASE + 154)
- The checksum is invalid.*

 - #define [RTPS_LOG_DATA_BATCH_ALREADY_RECEIVED_EC](#) (RTPS_LOG_BASE + 155)
- A DATA_BATCH submessage whose sequence number was previously received was discarded. This is not an error and may happen for example when a message is retransmitted.*

 - #define [RTPS_LOG_DATA_SN_OUT_OF_ORDER_EC](#) (RTPS_LOG_BASE + 156)

- Received a DATA submessage with an out-of-order sequence number.
- #define `RTPS_LOG_DATA_BATCH_SN_OUT_OF_ORDER_EC` (RTPS_LOG_BASE + 157)
- Received a DATA_BATCH submessage with an out-of-order sequence number.
- #define `RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM128_EC` (RTPS_LOG_BASE + 200)
- The property tries to override the BUILTIN128 checksum without setting `allow_builtin_override` to TRUE.
- #define `RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM32_EC` (RTPS_LOG_BASE + 201)
- The property tries to override the BUILTIN32 checksum without setting `allow_builtin_override` to TRUE.
- #define `RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM64_EC` (RTPS_LOG_BASE + 202)
- The property tries to override the BUILTIN64 checksum without setting `allow_builtin_override` to TRUE.
- #define `RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN128_EC` (RTPS_LOG_BASE + 203)
- The property override of the BUILTIN128 checksum is invalid. It is not legal to reuse any of the built-in functions.
- #define `RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN32_EC` (RTPS_LOG_BASE + 204)
- The property override of the BUILTIN32 checksum is invalid. It is not legal to reuse any of the built-in functions.
- #define `RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN64_EC` (RTPS_LOG_BASE + 205)
- The property override of the BUILTIN64 checksum is invalid. It is not legal to reuse any of the built-in functions.
- #define `RTPS_LOG_CHECKSUM_INVALID_CUSTOM256_EC` (RTPS_LOG_BASE + 206)
- The property specifies an invalid custom256 checksum function.
- #define `RTPS_LOG_CHECKSUM_INVALID_CUSTOM32_EC` (RTPS_LOG_BASE + 207)
- The property specifies an invalid custom32 checksum function.
- #define `RTPS_LOG_CHECKSUM_INVALID_CUSTOM64_EC` (RTPS_LOG_BASE + 208)
- The property specifies an invalid custom64 checksum function.
- #define `RTPS_LOG_CHECKSUM_INVALID_CUSTOM128_EC` (RTPS_LOG_BASE + 209)
- The property specifies an invalid custom128 checksum function.
- #define `RTPS_LOG_CHECKSUM_CHECKSUM_CALCULATION_FAILED_EC` (RTPS_LOG_BASE + 210)
- RTPS_Interface_calculate_checksum() failed to calculate the checksum.
- #define `RTPS_LOG_UNSUPPORTED_MANDATORY_HDR_EXT_PID_EC` (RTPS_LOG_BASE + 211)
- The HeaderExtension contained a mandatory PID that was not understood.
- #define `RTPS_LOG_INVALID_HDR_EXT_PID_LIST_EC` (RTPS_LOG_BASE + 212)
- An invalid HeaderExtension PID was detected.
- #define `RTPS_LOG_INVALID_HDR_EXT_PID_ERROR_EC` (RTPS_LOG_BASE + 213)
- Failed to decode the HeaderExtension PID list.
- #define `RTPS_LOG_INCONSISTENT_HDR_EXT_LENGTH_ERROR_EC` (RTPS_LOG_BASE + 214)
- The HeaderExtension length is inconsistent with the decoded encoded HeaderExtension length.
- #define `RTPS_LOG_MESSAGE_EXCEED_LENGTH_ERROR_EC` (RTPS_LOG_BASE + 215)
- A RTPS message which was longer than the indicated message length in the RTPS header was received. This causes the rest of the RTPS message to be invalidated and the remaining submessages to be ignored.
- #define `RTPS_LOG_CHECKSUM_INVALID_CRC32_FLAGS_EC` (RTPS_LOG_BASE + 216)
- The CRC32 submsg flags are invalid.

8.16.1 Detailed Description

RTPS module log codes. Log codes of the RTPS module

8.16.2 Macro Definition Documentation

8.16.2.1 #define RTPS_LOG_DB_REMOVE_ROUTE_ENTRY_EC (RTPS_LOG_BASE + 14)

Failed to remove entry from route table.

Failure reason given by database return code (dbrc)

8.17 rtps_rtps_impl.h File Reference

Implementation of RTPS interface functions and types.

8.17.1 Detailed Description

Implementation of RTPS interface functions and types. RTPS protocol defined types, implemented in C.

8.18 wh_sm_log.h File Reference

WH module log codes.

Macros

- #define `WHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC` (WHSM_LOG_BASE + 1)
KEEP_ALL History kind is not supported.
- #define `WHSM_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC` (WHSM_LOG_BASE + 2)
Unlimited length resource limits unsupported.
- #define `WHSM_LOG_MAX_SAMPLES_TOO_SMALL_EC` (WHSM_LOG_BASE + 3)
DataWriterQos.resource_limits.max_samples set too small.
- #define `WHSM_LOG_HISTORY_OBJECT` 1
A history object.
- #define `WHSM_LOG_KEY_OBJECT` 2
A key object.
- #define `WHSM_LOG_SAMPLE_OBJECT` 2
A sample object.
- #define `WHSM_LOG_KEYPOOL_OBJECT` 3
A key-pool object.
- #define `WHSM_LOG_SAMPLEPOOL_OBJECT` 4
A sample-pool object.
- #define `WHSM_LOG_KEYINDEX_OBJECT` 5
A key index pool.
- #define `WHSM_LOG_HISTORYINDEX_OBJECT` 6
A key index pool.
- #define `WHSM_LOG_SAMPLEINDEX_OBJECT` 5
A key index pool.
- #define `WHSM_LOG_OBJECT_ALLOCATE_EC` (WHSM_LOG_BASE + 4)
Failed to allocate object of the specified kind.
- #define `WHSM_LOG_OBJECT_DELETE_EC` (WHSM_LOG_BASE + 5)
Failed to delete object of the specified kind.
- #define `WHSM_LOG_OBJECT_INDEX_EC` (WHSM_LOG_BASE + 6)
Failed to index an object of the specified kind.
- #define `WHSM_LOG_NO_PROPERTY_EC` (WHSM_LOG_BASE + 7)
No property when creating a writer history instance.
- #define `WHSM_LOG_INVALID_INDEX_OBJECT_EC` (WHSM_LOG_BASE + 8)
The sample removed from an index did not match sample's key.

8.18.1 Detailed Description

WH module log codes.

8.18.2 Macro Definition Documentation

8.18.2.1 `#define WHSM_LOG_HISTORY_OBJECT 1`

A history object.

8.18.2.2 `#define WHSM_LOG_KEY_OBJECT 2`

A key object.

8.18.2.3 `#define WHSM_LOG_SAMPLE_OBJECT 2`

A sample object.

8.18.2.4 `#define WHSM_LOG_KEYPOOL_OBJECT 3`

A key-pool object.

8.18.2.5 `#define WHSM_LOG_SAMPLEPOOL_OBJECT 4`

A sample-pool object.

8.18.2.6 `#define WHSM_LOG_KEYINDEX_OBJECT 5`

A key index pool.

8.18.2.7 `#define WHSM_LOG_HISTORYINDEX_OBJECT 6`

A key index pool.

8.18.2.8 `#define WHSM_LOG_SAMPLEINDEX_OBJECT 5`

A key index pool.

Index

- ~FooSeq
 - FooSeq, [421](#)
- _contiguous_buffer
 - FooSeq, [429](#)
- _element_size
 - FooSeq, [429](#)
- _flags
 - FooSeq, [429](#)
- _length
 - FooSeq, [429](#)
- _maximum
 - FooSeq, [429](#)
- _token1
 - FooSeq, [429](#)
- _token2
 - FooSeq, [429](#)
- accept_unknown_peers
 - DDS_DiscoveryQosPolicy, [251](#)
- access_scope
 - DDS_PresentationQosPolicy, [276](#)
- active_count
 - DDS_ReliableReaderActivityChangedStatus, [283](#)
- active_count_change
 - DDS_ReliableReaderActivityChangedStatus, [284](#)
- add_entry
 - UDP Transport API, [59](#)
- add_peer
 - DDSDomainParticipant, [353](#)
- address
 - DDS_Locator_t, [266](#)
 - UDP_InterfaceTableEntry, [448](#)
- alive_count
 - DDS_LivelinessChangedStatus, [262](#)
- alive_count_change
 - DDS_LivelinessChangedStatus, [262](#)
- allow_builtin_override
 - RTPS Transport API, [63](#)
- allow_interface
 - UDP_InterfaceFactoryProperty, [446](#)
- allowed_crc_mask
 - DDS_ChecksumProperty, [238](#)
 - DDS_ChecksumProperty_t, [238](#)
 - DDS_WireProtocolQosPolicy, [309](#)
- as_entity
 - DDSDataReader, [312](#)
 - DDSDomainParticipant, [342](#)
 - DDSTopic, [386](#)
- as_topicdescription
 - DDSTopic, [386](#)
- assert_liveliness
 - DDSDataWriter, [328](#)
- attach_condition
 - DDSWaitSet, [394](#)
- autoenable_created_entities
 - DDS_EntityFactoryQosPolicy, [259](#)
- builtin_checksum128_class
 - RTPS Transport API, [63](#)
- builtin_checksum32_class
 - RTPS Transport API, [63](#)
- builtin_checksum64_class
 - RTPS Transport API, [63](#)
- builtin_endpoint_reader_nack_period
 - DPDE_DiscoveryPluginProperty, [398](#)
- builtin_multicast_port_offset
 - DDS_RtpsWellKnownPorts_t, [293](#)
- builtin_unicast_port_offset
 - DDS_RtpsWellKnownPorts_t, [294](#)
- builtin_writer_heartbeat_period
 - DPDE_DiscoveryPluginProperty, [398](#)
- builtin_writer_heartbeats_per_max_samples
 - DPDE_DiscoveryPluginProperty, [398](#)
- builtin_writer_max_heartbeat_retries
 - DPDE_DiscoveryPluginProperty, [398](#)
- CDR, [180](#)
 - CDR_LOG_INCR_OFFSET_EC, [180](#)
 - CDR_LOG_INVALID_ENUM_EC, [180](#)
 - CDR_LOG_SET_OFFSET_EC, [180](#)
 - CDR_LOG_STREAM_ALLOC_EC, [180](#)
 - CDR_LOG_UNSUPPORTED_OPTIONS_EC, [180](#)
- CDR_LOG_INCR_OFFSET_EC
 - CDR, [180](#)
- CDR_LOG_INVALID_ENUM_EC
 - CDR, [180](#)
- CDR_LOG_SET_OFFSET_EC
 - CDR, [180](#)
- CDR_LOG_STREAM_ALLOC_EC
 - CDR, [180](#)
- CDR_LOG_UNSUPPORTED_OPTIONS_EC
 - CDR, [180](#)
- cache_serialized_samples
 - DPDE_DiscoveryPluginProperty, [396](#)
- cdr_log.h, [450](#)
- check_crc
 - DDS_WireProtocolQosPolicy, [308](#)
- checksum
 - DDS_ParticipantBuiltinTopicData, [273](#)
 - RTPS Transport API, [63](#)
- checksum128
 - RTPS Transport API, [63](#)
- checksum32
 - RTPS Transport API, [63](#)
- checksum64
 - RTPS Transport API, [63](#)
- checksum_calculate
 - RTPS Transport API, [63](#)
- checksum_tx_mode
 - RTPS Transport API, [63](#)
- class_id

- RTPS Transport API, [63](#)
- coherent_access
 - DDS_PresentationQosPolicy, [276](#)
- compute_crc
 - DDS_WireProtocolQosPolicy, [308](#)
- computed_crc_kind
 - DDS_ChecksumProperty, [238](#)
 - DDS_ChecksumProperty_t, [238](#)
 - DDS_WireProtocolQosPolicy, [308](#)
- Conditions and WaitSets, [116](#)
- Configuring Resource Limits, [231](#)
- context
 - RTPS Transport API, [63](#)
- copy_data
 - FooTypeSupport, [432](#)
- copy_no_alloc
 - FooSeq, [423](#)
- create_data
 - FooTypeSupport, [431](#)
- create_datareader
 - DDSSubscriber, [380](#)
- create_datawriter
 - DDSPublisher, [372](#)
- create_participant
 - DDSDomainParticipantFactory, [356](#)
- create_publisher
 - DDSDomainParticipant, [342](#)
- create_subscriber
 - DDSDomainParticipant, [343](#)
- create_topic
 - DDSDomainParticipant, [344](#)
- current_count
 - DDS_PublicationMatchedStatus, [281](#)
 - DDS_SubscriptionMatchedStatus, [302](#)
- current_count_change
 - DDS_PublicationMatchedStatus, [281](#)
 - DDS_SubscriptionMatchedStatus, [302](#)
- DATA_READER_PROTOCOL, [112](#)
 - DDS_DATAREADERPROTOCOL_QOS_POLICY_NAME, [112](#)
- DATA_WRITER_PROTOCOL, [113](#)
 - DDS_DATAWRITERPROTOCOL_QOS_POLICY_NAME, [113](#)
- DB, [176](#)
 - DB_LOG_ALLOC_CURSOR_POOL_EC, [178](#)
 - DB_LOG_ALLOC_DATABASE_EC, [178](#)
 - DB_LOG_ALLOC_INDEX_POOL_EC, [178](#)
 - DB_LOG_ALLOC_RECORD_POOL_EC, [178](#)
 - DB_LOG_ALLOC_TABLE_POOL_EC, [177](#)
 - DB_LOG_CURSOR_INVALIDATED_EC, [178](#)
 - DB_LOG_CURSORS_INUSE_EC, [178](#)
 - DB_LOG_ILLEGAL_LOCK_MODE_EC, [177](#)
 - DB_LOG_ILLEGAL_RECORD_COUNT_EC, [177](#)
 - DB_LOG_ILLEGAL_TABLE_SIZE_EC, [177](#)
 - DB_LOG_INDEX_INUSE_EC, [178](#)
 - DB_LOG_LOCK_FAILURE_EC, [178](#)
 - DB_LOG_MUTEX_ALLOC_EC, [177](#)
 - DB_LOG_NAME_TOO_LONG_EC, [177](#)
 - DB_LOG_OUT_OF_CURSORS_EC, [178](#)
 - DB_LOG_OUT_OF_INDICES_EC, [178](#)
 - DB_LOG_OUT_OF_RECORDS_EC, [177](#)
 - DB_LOG_OUT_OF_TABLES_EC, [177](#)
 - DB_LOG_RECORD_ALREADY_EXISTS_EC, [178](#)
 - DB_LOG_RECORD_DOES_NOT_EXIST_EC, [178](#)
 - DB_LOG_RECORDS_INUSE_EC, [178](#)
 - DB_LOG_SORTED_ALLOC_EC, [177](#)
- DB_LOG_OUT_OF_CURSORS_EC, [178](#)
- DB_LOG_OUT_OF_INDICES_EC, [178](#)
- DB_LOG_OUT_OF_RECORDS_EC, [177](#)
- DB_LOG_OUT_OF_TABLES_EC, [177](#)
- DB_LOG_RECORD_ALREADY_EXISTS_EC, [178](#)
- DB_LOG_RECORD_DOES_NOT_EXIST_EC, [178](#)
- DB_LOG_RECORDS_INUSE_EC, [178](#)
- DB_LOG_SORTED_ALLOC_EC, [177](#)

- DB_LOG_TABLE_EXISTS_EC
 - DB, [177](#)
- DB_LOG_TABLE_NAME_TOO_LONG_EC
 - DB, [177](#)
- DB_LOG_TABLE_NOT_INUSE_EC
 - DB, [178](#)
- DB_LOG_TABLES_INUSE_EC
 - DB, [177](#)
- DB_LOG_UNLOCK_FAILURE_EC
 - DB, [178](#)
- DDS C++ API, [50](#)
- DDS-Specific Primitive Types, [70](#)
 - DDS_BOOLEAN_FALSE, [71](#)
 - DDS_BOOLEAN_TRUE, [71](#)
 - DDS_Boolean, [72](#)
 - DDS_Char, [71](#)
 - DDS_Double, [72](#)
 - DDS_Enum, [72](#)
 - DDS_Float, [72](#)
 - DDS_Long, [71](#)
 - DDS_LongDouble, [72](#)
 - DDS_LongLong, [71](#)
 - DDS_Octet, [71](#)
 - DDS_Short, [71](#)
 - DDS_String, [72](#)
 - DDS_UnsignedLong, [71](#)
 - DDS_UnsignedLongLong, [71](#)
 - DDS_UnsignedShort, [71](#)
 - DDS_Wchar, [71](#)
 - DDS_Wstring, [72](#)
- DDS_ALIVE_INSTANCE_STATE
 - Instance States, [46](#)
- DDS_ANY_INSTANCE_STATE
 - Instance States, [46](#)
- DDS_ANY_SAMPLE_STATE
 - Sample States, [42](#)
- DDS_ANY_VIEW_STATE
 - View States, [44](#)
- DDS_AUTO_NAME_ENTITY
 - ENTITY_NAME, [118](#)
- DDS_AUTOMATIC_LIVELINESS_QOS
 - LIVELINESS, [95](#)
- DDS_BEST_EFFORT_RELIABILITY_QOS
 - RELIABILITY, [97](#)
- DDS_BOOLEAN_FALSE
 - DDS-Specific Primitive Types, [71](#)
- DDS_BOOLEAN_TRUE
 - DDS-Specific Primitive Types, [71](#)
- DDS_BY_RECEPTION_TIMESTAMP_DESTINATIONORDER_QOS
 - DESTINATION_ORDER, [102](#)
- DDS_BY_SOURCE_TIMESTAMP_DESTINATIONORDER_QOS
 - DESTINATION_ORDER, [102](#)
- DDS_Boolean
 - DDS-Specific Primitive Types, [72](#)
- DDS_BuiltinTopicKey_t, [237](#)
 - Discovery, [24](#)
 - value, [237](#)
- DDS_C, [119](#)
 - DDSC_LOG_ADDRESSRESOLVER_OBJECT, [137](#)
 - DDSC_LOG_BIND_RECORD, [135](#)
 - DDSC_LOG_BINDRESOLVER_OBJECT, [137](#)
 - DDSC_LOG_CDR_BUFFER_SET_EC, [146](#)
 - DDSC_LOG_CDR_DESERIALIZE_DATA_EC, [147](#)
 - DDSC_LOG_CDR_DESERIALIZE_HEADER_EC, [147](#)
 - DDSC_LOG_CDR_DESERIALIZE_KEY_EC, [147](#)
 - DDSC_LOG_CDR_DESERIALIZE_KEYHASH_EC, [147](#)
 - DDSC_LOG_CDR_DESERIALIZE_PID_EC, [147](#)
 - DDSC_LOG_CDR_DESERIALIZE_PID_LENGTH_EC, [147](#)
 - DDSC_LOG_CDR_FINALIZE_SAMPLE_EC, [147](#)
 - DDSC_LOG_CDR_INCREMENT_POS_EC, [147](#)
 - DDSC_LOG_CDR_INITIALIZE_SAMPLE_EC, [147](#)
 - DDSC_LOG_CDR_POOL_ALLOC_EC, [146](#)
 - DDSC_LOG_CDR_POOL_DELETE_EC, [146](#)
 - DDSC_LOG_CDR_SERIALIZE_DATA_EC, [146](#)
 - DDSC_LOG_CDR_SERIALIZE_KEYHASH_EC, [146](#)
 - DDSC_LOG_CDR_SERIALIZE_PID_EC, [146](#)
 - DDSC_LOG_CDR_SERIALIZE_PID_LENGTH_EC, [146](#)
 - DDSC_LOG_CDR_SERIALIZE_STATUS_INFO_EC, [147](#)
 - DDSC_LOG_CDR_SET_POS_EC, [147](#)
 - DDSC_LOG_CHECKSUM_INCOMPATIBLE_EC, [152](#)
 - DDSC_LOG_CHECKSUM_INCONSISTENT_ALL-OW_EC, [152](#)
 - DDSC_LOG_CHECKSUM_INCONSISTENT_CLASS_EC, [152](#)
 - DDSC_LOG_CHECKSUM_INCONSISTENT_COMPUTE_EC, [152](#)
 - DDSC_LOG_CHECKSUM_REQUIRED_EC, [152](#)
 - DDSC_LOG_COMPONENT_CREATE_EC, [141](#)
 - DDSC_LOG_COMPONENT_DELETE_EC, [141](#)
 - DDSC_LOG_COMPONENT_LOOKUP_EC, [141](#)
 - DDSC_LOG_CONDITION_OBJECT, [137](#)
 - DDSC_LOG_CONDREF_OBJECT, [137](#)
 - DDSC_LOG_CREATE_INDEX_EC, [134](#)
 - DDSC_LOG_DATABASE_CREATE_EC, [133](#)
 - DDSC_LOG_DATABASE_DELETE_EC, [133](#)
 - DDSC_LOG_DATAREADER_CDR, [146](#)
 - DDSC_LOG_DATAREADER_ENTITY, [145](#)
 - DDSC_LOG_DATAREADER_LISTENER, [144](#)
 - DDSC_LOG_DATAREADER_NETIO_KIND, [148](#)
 - DDSC_LOG_DATAREADER_QOS, [141](#)
 - DDSC_LOG_DATAREADER_RECORD, [134](#)
 - DDSC_LOG_DATAREADER_RESOURCE_QOS_POLICY, [143](#)
 - DDSC_LOG_DATAREADERIO_COMPONENT, [140](#)

- DDSC_LOG_DATAREADERIO_OBJECT, [137](#)
- DDSC_LOG_DATAREADERQOS_OBJECT, [136](#)
- DDSC_LOG_DATAWRITER_CDR, [146](#)
- DDSC_LOG_DATAWRITER_ENTITY, [145](#)
- DDSC_LOG_DATAWRITER_LISTENER, [144](#)
- DDSC_LOG_DATAWRITER_NETIO_KIND, [148](#)
- DDSC_LOG_DATAWRITER_QOS, [141](#)
- DDSC_LOG_DATAWRITER_RECORD, [134](#)
- DDSC_LOG_DATAWRITER_RESOURCE_QOS-
_POLICY, [143](#)
- DDSC_LOG_DATAWRITERIO_COMPONENT,
[140](#)
- DDSC_LOG_DATAWRITERIO_OBJECT, [137](#)
- DDSC_LOG_DATAWRITERQOS_OBJECT, [136](#)
- DDSC_LOG_DB_CURSOR_INVALIDATED_EC,
[134](#)
- DDSC_LOG_DEADLINE_QOS_POLICY, [142](#)
- DDSC_LOG_DEFAULTDATAREADER_QOS, [142](#)
- DDSC_LOG_DEFAULTDATAWRITER_QOS, [142](#)
- DDSC_LOG_DEFAULTPARTICIPANT_QOS, [142](#)
- DDSC_LOG_DEFAULTPUBLISHER_QOS, [142](#)
- DDSC_LOG_DEFAULTSUBSCRIBER_QOS, [142](#)
- DDSC_LOG_DEFAULTTOPIC_QOS, [142](#)
- DDSC_LOG_DELETE_INDEX_EC, [134](#)
- DDSC_LOG_DESERIALIZE_BAD_PID_LENGTH-
_EC, [146](#)
- DDSC_LOG_DESTINATION_ORDER_POLICY,
[143](#)
- DDSC_LOG_DESTINATION_SEQUENCE, [139](#)
- DDSC_LOG_DISC_ADD_PEER_EC, [152](#)
- DDSC_LOG_DISC_AFTER_LOCAL_DATAREAD-
ER_DELETED_EC, [152](#)
- DDSC_LOG_DISC_AFTER_LOCAL_DATAREAD-
ER_ENABLED_EC, [152](#)
- DDSC_LOG_DISC_AFTER_LOCAL_DATAWRIT-
ER_DELETED_EC, [152](#)
- DDSC_LOG_DISC_AFTER_LOCAL_DATAWRIT-
ER_ENABLED_EC, [152](#)
- DDSC_LOG_DISC_AFTER_LOCAL_PARTICIPA-
NT_CREATED_EC, [151](#)
- DDSC_LOG_DISC_BEFORE_LOCAL_PARTICIP-
ANT_CREATED_EC, [151](#)
- DDSC_LOG_DISC_LOCAL_PARTICIPANT_ENA-
BLED_EC, [151](#)
- DDSC_LOG_DISC_REMOTE_PARTICIPANT_E-
XPIRED_EC, [152](#)
- DDSC_LOG_DISCOVERY_COMPONENT, [140](#)
- DDSC_LOG_DISCOVERY_TRAFFIC_QOS_POL-
ICY, [143](#)
- DDSC_LOG_DR_COMMIT_ENTRY_EC, [150](#)
- DDSC_LOG_DR_COMMIT_SAMPLE_EC, [150](#)
- DDSC_LOG_DR_COPY_DATA_SAMPLE_EC,
[150](#)
- DDSC_LOG_DR_CREATE_TYPED_READER_E-
C, [149](#)
- DDSC_LOG_DR_DESERIALIZE_KEYHASH_EC,
[150](#)
- DDSC_LOG_DR_DISPOSE_KEY_EC, [150](#)
- DDSC_LOG_DR_FILTER_ERROR_EC, [150](#)
- DDSC_LOG_DR_GET_ENTRY_FAILED_EC, [150](#)
- DDSC_LOG_DR_INSTANCE_TO_KEYHASH_E-
C, [150](#)
- DDSC_LOG_DR_READ_TAKE_FAILURE_EC,
[150](#)
- DDSC_LOG_DR_UNREGISTER_KEY_EC, [150](#)
- DDSC_LOG_DUPLICATE_GUID_PREFIX_EC,
[151](#)
- DDSC_LOG_DURABILITY_QOS_POLICY, [143](#)
- DDSC_LOG_DW_ACKNACK_FAILED_EC, [149](#)
- DDSC_LOG_DW_COMMIT_EC, [149](#)
- DDSC_LOG_DW_CREATE_TYPED_WRITER_E-
C, [149](#)
- DDSC_LOG_DW_HISTORY_REGISTER_KEY_E-
C, [149](#)
- DDSC_LOG_DW_ILLEGAL_KEY_KIND_EC, [149](#)
- DDSC_LOG_DW_KEYHASH_CREATE_EC, [149](#)
- DDSC_LOG_ENABLED_DISCOVERY_TRANSP-
ORT_SEQUENCE, [139](#)
- DDSC_LOG_ENABLED_TRANSPORT_SEQUE-
NCE, [139](#)
- DDSC_LOG_ENABLED_USER_TRANSPORT_S-
EQUENCE, [139](#)
- DDSC_LOG_ENDPOINT_NOT_CHILD_OF_PAR-
TICIPANT_EC, [132](#)
- DDSC_LOG_ENTITY_DIFFERENT_FACTORY_-
EC, [146](#)
- DDSC_LOG_ENTITY_ENABLE_EC, [145](#)
- DDSC_LOG_ENTITY_FINALIZE_EC, [146](#)
- DDSC_LOG_ENTITY_INITIALIZE_EC, [146](#)
- DDSC_LOG_ENTITY_NOT_EMPTY_EC, [146](#)
- DDSC_LOG_ENTITY_NOT_ENABLED_EC, [146](#)
- DDSC_LOG_ENTITY_OBJECT, [136](#)
- DDSC_LOG_FAILED_UPDATE_STATUS_COND-
ITION_EC, [133](#)
- DDSC_LOG_FIND_PUBLICATION_PARENT_EC,
[132](#)
- DDSC_LOG_FIND_SUBSCRIPTION_PARENT_-
EC, [132](#)
- DDSC_LOG_GET_NEXT_OBJECT_ID_EC, [131](#)
- DDSC_LOG_HEARTBEATS_QOS_POLICY, [143](#)
- DDSC_LOG_HISTORY_QOS_POLICY, [142](#)
- DDSC_LOG_HISTORY_RESOURCE, [141](#)
- DDSC_LOG_ILLEGAL_OBJECTID_EC, [133](#)
- DDSC_LOG_INITIAL_PEER_SEQUENCE, [139](#)
- DDSC_LOG_INTRA_NETIO_KIND, [148](#)
- DDSC_LOG_INVALID_DOMAINID_EC, [133](#)
- DDSC_LOG_INVALID_DURATION_EC, [131](#)
- DDSC_LOG_INVALID_ENDPOINT_GUID_EC,
[132](#)
- DDSC_LOG_INVALID_PARTICIPANT_GUID_PR-
EFIX_EC, [131](#)
- DDSC_LOG_INVALID_PARTICIPANT_NAME_E-
C, [131](#)
- DDSC_LOG_IO_SNPRINTF_FAILED_EC, [131](#)
- DDSC_LOG_LISTENER_GET_EC, [145](#)
- DDSC_LOG_LISTENER_INCONSISTENT_EC,

- 145
- DDSC_LOG_LISTENER_SET_EC, 145
- DDSC_LOG_LISTENER_SET_ILLEGAL_NULL_EC, 145
- DDSC_LOG_LIVELINESS_QOS_POLICY, 142
- DDSC_LOG_LOG_OBJECT, 137
- DDSC_LOG_LOOKUP_TYPE_PLUGIN_EC, 151
- DDSC_LOG_MAX_PARTICIPANT_ID_REACHED_EC, 132
- DDSC_LOG_MD5STREAM_OBJECT, 137
- DDSC_LOG_METAMULTICAST_SEQUENCE, 139
- DDSC_LOG_METAUNICAST_SEQUENCE, 139
- DDSC_LOG_MUTEX_OBJECT, 137
- DDSC_LOG_NETIO_ADD_ANON_TOPIC_ROUTE_EC, 147
- DDSC_LOG_NETIO_ADD_ROUTE_EC, 148
- DDSC_LOG_NETIO_ADD_TOPIC_ROUTE_EC, 147
- DDSC_LOG_NETIO_BIND_EC, 148
- DDSC_LOG_NETIO_BIND_EXTERNAL_EC, 148
- DDSC_LOG_NETIO_COMPONENT, 140
- DDSC_LOG_NETIO_DELETE_ROUTE_EC, 148
- DDSC_LOG_NETIO_DELETE_TOPIC_ROUTE_EC, 147
- DDSC_LOG_NETIO_FORCED_REMOVE_EC, 149
- DDSC_LOG_NETIO_FORWARD_TOPIC_EC, 147
- DDSC_LOG_NETIO_GET_EXTERNAL_INTF_EC, 148
- DDSC_LOG_NETIO_GET_ROUTE_TABLE_FAILED_EC, 148
- DDSC_LOG_NETIO_NETIO_KIND, 148
- DDSC_LOG_NETIO_NO_ROUTE_EC, 148
- DDSC_LOG_NETIO_PEER_LOOKUP_EC, 149
- DDSC_LOG_NETIO_ROUTE_LOOKUP_FAILED_EC, 148
- DDSC_LOG_NETIO_SEND_FAILED_EC, 149
- DDSC_LOG_NETIO_SET_STATE_EC, 149
- DDSC_LOG_NETIO_UNBIND_EC, 148
- DDSC_LOG_NETIO_UNBIND_EXTERNAL_EC, 148
- DDSC_LOG_NETMASK_SEQUENCE, 139
- DDSC_LOG_OBJECT_ALLOCATE_EC, 138
- DDSC_LOG_OBJECT_COPY_EC, 138
- DDSC_LOG_OBJECT_DELETE_EC, 138
- DDSC_LOG_OBJECT_EMPTY_EC, 139
- DDSC_LOG_OBJECT_FINALIZE_EC, 138
- DDSC_LOG_OBJECT_GET_PROPERTY_EC, 138
- DDSC_LOG_OBJECT_INITIALIZE_EC, 138
- DDSC_LOG_OBJECT_INUSECOUNT_EC, 139
- DDSC_LOG_OBJECT_REFCOUNT_EC, 138
- DDSC_LOG_OBJECT_SET_PROPERTY_EC, 139
- DDSC_LOG_OWNERSHIP_QOS_POLICY, 143
- DDSC_LOG_OWNERSHIP_STRENGTH_QOS_POLICY, 143
- DDSC_LOG_PACKET_INIT_EC, 149
- DDSC_LOG_PACKET_SET_HEAD_EC, 149
- DDSC_LOG_PACKET_SET_TAIL_EC, 149
- DDSC_LOG_PARTICIPANT_DOES_NOT_EXIST_EC, 132
- DDSC_LOG_PARTICIPANT_ENTITY, 145
- DDSC_LOG_PARTICIPANT_ID_QOS_POLICY, 143
- DDSC_LOG_PARTICIPANT_LISTENER, 145
- DDSC_LOG_PARTICIPANT_LOOKUP_EC, 132
- DDSC_LOG_PARTICIPANT_POOL_OBJECT, 138
- DDSC_LOG_PARTICIPANT_QOS, 142
- DDSC_LOG_PARTICIPANT_RECORD, 134
- DDSC_LOG_PARTICIPANT_RESOURCES, 141
- DDSC_LOG_PARTICIPANTDATA_OBJECT, 136
- DDSC_LOG_PARTICIPANTFACTORY_LISTENER, 145
- DDSC_LOG_PARTICIPANTFACTORY_QOS, 142
- DDSC_LOG_PARTICIPANTFACTORYQOS_OBJECT, 137
- DDSC_LOG_PARTICIPANTQOS_OBJECT, 136
- DDSC_LOG_PROTOCOL_QOS_POLICY, 142
- DDSC_LOG_PUBLICATION_ENTITY, 145
- DDSC_LOG_PUBLICATION_RECORD, 134
- DDSC_LOG_PUBLICATIONDATA_OBJECT, 136
- DDSC_LOG_PUBLISHER_ENTITY, 145
- DDSC_LOG_PUBLISHER_LISTENER, 144
- DDSC_LOG_PUBLISHER_QOS, 142
- DDSC_LOG_PUBLISHER_RECORD, 134
- DDSC_LOG_PUBLISHERQOS_OBJECT, 136
- DDSC_LOG_QOS_CHANGED_EC, 144
- DDSC_LOG_QOS_COPY_EC, 144
- DDSC_LOG_QOS_FINALIZE_EC, 144
- DDSC_LOG_QOS_GET_EC, 144
- DDSC_LOG_QOS_IMMUTABLE_EC, 144
- DDSC_LOG_QOS_INCONSISTENT_EC, 144
- DDSC_LOG_QOS_INCONSISTENT_POLICIES_EC, 144
- DDSC_LOG_QOS_INCONSISTENT_POLICY_EC, 143
- DDSC_LOG_QOS_INITIALIZE_EC, 144
- DDSC_LOG_QOS_SET_EC, 144
- DDSC_LOG_QOS_SET_ON_ENABLED_EC, 144
- DDSC_LOG_READERHISTORY_COMPONENT, 140
- DDSC_LOG_READTAKE_SEQUENCE, 140
- DDSC_LOG_RECORD_CREATE_EC, 135
- DDSC_LOG_RECORD_DELETE_EC, 135
- DDSC_LOG_RECORD_ERROR_EC, 135
- DDSC_LOG_RECORD_EXISTS_EC, 135
- DDSC_LOG_RECORD_FINALIZE_EC, 136
- DDSC_LOG_RECORD_INITIALIZE_EC, 136
- DDSC_LOG_RECORD_INSERT_EC, 135
- DDSC_LOG_RECORD_LOOKUP_EC, 135
- DDSC_LOG_RECORD_NOT_EXISTS_EC, 136
- DDSC_LOG_RECORD_REMOVE_EC, 136

- DDSC_LOG_RECORD_SELECT_EC, [136](#)
- DDSC_LOG_REFRESH_REM_PARTICIPANT_EC, [132](#)
- DDSC_LOG_REFRESH_REM_PARTICIPANT_TIMEOUT_EC, [132](#)
- DDSC_LOG_RELEASE_META_MC_EC, [133](#)
- DDSC_LOG_RELEASE_META_UC_EC, [133](#)
- DDSC_LOG_RELEASE_USER_MC_EC, [133](#)
- DDSC_LOG_RELEASE_USER_UC_EC, [133](#)
- DDSC_LOG_RELIABILITY_QOS_POLICY, [143](#)
- DDSC_LOG_REMOTE_PARTICIPANT_KEY_NOT_EQUAL_EC, [132](#)
- DDSC_LOG_REMOVE_PUBLICATION_EC, [132](#)
- DDSC_LOG_REMOVE_SUBSCRIPTION_EC, [132](#)
- DDSC_LOG_RESERVE_LOCATORS_EC, [133](#)
- DDSC_LOG_RESERVED_SEQUENCE, [139](#)
- DDSC_LOG_RESOURCE_EXCEEDED_EC, [141](#)
- DDSC_LOG_RESOURCE_LIMIT_QOS_POLICY, [142](#)
- DDSC_LOG_ROUTE_RECORD, [135](#)
- DDSC_LOG_ROUTE_SEQUENCE, [139](#)
- DDSC_LOG_ROUTERESOLVER_OBJECT, [137](#)
- DDSC_LOG_RT_OBJECT, [137](#)
- DDSC_LOG_RTPS_COMPONENT, [140](#)
- DDSC_LOG_RTPS_NETIO_KIND, [148](#)
- DDSC_LOG_SAMPLE_RESOURCES, [141](#)
- DDSC_LOG_SEQUENCE_COPY_EC, [140](#)
- DDSC_LOG_SEQUENCE_FINALIZE_EC, [140](#)
- DDSC_LOG_SEQUENCE_GETREF_EC, [140](#)
- DDSC_LOG_SEQUENCE_INITIALIZE_EC, [140](#)
- DDSC_LOG_SEQUENCE_INVALID_EC, [140](#)
- DDSC_LOG_SEQUENCE_SETLENGTH_EC, [140](#)
- DDSC_LOG_SEQUENCE_SETMAX_EC, [140](#)
- DDSC_LOG_SET_ENTITY_NAME_EC, [131](#)
- DDSC_LOG_STRING_OBJECT, [138](#)
- DDSC_LOG_STRING_RECORD, [135](#)
- DDSC_LOG_STRINGMANAGER_OBJECT, [138](#)
- DDSC_LOG_SUBSCRIBER_ENTITY, [145](#)
- DDSC_LOG_SUBSCRIBER_LISTENER, [144](#)
- DDSC_LOG_SUBSCRIBER_QOS, [142](#)
- DDSC_LOG_SUBSCRIBER_RECORD, [134](#)
- DDSC_LOG_SUBSCRIBERQOS_OBJECT, [136](#)
- DDSC_LOG_SUBSCRIPTION_ENTITY, [145](#)
- DDSC_LOG_SUBSCRIPTION_RECORD, [134](#)
- DDSC_LOG_SUBSCRIPTIONDATA_OBJECT, [136](#)
- DDSC_LOG_SYS_GET_HOSTNAME_EC, [131](#)
- DDSC_LOG_SYS_GETTIME_EC, [131](#)
- DDSC_LOG_SYSTEM_OBJECT, [137](#)
- DDSC_LOG_TABLE_CREATE_EC, [134](#)
- DDSC_LOG_TABLE_DELETE_EC, [134](#)
- DDSC_LOG_TABLE_INUSE_EC, [134](#)
- DDSC_LOG_TABLE_SELECT_EC, [134](#)
- DDSC_LOG_TIMER_CREATE_TIMEOUT_EC, [133](#)
- DDSC_LOG_TIMER_OBJECT, [138](#)
- DDSC_LOG_TIMEROUT_OBJECT, [138](#)
- DDSC_LOG_TOPIC_ENTITY, [145](#)
- DDSC_LOG_TOPIC_FIND_EC, [131](#)
- DDSC_LOG_TOPIC_LISTENER, [144](#)
- DDSC_LOG_TOPIC_NAME_CMP_EC, [150](#)
- DDSC_LOG_TOPIC_NARROW_EC, [133](#)
- DDSC_LOG_TOPIC_OBJECT, [138](#)
- DDSC_LOG_TOPIC_QOS, [141](#)
- DDSC_LOG_TOPIC_RECORD, [134](#)
- DDSC_LOG_TOPIC_TOO_LONG_EC, [151](#)
- DDSC_LOG_TOPICQOS_OBJECT, [136](#)
- DDSC_LOG_TRANSPORT_COMPONENT, [141](#)
- DDSC_LOG_TRANSPORT_QOS_POLICY, [143](#)
- DDSC_LOG_TYPE_FUNCTION_COPY_SAMPLE, [151](#)
- DDSC_LOG_TYPE_FUNCTION_CREATE_SAMPLE, [151](#)
- DDSC_LOG_TYPE_FUNCTION_DELETE_SAMPLE, [151](#)
- DDSC_LOG_TYPE_FUNCTION_DESERIALIZE_DATA, [150](#)
- DDSC_LOG_TYPE_FUNCTION_GET_KEY_KIND, [151](#)
- DDSC_LOG_TYPE_FUNCTION_GET_SERIALIZED_SAMPLE_MAX_SIZE, [150](#)
- DDSC_LOG_TYPE_FUNCTION_INSTANCE_TO_KEYHASH, [151](#)
- DDSC_LOG_TYPE_FUNCTION_NULL_EC, [151](#)
- DDSC_LOG_TYPE_FUNCTION_SERIALIZE_DATA, [150](#)
- DDSC_LOG_TYPE_KEY_TYPE_EC, [151](#)
- DDSC_LOG_TYPE_NAME_CMP_EC, [150](#)
- DDSC_LOG_TYPE_OBJECT, [137](#)
- DDSC_LOG_TYPE_RECORD, [135](#)
- DDSC_LOG_TYPE_SUPPORT_QOS_POLICY, [143](#)
- DDSC_LOG_TYPE_TOO_LONG_EC, [151](#)
- DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_KEY_EC, [132](#)
- DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_NAME_EC, [132](#)
- DDSC_LOG_USER_TRAFFIC_QOS_POLICY, [143](#)
- DDSC_LOG_USERMULTICAST_SEQUENCE, [139](#)
- DDSC_LOG_USERUNICAST_SEQUENCE, [139](#)
- DDSC_LOG_UUID_OBJECT, [138](#)
- DDSC_LOG_WAITSET_OBJECT, [138](#)
- DDSC_LOG_WRITERHISTORY_COMPONENT, [140](#)
- DDSC_LOG_WS_ADD_COND_REFERENCE_EC, [133](#)
- DDSC_LOG_WS_REMOVE_COND_REFERENCE_EC, [133](#)
- DDSC_LOG_WSREF_OBJECT, [137](#)
- DDS_CHECKSUM_AUTO_WIRE_PROTOCOL, [109](#)
- DDS_CHECKSUM_BUILTIN28_WIRE_PROTOCOL, [109](#)

- DDS_CHECKSUM_BUILTIN32
 - WIRE_PROTOCOL, [109](#)
- DDS_CHECKSUM_BUILTIN64
 - WIRE_PROTOCOL, [109](#)
- DDS_CHECKSUM_NONE
 - WIRE_PROTOCOL, [109](#)
- DDS_Char
 - DDS-Specific Primitive Types, [71](#)
- DDS_ChecksumKind_t
 - WIRE_PROTOCOL, [109](#)
- DDS_ChecksumKindMask_t
 - WIRE_PROTOCOL, [109](#)
- DDS_ChecksumProperty, [237](#)
 - allowed_crc_mask, [238](#)
 - computed_crc_kind, [238](#)
 - require_crc, [238](#)
- DDS_ChecksumProperty_t, [238](#)
 - allowed_crc_mask, [238](#)
 - computed_crc_kind, [238](#)
 - require_crc, [238](#)
- DDS_DATA_AVAILABLE_STATUS
 - Status Kinds, [87](#)
- DDS_DATA_ON_READERS_STATUS
 - Status Kinds, [87](#)
- DDS_DATAREADER_CPP
 - User Data Type Support, [28](#)
- DDS_DATAREADER_QOS_DEFAULT
 - Subscriber, [36](#)
- DDS_DATAREADERPROTOCOL_QOS_POLICY_NAME
 - DATA_READER_PROTOCOL, [112](#)
- DDS_DATAREADERRESOURCELIMITS_QOS_POLICY_NAME
 - DataReader Resource Limits, [103](#)
- DDS_DATAWRITER_CPP
 - User Data Type Support, [28](#)
- DDS_DATAWRITER_QOS_DEFAULT
 - Publisher, [31](#)
- DDS_DATAWRITERPROTOCOL_QOS_POLICY_NAME
 - DATA_WRITER_PROTOCOL, [113](#)
- DDS_DATAWRITERRESOURCELIMITS_QOS_POLICY_NAME
 - DataWriter Resource Limits, [104](#)
- DDS_DEADLINE_QOS_POLICY_ID
 - QoS Policies, [91](#)
- DDS_DEADLINE_QOS_POLICY_NAME
 - DEADLINE, [92](#)
- DDS_DISCOVERY_QOS_POLICY_NAME
 - DISCOVERY, [76](#)
- DDS_DOMAINPARTICIPANTRESOURCELIMITS_QOS_POLICY_NAME
 - DOMAIN_PARTICIPANT_RESOURCE_LIMITS, [115](#)
- DDS_DURABILITY_QOS_POLICY_NAME
 - DURABILITY, [99](#)
- DDS_DURATION_INFINITE
 - Time Support, [78](#)
- DDS_DURATION_INFINITE_NSEC
 - Time Support, [78](#)
- DDS_DURATION_INFINITE_SEC
 - Time Support, [78](#)
- DDS_DURATION_ZERO
 - Time Support, [78](#)
- DDS_DURATION_ZERO_NSEC
 - Time Support, [78](#)
- DDS_DURATION_ZERO_SEC
 - Time Support, [78](#)
- DDS_DataReaderInstanceReplacedStatus, [238](#)
 - last_instance_handle, [239](#)
 - last_replacement_instance, [239](#)
 - lost_samples, [239](#)
 - publication_handle, [239](#)
 - total_count, [239](#)
 - total_count_change, [239](#)
- DDS_DataReaderProtocolQosPolicy, [239](#)
 - rtps_object_id, [240](#)
 - rtps_reliable_reader, [240](#)
- DDS_DataReaderQos, [240](#)
 - deadline, [241](#)
 - destination_order, [241](#)
 - durability, [241](#)
 - history, [241](#)
 - liveliness, [241](#)
 - ownership, [241](#)
 - protocol, [242](#)
 - reader_resource_limits, [242](#)
 - resource_limits, [241](#)
 - transport, [242](#)
- DDS_DataReaderResourceLimitsInstanceReplacementKind
 - DataReader Resource Limits, [103](#)
- DDS_DataReaderResourceLimitsQosPolicy, [242](#)
 - instance_replacement, [243](#)
 - max_outstanding_reads, [243](#)
 - max_remote_writers, [242](#)
 - max_remote_writers_per_instance, [243](#)
 - max_routes_per_writer, [243](#)
 - max_samples_per_remote_writer, [243](#)
- DDS_DataWriterListener_ReliableReaderActivity-ChangedCallback
 - DataWriter, [33](#)
- DDS_DataWriterListener_ReliableSampleUnacknowledged-Callback
 - DataWriter, [33](#)
- DDS_DataWriterProtocolQosPolicy, [244](#)
 - rtps_object_id, [244](#)
 - rtps_reliable_writer, [244](#)
- DDS_DataWriterQos, [245](#)
 - deadline, [246](#)
 - destination_order, [246](#)
 - durability, [246](#)
 - history, [246](#)
 - liveliness, [246](#)
 - ownership, [246](#)
 - ownership_strength, [246](#)

- protocol, 246
- reliability, 246
- resource_limits, 246
- transport, 247
- writer_resource_limits, 247
- DDS_DataWriterResourceLimitsQosPolicy, 247
 - max_remote_readers, 247
 - max_routes_per_reader, 247
- DDS_DeadlineQosPolicy, 248
 - period, 249
- DDS_DestinationOrderQosPolicy, 249
 - kind, 249
 - source_timestamp_tolerance, 249
- DDS_DestinationOrderQosPolicyKind
 - DESTINATION_ORDER, 102
- DDS_DiscoveryComponent, 249
 - name, 250
- DDS_DiscoveryQosPolicy, 250
 - accept_unknown_peers, 251
 - discovery, 251
 - enabled_transports, 251
 - initial_peers, 250
- DDS_DomainId_t
 - Domain Module, 16
- DDS_DomainParticipantFactoryQos, 251
 - entity_factory, 252
 - resource_limits, 252
- DDS_DomainParticipantQos, 252
 - discovery, 253
 - entity_factory, 253
 - participant_name, 253
 - protocol, 253
 - resource_limits, 253
 - transports, 253
 - user_traffic, 253
- DDS_DomainParticipantResourceLimitsQosPolicy, 253
 - local_publisher_allocation, 254
 - local_reader_allocation, 254
 - local_subscriber_allocation, 255
 - local_topic_allocation, 255
 - local_type_allocation, 255
 - local_writer_allocation, 254
 - matching_writer_reader_pair_allocation, 255
 - max_destination_ports, 256
 - max_receive_ports, 256
 - remote_participant_allocation, 255
 - remote_reader_allocation, 255
 - remote_writer_allocation, 255
- DDS_Double
 - DDS-Specific Primitive Types, 72
- DDS_DurabilityQosPolicy, 257
 - kind, 257
- DDS_DurabilityQosPolicyKind
 - DURABILITY, 99
- DDS_Duration_t, 257
 - nanosec, 258
 - sec, 258
- DDS_ENTITYFACTORY_QOS_POLICY_ID
 - QoS Policies, 91
- DDS_ENTITYFACTORY_QOS_POLICY_NAME
 - ENTITY_FACTORY, 105
- DDS_ENTITYNAME_QOS_NAME_MAX
 - ENTITY_NAME, 118
- DDS_ENTITYNAME_QOS_POLICY_NAME
 - ENTITY_NAME, 118
- DDS_EXCLUSIVE_OWNERSHIP_QOS
 - OWNERSHIP, 93
- DDS_EntityFactoryQosPolicy, 258
 - autoenable_created_entities, 259
- DDS_EntityNameQosPolicy, 259
 - name, 259
- DDS_Enum
 - DDS-Specific Primitive Types, 72
- DDS_Float
 - DDS-Specific Primitive Types, 72
- DDS_GROUP_PRESENTATION_QOS
 - Presentation, 101
- DDS_GUID_UNKNOWN
 - GUID Support, 79
- DDS_GUID_t, 260
 - value, 260
- DDS_HANDLE_NIL
 - User Data Type Support, 29
- DDS_HIGHEST_OFFERED_PRESENTATION_QOS
 - Presentation, 101
- DDS_HISTORY_QOS_POLICY_ID
 - QoS Policies, 91
- DDS_HISTORY_QOS_POLICY_NAME
 - HISTORY, 98
- DDS_HistoryQosPolicy, 260
 - depth, 261
 - kind, 261
- DDS_HistoryQosPolicyKind
 - HISTORY, 98
- DDS_INCONSISTENT_TOPIC_STATUS
 - Status Kinds, 86
- DDS_INSTANCE_PRESENTATION_QOS
 - Presentation, 101
- DDS_INSTANCE_REPLACED_STATUS
 - Status Kinds, 88
- DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS
 - WIRE_PROTOCOL, 110
- DDS_INVALID_QOS_POLICY_ID
 - QoS Policies, 91
- DDS_InconsistentTopicStatus, 261
- DDS_InstanceHandle_equals
 - User Data Type Support, 28
- DDS_InstanceHandle_is_nil
 - User Data Type Support, 29
- DDS_InstanceHandle_t
 - User Data Type Support, 28
- DDS_InstanceStateKind
 - Instance States, 45
- DDS_InstanceStateMask
 - Instance States, 45

- DDS_KEEP_LAST_HISTORY_QOS
 - HISTORY, [98](#)
- DDS_LENGTH_AUTO
 - RESOURCE_LIMITS, [100](#)
- DDS_LENGTH_UNLIMITED
 - RESOURCE_LIMITS, [100](#)
- DDS_LIVELINESS_CHANGED_STATUS
 - Status Kinds, [88](#)
- DDS_LIVELINESS_LOST_STATUS
 - Status Kinds, [87](#)
- DDS_LIVELINESS_QOS_POLICY_ID
 - QoS Policies, [91](#)
- DDS_LIVELINESS_QOS_POLICY_NAME
 - LIVELINESS, [96](#)
- DDS_LOCATOR_ADDRESS_INVALID
 - Discovery, [23](#)
- DDS_LOCATOR_INVALID
 - Discovery, [22](#)
- DDS_LOCATOR_KIND_INVALID
 - Discovery, [23](#)
- DDS_LOCATOR_KIND_RESERVED
 - Discovery, [23](#)
- DDS_LOCATOR_KIND_SHMEM
 - Discovery, [23](#)
- DDS_LOCATOR_KIND_UDPv4
 - Discovery, [23](#)
- DDS_LOCATOR_KIND_UDPv6
 - Discovery, [23](#)
- DDS_LOCATOR_PORT_INVALID
 - Discovery, [23](#)
- DDS_LivelinessChangedStatus, [262](#)
 - alive_count, [262](#)
 - alive_count_change, [262](#)
 - last_publication_handle, [262](#)
 - not_alive_count, [262](#)
 - not_alive_count_change, [262](#)
- DDS_LivelinessLostStatus, [262](#)
 - total_count, [263](#)
 - total_count_change, [263](#)
- DDS_LivelinessQosPolicy, [263](#)
 - kind, [265](#)
 - lease_duration, [265](#)
- DDS_LivelinessQosPolicyKind
 - LIVELINESS, [95](#)
- DDS_Locator_t, [265](#)
 - address, [266](#)
 - kind, [266](#)
 - port, [266](#)
- DDS_LocatorSeq, [266](#)
- DDS_Long
 - DDS-Specific Primitive Types, [71](#)
- DDS_LongDouble
 - DDS-Specific Primitive Types, [72](#)
- DDS_LongLong
 - DDS-Specific Primitive Types, [71](#)
- DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS
 - LIVELINESS, [95](#)
- DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS
 - LIVELINESS, [95](#)
- DDS_NEW_VIEW_STATE
 - View States, [44](#)
- DDS_NO_INSTANCE_REPLACEMENT_QOS
 - DataReader Resource Limits, [103](#)
- DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE
 - Instance States, [46](#)
- DDS_NOT_ALIVE_INSTANCE_STATE
 - Instance States, [46](#)
- DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE
 - Instance States, [46](#)
- DDS_NOT_NEW_VIEW_STATE
 - View States, [44](#)
- DDS_NOT_READ_SAMPLE_STATE
 - Sample States, [41](#)
- DDS_NOT_REJECTED
 - DataReader, [38](#)
- DDS_OFFERED_DEADLINE_MISSED_STATUS
 - Status Kinds, [86](#)
- DDS_OFFERED_INCOMPATIBLE_QOS_STATUS
 - Status Kinds, [86](#)
- DDS_OWNERSHIP_QOS_POLICY_ID
 - QoS Policies, [91](#)
- DDS_OWNERSHIP_QOS_POLICY_NAME
 - OWNERSHIP, [93](#)
- DDS_OWNERSHIPSTRENGTH_QOS_POLICY_ID
 - QoS Policies, [91](#)
- DDS_OWNERSHIPSTRENGTH_QOS_POLICY_NAME
 - OWNERSHIP_STRENGTH, [94](#)
- DDS_Octet
 - DDS-Specific Primitive Types, [71](#)
- DDS_OfferedDeadlineMissedStatus, [266](#)
 - last_instance_handle, [267](#)
 - total_count, [267](#)
 - total_count_change, [267](#)
- DDS_OfferedIncompatibleQosStatus, [267](#)
 - last_policy_id, [268](#)
 - policies, [268](#)
 - total_count, [268](#)
 - total_count_change, [268](#)
- DDS_OwnershipQosPolicy, [268](#)
 - kind, [271](#)
- DDS_OwnershipQosPolicyKind
 - OWNERSHIP, [93](#)
- DDS_OwnershipStrengthQosPolicy, [271](#)
 - value, [272](#)
- DDS_PARTICIPANT_FACTORY_QOS_DEFAULT
 - DomainParticipantFactory, [18](#)
- DDS_PARTICIPANT_QOS_DEFAULT
 - DomainParticipantFactory, [18](#)
- DDS_PARTICIPANT_TOPIC_NAME
 - Participant Built-in Topic, [73](#)
- DDS_PRESENTATION_QOS_POLICY_ID
 - QoS Policies, [91](#)
- DDS_PROTOCOLVERSION

- Discovery, [23](#)
- DDS_PROTOCOLVERSION_1_0
 - Discovery, [23](#)
- DDS_PROTOCOLVERSION_1_1
 - Discovery, [23](#)
- DDS_PROTOCOLVERSION_1_2
 - Discovery, [23](#)
- DDS_PROTOCOLVERSION_2_0
 - Discovery, [23](#)
- DDS_PROTOCOLVERSION_2_1
 - Discovery, [23](#)
- DDS_PUBLICATION_MATCHED_STATUS
 - Status Kinds, [88](#)
- DDS_PUBLICATION_TOPIC_NAME
 - Publication Built-in Topic, [74](#)
- DDS_PUBLISHER_QOS_DEFAULT
 - DomainParticipant, [19](#)
- DDS_ParticipantBuiltinTopicData, [272](#)
 - checksum, [273](#)
 - dds_builtin_endpoints, [273](#)
 - default_multicast_locators, [273](#)
 - default_unicast_locators, [273](#)
 - key, [273](#)
 - liveliness_lease_duration, [273](#)
 - metatraffic_multicast_locators, [273](#)
 - metatraffic_unicast_locators, [273](#)
 - participant_name, [273](#)
 - product_version, [273](#)
 - rtps_protocol_version, [273](#)
 - rtps_vendor_id, [273](#)
- DDS_PresentationQosPolicy, [273](#)
 - access_scope, [276](#)
 - coherent_access, [276](#)
 - ordered_access, [276](#)
- DDS_PresentationQosPolicyAccessScopeKind PRESENTATION, [101](#)
- DDS_ProductVersion, [276](#)
 - major, [276](#)
 - minor, [277](#)
 - release, [277](#)
 - revision, [277](#)
- DDS_ProductVersion_t, [277](#)
 - major, [277](#)
 - minor, [277](#)
 - release, [277](#)
 - revision, [277](#)
- DDS_ProtocolVersion, [277](#)
 - major, [278](#)
 - minor, [278](#)
- DDS_ProtocolVersion_t
 - Discovery, [23](#)
- DDS_PublicationBuiltinTopicData, [278](#)
 - deadline, [279](#)
 - destination_order, [280](#)
 - durability, [280](#)
 - key, [279](#)
 - liveliness, [280](#)
 - ownership, [279](#)
 - ownership_strength, [279](#)
 - participant_key, [279](#)
 - reliability, [279](#)
 - topic_name, [279](#)
 - type_name, [279](#)
 - unicast_locator, [280](#)
- DDS_PublicationMatchedStatus, [280](#)
 - current_count, [281](#)
 - current_count_change, [281](#)
 - last_subscription_handle, [281](#)
 - total_count, [280](#)
 - total_count_change, [280](#)
- DDS_PublisherQos, [281](#)
 - entity_factory, [281](#)
- DDS_QosPolicyId_t
 - QoS Policies, [91](#)
- DDS_READ_SAMPLE_STATE
 - Sample States, [41](#)
- DDS_REJECTED_BY_INSTANCES_LIMIT
 - DataReader, [38](#)
- DDS_REJECTED_BY_REMOTE_WRITERS_LIMIT
 - DataReader, [38](#)
- DDS_REJECTED_BY_SAMPLES_LIMIT
 - DataReader, [38](#)
- DDS_REJECTED_BY_SAMPLES_PER_INSTANCE_LIMIT
 - DataReader, [38](#)
- DDS_REJECTED_BY_SAMPLES_PER_REMOTE_WRITER_LIMIT
 - DataReader, [38](#)
- DDS_RELIABILITY_QOS_POLICY_ID
 - QoS Policies, [91](#)
- DDS_RELIABILITY_QOS_POLICY_NAME
 - RELIABILITY, [97](#)
- DDS_RELIABLE_READER_ACTIVITY_CHANGED_STATUS
 - Status Kinds, [88](#)
- DDS_RELIABLE_RELIABILITY_QOS
 - RELIABILITY, [97](#)
- DDS_REPLACE_OLDEST_INSTANCE_REPLACEMENT_QOS
 - DataReader Resource Limits, [103](#)
- DDS_REQUESTED_DEADLINE_MISSED_STATUS
 - Status Kinds, [86](#)
- DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS
 - Status Kinds, [86](#)
- DDS_RESOURCELIMITS_QOS_POLICY_NAME
 - RESOURCE_LIMITS, [100](#)
- DDS_RETCODE_ALREADY_DELETED
 - Return Codes, [82](#)
- DDS_RETCODE_BAD_PARAMETER
 - Return Codes, [81](#)
- DDS_RETCODE_ERROR
 - Return Codes, [81](#)
- DDS_RETCODE_ILLEGAL_OPERATION
 - Return Codes, [82](#)
- DDS_RETCODE_IMMUTABLE_POLICY
 - Return Codes, [82](#)

- DDS_RETCODE_INCONSISTENT_POLICY
 - Return Codes, [82](#)
- DDS_RETCODE_NO_DATA
 - Return Codes, [82](#)
- DDS_RETCODE_NOT_ENABLED
 - Return Codes, [82](#)
- DDS_RETCODE_OK
 - Return Codes, [81](#)
- DDS_RETCODE_OUT_OF_RESOURCES
 - Return Codes, [82](#)
- DDS_RETCODE_PRECONDITION_NOT_MET
 - Return Codes, [82](#)
- DDS_RETCODE_TIMEOUT
 - Return Codes, [82](#)
- DDS_RETCODE_UNSUPPORTED
 - Return Codes, [81](#)
- DDS_RTI_BACKWARDS_COMPATIBLE_RTPS_WELL_KNOWN_PORTS
 - WIRE_PROTOCOL, [110](#)
- DDS_ReliabilityQosPolicy, [281](#)
 - kind, [282](#)
 - max_blocking_time, [282](#)
- DDS_ReliabilityQosPolicyKind
 - RELIABILITY, [97](#)
- DDS_ReliableReaderActivityChangedStatus, [283](#)
 - active_count, [283](#)
 - active_count_change, [284](#)
 - inactive_count, [283](#)
 - inactive_count_change, [284](#)
 - last_instance_handle, [284](#)
- DDS_ReliableReaderActivityChangedStatus_INITIALIZER
 - DataWriter, [33](#)
- DDS_ReliableSampleUnacknowledgedStatus, [284](#)
 - instance_handle, [284](#)
 - sequence_number, [284](#)
 - unacknowledged_count, [284](#)
- DDS_RequestedDeadlineMissedStatus, [284](#)
 - last_instance_handle, [285](#)
 - total_count, [285](#)
 - total_count_change, [285](#)
- DDS_RequestedIncompatibleQosStatus, [285](#)
 - last_policy_id, [286](#)
 - policies, [286](#)
 - total_count, [286](#)
 - total_count_change, [286](#)
- DDS_ResourceLimitsQosPolicy, [286](#)
 - max_instances, [288](#)
 - max_samples, [287](#)
 - max_samples_per_instance, [288](#)
- DDS_ReturnCode_t
 - Return Codes, [81](#)
- DDS_RtpsReliableReaderProtocol_t, [289](#)
 - nack_period, [289](#)
- DDS_RtpsReliableWriterProtocol_t, [290](#)
 - heartbeat_period, [290](#)
 - heartbeats_per_max_samples, [290](#)
 - max_heartbeat_retries, [291](#)
 - max_send_window, [291](#)
- DDS_RtpsWellKnownPorts_t, [291](#)
 - builtin_multicast_port_offset, [293](#)
 - builtin_unicast_port_offset, [294](#)
 - domain_id_gain, [292](#)
 - participant_id_gain, [293](#)
 - port_base, [292](#)
 - user_multicast_port_offset, [294](#)
 - user_unicast_port_offset, [294](#)
- DDS_SAMPLE_LOST_BY_DATAWRITER
 - DataReader, [38](#)
- DDS_SAMPLE_LOST_BY_HISTORY_DEPTH_LIMIT
 - DataReader, [39](#)
- DDS_SAMPLE_LOST_BY_MAX_SAMPLES_LIMIT
 - DataReader, [38](#)
- DDS_SAMPLE_LOST_BY_META_SAMPLE_LIMIT
 - DataReader, [39](#)
- DDS_SAMPLE_LOST_BY_NOT_READ_ON_CACHE_DELETION
 - DataReader, [39](#)
- DDS_SAMPLE_LOST_NOT_LOST
 - DataReader, [38](#)
- DDS_SAMPLE_LOST_STATUS
 - Status Kinds, [87](#)
- DDS_SAMPLE_REJECTED_STATUS
 - Status Kinds, [87](#)
- DDS_SHARED_OWNERSHIP_QOS
 - OWNERSHIP, [93](#)
- DDS_STATUS_MASK_ALL
 - Status Kinds, [85](#)
- DDS_STATUS_MASK_NONE
 - Status Kinds, [85](#)
- DDS_SUBSCRIBER_QOS_DEFAULT
 - DomainParticipant, [19](#)
- DDS_SUBSCRIPTION_MATCHED_STATUS
 - Status Kinds, [88](#)
- DDS_SUBSCRIPTION_TOPIC_NAME
 - Subscription Built-in Topic, [75](#)
- DDS_SYSTEMRESOURCELIMITS_QOS_POLICY_NAME
 - SYSTEM_RESOURCE_LIMITS, [107](#)
- DDS_SampleInfo, [294](#)
 - instance_handle, [296](#)
 - instance_state, [295](#)
 - publication_handle, [296](#)
 - publication_sequence_number, [296](#)
 - reception_timestamp, [296](#)
 - sample_state, [295](#)
 - source_timestamp, [295](#)
 - valid_data, [296](#)
 - view_state, [295](#)
- DDS_SampleInfoSeq, [296](#)
- DDS_SampleLostStatus, [297](#)
 - reason, [297](#)
 - sample_info, [297](#)
 - total_count, [297](#)
 - total_count_change, [297](#)
- DDS_SampleLostStatusKind

- DataReader, [38](#)
- DDS_SampleRejectedStatus, [297](#)
 - last_instance_handle, [298](#)
 - last_reason, [298](#)
 - total_count, [298](#)
 - total_count_change, [298](#)
- DDS_SampleRejectedStatusKind
 - DataReader, [38](#)
- DDS_SampleStateKind
 - Sample States, [41](#)
- DDS_SampleStateMask
 - Sample States, [41](#)
- DDS_SequenceNumber_compare
 - Sequence Number Support, [80](#)
- DDS_SequenceNumber_t, [298](#)
 - high, [299](#)
 - low, [299](#)
- DDS_Short
 - DDS-Specific Primitive Types, [71](#)
- DDS_StatusKind
 - Status Kinds, [85](#)
- DDS_StatusMask
 - Status Kinds, [85](#)
- DDS_String
 - DDS-Specific Primitive Types, [72](#)
- DDS_String_alloc
 - String Support, [155](#)
- DDS_String_cmp
 - String Support, [156](#)
- DDS_String_dup
 - String Support, [155](#)
- DDS_String_free
 - String Support, [156](#)
- DDS_String_length
 - String Support, [156](#)
- DDS_String_ncmp
 - String Support, [156](#)
- DDS_SubscriberQos, [299](#)
 - entity_factory, [299](#)
- DDS_SubscriptionBuiltinTopicData, [299](#)
 - deadline, [300](#)
 - destination_order, [301](#)
 - durability, [301](#)
 - key, [300](#)
 - liveliness, [301](#)
 - multicast_locator, [301](#)
 - ownership, [301](#)
 - participant_key, [300](#)
 - reliability, [301](#)
 - topic_name, [300](#)
 - type_name, [300](#)
 - unicast_locator, [301](#)
- DDS_SubscriptionMatchedReader, [301](#)
 - current_count, [302](#)
 - current_count_change, [302](#)
 - last_publication_handle, [302](#)
 - total_count, [302](#)
 - total_count_change, [302](#)
- DDS_SystemResourceLimitsQosPolicy, [302](#)
 - max_components, [303](#)
 - max_participants, [303](#)
- DDS_TIME_INVALID
 - Time Support, [78](#)
- DDS_TIME_INVALID_NSEC
 - Time Support, [77](#)
- DDS_TIME_INVALID_SEC
 - Time Support, [77](#)
- DDS_TIME_ZERO
 - Time Support, [77](#)
- DDS_TOPIC_PRESENTATION_QOS
 - Presentation, [101](#)
- DDS_TOPIC_QOS_DEFAULT
 - DomainParticipant, [19](#)
- DDS_TRANSIENT_LOCAL_DURABILITY_QOS
 - Durability, [99](#)
- DDS_TRANSPORT_QOS_POLICY_NAME
 - Transport, [114](#)
- DDS_TYPESUPPORT_CPP
 - User Data Type Support, [28](#)
- DDS_Time_t, [303](#)
 - nanosec, [303](#)
 - sec, [303](#)
- DDS_TopicQos, [303](#)
- DDS_TransportQosPolicy, [304](#)
 - enabled_transports, [304](#)
- DDS_UnsignedLong
 - DDS-Specific Primitive Types, [71](#)
- DDS_UnsignedLongLong
 - DDS-Specific Primitive Types, [71](#)
- DDS_UnsignedShort
 - DDS-Specific Primitive Types, [71](#)
- DDS_UserTrafficQosPolicy, [304](#)
 - enabled_transports, [305](#)
- DDS_VENDOR_ID_LENGTH_MAX
 - Discovery, [23](#)
- DDS_VOLATILE_DURABILITY_QOS
 - Durability, [99](#)
- DDS_VendorId, [305](#)
 - vendorId, [305](#)
- DDS_VendorId_t, [305](#)
 - vendorId, [306](#)
- DDS_ViewStateKind
 - View States, [43](#)
- DDS_ViewStateMask
 - View States, [43](#)
- DDS_WIREPROTOCOL_QOS_POLICY_NAME
 - Wire_Protocol, [110](#)
- DDS_Wchar
 - DDS-Specific Primitive Types, [71](#)
- DDS_WireProtocolQosPolicy, [306](#)
 - allowed_crc_mask, [309](#)
 - check_crc, [308](#)
 - compute_crc, [308](#)
 - computed_crc_kind, [308](#)
 - participant_id, [307](#)
 - require_crc, [308](#)

- rtps_app_id, [307](#)
 - rtps_host_id, [307](#)
 - rtps_instance_id, [307](#)
 - rtps_well_known_ports, [308](#)
- DDS_Wstring
 - DDS-Specific Primitive Types, [72](#)
- DDS_Wstring_compare
 - String Support, [157](#)
- DDSC_LOG_ADDRESSRESOLVER_OBJECT
 - DDS_C, [137](#)
- DDSC_LOG_BIND_RECORD
 - DDS_C, [135](#)
- DDSC_LOG_BINDRESOLVER_OBJECT
 - DDS_C, [137](#)
- DDSC_LOG_CDR_BUFFER_SET_EC
 - DDS_C, [146](#)
- DDSC_LOG_CDR_DESERIALIZE_DATA_EC
 - DDS_C, [147](#)
- DDSC_LOG_CDR_DESERIALIZE_HEADER_EC
 - DDS_C, [147](#)
- DDSC_LOG_CDR_DESERIALIZE_KEY_EC
 - DDS_C, [147](#)
- DDSC_LOG_CDR_DESERIALIZE_KEYHASH_EC
 - DDS_C, [147](#)
- DDSC_LOG_CDR_DESERIALIZE_PID_EC
 - DDS_C, [147](#)
- DDSC_LOG_CDR_DESERIALIZE_PID_LENGTH_EC
 - DDS_C, [147](#)
- DDSC_LOG_CDR_FINALIZE_SAMPLE_EC
 - DDS_C, [147](#)
- DDSC_LOG_CDR_INCREMENT_POS_EC
 - DDS_C, [147](#)
- DDSC_LOG_CDR_INITIALIZE_SAMPLE_EC
 - DDS_C, [147](#)
- DDSC_LOG_CDR_POOL_ALLOC_EC
 - DDS_C, [146](#)
- DDSC_LOG_CDR_POOL_DELETE_EC
 - DDS_C, [146](#)
- DDSC_LOG_CDR_SERIALIZE_DATA_EC
 - DDS_C, [146](#)
- DDSC_LOG_CDR_SERIALIZE_KEYHASH_EC
 - DDS_C, [146](#)
- DDSC_LOG_CDR_SERIALIZE_PID_EC
 - DDS_C, [146](#)
- DDSC_LOG_CDR_SERIALIZE_PID_LENGTH_EC
 - DDS_C, [146](#)
- DDSC_LOG_CDR_SERIALIZE_STATUS_INFO_EC
 - DDS_C, [147](#)
- DDSC_LOG_CDR_SET_POS_EC
 - DDS_C, [147](#)
- DDSC_LOG_CHECKSUM_INCOMPATIBLE_EC
 - DDS_C, [152](#)
- DDSC_LOG_CHECKSUM_INCONSISTENT_ALLOW_-EC
 - DDS_C, [152](#)
- DDSC_LOG_CHECKSUM_INCONSISTENT_CLASS_-EC
 - DDS_C, [152](#)
- DDSC_LOG_CHECKSUM_INCONSISTENT_COMPU-TE_EC
 - DDS_C, [152](#)
- DDSC_LOG_CHECKSUM_REQUIRED_EC
 - DDS_C, [152](#)
- DDSC_LOG_COMPONENT_CREATE_EC
 - DDS_C, [141](#)
- DDSC_LOG_COMPONENT_DELETE_EC
 - DDS_C, [141](#)
- DDSC_LOG_COMPONENT_LOOKUP_EC
 - DDS_C, [141](#)
- DDSC_LOG_CONDITION_OBJECT
 - DDS_C, [137](#)
- DDSC_LOG_CONDREF_OBJECT
 - DDS_C, [137](#)
- DDSC_LOG_CREATE_INDEX_EC
 - DDS_C, [134](#)
- DDSC_LOG_DATABASE_CREATE_EC
 - DDS_C, [133](#)
- DDSC_LOG_DATABASE_DELETE_EC
 - DDS_C, [133](#)
- DDSC_LOG_DATAREADER_CDR
 - DDS_C, [146](#)
- DDSC_LOG_DATAREADER_ENTITY
 - DDS_C, [145](#)
- DDSC_LOG_DATAREADER_LISTENER
 - DDS_C, [144](#)
- DDSC_LOG_DATAREADER_NETIO_KIND
 - DDS_C, [148](#)
- DDSC_LOG_DATAREADER_QOS
 - DDS_C, [141](#)
- DDSC_LOG_DATAREADER_RECORD
 - DDS_C, [134](#)
- DDSC_LOG_DATAREADER_RESOURCE_QOS_PO-LICY
 - DDS_C, [143](#)
- DDSC_LOG_DATAREADERIO_COMPONENT
 - DDS_C, [140](#)
- DDSC_LOG_DATAREADERIO_OBJECT
 - DDS_C, [137](#)
- DDSC_LOG_DATAREADERQOS_OBJECT
 - DDS_C, [136](#)
- DDSC_LOG_DATAWRITER_CDR
 - DDS_C, [146](#)
- DDSC_LOG_DATAWRITER_ENTITY
 - DDS_C, [145](#)
- DDSC_LOG_DATAWRITER_LISTENER
 - DDS_C, [144](#)
- DDSC_LOG_DATAWRITER_NETIO_KIND
 - DDS_C, [148](#)
- DDSC_LOG_DATAWRITER_QOS
 - DDS_C, [141](#)
- DDSC_LOG_DATAWRITER_RECORD
 - DDS_C, [134](#)
- DDSC_LOG_DATAWRITER_RESOURCE_QOS_POL-ICY
 - DDS_C, [143](#)
- DDSC_LOG_DATAWRITERIO_COMPONENT

- DDS_C, [140](#)
- DDSC_LOG_DATAWRITERIO_OBJECT
 - DDS_C, [137](#)
- DDSC_LOG_DATAWRITERQOS_OBJECT
 - DDS_C, [136](#)
- DDSC_LOG_DB_CURSOR_INVALIDATED_EC
 - DDS_C, [134](#)
- DDSC_LOG_DEADLINE_QOS_POLICY
 - DDS_C, [142](#)
- DDSC_LOG_DEFAULTDATAREADER_QOS
 - DDS_C, [142](#)
- DDSC_LOG_DEFAULTDATAWRITER_QOS
 - DDS_C, [142](#)
- DDSC_LOG_DEFAULTPARTICIPANT_QOS
 - DDS_C, [142](#)
- DDSC_LOG_DEFAULTPUBLISHER_QOS
 - DDS_C, [142](#)
- DDSC_LOG_DEFAULTSUBSCRIBER_QOS
 - DDS_C, [142](#)
- DDSC_LOG_DEFAULTTOPIC_QOS
 - DDS_C, [142](#)
- DDSC_LOG_DELETE_INDEX_EC
 - DDS_C, [134](#)
- DDSC_LOG_DESERIALIZE_BAD_PID_LENGTH_EC
 - DDS_C, [146](#)
- DDSC_LOG_DESTINATION_ORDER_POLICY
 - DDS_C, [143](#)
- DDSC_LOG_DESTINATION_SEQUENCE
 - DDS_C, [139](#)
- DDSC_LOG_DISC_ADD_PEER_EC
 - DDS_C, [152](#)
- DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_-DELETED_EC
 - DDS_C, [152](#)
- DDSC_LOG_DISC_AFTER_LOCAL_DATAREADER_-ENABLED_EC
 - DDS_C, [152](#)
- DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_-DELETED_EC
 - DDS_C, [152](#)
- DDSC_LOG_DISC_AFTER_LOCAL_DATAWRITER_-ENABLED_EC
 - DDS_C, [152](#)
- DDSC_LOG_DISC_AFTER_LOCAL_PARTICIPANT_-CREATED_EC
 - DDS_C, [151](#)
- DDSC_LOG_DISC_BEFORE_LOCAL_PARTICIPANT_-CREATED_EC
 - DDS_C, [151](#)
- DDSC_LOG_DISC_LOCAL_PARTICIPANT_ENABLE-D_EC
 - DDS_C, [151](#)
- DDSC_LOG_DISC_REMOTE_PARTICIPANT_EXPIR-ED_EC
 - DDS_C, [152](#)
- DDSC_LOG_DISCOVERY_COMPONENT
 - DDS_C, [140](#)
- DDSC_LOG_DISCOVERY_TRAFFIC_QOS_POLICY
 - DDS_C, [143](#)
- DDSC_LOG_DR_COMMIT_ENTRY_EC
 - DDS_C, [150](#)
- DDSC_LOG_DR_COMMIT_SAMPLE_EC
 - DDS_C, [150](#)
- DDSC_LOG_DR_COPY_DATA_SAMPLE_EC
 - DDS_C, [150](#)
- DDSC_LOG_DR_CREATE_TYPED_READER_EC
 - DDS_C, [149](#)
- DDSC_LOG_DR_DESERIALIZE_KEYHASH_EC
 - DDS_C, [150](#)
- DDSC_LOG_DR_DISPOSE_KEY_EC
 - DDS_C, [150](#)
- DDSC_LOG_DR_FILTER_ERROR_EC
 - DDS_C, [150](#)
- DDSC_LOG_DR_GET_ENTRY_FAILED_EC
 - DDS_C, [150](#)
- DDSC_LOG_DR_INSTANCE_TO_KEYHASH_EC
 - DDS_C, [150](#)
- DDSC_LOG_DR_READ_TAKE_FAILURE_EC
 - DDS_C, [150](#)
- DDSC_LOG_DR_UNREGISTER_KEY_EC
 - DDS_C, [150](#)
- DDSC_LOG_DUPLICATE_GUID_PREFIX_EC
 - DDS_C, [151](#)
- DDSC_LOG_DURABILITY_QOS_POLICY
 - DDS_C, [143](#)
- DDSC_LOG_DW_ACKNACK_FAILED_EC
 - DDS_C, [149](#)
- DDSC_LOG_DW_COMMIT_EC
 - DDS_C, [149](#)
- DDSC_LOG_DW_CREATE_TYPED_WRITER_EC
 - DDS_C, [149](#)
- DDSC_LOG_DW_HISTORY_REGISTER_KEY_EC
 - DDS_C, [149](#)
- DDSC_LOG_DW_ILLEGAL_KEY_KIND_EC
 - DDS_C, [149](#)
- DDSC_LOG_DW_KEYHASH_CREATE_EC
 - DDS_C, [149](#)
- DDSC_LOG_ENABLED_DISCOVERY_TRANSPORT-SEQUENCE
 - DDS_C, [139](#)
- DDSC_LOG_ENABLED_TRANSPORT_SEQUENCE
 - DDS_C, [139](#)
- DDSC_LOG_ENABLED_USER_TRANSPORT_SEQU-ENCE
 - DDS_C, [139](#)
- DDSC_LOG_ENDPOINT_NOT_CHILD_OF_PARTICI-PANT_EC
 - DDS_C, [132](#)
- DDSC_LOG_ENTITY_DIFFERENT_FACTORY_EC
 - DDS_C, [146](#)
- DDSC_LOG_ENTITY_ENABLE_EC
 - DDS_C, [145](#)
- DDSC_LOG_ENTITY_FINALIZE_EC
 - DDS_C, [146](#)
- DDSC_LOG_ENTITY_INITIALIZE_EC
 - DDS_C, [146](#)

DDSC_LOG_ENTITY_NOT_EMPTY_EC
DDS_C, 146

DDSC_LOG_ENTITY_NOT_ENABLED_EC
DDS_C, 146

DDSC_LOG_ENTITY_OBJECT
DDS_C, 136

DDSC_LOG_FAILED_UPDATE_STATUS_CONDITION_EC
DDS_C, 133

DDSC_LOG_FIND_PUBLICATION_PARENT_EC
DDS_C, 132

DDSC_LOG_FIND_SUBSCRIPTION_PARENT_EC
DDS_C, 132

DDSC_LOG_GET_NEXT_OBJECT_ID_EC
DDS_C, 131

DDSC_LOG_HEARTBEATS_QOS_POLICY
DDS_C, 143

DDSC_LOG_HISTORY_QOS_POLICY
DDS_C, 142

DDSC_LOG_HISTORY_RESOURCE
DDS_C, 141

DDSC_LOG_ILLEGAL_OBJECTID_EC
DDS_C, 133

DDSC_LOG_INITIAL_PEER_SEQUENCE
DDS_C, 139

DDSC_LOG_INTRA_NETIO_KIND
DDS_C, 148

DDSC_LOG_INVALID_DOMAINID_EC
DDS_C, 133

DDSC_LOG_INVALID_DURATION_EC
DDS_C, 131

DDSC_LOG_INVALID_ENDPOINT_GUID_EC
DDS_C, 132

DDSC_LOG_INVALID_PARTICIPANT_GUID_PREFIX_EC
DDS_C, 131

DDSC_LOG_INVALID_PARTICIPANT_NAME_EC
DDS_C, 131

DDSC_LOG_IO_SNPRINTF_FAILED_EC
DDS_C, 131

DDSC_LOG_LISTENER_GET_EC
DDS_C, 145

DDSC_LOG_LISTENER_INCONSISTENT_EC
DDS_C, 145

DDSC_LOG_LISTENER_SET_EC
DDS_C, 145

DDSC_LOG_LISTENER_SET_ILLEGAL_NULL_EC
DDS_C, 145

DDSC_LOG_LIVELINESS_QOS_POLICY
DDS_C, 142

DDSC_LOG_LOG_OBJECT
DDS_C, 137

DDSC_LOG_LOOKUP_TYPE_PLUGIN_EC
DDS_C, 151

DDSC_LOG_MAX_PARTICIPANT_ID_REACHED_EC
DDS_C, 132

DDSC_LOG_MD5STREAM_OBJECT
DDS_C, 137

DDSC_LOG_METAMULTICAST_SEQUENCE
DDS_C, 139

DDSC_LOG_METAUNICAST_SEQUENCE
DDS_C, 139

DDSC_LOG_MUTEX_OBJECT
DDS_C, 137

DDSC_LOG_NETIO_ADD_ANON_TOPIC_ROUTE_EC
DDS_C, 147

DDSC_LOG_NETIO_ADD_ROUTE_EC
DDS_C, 148

DDSC_LOG_NETIO_ADD_TOPIC_ROUTE_EC
DDS_C, 147

DDSC_LOG_NETIO_BIND_EC
DDS_C, 148

DDSC_LOG_NETIO_BIND_EXTERNAL_EC
DDS_C, 148

DDSC_LOG_NETIO_COMPONENT
DDS_C, 140

DDSC_LOG_NETIO_DELETE_ROUTE_EC
DDS_C, 148

DDSC_LOG_NETIO_DELETE_TOPIC_ROUTE_EC
DDS_C, 147

DDSC_LOG_NETIO_FORCED_REMOVE_EC
DDS_C, 149

DDSC_LOG_NETIO_FORWARD_TOPIC_EC
DDS_C, 147

DDSC_LOG_NETIO_GET_EXTERNAL_INTF_EC
DDS_C, 148

DDSC_LOG_NETIO_GET_ROUTE_TABLE_FAILED_EC
DDS_C, 148

DDSC_LOG_NETIO_NETIO_KIND
DDS_C, 148

DDSC_LOG_NETIO_NO_ROUTE_EC
DDS_C, 148

DDSC_LOG_NETIO_PEER_LOOKUP_EC
DDS_C, 149

DDSC_LOG_NETIO_ROUTE_LOOKUP_FAILED_EC
DDS_C, 148

DDSC_LOG_NETIO_SEND_FAILED_EC
DDS_C, 149

DDSC_LOG_NETIO_SET_STATE_EC
DDS_C, 149

DDSC_LOG_NETIO_UNBIND_EC
DDS_C, 148

DDSC_LOG_NETIO_UNBIND_EXTERNAL_EC
DDS_C, 148

DDSC_LOG_NETMASK_SEQUENCE
DDS_C, 139

DDSC_LOG_OBJECT_ALLOCATE_EC
DDS_C, 138

DDSC_LOG_OBJECT_COPY_EC
DDS_C, 138

DDSC_LOG_OBJECT_DELETE_EC
DDS_C, 138

DDSC_LOG_OBJECT_EMPTY_EC
DDS_C, 139

DDSC_LOG_OBJECT_FINALIZE_EC
 DDS_C, 138
 DDSC_LOG_OBJECT_GET_PROPERTY_EC
 DDS_C, 138
 DDSC_LOG_OBJECT_INITIALIZE_EC
 DDS_C, 138
 DDSC_LOG_OBJECT_INUSECOUNT_EC
 DDS_C, 139
 DDSC_LOG_OBJECT_REFCOUNT_EC
 DDS_C, 138
 DDSC_LOG_OBJECT_SET_PROPERTY_EC
 DDS_C, 139
 DDSC_LOG_OWNERSHIP_QOS_POLICY
 DDS_C, 143
 DDSC_LOG_OWNERSHIP_STRENGTH_QOS_POLICY
 DDS_C, 143
 DDSC_LOG_PACKET_INIT_EC
 DDS_C, 149
 DDSC_LOG_PACKET_SET_HEAD_EC
 DDS_C, 149
 DDSC_LOG_PACKET_SET_TAIL_EC
 DDS_C, 149
 DDSC_LOG_PARTICIPANT_DOES_NOT_EXIST_EC
 DDS_C, 132
 DDSC_LOG_PARTICIPANT_ENTITY
 DDS_C, 145
 DDSC_LOG_PARTICIPANT_ID_QOS_POLICY
 DDS_C, 143
 DDSC_LOG_PARTICIPANT_LISTENER
 DDS_C, 145
 DDSC_LOG_PARTICIPANT_LOOKUP_EC
 DDS_C, 132
 DDSC_LOG_PARTICIPANT_POOL_OBJECT
 DDS_C, 138
 DDSC_LOG_PARTICIPANT_QOS
 DDS_C, 142
 DDSC_LOG_PARTICIPANT_RECORD
 DDS_C, 134
 DDSC_LOG_PARTICIPANT_RESOURCES
 DDS_C, 141
 DDSC_LOG_PARTICIPANTDATA_OBJECT
 DDS_C, 136
 DDSC_LOG_PARTICIPANTFACTORY_LISTENER
 DDS_C, 145
 DDSC_LOG_PARTICIPANTFACTORY_QOS
 DDS_C, 142
 DDSC_LOG_PARTICIPANTFACTORYQOS_OBJECT
 DDS_C, 137
 DDSC_LOG_PARTICIPANTQOS_OBJECT
 DDS_C, 136
 DDSC_LOG_PROTOCOL_QOS_POLICY
 DDS_C, 142
 DDSC_LOG_PUBLICATION_ENTITY
 DDS_C, 145
 DDSC_LOG_PUBLICATION_RECORD
 DDS_C, 134
 DDSC_LOG_PUBLICATIONDATA_OBJECT
 DDS_C, 136
 DDSC_LOG_PUBLISHER_ENTITY
 DDS_C, 145
 DDSC_LOG_PUBLISHER_LISTENER
 DDS_C, 144
 DDSC_LOG_PUBLISHER_QOS
 DDS_C, 142
 DDSC_LOG_PUBLISHER_RECORD
 DDS_C, 134
 DDSC_LOG_PUBLISHERQOS_OBJECT
 DDS_C, 136
 DDSC_LOG_QOS_CHANGED_EC
 DDS_C, 144
 DDSC_LOG_QOS_COPY_EC
 DDS_C, 144
 DDSC_LOG_QOS_FINALIZE_EC
 DDS_C, 144
 DDSC_LOG_QOS_GET_EC
 DDS_C, 144
 DDSC_LOG_QOS_IMMUTABLE_EC
 DDS_C, 144
 DDSC_LOG_QOS_INCONSISTENT_EC
 DDS_C, 144
 DDSC_LOG_QOS_INCONSISTENT_POLICIES_EC
 DDS_C, 144
 DDSC_LOG_QOS_INCONSISTENT_POLICY_EC
 DDS_C, 143
 DDSC_LOG_QOS_INITIALIZE_EC
 DDS_C, 144
 DDSC_LOG_QOS_SET_EC
 DDS_C, 144
 DDSC_LOG_QOS_SET_ON_ENABLED_EC
 DDS_C, 144
 DDSC_LOG_READERHISTORY_COMPONENT
 DDS_C, 140
 DDSC_LOG_READTAKE_SEQUENCE
 DDS_C, 140
 DDSC_LOG_RECORD_CREATE_EC
 DDS_C, 135
 DDSC_LOG_RECORD_DELETE_EC
 DDS_C, 135
 DDSC_LOG_RECORD_ERROR_EC
 DDS_C, 135
 DDSC_LOG_RECORD_EXISTS_EC
 DDS_C, 135
 DDSC_LOG_RECORD_FINALIZE_EC
 DDS_C, 136
 DDSC_LOG_RECORD_INITIALIZE_EC
 DDS_C, 136
 DDSC_LOG_RECORD_INSERT_EC
 DDS_C, 135
 DDSC_LOG_RECORD_LOOKUP_EC
 DDS_C, 135
 DDSC_LOG_RECORD_NOT_EXISTS_EC
 DDS_C, 136
 DDSC_LOG_RECORD_REMOVE_EC
 DDS_C, 136
 DDSC_LOG_RECORD_SELECT_EC

DDS_C, [136](#)
 DDSC_LOG_REFRESH_REM_PARTICIPANT_EC
 DDS_C, [132](#)
 DDSC_LOG_REFRESH_REM_PARTICIPANT_TIME-
 OUT_EC
 DDS_C, [132](#)
 DDSC_LOG_RELEASE_META_MC_EC
 DDS_C, [133](#)
 DDSC_LOG_RELEASE_META_UC_EC
 DDS_C, [133](#)
 DDSC_LOG_RELEASE_USER_MC_EC
 DDS_C, [133](#)
 DDSC_LOG_RELEASE_USER_UC_EC
 DDS_C, [133](#)
 DDSC_LOG_RELIABILITY_QOS_POLICY
 DDS_C, [143](#)
 DDSC_LOG_REMOTE_PARTICIPANT_KEY_NOT_E-
 QUAL_EC
 DDS_C, [132](#)
 DDSC_LOG_REMOVE_PUBLICATION_EC
 DDS_C, [132](#)
 DDSC_LOG_REMOVE_SUBSCRIPTION_EC
 DDS_C, [132](#)
 DDSC_LOG_RESERVE_LOCATORS_EC
 DDS_C, [133](#)
 DDSC_LOG_RESERVED_SEQUENCE
 DDS_C, [139](#)
 DDSC_LOG_RESOURCE_EXCEEDED_EC
 DDS_C, [141](#)
 DDSC_LOG_RESOURCE_LIMIT_QOS_POLICY
 DDS_C, [142](#)
 DDSC_LOG_ROUTE_RECORD
 DDS_C, [135](#)
 DDSC_LOG_ROUTE_SEQUENCE
 DDS_C, [139](#)
 DDSC_LOG_ROUTERESOLVER_OBJECT
 DDS_C, [137](#)
 DDSC_LOG_RT_OBJECT
 DDS_C, [137](#)
 DDSC_LOG_RTPS_COMPONENT
 DDS_C, [140](#)
 DDSC_LOG_RTPS_NETIO_KIND
 DDS_C, [148](#)
 DDSC_LOG_SAMPLE_RESOURCES
 DDS_C, [141](#)
 DDSC_LOG_SEQUENCE_COPY_EC
 DDS_C, [140](#)
 DDSC_LOG_SEQUENCE_FINALIZE_EC
 DDS_C, [140](#)
 DDSC_LOG_SEQUENCE_GETREF_EC
 DDS_C, [140](#)
 DDSC_LOG_SEQUENCE_INITIALIZE_EC
 DDS_C, [140](#)
 DDSC_LOG_SEQUENCE_INVALID_EC
 DDS_C, [140](#)
 DDSC_LOG_SEQUENCE_SETLENGTH_EC
 DDS_C, [140](#)
 DDSC_LOG_SEQUENCE_SETMAX_EC
 DDS_C, [140](#)
 DDSC_LOG_SET_ENTITY_NAME_EC
 DDS_C, [131](#)
 DDSC_LOG_STRING_OBJECT
 DDS_C, [138](#)
 DDSC_LOG_STRING_RECORD
 DDS_C, [135](#)
 DDSC_LOG_STRINGMANAGER_OBJECT
 DDS_C, [138](#)
 DDSC_LOG_SUBSCRIBER_ENTITY
 DDS_C, [145](#)
 DDSC_LOG_SUBSCRIBER_LISTENER
 DDS_C, [144](#)
 DDSC_LOG_SUBSCRIBER_QOS
 DDS_C, [142](#)
 DDSC_LOG_SUBSCRIBER_RECORD
 DDS_C, [134](#)
 DDSC_LOG_SUBSCRIBERQOS_OBJECT
 DDS_C, [136](#)
 DDSC_LOG_SUBSCRIPTION_ENTITY
 DDS_C, [145](#)
 DDSC_LOG_SUBSCRIPTION_RECORD
 DDS_C, [134](#)
 DDSC_LOG_SUBSCRIPTIONDATA_OBJECT
 DDS_C, [136](#)
 DDSC_LOG_SYS_GET_HOSTNAME_EC
 DDS_C, [131](#)
 DDSC_LOG_SYS_GETTIME_EC
 DDS_C, [131](#)
 DDSC_LOG_SYSTEM_OBJECT
 DDS_C, [137](#)
 DDSC_LOG_TABLE_CREATE_EC
 DDS_C, [134](#)
 DDSC_LOG_TABLE_DELETE_EC
 DDS_C, [134](#)
 DDSC_LOG_TABLE_INUSE_EC
 DDS_C, [134](#)
 DDSC_LOG_TABLE_SELECT_EC
 DDS_C, [134](#)
 DDSC_LOG_TIMER_CREATE_TIMEOUT_EC
 DDS_C, [133](#)
 DDSC_LOG_TIMER_OBJECT
 DDS_C, [138](#)
 DDSC_LOG_TIMEROUT_OBJECT
 DDS_C, [138](#)
 DDSC_LOG_TOPIC_ENTITY
 DDS_C, [145](#)
 DDSC_LOG_TOPIC_FIND_EC
 DDS_C, [131](#)
 DDSC_LOG_TOPIC_LISTENER
 DDS_C, [144](#)
 DDSC_LOG_TOPIC_NAME_CMP_EC
 DDS_C, [150](#)
 DDSC_LOG_TOPIC_NARROW_EC
 DDS_C, [133](#)
 DDSC_LOG_TOPIC_OBJECT
 DDS_C, [138](#)
 DDSC_LOG_TOPIC_QOS

- DDS_C, [141](#)
- DDSC_LOG_TOPIC_RECORD
 - DDS_C, [134](#)
- DDSC_LOG_TOPIC_TOO_LONG_EC
 - DDS_C, [151](#)
- DDSC_LOG_TOPICQOS_OBJECT
 - DDS_C, [136](#)
- DDSC_LOG_TRANSPORT_COMPONENT
 - DDS_C, [141](#)
- DDSC_LOG_TRANSPORT_QOS_POLICY
 - DDS_C, [143](#)
- DDSC_LOG_TYPE_FUNCTION_COPY_SAMPLE
 - DDS_C, [151](#)
- DDSC_LOG_TYPE_FUNCTION_CREATE_SAMPLE
 - DDS_C, [151](#)
- DDSC_LOG_TYPE_FUNCTION_DELETE_SAMPLE
 - DDS_C, [151](#)
- DDSC_LOG_TYPE_FUNCTION_DESERIALIZE_DATA
 - DDS_C, [150](#)
- DDSC_LOG_TYPE_FUNCTION_GET_KEY_KIND
 - DDS_C, [151](#)
- DDSC_LOG_TYPE_FUNCTION_GET_SERIALIZED_-
SAMPLE_MAX_SIZE
 - DDS_C, [150](#)
- DDSC_LOG_TYPE_FUNCTION_INSTANCE_TO_KEYHASH
 - DDS_C, [151](#)
- DDSC_LOG_TYPE_FUNCTION_NULL_EC
 - DDS_C, [151](#)
- DDSC_LOG_TYPE_FUNCTION_SERIALIZE_DATA
 - DDS_C, [150](#)
- DDSC_LOG_TYPE_KEY_TYPE_EC
 - DDS_C, [151](#)
- DDSC_LOG_TYPE_NAME_CMP_EC
 - DDS_C, [150](#)
- DDSC_LOG_TYPE_OBJECT
 - DDS_C, [137](#)
- DDSC_LOG_TYPE_RECORD
 - DDS_C, [135](#)
- DDSC_LOG_TYPE_SUPPORT_QOS_POLICY
 - DDS_C, [143](#)
- DDSC_LOG_TYPE_TOO_LONG_EC
 - DDS_C, [151](#)
- DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_-
KEY_EC
 - DDS_C, [132](#)
- DDSC_LOG_UNKNOWN_REMOTE_PARTICIPANT_-
NAME_EC
 - DDS_C, [132](#)
- DDSC_LOG_USER_TRAFFIC_QOS_POLICY
 - DDS_C, [143](#)
- DDSC_LOG_USERMULTICAST_SEQUENCE
 - DDS_C, [139](#)
- DDSC_LOG_USERUNICAST_SEQUENCE
 - DDS_C, [139](#)
- DDSC_LOG_UUID_OBJECT
 - DDS_C, [138](#)
- DDSC_LOG_WAITSET_OBJECT
 - DDS_C, [138](#)
- DDSC_LOG_WRITERHISTORY_COMPONENT
 - DDS_C, [140](#)
- DDSC_LOG_WS_ADD_COND_REFERENCE_EC
 - DDS_C, [133](#)
- DDSC_LOG_WS_REMOVE_COND_REFERENCE_EC
 - DDS_C, [133](#)
- DDSC_LOG_WSREF_OBJECT
 - DDS_C, [137](#)
- DDSCondition, [309](#)
- DDSConditionSeq, [309](#)
- DDSDataReader, [310](#)
 - as_entity, [312](#)
 - get_instance_replaced_missed_status, [322](#)
 - get_listener, [313](#)
 - get_liveliness_changed_status, [320](#)
 - get_qos, [313](#)
 - get_requested_deadline_missed_status, [321](#)
 - get_requested_incompatible_qos_status, [322](#)
 - get_sample_lost_status, [321](#)
 - get_sample_rejected_status, [321](#)
 - get_subscriber, [312](#)
 - get_subscription_matched_status, [320](#)
 - get_topicdescription, [312](#)
 - lookup_instance_untyped, [319](#)
 - read_instance_untyped, [317](#)
 - read_next_sample_untyped, [318](#)
 - read_untyped, [313](#)
 - return_loan_untyped, [319](#)
 - set_listener, [313](#)
 - set_qos, [312](#)
 - take_instance_untyped, [317](#)
 - take_next_sample_untyped, [318](#)
 - take_untyped, [314](#)
- DDSDataReaderListener, [322](#)
 - on_before_sample_commit, [325](#)
 - on_before_sample_deserialize, [324](#)
 - on_data_available, [323](#)
 - on_instance_replaced, [324](#)
 - on_liveliness_changed, [324](#)
 - on_requested_deadline_missed, [323](#)
 - on_requested_incompatible_qos, [324](#)
 - on_sample_lost, [324](#)
 - on_sample_rejected, [324](#)
 - on_subscription_matched, [324](#)
- DDSDataWriter, [325](#)
 - assert_liveliness, [328](#)
 - dispose_untyped, [333](#)
 - dispose_w_timestamp_untyped, [334](#)
 - get_listener, [330](#)
 - get_liveliness_lost_status, [336](#)
 - get_offered_deadline_missed_status, [336](#)
 - get_offered_incompatible_qos_status, [337](#)
 - get_publication_matched_status, [336](#)
 - get_publisher, [328](#)
 - get_qos, [329](#)
 - get_topic, [328](#)
 - register_instance_untyped, [330](#)

- register_instance_w_timestamp_untyped, 331
- set_listener, 329
- set_qos, 329
- unregister_instance_untyped, 331
- unregister_instance_w_timestamp_untyped, 332
- write_untyped, 327
- write_w_timestamp_untyped, 335
- DDSDDataWriterListener, 337
 - on_liveliness_lost, 338
 - on_offered_deadline_missed, 338
 - on_offered_incompatible_qos, 338
 - on_publication_matched, 339
 - on_reliable_reader_activity_changed, 339
 - on_reliable_sample_unacknowledged, 339
- DDSDDomainEntity, 340
- DDSDDomainParticipant, 340
 - add_peer, 353
 - as_entity, 342
 - create_publisher, 342
 - create_subscriber, 343
 - create_topic, 344
 - delete_contained_entities, 350
 - delete_publisher, 343
 - delete_subscriber, 344
 - delete_topic, 345
 - find_topic, 346
 - get_current_time, 353
 - get_default_publisher_qos, 347
 - get_default_subscriber_qos, 348
 - get_default_topic_qos, 349
 - get_listener, 353
 - get_qos, 346
 - lookup_topicdescription, 350
 - register_type, 351
 - set_default_publisher_qos, 347
 - set_default_subscriber_qos, 348
 - set_default_topic_qos, 349
 - set_listener, 352
 - set_qos, 346
 - unregister_type, 351
- DDSDDomainParticipantFactory, 354
 - create_participant, 356
 - delete_participant, 357
 - finalize_instance, 356
 - get_default_participant_qos, 358
 - get_instance, 355
 - get_qos, 358
 - get_registry, 359
 - lookup_participant, 357
 - set_default_participant_qos, 358
 - set_qos, 359
- DDSDomainParticipantListener, 360
 - on_before_sample_commit, 362
 - on_before_sample_deserialize, 362
 - on_data_available, 361
 - on_data_on_readers, 363
 - on_inconsistent_topic, 364
 - on_instance_replaced, 362
 - on_liveliness_changed, 361
 - on_liveliness_lost, 363
 - on_offered_deadline_missed, 363
 - on_offered_incompatible_qos, 363
 - on_publication_matched, 364
 - on_reliable_reader_activity_changed, 364
 - on_requested_deadline_missed, 361
 - on_requested_incompatible_qos, 361
 - on_sample_lost, 362
 - on_sample_rejected, 361
 - on_subscription_matched, 361
- DDSEntity, 364
 - enable, 366
 - get_instance_handle, 368
 - get_status_changes, 367
 - get_statuscondition, 367
- DDSGuardCondition, 368
 - set_trigger_value, 368
- DDSLListener, 369
- DDSPublisher, 371
 - create_datawriter, 372
 - delete_contained_entities, 375
 - delete_datawriter, 373
 - get_default_datawriter_qos, 373
 - get_listener, 376
 - get_participant, 374
 - get_qos, 375
 - lookup_datawriter, 374
 - set_default_datawriter_qos, 374
 - set_listener, 376
 - set_qos, 375
- DDSPublisherListener, 376
- DDSStatusCondition, 377
 - get_enabled_statuses, 378
 - set_enabled_statuses, 378
- DDSSubscriber, 378
 - create_datareader, 380
 - delete_contained_entities, 382
 - delete_datareader, 380
 - get_default_datareader_qos, 381
 - get_listener, 384
 - get_participant, 383
 - get_qos, 383
 - lookup_datareader, 382
 - set_default_datareader_qos, 381
 - set_listener, 384
 - set_qos, 383
- DDSSubscriberListener, 384
 - on_data_on_readers, 385
- DDSTheParticipantFactory
 - Domain Module, 16
- DDSTopic, 385
 - as_entity, 386
 - as_topicdescription, 386
 - get_inconsistent_topic_status, 387
 - get_listener, 388
 - get_name, 388
 - get_participant, 389

- get_qos, 387
- get_type_name, 388
- narrow, 386
- set_listener, 388
- set_qos, 387
- DDSTopicDescription, 389
 - get_name, 390
 - get_participant, 390
 - get_type_name, 390
- DDSTopicListener, 391
 - on_inconsistent_topic, 391
- DDSWaitSet, 392
 - attach_condition, 394
 - detach_condition, 394
 - get_conditions, 394
 - wait, 393
- DEADLINE, 92
 - DDS_DEADLINE_QOS_POLICY_NAME, 92
- DESTINATION_ORDER, 102
 - DDS_BY_RECEPTION_TIMESTAMP_DESTINATIONORDER_QOS, 102
 - DDS_BY_SOURCE_TIMESTAMP_DESTINATIONORDER_QOS, 102
 - DDS_DestinationOrderQosPolicyKind, 102
- DISCOVERY, 76
 - DDS_DISCOVERY_QOS_POLICY_NAME, 76
- DOMAIN_PARTICIPANT_RESOURCE_LIMITS, 115
 - DDS_DOMAINPARTICIPANTRESOURCELIMITS_QOS_POLICY_NAME, 115
- DPDE, 224
 - DPDE_LOG_ADD_PEER_EC, 230
 - DPDE_LOG_ADVANCE_SN_EC, 228
 - DPDE_LOG_ALLOCATE_DPDE_EC, 227
 - DPDE_LOG_ANNOUNCE_WRITE_EC, 228
 - DPDE_LOG_ANNOUNCEMENT_EC, 228
 - DPDE_LOG_ASSERT_REMOTE_PARTICIPANT_EC, 228
 - DPDE_LOG_ASSERT_REMOTE_PUBLICATION_EC, 229
 - DPDE_LOG_ASSERT_REMOTE_SUBSCRIPTION_EC, 229
 - DPDE_LOG_CDR_SET_OFFSET_EC, 226
 - DPDE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC, 230
 - DPDE_LOG_CREATE_PARTICIPANT_DISCOVERY_EC, 227
 - DPDE_LOG_CREATE_PUBLICATION_DISCOVERY_EC, 227
 - DPDE_LOG_CREATE_PUBLISHER_EC, 227
 - DPDE_LOG_CREATE_READER_EC, 228
 - DPDE_LOG_CREATE_SUBSCRIBER_EC, 227
 - DPDE_LOG_CREATE_SUBSCRIPTION_DISCOVERY_EC, 227
 - DPDE_LOG_CREATE_TOPIC_EC, 228
 - DPDE_LOG_CREATE_WRITER_EC, 228
 - DPDE_LOG_DELETE_PARTICIPANT_TOPIC_EC, 227
 - DPDE_LOG_DELETE_PUBLICATION_TOPIC_EC, 227
 - DPDE_LOG_DELETE_SUBSCRIPTION_TOPIC_EC, 227
 - DPDE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC, 228
 - DPDE_LOG_DESERIALIZE_LOCATOR_EC, 230
 - DPDE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC, 227
 - DPDE_LOG_DESERIALIZE_UNKNOWN_PID_EC, 228
 - DPDE_LOG_DESERIALIZE_VENDOR_ID_EC, 227
 - DPDE_LOG_DISPOSE_PARTICIPANT_EC, 227
 - DPDE_LOG_DISPOSE_PUBLICATION_EC, 229
 - DPDE_LOG_DISPOSE_SUBSCRIPTION_EC, 229
 - DPDE_LOG_ENABLE_REMOTE_PARTICIPANT_EC, 228
 - DPDE_LOG_INVALID_PARTICIPANT_SAMPLE_EC, 229
 - DPDE_LOG_INVALID_PUBLICATION_SAMPLE_EC, 229
 - DPDE_LOG_INVALID_SUBSCRIPTION_SAMPLE_EC, 229
 - DPDE_LOG_LOCATORS_FULL_EC, 230
 - DPDE_LOG_REFRESH_REMOTE_PARTICIPANT_EC, 228
 - DPDE_LOG_REGISTER_TYPE_EC, 228
 - DPDE_LOG_REMOVE_PEER_EC, 230
 - DPDE_LOG_REMOVE_REMOTE_PUBLICATION_EC, 229
 - DPDE_LOG_REMOVE_REMOTE_SUBSCRIPTION_EC, 229
 - DPDE_LOG_RETURN_PARTICIPANT_SAMPLE_EC, 229
 - DPDE_LOG_RETURN_PUBLICATION_SAMPLE_EC, 229
 - DPDE_LOG_RETURN_SUBSCRIPTION_SAMPLE_EC, 229
 - DPDE_LOG_SCHEDULE_FAST_ASSERTION_EC, 228
 - DPDE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC, 228
 - DPDE_LOG_SERIALIZE_DEFAULT_UNICAST_EC, 226
 - DPDE_LOG_SERIALIZE_ENTITY_NAME_EC, 226
 - DPDE_LOG_SERIALIZE_GUID_EC, 226
 - DPDE_LOG_SERIALIZE_LEASE_DURATION_EC, 227
 - DPDE_LOG_SERIALIZE_PRESENTATION_EC, 230
 - DPDE_LOG_SERIALIZE_PRODUCT_VERSION_EC, 227
 - DPDE_LOG_SERIALIZE_PROTOCOL_VERSION_EC, 226
 - DPDE_LOG_SERIALIZE_TOPIC_NAME_EC, 226

- DPDE_LOG_SERIALIZE_TYPE_NAME_EC, [226](#)
- DPDE_LOG_SERIALIZE_VENDOR_ID_EC, [227](#)
- DPDE_LOG_TAKE_PARTICIPANT_SAMPLE_EC, [229](#)
- DPDE_LOG_TAKE_PUBLICATION_SAMPLE_EC, [229](#)
- DPDE_LOG_TAKE_SUBSCRIPTION_SAMPLE_EC, [229](#)
- DPDE_LOG_UNSUPPORTED_LOCATOR_EC, [230](#)
- DPDE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC, [228](#)
- DPDE Dynamic Discovery API, [54](#)
 - DPDE_DiscoveryPluginProperty_INITIALIZER, [54](#)
- DPDE_DiscoveryPluginProperty, [395](#)
 - builtin_endpoint_reader_nack_period, [398](#)
 - builtin_writer_heartbeat_period, [398](#)
 - builtin_writer_heartbeats_per_max_samples, [398](#)
 - builtin_writer_max_heartbeat_retries, [398](#)
 - cache_serialized_samples, [396](#)
 - initial_participant_announcement_period, [396](#)
 - initial_participant_announcements, [396](#)
 - max_locators_per_discovered_participant, [397](#)
 - max_participant_locators, [396](#)
 - max_samples_per_builtin_endpoint_reader, [397](#)
 - max_samples_per_remote_builtin_endpoint_writer, [397](#)
 - participant_liveliness_assert_period, [396](#)
 - participant_liveliness_lease_duration, [396](#)
- DPDE_DiscoveryPluginProperty_INITIALIZER
 - DPDE Dynamic Discovery API, [54](#)
- DPDE_LOG_ADD_PEER_EC
 - DPDE, [230](#)
- DPDE_LOG_ADVANCE_SN_EC
 - DPDE, [228](#)
- DPDE_LOG_ALLOCATE_DPDE_EC
 - DPDE, [227](#)
- DPDE_LOG_ANNOUNCE_WRITE_EC
 - DPDE, [228](#)
- DPDE_LOG_ANNOUNCEMENT_EC
 - DPDE, [228](#)
- DPDE_LOG_ASSERT_REMOTE_PARTICIPANT_EC
 - DPDE, [228](#)
- DPDE_LOG_ASSERT_REMOTE_PUBLICATION_EC
 - DPDE, [229](#)
- DPDE_LOG_ASSERT_REMOTE_SUBSCRIPTION_EC
 - DPDE, [229](#)
- DPDE_LOG_CDR_SET_OFFSET_EC
 - DPDE, [226](#)
- DPDE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC
 - DPDE, [230](#)
- DPDE_LOG_CREATE_PARTICIPANT_DISCOVERY_EC
 - DPDE, [227](#)
- DPDE_LOG_CREATE_PUBLICATION_DISCOVERY_EC
 - DPDE, [227](#)
- DPDE_LOG_CREATE_PUBLISHER_EC
 - DPDE, [227](#)
- DPDE_LOG_CREATE_READER_EC
 - DPDE, [228](#)
- DPDE_LOG_CREATE_SUBSCRIBER_EC
 - DPDE, [227](#)
- DPDE_LOG_CREATE_SUBSCRIPTION_DISCOVERY_EC
 - DPDE, [227](#)
- DPDE_LOG_CREATE_TOPIC_EC
 - DPDE, [228](#)
- DPDE_LOG_CREATE_WRITER_EC
 - DPDE, [228](#)
- DPDE_LOG_DELETE_PARTICIPANT_TOPIC_EC
 - DPDE, [227](#)
- DPDE_LOG_DELETE_PUBLICATION_TOPIC_EC
 - DPDE, [227](#)
- DPDE_LOG_DELETE_SUBSCRIPTION_TOPIC_EC
 - DPDE, [227](#)
- DPDE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC
 - DPDE, [228](#)
- DPDE_LOG_DESERIALIZE_LOCATOR_EC
 - DPDE, [230](#)
- DPDE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC
 - DPDE, [227](#)
- DPDE_LOG_DESERIALIZE_UNKNOWN_PID_EC
 - DPDE, [228](#)
- DPDE_LOG_DESERIALIZE_VENDOR_ID_EC
 - DPDE, [227](#)
- DPDE_LOG_DISPOSE_PARTICIPANT_EC
 - DPDE, [227](#)
- DPDE_LOG_DISPOSE_PUBLICATION_EC
 - DPDE, [229](#)
- DPDE_LOG_DISPOSE_SUBSCRIPTION_EC
 - DPDE, [229](#)
- DPDE_LOG_ENABLE_REMOTE_PARTICIPANT_EC
 - DPDE, [228](#)
- DPDE_LOG_INVALID_PARTICIPANT_SAMPLE_EC
 - DPDE, [229](#)
- DPDE_LOG_INVALID_PUBLICATION_SAMPLE_EC
 - DPDE, [229](#)
- DPDE_LOG_INVALID_SUBSCRIPTION_SAMPLE_EC
 - DPDE, [229](#)
- DPDE_LOG_LOCATORS_FULL_EC
 - DPDE, [230](#)
- DPDE_LOG_REFRESH_REMOTE_PARTICIPANT_EC
 - DPDE, [228](#)
- DPDE_LOG_REGISTER_TYPE_EC
 - DPDE, [228](#)
- DPDE_LOG_REMOVE_PEER_EC
 - DPDE, [230](#)
- DPDE_LOG_REMOVE_REMOTE_PUBLICATION_EC
 - DPDE, [229](#)
- DPDE_LOG_REMOVE_REMOTE_SUBSCRIPTION_EC
 - DPDE, [229](#)

- DPDE, [229](#)
- DPDE_LOG_RETURN_PARTICIPANT_SAMPLE_EC
 - DPDE, [229](#)
- DPDE_LOG_RETURN_PUBLICATION_SAMPLE_EC
 - DPDE, [229](#)
- DPDE_LOG_RETURN_SUBSCRIPTION_SAMPLE_EC
 - DPDE, [229](#)
- DPDE_LOG_SCHEDULE_FAST_ASSERTION_EC
 - DPDE, [228](#)
- DPDE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC
 - DPDE, [228](#)
- DPDE_LOG_SERIALIZE_DEFAULT_UNICAST_EC
 - DPDE, [226](#)
- DPDE_LOG_SERIALIZE_ENTITY_NAME_EC
 - DPDE, [226](#)
- DPDE_LOG_SERIALIZE_GUID_EC
 - DPDE, [226](#)
- DPDE_LOG_SERIALIZE_LEASE_DURATION_EC
 - DPDE, [227](#)
- DPDE_LOG_SERIALIZE_PRESENTATION_EC
 - DPDE, [230](#)
- DPDE_LOG_SERIALIZE_PRODUCT_VERSION_EC
 - DPDE, [227](#)
- DPDE_LOG_SERIALIZE_PROTOCOL_VERSION_EC
 - DPDE, [226](#)
- DPDE_LOG_SERIALIZE_TOPIC_NAME_EC
 - DPDE, [226](#)
- DPDE_LOG_SERIALIZE_TYPE_NAME_EC
 - DPDE, [226](#)
- DPDE_LOG_SERIALIZE_VENDOR_ID_EC
 - DPDE, [227](#)
- DPDE_LOG_TAKE_PARTICIPANT_SAMPLE_EC
 - DPDE, [229](#)
- DPDE_LOG_TAKE_PUBLICATION_SAMPLE_EC
 - DPDE, [229](#)
- DPDE_LOG_TAKE_SUBSCRIPTION_SAMPLE_EC
 - DPDE, [229](#)
- DPDE_LOG_UNSUPPORTED_LOCATOR_EC
 - DPDE, [230](#)
- DPDE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC
 - DPDE, [228](#)
- DPDEDiscoveryFactory, [398](#)
- get_interface, [398](#)
- DPSE, [216](#)
 - DPSE_LOG_ADD_ANONYMOUS_ROUTE_EC, [222](#)
 - DPSE_LOG_ADVANCE_SN_EC, [221](#)
 - DPSE_LOG_ANNOUNCE_LOCAL_PARTICIPANT_DELETION_EC, [219](#)
 - DPSE_LOG_ANNOUNCE_WRITE_EC, [221](#)
 - DPSE_LOG_ANNOUNCEMENT_EC, [221](#)
 - DPSE_LOG_ASSERT_REMOTE_PARTICIPANT_EC, [221](#)
 - DPSE_LOG_CDR_DESERIALIZE_EC, [220](#)
 - DPSE_LOG_CDR_SERIALIZE_EC, [220](#)
 - DPSE_LOG_CDR_SET_POSITION_EC, [220](#)
 - DPSE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC, [223](#)
 - DPSE_LOG_CREATE_DISCOVERY_PUBLISHER_EC, [218](#)
 - DPSE_LOG_CREATE_DISCOVERY_SUBSCRIBER_EC, [219](#)
 - DPSE_LOG_CREATE_READER_EC, [219](#)
 - DPSE_LOG_CREATE_TOPIC_EC, [219](#)
 - DPSE_LOG_CREATE_WRITER_EC, [219](#)
 - DPSE_LOG_DELETE_ANONYMOUS_ROUTE_EC, [222](#)
 - DPSE_LOG_DELETE_PUBLISHER_EC, [219](#)
 - DPSE_LOG_DELETE_READER_EC, [219](#)
 - DPSE_LOG_DELETE_SUBSCRIBER_EC, [219](#)
 - DPSE_LOG_DELETE_TOPIC_EC, [219](#)
 - DPSE_LOG_DELETE_WRITER_EC, [219](#)
 - DPSE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC, [221](#)
 - DPSE_LOG_DESERIALIZE_GUID_EC, [221](#)
 - DPSE_LOG_DESERIALIZE_PARTICIPANT_NAME_EC, [221](#)
 - DPSE_LOG_DESERIALIZE_PRODUCT_VERSION_EC, [220](#)
 - DPSE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC, [221](#)
 - DPSE_LOG_DESERIALIZE_TOO_MANY_LOCATORS_EC, [221](#)
 - DPSE_LOG_DESERIALIZE_UNKNOWN_PID_EC, [221](#)
 - DPSE_LOG_DESERIALIZE_VENDOR_ID_EC, [221](#)
 - DPSE_LOG_DISPOSE_EC, [219](#)
 - DPSE_LOG_ENTITY_ENABLE_EC, [222](#)
 - DPSE_LOG_GET_DDS_PROPERTIES_EC, [222](#)
 - DPSE_LOG_GET_TIMER_EC, [223](#)
 - DPSE_LOG_INVALID_DISCOVERY_SAMPLE_EC, [222](#)
 - DPSE_LOG_INVALID_PEER_ADDRESS_EC, [218](#)
 - DPSE_LOG_MAX_REMOTE_PARTICIPANT_EC, [222](#)
 - DPSE_LOG_OBJECT_ALLOCATE_EC, [219](#)
 - DPSE_LOG_OBJECT_DELETE_EC, [220](#)
 - DPSE_LOG_OBJECT_FINALIZE_EC, [219](#)
 - DPSE_LOG_OBJECT_INITIALIZE_EC, [219](#)
 - DPSE_LOG_OBJECT_INVALID_EC, [220](#)
 - DPSE_LOG_ON_ASSERT_REMOTE_PARTICIPANT_EC, [222](#)
 - DPSE_LOG_PARTICIPANT_TAKE_EC, [221](#)
 - DPSE_LOG_REFRESH_REMOTE_PARTICIPANT_EC, [222](#)
 - DPSE_LOG_REGISTER_TYPE_EC, [219](#)
 - DPSE_LOG_RESET_REMOTE_PARTICIPANT_EC, [222](#)
 - DPSE_LOG_RETURN_DISCOVERY_SAMPLE_EC, [222](#)
 - DPSE_LOG_SCHEDULE_FAST_ASSERTION_EC, [221](#)

- DPSE_LOG_SEQUENCE_COPY_EC, [223](#)
- DPSE_LOG_SEQUENCE_FINALIZE_EC, [223](#)
- DPSE_LOG_SEQUENCE_GETREF_EC, [222](#)
- DPSE_LOG_SEQUENCE_INITIALIZE_EC, [222](#)
- DPSE_LOG_SEQUENCE_SETLENGTH_EC, [222](#)
- DPSE_LOG_SEQUENCE_SETMAX_EC, [222](#)
- DPSE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC, [220](#)
- DPSE_LOG_SERIALIZE_DEFAULT_UNICAST_EC, [220](#)
- DPSE_LOG_SERIALIZE_GUID_EC, [220](#)
- DPSE_LOG_SERIALIZE_LEASE_DURATION_EC, [220](#)
- DPSE_LOG_SERIALIZE_META_MULTICAST_EC, [220](#)
- DPSE_LOG_SERIALIZE_META_UNICAST_EC, [220](#)
- DPSE_LOG_SERIALIZE_MULTICAST_EC, [222](#)
- DPSE_LOG_SERIALIZE_PARTICIPANT_NAME_EC, [221](#)
- DPSE_LOG_SERIALIZE_PRODUCT_VERSION_EC, [220](#)
- DPSE_LOG_SERIALIZE_PROTOCOL_VERSION_EC, [220](#)
- DPSE_LOG_SERIALIZE_VENDOR_ID_EC, [220](#)
- DPSE_LOG_UNSUPPORTED_LOCATOR_EC, [223](#)
- DPSE_LOG_UPDATE_PARTICIPANT_ASSERT_PERIOD_EC, [221](#)
- DPSE Static Discovery API, [53](#)
 - DPSE_DiscoveryPluginProperty_INITIALIZER, [53](#)
- DPSE_DISCOVERYPLUGIN_OBJECT
 - disc_dpse_log.h, [471](#)
- DPSE_DiscoveryPluginProperty, [399](#)
 - initial_participant_announcement_period, [400](#)
 - initial_participant_announcements, [399](#)
 - max_locators_per_discovered_participant, [400](#)
 - max_participant_locators, [400](#)
 - participant_liveliness_assert_period, [399](#)
 - participant_liveliness_lease_duration, [399](#)
- DPSE_DiscoveryPluginProperty_INITIALIZER
 - DPSE Static Discovery API, [53](#)
- DPSE_ENABLEDPARTICIPANTINDEX_OBJECT
 - disc_dpse_log.h, [471](#)
- DPSE_KEYLIST_OBJECT
 - disc_dpse_log.h, [471](#)
- DPSE_LOG_ADD_ANONYMOUS_ROUTE_EC
 - DPSE, [222](#)
- DPSE_LOG_ADVANCE_SN_EC
 - DPSE, [221](#)
- DPSE_LOG_ANNOUNCE_LOCAL_PARTICIPANT_DELETION_EC
 - DPSE, [219](#)
- DPSE_LOG_ANNOUNCE_WRITE_EC
 - DPSE, [221](#)
- DPSE_LOG_ANNOUNCEMENT_EC
 - DPSE, [221](#)
- DPSE_LOG_ASSERT_REMOTE_PARTICIPANT_EC
 - DPSE, [221](#)
- DPSE, [221](#)
- DPSE_LOG_CDR_DESERIALIZE_EC
 - DPSE, [220](#)
- DPSE_LOG_CDR_LONG_KIND
 - disc_dpse_log.h, [471](#)
- DPSE_LOG_CDR_PID_KIND
 - disc_dpse_log.h, [471](#)
- DPSE_LOG_CDR_SERIALIZE_EC
 - DPSE, [220](#)
- DPSE_LOG_CDR_SET_POSITION_EC
 - DPSE, [220](#)
- DPSE_LOG_CHECKSUM_INCOMPATIBLE_PARTICIPANT_IGNORED_EC
 - DPSE, [223](#)
- DPSE_LOG_CREATE_DISCOVERY_PUBLISHER_EC
 - DPSE, [218](#)
- DPSE_LOG_CREATE_DISCOVERY_SUBSCRIBER_EC
 - DPSE, [219](#)
- DPSE_LOG_CREATE_READER_EC
 - DPSE, [219](#)
- DPSE_LOG_CREATE_TOPIC_EC
 - DPSE, [219](#)
- DPSE_LOG_CREATE_WRITER_EC
 - DPSE, [219](#)
- DPSE_LOG_DEFAULT_MULTICAST_LOCATOR_SEQUENCE
 - disc_dpse_log.h, [472](#)
- DPSE_LOG_DEFAULT_UNICAST_LOCATOR_SEQUENCE
 - disc_dpse_log.h, [472](#)
- DPSE_LOG_DELETE_ANONYMOUS_ROUTE_EC
 - DPSE, [222](#)
- DPSE_LOG_DELETE_PUBLISHER_EC
 - DPSE, [219](#)
- DPSE_LOG_DELETE_READER_EC
 - DPSE, [219](#)
- DPSE_LOG_DELETE_SUBSCRIBER_EC
 - DPSE, [219](#)
- DPSE_LOG_DELETE_TOPIC_EC
 - DPSE, [219](#)
- DPSE_LOG_DELETE_WRITER_EC
 - DPSE, [219](#)
- DPSE_LOG_DESERIALIZE_BUILTIN_ENDPOINTS_EC
 - DPSE, [221](#)
- DPSE_LOG_DESERIALIZE_GUID_EC
 - DPSE, [221](#)
- DPSE_LOG_DESERIALIZE_PARTICIPANT_NAME_EC
 - DPSE, [221](#)
- DPSE_LOG_DESERIALIZE_PRODUCT_VERSION_EC
 - DPSE, [220](#)
- DPSE_LOG_DESERIALIZE_PROTOCOL_VERSION_EC
 - DPSE, [221](#)

- DPSE_LOG_DESERIALIZE_TOO_MANY_LOCATOR-
S_EC
DPSE, [221](#)
- DPSE_LOG_DESERIALIZE_UNKNOWN_PID_EC
DPSE, [221](#)
- DPSE_LOG_DESERIALIZE_VENDOR_ID_EC
DPSE, [221](#)
- DPSE_LOG_DISPOSE_EC
DPSE, [219](#)
- DPSE_LOG_ENTITY_ENABLE_EC
DPSE, [222](#)
- DPSE_LOG_GET_DDS_PROPERTIES_EC
DPSE, [222](#)
- DPSE_LOG_GET_TIMER_EC
DPSE, [223](#)
- DPSE_LOG_INVALID_DISCOVERY_SAMPLE_EC
DPSE, [222](#)
- DPSE_LOG_INVALID_PEER_ADDRESS_EC
DPSE, [218](#)
- DPSE_LOG_MAX_REMOTE_PARTICIPANT_EC
DPSE, [222](#)
- DPSE_LOG_META_MULTICAST_LOCATOR_SEQUE-
NCE
disc_dpse_log.h, [472](#)
- DPSE_LOG_META_UNICAST_LOCATOR_SEQUEN-
CE
disc_dpse_log.h, [472](#)
- DPSE_LOG_OBJECT_ALLOCATE_EC
DPSE, [219](#)
- DPSE_LOG_OBJECT_DELETE_EC
DPSE, [220](#)
- DPSE_LOG_OBJECT_FINALIZE_EC
DPSE, [219](#)
- DPSE_LOG_OBJECT_INITIALIZE_EC
DPSE, [219](#)
- DPSE_LOG_OBJECT_INVALID_EC
DPSE, [220](#)
- DPSE_LOG_ON_ASSERT_REMOTE_PARTICIPANT-
_EC
DPSE, [222](#)
- DPSE_LOG_PARTICIPANT_TAKE_EC
DPSE, [221](#)
- DPSE_LOG_REFRESH_REMOTE_PARTICIPANT_EC
DPSE, [222](#)
- DPSE_LOG_REGISTER_TYPE_EC
DPSE, [219](#)
- DPSE_LOG_RESET_REMOTE_PARTICIPANT_EC
DPSE, [222](#)
- DPSE_LOG_RETURN_DISCOVERY_SAMPLE_EC
DPSE, [222](#)
- DPSE_LOG_SCHEDULE_FAST_ASSERTION_EC
DPSE, [221](#)
- DPSE_LOG_SEQUENCE_COPY_EC
DPSE, [223](#)
- DPSE_LOG_SEQUENCE_FINALIZE_EC
DPSE, [223](#)
- DPSE_LOG_SEQUENCE_GETREF_EC
DPSE, [222](#)
- DPSE_LOG_SEQUENCE_INITIALIZE_EC
DPSE, [222](#)
- DPSE_LOG_SEQUENCE_SETLENGTH_EC
DPSE, [222](#)
- DPSE_LOG_SEQUENCE_SETMAX_EC
DPSE, [222](#)
- DPSE_LOG_SERIALIZE_BUILTIN_ENDPOINTS_EC
DPSE, [220](#)
- DPSE_LOG_SERIALIZE_DEFAULT_UNICAST_EC
DPSE, [220](#)
- DPSE_LOG_SERIALIZE_GUID_EC
DPSE, [220](#)
- DPSE_LOG_SERIALIZE_LEASE_DURATION_EC
DPSE, [220](#)
- DPSE_LOG_SERIALIZE_META_MULTICAST_EC
DPSE, [220](#)
- DPSE_LOG_SERIALIZE_META_UNICAST_EC
DPSE, [220](#)
- DPSE_LOG_SERIALIZE_MULTICAST_EC
DPSE, [222](#)
- DPSE_LOG_SERIALIZE_PARTICIPANT_NAME_EC
DPSE, [221](#)
- DPSE_LOG_SERIALIZE_PRODUCT_VERSION_EC
DPSE, [220](#)
- DPSE_LOG_SERIALIZE_PROTOCOL_VERSION_EC
DPSE, [220](#)
- DPSE_LOG_SERIALIZE_VENDOR_ID_EC
DPSE, [220](#)
- DPSE_LOG_UNSUPPORTED_LOCATOR_EC
DPSE, [223](#)
- DPSE_LOG_UPDATE_PARTICIPANT_ASSERT_PER-
IOD_EC
DPSE, [221](#)
- DPSE_PARTICIPANTANNOUCEMENT_OBJECT
disc_dpse_log.h, [471](#)
- DPSE_PARTICIPANTBUILTINTOPICDATA_OBJECT
disc_dpse_log.h, [471](#)
- DPSE_PARTICIPANTENTITYNAME_OBJECT
disc_dpse_log.h, [471](#)
- DPSE_PARTICIPANTMAMES_OBJECT
disc_dpse_log.h, [471](#)
- DPSE_PARTICIPANTNAMEINDEX_OBJECT
disc_dpse_log.h, [471](#)
- DPSE_PARTICIPANTPOOL_OBJECT
disc_dpse_log.h, [471](#)
- DPSE_PUBLISHER_ENTITY
disc_dpse_log.h, [471](#)
- DPSE_SUBSCRIBER_ENTITY
disc_dpse_log.h, [472](#)
- DPSE_TOPIC_ENTITY
disc_dpse_log.h, [471](#)
- DPSEDiscoveryFactory, [400](#)
get_interface, [401](#)
- DPSEDiscoveryPlugin, [401](#)
RemoteParticipant_assert, [402](#)
RemotePublication_assert, [402](#)
RemoteSubscription_assert, [402](#)
- DURABILITY, [99](#)

- DDS_DURABILITY_QOS_POLICY_NAME, 99
- DDS_DurabilityQosPolicyKind, 99
- DDS_TRANSIENT_LOCAL_DURABILITY_QOS, 99
- DDS_VOLATILE_DURABILITY_QOS, 99
- Data Sample, 40
- DataReader, 37
 - DDS_NOT_REJECTED, 38
 - DDS_REJECTED_BY_INSTANCES_LIMIT, 38
 - DDS_REJECTED_BY_REMOTE_WRITERS_LIMIT, 38
 - DDS_REJECTED_BY_SAMPLES_LIMIT, 38
 - DDS_REJECTED_BY_SAMPLES_PER_INSTANCE_LIMIT, 38
 - DDS_REJECTED_BY_SAMPLES_PER_REMOTE_WRITER_LIMIT, 38
 - DDS_SAMPLE_LOST_BY_DATAWRITER, 38
 - DDS_SAMPLE_LOST_BY_HISTORY_DEPTH_LIMIT, 39
 - DDS_SAMPLE_LOST_BY_MAX_SAMPLES_LIMIT, 38
 - DDS_SAMPLE_LOST_BY_META_SAMPLE_LIMIT, 39
 - DDS_SAMPLE_LOST_BY_NOT_READ_ON_CACHE_DELETION, 39
 - DDS_SAMPLE_LOST_NOT_LOST, 38
 - DDS_SampleLostStatusKind, 38
 - DDS_SampleRejectedStatusKind, 38
- DataReader Resource Limits, 103
 - DDS_DATAREADERRESOURCELIMITS_QOS_POLICY_NAME, 103
 - DDS_DataReaderResourceLimitsInstanceReplacementKind, 103
 - DDS_NO_INSTANCE_REPLACEMENT_QOS, 103
 - DDS_REPLACE_OLDEST_INSTANCE_REPLACEMENT_QOS, 103
- DataWriter, 32
 - DDS_DataWriterListener_ReliableReaderActivityChangedCallback, 33
 - DDS_DataWriterListener_ReliableSampleUnacknowledgedCallback, 33
 - DDS_ReliableReaderActivityChangedStatus_INITIALIZER, 33
- DataWriter Resource Limits, 104
 - DDS_DATAWRITERRESOURCELIMITS_QOS_POLICY_NAME, 104
- db_log.h, 451
- dds_builtin_endpoints
 - DDS_ParticipantBuiltinTopicData, 273
- dds_c_config.h, 452
- dds_c_log.h, 452
- deadline
 - DDS_DataReaderQos, 241
 - DDS_DataWriterQos, 246
 - DDS_PublicationBuiltinTopicData, 279
 - DDS_SubscriptionBuiltinTopicData, 300
- default_multicast_locators
 - DDS_ParticipantBuiltinTopicData, 273
- default_unicast_locators
 - DDS_ParticipantBuiltinTopicData, 273
- delete_contained_entities
 - DDSDomainParticipant, 350
 - DDSPublisher, 375
 - DDSSubscriber, 382
- delete_data
 - FooTypeSupport, 431
- delete_datareader
 - DDSSubscriber, 380
- delete_datawriter
 - DDSPublisher, 373
- delete_participant
 - DDSDomainParticipantFactory, 357
- delete_publisher
 - DDSDomainParticipant, 343
- delete_subscriber
 - DDSDomainParticipant, 344
- delete_topic
 - DDSDomainParticipant, 345
- deny_interface
 - UDP_InterfaceFactoryProperty, 446
- depth
 - DDS_HistoryQosPolicy, 261
- destination_order
 - DDS_DataReaderQos, 241
 - DDS_DataWriterQos, 246
 - DDS_PublicationBuiltinTopicData, 280
 - DDS_SubscriptionBuiltinTopicData, 301
- destination_rules
 - UDP_InterfaceFactoryProperty, 447
- detach_condition
 - DDSWaitSet, 394
- disc_dpde_log.h, 465
- disc_dpse_log.h, 467
 - DPSE_DISCOVERYPLUGIN_OBJECT, 471
 - DPSE_ENABLEDPARTICIPANTINDEX_OBJECT, 471
 - DPSE_KEYLIST_OBJECT, 471
 - DPSE_LOG_CDR_LONG_KIND, 471
 - DPSE_LOG_CDR_PID_KIND, 471
 - DPSE_LOG_DEFAULT_MULTICAST_LOCATOR_SEQUENCE, 472
 - DPSE_LOG_DEFAULT_UNICAST_LOCATOR_SEQUENCE, 472
 - DPSE_LOG_META_MULTICAST_LOCATOR_SEQUENCE, 472
 - DPSE_LOG_META_UNICAST_LOCATOR_SEQUENCE, 472
 - DPSE_PARTICIPANTANNOUNCEMENT_OBJECT, 471
 - DPSE_PARTICIPANTBUILTINTOPICDATA_OBJECT, 471
 - DPSE_PARTICIPANTENTITYNAME_OBJECT, 471
 - DPSE_PARTICIPANTMAMES_OBJECT, 471
 - DPSE_PARTICIPANTNAMEINDEX_OBJECT, 471

- DPSE_PARTICIPANTPOOL_OBJECT, 471
- DPSE_PUBLISHER_ENTITY, 471
- DPSE_SUBSCRIBER_ENTITY, 472
- DPSE_TOPIC_ENTITY, 471
- Discovery, 21
 - DDS_BuiltinTopicKey_t, 24
 - DDS_LOCATOR_ADDRESS_INVALID, 23
 - DDS_LOCATOR_INVALID, 22
 - DDS_LOCATOR_KIND_INVALID, 23
 - DDS_LOCATOR_KIND_RESERVED, 23
 - DDS_LOCATOR_KIND_SHMEM, 23
 - DDS_LOCATOR_KIND_UDPv4, 23
 - DDS_LOCATOR_KIND_UDPv6, 23
 - DDS_LOCATOR_PORT_INVALID, 23
 - DDS_PROTOCOLVERSION, 23
 - DDS_PROTOCOLVERSION_1_0, 23
 - DDS_PROTOCOLVERSION_1_1, 23
 - DDS_PROTOCOLVERSION_1_2, 23
 - DDS_PROTOCOLVERSION_2_0, 23
 - DDS_PROTOCOLVERSION_2_1, 23
 - DDS_ProtocolVersion_t, 23
 - DDS_VENDOR_ID_LENGTH_MAX, 23
- discovery
 - DDS_DiscoveryQosPolicy, 251
 - DDS_DomainParticipantQos, 253
- dispose
 - FooDataWriter, 417
- dispose_untyped
 - DDSDDataWriter, 333
- dispose_w_timestamp
 - FooDataWriter, 418
- dispose_w_timestamp_untyped
 - DDSDDataWriter, 334
- Documentation Guide, 48
- Domain Module, 16
 - DDS_DomainId_t, 16
 - DDSTheParticipantFactory, 16
- domain_id_gain
 - DDS_RtpsWellKnownPorts_t, 292
- DomainParticipant, 19
 - DDS_PUBLISHER_QOS_DEFAULT, 19
 - DDS_SUBSCRIBER_QOS_DEFAULT, 19
 - DDS_TOPIC_QOS_DEFAULT, 19
- DomainParticipantFactory, 18
 - DDS_PARTICIPANT_FACTORY_QOS_DEFAULT, 18
 - DDS_PARTICIPANT_QOS_DEFAULT, 18
- durability
 - DDS_DataReaderQos, 241
 - DDS_DataWriterQos, 246
 - DDS_PublicationBuiltinTopicData, 280
 - DDS_SubscriptionBuiltinTopicData, 301
- Dynamic Participant Dynamic Endpoint (DPDE), 232
- ENTITY_FACTORY, 105
 - DDS_ENTITYFACTORY_QOS_POLICY_NAME, 105
- ENTITY_NAME, 118
 - DDS_AUTO_NAME_ENTITY, 118
 - DDS_ENTITYNAME_QOS_NAME_MAX, 118
 - DDS_ENTITYNAME_QOS_POLICY_NAME, 118
- enable
 - DDSEntity, 366
- enabled_transports
 - DDS_DiscoveryQosPolicy, 251
 - DDS_TransportQosPolicy, 304
 - DDS_UserTrafficQosPolicy, 305
- ensure_length
 - FooSeq, 426
- Entity Support, 117
- entity_factory
 - DDS_DomainParticipantFactoryQos, 252
 - DDS_DomainParticipantQos, 253
 - DDS_PublisherQos, 281
 - DDS_SubscriberQos, 299
- Extended Qos Support, 106
- finalize
 - OSAPI_SystemI, 439
- finalize_data
 - FooTypeSupport, 430
- finalize_instance
 - DDSDomainParticipantFactory, 356
- find_topic
 - DDSDomainParticipant, 346
- first_alarm
 - OSAPI_PortProperty, 437
- first_give_event
 - OSAPI_PortProperty, 437
- first_resource_id
 - OSAPI_PortProperty, 436
- first_timeout_event
 - OSAPI_PortProperty, 437
- flags
 - UDP_InterfaceTableEntry, 448
- FooDataReader, 403
 - narrow, 404
 - read, 406
 - read_instance, 410
 - read_next_sample, 405
 - return_loan, 404
 - take, 407
 - take_instance, 411
 - take_next_sample, 405
- FooDataWriter, 412
 - dispose, 417
 - dispose_w_timestamp, 418
 - narrow, 412
 - register_instance, 414
 - register_instance_w_timestamp, 414
 - unregister_instance, 415
 - unregister_instance_w_timestamp, 416
 - write, 413
 - write_w_timestamp, 419
- FooSeq, 420
 - ~FooSeq, 421
 - _contiguous_buffer, 429
 - _element_size, 429

- [_flags, 429](#)
- [_length, 429](#)
- [_maximum, 429](#)
- [_token1, 429](#)
- [_token2, 429](#)
- [copy_no_alloc, 423](#)
- [ensure_length, 426](#)
- [FooSeq, 421, 422](#)
- [FooSeq, 421, 422](#)
- [from_array, 423](#)
- [get_contiguous_buffer, 429](#)
- [has_ownership, 429](#)
- [length, 425](#)
- [loan_contiguous, 427](#)
- [max_alloc_size, 429](#)
- [maximum, 426, 427](#)
- [operator=, 422](#)
- [operator\[\], 424, 425](#)
- [to_array, 424](#)
- [unloan, 428](#)
- [FooTypeSupport, 430](#)
 - [copy_data, 432](#)
 - [create_data, 431](#)
 - [delete_data, 431](#)
 - [finalize_data, 430](#)
 - [initialize_data, 430](#)
 - [register_type, 432](#)
 - [unregister_type, 432](#)
- [from_array](#)
 - [FooSeq, 423](#)
- [GUID Support, 79](#)
 - [DDS_GUID_UNKNOWN, 79](#)
- [generate_uuid](#)
 - [OSAPI_SystemI, 440](#)
- [get_conditions](#)
 - [DDSWaitSet, 394](#)
- [get_contiguous_buffer](#)
 - [FooSeq, 429](#)
- [get_current_time](#)
 - [DDSDomainParticipant, 353](#)
- [get_default_datareader_qos](#)
 - [DDSSubscriber, 381](#)
- [get_default_datawriter_qos](#)
 - [DDSPublisher, 373](#)
- [get_default_participant_qos](#)
 - [DDSDomainParticipantFactory, 358](#)
- [get_default_publisher_qos](#)
 - [DDSDomainParticipant, 347](#)
- [get_default_subscriber_qos](#)
 - [DDSDomainParticipant, 348](#)
- [get_default_topic_qos](#)
 - [DDSDomainParticipant, 349](#)
- [get_enabled_statuses](#)
 - [DDSStatusCondition, 378](#)
- [get_hostname](#)
 - [OSAPI_SystemI, 440](#)
- [get_inconsistent_topic_status](#)
 - [DDSTopic, 387](#)
- [get_instance](#)
 - [DDSDomainParticipantFactory, 355](#)
- [get_instance_handle](#)
 - [DDSEntity, 368](#)
- [get_instance_replaced_missed_status](#)
 - [DDSDataReader, 322](#)
- [get_interface](#)
 - [DPEDiscoveryFactory, 398](#)
 - [DPSEDiscoveryFactory, 401](#)
 - [RHSMHistoryFactory, 442](#)
 - [UDPInterfaceFactory, 449](#)
 - [WHSMHistoryFactory, 450](#)
- [get_listener](#)
 - [DDSDataReader, 313](#)
 - [DDSDataWriter, 330](#)
 - [DDSDomainParticipant, 353](#)
 - [DDSPublisher, 376](#)
 - [DDSSubscriber, 384](#)
 - [DDSTopic, 388](#)
- [get_liveliness_changed_status](#)
 - [DDSDataReader, 320](#)
- [get_liveliness_lost_status](#)
 - [DDSDataWriter, 336](#)
- [get_name](#)
 - [DDSTopic, 388](#)
 - [DDSTopicDescription, 390](#)
- [get_offered_deadline_missed_status](#)
 - [DDSDataWriter, 336](#)
- [get_offered_incompatible_qos_status](#)
 - [DDSDataWriter, 337](#)
- [get_participant](#)
 - [DDSPublisher, 374](#)
 - [DDSSubscriber, 383](#)
 - [DDSTopic, 389](#)
 - [DDSTopicDescription, 390](#)
- [get_publication_matched_status](#)
 - [DDSDataWriter, 336](#)
- [get_publisher](#)
 - [DDSDataWriter, 328](#)
- [get_qos](#)
 - [DDSDataReader, 313](#)
 - [DDSDataWriter, 329](#)
 - [DDSDomainParticipant, 346](#)
 - [DDSDomainParticipantFactory, 358](#)
 - [DDSPublisher, 375](#)
 - [DDSSubscriber, 383](#)
 - [DDSTopic, 387](#)
- [get_registry](#)
 - [DDSDomainParticipantFactory, 359](#)
- [get_requested_deadline_missed_status](#)
 - [DDSDataReader, 321](#)
- [get_requested_incompatible_qos_status](#)
 - [DDSDataReader, 322](#)
- [get_sample_lost_status](#)
 - [DDSDataReader, 321](#)
- [get_sample_rejected_status](#)
 - [DDSDataReader, 321](#)
- [get_socket](#)

- OSAPI_PortProperty, 438
- get_status_changes
 - DDSEntity, 367
- get_statuscondition
 - DDSEntity, 367
- get_subscriber
 - DDSDataReader, 312
- get_subscription_matched_status
 - DDSDataReader, 320
- get_ticktime
 - OSAPI_SystemI, 440
- get_time
 - OSAPI_SystemI, 439
- get_timer_resolution
 - OSAPI_SystemI, 439
- get_topic
 - DDSDataWriter, 328
- get_topicdescription
 - DDSDataReader, 312
- get_type_name
 - DDSTopic, 388
 - DDSTopicDescription, 390
- HISTORY, 98
 - DDS_HISTORY_QOS_POLICY_NAME, 98
 - DDS_HistoryQosPolicyKind, 98
 - DDS_KEEP_LAST_HISTORY_QOS, 98
- has_ownership
 - FooSeq, 429
- heap_area
 - OSAPI_PortProperty, 436
- heap_area_size
 - OSAPI_PortProperty, 436
- heartbeat_period
 - DDS_RtpsReliableWriterProtocol_t, 290
- heartbeats_per_max_samples
 - DDS_RtpsReliableWriterProtocol_t, 290
- high
 - DDS_SequenceNumber_t, 299
- history
 - DDS_DataReaderQos, 241
 - DDS_DataWriterQos, 246
- hostname
 - OSAPI_SystemProperty, 440
- if_table
 - UDP_InterfaceFactoryProperty, 447
- ifname
 - UDP_InterfaceTableEntry, 448
- inactive_count
 - DDS_ReliableReaderActivityChangedStatus, 283
- inactive_count_change
 - DDS_ReliableReaderActivityChangedStatus, 284
- Infrastructure Module, 47
- initial_participant_announcement_period
 - DPDE_DiscoveryPluginProperty, 396
 - DPSE_DiscoveryPluginProperty, 400
- initial_participant_announcements
 - DPDE_DiscoveryPluginProperty, 396
- DPSE_DiscoveryPluginProperty, 399
- initial_peers
 - DDS_DiscoveryQosPolicy, 250
- initialize
 - OSAPI_SystemI, 439
- initialize_data
 - FooTypeSupport, 430
- Instance States, 45
 - DDS_ALIVE_INSTANCE_STATE, 46
 - DDS_ANY_INSTANCE_STATE, 46
 - DDS_InstanceStateKind, 45
 - DDS_InstanceStateMask, 45
 - DDS_NOT_ALIVE_DISPOSED_INSTANCE_STATE, 46
 - DDS_NOT_ALIVE_INSTANCE_STATE, 46
 - DDS_NOT_ALIVE_NO_WRITERS_INSTANCE_STATE, 46
- instance_handle
 - DDS_ReliableSampleUnacknowledgedStatus, 284
 - DDS_SampleInfo, 296
- instance_replacement
 - DDS_DataReaderResourceLimitsQosPolicy, 243
- instance_state
 - DDS_SampleInfo, 295
- is_anonymous
 - RTI_ManagementQosPolicy, 442
- is_default_interface
 - UDP_InterfaceFactoryProperty, 447
- key
 - DDS_ParticipantBuiltinTopicData, 273
 - DDS_PublicationBuiltinTopicData, 279
 - DDS_SubscriptionBuiltinTopicData, 300
- kind
 - DDS_DestinationOrderQosPolicy, 249
 - DDS_DurabilityQosPolicy, 257
 - DDS_HistoryQosPolicy, 261
 - DDS_LivelinessQosPolicy, 265
 - DDS_Locator_t, 266
 - DDS_OwnershipQosPolicy, 271
 - DDS_ReliabilityQosPolicy, 282
- LIVELINESS, 95
 - DDS_AUTOMATIC_LIVELINESS_QOS, 95
 - DDS_LIVELINESS_QOS_POLICY_NAME, 96
 - DDS_LivelinessQosPolicyKind, 95
 - DDS_MANUAL_BY_PARTICIPANT_LIVELINESS_QOS, 95
 - DDS_MANUAL_BY_TOPIC_LIVELINESS_QOS, 95
- last_instance_handle
 - DDS_DataReaderInstanceReplacedStatus, 239
 - DDS_OfferedDeadlineMissedStatus, 267
 - DDS_ReliableReaderActivityChangedStatus, 284
 - DDS_RequestedDeadlineMissedStatus, 285
 - DDS_SampleRejectedStatus, 298
- last_policy_id
 - DDS_OfferedIncompatibleQosStatus, 268
 - DDS_RequestedIncompatibleQosStatus, 286

- last_publication_handle
 - DDS_LivelinessChangedStatus, 262
 - DDS_SubscriptionMatchedStatus, 302
- last_reason
 - DDS_SampleRejectedStatus, 298
- last_replacement_instance
 - DDS_DataReaderInstanceReplacedStatus, 239
- last_resource_id
 - OSAPI_PortProperty, 436
- last_subscription_handle
 - DDS_PublicationMatchedStatus, 281
- lease_duration
 - DDS_LivelinessQosPolicy, 265
- length
 - FooSeq, 425
- liveliness
 - DDS_DataReaderQos, 241
 - DDS_DataWriterQos, 246
 - DDS_PublicationBuiltinTopicData, 280
 - DDS_SubscriptionBuiltinTopicData, 301
- liveliness_lease_duration
 - DDS_ParticipantBuiltinTopicData, 273
- loan_contiguous
 - FooSeq, 427
- local_publisher_allocation
 - DDS_DomainParticipantResourceLimitsQosPolicy, 254
- local_reader_allocation
 - DDS_DomainParticipantResourceLimitsQosPolicy, 254
- local_subscriber_allocation
 - DDS_DomainParticipantResourceLimitsQosPolicy, 255
- local_topic_allocation
 - DDS_DomainParticipantResourceLimitsQosPolicy, 255
- local_type_allocation
 - DDS_DomainParticipantResourceLimitsQosPolicy, 255
- local_writer_allocation
 - DDS_DomainParticipantResourceLimitsQosPolicy, 254
- Log API, 65
 - OSAPI_LOG_VERBOSITY_DEBUG, 66
 - OSAPI_LOG_VERBOSITY_ERROR, 66
 - OSAPI_LOG_VERBOSITY_SILENT, 66
 - OSAPI_LOG_VERBOSITY_WARNING, 66
 - OSAPI_Log_clear, 68
 - OSAPI_Log_get_display_handler, 68
 - OSAPI_Log_get_last_error_code, 69
 - OSAPI_Log_get_log_handler, 67
 - OSAPI_Log_get_property, 69
 - OSAPI_Log_get_trace_handler, 67
 - OSAPI_Log_set_display_handler, 68
 - OSAPI_Log_set_log_handler, 66
 - OSAPI_Log_set_property, 69
 - OSAPI_Log_set_trace_handler, 67
 - OSAPI_Log_set_verbosity, 68
 - OSAPI_Log_write_buffer_T, 66
 - OSAPI_LogHandler_T, 66
 - OSAPI_LogVerbosity_T, 66
 - OSAPI_Trace_set_trace_mask, 68
- Log Codes, 236
- log_detail
 - OSAPI_LogProperty, 434
- log_dst_address
 - OSAPI AutoSAR, 168
- log_dst_port
 - OSAPI AutoSAR, 168
- lookup_datareader
 - DDSSubscriber, 382
- lookup_datawriter
 - DDSPublisher, 374
- lookup_instance_untyped
 - DDSDataReader, 319
- lookup_participant
 - DDSDomainParticipantFactory, 357
- lookup_topicdescription
 - DDSDomainParticipant, 350
- lost_samples
 - DDS_DataReaderInstanceReplacedStatus, 239
- low
 - DDS_SequenceNumber_t, 299
- major
 - DDS_ProductVersion, 276
 - DDS_ProductVersion_t, 277
 - DDS_ProtocolVersion, 278
- majorRev
 - NDDS_Type_PluginVersion, 434
- matching_writer_reader_pair_allocation
 - DDS_DomainParticipantResourceLimitsQosPolicy, 255
- max_alloc_size
 - FooSeq, 429
- max_blocking_time
 - DDS_ReliabilityQosPolicy, 282
- max_buffer_size
 - OSAPI_LogProperty, 434
- max_components
 - DDS_SystemResourceLimitsQosPolicy, 303
- max_destination_ports
 - DDS_DomainParticipantResourceLimitsQosPolicy, 256
- max_heartbeat_retries
 - DDS_RtpsReliableWriterProtocol_t, 291
- max_instances
 - DDS_ResourceLimitsQosPolicy, 288
- max_local_addr_id
 - OSAPI_PortProperty, 438
- max_locators_per_discovered_participant, 234
 - DPDE_DiscoveryPluginProperty, 397
 - DPSE_DiscoveryPluginProperty, 400
- max_message_size
 - UDP_InterfaceFactoryProperty, 447
- max_outstanding_reads
 - DDS_DataReaderResourceLimitsQosPolicy, 243

- max_participant_locators, [233](#)
 - DPDE_DiscoveryPluginProperty, [396](#)
 - DPSE_DiscoveryPluginProperty, [400](#)
- max_participants
 - DDS_SystemResourceLimitsQosPolicy, [303](#)
- max_receive_buffer_size
 - UDP_InterfaceFactoryProperty, [447](#)
- max_receive_ports
 - DDS_DomainParticipantResourceLimitsQosPolicy, [256](#)
- max_receive_sockets
 - OSAPI_PortProperty, [437](#)
- max_remote_readers
 - DDS_DataWriterResourceLimitsQosPolicy, [247](#)
- max_remote_writers
 - DDS_DataReaderResourceLimitsQosPolicy, [242](#)
- max_remote_writers_per_instance
 - DDS_DataReaderResourceLimitsQosPolicy, [243](#)
- max_routes_per_reader
 - DDS_DataWriterResourceLimitsQosPolicy, [247](#)
- max_routes_per_writer
 - DDS_DataReaderResourceLimitsQosPolicy, [243](#)
- max_samples
 - DDS_ResourceLimitsQosPolicy, [287](#)
- max_samples_per_builtin_endpoint_reader, [235](#)
 - DPDE_DiscoveryPluginProperty, [397](#)
- max_samples_per_instance
 - DDS_ResourceLimitsQosPolicy, [288](#)
- max_samples_per_remote_builtin_endpoint_writer
 - DPDE_DiscoveryPluginProperty, [397](#)
- max_samples_per_remote_writer
 - DDS_DataReaderResourceLimitsQosPolicy, [243](#)
- max_send_buffer_size
 - UDP_InterfaceFactoryProperty, [446](#)
- max_send_message_size
 - UDP_InterfaceFactoryProperty, [447](#)
- max_send_window
 - DDS_RtpsReliableWriterProtocol_t, [291](#)
- maximum
 - FooSeq, [426](#), [427](#)
- metatraffic_multicast_locators
 - DDS_ParticipantBuiltinTopicData, [273](#)
- metatraffic_unicast_locators
 - DDS_ParticipantBuiltinTopicData, [273](#)
- minor
 - DDS_ProductVersion, [277](#)
 - DDS_ProductVersion_t, [277](#)
 - DDS_ProtocolVersion, [278](#)
- minorRev
 - NDDS_Type_PluginVersion, [434](#)
- multicast_interface
 - UDP_InterfaceFactoryProperty, [447](#)
- multicast_locator
 - DDS_SubscriptionBuiltinTopicData, [301](#)
- multicast_ttl
 - UDP_InterfaceFactoryProperty, [447](#)
- NDDS_Discovery_Property, [433](#)
- NDDS_Type_Plugin, [433](#)
- NDDS_Type_PluginVersion, [434](#)
 - majorRev, [434](#)
 - minorRev, [434](#)
- NETIO, [183](#)
 - NETIO_LOG_AR_ADD_INVALID_INTERFACE_EC, [188](#)
 - NETIO_LOG_AR_ADDRESS_RESOLVE_FAILED_EC, [188](#)
 - NETIO_LOG_AR_ADDRESS_STRING_EXCEEDED_EC, [188](#)
 - NETIO_LOG_AR_ALLOC_FAILED_EC, [188](#)
 - NETIO_LOG_AR_CONTEXT_UNINITIALIZED_EC, [188](#)
 - NETIO_LOG_AR_INVALID_TOKEN_EC, [188](#)
 - NETIO_LOG_AR_PORT_RESOLVE_FAILED_EC, [188](#)
 - NETIO_LOG_BIND_CREATE_RTABLE_EC, [187](#)
 - NETIO_LOG_BIND_DELETE_RTABLE_EC, [187](#)
 - NETIO_LOG_BIND_EXTERNAL_FAILED_EC, [187](#)
 - NETIO_LOG_BIND_NEW_EC, [187](#)
 - NETIO_LOG_BIND_NEW_RTABLE_EC, [187](#)
 - NETIO_LOG_BIND_SET_LENGTH_EC, [187](#)
 - NETIO_LOG_BIND_SET_MAX_EC, [187](#)
 - NETIO_LOG_GETHOST_BYNAME_EC, [187](#)
 - NETIO_LOG_ILLEGAL_ROUTE_OPERATION_EC, [188](#)
 - NETIO_LOG_INVALID_ADDRESS_INDEX_EC, [188](#)
 - NETIO_LOG_LOOP_BINDX_DB_EC, [189](#)
 - NETIO_LOG_LOOP_CREATE_TABLE_EC, [189](#)
 - NETIO_LOG_LOOP_CURSOR_ERROR_EC, [189](#)
 - NETIO_LOG_LOOP_DELETE_TABLE_EC, [189](#)
 - NETIO_LOG_LOOP_FWD_EC, [189](#)
 - NETIO_LOG_LOOP_INITIALIZE_FAILED_EC, [189](#)
 - NETIO_LOG_LOOP_INVALID_COMPONENT_EC, [189](#)
 - NETIO_LOG_LOOP_INVALID_FACTORY_EC, [189](#)
 - NETIO_LOG_LOOP_INVALID_PROPERTY_EC, [189](#)
 - NETIO_LOG_LOOP_SELECT_TABLE_EC, [189](#)
 - NETIO_LOG_LOOP_SET_LENGTH_EC, [189](#)
 - NETIO_LOG_LOOP_UNBINDX_DB_EC, [189](#)
 - NETIO_LOG_PACKET_INITIALIZE_EC, [188](#)
 - NETIO_LOG_PACKET_SET_HEAD_EC, [188](#)
 - NETIO_LOG_PACKET_SET_TAIL_EC, [188](#)
 - NETIO_LOG_ROUTE_RTABLE_ADD_EC, [187](#)
 - NETIO_LOG_ROUTE_RTABLE_ALLOC_EC, [187](#)
 - NETIO_LOG_ROUTE_RTABLE_CREATE_EC, [187](#)
 - NETIO_LOG_ROUTE_RTABLE_DELETE_EC, [187](#)
 - NETIO_LOG_ROUTE_RTABLE_UPDATE_EC, [187](#)
 - NETIO_LOG_SET_NAME_EC, [188](#)
 - NETIO_LOG_UNBIND_EXTERNAL_FAILED_EC, [187](#)

- 187
- UDP_LOG_ALLOC_EC, 192
- UDP_LOG_CURSOR_ERROR_EC, 192
- UDP_LOG_DROPGRP_EC, 192
- UDP_LOG_FINALIZE_EC, 190
- UDP_LOG_GET_BUF_ALLOC_EC, 191
- UDP_LOG_GET_BUF_SIZE_EC, 191
- UDP_LOG_GET_IFLIST_EC, 191
- UDP_LOG_GET_LENGTH_EC, 192
- UDP_LOG_GET_NAMEINFO_EC, 191
- UDP_LOG_GETHOSTNAME_EC, 189
- UDP_LOG_INCONSISTENT_MAX_MESSAGE_SIZE_EC, 192
- UDP_LOG_INITIALIZE_FAILED_EC, 192
- UDP_LOG_LOADLIB_EC, 191
- UDP_LOG_MULTICAST_ENABLE_EC, 191
- UDP_LOG_MULTICAST_NOT_ENABLED_EC, 192
- UDP_LOG_NAT_DELETE_EC, 190
- UDP_LOG_PACKET_FWD_EC, 190
- UDP_LOG_PACKET_HEAD_EC, 190
- UDP_LOG_PACKET_INIT_EC, 190
- UDP_LOG_PORT_ADD_EC, 191
- UDP_LOG_PORT_ALLOC_EC, 190
- UDP_LOG_PORT_ENTRY_EC, 190
- UDP_LOG_PORT_NOT_FOUND_EC, 192
- UDP_LOG_PORT_THREAD_EC, 191
- UDP_LOG_PROPERTY_FINALIZE_EC, 192
- UDP_LOG_RECORD_EC, 191
- UDP_LOG_RECV_ERROR_EC, 193
- UDP_LOG_SEND_ERROR_EC, 192
- UDP_LOG_SEND_PING_EC, 192
- UDP_LOG_SET_LENGTH_EC, 192
- UDP_LOG_SOCKET_ADDGRP_EC, 190
- UDP_LOG_SOCKET_BIND_EC, 190
- UDP_LOG_SOCKET_CLOSE_EC, 191
- UDP_LOG_SOCKET_CREATE_EC, 190
- UDP_LOG_SOCKET_IFFLAGS_EC, 191
- UDP_LOG_SOCKET_IFLIST_EC, 191
- UDP_LOG_SOCKET_INIT_EC, 189
- UDP_LOG_SOCKET_REUSE_PORT_EC, 190
- UDP_LOG_SOCKET_RXBUF_EC, 191
- UDP_LOG_SOCKET_SET_MCAST_EC, 189
- UDP_LOG_SOCKET_SET_MCASTIF_EC, 190
- UDP_LOG_SOCKET_SETFD_EC, 190
- UDP_LOG_SOCKET_SNDBUF_EC, 190
- UDP_LOG_SOCKET_TTL_EC, 190
- UDP_LOG_TRANSPORT_NO_INTERFACES_EC, 193
- UDP_LOG_UDP_CREATE_INDEX_EC, 191
- UDP_LOG_UDP_CREATE_TABLE_EC, 191
- UDP_LOG_WSOCKET_INCOMPATIBLE_EC, 192
- UDP_LOG_WSA_STARTUP_EC, 192
- UDP_TRANSFORM_LOG_ADD_EC, 193
- UDP_TRANSFORM_LOG_ALLOC_EC, 193
- UDP_TRANSFORM_LOG_CREATE_EC, 193
- UDP_TRANSFORM_LOG_DUPLICATE_EC, 193
- UDP_TRANSFORM_LOG_FACTORY_NOT_FOUND_EC, 193
- UDP_TRANSFORM_LOG_FIND_EC, 193
- UDP_TRANSFORM_LOG_INVALID_FACTORY_REFERENCE_EC, 193
- UDP_TRANSFORM_LOG_INVALID_TUDP_LOGATOR_KIND_EC, 193
- UDP_TRANSFORM_LOG_INVALID_UDP_MODE_EC, 193
- UDP_TRANSFORM_LOG_TRANSFORM_EC, 193
- NETIO_Autosar_Tcplp_get_socket OSAPI AutoSAR, 166
- NETIO_Autosar_Tcplp_pdu_callout OSAPI AutoSAR, 167
- NETIO_Autosar_Tcplp_send_dyn_data OSAPI AutoSAR, 167
- NETIO_Autosar_Tcplp_udp_rx_indication OSAPI AutoSAR, 168
- NETIO_LOG_AR_ADD_INVALID_INTERFACE_EC NETIO, 188
- NETIO_LOG_AR_ADDRESS_RESOLVE_FAILED_EC NETIO, 188
- NETIO_LOG_AR_ADDRESS_STRING_EXCEEDED_EC NETIO, 188
- NETIO_LOG_AR_ALLOC_FAILED_EC NETIO, 188
- NETIO_LOG_AR_CONTEXT_UNINITIALIZED_EC NETIO, 188
- NETIO_LOG_AR_INVALID_TOKEN_EC NETIO, 188
- NETIO_LOG_AR_PORT_RESOLVE_FAILED_EC NETIO, 188
- NETIO_LOG_BIND_CREATE_RTABLE_EC NETIO, 187
- NETIO_LOG_BIND_DELETE_RTABLE_EC NETIO, 187
- NETIO_LOG_BIND_EXTERNAL_FAILED_EC NETIO, 187
- NETIO_LOG_BIND_NEW_EC NETIO, 187
- NETIO_LOG_BIND_NEW_RTABLE_EC NETIO, 187
- NETIO_LOG_BIND_SET_LENGTH_EC NETIO, 187
- NETIO_LOG_BIND_SET_MAX_EC NETIO, 187
- NETIO_LOG_GETHOST_BYNAME_EC NETIO, 187
- NETIO_LOG_ILLEGAL_ROUTE_OPERATION_EC NETIO, 188
- NETIO_LOG_INVALID_ADDRESS_INDEX_EC NETIO, 188
- NETIO_LOG_LOOP_BINDX_DB_EC NETIO, 189
- NETIO_LOG_LOOP_CREATE_TABLE_EC NETIO, 189

- NETIO_LOG_LOOP_CURSOR_ERROR_EC
 - NETIO, [189](#)
- NETIO_LOG_LOOP_DELETE_TABLE_EC
 - NETIO, [189](#)
- NETIO_LOG_LOOP_FWD_EC
 - NETIO, [189](#)
- NETIO_LOG_LOOP_INITIALIZE_FAILED_EC
 - NETIO, [189](#)
- NETIO_LOG_LOOP_INVALID_COMPONENT_EC
 - NETIO, [189](#)
- NETIO_LOG_LOOP_INVALID_FACTORY_EC
 - NETIO, [189](#)
- NETIO_LOG_LOOP_INVALID_PROPERTY_EC
 - NETIO, [189](#)
- NETIO_LOG_LOOP_SELECT_TABLE_EC
 - NETIO, [189](#)
- NETIO_LOG_LOOP_SET_LENGTH_EC
 - NETIO, [189](#)
- NETIO_LOG_LOOP_UNBINDX_DB_EC
 - NETIO, [189](#)
- NETIO_LOG_PACKET_INITIALIZE_EC
 - NETIO, [188](#)
- NETIO_LOG_PACKET_SET_HEAD_EC
 - NETIO, [188](#)
- NETIO_LOG_PACKET_SET_TAIL_EC
 - NETIO, [188](#)
- NETIO_LOG_ROUTE_RTABLE_ADD_EC
 - NETIO, [187](#)
- NETIO_LOG_ROUTE_RTABLE_ALLOC_EC
 - NETIO, [187](#)
- NETIO_LOG_ROUTE_RTABLE_CREATE_EC
 - NETIO, [187](#)
- NETIO_LOG_ROUTE_RTABLE_DELETE_EC
 - NETIO, [187](#)
- NETIO_LOG_ROUTE_RTABLE_UPDATE_EC
 - NETIO, [187](#)
- NETIO_LOG_SET_NAME_EC
 - NETIO, [188](#)
- NETIO_LOG_UNBIND_EXTERNAL_FAILED_EC
 - NETIO, [187](#)
- nack_period
 - DDS_RtpsReliableReaderProtocol_t, [289](#)
- name
 - DDS_DiscoveryComponent, [250](#)
 - DDS_EntityNameQosPolicy, [259](#)
- nanosec
 - DDS_Duration_t, [258](#)
 - DDS_Time_t, [303](#)
- narrow
 - DDSTopic, [386](#)
 - FooDataReader, [404](#)
 - FooDataWriter, [412](#)
- nat
 - UDP_InterfaceFactoryProperty, [447](#)
- netio_log.h, [472](#)
 - UDP_TRANSFORM_LOG_KIND_DST_INDEX, [477](#)
 - UDP_TRANSFORM_LOG_KIND_ENTRY, [476](#)
 - UDP_TRANSFORM_LOG_KIND_FACTORY, [476](#)
 - UDP_TRANSFORM_LOG_KIND_FACTORY_INDEX, [477](#)
 - UDP_TRANSFORM_LOG_KIND_RULE, [476](#)
 - UDP_TRANSFORM_LOG_KIND_SRC_INDEX, [477](#)
 - UDP_TRANSFORM_LOG_KIND_TRANSFORM_DESTINATION, [477](#)
 - UDP_TRANSFORM_LOG_KIND_TRANSFORM_NETMASK_INDEX, [477](#)
 - UDP_TRANSFORM_LOG_KIND_TRANSFORM_SOURCE, [477](#)
- netmask
 - UDP_InterfaceTableEntry, [448](#)
- not_alive_count
 - DDS_LivelinessChangedStatus, [262](#)
- not_alive_count_change
 - DDS_LivelinessChangedStatus, [262](#)
- number_of_heap_areas
 - OSAPI_PortProperty, [436](#)
- number_of_rcv_buffers
 - OSAPI_PortProperty, [437](#)
- OS API, [64](#)
- OSAPI AutoSAR, [166](#)
 - log_dst_address, [168](#)
 - log_dst_port, [168](#)
 - NETIO_Autosar_Tcplp_get_socket, [166](#)
 - NETIO_Autosar_Tcplp_pdu_callout, [167](#)
 - NETIO_Autosar_Tcplp_send_dyn_data, [167](#)
 - NETIO_Autosar_Tcplp_udp_rx_indication, [168](#)
 - OSAPI_AUTOSAR_SYNCKIND_RESOURCES, [167](#)
 - OSAPI_Autosar_SyncKind_T, [167](#)
- OSAPI Heap, [160](#)
 - OSAPI_Heap_allocate, [160](#)
 - OSAPI_Heap_allocate_array, [161](#)
 - OSAPI_Heap_allocate_buffer, [162](#)
 - OSAPI_Heap_allocate_string, [162](#)
 - OSAPI_Heap_allocate_struct, [161](#)
 - OSAPI_Heap_disable_alloc, [163](#)
 - OSAPI_Heap_free, [161](#)
 - OSAPI_Heap_free_array, [162](#)
 - OSAPI_Heap_free_buffer, [163](#)
 - OSAPI_Heap_free_string, [162](#)
 - OSAPI_Heap_free_struct, [161](#)
 - OSAPI_Heap_get_allocated_byte_count, [163](#)
 - OSAPI_Heap_realloc, [161](#)
 - OSAPI_gv_AllocatedByteCount, [163](#)
- OSAPI Mutex, [164](#)
 - OSAPI_Mutex_T, [164](#)
 - OSAPI_Mutex_delete, [164](#)
 - OSAPI_Mutex_give, [165](#)
 - OSAPI_Mutex_new, [164](#)
 - OSAPI_Mutex_take, [165](#)
- OSAPI Semaphore, [169](#)
 - OSAPI_Semaphore_T, [169](#)
 - OSAPI_Semaphore_delete, [169](#)
 - OSAPI_Semaphore_give, [170](#)

- OSAPI_Semaphore_new, [169](#)
- OSAPI_Semaphore_take, [170](#)
- OSAPI System API, [171](#)
 - OSAPI_SYSTEM_MAX_HOSTNAME, [171](#)
- OSAPI Thread API, [172](#)
 - OSAPI_THREAD_DEFAULT_OPTIONS, [173](#)
 - OSAPI_THREAD_FLOATING_POINT, [173](#)
 - OSAPI_THREAD_PRIORITY_ABOVE_NORMAL, [173](#)
 - OSAPI_THREAD_PRIORITY_BELOW_NORMAL, [172](#)
 - OSAPI_THREAD_PRIORITY_HIGH, [173](#)
 - OSAPI_THREAD_PRIORITY_INHERIT, [173](#)
 - OSAPI_THREAD_PRIORITY_LOW, [172](#)
 - OSAPI_THREAD_PRIORITY_NORMAL, [173](#)
 - OSAPI_THREAD_PROPERTY_DEFAULT, [173](#)
 - OSAPI_THREAD_REALTIME_PRIORITY, [173](#)
 - OSAPI_THREAD_STDIO, [173](#)
 - OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE, [173](#)
 - OSAPI_ThreadOptions, [173](#)
 - OSAPI_ThreadProperty_INITIALIZER, [173](#)
- OSAPI Types, [174](#)
 - RTI_BOOL, [175](#)
 - RTI_FALSE, [175](#)
 - RTI_INT16, [174](#)
 - RTI_INT32, [174](#)
 - RTI_INT64, [175](#)
 - RTI_INT8, [174](#)
 - RTI_SIZE_T, [175](#)
 - RTI_TRUE, [175](#)
 - RTI_UINT16, [174](#)
 - RTI_UINT32, [174](#)
 - RTI_UINT64, [175](#)
 - RTI_UINT8, [174](#)
- OSAPI_AUTOSAR_SYNCKIND_RESOURCES
 - OSAPI AutoSAR, [167](#)
- OSAPI_Autosar_SyncKind_T
 - OSAPI AutoSAR, [167](#)
- OSAPI_Heap_allocate
 - OSAPI Heap, [160](#)
- OSAPI_Heap_allocate_array
 - OSAPI Heap, [161](#)
- OSAPI_Heap_allocate_buffer
 - OSAPI Heap, [162](#)
- OSAPI_Heap_allocate_string
 - OSAPI Heap, [162](#)
- OSAPI_Heap_allocate_struct
 - OSAPI Heap, [161](#)
- OSAPI_Heap_disable_alloc
 - OSAPI Heap, [163](#)
- OSAPI_Heap_free
 - OSAPI Heap, [161](#)
- OSAPI_Heap_free_array
 - OSAPI Heap, [162](#)
- OSAPI_Heap_free_buffer
 - OSAPI Heap, [163](#)
- OSAPI_Heap_free_string
 - OSAPI Heap, [162](#)
- OSAPI_Heap_free_struct
 - OSAPI Heap, [161](#)
- OSAPI_Heap_get_allocated_byte_count
 - OSAPI Heap, [163](#)
- OSAPI_Heap_realloc
 - OSAPI Heap, [161](#)
- OSAPI_LOG_VERBOSITY_DEBUG
 - Log API, [66](#)
- OSAPI_LOG_VERBOSITY_ERROR
 - Log API, [66](#)
- OSAPI_LOG_VERBOSITY_SILENT
 - Log API, [66](#)
- OSAPI_LOG_VERBOSITY_WARNING
 - Log API, [66](#)
- OSAPI_Log_clear
 - Log API, [68](#)
- OSAPI_Log_get_display_handler
 - Log API, [68](#)
- OSAPI_Log_get_last_error_code
 - Log API, [69](#)
- OSAPI_Log_get_log_handler
 - Log API, [67](#)
- OSAPI_Log_get_property
 - Log API, [69](#)
- OSAPI_Log_get_trace_handler
 - Log API, [67](#)
- OSAPI_Log_set_display_handler
 - Log API, [68](#)
- OSAPI_Log_set_log_handler
 - Log API, [66](#)
- OSAPI_Log_set_property
 - Log API, [69](#)
- OSAPI_Log_set_trace_handler
 - Log API, [67](#)
- OSAPI_Log_set_verbosity
 - Log API, [68](#)
- OSAPI_Log_write_buffer_T
 - Log API, [66](#)
- OSAPI_LogHandler_T
 - Log API, [66](#)
- OSAPI_LogProperty, [434](#)
 - log_detail, [434](#)
 - max_buffer_size, [434](#)
 - write_buffer, [435](#)
- OSAPI_LogVerbosity_T
 - Log API, [66](#)
- OSAPI_Mutex_T
 - OSAPI Mutex, [164](#)
- OSAPI_Mutex_delete
 - OSAPI Mutex, [164](#)
- OSAPI_Mutex_give
 - OSAPI Mutex, [165](#)
- OSAPI_Mutex_new
 - OSAPI Mutex, [164](#)
- OSAPI_Mutex_take
 - OSAPI Mutex, [165](#)
- OSAPI_MutexBase, [435](#)

- OSAPI_PortProperty, 435
 - first_alarm, 437
 - first_give_event, 437
 - first_resource_id, 436
 - first_timeout_event, 437
 - get_socket, 438
 - heap_area, 436
 - heap_area_size, 436
 - last_resource_id, 436
 - max_local_addr_id, 438
 - max_receive_sockets, 437
 - number_of_heap_areas, 436
 - number_of_rcv_buffers, 437
 - rcv_buffer_size, 438
 - semaphore_max_count, 437
 - send_data, 438
 - spinlock_id, 436
 - sync_type, 436
 - timer_resolution_ms, 436
 - udp_packet_received_event, 438
 - udp_receive_task_id, 438
 - use_socket_owner, 437
 - use_udp_thread, 438
- OSAPI_SEMAPHORE_RESULT_ERROR
 - osapi_semaphore.h, 478
- OSAPI_SEMAPHORE_RESULT_OK
 - osapi_semaphore.h, 478
- OSAPI_SEMAPHORE_RESULT_TIMEOUT
 - osapi_semaphore.h, 478
- OSAPI_SEMAPHORE_TIMEOUT_INFINITE
 - osapi_semaphore.h, 478
- OSAPI_SYSTEM_MAX_HOSTNAME
 - OSAPI System API, 171
- OSAPI_Semaphore_T
 - OSAPI Semaphore, 169
- OSAPI_Semaphore_delete
 - OSAPI Semaphore, 169
- OSAPI_Semaphore_give
 - OSAPI Semaphore, 170
- OSAPI_Semaphore_new
 - OSAPI Semaphore, 169
- OSAPI_Semaphore_take
 - OSAPI Semaphore, 170
- OSAPI_SystemI, 439
 - finalize, 439
 - generate_uuid, 440
 - get_hostname, 440
 - get_ticktime, 440
 - get_time, 439
 - get_timer_resolution, 439
 - initialize, 439
 - start_timer, 439
 - stop_timer, 439
- OSAPI_SystemProperty, 440
 - hostname, 440
 - port_property, 440
 - timer_property, 440
- OSAPI_THREAD_DEFAULT_OPTIONS
 - OSAPI Thread API, 173
- OSAPI_THREAD_FLOATING_POINT
 - OSAPI Thread API, 173
- OSAPI_THREAD_PRIORITY_ABOVE_NORMAL
 - OSAPI Thread API, 173
- OSAPI_THREAD_PRIORITY_BELOW_NORMAL
 - OSAPI Thread API, 172
- OSAPI_THREAD_PRIORITY_HIGH
 - OSAPI Thread API, 173
- OSAPI_THREAD_PRIORITY_INHERIT
 - OSAPI Thread API, 173
- OSAPI_THREAD_PRIORITY_LOW
 - OSAPI Thread API, 172
- OSAPI_THREAD_PRIORITY_NORMAL
 - OSAPI Thread API, 173
- OSAPI_THREAD_PROPERTY_DEFAULT
 - OSAPI Thread API, 173
- OSAPI_THREAD_REALTIME_PRIORITY
 - OSAPI Thread API, 173
- OSAPI_THREAD_STDIO
 - OSAPI Thread API, 173
- OSAPI_THREAD_USE_OSDEFAULT_STACKSIZE
 - OSAPI Thread API, 173
- OSAPI_ThreadOptions
 - OSAPI Thread API, 173
- OSAPI_ThreadProperty, 441
 - options, 441
 - priority, 441
 - stack_size, 441
- OSAPI_ThreadProperty_INITIALIZER
 - OSAPI Thread API, 173
- OSAPI_TimerProperty, 441
 - thread, 441
- OSAPI_Trace_set_trace_mask
 - Log API, 68
- OSAPI_gv_AllocatedByteCount
 - OSAPI Heap, 163
- OWNERSHIP, 93
 - DDS_EXCLUSIVE_OWNERSHIP_QOS, 93
 - DDS_OWNERSHIP_QOS_POLICY_NAME, 93
 - DDS_OwnershipQosPolicyKind, 93
 - DDS_SHARED_OWNERSHIP_QOS, 93
- OWNERSHIP_STRENGTH, 94
 - DDS_OWNERSHIPSTRENGTH_QOS_POLICY_NAME, 94
- on_before_sample_commit
 - DDSDDataReaderListener, 325
 - DDSDomainParticipantListener, 362
- on_before_sample_deserialize
 - DDSDDataReaderListener, 324
 - DDSDomainParticipantListener, 362
- on_data_available
 - DDSDDataReaderListener, 323
 - DDSDomainParticipantListener, 361
- on_data_on_readers
 - DDSDomainParticipantListener, 363
 - DDSSubscriberListener, 385
- on_inconsistent_topic

- DDSDomainParticipantListener, 364
- DDSTopicListener, 391
- on_instance_replaced
 - DDSDataReaderListener, 324
 - DDSDomainParticipantListener, 362
- on_liveliness_changed
 - DDSDataReaderListener, 324
 - DDSDomainParticipantListener, 361
- on_liveliness_lost
 - DDSDataWriterListener, 338
 - DDSDomainParticipantListener, 363
- on_offered_deadline_missed
 - DDSDataWriterListener, 338
 - DDSDomainParticipantListener, 363
- on_offered_incompatible_qos
 - DDSDataWriterListener, 338
 - DDSDomainParticipantListener, 363
- on_publication_matched
 - DDSDataWriterListener, 339
 - DDSDomainParticipantListener, 364
- on_reliable_reader_activity_changed
 - DDSDataWriterListener, 339
 - DDSDomainParticipantListener, 364
- on_reliable_sample_unacknowledged
 - DDSDataWriterListener, 339
- on_requested_deadline_missed
 - DDSDataReaderListener, 323
 - DDSDomainParticipantListener, 361
- on_requested_incompatible_qos
 - DDSDataReaderListener, 324
 - DDSDomainParticipantListener, 361
- on_sample_lost
 - DDSDataReaderListener, 324
 - DDSDomainParticipantListener, 362
- on_sample_rejected
 - DDSDataReaderListener, 324
 - DDSDomainParticipantListener, 361
- on_subscription_matched
 - DDSDataReaderListener, 324
 - DDSDomainParticipantListener, 361
- operator=
 - FooSeq, 422
- operator[]
 - FooSeq, 424, 425
- options
 - OSAPI_ThreadProperty, 441
- ordered_access
 - DDS_PresentationQosPolicy, 276
- osapi_semaphore.h, 477
 - OSAPI_SEMAPHORE_RESULT_ERROR, 478
 - OSAPI_SEMAPHORE_RESULT_OK, 478
 - OSAPI_SEMAPHORE_RESULT_TIMEOUT, 478
 - OSAPI_SEMAPHORE_TIMEOUT_INFINITE, 478
- osapi_stdio.h, 478
- osapi_time.h, 478
- ownership
 - DDS_DataReaderQos, 241
 - DDS_DataWriterQos, 246
 - DDS_PublicationBuiltinTopicData, 279
 - DDS_SubscriptionBuiltinTopicData, 301
 - ownership_strength
 - DDS_DataWriterQos, 246
 - DDS_PublicationBuiltinTopicData, 279
- PRESENTATION, 101
 - DDS_GROUP_PRESENTATION_QOS, 101
 - DDS_HIGHEST_OFFERED_PRESENTATION_QOS, 101
 - DDS_INSTANCE_PRESENTATION_QOS, 101
 - DDS_PresentationQosPolicyAccessScopeKind, 101
 - DDS_TOPIC_PRESENTATION_QOS, 101
- Participant Built-in Topic, 73
 - DDS_PARTICIPANT_TOPIC_NAME, 73
- participant_id
 - DDS_WireProtocolQosPolicy, 307
- participant_id_gain
 - DDS_RtpsWellKnownPorts_t, 293
- participant_key
 - DDS_PublicationBuiltinTopicData, 279
 - DDS_SubscriptionBuiltinTopicData, 300
- participant_liveliness_assert_period
 - DPDE_DiscoveryPluginProperty, 396
 - DPSE_DiscoveryPluginProperty, 399
- participant_liveliness_lease_duration
 - DPDE_DiscoveryPluginProperty, 396
 - DPSE_DiscoveryPluginProperty, 399
- participant_name
 - DDS_DomainParticipantQos, 253
 - DDS_ParticipantBuiltinTopicData, 273
- period
 - DDS_DeadlineQosPolicy, 249
- policies
 - DDS_OfferedIncompatibleQosStatus, 268
 - DDS_RequestedIncompatibleQosStatus, 286
- port
 - DDS_Locator_t, 266
- port_base
 - DDS_RtpsWellKnownPorts_t, 292
- port_property
 - OSAPI_SystemProperty, 440
- priority
 - OSAPI_ThreadProperty, 441
- product_version
 - DDS_ParticipantBuiltinTopicData, 273
- protocol
 - DDS_DataReaderQos, 242
 - DDS_DataWriterQos, 246
 - DDS_DomainParticipantQos, 253
- Publication Built-in Topic, 74
 - DDS_PUBLICATION_TOPIC_NAME, 74
- Publication Module, 30
- publication_handle
 - DDS_DataReaderInstanceReplacedStatus, 239
 - DDS_SampleInfo, 296
- publication_sequence_number
 - DDS_SampleInfo, 296

- RHSM_LOG_RW_PRUNE_FAILED_EC, [213](#)
- RHSM_LOG_SAMPLE_INFO_POOL_EC, [212](#)
- RHSM_LOG_SAMPLE_POOL_EC, [212](#)
- RHSM_LOG_SAMPLE_POOL_EMPTY_EC, [213](#)
- RHSM_LOG_SAMPLE_PTR_ARRAY_EC, [212](#)
- RHSM_LOG_TBF_NOT_SUPPORTED_EC, [212](#)
- RHSM Reader History API, [56](#)
- RHSM_LOG_DESTINATION_BY_SOURCE_NOT_SUPPORTED_EC
 - RH, [213](#)
- RHSM_LOG_ENTRY_RESERVATION_FAILED_EC
 - RH, [213](#)
- RHSM_LOG_FAILED_TO_REMOVE_OLDEST_EC
 - RH, [213](#)
- RHSM_LOG_GET_RECEPTION_TIMESTAMP_EC
 - RH, [212](#)
- RHSM_LOG_GET_TIME_EC
 - RH, [213](#)
- RHSM_LOG_HISTORY_OBJECT
 - rh_sm_log.h, [481](#)
- RHSM_LOG_HISTORY_UPDATE_EC
 - RH, [213](#)
- RHSM_LOG_INFO_ARRAY_EC
 - RH, [212](#)
- RHSM_LOG_INSTANCEHANDLE_OBJECT
 - rh_sm_log.h, [482](#)
- RHSM_LOG_INVALID_INSTANCE_REPLACEMENT_EC
 - RH, [212](#)
- RHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC
 - RH, [212](#)
- RHSM_LOG_KEY_OBJECT
 - rh_sm_log.h, [481](#)
- RHSM_LOG_KEYINDEXPOOL_OBJECT
 - rh_sm_log.h, [481](#)
- RHSM_LOG_KEYPOOL_OBJECT
 - rh_sm_log.h, [481](#)
- RHSM_LOG_MAX_SAMPLE_TOO_SMALL_EC
 - RH, [212](#)
- RHSM_LOG_NO_PROPERTY_EC
 - RH, [213](#)
- RHSM_LOG_NO_PROPERTY_QOS_EC
 - RH, [212](#)
- RHSM_LOG_OBJECT_ALLOCATE_EC
 - RH, [213](#)
- RHSM_LOG_OBJECT_DELETE_EC
 - RH, [213](#)
- RHSM_LOG_OBJECT_INDEX_EC
 - RH, [213](#)
- RHSM_LOG_OBJECT_INVALID_EC
 - RH, [213](#)
- RHSM_LOG_OUTSTANDING_SAMPLE_EC
 - RH, [212](#)
- RHSM_LOG_OWNER_OBJECT
 - rh_sm_log.h, [481](#)
- RHSM_LOG_RECOMMIT_FAILURE_EC
 - RH, [213](#)
- RHSM_LOG_REMOVELIST_OBJECT
 - rh_sm_log.h, [481](#)
- RHSM_LOG_RETURN_SAMPLE_EC
 - RH, [213](#)
- RHSM_LOG_RW_OBJECT
 - rh_sm_log.h, [481](#)
- RHSM_LOG_RW_PRUNE_FAILED_EC
 - RH, [213](#)
- RHSM_LOG_RWINDEX_OBJECT
 - rh_sm_log.h, [481](#)
- RHSM_LOG_RWPOOL_OBJECT
 - rh_sm_log.h, [481](#)
- RHSM_LOG_SAMPLE_INFO_POOL_EC
 - RH, [212](#)
- RHSM_LOG_SAMPLE_POOL_EC
 - RH, [212](#)
- RHSM_LOG_SAMPLE_POOL_EMPTY_EC
 - RH, [213](#)
- RHSM_LOG_SAMPLE_PTR_ARRAY_EC
 - RH, [212](#)
- RHSM_LOG_SAMPLEINFO_OBJECT
 - rh_sm_log.h, [482](#)
- RHSM_LOG_SAMPLEPOOL_OBJECT
 - rh_sm_log.h, [481](#)
- RHSM_LOG_SAMPLEPTPOOL_OBJECT
 - rh_sm_log.h, [481](#)
- RHSM_LOG_TBF_NOT_SUPPORTED_EC
 - RH, [212](#)
- RHSMHistoryFactory, [442](#)
 - get_interface, [442](#)
- RT, [181](#)
 - RT_LOG_REGISTRY_EXISTS_EC, [181](#)
 - RT_LOG_REGISTRY_FINALIZE_EC, [181](#)
 - RT_LOG_REGISTRY_INCONSISTENT_CID_EC, [182](#)
 - RT_LOG_REGISTRY_INIT_FACTORY_EC, [181](#)
 - RT_LOG_REGISTRY_INIT_FAILURE_EC, [181](#)
 - RT_LOG_REGISTRY_NAME_TOO_LONG_EC, [181](#)
 - RT_LOG_REGISTRY_NOT_INITIALIZED_EC, [182](#)
 - RT_LOG_REGISTRY_REGISTER_EC, [181](#)
 - RT_LOG_SET_IMMUTABLE_PROPERTY_EC, [181](#)
- RT Run-Time API, [55](#)
- RT_LOG_REGISTRY_EXISTS_EC
 - RT, [181](#)
- RT_LOG_REGISTRY_FINALIZE_EC
 - RT, [181](#)
- RT_LOG_REGISTRY_INCONSISTENT_CID_EC
 - RT, [182](#)
- RT_LOG_REGISTRY_INIT_FACTORY_EC
 - RT, [181](#)
- RT_LOG_REGISTRY_INIT_FAILURE_EC
 - RT, [181](#)
- RT_LOG_REGISTRY_NAME_TOO_LONG_EC
 - RT, [181](#)
- RT_LOG_REGISTRY_NOT_INITIALIZED_EC

- RT, [182](#)
- RT_LOG_REGISTRY_REGISTER_EC
 - RT, [181](#)
- RT_LOG_SET_IMMUTABLE_PROPERTY_EC
 - RT, [181](#)
- RTI_BOOL
 - OSAPI Types, [175](#)
- RTI_FALSE
 - OSAPI Types, [175](#)
- RTI_INT16
 - OSAPI Types, [174](#)
- RTI_INT32
 - OSAPI Types, [174](#)
- RTI_INT64
 - OSAPI Types, [175](#)
- RTI_INT8
 - OSAPI Types, [174](#)
- RTI_ManagementQosPolicy, [442](#)
 - is_anonymous, [442](#)
- RTI_SIZE_T
 - OSAPI Types, [175](#)
- RTI_TRUE
 - OSAPI Types, [175](#)
- RTI_UINT16
 - OSAPI Types, [174](#)
- RTI_UINT32
 - OSAPI Types, [174](#)
- RTI_UINT64
 - OSAPI Types, [175](#)
- RTI_UINT8
 - OSAPI Types, [174](#)
- RTPS, [195](#)
 - RTPS_LOG_ACK_EC, [203](#)
 - RTPS_LOG_ALLOCATE_EC, [200](#)
 - RTPS_LOG_ASSERT_PEER_EC, [203](#)
 - RTPS_LOG_ASSERT_TRANSPORT_EC, [203](#)
 - RTPS_LOG_BAD_PARAMETER_EC, [202](#)
 - RTPS_LOG_BITCOUNT_OUT_OF_BOUNDS_EC, [205](#)
 - RTPS_LOG_BITMAP_SET_BIT_EC, [205](#)
 - RTPS_LOG_BITMAP_SHIFT_EC, [207](#)
 - RTPS_LOG_BUFFERPOOL_DELETE_EC, [207](#)
 - RTPS_LOG_CHECKSUM_CHECKSUM_CALCULATION_FAILED_EC, [210](#)
 - RTPS_LOG_CHECKSUM_CHECKSUM_DESERIALIZE_ERROR_EC, [209](#)
 - RTPS_LOG_CHECKSUM_CHECKSUM_ERROR_EC, [209](#)
 - RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM128_EC, [209](#)
 - RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM32_EC, [209](#)
 - RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM64_EC, [209](#)
 - RTPS_LOG_CHECKSUM_INCONSISTENT_LENGTH_EC, [209](#)
 - RTPS_LOG_CHECKSUM_INVALID_CRC32_FLAGS_EC, [210](#)
 - RTPS_LOG_CHECKSUM_INVALID_CUSTOM128_EC, [210](#)
 - RTPS_LOG_CHECKSUM_INVALID_CUSTOM256_EC, [210](#)
 - RTPS_LOG_CHECKSUM_INVALID_CUSTOM32_EC, [210](#)
 - RTPS_LOG_CHECKSUM_INVALID_CUSTOM64_EC, [210](#)
 - RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN128_EC, [209](#)
 - RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN32_EC, [209](#)
 - RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN64_EC, [210](#)
 - RTPS_LOG_CHECKSUM_LENGTH_DESERIALIZE_ERROR_EC, [209](#)
 - RTPS_LOG_CHECKSUM_MISSING_CHECKSUM_LENGTH_EC, [209](#)
 - RTPS_LOG_CHECKSUM_UNSUPPORTED_EC, [209](#)
 - RTPS_LOG_CREATE_BIND_TABLE_EC, [206](#)
 - RTPS_LOG_CREATE_BUFFER_POOL_EC, [208](#)
 - RTPS_LOG_CREATE_EVENT_TIMER_EC, [205](#)
 - RTPS_LOG_CREATE_HB_EVENT_EC, [204](#)
 - RTPS_LOG_CREATE_PEERS_INDEX_EC, [208](#)
 - RTPS_LOG_CREATE_ROUTE_TABLE_EC, [201](#)
 - RTPS_LOG_DATA_ALREADY_RECEIVED_EC, [204](#)
 - RTPS_LOG_DATA_BATCH_ALREADY_RECEIVED_EC, [209](#)
 - RTPS_LOG_DATA_BATCH_SN_OUT_OF_ORDER_EC, [209](#)
 - RTPS_LOG_DATA_OUT_OF_RANGE_EC, [204](#)
 - RTPS_LOG_DATA_SN_OUT_OF_ORDER_EC, [209](#)
 - RTPS_LOG_DB_CREATE_BIND_ENTRY_EC, [201](#)
 - RTPS_LOG_DB_CREATE_BIND_PEER_INDEX_EC, [201](#)
 - RTPS_LOG_DB_CREATE_DIRECT_INDEX_EC, [208](#)
 - RTPS_LOG_DB_CREATE_EXT_BIND_ENTRY_EC, [205](#)
 - RTPS_LOG_DB_CREATE_GROUP_INDEX_EC, [208](#)
 - RTPS_LOG_DB_CREATE_ROUTE_ENTRY_EC, [201](#)
 - RTPS_LOG_DB_CREATE_ROUTE_PEER_INDEX_EC, [201](#)
 - RTPS_LOG_DB_CREATE_ROUTE_XPORT_INDEX_EC, [201](#)
 - RTPS_LOG_DB_CREATE_SELECTED_GROUP_INDEX_EC, [208](#)
 - RTPS_LOG_DB_DELETE_EXTERNAL_BIND_ENTRY_EC, [202](#)
 - RTPS_LOG_DB_DELETE_ROUTE_ENTRY_EC, [201](#)
 - RTPS_LOG_DB_INSERT_BIND_ENTRY_EC, [202](#)

- RTPS_LOG_DB_INSERT_ROUTE_ENTRY_EC, 201
- RTPS_LOG_DB_LOCK_EC, 206
- RTPS_LOG_DB_REMOVE_BIND_ENTRY_EC, 202
- RTPS_LOG_DB_REMOVE_EXTERNAL_BIND_ENTRY_EC, 202
- RTPS_LOG_DB_SELECT_ALL_EC, 201
- RTPS_LOG_DB_SELECT_MATCH_EC, 201
- RTPS_LOG_DB_SELECT_RANGE_EC, 201
- RTPS_LOG_DB_UNLOCK_EC, 206
- RTPS_LOG_DELETE_BUFFER_POOL_EC, 208
- RTPS_LOG_DELETE_INDEX_EC, 206
- RTPS_LOG_DELETE_INDEXER_EC, 208
- RTPS_LOG_DELETE_TABLE_EC, 206
- RTPS_LOG_DESERIALIZE_APP_ID_EC, 205
- RTPS_LOG_DESERIALIZE_HOST_ID_EC, 205
- RTPS_LOG_DESERIALIZE_INSTANCE_ID_EC, 205
- RTPS_LOG_DESERIALIZE_OBJECT_ID_EC, 205
- RTPS_LOG_DIRECT_SEND_EC, 207
- RTPS_LOG_EXCEEDED_LIMIT_PEERS_EC, 203
- RTPS_LOG_EXCEEDED_LIMIT_TRANSPORTS_EC, 203
- RTPS_LOG_FINALIZE_INTERFACE_EC, 207
- RTPS_LOG_FIND_EXTERNAL_INTERFACE_EC, 203
- RTPS_LOG_FIRST_SN_GREATER_LAST_SN_EC, 205
- RTPS_LOG_FORWARD_UPSTREAM_EC, 202
- RTPS_LOG_FULL_SEND_WINDOW_EC, 202
- RTPS_LOG_FULLY_ACKED_READER_EC, 204
- RTPS_LOG_FUNC_UNSUPPORTED_EC, 203
- RTPS_LOG_GET_PEER_BUFFER_EC, 208
- RTPS_LOG_GET_WINDOW_BUFFER_EC, 206
- RTPS_LOG_INCONSISTENT_HDR_EXT_LENGTH_ERROR_EC, 210
- RTPS_LOG_INDEX_ADD_ENTRY_EC, 201
- RTPS_LOG_INDEXER_REMOVE_ENTRY_EC, 207
- RTPS_LOG_INITIALIZE_INTERFACE_EC, 200
- RTPS_LOG_INSERT_WINDOW_FULL_EC, 206
- RTPS_LOG_INTERFACE_MISMATCH_EC, 202
- RTPS_LOG_INTERFACE_NOT_ENABLED_EC, 204
- RTPS_LOG_INTF_MODE_UNDEF_EC, 208
- RTPS_LOG_INVALID_HDR_EXT_PID_ERROR_EC, 210
- RTPS_LOG_INVALID_HDR_EXT_PID_LIST_EC, 210
- RTPS_LOG_INVALID_PACKET_EC, 204
- RTPS_LOG_LOOKUP_EXT_BIND_ENTRY_EC, 207
- RTPS_LOG_LOOKUP_PEER_EC, 207
- RTPS_LOG_MESSAGE_EXCEED_LENGTH_ERROR_EC, 210
- RTPS_LOG_NEGATIVE_RESERVATION_COUNT_EC, 208
- RTPS_LOG_NETIO_PACKET_INSUFFICIENT_BUFFER_EC, 208
- RTPS_LOG_NETIO_PACKET_SET_HEAD_EC, 204
- RTPS_LOG_NETIO_PACKET_SET_TAIL_EC, 202
- RTPS_LOG_NONEXISTENT_BIND_EC, 203
- RTPS_LOG_NONEXISTENT_EXTERNAL_BIND_EC, 203
- RTPS_LOG_NONEXISTENT_ROUTE_EC, 203
- RTPS_LOG_NOT_ENABLED_EC, 202
- RTPS_LOG_PACKET_INITIALIZE_EC, 207
- RTPS_LOG_PROCESS_ACKNACK_EC, 204
- RTPS_LOG_PROCESS_DATA_BATCH_EC, 208
- RTPS_LOG_PROCESS_DATA_EC, 204
- RTPS_LOG_PROCESS_GAP_EC, 204
- RTPS_LOG_PROCESS_HEARTBEAT_BATCH_EC, 208
- RTPS_LOG_PROCESS_HEARTBEAT_EC, 204
- RTPS_LOG_READER_SEND_ACKNACK_EC, 207
- RTPS_LOG_READER_UNSUPPORTED_EC, 202
- RTPS_LOG_RECEIVE_EC, 202
- RTPS_LOG_REQUEST_EC, 203
- RTPS_LOG_RETURN_LOAN_EC, 203
- RTPS_LOG_ROUTE_PACKET_EC, 202
- RTPS_LOG_SEND_EC, 202
- RTPS_LOG_SEND_HEARTBEAT_EC, 204
- RTPS_LOG_SEQ_INIT_EC, 207
- RTPS_LOG_SEQ_LEN_EC, 207
- RTPS_LOG_SEQ_MAX_EC, 207
- RTPS_LOG_SEQ_NUM_OUT_OF_RANGE_EC, 205
- RTPS_LOG_SERIALIZE_APP_ID_EC, 205
- RTPS_LOG_SERIALIZE_HOST_ID_EC, 205
- RTPS_LOG_SERIALIZE_INSTANCE_ID_EC, 205
- RTPS_LOG_SERIALIZE_OBJECT_ID_EC, 205
- RTPS_LOG_SET_GAP_EC, 206
- RTPS_LOG_SHIFT_BITMAP_EC, 204
- RTPS_LOG_SHIFT_SN_OUT_OF_RANGE_EC, 205
- RTPS_LOG_STALE_ACK_EPOCH_EC, 203
- RTPS_LOG_STALE_HB_EPOCH_EC, 203
- RTPS_LOG_STATUS_CHANGE_EC, 206
- RTPS_LOG_TIMER_DELETE_TIMEOUT_EC, 206
- RTPS_LOG_UNKNOWN_SUBMESSAGE_EC, 204
- RTPS_LOG_UNSUPPORTED_INFO_REPLY_EC, 206
- RTPS_LOG_UNSUPPORTED_INFO_REPLY_IP4_EC, 206
- RTPS_LOG_UNSUPPORTED_INFO_SRC_EC, 206
- RTPS_LOG_UNSUPPORTED_INTERFACE_EC, 204

- RTPS_LOG_UNSUPPORTED_MANDATORY_HEADER_EXT_PID_EC, [210](#)
- RTPS_LOG_WINDOW_ADVANCE_EC, [207](#)
- RTPS_LOG_WINDOW_ADVANCE_UNACKED_EC, [207](#)
- RTPS_LOG_WINDOW_INSERT_EC, [206](#)
- RTPS_LOG_WINDOW_POOL_ALLOC_EC, [206](#)
- RTPS_LOG_WRITER_REQUEST_SENT_HISTORY_EC, [207](#)
- RTPS Transport API, [61](#)
 - allow_builtin_override, [63](#)
 - builtin_checksum128_class, [63](#)
 - builtin_checksum32_class, [63](#)
 - builtin_checksum64_class, [63](#)
 - checksum, [63](#)
 - checksum128, [63](#)
 - checksum32, [63](#)
 - checksum64, [63](#)
 - checksum_calculate, [63](#)
 - checksum_tx_mode, [63](#)
 - class_id, [63](#)
 - context, [63](#)
 - RTPS_CHECKSUM_TXMODE_OMG, [62](#)
 - RTPS_CHECKSUM_TXMODE_RTICRC32, [62](#)
 - RTPS_Checksum_T, [62](#)
 - RTPS_ChecksumCalculate_T, [62](#)
 - RTPS_ChecksumClass_T, [62](#)
 - RTPS_ChecksumTxMode_T, [62](#)
 - RTPS_INTERFACE_FACTORY_DEFAULT, [63](#)
 - RTPS_InterfaceFactoryProperty_T, [62](#)
- RTPS_CHECKSUM_TXMODE_OMG
 - RTPS Transport API, [62](#)
- RTPS_CHECKSUM_TXMODE_RTICRC32
 - RTPS Transport API, [62](#)
- RTPS_Checksum, [443](#)
- RTPS_Checksum_T
 - RTPS Transport API, [62](#)
- RTPS_ChecksumCalculate_T
 - RTPS Transport API, [62](#)
- RTPS_ChecksumClass, [443](#)
- RTPS_ChecksumClass_T
 - RTPS Transport API, [62](#)
- RTPS_ChecksumProperty, [443](#)
- RTPS_ChecksumTxMode_T
 - RTPS Transport API, [62](#)
- RTPS_INTERFACE_FACTORY_DEFAULT
 - RTPS Transport API, [63](#)
- RTPS_InterfaceFactoryProperty, [444](#)
- RTPS_InterfaceFactoryProperty_T
 - RTPS Transport API, [62](#)
- RTPS_LOG_ACK_EC
 - RTPS, [203](#)
- RTPS_LOG_ALLOCATE_EC
 - RTPS, [200](#)
- RTPS_LOG_ASSERT_PEER_EC
 - RTPS, [203](#)
- RTPS_LOG_ASSERT_TRANSPORT_EC
 - RTPS, [203](#)
- RTPS_LOG_BAD_PARAMETER_EC
 - RTPS, [202](#)
- RTPS_LOG_BITCOUNT_OUT_OF_BOUNDS_EC
 - RTPS, [205](#)
- RTPS_LOG_BITMAP_SET_BIT_EC
 - RTPS, [205](#)
- RTPS_LOG_BITMAP_SHIFT_EC
 - RTPS, [207](#)
- RTPS_LOG_BUFFERPOOL_DELETE_EC
 - RTPS, [207](#)
- RTPS_LOG_CHECKSUM_CHECKSUM_CALCULATION_FAILED_EC
 - RTPS, [210](#)
- RTPS_LOG_CHECKSUM_CHECKSUM_DESERIALIZE_ERROR_EC
 - RTPS, [209](#)
- RTPS_LOG_CHECKSUM_CHECKSUM_ERROR_EC
 - RTPS, [209](#)
- RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM128_EC
 - RTPS, [209](#)
- RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM32_EC
 - RTPS, [209](#)
- RTPS_LOG_CHECKSUM_ILLEGAL_OVERRIDE_CHECKSUM64_EC
 - RTPS, [209](#)
- RTPS_LOG_CHECKSUM_INCONSISTENT_LENGTH_EC
 - RTPS, [209](#)
- RTPS_LOG_CHECKSUM_INVALID_CRC32_FLAGS_EC
 - RTPS, [210](#)
- RTPS_LOG_CHECKSUM_INVALID_CUSTOM128_EC
 - RTPS, [210](#)
- RTPS_LOG_CHECKSUM_INVALID_CUSTOM256_EC
 - RTPS, [210](#)
- RTPS_LOG_CHECKSUM_INVALID_CUSTOM32_EC
 - RTPS, [210](#)
- RTPS_LOG_CHECKSUM_INVALID_CUSTOM64_EC
 - RTPS, [210](#)
- RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN128_EC
 - RTPS, [209](#)
- RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN32_EC
 - RTPS, [209](#)
- RTPS_LOG_CHECKSUM_INVALID_OVERRIDE_BUILTIN64_EC
 - RTPS, [210](#)
- RTPS_LOG_CHECKSUM_LENGTH_DESERIALIZE_ERROR_EC
 - RTPS, [209](#)
- RTPS_LOG_CHECKSUM_MISSING_CHECKSUM_LENGTH_EC
 - RTPS, [209](#)
- RTPS_LOG_CHECKSUM_UNSUPPORTED_EC
 - RTPS, [209](#)

- RTPS_LOG_CREATE_BIND_TABLE_EC
RTPS, [206](#)
- RTPS_LOG_CREATE_BUFFER_POOL_EC
RTPS, [208](#)
- RTPS_LOG_CREATE_EVENT_TIMER_EC
RTPS, [205](#)
- RTPS_LOG_CREATE_HB_EVENT_EC
RTPS, [204](#)
- RTPS_LOG_CREATE_PEERS_INDEX_EC
RTPS, [208](#)
- RTPS_LOG_CREATE_ROUTE_TABLE_EC
RTPS, [201](#)
- RTPS_LOG_DATA_ALREADY_RECEIVED_EC
RTPS, [204](#)
- RTPS_LOG_DATA_BATCH_ALREADY_RECEIVED_-
EC
RTPS, [209](#)
- RTPS_LOG_DATA_BATCH_SN_OUT_OF_ORDER_-
EC
RTPS, [209](#)
- RTPS_LOG_DATA_OUT_OF_RANGE_EC
RTPS, [204](#)
- RTPS_LOG_DATA_SN_OUT_OF_ORDER_EC
RTPS, [209](#)
- RTPS_LOG_DB_CREATE_BIND_ENTRY_EC
RTPS, [201](#)
- RTPS_LOG_DB_CREATE_BIND_PEER_INDEX_EC
RTPS, [201](#)
- RTPS_LOG_DB_CREATE_DIRECT_INDEX_EC
RTPS, [208](#)
- RTPS_LOG_DB_CREATE_EXT_BIND_ENTRY_EC
RTPS, [205](#)
- RTPS_LOG_DB_CREATE_GROUP_INDEX_EC
RTPS, [208](#)
- RTPS_LOG_DB_CREATE_ROUTE_ENTRY_EC
RTPS, [201](#)
- RTPS_LOG_DB_CREATE_ROUTE_PEER_INDEX_E-
C
RTPS, [201](#)
- RTPS_LOG_DB_CREATE_ROUTE_XPORT_INDEX_-
EC
RTPS, [201](#)
- RTPS_LOG_DB_CREATE_SELECTED_GROUP_IND-
EX_EC
RTPS, [208](#)
- RTPS_LOG_DB_DELETE_EXTERNAL_BIND_ENTR-
Y_EC
RTPS, [202](#)
- RTPS_LOG_DB_DELETE_ROUTE_ENTRY_EC
RTPS, [201](#)
- RTPS_LOG_DB_INSERT_BIND_ENTRY_EC
RTPS, [202](#)
- RTPS_LOG_DB_INSERT_ROUTE_ENTRY_EC
RTPS, [201](#)
- RTPS_LOG_DB_LOCK_EC
RTPS, [206](#)
- RTPS_LOG_DB_REMOVE_BIND_ENTRY_EC
RTPS, [202](#)
- RTPS_LOG_DB_REMOVE_EXTERNAL_BIND_ENTR-
Y_EC
RTPS, [202](#)
- RTPS_LOG_DB_REMOVE_ROUTE_ENTRY_EC
rtps_log.h, [488](#)
- RTPS_LOG_DB_SELECT_ALL_EC
RTPS, [201](#)
- RTPS_LOG_DB_SELECT_MATCH_EC
RTPS, [201](#)
- RTPS_LOG_DB_SELECT_RANGE_EC
RTPS, [201](#)
- RTPS_LOG_DB_UNLOCK_EC
RTPS, [206](#)
- RTPS_LOG_DELETE_BUFFER_POOL_EC
RTPS, [208](#)
- RTPS_LOG_DELETE_INDEX_EC
RTPS, [206](#)
- RTPS_LOG_DELETE_INDEXER_EC
RTPS, [208](#)
- RTPS_LOG_DELETE_TABLE_EC
RTPS, [206](#)
- RTPS_LOG_DESERIALIZE_APP_ID_EC
RTPS, [205](#)
- RTPS_LOG_DESERIALIZE_HOST_ID_EC
RTPS, [205](#)
- RTPS_LOG_DESERIALIZE_INSTANCE_ID_EC
RTPS, [205](#)
- RTPS_LOG_DESERIALIZE_OBJECT_ID_EC
RTPS, [205](#)
- RTPS_LOG_DIRECT_SEND_EC
RTPS, [207](#)
- RTPS_LOG_EXCEEDED_LIMIT_PEERS_EC
RTPS, [203](#)
- RTPS_LOG_EXCEEDED_LIMIT_TRANSPORTS_EC
RTPS, [203](#)
- RTPS_LOG_FINALIZE_INTERFACE_EC
RTPS, [207](#)
- RTPS_LOG_FIND_EXTERNAL_INTERFACE_EC
RTPS, [203](#)
- RTPS_LOG_FIRST_SN_GREATER_LAST_SN_EC
RTPS, [205](#)
- RTPS_LOG_FORWARD_UPSTREAM_EC
RTPS, [202](#)
- RTPS_LOG_FULL_SEND_WINDOW_EC
RTPS, [202](#)
- RTPS_LOG_FULLY_ACKED_READER_EC
RTPS, [204](#)
- RTPS_LOG_FUNC_UNSUPPORTED_EC
RTPS, [203](#)
- RTPS_LOG_GET_PEER_BUFFER_EC
RTPS, [208](#)
- RTPS_LOG_GET_WINDOW_BUFFER_EC
RTPS, [206](#)
- RTPS_LOG_INCONSISTENT_HDR_EXT_LENGTH_E-
RROR_EC
RTPS, [210](#)
- RTPS_LOG_INDEX_ADD_ENTRY_EC
RTPS, [201](#)

- RTPS_LOG_INDEXER_REMOVE_ENTRY_EC
RTPS, [207](#)
- RTPS_LOG_INITIALIZE_INTERFACE_EC
RTPS, [200](#)
- RTPS_LOG_INSERT_WINDOW_FULL_EC
RTPS, [206](#)
- RTPS_LOG_INTERFACE_MISMATCH_EC
RTPS, [202](#)
- RTPS_LOG_INTERFACE_NOT_ENABLED_EC
RTPS, [204](#)
- RTPS_LOG_INTF_MODE_UNDEF_EC
RTPS, [208](#)
- RTPS_LOG_INVALID_HDR_EXT_PID_ERROR_EC
RTPS, [210](#)
- RTPS_LOG_INVALID_HDR_EXT_PID_LIST_EC
RTPS, [210](#)
- RTPS_LOG_INVALID_PACKET_EC
RTPS, [204](#)
- RTPS_LOG_LOOKUP_EXT_BIND_ENTRY_EC
RTPS, [207](#)
- RTPS_LOG_LOOKUP_PEER_EC
RTPS, [207](#)
- RTPS_LOG_MESSAGE_EXCEED_LENGTH_ERROR_EC
RTPS, [210](#)
- RTPS_LOG_NEGATIVE_RESERVATION_COUNT_EC
RTPS, [208](#)
- RTPS_LOG_NETIO_PACKET_INSUFFICIENT_BUFFER_EC
RTPS, [208](#)
- RTPS_LOG_NETIO_PACKET_SET_HEAD_EC
RTPS, [204](#)
- RTPS_LOG_NETIO_PACKET_SET_TAIL_EC
RTPS, [202](#)
- RTPS_LOG_NONEXISTENT_BIND_EC
RTPS, [203](#)
- RTPS_LOG_NONEXISTENT_EXTERNAL_BIND_EC
RTPS, [203](#)
- RTPS_LOG_NONEXISTENT_ROUTE_EC
RTPS, [203](#)
- RTPS_LOG_NOT_ENABLED_EC
RTPS, [202](#)
- RTPS_LOG_PACKET_INITIALIZE_EC
RTPS, [207](#)
- RTPS_LOG_PROCESS_ACKNACK_EC
RTPS, [204](#)
- RTPS_LOG_PROCESS_DATA_BATCH_EC
RTPS, [208](#)
- RTPS_LOG_PROCESS_DATA_EC
RTPS, [204](#)
- RTPS_LOG_PROCESS_GAP_EC
RTPS, [204](#)
- RTPS_LOG_PROCESS_HEARTBEAT_BATCH_EC
RTPS, [208](#)
- RTPS_LOG_PROCESS_HEARTBEAT_EC
RTPS, [204](#)
- RTPS_LOG_READER_SEND_ACKNACK_EC
RTPS, [207](#)
- RTPS_LOG_READER_UNSUPPORTED_EC
RTPS, [202](#)
- RTPS_LOG_RECEIVE_EC
RTPS, [202](#)
- RTPS_LOG_REQUEST_EC
RTPS, [203](#)
- RTPS_LOG_RETURN_LOAN_EC
RTPS, [203](#)
- RTPS_LOG_ROUTE_PACKET_EC
RTPS, [202](#)
- RTPS_LOG_SEND_EC
RTPS, [202](#)
- RTPS_LOG_SEND_HEARTBEAT_EC
RTPS, [204](#)
- RTPS_LOG_SEQ_INIT_EC
RTPS, [207](#)
- RTPS_LOG_SEQ_LEN_EC
RTPS, [207](#)
- RTPS_LOG_SEQ_MAX_EC
RTPS, [207](#)
- RTPS_LOG_SEQ_NUM_OUT_OF_RANGE_EC
RTPS, [205](#)
- RTPS_LOG_SERIALIZE_APP_ID_EC
RTPS, [205](#)
- RTPS_LOG_SERIALIZE_HOST_ID_EC
RTPS, [205](#)
- RTPS_LOG_SERIALIZE_INSTANCE_ID_EC
RTPS, [205](#)
- RTPS_LOG_SERIALIZE_OBJECT_ID_EC
RTPS, [205](#)
- RTPS_LOG_SET_GAP_EC
RTPS, [206](#)
- RTPS_LOG_SHIFT_BITMAP_EC
RTPS, [204](#)
- RTPS_LOG_SHIFT_SN_OUT_OF_RANGE_EC
RTPS, [205](#)
- RTPS_LOG_STALE_ACK_EPOCH_EC
RTPS, [203](#)
- RTPS_LOG_STALE_HB_EPOCH_EC
RTPS, [203](#)
- RTPS_LOG_STATUS_CHANGE_EC
RTPS, [206](#)
- RTPS_LOG_TIMER_DELETE_TIMEOUT_EC
RTPS, [206](#)
- RTPS_LOG_UNKNOWN_SUBMESSAGE_EC
RTPS, [204](#)
- RTPS_LOG_UNSUPPORTED_INFO_REPLY_EC
RTPS, [206](#)
- RTPS_LOG_UNSUPPORTED_INFO_REPLY_IP4_EC
RTPS, [206](#)
- RTPS_LOG_UNSUPPORTED_INFO_SRC_EC
RTPS, [206](#)
- RTPS_LOG_UNSUPPORTED_INTERFACE_EC
RTPS, [204](#)
- RTPS_LOG_UNSUPPORTED_MANDATORY_HDR_EXT_PID_EC
RTPS, [210](#)
- RTPS_LOG_WINDOW_ADVANCE_EC

- RTPS, [207](#)
- RTPS_LOG_WINDOW_ADVANCE_UNACKED_EC
 - RTPS, [207](#)
- RTPS_LOG_WINDOW_INSERT_EC
 - RTPS, [206](#)
- RTPS_LOG_WINDOW_POOL_ALLOC_EC
 - RTPS, [206](#)
- RTPS_LOG_WRITER_REQUEST_SENT_HISTORY_-EC
 - RTPS, [207](#)
- RTRegistry, [444](#)
 - register_component, [445](#)
 - unregister, [445](#)
- rcv_buffer_size
 - OSAPI_PortProperty, [438](#)
- read
 - FooDataReader, [406](#)
- read_instance
 - FooDataReader, [410](#)
- read_instance_untyped
 - DDSDataReader, [317](#)
- read_next_sample
 - FooDataReader, [405](#)
- read_next_sample_untyped
 - DDSDataReader, [318](#)
- read_untyped
 - DDSDataReader, [313](#)
- reader_resource_limits
 - DDS_DataReaderQos, [242](#)
- reason
 - DDS_SampleLostStatus, [297](#)
- reception_timestamp
 - DDS_SampleInfo, [296](#)
- recv_thread
 - UDP_InterfaceFactoryProperty, [447](#)
- reda_log.h, [478](#)
- register_component
 - RTRegistry, [445](#)
- register_instance
 - FooDataWriter, [414](#)
- register_instance_untyped
 - DDSDataWriter, [330](#)
- register_instance_w_timestamp
 - FooDataWriter, [414](#)
- register_instance_w_timestamp_untyped
 - DDSDataWriter, [331](#)
- register_type
 - DDSDomainParticipant, [351](#)
 - FooTypeSupport, [432](#)
- release
 - DDS_ProductVersion, [277](#)
 - DDS_ProductVersion_t, [277](#)
- reliability
 - DDS_DataWriterQos, [246](#)
 - DDS_PublicationBuiltinTopicData, [279](#)
 - DDS_SubscriptionBuiltinTopicData, [301](#)
- remote_participant_allocation
 - DDS_DomainParticipantResourceLimitsQosPolicy, [255](#)
- remote_reader_allocation
 - DDS_DomainParticipantResourceLimitsQosPolicy, [255](#)
- remote_writer_allocation
 - DDS_DomainParticipantResourceLimitsQosPolicy, [255](#)
- RemoteParticipant_assert
 - DPSEDiscoveryPlugin, [402](#)
- RemotePublication_assert
 - DPSEDiscoveryPlugin, [402](#)
- RemoteSubscription_assert
 - DPSEDiscoveryPlugin, [402](#)
- require_crc
 - DDS_ChecksumProperty, [238](#)
 - DDS_ChecksumProperty_t, [238](#)
 - DDS_WireProtocolQosPolicy, [308](#)
- resource_limits
 - DDS_DataReaderQos, [241](#)
 - DDS_DataWriterQos, [246](#)
 - DDS_DomainParticipantFactoryQos, [252](#)
 - DDS_DomainParticipantQos, [253](#)
- Return Codes, [81](#)
 - DDS_RETCODE_ALREADY_DELETED, [82](#)
 - DDS_RETCODE_BAD_PARAMETER, [81](#)
 - DDS_RETCODE_ERROR, [81](#)
 - DDS_RETCODE_ILLEGAL_OPERATION, [82](#)
 - DDS_RETCODE_IMMUTABLE_POLICY, [82](#)
 - DDS_RETCODE_INCONSISTENT_POLICY, [82](#)
 - DDS_RETCODE_NO_DATA, [82](#)
 - DDS_RETCODE_NOT_ENABLED, [82](#)
 - DDS_RETCODE_OK, [81](#)
 - DDS_RETCODE_OUT_OF_RESOURCES, [82](#)
 - DDS_RETCODE_PRECONDITION_NOT_MET, [82](#)
 - DDS_RETCODE_TIMEOUT, [82](#)
 - DDS_RETCODE_UNSUPPORTED, [81](#)
 - DDS_ReturnCode_t, [81](#)
- return_loan
 - FooDataReader, [404](#)
- return_loan_untyped
 - DDSDataReader, [319](#)
- revision
 - DDS_ProductVersion, [277](#)
 - DDS_ProductVersion_t, [277](#)
- rh_sm_log.h, [479](#)
 - RHSM_LOG_HISTORY_OBJECT, [481](#)
 - RHSM_LOG_INSTANCEHANDLE_OBJECT, [482](#)
 - RHSM_LOG_KEY_OBJECT, [481](#)
 - RHSM_LOG_KEYINDEXPOOL_OBJECT, [481](#)
 - RHSM_LOG_KEYPOOL_OBJECT, [481](#)
 - RHSM_LOG_OWNER_OBJECT, [481](#)
 - RHSM_LOG_REMOVELIST_OBJECT, [481](#)
 - RHSM_LOG_RW_OBJECT, [481](#)
 - RHSM_LOG_RWINDEX_OBJECT, [481](#)
 - RHSM_LOG_RWPOOL_OBJECT, [481](#)
 - RHSM_LOG_SAMPLEINFO_OBJECT, [482](#)

- RHSM_LOG_SAMPLEPOOL_OBJECT, 481
- RHSM_LOG_SAMPLEPTPOOL_OBJECT, 481
- rt_log.h, 482
- rtps_app_id
 - DDS_WireProtocolQosPolicy, 307
- rtps_config.h, 482
- rtps_dll.h, 482
- rtps_host_id
 - DDS_WireProtocolQosPolicy, 307
- rtps_instance_id
 - DDS_WireProtocolQosPolicy, 307
- rtps_log.h, 483
 - RTPS_LOG_DB_REMOVE_ROUTE_ENTRY_EC, 488
- rtps_object_id
 - DDS_DataReaderProtocolQosPolicy, 240
 - DDS_DataWriterProtocolQosPolicy, 244
- rtps_protocol_version
 - DDS_ParticipantBuiltinTopicData, 273
- rtps_reliable_reader
 - DDS_DataReaderProtocolQosPolicy, 240
- rtps_reliable_writer
 - DDS_DataWriterProtocolQosPolicy, 244
- rtps_rtps_impl.h, 489
- rtps_vendor_id
 - DDS_ParticipantBuiltinTopicData, 273
- rtps_well_known_ports
 - DDS_WireProtocolQosPolicy, 308
- SYSTEM_RESOURCE_LIMITS, 107
 - DDS_SYSTEMRESOURCELIMITS_QOS_POLICY_NAME, 107
- Sample States, 41
 - DDS_ANY_SAMPLE_STATE, 42
 - DDS_NOT_READ_SAMPLE_STATE, 41
 - DDS_READ_SAMPLE_STATE, 41
 - DDS_SampleStateKind, 41
 - DDS_SampleStateMask, 41
- sample_info
 - DDS_SampleLostStatus, 297
- sample_state
 - DDS_SampleInfo, 295
- sec
 - DDS_Duration_t, 258
 - DDS_Time_t, 303
- semaphore_max_count
 - OSAPI_PortProperty, 437
- send_data
 - OSAPI_PortProperty, 438
- Sequence Number Support, 80
 - DDS_SequenceNumber_compare, 80
- Sequence Support, 153
- sequence_number
 - DDS_ReliableSampleUnacknowledgedStatus, 284
- set_default_datareader_qos
 - DDSSubscriber, 381
- set_default_datawriter_qos
 - DDSPublisher, 374
- set_default_participant_qos
 - DDSDomainParticipantFactory, 358
- set_default_publisher_qos
 - DDSDomainParticipant, 347
- set_default_subscriber_qos
 - DDSDomainParticipant, 348
- set_default_topic_qos
 - DDSDomainParticipant, 349
- set_enabled_statuses
 - DDSStatusCondition, 378
- set_listener
 - DDSDataReader, 313
 - DDSDataWriter, 329
 - DDSDomainParticipant, 352
 - DDSPublisher, 376
 - DDSSubscriber, 384
 - DDSTopic, 388
- set_qos
 - DDSDataReader, 312
 - DDSDataWriter, 329
 - DDSDomainParticipant, 346
 - DDSDomainParticipantFactory, 359
 - DDSPublisher, 375
 - DDSSubscriber, 383
 - DDSTopic, 387
- set_trigger_value
 - DDSGuardCondition, 368
- source_rules
 - UDP_InterfaceFactoryProperty, 447
- source_timestamp
 - DDS_SampleInfo, 295
- source_timestamp_tolerance
 - DDS_DestinationOrderQosPolicy, 249
- spinlock_id
 - OSAPI_PortProperty, 436
- stack_size
 - OSAPI_ThreadProperty, 441
- start_timer
 - OSAPI_SystemI, 439
- Status Kinds, 83
 - DDS_DATA_AVAILABLE_STATUS, 87
 - DDS_DATA_ON_READERS_STATUS, 87
 - DDS_INCONSISTENT_TOPIC_STATUS, 86
 - DDS_INSTANCE_REPLACED_STATUS, 88
 - DDS_LIVELINESS_CHANGED_STATUS, 88
 - DDS_LIVELINESS_LOST_STATUS, 87
 - DDS_OFFERED_DEADLINE_MISSED_STATUS, 86
 - DDS_OFFERED_INCOMPATIBLE_QOS_STATUSES, 86
 - DDS_PUBLICATION_MATCHED_STATUS, 88
 - DDS_RELIABLE_READER_ACTIVITY_CHANGED_STATUS, 88
 - DDS_REQUESTED_DEADLINE_MISSED_STATUS, 86
 - DDS_REQUESTED_INCOMPATIBLE_QOS_STATUS, 86
 - DDS_SAMPLE_LOST_STATUS, 87
 - DDS_SAMPLE_REJECTED_STATUS, 87

- DDS_STATUS_MASK_ALL, 85
- DDS_STATUS_MASK_NONE, 85
- DDS_SUBSCRIPTION_MATCHED_STATUS, 88
- DDS_StatusKind, 85
- DDS_StatusMask, 85
- stop_timer
 - OSAPI_SystemI, 439
- String Support, 154
 - DDS_String_alloc, 155
 - DDS_String_cmp, 156
 - DDS_String_dup, 155
 - DDS_String_free, 156
 - DDS_String_length, 156
 - DDS_String_ncmp, 156
 - DDS_Wstring_compare, 157
- Subscriber, 36
 - DDS_DATAREADER_QOS_DEFAULT, 36
- Subscription Built-in Topic, 75
 - DDS_SUBSCRIPTION_TOPIC_NAME, 75
- Subscription Module, 34
- sync_type
 - OSAPI_PortProperty, 436
- TRANSPORT, 114
 - DDS_TRANSPORT_QOS_POLICY_NAME, 114
- take
 - FooDataReader, 407
- take_instance
 - FooDataReader, 411
- take_instance_untyped
 - DDSDDataReader, 317
- take_next_sample
 - FooDataReader, 405
- take_next_sample_untyped
 - DDSDDataReader, 318
- take_untyped
 - DDSDDataReader, 314
- thread
 - OSAPI_TimerProperty, 441
- Time Support, 77
 - DDS_DURATION_INFINITE, 78
 - DDS_DURATION_INFINITE_NSEC, 78
 - DDS_DURATION_INFINITE_SEC, 78
 - DDS_DURATION_ZERO, 78
 - DDS_DURATION_ZERO_NSEC, 78
 - DDS_DURATION_ZERO_SEC, 78
 - DDS_TIME_INVALID, 78
 - DDS_TIME_INVALID_NSEC, 77
 - DDS_TIME_INVALID_SEC, 77
 - DDS_TIME_ZERO, 77
- timer_property
 - OSAPI_SystemProperty, 440
- timer_resolution_ms
 - OSAPI_PortProperty, 436
- to_array
 - FooSeq, 424
- Topic, 25, 26
- topic_name
 - DDS_PublicationBuiltinTopicData, 279
 - DDS_SubscriptionBuiltinTopicData, 300
- total_count
 - DDS_DataReaderInstanceReplacedStatus, 239
 - DDS_LivelinessLostStatus, 263
 - DDS_OfferedDeadlineMissedStatus, 267
 - DDS_OfferedIncompatibleQosStatus, 268
 - DDS_PublicationMatchedStatus, 280
 - DDS_RequestedDeadlineMissedStatus, 285
 - DDS_RequestedIncompatibleQosStatus, 286
 - DDS_SampleLostStatus, 297
 - DDS_SampleRejectedStatus, 298
 - DDS_SubscriptionMatchedStatus, 302
- total_count_change
 - DDS_DataReaderInstanceReplacedStatus, 239
 - DDS_LivelinessLostStatus, 263
 - DDS_OfferedDeadlineMissedStatus, 267
 - DDS_OfferedIncompatibleQosStatus, 268
 - DDS_PublicationMatchedStatus, 280
 - DDS_RequestedDeadlineMissedStatus, 285
 - DDS_RequestedIncompatibleQosStatus, 286
 - DDS_SampleLostStatus, 297
 - DDS_SampleRejectedStatus, 298
 - DDS_SubscriptionMatchedStatus, 302
- transform_locator_kind
 - UDP_InterfaceFactoryProperty, 448
- transform_udp_mode
 - UDP_InterfaceFactoryProperty, 447
- transport
 - DDS_DataReaderQos, 242
 - DDS_DataWriterQos, 247
- transports
 - DDS_DomainParticipantQos, 253
- type_name
 - DDS_PublicationBuiltinTopicData, 279
 - DDS_SubscriptionBuiltinTopicData, 300
- UDP Transform API, 60
- UDP Transport API, 58
 - add_entry, 59
 - UDP_INTERFACE_INTERFACE_MULTICAST_FLAG, 58
 - UDP_INTERFACE_INTERFACE_UP_FLAG, 58
 - UDP_INTERFACE_MAX_IFNAME, 58
 - UDP_INTERFACE_MAX_NETMASK_BITS, 58
 - UDP_INTERFACE_INTERFACE_MULTICAST_FLAG
 - UDP Transport API, 58
 - UDP_INTERFACE_INTERFACE_UP_FLAG
 - UDP Transport API, 58
 - UDP_INTERFACE_MAX_IFNAME
 - UDP Transport API, 58
 - UDP_INTERFACE_MAX_NETMASK_BITS
 - UDP Transport API, 58
 - UDP_InterfaceFactoryProperty, 446
 - allow_interface, 446
 - deny_interface, 446
 - destination_rules, 447
 - if_table, 447
 - is_default_interface, 447
 - max_message_size, 447

- max_receive_buffer_size, [447](#)
- max_send_buffer_size, [446](#)
- max_send_message_size, [447](#)
- multicast_interface, [447](#)
- multicast_ttl, [447](#)
- nat, [447](#)
- recv_thread, [447](#)
- source_rules, [447](#)
- transform_locator_kind, [448](#)
- transform_udp_mode, [447](#)
- UDP_InterfaceTableEntry, [448](#)
 - address, [448](#)
 - flags, [448](#)
 - ifname, [448](#)
 - netmask, [448](#)
- UDP_InterfaceTableEntrySeq, [448](#)
- UDP_LOG_ALLOC_EC
 - NETIO, [192](#)
- UDP_LOG_CURSOR_ERROR_EC
 - NETIO, [192](#)
- UDP_LOG_DROPGRP_EC
 - NETIO, [192](#)
- UDP_LOG_FINALIZE_EC
 - NETIO, [190](#)
- UDP_LOG_GET_BUF_ALLOC_EC
 - NETIO, [191](#)
- UDP_LOG_GET_BUF_SIZE_EC
 - NETIO, [191](#)
- UDP_LOG_GET_IFLIST_EC
 - NETIO, [191](#)
- UDP_LOG_GET_LENGTH_EC
 - NETIO, [192](#)
- UDP_LOG_GET_NAMEINFO_EC
 - NETIO, [191](#)
- UDP_LOG_GETHOSTNAME_EC
 - NETIO, [189](#)
- UDP_LOG_INCONSISTENT_MAX_MESSAGE_SIZE_EC
 - NETIO, [192](#)
- UDP_LOG_INITIALIZE_FAILED_EC
 - NETIO, [192](#)
- UDP_LOG_LOADLIB_EC
 - NETIO, [191](#)
- UDP_LOG_MULTICAST_ENABLE_EC
 - NETIO, [191](#)
- UDP_LOG_MULTICAST_NOT_ENABLED_EC
 - NETIO, [192](#)
- UDP_LOG_NAT_DELETE_EC
 - NETIO, [190](#)
- UDP_LOG_PACKET_FWD_EC
 - NETIO, [190](#)
- UDP_LOG_PACKET_HEAD_EC
 - NETIO, [190](#)
- UDP_LOG_PACKET_INIT_EC
 - NETIO, [190](#)
- UDP_LOG_PORT_ADD_EC
 - NETIO, [191](#)
- UDP_LOG_PORT_ALLOC_EC
 - NETIO, [190](#)
- UDP_LOG_PORT_ENTRY_EC
 - NETIO, [190](#)
- UDP_LOG_PORT_NOT_FOUND_EC
 - NETIO, [192](#)
- UDP_LOG_PORT_THREAD_EC
 - NETIO, [191](#)
- UDP_LOG_PROPERTY_FINALIZE_EC
 - NETIO, [192](#)
- UDP_LOG_RECORD_EC
 - NETIO, [191](#)
- UDP_LOG_RECV_ERROR_EC
 - NETIO, [193](#)
- UDP_LOG_SEND_ERROR_EC
 - NETIO, [192](#)
- UDP_LOG_SEND_PING_EC
 - NETIO, [192](#)
- UDP_LOG_SET_LENGTH_EC
 - NETIO, [192](#)
- UDP_LOG_SOCKET_ADDGRP_EC
 - NETIO, [190](#)
- UDP_LOG_SOCKET_BIND_EC
 - NETIO, [190](#)
- UDP_LOG_SOCKET_CLOSE_EC
 - NETIO, [191](#)
- UDP_LOG_SOCKET_CREATE_EC
 - NETIO, [190](#)
- UDP_LOG_SOCKET_IFFLAGS_EC
 - NETIO, [191](#)
- UDP_LOG_SOCKET_IFLIST_EC
 - NETIO, [191](#)
- UDP_LOG_SOCKET_INIT_EC
 - NETIO, [189](#)
- UDP_LOG_SOCKET_REUSE_PORT_EC
 - NETIO, [190](#)
- UDP_LOG_SOCKET_RXBUF_EC
 - NETIO, [191](#)
- UDP_LOG_SOCKET_SET_MCAST_EC
 - NETIO, [189](#)
- UDP_LOG_SOCKET_SET_MCASTIF_EC
 - NETIO, [190](#)
- UDP_LOG_SOCKET_SETFD_EC
 - NETIO, [190](#)
- UDP_LOG_SOCKET_SNDBUF_EC
 - NETIO, [190](#)
- UDP_LOG_SOCKET_TTL_EC
 - NETIO, [190](#)
- UDP_LOG_TRANSPORT_NO_INTERFACES_EC
 - NETIO, [193](#)
- UDP_LOG_UDP_CREATE_INDEX_EC
 - NETIO, [191](#)
- UDP_LOG_UDP_CREATE_TABLE_EC
 - NETIO, [191](#)
- UDP_LOG_WINSOCK_INCOMPATIBLE_EC
 - NETIO, [192](#)
- UDP_LOG_WSA_STARTUP_EC
 - NETIO, [192](#)
- UDP_NatEntry, [449](#)

- UDP_NatEntrySeq, [449](#)
- UDP_TRANSFORM_LOG_ADD_EC
 - NETIO, [193](#)
- UDP_TRANSFORM_LOG_ALLOC_EC
 - NETIO, [193](#)
- UDP_TRANSFORM_LOG_CREATE_EC
 - NETIO, [193](#)
- UDP_TRANSFORM_LOG_DUPLICATE_EC
 - NETIO, [193](#)
- UDP_TRANSFORM_LOG_FACTORY_NOT_FOUND_-EC
 - NETIO, [193](#)
- UDP_TRANSFORM_LOG_FIND_EC
 - NETIO, [193](#)
- UDP_TRANSFORM_LOG_INVALID_FACTORY_REFERENCE_EC
 - NETIO, [193](#)
- UDP_TRANSFORM_LOG_INVALID_TUDP_LOCATOR_KIND_EC
 - NETIO, [193](#)
- UDP_TRANSFORM_LOG_INVALID_UDP_MODE_EC
 - NETIO, [193](#)
- UDP_TRANSFORM_LOG_KIND_DST_INDEX
 - netio_log.h, [477](#)
- UDP_TRANSFORM_LOG_KIND_ENTRY
 - netio_log.h, [476](#)
- UDP_TRANSFORM_LOG_KIND_FACTORY
 - netio_log.h, [476](#)
- UDP_TRANSFORM_LOG_KIND_FACTORY_INDEX
 - netio_log.h, [477](#)
- UDP_TRANSFORM_LOG_KIND_RULE
 - netio_log.h, [476](#)
- UDP_TRANSFORM_LOG_KIND_SRC_INDEX
 - netio_log.h, [477](#)
- UDP_TRANSFORM_LOG_KIND_TRANSFORM_DESTINATION
 - netio_log.h, [477](#)
- UDP_TRANSFORM_LOG_KIND_TRANSFORM_NETWORK_MASK_INDEX
 - netio_log.h, [477](#)
- UDP_TRANSFORM_LOG_KIND_TRANSFORM_SOURCE
 - netio_log.h, [477](#)
- UDP_TRANSFORM_LOG_TRANSFORM_EC
 - NETIO, [193](#)
- UDPInterfaceFactory, [449](#)
 - get_interface, [449](#)
- UDPInterfaceTable, [450](#)
- udp_packet_received_event
 - OSAPI_PortProperty, [438](#)
- udp_receive_task_id
 - OSAPI_PortProperty, [438](#)
- unacknowledged_count
 - DDS_ReliableSampleUnacknowledgedStatus, [284](#)
- unicast_locator
 - DDS_PublicationBuiltinTopicData, [280](#)
 - DDS_SubscriptionBuiltinTopicData, [301](#)
- unloan
 - FooSeq, [428](#)
- unregister
 - RTRegistry, [445](#)
- unregister_instance
 - FooDataWriter, [415](#)
- unregister_instance_untyped
 - DDSDataWriter, [331](#)
- unregister_instance_w_timestamp
 - FooDataWriter, [416](#)
- unregister_instance_w_timestamp_untyped
 - DDSDataWriter, [332](#)
- unregister_type
 - DDSDomainParticipant, [351](#)
 - FooTypeSupport, [432](#)
- use_socket_owner
 - OSAPI_PortProperty, [437](#)
- use_udp_thread
 - OSAPI_PortProperty, [438](#)
- User Data Type Support, [27](#)
 - DDS_DATAREADER_CPP, [28](#)
 - DDS_DATAWRITER_CPP, [28](#)
 - DDS_HANDLE_NIL, [29](#)
 - DDS_InstanceHandle_equals, [28](#)
 - DDS_InstanceHandle_is_nil, [29](#)
 - DDS_InstanceHandle_t, [28](#)
 - DDS_TYPESUPPORT_CPP, [28](#)
- user_multicast_port_offset
 - DDS_RtpsWellKnownPorts_t, [294](#)
- user_traffic
 - DDS_DomainParticipantQos, [253](#)
- user_unicast_port_offset
 - DDS_RtpsWellKnownPorts_t, [294](#)
- valid_data
 - DDS_SampleInfo, [296](#)
- value
 - DDS_BuiltinTopicKey_t, [237](#)
 - DDS_GUID_t, [260](#)
 - DDS_OwnershipStrengthQosPolicy, [272](#)
- vendorId
 - DDS_VendorId, [305](#)
 - DDS_VendorId_t, [306](#)
- View States, [43](#)
 - DDS_ANY_VIEW_STATE, [44](#)
 - DDS_NEW_VIEW_STATE, [44](#)
 - DDS_NOT_NEW_VIEW_STATE, [44](#)
 - DDS_ViewStateKind, [43](#)
 - DDS_ViewStateMask, [43](#)
- view_state
 - DDS_SampleInfo, [295](#)
- WH, [214](#)
 - WHSM_LOG_INVALID_INDEX_OBJECT_EC, [215](#)
 - WHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC, [214](#)
 - WHSM_LOG_MAX_SAMPLES_TOO_SMALL_EC, [214](#)
 - WHSM_LOG_NO_PROPERTY_EC, [214](#)
 - WHSM_LOG_OBJECT_ALLOCATE_EC, [214](#)

- WHSM_LOG_OBJECT_DELETE_EC, [214](#)
- WHSM_LOG_OBJECT_INDEX_EC, [214](#)
- WHSM_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC, [214](#)
- WHSM Writer History API, [57](#)
- WHSM_LOG_HISTORY_OBJECT
 - wh_sm_log.h, [490](#)
- WHSM_LOG_HISTORYINDEX_OBJECT
 - wh_sm_log.h, [490](#)
- WHSM_LOG_INVALID_INDEX_OBJECT_EC
 - WH, [215](#)
- WHSM_LOG_KEEP_ALL_HISTORY_NOT_SUPPORTED_EC
 - WH, [214](#)
- WHSM_LOG_KEY_OBJECT
 - wh_sm_log.h, [490](#)
- WHSM_LOG_KEYINDEX_OBJECT
 - wh_sm_log.h, [490](#)
- WHSM_LOG_KEYPOOL_OBJECT
 - wh_sm_log.h, [490](#)
- WHSM_LOG_MAX_SAMPLES_TOO_SMALL_EC
 - WH, [214](#)
- WHSM_LOG_NO_PROPERTY_EC
 - WH, [214](#)
- WHSM_LOG_OBJECT_ALLOCATE_EC
 - WH, [214](#)
- WHSM_LOG_OBJECT_DELETE_EC
 - WH, [214](#)
- WHSM_LOG_OBJECT_INDEX_EC
 - WH, [214](#)
- WHSM_LOG_SAMPLE_OBJECT
 - wh_sm_log.h, [490](#)
- WHSM_LOG_SAMPLEINDEX_OBJECT
 - wh_sm_log.h, [490](#)
- WHSM_LOG_SAMPLEPOOL_OBJECT
 - wh_sm_log.h, [490](#)
- WHSM_LOG_UNLIMITED_HISTORY_NOT_SUPPORTED_EC
 - WH, [214](#)
- WHSMHistoryFactory, [450](#)
 - get_interface, [450](#)
- WIRE_PROTOCOL, [108](#)
 - DDS_CHECKSUM_AUTO, [109](#)
 - DDS_CHECKSUM_BUILTIN128, [109](#)
 - DDS_CHECKSUM_BUILTIN32, [109](#)
 - DDS_CHECKSUM_BUILTIN64, [109](#)
 - DDS_CHECKSUM_NONE, [109](#)
 - DDS_ChecksumKind_t, [109](#)
 - DDS_ChecksumKindMask_t, [109](#)
 - DDS_INTEROPERABLE_RTPS_WELL_KNOWN_PORTS, [110](#)
 - DDS_RTI_BACKWARDS_COMPATIBLE_RTPS_WELL_KNOWN_PORTS, [110](#)
 - DDS_WIREPROTOCOL_QOS_POLICY_NAME, [110](#)
- wait
 - DDSWaitSet, [393](#)
- wh_sm_log.h, [489](#)
- WHSM_LOG_HISTORY_OBJECT, [490](#)
- WHSM_LOG_HISTORYINDEX_OBJECT, [490](#)
- WHSM_LOG_KEY_OBJECT, [490](#)
- WHSM_LOG_KEYINDEX_OBJECT, [490](#)
- WHSM_LOG_KEYPOOL_OBJECT, [490](#)
- WHSM_LOG_SAMPLE_OBJECT, [490](#)
- WHSM_LOG_SAMPLEINDEX_OBJECT, [490](#)
- WHSM_LOG_SAMPLEPOOL_OBJECT, [490](#)
- write
 - FooDataWriter, [413](#)
- write_buffer
 - OSAPI_LogProperty, [435](#)
- write_untyped
 - DDSDDataWriter, [327](#)
- write_w_timestamp
 - FooDataWriter, [419](#)
- write_w_timestamp_untyped
 - DDSDDataWriter, [335](#)
- writer_resource_limits
 - DDS_DataWriterQos, [247](#)